
工业物联网操作系统砵璁及实验开发详解

序言

目录

第 1 篇 前言	8
第 2 篇 基本介绍	10
第 1 章 XiUOS 介绍	10
1.1 背景.....	10
1.2 定位.....	10
1.3 特色.....	11
1.4 部署.....	12
1.5 结构.....	14
第 2 章 基于 ARM 架构的矽璜硬件终端	17
2.1 ARM 硬件系统介绍	17
2.2 源码下载.....	24
2.3 工具安装.....	25
2.4 代码编译.....	27
2.5 系统烧录.....	28
2.6 系统调试.....	29
第 3 章 基于 RISC-V 架构的矽璜硬件终端	32
3.1 RISC-V 硬件系统介绍	32
3.2 源码下载.....	38
3.3 工具安装.....	39
3.4 代码编译.....	40
3.5 系统烧录.....	41
3.6 系统调试.....	43
第 3 篇 XiUOS 内核 (XiZi)	46
第 1 章 任务管理	46
1.1 任务管理介绍.....	46
1.2 任务管理实验.....	59
第 2 章 内存管理	92
2.1 内存管理介绍.....	92
2.2 内存堆管理接口.....	98
2.3 内存池管理接口.....	98
2.4 内存管理及实验.....	99
第 3 章 中断管理及实验	111

3.1 中断管理介绍.....	111
3.2 中断管理接口.....	113
3.3 中断管理实验.....	114
第4章 节拍机制及实验	121
4.1 节拍机制介绍.....	121
4.2 节拍设置接口.....	122
4.3 节拍机制实验.....	124
第5章 文件系统及实验	126
5.1 文件系统介绍.....	126
5.2 文件系统接口.....	127
5.3 文件系统实验.....	130
第6章 驱动管理及实验	132
6.1 驱动管理介绍.....	132
6.2 驱动管理接口.....	134
6.3 关键数据结构.....	135
6.4 函数接口定义.....	138
6.5 驱动模型实验.....	142
第4篇 驱动开发实验-ARM 篇	145
第1章 GPIO 驱动	145
1.1 GPIO 介绍	145
1.2 GPIO 驱动介绍	150
1.3 GPIO 设备应用	155
第2章 TIMER 驱动	157
2.1 TIMER 介绍	157
2.2 TIMER 驱动介绍	161
2.3 TIMER 设备应用	166
第3章 RTC 驱动	168
3.1 RTC 介绍.....	168
3.2 RTC 驱动介绍.....	177
3.3 RTC 设备应用.....	184
第4章 WATCHDOG 驱动	186
4.1 WATCHDOG 介绍	186
4.2 WATCHDOG 驱动介绍	188
4.3 WATCHDOG 设备应用	193

第 5 章 UART 驱动	194
5.1 UART 介绍.....	194
5.2 UART 驱动介绍.....	201
5.3 UART 设备应用.....	207
第 6 章 I2C 驱动	208
6.1 I2C 介绍.....	208
6.2 I2C 驱动介绍.....	219
6.3 I2C 设备应用.....	225
第 7 章 SPI 驱动	226
7.1 SPI 介绍.....	226
7.2 SPI 驱动介绍.....	230
7.3 SPI 设备应用.....	236
第 8 章 QSPI flash 驱动	237
8.1 QSPI 介绍.....	237
8.2 QSPI flash 驱动介绍	242
8.3 QSPI flash 设备应用	251
第 9 章 ADC 驱动开发	252
9.1 ADC 介绍	252
9.2 ADC 驱动介绍	255
9.3 ADC 设备应用	260
第 10 章 DAC 驱动	261
10.1 DAC 介绍	261
10.2 DAC 驱动介绍	263
10.3 DAC 设备应用	268
第 11 章 USB 驱动	269
11.1 USB 介绍.....	269
11.2 USB 驱动介绍.....	274
11.3 USB 设备应用.....	278
第 12 章 SDIO 驱动	280
12.1 SDIO 介绍	280
12.2 SDIO 驱动介绍	284
12.3 SDIO 设备应用	288
第 13 章 CAN 驱动	290
13.1 CAN 介绍	290

13.2 CAN 驱动介绍	293
13.3 CAN 设备应用	299
第 14 章 ETH 驱动	301
14.1 以太网 MAC 控制器介绍	301
14.2 ETH 驱动介绍	307
14.3 ETH 设备应用	308
第 5 篇 驱动开发—RISC-V 篇	311
第 1 章 GPIO 驱动	311
1.1 GPIO 介绍	311
1.2 GPIO 驱动程序	319
1.3 GPIO 应用实例	323
第 2 章 TIMER 驱动	324
2.1 TIMER 介绍	324
2.2 TIMER 驱动程序	333
2.3 TIMER 应用实例	335
第 3 章 RTC 驱动	337
3.1 RTC 介绍	337
3.2 RTC 驱动程序	340
3.3 RTC 应用实例	345
第 4 章 WATCHDOG 驱动	346
4.1 WATCHDOG 介绍	346
4.2 WATCHDOG 驱动程序	348
4.3 WATCHDOG 应用实例	352
第 5 章 UART 驱动	353
5.1 UART 介绍	353
5.2 UART 驱动程序	360
5.3 UART 应用实例	363
第 6 章 I2C 驱动	366
6.1 I2C 介绍	366
6.2 I2C 驱动程序	394
6.3 I2C 应用实例	398
第 7 章 TOUCH 驱动	399
7.1 TOUCH 介绍	399
7.2 TOUCH 驱动程序	400

7.3 TOUCH 应用实例.....	405
第 8 章 SPI 驱动	407
8.1 SPI 介绍.....	407
8.2 SPI 驱动程序.....	408
第 9 章 USB 驱动开发	415
9.1 USB 介绍.....	415
9.2 USB 驱动程序.....	421
9.3 USB 设备应用.....	425
第 10 章 TF 卡驱动	427
10.1 TF 卡介绍	427
10.2 TF 卡驱动程序.....	428
10.3 TF 卡设备应用	432
第 11 章 ETH 驱动	432
11.1 ETH 介绍.....	432
11.2 ETH 驱动程序.....	438
11.3 ETH 设备应用.....	438
第 12 章 LCD 驱动	440
12.1 LCD 介绍.....	440
12.2 LCD 驱动程序.....	442
12.3 LCD 设备应用.....	449
第 13 章 CAMERA 驱动	450
13.1 CAMERA 介绍	450
13.2 CAMERA 驱动程序	454
13.3 CAMERA 设备应用	460
第 6 篇 感联知控框架开发	463
第 1 章 “全面感知”实践.....	464
1.1 “全面感知”框架介绍	464
1.2 ARM 架构的传感器	469
1.3 RISC-V 架构下的传感器	477
第 2 章 “泛在互联”实践.....	488
2.1 “泛在互联”框架介绍	488
2.2 ARM 架构的网络连接	494
2.3 RISC-V 架构的网络连接	500
第 3 章 “实时认知”实践.....	504

3.1 “实时认知”框架介绍	504
3.2 ARM 架构下的“知”框架应用	509
3.3 RISC-V 架构下 K210 的“知”框架	517
第 4 章 “精准控制”实践	528
4.1 “精准控制”框架介绍	528
4.2 PLC 协议介绍	534
4.3 PLC 应用	538
附录 A	550
附录 B	557

第1篇 前言

本教材依托开源工业物联操作系统矽琇，面向高校教师和学生等读者群体，系统性介绍矽琇内核、驱动及应用框架，配套基于 ARM 和 RISC-V 架构的矽琇硬件终端设计相应的实验课程。

全书分为三个部分。第一部分重点介绍矽琇的背景、特色和架构，以及两款基于 ARM 架构和 RISC-V 架构的教学配套矽琇硬件终端，详细介绍了矽琇源码下载编译、系统烧录调试的方法，为后续实验奠定基础。第二部分详细讲解矽琇内核原理，如任务管理、内存管理、中断管理、节拍机制、文件系统和驱动管理等，并专门设计相应的内核实验，帮助读者充分理解和学习复杂的操作系统内核原理。第三部分则主要介绍驱动和应用框架开发实验案例和源码，均可在教学配套矽琇硬件终端上运行，涵盖了常见的外设驱动和嵌入式应用，在满足课程实验教学的前提下，也有助于读者搭建应用项目。

操作系统课程作为计算机和电子等专业重点课程，其应用范围之广、重要程度之大不言而喻，对教师教学方式和学生学习实践都有很高的要求。笔者调研发现，目前由于操作系统课程存在理论性强、知识点多和涉及面广等特点，在教学过程中存在教学手段单一、理论教学与其他学科衔接不紧密、理论与实践分离等问题，导致教学效果一般，引发学生课程学习难度大、学习成果见效慢和理论脱离实验等后果，长期以来容易心生畏惧，从而难以达到操作系统课程的教学目的。

针对以上问题，笔者结合开源工业物联操作系统矽琇，提供免费的实验硬件终端，设计代码精简、逻辑清晰和上手难度低的实验代码，用易于理解的术语和丰富的图表描述复杂的操作系统原理，突显实验内容丰富、实验形式多样化和实验层次清晰的特点，旨在解决目前操作系统课程中“重理论、轻实验”、学习难度高、缺乏课后实践等痛点问题，使读者掌握每个实验模块原理和代码，真正理解操作系统核心思想，成长为一名合格的操作系统工程师，为我国操作系统事业贡献力量。

致谢：

本书由浙江省北大信息技术高等研究院工业物联操作系统实验室负责修订完成, 特别感谢薛栋梁、刘伟超、王维根、田春雨、杜婉婧等员工和吴政、赵云、杨泽伟、黄琛、莫佳洋、胡超等实习生的尽力参与, 他们严谨细致、坚持不懈的精神大大推进了本书的修订进度。

第2篇 基本介绍

第1章 XiUOS 介绍

1.1 背景

XiUOS (X Indust Ubiquitous Operating System) 矽琇工业物联操作系统是一款面向工业物联场景的泛在操作系统，来自泛在操作系统研究计划。所谓泛在操作系统（UOS: Ubiquitous Operating Systems），是支持互联网时代人机物融合泛在计算应用模式的新型操作系统，是传统操作系统概念的泛化与延伸。在泛在操作系统技术体系中，不同的泛在计算设备和泛在应用场景需要符合各自特性的不同 UOS，XiUOS 即是面向工业物联场景的一种 UOS，主要由一个极简的微型实时操作系统(RTOS)内核和其上的智能工业物联框架构成，支持工业物联网（IIoT: Industrial Internet of Things）应用。

XiUOS 诞生于浙江省北京大学信息技术高等研究院，其研发获得了杭州市萧山区政府的大力支持。萧山是浙江乃至中国的制造业重镇，工业基础雄厚，正在谋求传统制造业数字化转型升级、实现新旧动能转换，寻求工业互联网和物联网等信息技术的助力。XiUOS 矽琇工业物联操作系统正是在这样的背景下应运而生，希望通过研发的工业物联网软硬件系统为助力，促进先进计算技术和工业场景的深度融合，助推工业企业的数字化转型，是产业应用需求和学术研究计划结合的产物。

关于“矽琇”，“矽”即硅，“琇”类玉，寓意从沙子到美石的演进升华之路。在开源社区的帮助下，未来矽琇去瑕，玉成其事。

1.2 定位

XiUOS 是一种工业物联网操作系统，目标是通过工业物联网的部署和应用，促进工业领域人、机、物的深度互联，促进工厂的数字化转型升级，使能智能化工业生产新体系。要实现该目标，工业物联网操作系统需要解决“感、联、知、控”四方面的问题：

- 感：复杂多样的工业生产实体智能地识别、感知和采集生产相关数据；
- 联：工业数据在互联互通的网络上进行传输和汇聚；
- 知：对这些网络化的工业大数据形成快速处理和实效分析；
- 控：将数据分析所得到的信息反馈到工业生产中，对生产活动进行改善。

其中，“感”和“联”主要在物联网节点级设备、传感器等设备上实现，“知”和“控”的核心主要在边缘计算设备和数据中心设备上实现。面向物联网节点级设备的矽璜工业物联网操作系统，主要定位于“感”、“联”和“控”的层次，支持接入市面上各种传感器，将感知获得的数据传递给边缘设备和数据中心设备，具有对 PLC 等的控制能力，能对“知”提供支持，如图 2-1-1 所示。

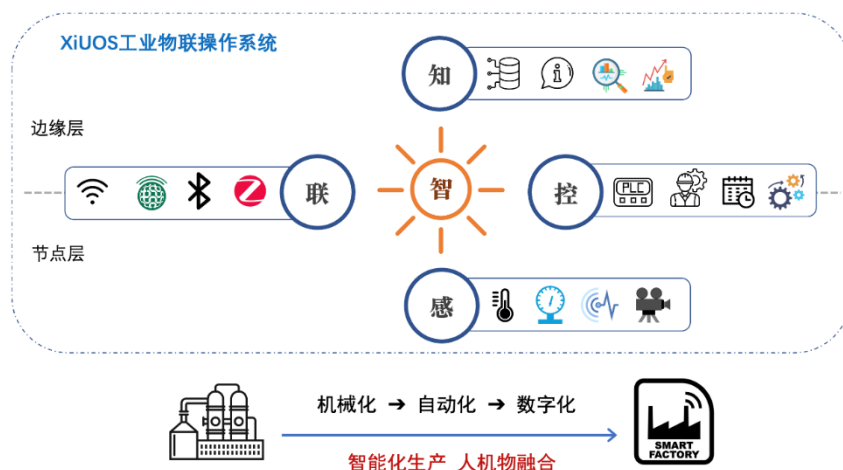


图 2-1-1 “感联知控”框架示意图

1.3 特色

目前市面上已有非常多成功的物联网操作系统，如国外的 FreeRTOS、Zephyr、RIOT、 $\mu\text{C}/\text{OS}$ ，国内的 RT-Thread、LiteOS、AliOS Things 等，这些系统多是从实时嵌入式系统（RTOS: Real time OS）发展而来，通常具有非常小巧的实时内核，多运行在资源受限的微控制单元（MCU: Microcontroller Unit）或单片机上，对低功耗有较高要求，常见的处理器为 ARM Cortex-M 系列。

矽璜工业物联网操作系统 XiUOS 和现有大多数的物联网 OS 关注点并不相同，具有自己的特色：

- XiUOS 不仅着眼于内核的基础功能，如简单实时的调度，静态内存的预分配与动态内存的碎片合并相结合的机制，任务隔离巧妙耦合硬件体系结构特征的算法；而且更关注基础功能之上的业务驱动的高级功能，如以感知数据本身为中心的传感器驱动方法，以方便用户设备联网的无线网络自动接入功能，在节点端启动图像与声音识别的智能化功能；

- XiUOS 不仅注重于 OS 本身在多领域的通用性，而且更寻求其在工业物联领域的独特性，将复杂的工业控制流程解耦为多个不可再分的“控制元”，以软件定义的方式对各种“控制元”进行再组合或构造，从而在节点端完成对工业设备的实时控制，满足工业企业的对精准控制的刚需；
- XiUOS 不仅关切单个节点设备本身功能和效用的发挥，而且更聚焦于在大规模工业生产环境中组成的网络化集群，强调对大规模部署和管理运维的支持，强调节点设备群构成的物联体系对工业生产的促进作用；
- XiUOS 吸取现有闭源和开源 RTOS 在各自细分领域的特征，更加关注生态的开放性，依托并逐步融入现有的主流开源生态体系，形成其在工业领域的重要角色和作用。

XiUOS 是软硬件协同的物联网解决方案，提供自研的有特色的硬件系统，支持 ARM 和 RISC-V 两种体系结构的处理器，在同一物理终端上同时支持多种通信方式和多种总线接口，可直接部署于工业生产环境，使得工业物联网用户不必关心底层各类硬件的实施细节，从而更加专注于应用逻辑本身的效能。

简而言之，现有物联网 OS 主要关注的是做优秀的单节点 OS，XiUOS 是在现有单节点 OS 的基础上构建面向多节点的网络化智能物联软件栈，为工业物联网领域应用开发、系统部署、管理运维提供友好方案，因此现有物联网 OS 和 XiUOS 功能是主要是互补的而非竞争的，如 Linux 之于 Android。

1.4 部署

XiUOS 是面向工业领域多节点设备的网络化 OS，其部署主要包括节点端的多节点集群和边缘端/云端的计算设施，如图 2-1-2 所示。



图 2-1-2 XIUOS 部署环境示意图

在节点端，支持多种工厂环境传感设备，既包括基础的环境传感器感知温湿度、烟雾气体、噪音等，也包括智能传感器可对人体、物体、声音进行识别，这些传感器支持多种网络通信互联协议，可以按需自组网络，通过网关设备向外交换数据，实现智能物联框架的“感、联”功能。网关设备对同一个集群中的节点设备进行管理，负责和边缘/云端系统交换数据，并可具备有限的“知、控”功能。

节点端感知的数据经网关传输给边缘/云端系统之后，边缘/云端的工业互联网平台可对汇聚而来的数据进行分析，将有价值的数据在控制中心的大屏、管理人员的移动设备上展示，并根据工业应用的业务规则，通知和调度“设备、排程、人员”，对生产过程进行优化，完成“知、控”。

目前，XiUOS 已在山东青岛海尔热水器厂产线和浙江大胜达包装股份有限公司彩印车间等场景部署了示范应用。

1.4.1 海尔热水器厂工艺数字化

海尔热水器厂工艺数字化方案主要负责采集卷圆、直缝焊、扩口、组队、环缝、水管焊接、试压、打砂、搪瓷、预装、氮氢试压、发泡机和打包机等产线工位的 PLC 数据采集，并通过无线蜂窝网络 4G/5G、短距离无线 LoRa 网络和以太网等多种通信方式上传至海尔智家平台数据库，实时监控各工位生产状况和进度，

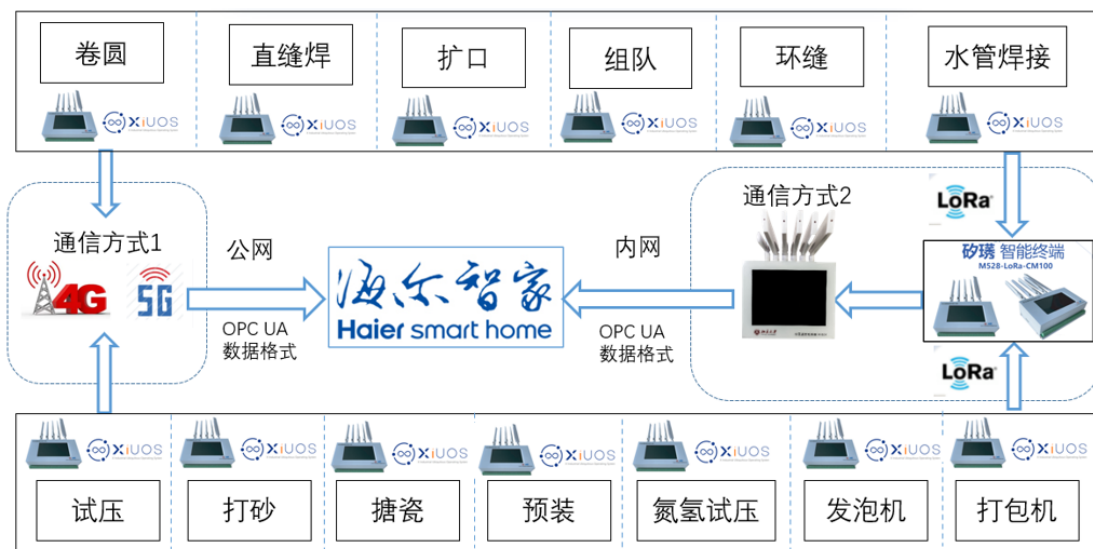


图 2-1-3 海尔热水器厂工艺数字化示意图

其中，砭琇工业物联智能终端运行 XiUOS，支持丰富的外设驱动接口，与产线工位 PLC 网口连接，解析通信协议并组装协议报文，实时采集 PLC 节拍数据，同时支持稳定的无线 LoRa 网络模组，通过私有协议组网上传数据至 LoRa 网关，根据 OPC UA 数据格式要求生成数据报文，存入海尔智家数据库。

1.4.2 大胜达智能环境感知系统

大胜达智能环境感知系统主要采集 PM1.0、PM2.5、PM10、AQS、TVOC、甲醛、乙醇、二氧化碳、噪音、甲烷、温度、湿度、气压、风速、风向等环境因子，此外还包括部分机器电能消耗的监测。该系统主要由 1 个网关、1 个电表监测节点和 6 组环境监测节点组成，每组包含 8 个监测节点，共 50 台设备，覆盖整个彩印车间，监测环境因子，如图 2-1-4 所示。

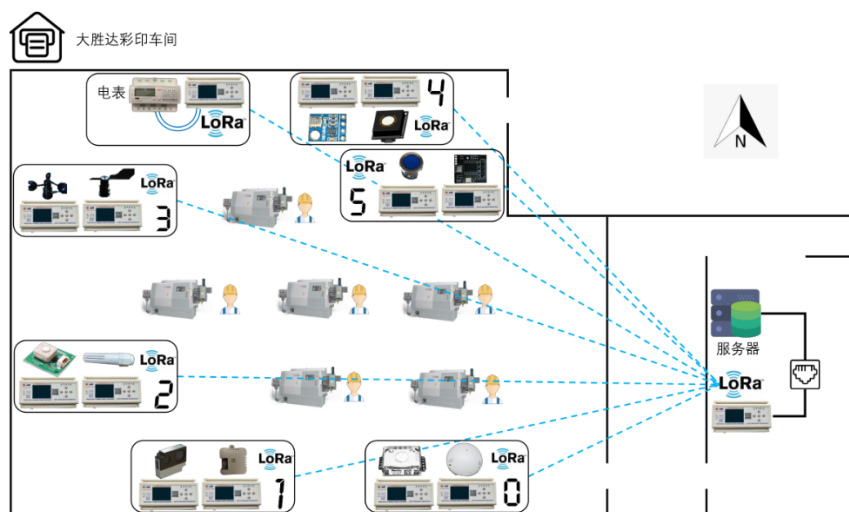


图 2-1-4 大胜达智能环境感知系统示意图

其中，监测节点由工业物联终端设备接入若干传感器组成，搭载 XiUOS，负责实时采集环境数据并上传到网关节点。网关与监测节点之间采用 LoRa 网络通信，网关节点则负责接收环境数据并转储到分布式内存数据库和分布式磁盘数据库中，供上层应用使用。该系统基于 XiUOS 的多任务管理，并发地管理多个主动上报和被动应答的传感器；基于 XiUOS 的感知框架，简化应用编程，轻松采集工业环境数据。

1.5 结构

面向智能制造的矽琇工业物联网操作系统通过高效管理工业物联网设备、支撑工业物联网应用，实现智能化的全面感知、泛在互联、实时认知、精准调控，促进以工业设备和工业控制系统为核心的人、机、物深度互联，帮助提升生产线的数字化和智能化水平。整体设计方案如图 2-1-5 所示。

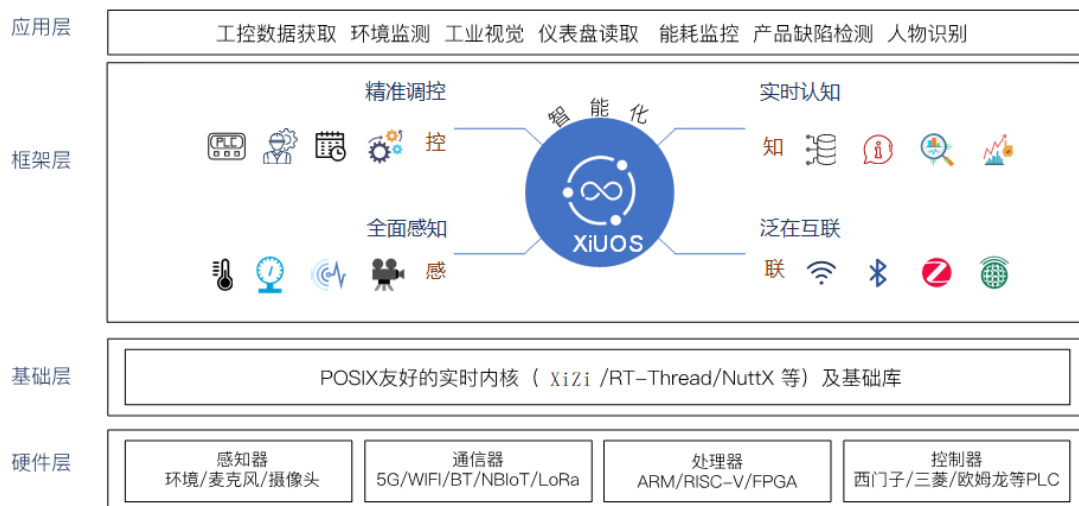


图 2-1-5 XiUOS 系统架构图

（1）硬件层：

硬件层主要是 XiUOS 支持的处理器、感知器和通信器，以及工业领域控制器：

- 处理器：支持 ARM Cortex-M 系列和 RISC-V 架构的处理器，以及 FPGA 芯片。
- 通信器：支持 5G/4G/WiFi/Bluetooth/NB-IoT/LoRa 等的网络通信器件。
- 感知器：支持工业领域的各种环境感知传感器，如烟雾、气体、噪声、电流电压等，还支持对声音和物体进行感知识别的摄像头、麦克风等音视频设备。
- 控制器：主要 PLC 厂商如西门子、施耐德、欧姆龙、三菱、台达等的 PLC 系统，XiUOS 可从 PLC 中获取工业生产相关数据。

自主研发的硬件系统对感知器、通信器、处理器和控制器提供综合支持，具有如下特点：

- XiUOS 的硬件系统支持 RISC-V 和 ARM 两种体系结构，强调 RISC-V 和 ARM 在工业应用中的共生共赢，又注重突出各自特点；
- XiUOS 的硬件系统在同一物理实体终端上同时支持 4G、NB-IoT、WiFi、ZigBee、LoRa、Bluetooth、Ethernet，在硬件上使得现有应用能够方便、快捷地联接各种网络；

- XiUOS 的硬件系统在同一物理实体终端上同时支持 SPI、IIC、UART、CAN、RS485、RS232、USB、SDIO 等总线接口，在硬件上满足工业物联网对外设多样性的需求；
- XiUOS 的硬件系统能够直接部署于工业生产环境中，而非验证性的开发板评估板，系统稳定可靠。

XiUOS 硬件系统中的处理器、通信器、传感器、控制器采用解耦的模块化设计，基于软件定义思想屏蔽各种硬件外设的差异，使得工业物联网用户不必关心底层硬件的实施细节，从而更加专注于应用逻辑本身的效能。

（2）基础层：

基础层是 XiUOS 关于实时内核的定义，支持自研 XiZi 内核，提供稳定的内核服务和驱动抽象。内核服务提供线程管理、内存管理、文件系统、中断管理和节拍机制等核心服务，支持多线程运行。驱动抽象框架遵循外设总线化和文件化的中心思想，驱动和应用解耦，支持海量异构的硬件设备。同时为拓展生态、吸引开发者，兼容 POSIX 友好的开源 RTOS 内核，如 RT-Thread 和 NuttX。

从 XiUOS 整体架构角度，基础层南向支持硬件层各类工业物联网设备的驱动，北向对接框架层“感、联、知、控”应用框架，除实时内核外，还提供丰富基础库，如 C/C++ 标准库、轻量级网络协议栈 LwIP 等。

基于对现有内核的功能特性与参数的归纳抽象，支持对内核进行软件定义，用户可选择基于何种内核运行 XiUOS，并对公共库、驱动等进行相应的定义，根据配置进行加载或生成。

（3）框架层：

框架层面向工业物联场景进行设计，核心是“感、联、知、控”应用框架。

- 感：支持工业领域的各种环境感知传感器，如温度、湿度、烟雾、气味、噪声、电流电压等，以及对声音和物体进行感知识别的摄像头等音视频采集设备，提供方便各种外接接口驱动，如 IIC、UART 等，便于接入新传感器。同时鉴于每种传感器在底层使用上都有区别，为了简化编程，提供统一的面向物理量的编程抽象，提供统一的面向数据的编程抽象，屏蔽底层设备的使用差异，开发者只需要使用 open/write/ioctl 等接口就可以对各种传感器进行操作和开发，简化传感数据的采集、解析和管理。

- 联：互联框架提供网络互联能力，尤其是复杂环境下的无线自组网能力，支持面向嵌入式领域的各种网络协议栈和消息通信库。其中协议栈包括 TCP/IP、NB-IoT、LoRa 等的软件栈支持，方便开发者开发通信功能。消息通信库则包括 MQTT、CoAP 等常用的 IoT 通信库。设计上采用以网络适配器为中心的抽象方式，发送与接收数据时，只需要像 socket 网络通信一样，仅关注发送与接收的数据，而不用把重点放在所使用的具体协议上，有效屏蔽种类繁多的网络协议细节，对外提供统一的网络接口，从而实现了网络数据收发功能与底层硬件的解耦，底层驱动实现具体的网络协议，从而将上层的网络数据传送需求映射到具体的硬件通信模组上。
- 控：控制框架提供工控设备控制和数据采集能力，提供对 PLC 的数据采集库，可对设备进行实时监控和远程控制，遵循软件定义工业控制的设计思路，支持和常见 PLC、工控机设备的互联互通，建立信息控制和交互的通道，实现自顶向下的执行流。
- 智：智能框架是 XiUOS 的一个特色，将智能识别和分析能力下沉到物联网节点设备上(通常为带加速器的 ARM Cortex-M/A 或 RISC-V 处理器)，使之在节点端即可实时对声音、物体、人体、缺陷模式等进行识别，大大拓展物联网节点的应用范围，内容包括 AI 算法加速库、微型 AI 引擎等。其中 AI 算法加速库提供工业物联网方面的常用 AI 模型，比如仪表盘识别模型等。微型 AI 引擎提供运行环境，如 TensorFlow Lite for microcontrollers，方便开发者开发常用 AI 算法。此外，还根据处理器和加速器的具体情况，提供加速器驱动，来高效利用节点端算力。

第2章 基于 ARM 架构的矽璜硬件终端

本节基于 ARM 矽璜硬件终端介绍硬件配置和 XiUOS 的编译及烧录方法。推荐使用操作系统 Ubuntu18.04，下载地址 <https://ubuntu.com/download/desktop>。开发工具推荐使用 VS Code，下载地址 <https://code.visualstudio.com/>，推荐下载地址为 http://vscode.cdn.azure.cn/stable/3c4e3df9e89829dce27b7b5c24508306b151f30d/code_1.55.2-1618307277_amd64.deb。

2.1 ARM 硬件系统介绍

ARM 矽璜硬件终端的模组分布如图 2-2-1 所示。

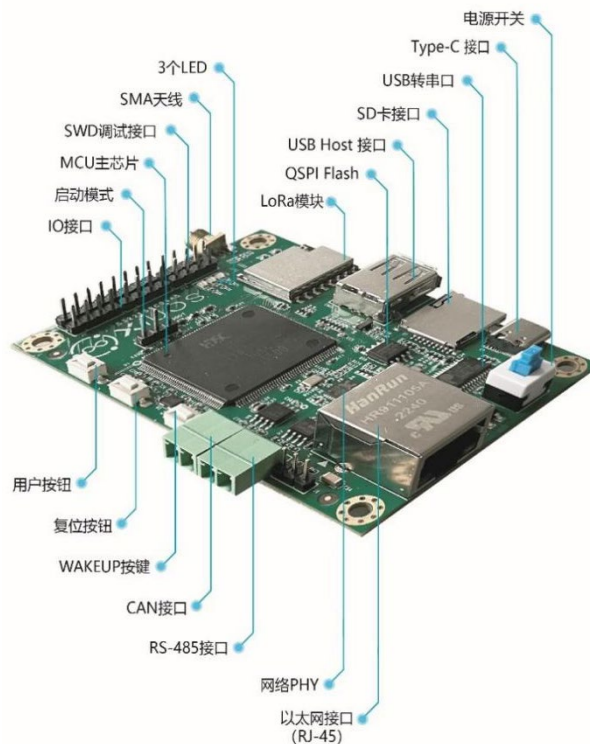


图 2-2-1 edu-arm 矽璞硬件终端

采用的MCU是华大半导体HC32F4A0型号芯片，HC32F4A0是基于ARM®Cortex®-M4 32-bit RISC CPU，最高工作频率240MHz的高性能MCU。同时，HC32F4A0集成了高速片上存储器，包括最大2MB的Flash，最大512KB的SRAM。集成了Flash访问加速单元，实现CPU在Flash上的单周期程序执行。具有丰富的片上资源，分别是：ADC、PGA、DAC、CMP、PWM、Timer、GPIO、I2C、UART、SPI、QSPI、CAN、I2S、SDIO、ETHMAC、USB、外部扩张总线控制器、DVP、MAU、FMAC。

ARM矽璞硬件终端根据HC32F4A0芯片的特点和教育需求，配置了QSPI Flash、LoRa模组、RJ-45、RS485、CAN、SMA天线、SWD调试接口、SD卡、USB等外设组件，可以全面、充分地发挥其各种功能。同时，ARM矽璞硬件终端上设计有通过PL2303实现的ISP自动下载电路，可通过单根USB Type-C对终端进行供电和程序烧录。

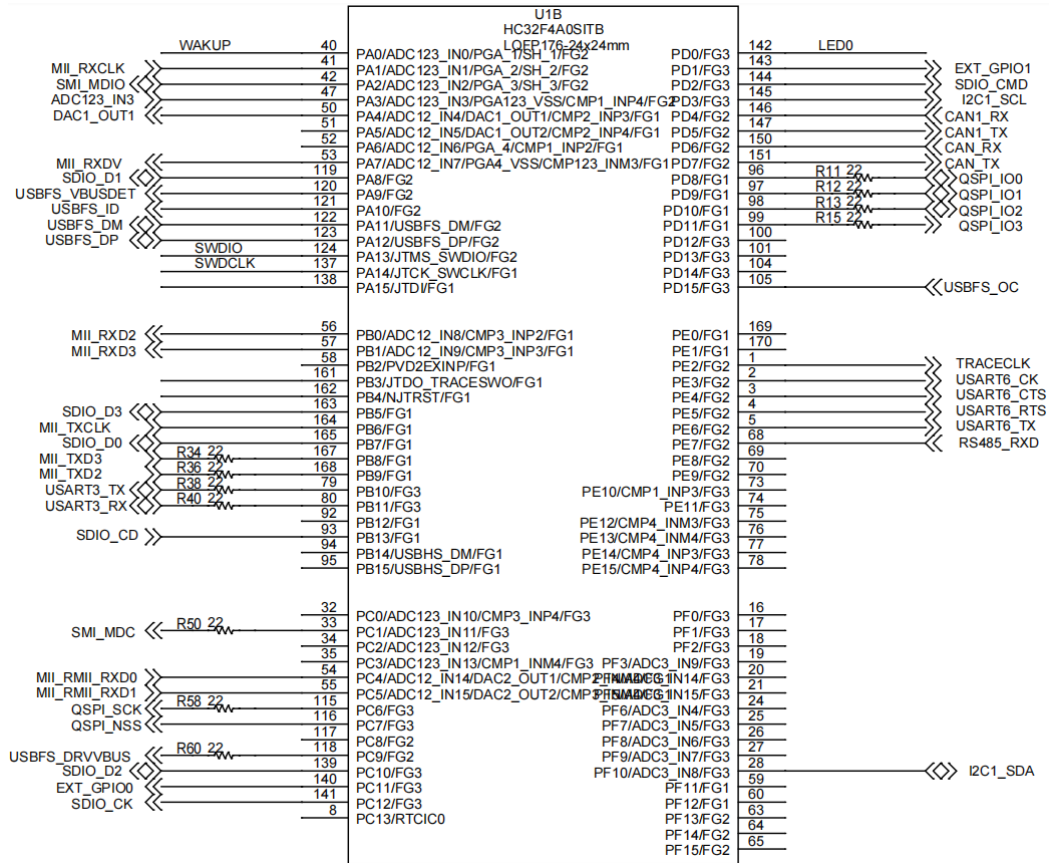


图 2-2-2 HC32F4A0 部分电路原理图

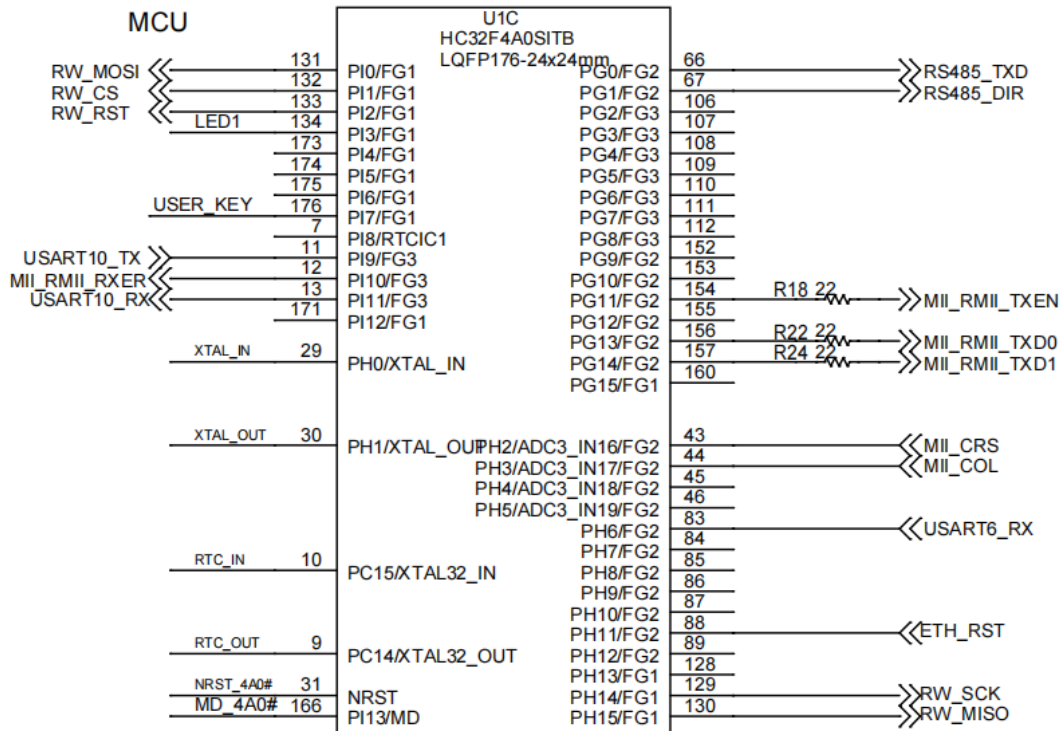


图 2-2-3 HC32F4A0 部分电路原理图

HC32F4A0 芯片部分的原理图如图 2-2-2 和 2-2-3 所示。

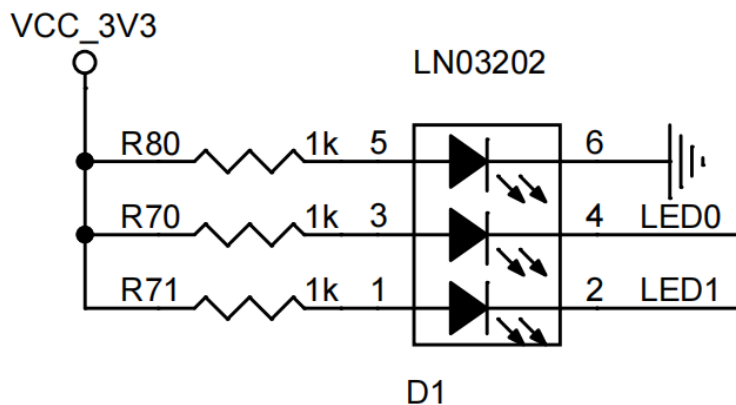


图 2-2-4 LED 电路原理图

在 HC32F4A0 上共搭载有 3 颗 LED，原理图如图 2-2-4 所示，其中一颗为电源指示灯，在电源供电正常时将保持常亮。LED0 和 LED1 的阴极引脚和 HC32F4A0 的 PD0 和 PI3 引脚相连，可通过 GPIO 或其他外设进行控制。

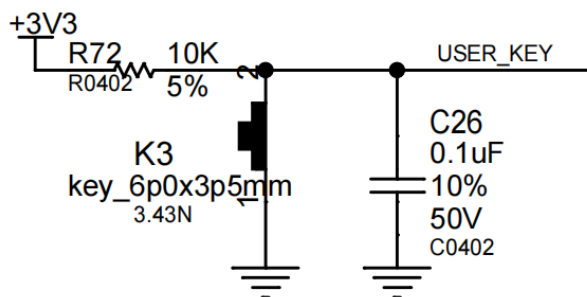


图 2-2-5 Key-3 按键电路原理图

用户自定义按键部分的原理图如图 2-2-5 所示。按键被一端接 GND，另一端通过一个 10K 的电阻和电源输出相连接，同时直接与 HC32F4A0 的 PI7 引脚相连接。

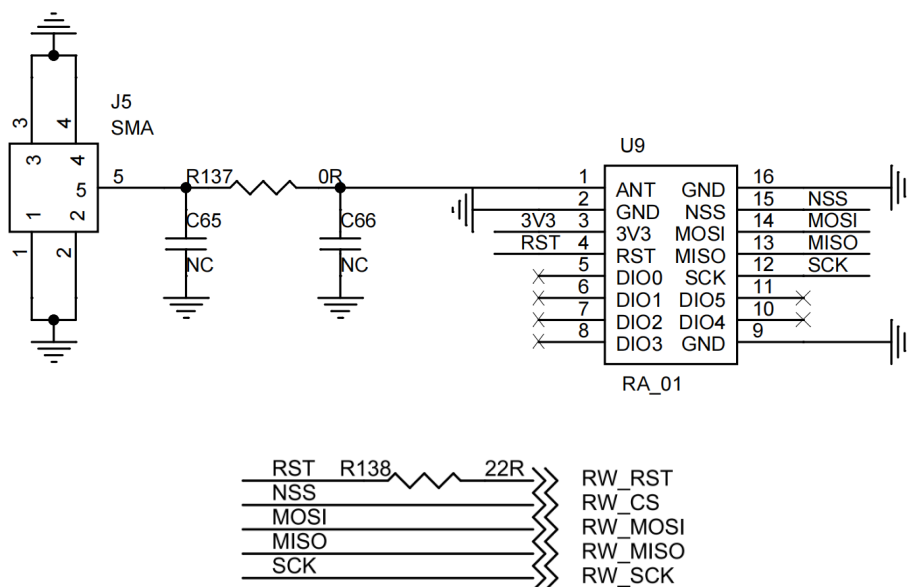


图 2-2-6 RA-01 Lora 通信模块原理图

如图 2-2-6 所示是 LoRa 模块的硬件原理图，其型号为 Ra-01H，其通过 SPI 接口与主芯片间进行通信。SPI 接口的 NSS、SCK、MOSI、MISO、RST 引脚分别连接至 HC32F4A0 的 PI1、PH14、PI0、PH15 和 PI2 对应的引脚。

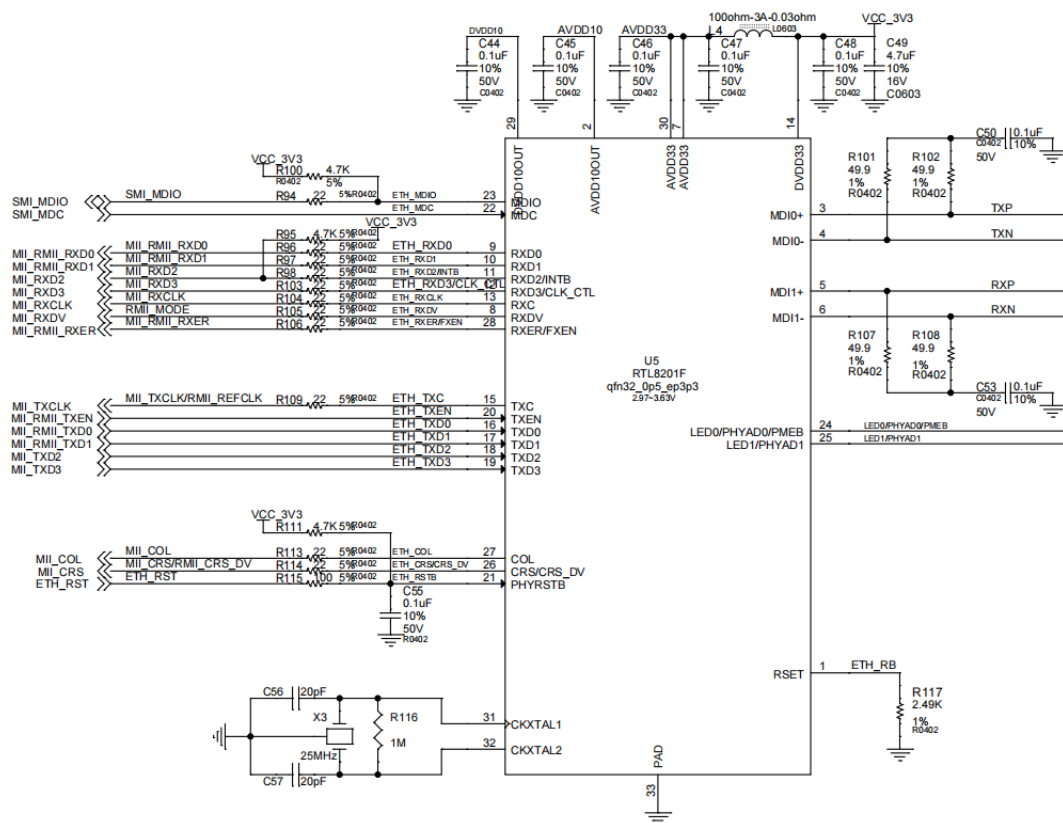


图 2-2-7 RTL8201F Ethernet 通信模块原理图

RTL8201F 芯片的硬件原理图如图 2-2-7 所示。RTL8201F 芯片主要通过 MII 接口与主控芯片进行通信，同时通过 MDIO+、MDIO-、MDI1+、MD1-与 RJ45 接口相连接。由于 MII 接口引脚数量较多，这里不一一列举，读者可参考附录 A HC32F4A0 引脚映射表来获得其硬件连接路线。

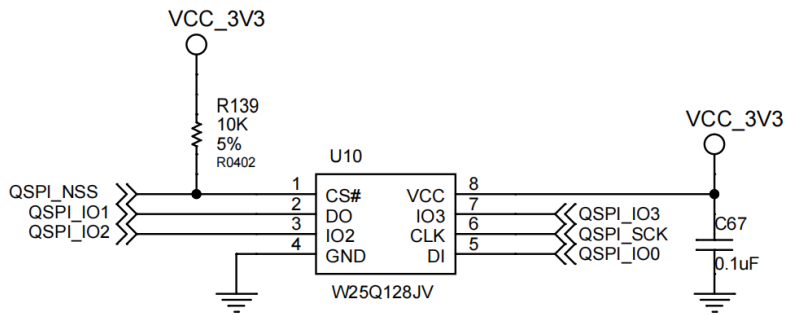


图 2-2-8 W25Q128 Flash 芯片原理图

图 2-2-8 所示的是 W25Q128 Flash 芯片的硬件原理图。W25Q128 是通过 QSPI 与 HC32F4A0 相连接。其 QSPI_IO0 至 QSPI_IO3 对应连接 HC32F4A0 的 PD8 至 PD11；QSPI_SCK 和 QSPI_NSS 分别连接到了 PC6 与 PC7。

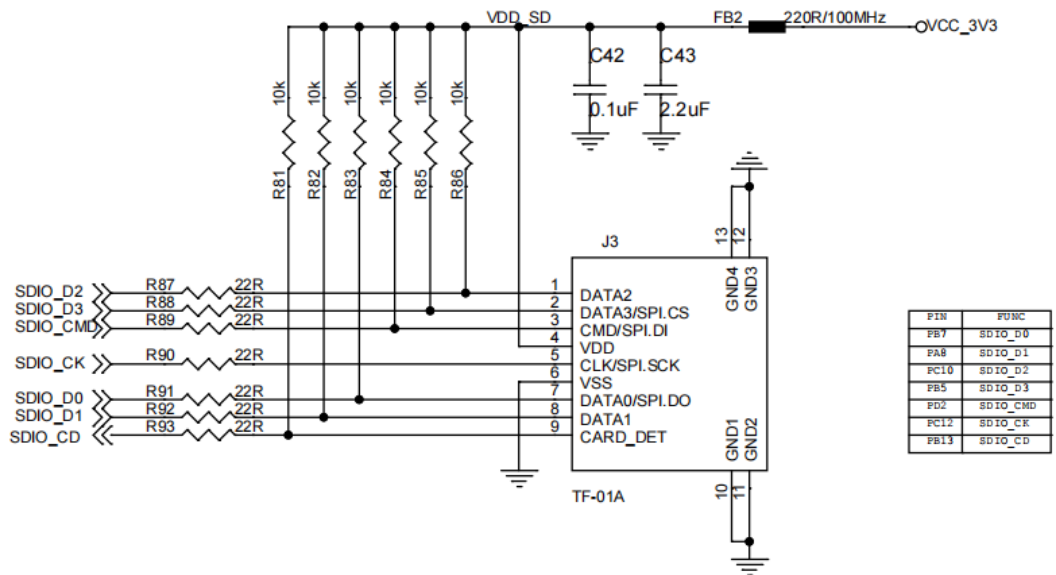


图 2-2-9 TF 卡槽原理图

TF 卡槽的对应连接方式如图 2-2-9 所示,其主要通过 SDIO 接口与 HC32F4A0 相连接,其中 SDIO_D0 至 SDIO_D3 分别连接到了 PB7、PA8、PC10 和 PB5; SDIO_CMD、SDIO_CK、SDIO_CD 分别连接到 PD2、PC12、PB13 对应引脚。

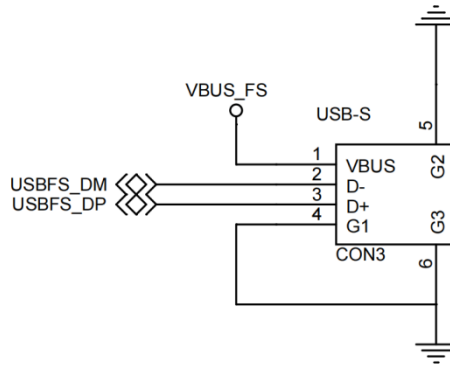


图 2-2-10 USB 接口原理图

HC32F4A0 集成了 USB 控制器，如图 2-2-10 所示，因而 USB 接口是直接与其 USB 控制引脚直接相连的。

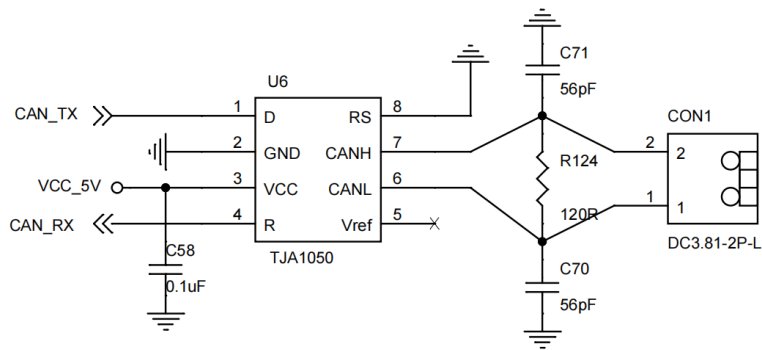


图 2-2-11 TJA1050 CAN 收发器芯片原理图

图 2-2-11 所示的是 CAN 口对应的硬件原理图。CAN 的电平转换器采用的是 TJA1050，TJA1050 的发送引脚 D 和接收引脚 R 分别与 PD7、PD6 相连接。

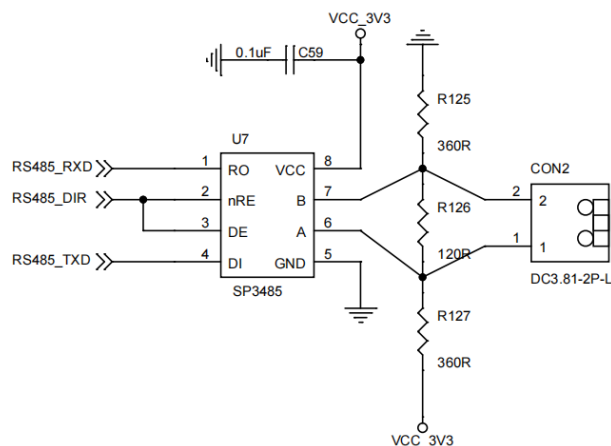


图 2-2-12 SP3485 RS485 收发器原理图

图 2-2-12 所示的是 RS485 通信接口对应的硬件原理图。RS485 通信的电平转换芯片采用了 SP3485 芯片，与 RISC-V 版本矽璞硬件终端相同。SP3485 的接收引脚 RO 和发送引脚 DI 对应了 UART 协议的 RS485_RXD 和 RS485_TXD，分别对应连接到 HC32F4A0 的 PE7 和 PG0；RS485 的输出输入方向选择对应了 nRE 和 DE 两个引脚，通过 PG1（对应 RS485_DIR）控制高低电平进行控制。

在拓展性方面，ARM 矽璞硬件终端提供了大量可用的 IO 引脚，支持包括 UART、I2C、SPI 在内的多种协议。提供 JTAG SW 调试引脚，可通过 JTAG 调试器实现在线调试，通过相关开发调试工具，可实现单步调试、断点调试等操作。

2.2 源码下载

XiUOS 的源码已经发布在确实开源平台，源码下载地址 <https://www.gitlink.org.cn/xuos/xiuos/>。首先使用 Linux 命令新建一个空文件夹 test，并进入该文件夹中。然后，使用命令下载源码至新建的 test 文件夹。具体用到的两条 Linux 命令如下：

```
$ mkdir test && cd test
$ git clone https://gitlink.org.cn/xuos/xiuos.git
```

XiUOS 源代码下载好之后，打开源码文件包，进入 test/Ubiquitous/XiUOS 可以看到多个文件，对应的说明如表 2-2-1 所示。

表 2-2-1 代码目录说明

名称	说明
----	----

application	应用代码
board	板级支持包
framework	应用框架
fs	文件系统
kernel	内核源码
resources	驱动文件
tool	系统工具

使用 VScode 打开源代码，可以进行代码查看和编写，打开方法为：打开系统终端，在源码文件夹下输入 `code` . 即可打开 VScode 开发环境，如图 2-2-13 所示。

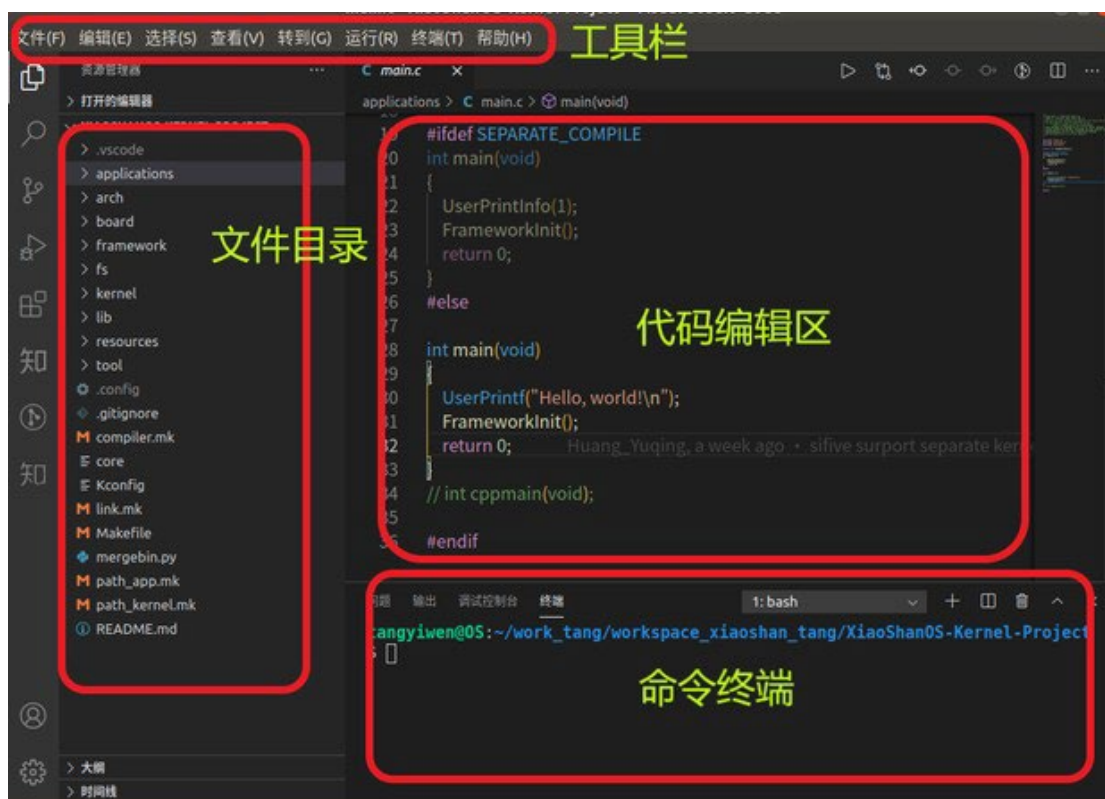


图 2-2-13 开发环境示意图

2.3 工具安装

(1) 依赖包安装:

Ubuntu 系统安装好之后，在安装软件之前，或者遇到依赖问题之时，需要安装一些依赖包，它聚合了一些常用的依赖。另外，还需要安装例如 gcc 编译器和 make 命令工具等。具体用到的两条 Linux 命令如下所示。

```
$ sudo apt install build-essential pkg-config  
  
$ sudo apt install gcc make libncurses5-dev openssl libssl-dev bison flex libelf-dev autoconf  
libtool gperf libc6-dev git
```

（2）配置工具安装：

Linux 编译器通过.config 文件确认哪些代码编译进内核，哪些被裁减掉，menuconfig 是生成.config 的一个工具。总结就是 menuconfig 是编辑内核的工具；kconfig 定义 menuconfig 菜单选项；生成的.config 是最后 menuconfig 配置完生成的文件；makefile 根据.config 宏定义判断是否编译驱动。

对于 XiUOS 的编译，首先需要下载对应的裁减配置工具：kconfig-frontends，下载地址 <https://gitlink.org.cn/xuos/kconfig-frontends.git>，然后进行安装即可。上述下载和安装操作可通过 Linux 命令执行，如下所示。第 1 条命令用于创建和进入一个文件夹 kfrontends；第 2 条命令是从确实开源平台下载一个 kconfig-frontends 至文件夹 kfrontends；第 3 条命令是运行 XiUOS 的 build 进行配置工具的安装。

```
$ mkdir kfrontends && cd kfrontends  
  
$ git clone https://gitlink.org.cn/xuos/kconfig-frontends.git  
  
$ ./xs_build.sh
```

（3）编译工具链安装：

做 C 开发，特别是嵌入式方向，需要涉及编译工具链和交叉编译工具链的相关内容，即 C 程序需要经过 gcc 等工具编译成二进制程序才能被计算机使用。这里的 gcc 通常是泛指，包括 gcc、g++、ar、as、ld 等一系列工具，这些工具用在程序文件的预编译、编译、链接等整个过程，这整套工具就被称为编译工具链（Toolchain）。

同样，XiUOS 的编译也需要安装编译工具链，对于 ARM 的 arm-none-eabi(gcc version 6.3.1)，默认安装到 Ubuntu 的/usr/bin/arm-none-eabi-。可使用如下命令行下载和安装。

```
$ sudo apt install gcc-arm-none-eabi
```

2.4 代码编译

使用 VScode 打开工程的方法有多种，本文介绍一种快捷键，在项目目录下将 `code .` 输入 `linux` 系统命令终端即可打开目标项目，然后打开 `APP_Framework/Applications` 文件夹下 `main.c`，在输出函数中写入 `Hello, world!!! \n`，即可完成测试代码编写。

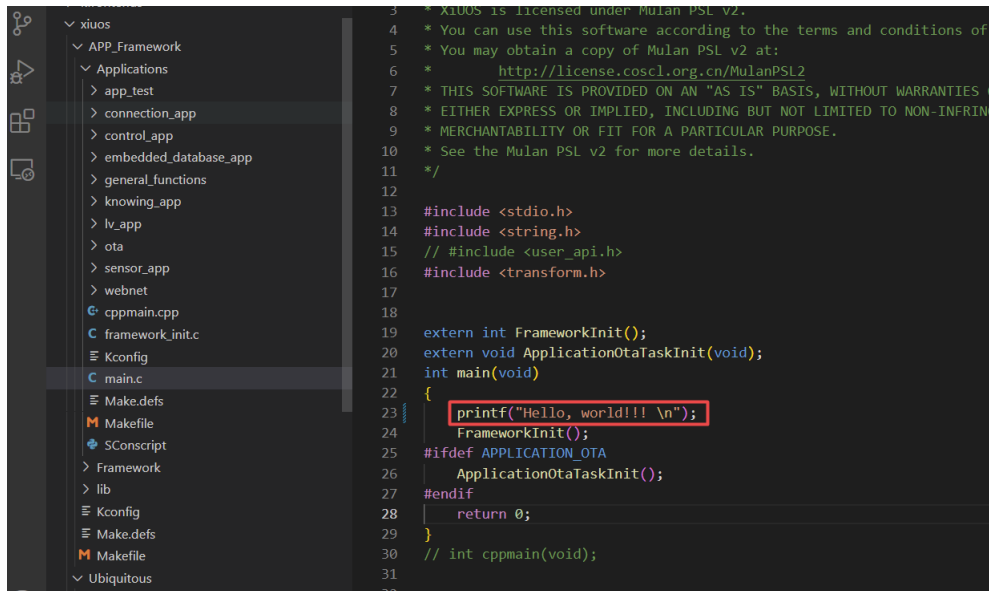


图 2-2-14 代码编写示例

代码编译具体步骤如下：

- (1) 在 VScode 的“命令终端”，执行命令 `make BOARD=hc32f4a0 menuconfig`，生成配置文件；在编译版本时，BOARD 需要选择 `hc32f4a0`；
- (2) 在 `menuconfig` 界面配置需要关闭和开启的功能，按回车键进入下级菜单，按 Y 键选中需要开启的功能，按 N 键选中需要关闭的功能，配置结束后选择 `Exit` 保存并退出。本实验无需选择任何选项，所以双击 `ESC` 结束选择，继续操作即可；

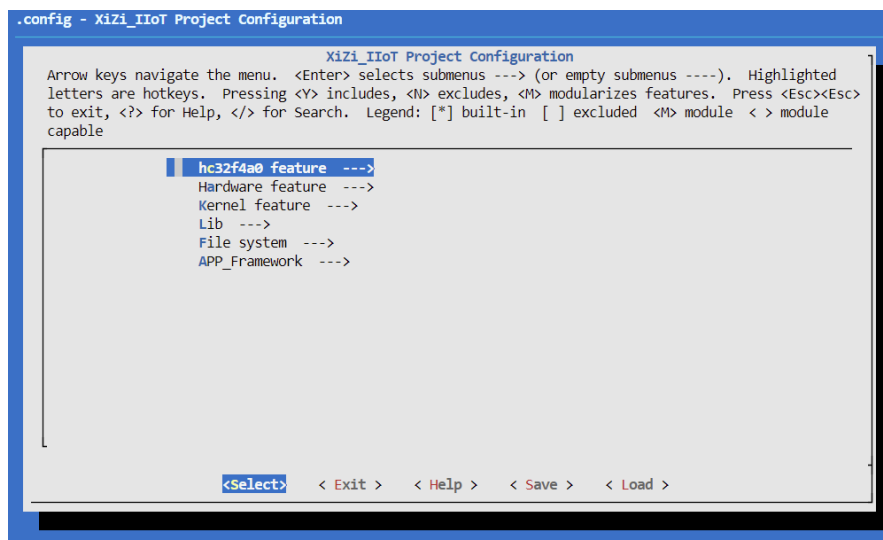


图 2-2-15 hc32f4a0 配置界面

(3) 继续在终端执行 `make BOARD=hc32f4a0` 命令，进行编译；

(4) 如果编译正确无误，会在 `build` 文件夹下生成 `XiZi_hc32f4a0.elf`、`XiZi_hc32f4a0.bin` 等文件。其中 `XiZi_hc32f4a0.hex` 需要烧写到设备中进行运行。如果重新编写代码后需要再编译，则可执行 `make BOARD=hc32f4a0 distclean` 命令，清除配置文件和编译生成的文件。

2.5 系统烧录

将终端通过 TYPE-C 接口转接到 PC，然后使用华大官方 HDSC ISP 工具进行烧写 bin 文件，HDSC ISP 工具可参考 <https://www.hdsc.com.cn/Category83-1496>。具体的 bin 文件烧录步骤如下：

(1) 烧录 bin 文件前，需要将终端 boot 引脚拉高，上电后再按下 reset 按键，方可进入 BOOT MODE；

(2) 打开 HDSC ISP 工具后，选择芯片类型、通信波特率、bin 文件路径和对应串口，如 step1 中所示。之后勾选擦除、空白检测、编程选项，点击执行，如 step2 中所示。若烧写无误，则如 step3 提示验证成功，代表烧录文件完成；

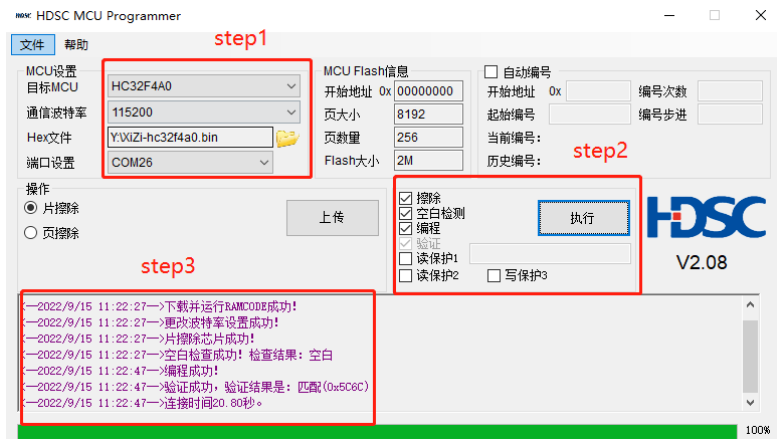


图 2-2-16 程序烧写界面

(3) 如果编译和烧写无误，将会在串口终端上看到信息打印输出。

```
serial-com27 x
A:::A A:::A i:::i II:::II II:::II T:::T T:::T T:::T
A:::A A:::A i:::i II:::II II:::II T:::T T:::T T:::T
AAAAAA AAAAAA iiiiii IIIIIIII IIIIIIII TTTTTTTTTT
*****
*****-----X Industrial Ubiquitous Operating System-----*****
*****2021 Copyright AIIT Ubiquitous-OS Team*****
*****
initialize vfs success
log sys_workq create success
initialize work sys workqueue success
initialize fatfs success
initialize libc_system success

[Letter-Shell]

Build: Sep 15 2022 11:15:35
Version: 3.0.5
Copyright: (c) 2020 Letter

letter:/$ initialize letter-shell system success
Hello, world!

letter:/$ ShowTask
*****
CURRENT RUNNING TASK[ letter-shell ] PRI[ 20 ]
SCHED POLICY [ RR_REMAINSLICE ]
*****
STAT ID TASK PRI STACK_DEPTH USED LEFT_TICKS ERROR_STAT
RUNNING 4 letter-shell 20 4096 14% 5 000
SUSPEND 3 sys_work 23 2048 35% 10 000
READY 1 ktaskidle0 0 1024 9% 7 000
SUSPEND 0 ZombieRecycleTask 31 2048 6% 9 000
*****
letter:/$
```

图 2-2-17 终端输出界面

2.6 系统调试

ARM 矽琇硬件终端提供了 SWD 接口，其可借助 J-Link、ST-Link 等调试器来与主机进行连接和调试。

主机操作系统为 Ubuntu 18.04，在完成上文的基本编译环境配置后，进行 GDB 调试的步骤如下：

1、安装 arm-none-eabi-gdb:

```
$ sudo apt-get install gdb-multiarch
$ ln -s /usr/bin/gdb-multiarch /usr/bin/arm-none-eabi-gdb
```

2、安装 J-Link 驱动和工具:

需要先在 JLink 官网上下载 deb 安装包，然后进行解压安装。下载页面链接为：<https://www.segger.com/downloads/jlink/>。本书所下载的版本为 V7.86b。

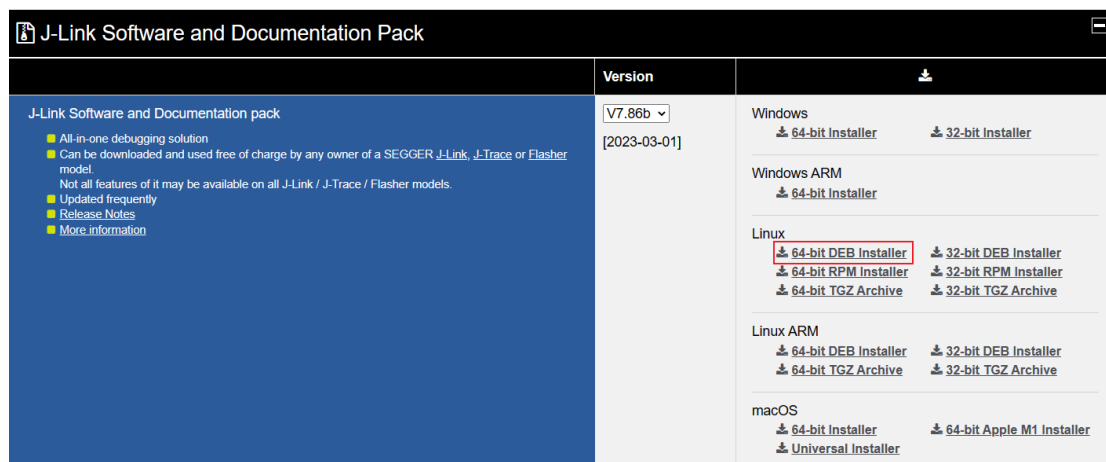


图 2-2-18 JLink 下载界面

然后使用安装命令,示例命令如下:

```
$ sudo dpkg -i JLink_Linux_V786b_i386.deb
```

3、添加 HC32F4A0 补丁:

安装完成需要跳转到 JLink 的安装目录/opt/SEGGER/JLink_V786a 下，添加 HDSC 的相关补丁包，补丁包的下载地址为：<https://www.hdsc.com.cn/cn/Index/downloadFile/modelid/66/id/63/key/2>，下载后相关路径为 hc32f4a0_ddl_Rev1.4.0/mcu/MDK/config。

4、打开 JLinkGDBServer 开启本地端口，运行 arm-none-eabi-gdb 进行调试:

在完成硬件连接后，使用如下命令开启 JLinkGDBServer 端口。

```
$ JLinkGDBServer -if SWD -device HC32F4A0_2M
```

如图 2-2-19 所示，在本示例中，对应的 TCP/IP 端口号为 2331。

```

Connecting to J-Link...
J-Link is connected.
Firmware: J-Link Pro V4 compiled Sep 22 2022 15:00:37
Hardware: V4.00
S/N: 174310352
Feature(s): RDI, FlashBP, FlashDL, JFlash, GDB
Checking target voltage...
Target voltage: 3.27 V
Listening on TCP/IP port 2331
Connecting to target...
Halting core...
Connected to target
Waiting for GDB connection...

```

图 2-2-19 打开 JLinkGDBServer 开启本地端口

另打开一个 shell 终端，运行 GDB 调试程序并连接至 JLinkGDBServer 端口，示例命令为：

```

$ sudo arm-none-cabi-gdb ./build/XiZi-hc32f4a0.elf --eval-command="target remote
127.0.0.1:2331"

```

其中“127.0.0.1:2331”应当对应于上一步骤中打开的 JLinkGDBServer 所打开的本地端口。

完成以上步骤后可以开始调试，在 GDB 控制台输入“load”命令即可将二进制文件烧录进入 HC32F4A0 中，最终终端显示如图 2-2-20 所示。

```

Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./build/XiZi-hc32f4a0.elf...done.
Remote debugging using 127.0.0.1:2331
HwHardFaultException (ExceptionInfo=0x1ffe213c <working_dir+164>)
  at /home/xiuos/test/workspace/Ubiquitous/XiZi_IIoT/arch/arm/shared/prepare_a
hwstack.c:334
334         while (1);
(gdb) load
Loading section .interrupts, size 0x280 lma 0x0
Loading section .icg_sec, size 0x60 lma 0x400
Loading section .text, size 0x37464 lma 0x480
Loading section .init, size 0x4 lma 0x378e4
Loading section .fini, size 0x4 lma 0x378e8
Loading section .ARM.exidx, size 0x8 lma 0x378ec
Loading section .fini_array, size 0x4 lma 0x378f4
Loading section .data, size 0xc70 lma 0x378f8
Start address 0xbc38, load size 230344
Transfer rate: 44989 KB/sec, 10470 bytes/write.

```

图 2-2-20 开启 GDB 页面

此外，调试环境还支持 Keil 或 IAR 等，需提前安装 IDE 支持包，下载可通过官网进行下载，具体参考链接为：<http://www.xhsc.com.cn/Productlist/info.aspx?itemid=1800>。

第3章 基于 RISC-V 架构的矽璜硬件终端

本节基于 RISC-V 矽璜硬件终端介绍硬件配置和 XiUOS 的编译及烧录方法，推荐使用操作系统 Ubuntu18.04，下载地址 <https://ubuntu.com/download/desktop>。开发工具推荐使用 VSCode，下载地址 <https://code.visualstudio.com/>，推荐下载地址为 http://vscode.cdn.azure.cn/stable/3c4e3df9e89829dce27b7b5c24508306b151f30d/code_1.55.2-1618307277_amd64.deb。

3.1 RISC-V 硬件系统介绍

RISC-V 矽璜硬件终端的模组分布如图 2-3-1 所示：

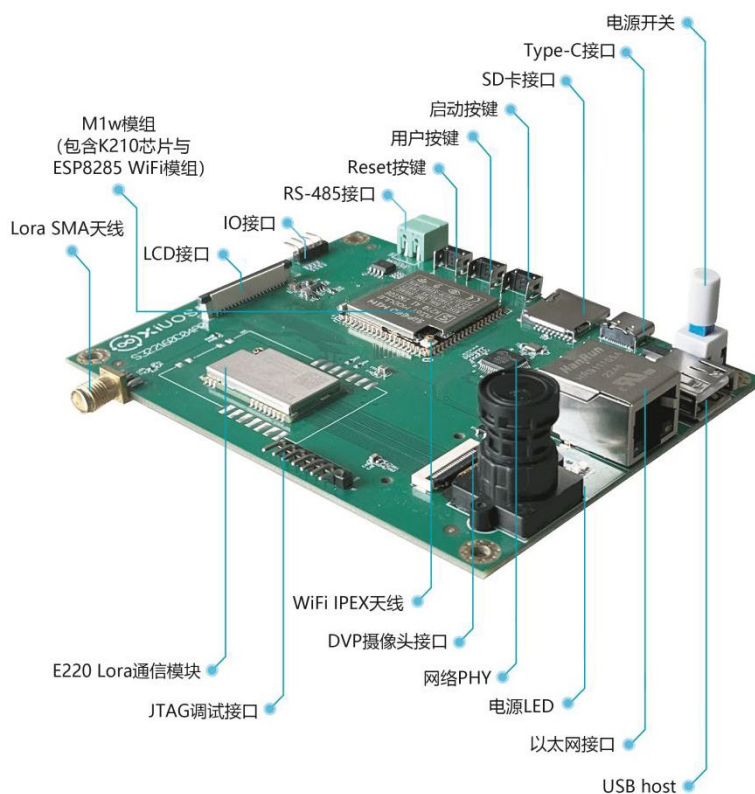
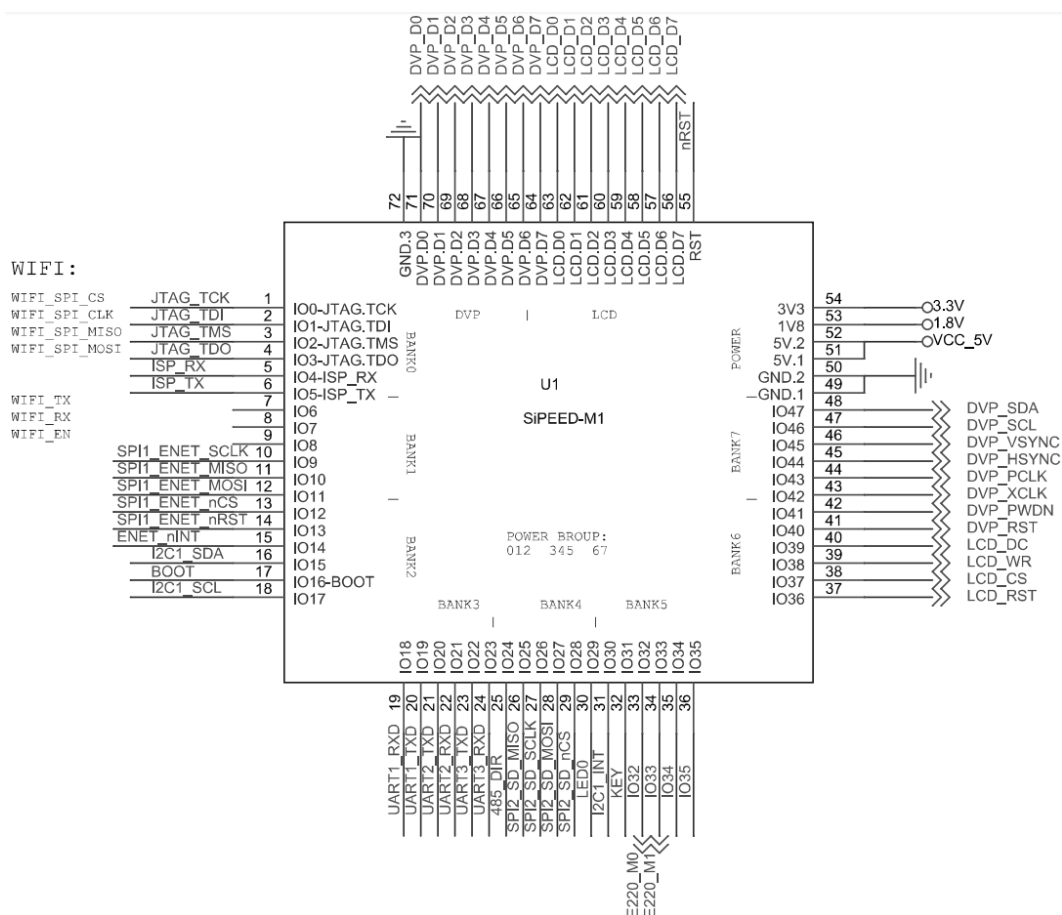


图 2-3-1 edu-riscv64 矽璜硬件终端

采用的芯片型号为 Kendryte K210。Kendryte K210 是集成机器视觉与机器听觉能力的系统级芯片 (SoC)。使用台积电 (TSMC) 超低功耗的 28 纳米先进制程，具有双核 64 位

处理器，拥有较好的功耗性能，稳定性与可靠性。其中，CPU 标称频率为 400MHz，片上 S RAM 为 8MB。同时，K210 具备丰富的外设单元，分别是：DVP、JTAG、OTP、FPIOA、GPIO、UART、SPI、RTC、I²S、I²C、WDT、Timer 与 PWM，可满足海量应用场景。

RISC-V 砒璜硬件终端根据 K210 芯片的特点和教育需求，配置了 Ethernet 有线网口、L ora、RS485、TF 卡、USB、摄像头、LCD 显示屏幕等外设组件，可以全面、充分地发挥其各种功能。同时，终端通过 PL2303 接口转换芯片实现了 ISP 下载，用户可以直接将终端通过 Type-C 连接至 PC 进行程序烧录。下文将对各外设组件功能的实现方式和电子线路连接进行简要介绍，具体的连接表另可参照附录 B。



块与 K210 芯片存在 UART 和 SPI 两种连接方式，分别对应于 IO6-IO8 引脚和 IO0-IO3 引脚。

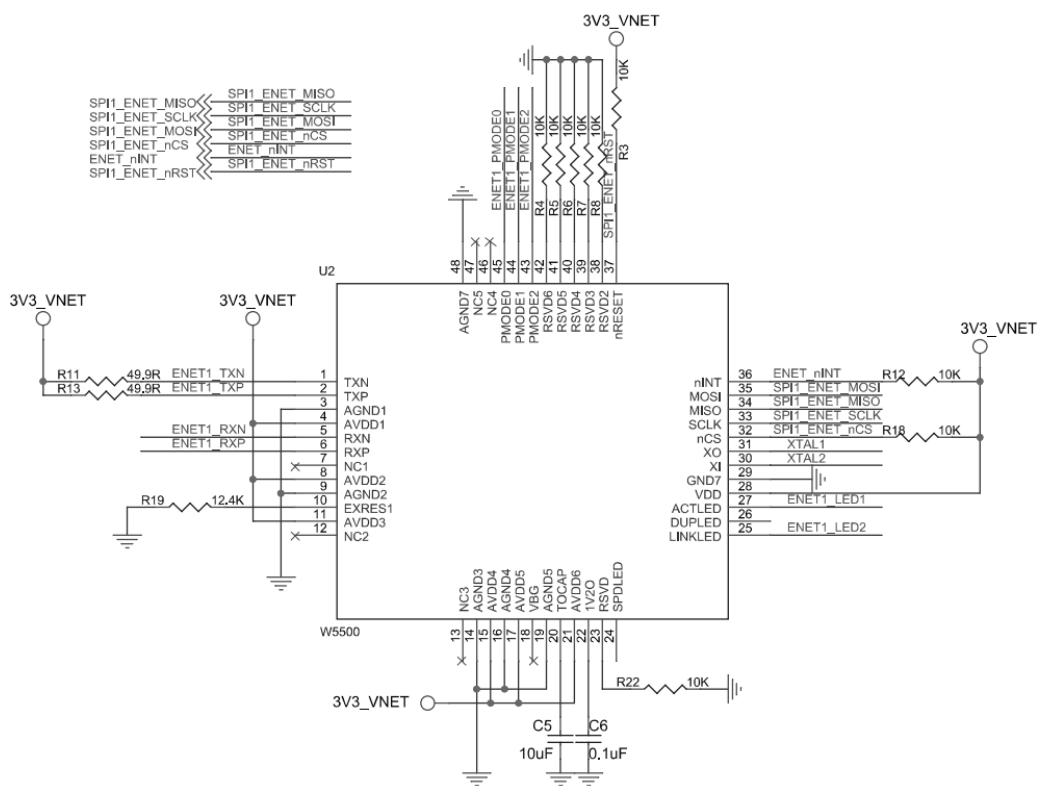
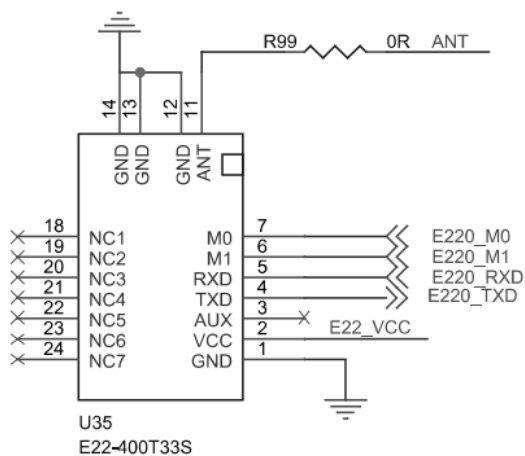


图 2-3-3 W5500 芯片原理图

如图 2-3-3 所示，板内通过集成 W5500 芯片以实现 Ethernet 通信，可通过有线网口连接至网络。W5500 芯片的控制引脚连接到 K210 芯片的 IO9-IO14 引脚，其通信协议为 SPI。



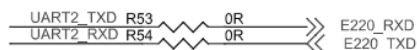


图 2-3-4 E22 Lora 模块电路原理图

搭载的 E22 Lora 通信模块如图 2-3-4 所示，该款通信模块的工作频段为 410.125~493.125MHz，支持定点传输、广播传输等通信方式，最大通信距离可达 5km。其通过 UART 串口与 K210 芯片进行信息交换，模块 UART 引脚 RXD、TXD 与控制引脚 M0、M1 分别连接到 K210 芯片的 IO20、IO21、IO32 和 IO33，可以实现透明传输或 WOR 收发传输。

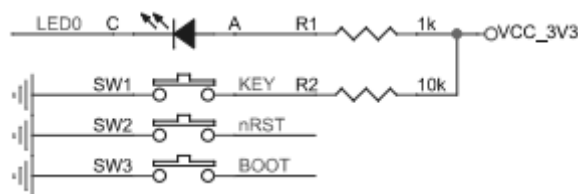


图 2-3-5 按键原理图

LED 与硬件按键如图 2-3-5 所示，其中 LED0（对应 IO29）连接在发光二极管的阴极引脚。SW2、SW3 按键分别对应终端的 nRST 与 BOOT，SW1 按键可自由开发，对应于 K210 芯片的 IO31。

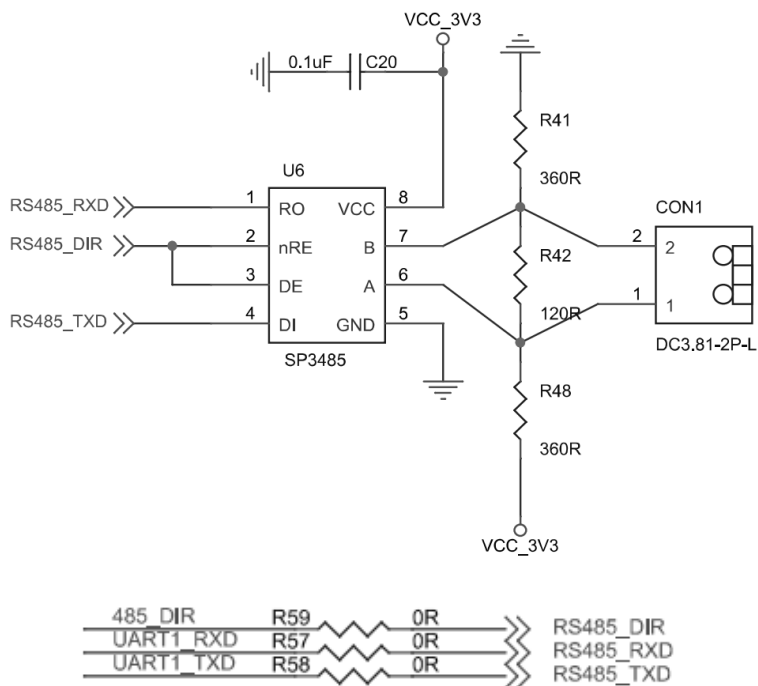


图 2-3-6 SP3485 RS485 收发芯片电路图

图 2-3-8 CH376T 文件管理控制芯片原理图

如图 2-3-8 所示，USB 接口依靠 CH376T 文件管理控制芯片来实现。该款芯片的通信协议为 UART 协议。CH376T 的 TXD 接收引脚连接 K210 的 IO22，RXD 引脚连接 IO23。

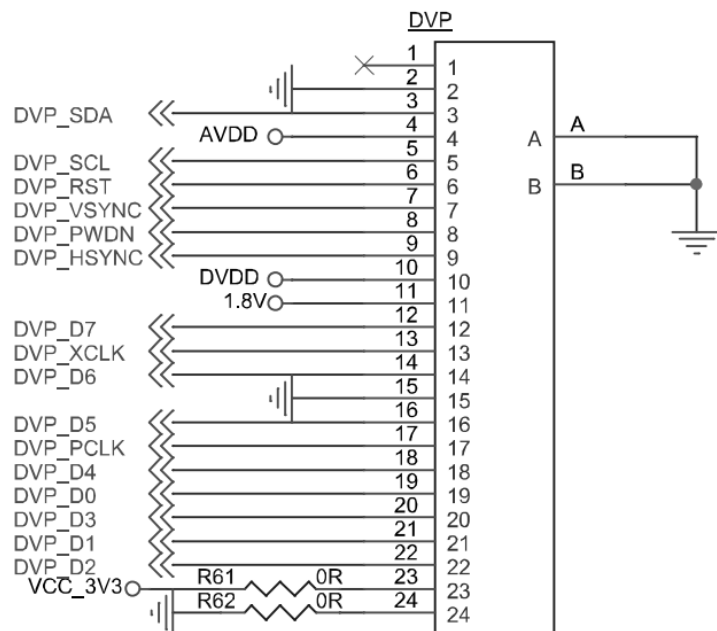


图 2-3-9 DVP 摄像头接口底座原理图

如图 2-3-9 所示，RISC-V 砒璜硬件终端设计有摄像头接口可连接 OV2640 等摄像模块。摄像头接口对应于 K210 芯片的 IO40-IO47 以及 DVP.D0 至 DVP.D7 专用引脚，其支持的通信协议包括 DVP 通信协议与 SCCB 摄像头控制协议。

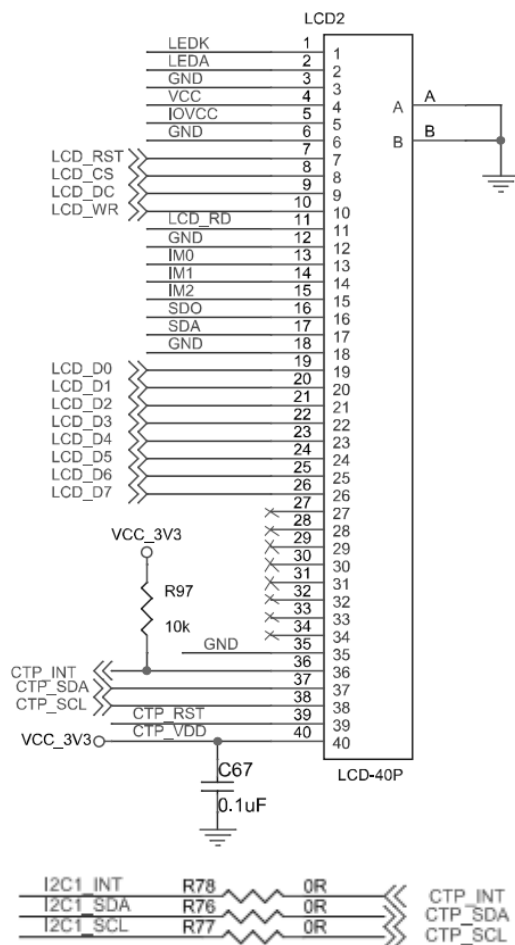


图 2-3-10 LCD 和 Touch 接口电路原理图

如图 2-3-10 所示，LCD 接口配备有专用的 LCD 显示模组，其显示分辨率为 320*320，同时支持触摸等交互方式。LCD 显示模组的显示接口对应于 K210 的 IO36-IO39 及 LCD.D0-LCD.D7 等硬件接口；电容式触摸屏主要通过 I2C 协议实现，SDA、SCL 及 INT 引脚分别对应于 K210 的 IO15、IO17 和 IO30。

在外部拓展性方面，RISC-V 矽琇硬件终端支持 RS485、USART、I2C 等通信协议，可以与数量众多的传感器等外部模块进行通信，以进一步增强终端的功能。同时，RISC-V 矽琇硬件终端支持 JTAG 接口调试，通过相关开发调试工具，可实现单步、断点等实时调试功能。

3.2 源码下载

RISC-V 矽琇硬件终端采用 XiUOS 源码，下载方式同 2.2 源码下载相同，这里不再叙述。

3.3 工具安装

(1) 依赖包安装:

Ubuntu 系统安装好之后，在安装软件之前，或者遇到依赖问题之时，需要安装一些依赖包，它聚合了一些常用的依赖。另外，还需要安装例如 gcc 编译器和 make 命令工具等。具体用到的两条 Linux 命令如下所示。

```
$ sudo apt install build-essential pkg-config  
  
$ sudo apt install gcc make libncurses5-dev openssl libssl-dev bison flex libelf-dev autoconf  
libtool gperf libc6-dev git
```

(2) 配置工具安装:

Linux 编译器通过.config 文件确认哪些代码编译进内核，哪些被裁减掉，menuconfig 是生成.config 的一个工具。总结就是 menuconfig 是编辑内核的工具；kconfig 定义 menuconfig 菜单选项；生成的.config 是最后 menuconfig 配置完生成的文件；makefile 根据.config 宏定义判断是否编译驱动。

对于 XiUOS 的编译，首先需要下载对应的裁减配置工具：kconfig-frontends，下载地址 <https://www.gitlink.org.cn/xuos/kconfig-frontends>，然后进行安装即可。上述下载和安装操作可通过 Linux 命令执行，如下所示。第 1 条命令用于创建和进入一个文件夹 kfrontends；第 2 条命令是从确实开源平台下载一个 kconfig-frontends 至文件夹 kfrontends；第 3 条命令是运行 XiUOS 的 build 进行配置工具的安装。

```
$ mkdir kfrontends && cd kfrontends  
  
$ git clone https://gitlink.org.cn/xuos/kconfig-frontends.git  
  
$ ./xs_build.sh
```

(3) 编译工具链安装:

做 C 开发，特别是嵌入式方向，需要涉及编译工具链和交叉编译工具链的相关内容，即 C 程序需要经过 gcc 等工具编译成二进制程序才能被计算机使用。这里的 gcc 通常是泛指，包括 gcc、g++、ar、as、ld 等一系列工具，这些工具用在程序文件的预编译、编译、链接等整个过程，这整套工具就被称为编译工具链（Toolchain）。

同样，XiUOS 的编译也需要安装编译工具链，对于 RISC-V 的 riscv-none-embed-，默认安装到 Ubuntu 的/opt/。下载网址：<https://github.com/ilg-archived/riscv-none-gcc/releases>，下载并解压的 Linux 命令如下。

```
$ tar -xjf gnu-mcu-eclipse.tar.bz2 -C /opt/
```

将上述解压的编译工具链的路径添加到 board/edu-riscv/config.mk 文件当中，使用的 Linux 命令如下。

```
$export CROSS_COMPILE ?=/opt/gnu-mcu-eclipse/riscv-none-gcc/8.2.0-2.1-20190425-1021/bin/riscv-none-embed-
```

若 CROSS_COMPILE 变量定义语句已经存在，将它替换成上面的语句。

3.4 代码编译

打开终端，输入 code .即可打开 XiUOS 源码工程，然后打开 xiuos/APP_Framework/Applications 文件夹下 main.c，在输出函数中写入 Hello, world!!!\n Running on edu-riscv\n，即可完成测试代码编写。

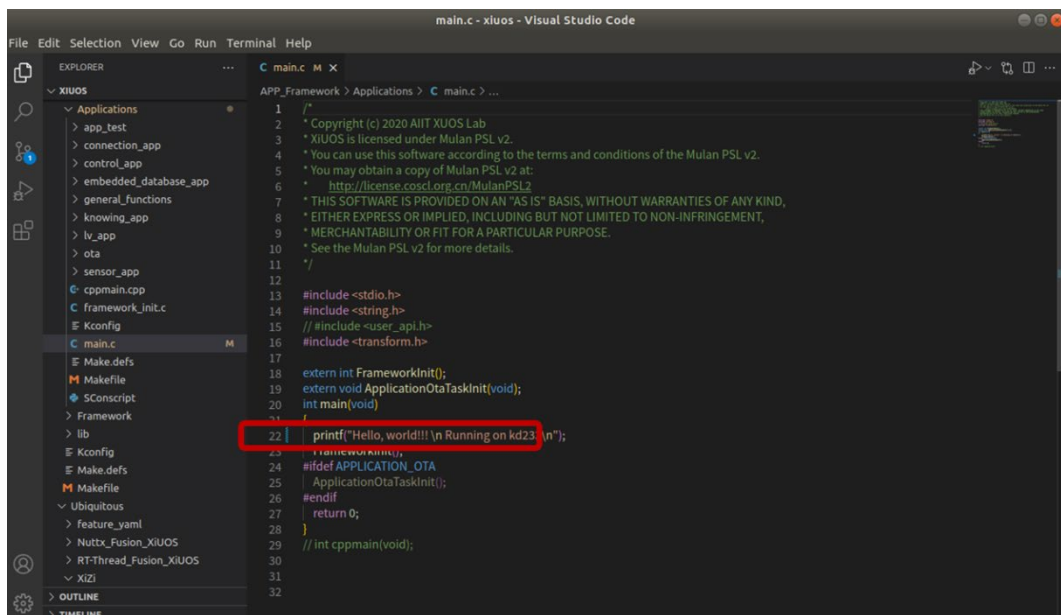


图 2-3-11 代码编辑示意图

代码编译具体步骤如下：

(1) 在 VScode 的“命令终端”，执行命令 `make menuconfig BOARD=edu-riscv64`，生成配置文件；在编译版本时，BRARD 需要选择 edu-riscv64；

(2) 在 `menuconfig` 界面配置需要关闭和开启的功能，按回车键进入下级菜单，按 Y 键选中需要开启的功能，按 N 键选中需要关闭的功能，配置结束后选择 `Exit` 保存并退出。本实验无需选择任何选项，所以双击 `ESC` 结束选择，继续操作即可；

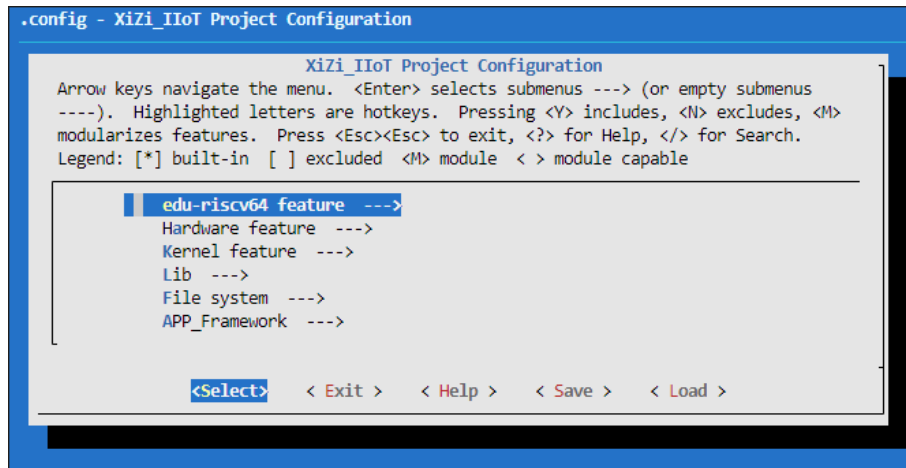


图 2-3-12 edu-riscv64 配置界面

(3) 继续在终端执行 `make` 命令，进行编译；

(4) 如果编译正确无误，会在 `build` 文件夹下生成 `XiZi_edu-riscv64.bin`、`XiZi_edu-riscv64.elf` 等文件。其中 `XiZi_edu-riscv64.bin` 需要烧写到设备中进行运行。

如果重新编写代码后需要再编译，则可执行 `make BOARD=edu-riscv64 distclean` 命令，清除配置文件和编译生成的文件。

3.5 系统烧录

连接终端串口（即 Type-C 口）到 USB 接口，终端即可通电开启。首先，安装 K-Flash，它是一款用于嵌入式开发的 Flash 烧录软件，具有文件烧录与校验、数据擦除、数据查空和数据读取等功能；然后，使用 K-Flash 工具进行烧录 `bin` 文件。具体的安装和 `bin` 文件烧录步骤如下：

(1) 执行以下命令安装 K-Flash 工具：

```
$ sudo apt install python3 python3-pip
$ sudo pip3 install pyserial
$ sudo pip3 install pyelftools
$ sudo pip3 install kflash
```

如果最后一步安装 kflash 出现错误，可以尝试以下命令。

```
$ sudo python -m pip install kflash
$ sudo python3 -m pip install kflash
$ sudo pip install kflash
$ sudo pip2 install kflash
```

(2) 代码根目录下执行 K-Flash 工具烧录，-p 为 USB 端口号，视实际情况而定：

```
$ sudo kflash -t build/XiZi_edu-riscv64.bin -p /dev/ttyUSB0
```

(3) 如果编译和烧写无误，将会在串口终端上看到 hello world 相关的打印信息，表明 XiUOS 成功编译和烧录至 K210 芯片（即 edu-riscv64 矽璞硬件终端）。

```
*****
AAAAAAAA      AAAAAAAAA      IIIIIIII      IIIIIIII      IIIIIIII      TTTTTTTTTTTTTT
A:::~A      A:::~A      I:::~I      I:::~I      II:::~II      TT:::~TTTTTTTTTT
A:::~A      A:::~A      iii~i      I:::~I      II:::~II      T:::~TTTTTTTTTT
A:::~A      A:::~A      i:::~i      II:::~II      I:::~III:::~III:::~IT:::~T      TTTTTT
AAA:::~A      A:::~AAA      iii~i      I:::~I      I:::~I      I:::~IT:::~T
  A:::~A      A:::~A      I:::~I      I:::~I      I:::~I      I:::~IT:::~T
    A:::~A:::~A      iiii~i      I:::~I      I:::~I      I:::~I      I:::~T:::~TTTT
      A:::~A:::~A      i:::~i      I:::~I      I:::~I      I:::~I      TT:::~TTTTTT
        A:::~A:::~A      i:::~i      I:::~I      I:::~I      I:::~I      TTT:::~TTTT
          A:::~A:::~A      i:::~i      I:::~I      I:::~I      I:::~I      TTTTT:::~T
            A:::~A      i:::~i      I:::~I      I:::~I      I:::~I      T:::~T
              AAA:::~A      A:::~AAA      i:::~i      I:::~I      I:::~I      I:::~I      T:::~T
                A:::~A      A:::~A      i:::~i      I:::~III:::~III:::~I      I:::~III:::~III:::~ITTTTTT      T:::~T
                  A:::~A      A:::~A      i:::~i      II:::~II:::~II      II:::~II:::~II      T:::~TTTTTTTT:::~T
                    A:::~A      A:::~A      i:::~i      II:::~II:::~II      II:::~II:::~II      T:::~TTTTTTTT:::~T
                      AAAAAA      AAAAAA      iiii~i      IIIIIIII      IIIIIIII      TTTTTTTTTTTTTT
*****
*****-----X Industrial Ubiquitous Operating System-----*****
*****2021 Copyright AIIT Ubiquitous-OS Team*****
*****
initialize vfs success
log sys_workq create success
initialize work sys workqueue success
initialize fatfs success
initialize libc_system success

Letter-shell

Build:      Feb 27 2023 16:16:34
Version:    3.0.5
Copyright:  (c) 2020 Letter

letter:/$ initialize letter-shell system success
Hello, world!
```

图 2-3-13 edu-riscv64 矽璞硬件终端显示界面

(4) XiUOS 提供多种命令行功能，输入 Help，会显示所有的已有命令，如图 2-3-14 所示。

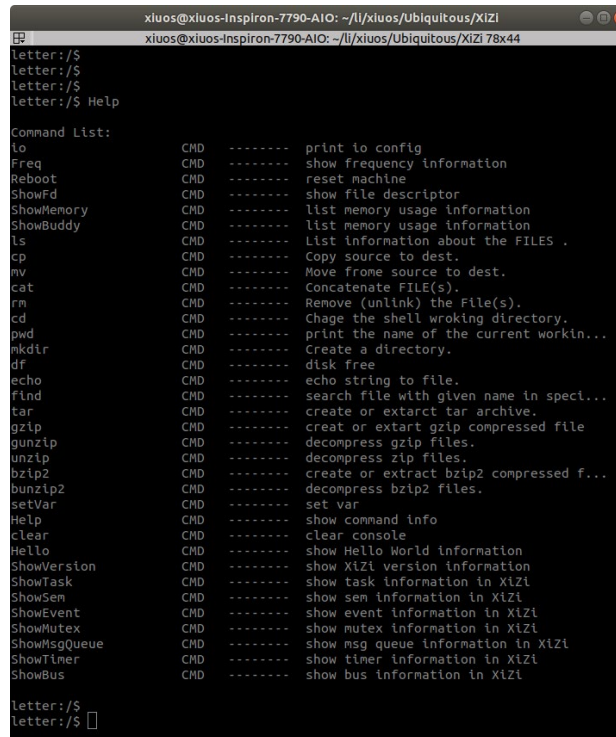


图 2-3-14 Help 命令界面

Command List 中列出了全部的 XiUOS 命令。例如，ShowVersion 命令是用于向用户显示当前 XiUOS 的版本。在控制台输入 ShowVersion，即可显示如下信息：

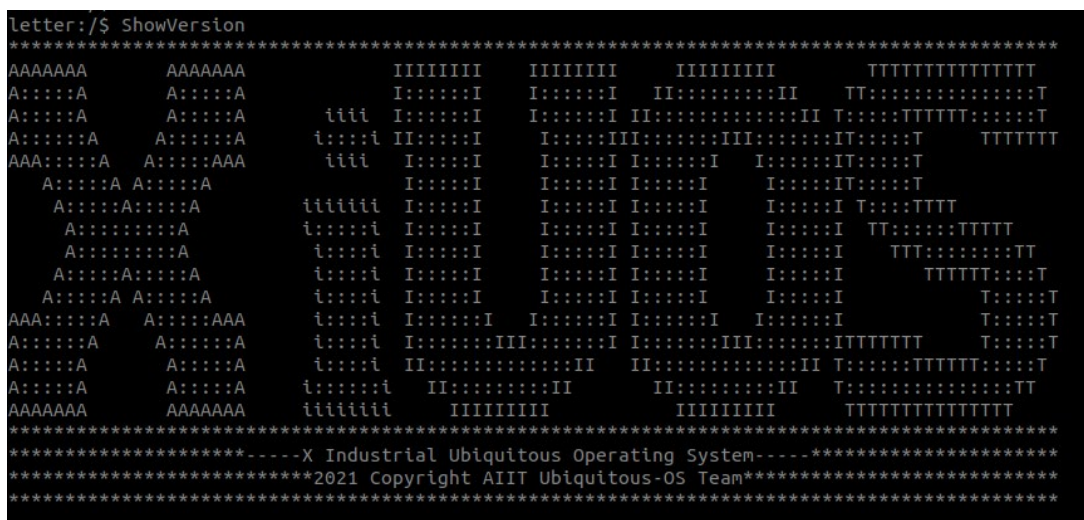


图 2-3-15 ShowVersion 命令功能展示

3.6 系统调试

edu-riscv64 矽璜硬件终端提供 SWD 调试接口，可以通过 Jlink 调试器进行在线调试。

在进行 GDB 调试前需要进行硬件跳线，将 IO8 与 GND 直接短接。

主机操作系统为 Ubuntu 18.04。具体调试步骤如下：

- 下载并安装 JLink 驱动，其下载地址为：<https://www.segger.com/downloads/jlink>。其安装指令为：

```
$ dpkg -i JLink_Linux_V632i_x86_64.deb
```

在 JLink 安装完成后可通过 JLinkExe 命令检查驱动是否完成安装和正常运行。

```
$ JLinkExe

SEGGER J-Link Commander V7. 0a (Compiled Apr 16 2021 15:36:25)

DLL version V7.00a, compiled Apr 16 2021 15:36:09

Connecting to J-Link via USB...O.K.

Firmware: J-Link OB-K22-SiFive compiled Mar  3 2021 14:13:05

Hardware version: V1.00

S/N: 979015641

VTref=3.300V

Type "connect" to establish a target connection, '?' for help

J-Link>
```

- 下载并安装 OpenOCD, 下载地址为：<https://github.com/kendryte/openocd-kendryte/releases>，通过如下指令进行安装：

```
$ sudo apt install libusb-dev libftdi-dev libhidapi-dev
$ sudo mv kendryte-openocd-0.2.3-ubuntu64.tar.gz /opt
$ cd /opt
$ sudo tar -zxvf kendryte-openocd-0.2.3-ubuntu64.tar.gz
```

安装完成后，需要修改 OpenOCD 的配置文件，命令如下：

```
$ sudo vim /opt/kendryte-openocd/tcl/openocd.cfg
```

如图 2-3-16 所示，需要修改内容如下，其中，“504504051”是 JLink 的序列号，序列号从 JLink 的背面标签可以查看，也可以在 JLink 连接计算机后执行 JLinkExe 命令看到。

```
# debug adapter
interface jlink
jlink serial 504504051

transport select jtag
adapter_khz 3000

# server port
gdb_port 3333
telnet_port 4444

# add cpu target
set _CHIPNAME riscv

jtag newtap $_CHIPNAME cpu -irlen 5 -expected-id 0x04e4796b

set _TARGETNAME $_CHIPNAME.cpu
target create $_TARGETNAME riscv -chain-position $_TARGETNAME

# command
init
halt

-- 插入 --
```

图 2-3-16 OpenOCD 配置文件修改

完成 OpenOCD 的安装和配置后，可以通过如下命令打开 OpenOCD：

```
$ cd /opt/kendryte-openocd
$ ./bin/openocd -f ./tcl/openocd.cfg
```

- 打开 OpenOCD 后，可直接通过运行 GDB 程序开始调试。示例命令如下：

```
$ riscv-none-embed-gdb (workspace)/build/XiUOS_kd233.elf --eval-command="target remote
127.0.0.1:3333"
```

其中，（workspace）为实际操作的 XiUOS 项目的绝对路径，XiUOS_kd233.elf 是编译生成的文件。127.0.0.1 表示本地 IP 地址，也可以指定远程地址。3333 是 OpenOCD 监听的端口号。成功连接 OpenOCD 后，可以使用 load、break、continue 等 gdb 命令来调试。

第3篇 XiUOS 内核 (XiZi)

第1章 任务管理

1.1 任务管理介绍

1.1.1 任务管理简介

任务 (Task) 是 XiUOS 中处理器使用权分配的最小单位。每个任务有自己的程序栈与寄存器上下文，在多处理器平台上可以互不干扰地同时运行，但单个处理器上任意时刻只能有一个任务在运行。用户可以使用 XiUOS 提供的接口创建任意数量的任务。

XiUOS 中的任务在任意时刻都处于就绪 (Ready)、运行 (Running)、阻塞/挂起 (Suspend)、退出 (Close) 四种状态之一。状态之间的变化关系如下图所示。

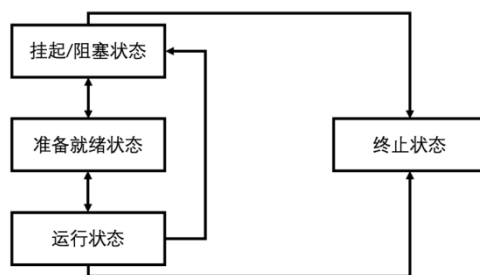


图 3-1-2 任务状态切换图

- 任务在创建完成后会进入就绪状态，并被加入就绪队列等待内核 CPU 调度；
- 当任务被调度开始运行时，会进入运行状态；
- 如果任务在运行过程中被更高优先级的任务抢占，则被抢占的任务会回到就绪队列，并再次进入就绪状态；
- 当任务在运行过程中申请资源失败时，任务会被挂起，并进入挂起状态。当所申请的资源能够被满足时，会回到就绪状态；
- 当任务执行完成，即从入口函数返回时，会进入终止状态，并由内核回收其相关资源。

每一个任务在内核中都有一个 TaskDescriptor 描述符，其结构体定义如下：

代码 3-3-1 TaskDescriptor 结构体

```
01 struct TaskDescriptor
```

```

02  {
03      void                *stack_point;
04      TaskDyncSchedMembeType  task_dync_sched_member;
05      TaskBaseInfoType        task_base_info;
06      #ifdef ARCH_SMP
07      TaskSmpInfoType          task_smp_info;
08      #endif
09      #if defined (KERNEL_EVENT)
10      uint32                  event_id_trigger : 29;
11      uint32                  event_mode : 3;
12      #endif
13      x_err_t                 exstatus;
14      DoubleLinklistType      link;
15      struct IdNode            id;
16      struct KTaskDone         *Done;
17  };

```

其中，`stack_point` 指向任务堆栈的起始地址；`task_dync_sched_member` 包含与任务调度相关的信息；`task_base_info` 记录任务的基本信息；`task_smp_info` 统计与多处理器相关的信息；`event_id_trigger/event_mode` 用于实现事件集机制（详见任务通信）；`exstatus` 为任务调用内核接口时最近的错误码，即用户线程在使用内核接口时可能会执行失败，此时内核接口返回-1，具体的错误码被保存在成员变量 `exstatus` 中，且在下一次调用内核接口失败时被覆盖，`link` 用于组织内核中所有的任务；`id` 用于表示一个线程；`Done` 提供所有的线程的操作函数。各复合成员的定义如下：

（1）TaskDyncSchedMember:

TaskDyncSchedMember 结构体用于记录与调度相关的信息，其结构体定义如下：

代码 3-4-2 TaskDyncSchedMember 结构体

```

01  struct TaskDyncSchedMember {
02      uint8                stat;
03      uint8                advance_cnt;
04      uint8                cur_prio;
05      ubase                origin_timeslice;
06      ubase                rest_timeslice;
07      #ifdef SEPARATE_COMPILE
08      uint8                isolation_flag;
09      void                 *isolation;

```

```

10      uint8          isolation_status;
11  #endif
12      union {
13          DoubleLinklistType  sched_link;
14          AvlNodeType          sched_avl;
15      };
16  #if KTASK_PRIORITY_MAX > 32
17      uint8  bitmap_offset;
18      uint8  bitmap_row;
19  #endif
20      uint32          bitmap_column;
21      delay_t         delay;
22  };
23  #define KTASK_INIT          0x00
24  #define KTASK_READY        0x01
25  #define KTASK_SUSPEND      0x02
26  #define KTASK_RUNNING      0x03
27  #define KTASK_CLOSE        0x04

```

其中，stat 记录任务的当前状态，可以为初始化（KTASK_INIT）、挂起（KTASK_SUSPEND）、就绪（KTASK_READY）、运行（KTASK_RUNNING）或退出（KTASK_CLOSE）；advance_cnt 表示在配置成短作业预先调度时优先处理的时间片周期个数；cur_prio 表示任务当前的优先级，用于优先级反转，该优先级可以高于任务创建时配置的优先级；origin_timeslice 表示在时间片轮转调度时，任务每次运行的时间片；isolation_flag 变量和指针 isolation 支持地址空间隔离；isolation_status 用于标志内核服务的过程（1 表示进入内核服务上下文）；sched_link 和 sched_avl 构成的联合体为就绪队列节点，XiUOS 中就绪队列可以组织为双链表（sched_link）或平衡二叉树（sched_avl）；task_timer 为任务睡眠的计数器。

（2）TaskBaseInfo 成员：

TaskBaseInfo 结构记录了任务的基本属性，包括任务的名称（name）、入口函数（func_entry）和参数（func_param）、栈大小（stack_depth）、初始优先级（origin_prio），其结构体定义如下：

代码 3-5-3 TaskBaseInfo 结构体

```

01  struct TaskBaseInfo{
02      char      name[NAME_NUM_MAX];
03      void      *func_entry;

```

```

04     void      *func_param;
05     uint8      origin_prio;
06     uint32     stack_depth;
07     void      *stack_start;
08 };

```

(3) TaskSmpInfo 成员:

TaskSmpInfo 结构包含多处理器相关的信息，其成员分别表示该任务绑定的 CPU ID 与正在运行的 CPU ID，其结构体定义如下：

代码 3-6-4 TaskSmpInfo 结构

```

01 struct TaskSmpInfo{
02     uint8      combined_coreid;
03     uint8      running_coreid;
04     uint16     critical_lock_cnt;
05 };

```

1.1.2 任务创建接口

XiUOS 中定义了一个任务函数接口，用于用户态的任务创建。任务的各个属性由 **utask** 结构表示，包括任务的名称、入口函数及参数、栈大小和优先级，在调用该函数时需要传入该结构的实例用于配置任务属性。任务创建成功后，内核会为其分配指定大小的栈及其他结构（如 **struct TaskDescriptor**），并返回任务 id。任务创建相关的函数接口定义如下：

代码 3-7-5 任务创建接口

```

01 struct utask{
02     char        name[NAME_NUM_MAX];
03     void        *func_entry;
04     void        *func_param;
05     uint32      stack_size;
06     uint8       prio;
07 };
08 typedef struct utask UtaskType;
09 int32_t UserTaskCreate (UtaskType task)

```

另外，XiUOS 中还定义一系列任务管理函数：

- **int32_t UserTaskCreate (UtaskType task)**，该函数用于创建一个任务。其中，参数 **task** 是需要创建的任务，类型是上述的 **utask**。

- `x_err_t UserTaskStartup (int32_t id)`, 该函数用于就绪一个任务, 交由调度器开始调度执行。其中, 参数 `id` 表示待就绪的任务 ID。
- `x_err_t UserTaskDelete (int32_t id)`, 该函数用于删除一个任务, 强制其进入退出状态, 若删除成功则返回 EOK, 否则返回-ERROR。其中, 参数 `id` 表示待删除的任务 ID。
- `x_err_t UserTaskCoreCombine (int32_t id, uint8_t core_id)`, 该函数用于将任务绑定至指定的处理器上, 若绑定成功则返回 EOK, 否则返回-ERROR。其中, 参数 `id` 表示待绑定的任务 ID; `core_id` 表示待绑定的处理器 ID。
- `x_err_t UserTaskCoreUnCombine (int32_t id)`, 该函数用于解除任务与处理器的绑定, 若解除成功则返回 EOK, 否则返回-ERROR。其中, 参数 `id` 表示待解除绑定的任务 ID。
- `x_err_t UserTaskDelay (int32_t ms)`, 该函数用于将当前任务挂起一定时间, 单位为 `ms`, 挂起时间结束后, 任务会进入就绪状态, 等待系统调用。其中, 参数 `ms` 表示任务的挂起时间, 单位为 `ms`。

1.1.3 任务调度简介

内核会对系统中的所有任务按照一定策略（抢占式优先级或先来先服务）进行调度, 最大限度地利用处理器资源。任务调度即从系统就绪队列中按一定策略选择某一任务, 使其进入运行状态的过程。XiUOS 支持以下三种调度方式:

- 时间片轮转 (Round-Robin, RR) 调度, 即任务按照分配的时间片执行, 时间片结束, 调度一个新的就绪任务执行, 当前任务重新就绪, 等待下一次的调度。
- 时间片轮转+剩余时间片 (RR_REMAINSLICE) 调度, 在 RR 调度策略中, 每个任务只运行一个剩余时间片, 结束后就将该任务放到等待队列中。若要使某个任务运行 `n` 个剩余时间片, 那么采用 RR_REMAINSLICE 的调度方式, 其中剩余时间片可以自定义设置。
- 先进先出 (First Input First Output, FIFO) 调度, 即任务按照被创建的顺序依次被执行。当一个任务运行完成后, 系统才会让下一个任务开始运行。

1.1.4 任务通信简介

XiUOS 提供多种任务间通信机制, 包括消息队列、信号量、互斥量与事件集。

(1) 消息队列:

消息队列（message queue）提供可供多个任务读或写的消息缓冲区，其中消息指固定长度的任意数据块。消息队列的容量有限，当消息队列满时，向消息队列写入的任务会被挂起；当消息队列空时，从消息队列读取的任务会被挂起。消息队列的结构定义及成员含义如下所示：

代码 3-8-6 消息队列结构定义

```

01 struct MsgQueue{
02     struct IdNode      id;           //用于唯一标识一个消息队列
03     void               *msg_buf;     //用于存储消息的缓冲区的首地址
04     int16              index;        //当前缓冲区中第一个消息的序号
05     uint16             num_msgs;     //当前缓冲区中的消息数量
06     uint16             each_len;     //每条消息的大小，单位为字节
07     uint16             max_msgs;     //缓冲区中最多能存放的消息数量
08     DoubleLinklistType send_pend_list; //被挂起的发送任务链表
09     DoubleLinklistType recv_pend_list; //被挂起的接收任务链表
10     DoubleLinklistType link;         //系统中所有消息队列构成的链表
11     struct MsgQueueDone *Done;       //消息队列的操作函数集
12 };

```

另外，XiUOS 中还定义一系列消息队列函数接口：

- `int32_t UserMsgQueueCreate (size_t msg_size, size_t max_msgs)`，该函数用于创建一个消息队列。创建成功后，新的消息队列会被加入内核的消息队列管理链表，并返回该消息队列的 ID，ID 默认范围 0-255，可配置。其中，参数 `msg_size` 表示每条消息的长度，单位为字节；`max_msgs` 表示缓冲区中最多存放的消息数量。
- `x_err_t UserMsgQueueDelete (int32_t mq)`，该函数用于删除一个已创建的消息队列。其中，参数 `mq` 表示待删除的消息队列 ID。
- `x_err_t UserMsgQueueSendwait (int32_t mq, const void *buffer, size_t size, int32_t wait_time)`，该函数用于等待一段时间后，再向消息队列发送一个消息。若消息发送成功则返回 EOK，若不成功（等待超时）则返回-ETIMEOUT。其中，参数 `mq` 表示目标消息队列 ID；`buffer` 表示消息数据首地址；`size` 表示消息长度；`wait_time` 表示等待时间上限，单位 ms，若为 0，则不等待，若为-1，表示永久等待。

- `x_err_t UserMsgQueueSend (int32_t mq, const void *buffer, size_t size)`, 该函数用于向消息队列发送一个消息。若消息发送成功则返回 EOK, 若不成功则返回-ETIMEOU T。其中, 参数 mq 目标消息队列 ID; buffer 表示消息数据首地址; size 表示消息长度。
- `x_err_t UserMsgQueueUrgentSend (int32_t mq, const void *buffer, size_t size)`, 该函数用于向消息队列优先发送一个消息。若消息发送成功则返回 EOK, 若不成功(等待超时)则返回-ETIMEOUT。其中, 参数 mq 目标消息队列 ID; buffer 表示消息数据首地址; size 表示消息长度。
- `x_err_t UserMsgQueueRecv (int32_t mq, void *buffer, size_t size,int32_t wait_time)`, 该函数用于从消息队列接收一个消息。若消息接收成功则返回 EOK, 若不成功(等待超时)则返回-ETIMEOUT。其中, 参数 mq 表示来源消息队列 ID; buffer 表示用于接收消息数据的缓冲区; size 表示缓冲区大小; wait_time 表示等待时间上限, 单位 ms; 若为 0, 则不等待。
- `x_err_t UserMsgQueueReinit (int32_t mq)`, 该函数用于将一个消息队列复位。其中, 参数 mq 表示消息队列 ID。

(2) 信号量:

信号量(semaphore)具有一个给定的初值。任务可以获取或释放一个信号量。当任务获取信号量时, 信号量的值递减, 释放信号量时, 信号量的值递增。当信号量的值递减至 0 时, 后续尝试获取信号量的任务会被挂起。当任务释放信号量时, 内核会从该信号量的挂起队列上唤醒一个任务。信号量可以实现任务间的同步与互斥。信号量结构定义及成员含义如下所示:

代码 3-9-7 信号量结构定义

```
01 struct Semaphore{
02     struct IdNode      id;          //信号量 ID, 用于唯一标识一个信号量
03     uint16             value;        //信号量的当前值
04     DoubleLinklistType pend_list;    //挂起任务链表
05     DoubleLinklistType link;         //系统中所有信号量构成的链表
06 };
```

另外, XiUOS 中还定义一系列信号量函数接口:

- `sem_t UserSemaphoreCreate (uint16_t val)`, 该函数用于创建一个信号量。创建成功后, 新的信号量会被加入内核的信号量管理链表, 并返回该信号量的 ID, ID 默认范围 0-255, 可配置。其中, 参数 `val` 表示信号量的初值。
- `x_err_t UserSemaphoreDelete (sem_t sem)`, 该函数用于删除一个已创建的信号量。其中, 参数 `sem` 表示待删除的信号量 ID。
- `x_err_t UserSemaphoreObtain (sem_t sem, int32_t wait_time)`, 该函数用于获取一个信号量。若获取成功则返回 EOK, 若不成功(等待超时)则返回-ETIMEOUT。其中, 参数 `sem` 表示待获取的信号量 ID; `wait_time` 表示等待时间上限, 单位 ms, 若为 0, 则不等待。
- `x_err_t UserSemaphoreAbandon (sem_t sem)`, 该函数用于释放一个信号量。其中, 参数 `sem` 表示待释放的信号量 ID。
- `x_err_t UserSemaphoreSetValue (sem_t sem, uint16_t val)`, 该函数用于将一个信号量的值进行重置。其中, 参数 `sem` 表示来源消息队列 ID; `val` 表示重置的信号量的值。

(3) 互斥量:

互斥量(`mutex`)可以视作一个初值为 1 的信号量, 同样有获取和释放操作。互斥量一般用于任务间的互斥。互斥量结构定义及成员含义如下所示:

代码 3-10-8 互斥量结构定义

```

01 struct Mutex{
02     struct IdNode    id;           //互斥量 ID, 用于唯一标识一个互斥量
03     uint16           val;          //互斥量的当前值
04     uint8            recursive_cnt; //持有互斥量的任务的原优先级
05     uint8            origin_prio;  //持有互斥量的任务获取互斥量的次数
06     struct TaskDescriptor *holder; //持有互斥量的任务
07     DoubleLinklistType pend_list;  //挂起任务链表
08     DoubleLinklistType link;       //系统中所有互斥量构成的链表
09 };
    
```

另外, XiUOS 中还定义一系列互斥量函数接口:

- `int32_t UserMutexCreate ()`, 该函数用于创建一个互斥量。创建成功后, 新的互斥量会被加入内核的互斥量管理链表, 并返回该互斥量的 ID, ID 默认范围 0-255, 可配置。

- `void UserMutexDelete (int32_t mutex)`, 该函数用于删除一个已创建的互斥量。其中, 参数 `mutex` 表示待删除的互斥量的 ID。
- `int32_t UserMutexObtain (int32_t mutex, int32_t wait_time)`, 该函数用于获取一个互斥量。若获取成功则返回 EOK, 若不成功(等待超时)则返回-ETIMEOUT。其中, 参数 `mutex` 表示待获取的互斥量的 ID; `wait_time` 表示等待时间上限, 单位 ms; 若为 0, 则不等待。
- `int32_t UserMutexAbandon (int32_t mutex)`, 该函数用于释放一个互斥量。其中, 参数 `mutex` 表示待释放的互斥量的 ID。

(4) 事件集:

事件集(event set)允许用户定义一个事件集合, 集合中的每个事件都可以被任务触发或等待。任务可以同时等待多个事件, 此时等待触发的条件可以配置为 AND 或者 OR: 当等待触发条件配置为 AND 时, 只有所有被等待的事件均被触发才视作等待结束, 称之为逻辑与触发; 当等待触发条件配置为 OR 时, 任意一个被等待的事件触发即视作等待结束, 称之为逻辑或触发。使用事件集可以实现多对多的任务间同步与互斥。事件集结构定义及成员含义如下所示:

代码 3-11-9 事件集结构定义

```

01 #define EVENT_AND          (1 << 0)
02 #define EVENT_OR           (1 << 1)
03 #define EVENT_AUTOCLEAN    (1 << 2)
04 #define EVENTS(x)          (1 << x)
05 struct Event{
06     struct IdNode      id;          //用于唯一标识一个事件集
07     uint32              options : 3; //高 3 位用于记录事件集属性
08     uint32              events : 29; //低 29 位用于表示至多 29 个事件
09     DoubleLinklistType pend_list;   //等待任务链表
10     DoubleLinklistType link;        //系统中所有事件集构成的链表
11 };

```

另外, XiUOS 中还定义一系列事件集函数接口:

- `eventIdType UserEventCreate (uint8_t flag)`, 该函数用于创建一个事件集。`flag` 参数用于配置事件集的属性。可配置的属性有等待触发方式(EVENT_AND 或 EVENT_OR)及等待触发后是否自动清空其他已触发的事件(EVENT_AUTOCLEAN)。创建成功后,

新的事件集会被加入内核的事件集管理链表，并返回该事件集的 ID，ID 默认范围 0-255，可配置。其中，参数 `flag` 表示事件集配置选项，须在 `EVENT_AND` 及 `EVENT_OR` 中指定其一，并可以按位或上 `EVENT_AUTOCLEAN`。

- `void UserEventDelete (EventIdType event)`，该函数用于删除一个已创建的事件集。其中，参数 `event` 表示待删除的事件集的 ID。
- `x_err_t UserEventTrigger (EventIdType event, uint32_t set)`，该函数用于触发事件集中的一组事件。其中，参数 `event` 表示事件集 ID；`set` 表示欲触发的事件，其中被置 1 的位标识被触发的事件，可以使用 `EVENTS` 宏按位或发送事件类型。
- `x_err_t UserEventProcess (EventIdType event, uint32_t set, uint8_t option, int32_t wait_time, uint32_t *Recved)`，该函数用于等待事件集中的一组事件。若等待成功则返回 `EOK`，此时若 `EVENT_AUTOCLEAN` 被打开则事件集中所有已触发事件会被清空；若等待失败（超时）则返回 `-ETIMEOUT`。其中，参数 `event` 表示事件集 ID；`set` 表示欲等待的事件，其中被置 1 的位标识被等待的事件，可以使用 `EVENTS` 宏按位或接收事件类型；`options` 表示用于记录事件集属性（等待触发条件、是否自动清空）；`wait_time` 表示等待时间上限，单位 `ms`；若为 0，则不等待；`Recved` 表示用于记录已经被处理完成的事件。

1.1.5 任务隔离简介

XiUOS 是一个支持多任务的操作系统，对任务的数量没有限制。在 XiUOS 中，每个任务都需要自己的堆栈，同时也可能会动态申请内存资源。任务在运行过程中发生内存溢出是实时操作系统（RTOS）中最常见的问题，所以限制任务的内存空间访问是保证 RTOS 稳定运行的关键。

ARM 和 RISC-V 在体系架构上都提供了内存访问的保护功能，可以通过对特定寄存器的硬编程实现对指定内存区域访问权限的设置。然而，现有的大多数物联网操作系统并没有使用体系结构提供的内存保护功能来对任务运行的地址空间进行隔离保护。XiUOS 充分考虑任务运行的安全问题，在不影响任务正常执行的情况下，对每个任务所允许访问的内存地址空间进行限制。除此之外，任务在动态申请内存、释放内存、内存共享时，XiUOS 也将提供隔离服务。

XiUOS 任务隔离的总体设计思想是将物理内存地址空间划分为信任地址空间和非信任地址空间。XiUOS 的内核任务运行在信任地址空间，可以访问所有信任地址空间和非信任地址空间；XiUOS 的用户程序运行在非信任地址空间，通过“内核服务表”的方式访问内核任务提供的功能。

ARM 和 RISC-V 在体系架构上支持机器在特权模式和非特权模式之间转换。XiUOS 的内核任务运行在特权模式下，可以访问体系结构支持的、可编程的所有硬件资源。XiUOS 的用户程序运行在非特权模式下，对硬件资源的访问权限是受限制的。为了实现用户程序受限的硬件资源访问以及内核任务提供的其他功能，XiUOS 的内核为用户程序提供一组服务接口来满足应用程序的这些需求，这一组内核服务接口称为内核服务表。应用程序访问内核服务接口的流程如下：

- 应用程序执行异常调用指令并指定相应内核服务号和参数；
- 通过软中断指令产生一个调用异常，之后 CPU 切换到特权模式并强制执行异常处理流程，异常处理流程提取内核服务号和参数，并将服务号作为索引；
- 根据索引从内核服务表中查找对应的内核服务接口；
- 在特权模式下，执行所需的内核服务，完成后切换回非特权模式继续执行。

另外，XiUOS 中通过任务描述符 TaskDescriptor 管理系统的任务，其中包含了任务优先级、任务名称、任务状态等信息。为了管理任务可访问的内存空间，TaskDescriptor 描述符在 TaskDyncSchedMember 子结构中增加任务隔离标志位 isolation_flag 和任务隔离成员 isolation、isolation_status,其定义如下：

代码 3-12-10 TaskDyncSchedMember 子结构

```

01 struct TaskDyncSchedMember {
02     ...
03     #ifdef SEPARATE_COMPILE
04         uint8      isolation_flag;
05         void        *isolation;
06         uint8      isolation_status;
07     #endif
08     ...
09 };

```

用户程序运行在非信任地址空间，默认开启隔离机制。在用户程序对应的任务被创建时，任务隔离标志位 `isolation_flag` 被置为 1，其允许访问的地址空间范围由 `isolation_table` 指针所指定。`isolation_table` 包含了任务可访问的多个内存地址空间，每个内存地址空间用一个 `isolation_region` 数据结构来描述，这个数据结构包括一段连续的地址空间范围和访问权限，其具体结构取决于体系架构提供的内存保护功能，如 ARM 提供的 MPU 和 RISC-V 提供的 PMP 功能。`isolation_region` 的定义如下：

代码 3-13-11 `isolation_region` 定义

```

01  #if defined (TASK_ISOLATION)
02  typedef struct isolation_region {
03      #ifdefined (XS_RISCV32)
04          xs_uint8  region_cfg;
05          xs_uint32 region_addr;
06      #elif defined (XS_RISCV64)
07          xs_uint8  region_cfg;
08          xs_uint64 region_addr;
09      #elif defined (XS_ARM32)
10          xs_uint32 region_cfg;
11          xs_uint32 region_addr;
12      #endif
13  }isolation_region;
14  #endif

```

XiUOS 中的任务在编译链接后形成 Linux 通用的 ELF 文件结构，其中包括 `.code`、`.data`、`.bss` 等段。在任务加载过程中，`.code`、`.data`、`.bss` 会被加载到对应的内存段中。在创建任务时，先判断任务类型，`UserTaskCreate` 用于创建任务，其中 `isolation_flag` 标志位会被置为 1。当为任务分配好栈空间后，`isolation_table` 中为 `.code`、`.data`、`.bss` 等段分别创建一个 `isolation_region`，并设置对应的地址范围和访问权限，如 `.code` 段对应的 `isolation_region` 为读和执行权限，`.data` 段对应的 `isolation_region` 为只读权限，`.bss` 段和栈对应的 `isolation_region` 为读写权限。

基于上述的任务内存结构分布及 `isolation_region`，下面介绍面向 RISC-V 架构的三种任务隔离机制：切换时的隔离，动态申请/释放内存时的隔离，RISC-V 64 架构任务隔离。

（1）任务切换时的隔离：

在 XiUOS 中，多个任务共享有限个 CPU 核，采用优先级加时间片的调度方式，当一个任务时间片耗尽或者主动让出 CPU 的使用权时，内核调度程序负责保存当前任务的上下文信息，并从等待队列中挑选下一个就绪任务，恢复其上下文信息，并允许其获取 CPU 的使用权。为了保证每个任务只能访问自己的内存空间，内核调度程序在恢复任务上下文时，会先根据 `isolation_flag` 标志判断该任务是否为用户程序，如果为用户程序，则当前 CPU 核在运行该任务时，只允许访问本任务 `isolation_table` 中定义的内存区域，对于其它区域没有访问权限。

（2）动态申请/释放内存时的隔离：

XiUOS 的任务通过 `malloc/free` 等内核服务接口来动态申请和释放内存空间，并能根据用户任务申请和释放的内存地址更新任务隔离表。当用户程序通过 `malloc` 等内核服务接口向内核申请指定大小的内存空间时，系统会在用户程序对应的任务中增加对内存空间的访问权限，同时将更新后的任务隔离表加载到内存保护单元配置寄存器中，使其生效。当用户程序通过 `free` 等内核服务接口向内核释放指定大小的内存空间时，系统会在用户程序对应的任务隔离表中清除对这段内存空间的访问权限，同时加载更新后的任务隔离表到内存保护单元寄存器中，使其生效。

此外，XiUOS 还支持共享内存的任务隔离，其基本思路同动态内存申请/释放时相同。进一步地，当用户程序因为某种原因试图访问没有权限的内存空间时，CPU 会产生一个访问错误的异常，并进入内核服务接口的异常处理流程，在这个流程中会将该任务直接杀死并回收任务资源。

（3）RISC-V 64 架构任务隔离：

大部分 RISC-V 架构的 CPU 拥有特权模式和非特权模式。这两种模式都为 CPU 核提供了物理内存保护（PMP，Physical Memory Protection）功能。通过编程 PMP，可以为指定大小的内存区域设置读、写、执行等访问权限。PMP 包括 8~16 组配置寄存器和地址寄存器，一组配置寄存器和地址寄存器称为一个 PMP entry，PMP entry 对应之前定义的 `isolation_region`，用于标识一段内存地址空间的访问权限。相关 PMP 操作的接口如下：

代码 3-14-12 PMP 操作

```
01 // isolation_table 增加一个 region
```

```

02 PMP_add_region (isolation_region *isolation_table,
03 void *region_address, xs_size_t size, int flag);
04 // isolation_table 增加清除一个 region
05 PMP_clear_region (isolation_region *isolation_table,
06 void *region_address, xs_size_t size);
07 // 将 isolation_table 加载到 PMP 中
08 PMP_load (isolation_region *isolation_table, xs_uint8 coreid);

```

对于一个正在运行用户任务，其 isolation_table 的内容大致如下：

代码 3-15-13 isolation_table

```

01 isolation_table[16] = {
02 { //code
03     //L=0, 用户任务遵守 RWX 指定的权限，内核任务拥有全部权限
04     .region_cfg = __PMP_REGION_CFG ( 0, )
05     .region_addr = __PMP_REGION_ADDR (text_start_addr,region_start_addr_size);
06 },
07 { //data
08     //L=0, 用户任务遵守 RWX 指定的权限，内核任务拥有全部权限
09     .region_cfg = __PMP_REGION_CFG ( 0, )
10     .region_addr = __PMP_REGION_ADDR (data_start_addr,region_start_addr_size);
11 },
12 { //bss 段
13     //L=0, 用户任务遵守 RWX 指定的权限，内核任务拥有全部权限
14     .region_cfg = __PMP_REGION_CFG ( 0, )
15     .region_addr = __PMP_REGION_ADDR (bss_start_addr,region_start_addr_size);
16 },
17 { // stack
18     //L=0, 用户任务遵守 RWX 指定的权限，内核任务拥有全部权限
19     .region_cfg = __PMP_REGION_CFG ( 0, )
20     .region_addr = __PMP_REGION_ADDR (stack_start_addr,region_start_addr_size);
21 },
22 ...
23 };

```

1.2 任务管理实验

1.2.1 任务创建实验

XiUOS 中提供了查看任务状态的命令 ShowTask。在控制台输入 ShowTask，可显示图 3-1-2 所示的任务状态。由图中可知，CPU 有两个核，第 0 和 1 核，当前 CPU[0]中运行的

任务是“ktaskidle0”，优先级是 0；CPU[1]中运行的任务是“letter-shell”，优先级是 20；使用的调度策略是 RR_REMAINSlice。

另外，由图 3-16-17 中的表可知，当前系统有 5 个任务，均为系统任务，分别为“letter-shell”、“sys_work”、“ktaskidle1”、“ktaskidle0”和“ZombieRecycleKTask”。其中，ID 表示给任务分配的编号，可以看出，此时 ID=3 未分配；STAT 表示每个任务对应的状态，目前有运行（RUNNING）、挂起（SUSPEND）和准备就绪（READY）这三种状态；PRI 表示任务的优先级，用于任务的执行调度；CORE 表示任务当前绑定的 CPU 核；STACK_DEPTH 表示任务的堆栈大小；USED 表示任务堆栈利用率；LEFT_TICKS 表示任务运行的剩余时间片；ERROR_STAT 表示任务运行错误状态值。

```
letter:/S ShowTask
*****
CPU[0] CURRENT RUNNING TASK[ ktaskidle0 ] PRI[ 0 ]
CPU[1] CURRENT RUNNING TASK[ letter-shell ] PRI[ 20 ]
SCHED POLICY [ RR_REMAINSlice ]
*****
STAT  ID  TASK                PRI  CORE  STACK_DEPTH  USED  LEFT_TICKS  ERROR_STAT
RUNNING  5  letter-shell            20    1      4096        29%    6           000
SUSPEND  4  sys_work                23    1      2048        34%   10           000
READY   2  ktaskidle1              0     1      1024        67%    3           000
RUNNING  1  ktaskidle0              0     0      1024        47%    9           000
SUSPEND  0  ZombieRecycleKTask      31    0      2048        31%   10           000
*****
letter:/S
```

图 3-18-19 初始任务状态

任务创建实验设计过程如图 3-1-3 所示。首先定义两个函数 task1_func 和 task2_func，然后将这两个函数分别作为参数赋值给任务 task1 和 task2；在任务 task1 中，完成任务的创建、CPU 核绑定和解绑、启动操作；在任务 task2 中，完成任务的创建、CPU 核绑定、启动、删除操作。具体实现过程如代码 3-1-14 中所示，该段代码可放到 XiUOS/Applications/main.c 中运行。

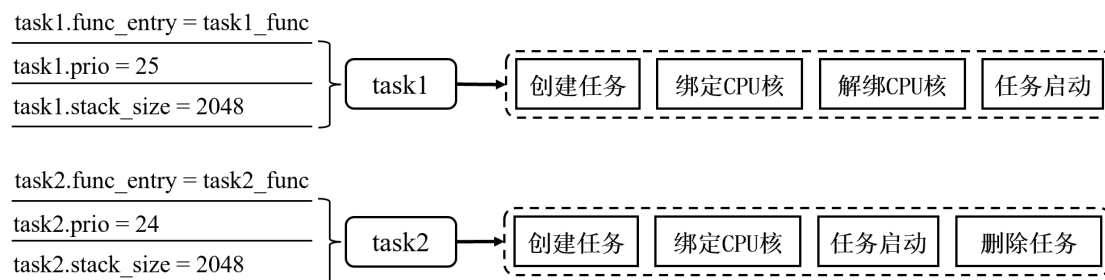


图 3-20-21 任务创建实验设计

代码 3-22-14 任务管理实验代码

```
01 void* task1_func(void* param) {
02     while(1) {
03         printf("task1 is running\n");
04         UserTaskDelay(20000);
05     }
06 }
07
08 void* task2_func(void* param) {
09     while(1) {
10         printf("task2 is running\n");
11         UserTaskDelay(20000);
12     }
13 }
14
15 int task_management(void) {
16     int ret = 0;
17     UtaskType task1;
18     task1.prio = 25;
19     task1.stack_size = 2048;
20     task1.func_entry = task1_func;
21     task1.func_param = NULL;
22     strncpy(task1.name,"task1",strlen("task1"));
23
24     int task1_pid = UserTaskCreate(task1);
25     if (task1_pid < 0)
26         return -1;
27     UserTaskCoreCombine(task1_pid,0);
28     UserTaskCoreUnCombine(task1_pid);
29     ret = UserTaskStartup(task1_pid);
30
31     UtaskType task2;
32     task2.prio = 24;
33     task2.stack_size = 2048;
34     task2.func_entry = task2_func;
35     task2.func_param = NULL;
36     strncpy(task2.name,"task2",strlen("task2"));
```

```
37
38     int task2_pid = UserTaskCreate(task2);
39     if (task2_pid < 0)
40         return -1;
41
42     UserTaskCoreCombine(task2_pid,1);
43     ret = UserTaskStartup(task2_pid);
44     UserTaskDelete(task2_pid);
45 }
```

代码行 1~6 和 8~13 分别创建了函数 `task1_func` 和 `task2_func`，这两个函数的功能相同，都是无限循环打印输出，且打印时间延时为 20000ms（20s），该延时是由 `UserTaskDelay` 函数在代码 4 和 7 中进行设置的。代码行 15~45 创建了一个任务管理函数 `task_management`，在其中创建了两个任务 `task1` 和 `task2`。以 `task1` 为例，在代码行 17~29 中，`task1.prio` 是给任务 `task1` 设置的优先级，范围是最大值可配置，即根据不同设置有不同的范围，且值越高，优先级越高；`task1.stack_size` 是设置的堆栈大小，无范围，根据任务的实际需求自由设置；`task1.func_entry` 是设置功能实体，这里把函数 `task1_func` 关联给任务 `task1`；`task1.func_param` 是设置任务的功能函数参数；代码行 24 通过调用 `UserTaskCreate` 函数创建一个任务，返回 `task1_pid`，然后在代码行 25~26 中检查生成的 `task1_pid` 是否符合要求，值得注意的是，任务编号是系统自动生成的；代码行 27 中通过 `UserTaskCoreCombine` 函数将该任务绑定到 CPU 的第 0 核上；紧接着代码行 28 使用 `UserTaskCoreUnCombine` 函数对 CPU 核进行解绑定，该行主要是为了验证 CPU 核的解绑功能；代码行 29 通过 `UserTaskStartup` 函数启动任务，让其运行。代码行 31~44 的 `task2` 创建和配置过程基本与 `task2` 的过程类似。不同之处在于代码行 42，给 `task2` 绑定的 CPU 核是 1；在代码行 43 启动任务 `task2` 之后，随即在代码行 44 通过 `UserTaskDelete` 函数对任务 `task2` 进行删除，此处是为了验证任务删除功能。最后，在 `main` 函数中运行 `task_management` 函数。

系统编译、烧录、运行之后的 XiUOS 控制台界面如图 3-1-4 所示。从图中可以看出，控制台只打印了任务 `task1` 的“`task1 is running`”，而任务 `task2` 的并没有打印输出，这是因为在代码行 42 中对 `task2` 进行了删除。

执行 XiUOS 的 `ShowTask` 命令行结果如图 3-1-5 所示。相比于图 3-23-24，图 3-1-5 中有 6 个任务，其中新增加的任务是 `task1`，而 `task2` 由于已经被删除，不在任务列表当中。由

于打印输出的延时是 20s，所以 task1 大部分时间处于挂起状态，所以此时的 STAT 是 SUSPEND；图 3-25-26 中的 ID=3 是未分配的，此时在图 3-1-5 中已经分配给了 task1；虽然在代码行 27 中给 task1 绑定了 CPU 的第 0 核，但随即在代码行 28 进行了解绑，所以给 task1 采用随机调度的方式，此时分配的是 CPU 的第 1 核。

```

letter-shell
Build:      Jun 30 2022 14:44:46
Version:    3.0.5
Copyright:  (c) 2020 Letter

letter:/$ initialize letter-shell system success
task 1 is running
task 1 is running
task 1 is running

```

图 3-27-28 运行任务创建实验的 XiUOS 界面

```

letter:/$ ShowTask
*****
CPU[0] CURRENT RUNNING TASK[ ktaskidle0 ] PRI[ 0 ]
CPU[1] CURRENT RUNNING TASK[ letter-shell ] PRI[ 20 ]
SCHED POLICY [ RR_REMAINSLICE ]
*****
STAT  ID  TASK                                PRI  CORE  STACK_DEPTH  USED  LEFT_TICKS  ERROR_STAT
SUSPEND 3  task_1                                25   1      2048         54%   10         000
RUNNING 5  letter-shell                          20   1      4096         29%   7          000
SUSPEND 4  sys_work                              23   1      2048         34%   10         000
READY  2  ktaskidle1                             0   1      1024         67%   2          000
RUNNING 1  ktaskidle0                             0   0      1024         47%   8          000
SUSPEND 0  ZombieRecycleKTask                     31   0      2048         31%   10         000
*****

```

图 3-29-30 任务创建实验时的 ShowTask 结果

1.2.2 任务调度实验

XiUOS 的任务调度策略是通过设置完成的。首先，在控制台的 XiZi 目录下输入 make menuconfig，会显示如图 3-31-32 所示的 XiZi Project Configuration 界面。接着，分别选择 Kernel feature→Task feature→Sheduler Policy Set，即从图 3-33-34 到 3-35-36 的选择过程，会显示图 3-37-38 的调度策略选择界面。其中，有三种策略供选择，分别是 RR、RR_REMAINSLICE 和 FIFO。

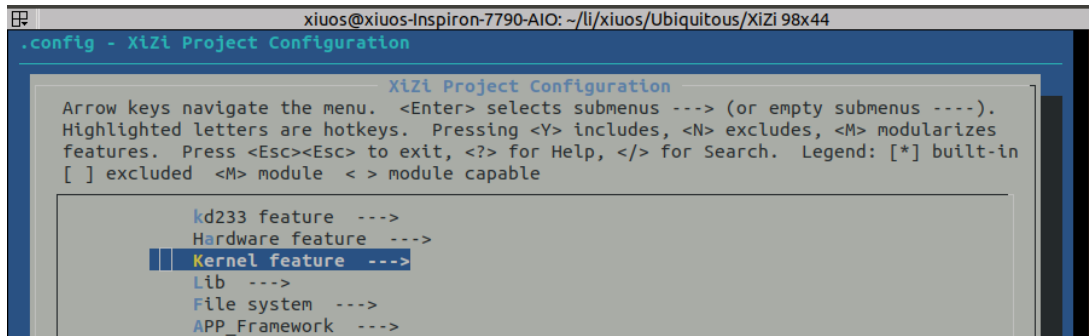


图 3-39-40 XiZi Project Configuration

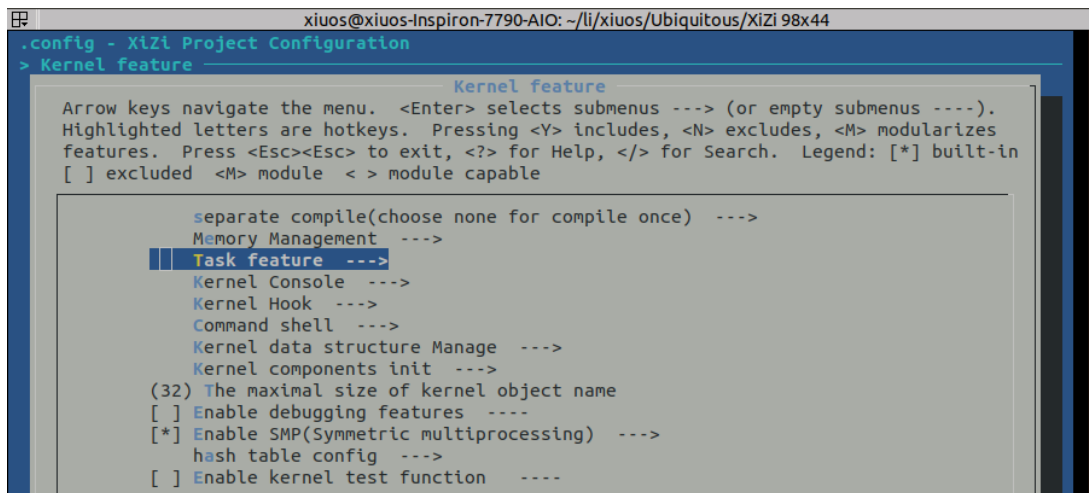


图 3-41-42 Kernel feature

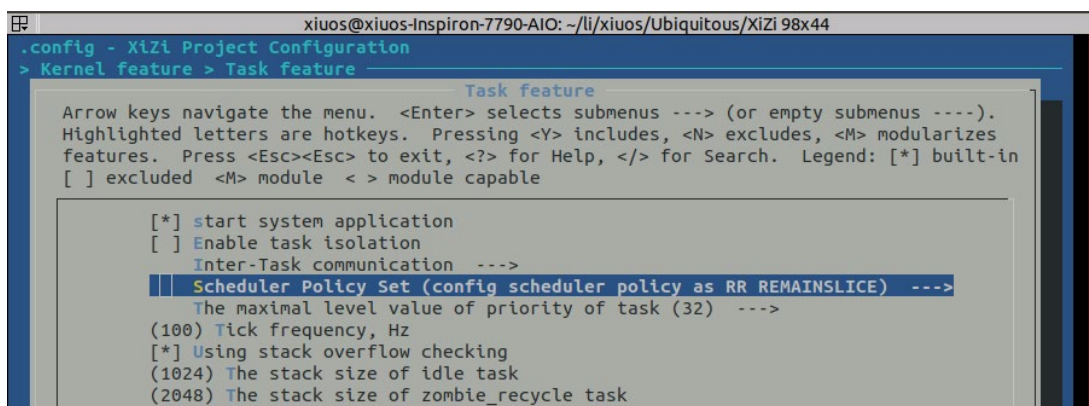


图 3-43-44 Task feature

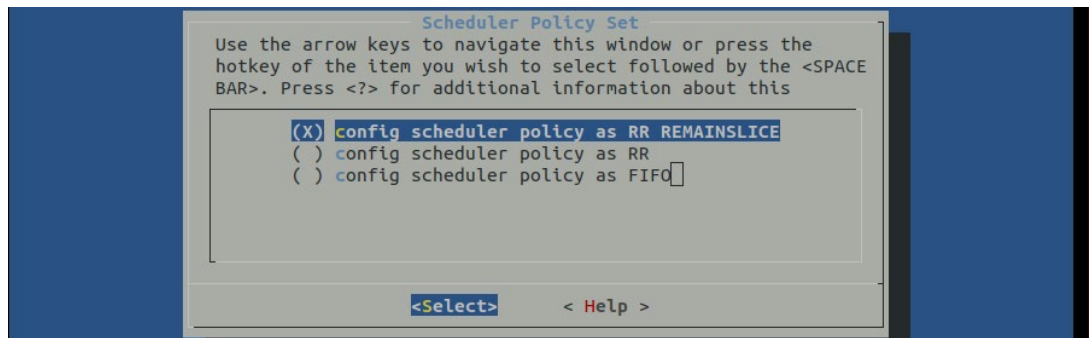


图 3-45-46 Sheduler Policy Set

(1) RR 调度实验:

我们在 XiUOS 中执行两次 ShowTask 命令，查看时间片轮转 RR 的调度结果，如图 3-47-48 和 3-1-11 中所示。从前后两次 ShowTask 结果看出，任务 letter-shell 的 LEFT-TICKS 从 5 减少到了 3。同样，任务 ktaskidle0 和 ktaskidle1 的 LEFT-TICKS 也相应的减少。需要说明的是，有时前后两次 ShowTask 结果中，有些任务的 LEFT-TICKS 是由少变多，这是因为该任务的当前一轮已经调度结束，在等待之后又重新被调度了。以上结果及结论都显示了 RR 调度策略的方式。值得注意的是，RR_REMAINS LICE 的调度过程与 RR 类似，这里不再叙述。

```
letter:/$ ShowTask
*****
CPU[0] CURRENT RUNNING TASK[ ktaskidle0 ] PRI[ 0 ]
CPU[1] CURRENT RUNNING TASK[ letter-shell ] PRI[ 20 ]
SCHED POLICY [ RR ]
*****
STAT  ID  TASK                PRI  CORE  STACK DEPTH USED  LEFT TICKS  ERROR STAT
-----
RUNNING 5  letter-shell             20    1    4096   29%    5    000
SUSPEND 4  sys_work                 23    1    2048   34%   10    000
READY  2  ktaskidle1               0     1    1024   65%    5    000
RUNNING 1  ktaskidle0               0     0    1024   47%    9    000
SUSPEND 0  ZombieRecycleKTask       31    1    2048   30%   10    000
*****
```

图 3-49-50 RR 任务调度的第 1 次 ShowTask

```
letter:/$ ShowTask
*****
CPU[0] CURRENT RUNNING TASK[ ktaskidle0 ] PRI[ 0 ]
CPU[1] CURRENT RUNNING TASK[ letter-shell ] PRI[ 20 ]
SCHED POLICY [ RR ]
*****
STAT  ID  TASK                PRI  CORE  STACK DEPTH USED  LEFT TICKS  ERROR STAT
-----
RUNNING 5  letter-shell             20    1    4096   29%    3    000
SUSPEND 4  sys_work                 23    1    2048   34%   10    000
READY  2  ktaskidle1               0     1    1024   65%    4    000
RUNNING 1  ktaskidle0               0     0    1024   47%    6    000
SUSPEND 0  ZombieRecycleKTask       31    1    2048   30%   10    000
*****
```

图 3-51-52 RR 任务调度的第 2 次 ShowTask

(2) FIFO 调度实验:

FIFO 任务调度实验的设计思路是：首先将 XiUOS 的任务调度策略设置为 FIFO；然后创建两个任务 task1 和 task2，并都绑定相同的 CPU 核。那么在这种调度策略下，一个任务执行完成之后，才能轮到另一个任务执行。

为了验证 FIFO 的任务调度方式，我们在 XiUOS/Applications/main.c 中，增加代码 3-1-15。其中，代码行 1~8 和 10~17 中分别定义了两个函数 task1_func 和 task2_func，这两个函数执行一定次数的求和操作。为了能够有足够的时间显示结果，这里把循环变量 idx 设置成一个很大的值。然后，在代码行 19~47 中，即函数 task_scheduling 中，分别创建两个任务 task1 和 task2。为了验证 FIFO 调度策略，这里将 task1 和 task2 的 CPU 核都绑定到第 0 核，如代码行 31 和 45 所示。其它代码行的作用，这里不再赘述。

代码 3-53-15 FIFO 任务调度实验代码

```

01 void* task1_func(void* param) {
02     int idx=1000000000;
03     int sum=0;
04     while(idx>=0) {
05         sum=sum+1;
06         idx=idx-1;
07     }
08 }
09
10 void* task2_func(void* param) {
11     int idx=1000000000;
12     int sum=0;
13     while(idx>=0) {
14         sum=sum+1;
15         idx=idx-1;
16     }
17 }
18
19 int task_scheduling(void) {
20     int ret = 0;
21     UtaskType task1;
22     task1.prio = 25;

```

```
23     task1.stack_size = 2048;
24     task1.func_entry = task1_func;
25     task1.func_param = NULL;
26     strncpy(task1.name,"task1",strlen("task1"));
27
28     int task1_pid = UserTaskCreate(task1);
29     if (task1_pid < 0)
30         return -1;
31     UserTaskCoreCombine(task1_pid,0);
32     ret = UserTaskStartup(task1_pid);
33
34     UtaskType task2;
35     task2.prio = 24;
36     task2.stack_size = 2048;
37     task2.func_entry = task2_func;
38     task2.func_param = NULL;
39     strncpy(task2.name,"task2",strlen("task2"));
40
41     int task2_pid = UserTaskCreate(task2);
42     if (task2_pid < 0)
43         return -1;
44
45     UserTaskCoreCombine(task2_pid,0);
46     ret = UserTaskStartup(task2_pid);
47 }
```

系统编译、烧录、运行之后，使用 ShowTask 命令显示结果如图 3-1-12 所示。从第 1 次 ShowTask 结果可以看出，即图 3-1-12 所示，此时 task1 处于 RUNNING 状态，而 task2 处于 READY 状态。这是因为两个任务都是绑定到 CPU 的第 0 核，由于任务 task1 是先进，所以 task2 要等到 task1 执行结束之后才会被调度。图 3-1-13 所示的是第 2 次 ShowTask 结果，此时任务 task1 已经执行结束，所以任务列表中已经没有 task1；而任务 task2 开始执行，其状态由原来的 READY 变为 RUNNING。


```

letter:/ $ ShowTask
*****
CPU[0] CURRENT RUNNING TASK[ task_1 ] PRI[ 25 ]
CPU[1] CURRENT RUNNING TASK[ letter-shell ] PRI[ 20 ]
SCHED POLICY [ FIFO ]
*****
STAT  ID  TASK                PRI  CORE  STACK  DEPTH  USED  ERROR  STAT
-----
READY  7  task_2                  24   2    2048    12%   000
RUNNING 3  task_1                  25   0    2048    24%   000
RUNNING 5  letter-shell            20   1    4096    29%   000
SUSPEND 4  sys_work                23   1    2048    34%   000
READY  2  ktaskidle1              0    1    1024    65%   000
READY  1  ktaskidle0              0    2    1024    25%   000
SUSPEND 0  ZombieRecycleKTask      31   1    2048    24%   000
*****

```

图 3-54-55 FIFO 调度实验的第 1 次 ShowTask

```

letter:/ $ ShowTask
*****
CPU[0] CURRENT RUNNING TASK[ task_2 ] PRI[ 24 ]
CPU[1] CURRENT RUNNING TASK[ letter-shell ] PRI[ 20 ]
SCHED POLICY [ FIFO ]
*****
STAT  ID  TASK                PRI  CORE  STACK  DEPTH  USED  ERROR  STAT
-----
RUNNING 7  task_2                  24   0    2048    24%   000
RUNNING 5  letter-shell            20   1    4096    29%   000
SUSPEND 4  sys_work                23   1    2048    34%   000
READY  2  ktaskidle1              0    1    1024    65%   000
READY  1  ktaskidle0              0    2    1024    25%   000
SUSPEND 0  ZombieRecycleKTask      31   0    2048    24%   000
*****

```

图 3-56-57 FIFO 调度实验的第 2 次 ShowTask

(3) 基于 ARM 开发板的任务切换性能测试：

为了测试系统的任务切换时间，考虑使用 GPIO 引脚进行测试，将 GPIO 在任务切换开始和结束时分别置为高电平和低电平。本实验选择 PD1 作为外部的输出引脚。为了实现对 GPIO 引脚电平的控制，需要编写相关程序对 GPIO 引脚配置；同时需要修改内核代码中的任务切换入口和出口，分别在入口和出口中拉高和拉低 IO 电平。

ARM cortex-m4 内核的 XIUOS 系统的任务切换是基于 pendSV 异常来实现的，根据这一特点需要区分场景进行测量：

- 只测量 SwitchKtaskContext 时间开销。在 SwitchKtaskContext 函数入口处将 C13 管脚置为高电平，在出口位置，将 C13 管脚置为低电平。得出的管脚电平时间即为不计算 pendSV 异常的的任务切换时间。
- 测量带 pendSV 异常的的任务切换的时间开销。在 SwitchKtaskContext 函数入口处将 C13 管脚置为高电平，在 pendSV 异常处理过程，保存现场之后，切换到目标任务之前将 C13 管脚置为低电平。得出的管脚电平时间即为带 pendSV 异常的的任务切换时间。

对于以上两种测量场景，在代码实现上的不同之处只有拉低 PD1 的时间节点有所不同。

代码 3-1-16 主要是对应于实验中的硬件配置过程。代码行 3~4 和 6~7 分别声明了 GPIO 引脚参数和总线配置相关的结构体变量。代码行 9~12 对配置参数进行了初始化；14~22 将 PD1 配置为输出模式并设置输出低电平。由于任务切换的过程时间较短，因而为了降低使用示波器捕获的难度，代码行 24~26 通过循环调用 DelayKTask 函数来使这一过程周期性地

代码 3-58-16 任务切换性能测试配置代码

```

01 void RealtimeTaskSwitchTest()
02 {
03     struct PinParam testpin_1;
04     struct PinStat testpin_1_stat;
05
06     struct BusConfigureInfo configure_info_1;
07     struct BusBlockWriteParam write_param_1;
08
09     KPrintf("entry to RealtimeTaskSwitchTest()\n");
10     configure_info_1.configure_cmd = OPE_CFG;
11     configure_info_1.private_data = (void *)&testpin_1;
12     write_param_1.buffer = (void *)&testpin_1_stat;
13
14     //#define GPIO_D1 143
15     testpin_1.cmd = GPIO_CONFIG_MODE;
16     testpin_1.pin = GPIO_D1;
17     testpin_1.mode = GPIO_CFG_OUTPUT;
18     BusDrvConfigure(pin->owner_driver, &configure_info_1);
19
20     testpin_1_stat.pin = GPIO_D1;
21     testpin_1_stat.val = GPIO_HIGH;
22     BusDevWriteData(pin->owner_haldev, &write_param_1);
23
24     while (RET_TRUE) {
25         DelayKTask(1);
26     }
27 }

```

代码 3-4 中行 3~6 是需要增加的部分内核代码，对应为任务切换操作的入口。其主要功能为将 PD1 引脚拉高。为了减少时间开销，故对内核修改的代码中采用汇编语言进行编写。通过查询 HC32F4A0 芯片的用户手册，可以查找到 GPIO Port D 对应的通用置位寄存器(P

OSRx)地址为 0x40053838。通过 LDR、MOV、STR 汇编指令分别将 POSRD 寄存器的第 2 比特置位即可使 PD1 引脚输出高电平。

代码 3-59-17 任务入口 GPIO 口电平控制

```

01 void __attribute__((naked)) HwInterruptcontextSwitch(x_ubase from, x_ubase to,
                                struct TaskDescriptor *to_task, void *context)
02 {
03     /* 将 GPIO D1 置为高电平 */
04     asm volatile("LDR r2, =0x40053838"); // 测试代码
05     asm volatile("MOV r3, #0x0002");    // 测试代码
06     asm volatile("STR r3, [r2]");        // 测试代码
07
08     asm volatile ("PUSH {R4}");
09     asm volatile ("PUSH {R5}");
10
11     asm volatile ("LDR r4, =KtaskSwitchInterruptFlag");
12     asm volatile ("LDR r5, [r4]");
13     asm volatile ("CMP r5, #1");
14
15     asm volatile ("POP {R5}");
16     asm volatile ("POP {R4}");
17
18     asm volatile ("BEQ Arm32SwitchReswitch");
19
20     asm volatile ("PUSH {R4}");
21     asm volatile ("PUSH {R5}");
22     asm volatile ("LDR r4, =KtaskSwitchInterruptFlag");
23
24     asm volatile ("MOV r5, #1");
25     asm volatile ("STR r5, [r4]");
26     asm volatile ("LDR r4, =InterruptFromKtask");
27     asm volatile ("STR r0, [r4]");
28
29     asm volatile ("POP {R5}");
30     asm volatile ("POP {R4}");
31
32     asm volatile ("B Arm32SwitchReswitch");
33 }

```

对于仅测量 SwitchKtaskContext 时间开销，需要修改的内核代码见代码 3-1-18。代码行 12~21 是增加在任务切换出口的代码部分。代码行 15~18 主要是将对应的通用置位寄存器 P ORRD（地址为 0x4005383A）的第 2 比特置位，即可使 PD1 引脚输出低电平。

代码 3-60-18 任务出口 GPIO 口电平控制

```

01 void __attribute__((naked)) Arm32SwitchReswitch()
02 {
03     asm volatile ("PUSH {R4}");
04
05     asm volatile ("LDR r4, =InterruptToKtask");
06     asm volatile ("STR r1, [r4]");
07     asm volatile ("LDR r4, =InterruptToKtaskDescriptor");
08     asm volatile ("STR r2, [r4]");
09     asm volatile ("LDR r0, =NVIC_INT_CTRL");
10     asm volatile ("LDR r1, =NVIC_PENDSVSET");
11     asm volatile ("STR r1, [r0]");
12
13     asm volatile ("POP {R4}");
14
15     /* 将 GPIO D1 置为低电平 */
16     asm volatile("LDR r2, =0x4005383A"); // 测试代码
17     asm volatile("MOV r3, #0x0002");    // 测试代码
18     asm volatile("STR r3, [r2]");        // 测试代码
19
20     asm volatile ("BX LR");
21 }

```

由于在每次 DelayKTask 函数结束后系统都将进行两次任务调度，分别从 letter-shell 线程切换到 idle 线程和从 idle 线程切换回 letter-shell 线程，两次任务切换分别位于 DelayKTask 延迟时间的开始和结束。这将使 PD01 产生周期性的脉冲信号。在代码修改完成后，编译并烧录程序，完成硬件相关引脚与示波器的连接。可以观察到周期性的脉冲波形。由于 SwitchKtaskContext 函数的代码相对固定，在不考虑流水线的情况下，在一个周期内任务的来回切换时间是大致相等的。如图 3-1-14 所示是一个周期内得到的两个脉冲波形。

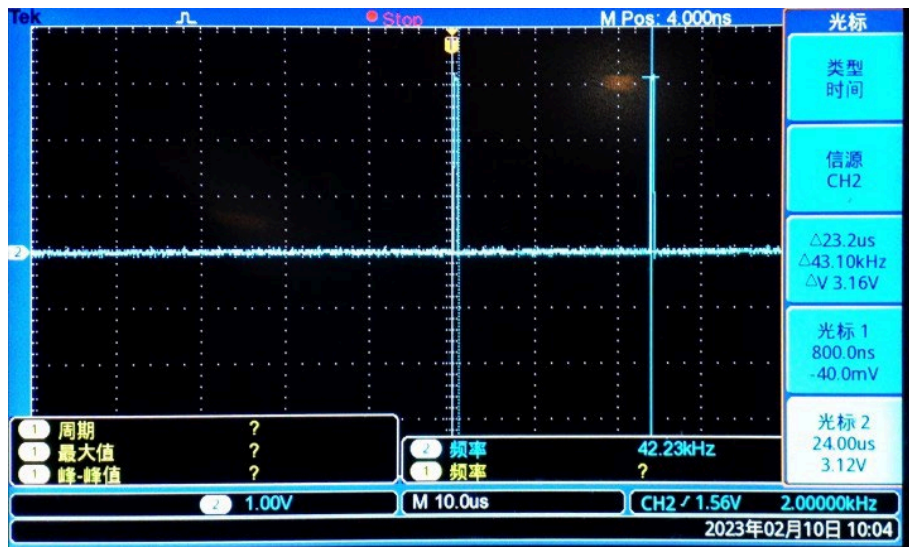


图 3-61-62 周期内的两个脉冲波形

通过进一步调整示波器的水平轴尺度，可以得到脉冲波的时间宽度为 440ns，这说明了 SwitchKtaskContext 函数的一次调用的时间开销为 440ns 左右。

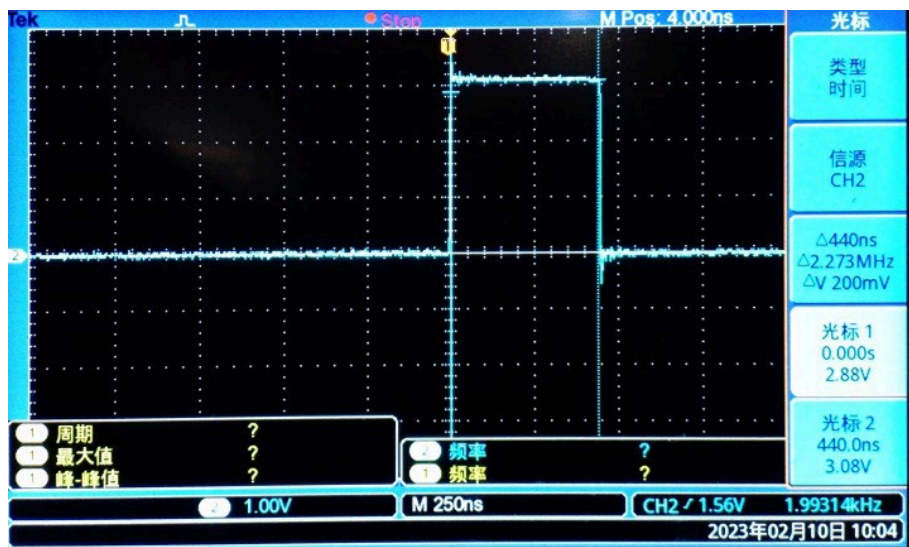


图 3-63-64 脉冲高电平时间的测量

同理地，对于带 pendSV 异常的任务切换的时间开销，可以在进行代码 3-1-16 与代码 3-1-17 处的修改的基础上，进行如代码 3-1-19 所示的代码修改。switch_to_task 代码段的调用位于 pendSV 异常回调函数的出口处。代码行 2~5 的功能为将 PD1 引脚输出设置为低电平。

代码 3-65-19 带 PendSV 任务出口 GPIO 口电平控制

```

01 switch_to_task:
02     /* 将 GPIO D1 置为低电平 */
03     LDR    r2, =0x4005383A      // 测试代码
04     MOV    r3, #0x0002         // 测试代码
05     STR    r3, [r2]            // 测试代码

```

由于 pendSV 异常的响应时间，而由于 DelayKTask 函数的延时结束恰好对应于节拍机制的时钟周期中断触发，因而在 DelayKTask 函数的延时结束后的第一次切换在 pendSV 异常响应延迟更大。图 3-1-16 是通过示波器观察到了两个脉冲波形。

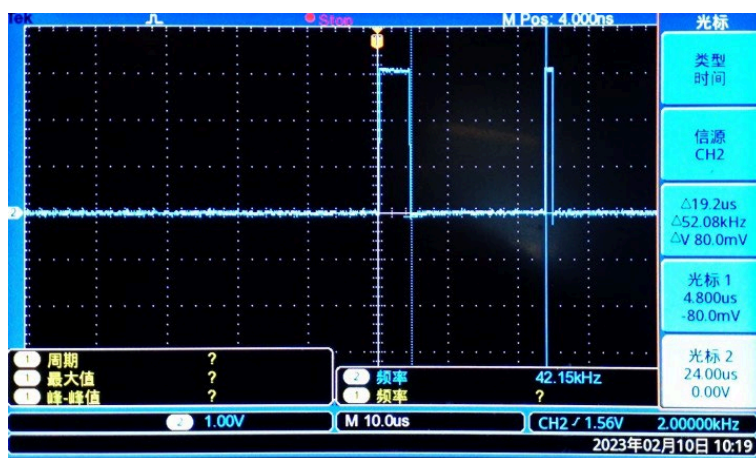


图 3-66-67 带 PendSV 的两个脉冲波形

其中图 3-1-17 所示的为调整水平轴尺度后，第一次任务切换所对应的脉冲波形，可以测得其时间宽度为 4.40us。



图 3-68-69 窄脉冲高电平时间

图 3-1-18 所示为在调整水平轴尺度后第二个脉冲波形，可以测得其时间宽度为 1.08us。



图 3-70-71 宽脉冲高电平时间

通过综合以上实验结果，可以得出以下结论：

- 任务切换函数的一次调用的时间开销为 440ns 左右
- 带 pendSV 异常的的任务切换的时间开销范围为几个微秒的量级。

（4）基于 RISC-V 开发板的任务切换性能测试：

为了测试系统的任务切换时间，考虑使用 GPIO 引脚进行测试，将 GPIO 在任务切换开始和结束时分别置为高电平和低电平。本实验选择 IO34 作为外部的输出引脚，并在板级支持包中的 drv_io_config.c 文件将 IO34 映射为 GPIOHS5。为了实现对 IO 引脚电平的控制，需要编写相关程序对 IO 引脚配置；同时需要修改内核代码中的任务切换入口和出口，分别在入口和出口中拉高和拉低 IO 电平。

代码 3-1-20 中行 4~12 是需要增加的部分内核代码，对应为任务切换操作的入口。为了减少时间开销，故对内核修改的代码中采用汇编语言进行编写。K210 控制 GPIO 引脚的高低电平可通过修改寄存器 0x3800100C 实现。代码行 4~7 主要是将 0x3800100C 地址值传入 a4 寄存器中；代码行 8~12 将 0x3800100C 中的数值与 0x00000005 进行或运算，并传回 0x3800100C 地址中。

代码 3-72-20 任务切换入口 IO 电平控制

```
01 void __attribute__((naked)) SwitchKtaskContext(x_ubase from,
x_ubase to, struct TaskDescriptor *to_task)
02 {
```

```

03      /* 将 IO34 (GPIOHS5) 置为高电平 */
04      asm volatile ("lui  a5, 0x38001"); // 测试代码
05      asm volatile ("addi a5, a5, 12"); // 测试代码
06      asm volatile ("lw   a5, 0(a5)"); // 测试代码
07      asm volatile ("sext.w a4, a5"); // 测试代码
08      asm volatile ("lui  a5, 0x38001"); // 测试代码
09      asm volatile ("addi a5, a5, 12"); // 测试代码
10      asm volatile ("ori  a4, a4, 5"); // 测试代码
11      asm volatile ("sext.w a4, a4"); // 测试代码
12      asm volatile ("sw   a4, 0(a5)"); // 测试代码
13
14      asm volatile ("addi sp, sp, -32 * " RegLengthS);
15      asm volatile (StoreDS " sp, (a0)");
16      asm volatile (StoreDS " x1, 0 * " RegLengthS "(sp)");
17      asm volatile (StoreDS " x1, 1 * " RegLengthS "(sp)");
18      asm volatile ("csrr a0, mstatus");
19      #ifndef TASK_ISOLATION
20          asm volatile ("andi a0, a0, 8");
21          asm volatile ("beqz a0, SaveMpie");
22          asm volatile ("li   a0, 0x80");
23      #endif
24      asm volatile ("j SaveMpie");
25  }

```

代码 3-1-21 中行 12~21 是增加在任务切换出口的代码部分。代码行 13~16 主要是将 0x3800100C 地址值传入 a4 寄存器中；代码行 17~21 将 0x3800100C 中的数值与 0x00000005 的反码进行或运算，并传回 0x3800100C 地址中。

代码 3-73-21 任务切换出口 IO 电平控制

```

01  void __attribute__((naked)) SaveMpie()
02  {
03      //...由于篇幅限制, 这里忽略了部分代码
04      asm volatile (StoreDS " t3, 28 * " RegLengthS "(sp)");
05      asm volatile (StoreDS " t4, 29 * " RegLengthS "(sp)");
06      asm volatile (StoreDS " t5, 30 * " RegLengthS "(sp)");
07      asm volatile (StoreDS " t6, 31 * " RegLengthS "(sp)");
08      asm volatile (LoadDS " sp, (a1)");
09      asm volatile ("mv   a0, a2");
10      asm volatile ("jal  RestoreCpusLockStatus");
11

```

```

12    /* 将 IO34 (GPIOHS5) 置为低电平 */
13    asm volatile ("lui  a5, 0x38001"); // 测试代码
14    asm volatile ("addi a5, a5, 12"); // 测试代码
15    asm volatile ("lw   a5, 0(a5)"); // 测试代码
16    asm volatile ("sext.w a4, a5"); // 测试代码
17    asm volatile ("lui  a5, 0x38001"); // 测试代码
18    asm volatile ("addi a5, a5, 12"); // 测试代码
19    asm volatile ("andi a4, a4, -6"); // 测试代码
20    asm volatile ("sext.w a4, a4"); // 测试代码
21    asm volatile ("sw   a4, 0(a5)"); // 测试代码
22
23    asm volatile ("j SwitchKTaskContextExit");
24 }

```

代码 3-1-22 主要是对应于实验中的硬件配置过程。代码行 3~4 和 6~7 分别声明了 GPIO 引脚参数和总线配置相关的结构体变量。代码行 9~11 对配置参数进行了初始化；13~20 将 GPIO34 配置为输出模式并设置输出低电平。由于任务切换的过程时间较短，因而为了降低使用示波器捕获的难度，代码行 22~24 通过循环调用 DelayKTask 函数来使这一过程周期性地进行的。

代码 3-74-22 任务切换测试配置代码

```

01 void RealtimeTaskSwitchTest()
02 {
03     struct PinParam testpin_1;
04     struct PinStat testpin_1_stat;
05
06     struct BusConfigureInfo configure_info_1;
07     struct BusBlockWriteParam write_param_1;
08
09     configure_info_1.configure_cmd = OPE_CFG;
10     configure_info_1.private_data = (void *)&testpin_1;
11     write_param_1.buffer = (void *)&testpin_1_stat;
12
13     testpin_1.cmd = GPIO_CONFIG_MODE;
14     testpin_1.pin = GPIO_34;
15     testpin_1.mode = GPIO_CFG_OUTPUT;
16     BusDrvConfigure(pin->owner_driver, &configure_info_1);
17
18     testpin_1_stat.pin = GPIO_34;

```

```

19     testpin_1_stat.val = GPIO_LOW;
20     BusDevWriteData(pin->owner_haldev, &write_param_1);
21
22     while (RET_TRUE) {
23         DelayKTask(1);
24     }
25 }

```

由于在每次 DelayKTask 函数结束后系统都将进行两次任务调度，分别从 letter-shell 线程切换到 idle 线程和从 idle 线程切换回 letter-shell 线程，两次任务切换分别位于 DelayKTask 延迟时间的开始和结束。这将使 IO34 产生周期性的脉冲信号。在完成程序的编译和烧录，并将开发板的相关引脚与示波器对应连接完成后，对示波器的水平轴、垂直轴和触发电平等进行合适的设置（具体参数可参考截图），可以得到如图 3-1-19 所示的结果。

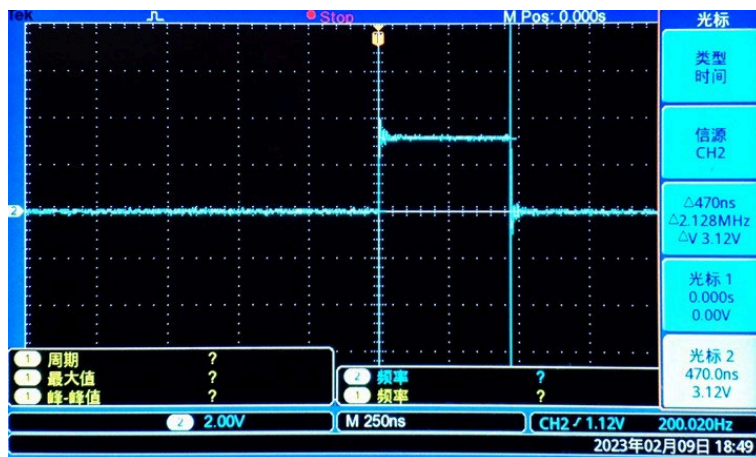


图 3-75-76 脉冲高电平时间测量

从示波器的测试结果上来看，基于 XIUOS 的 K210 开发板的任务调度的时间开销为 470ns。

1.2.3 任务通信实验

（1）消息队列实验：

消息队列实验设计过程如图 3-1-20 所示。首先创建了两个任务 task1 和 task2；任务 task1 创建一个消息队列，然后不停地向该消息队列发送消息，而任务 task2 不断地从消息队列接收消息。该过程循环若干次之后，由任务 task1 删除消息队列。整个实验过程涉及消息队列的创建，消息发送、接收和删除操作，具体实现过程如代码 3-1-23 中所示。

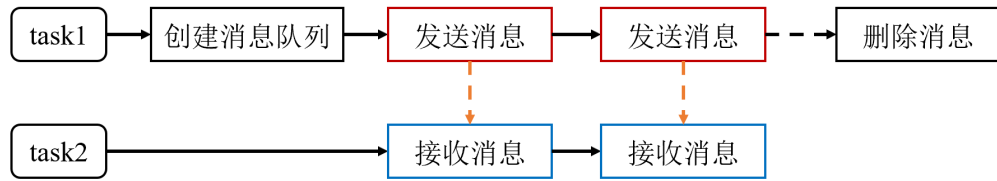


图 3-77-78 消息队列实验设计

在代码 3-1-23 中。代码行 1 定义了一个 `task_msgQueue` 函数，改函数的内容与代码 3-1-1 5 中的 `task_scheduling` 函数内容相同，主要作用是创建两个任务，这里不再详细列出；代码行 2~4 是 3 个宏定义，分别定义了消息等待时间、发送消息的大小，以及发送的消息数量；代码行 5 是设置一个消息 ID。

代码行 7~24 和 26~40 分别实现了两个函数 `task1_func` 和 `task2_func`。在函数 `task1_func` 中，主要创建了一个消息队列，返回一个 `msg_id`（代码行 10）；消息 ID 是系统自动分配的，是一个大于等于 0 的值，创建消息队列有可能会失败，所以需要判断一下是否创建成功（代码行 12~15）；然后，函数 `task1_func` 会每隔 1s 秒，周期地向消息队列发送消息，发送的内容即为 `msgBuf` 的内容（代码行 16~20）；循环发送 5 次消息后，对该消息队列进删除，但是此时的 `msg_id` 还是保留的，然后并将消息 ID 的值置为 -1，即告知其他任务，此消息队列已经不存在（代码行 22~23）。函数 `task2_func` 中首先声明了一个字符串数组 `buf`，用于接收 `task2_func` 发送过来的消息（代码行 27~28）；然后，在代码行 30~39 中，一直循环等待接收消息队列的 ID 为 `msg_id` 的消息，如代码行 32 所示，其中的 `WAIT_TIME` 设置为 -1，表示永久等待；当函数 `task1_func` 中删除消息队列，然后将 `msg_id` 置为 -1 之后，函数 `task2_func` 中的 `while` 循环结束（代码行 37~38）。需要提示的是，消息队列函数较多，这里不一一验证功能，可根据实际需要，自行应用。

代码 3-79-23 消息队列实验代码

```

01 int task_msgQueue(void);
02 #define WAIT_TIME -1
03 #define MSG_SIZE 20
04 #define MSG_NUMS 5
05 static int msg_id = -1;
06
07 void* task1_func(void* param) {

```

```

08     int count = 0;
09     char msgBuf [20] = "Hello World";
10     msg_id = UserMsgQueueCreate(MSG_SIZE, MSG_NUMS);
11     printf("msg id = %d\n", msg_id);
12     if(msg_id < 0){
13         printf("Failed to create msg\n");
14         return NULL;
15     }
16     while (count < 5) {
17         UserMsgQueueSend(msg_id, msgBuf,20);
18         UserTaskDelay(10000);
19         count++;
20     }
21
22     UserMsgQueueDelete(msg_id);
23     msg_id = -1;
24 }
25
26 void* task2_func(void* param) {
27     char buf [20];
28     memset(buf, 0, sizeof(buf));
29     printf("task2 is waiting msg\n");
30     while(1) {
31         if(msg_id >= 0) {
32             UserMsgQueueRecv(msg_id, buf, sizeof(buf), WAIT_TIME);
33             if(buf) {
34                 printf("task2 receives %s \n",buf);
35                 memset(buf, 0, sizeof(buf));
36             }
37         } else
38             break;
39     }
40 }

```

系统编译、烧录、运行之后的界面如图 3-1-21 所示。首先，可看到分配给消息队列的 msg_id 为 0；然后，task1_func 函数一共发送了 5 个消息到队列中，全部被 task2_func 接收并打印输出。

XiUOS 提供了消息队列的查看命令 ShowMsgQueue。图 3-1-22 和 3-1-23 中显示了两次 ShowMsgQueue 的结果，第 1 次是未进行消息队列实验的查询结果（图 3-1-22），第 2 次是

正在进行消息队列实验时的查询结果（图 3-1-23）。图 3-1-22 中此时没有消息队列消息，所以整个表都为空；相反，图 3-1-23 中有一条记录，记录当前消息队列的 ID、EACH_LEN、MAX_MSGS 等信息。其中 RECV_SUSPEND_TASK 为 task2，因为 task2 是永久等待消息，所以在未接受到消息时，其处于挂起状态。

```

Letter-shell
Build:      Jul  6 2022 10:34:09
Version:    3.0.5
Copyright:  (c) 2020 Letter

letter:/$ initialize letter-shell system success
msg id = 0
task2 is waiting msg
task 2 receives Hello World
task 2 receives Hello World
task 2 receives Hello World
task 2 receives Hello World
task 2 receives Hello World

```

图 3-80-81 消息队列实验结果

```

letter:/$ ShowMsgQueue
*****
MSG_ID NUM_MSGS EACH_LEN MAX_MSGS SEND_SUSPEND_TASK RECV_SUSPEND_TASK
*****

```

图 3-82-83 未进行消息队列实验的 ShowMsgQueue

```

letter:/$ ShowMsgQueue
*****
MSG_ID NUM_MSGS EACH_LEN MAX_MSGS SEND_SUSPEND_TASK RECV_SUSPEND_TASK
0      0      24      5      task_2 |
*****

```

图 3-84-85 消息队列实验的 ShowMsgQueue 结果

（2）信号量实验：

信号量实验设计过程如图 3-1-24 所示，首先创建两个任务 task1 和 task2；任务 task1 负责创建一个信号量，然后释放信号量，然后该信号量由任务 task2 获得；在一定条件下，任务 task1 会对信号量的值进行重新设置，这样 task2 可能会连续获得多个信号量；最后由任务 task1 删除信号量。该实验实现过程由代码 3-1-24 中所示，其中涉及了信号量的创建、释放、获取、删除，以及信号量值设置操作。

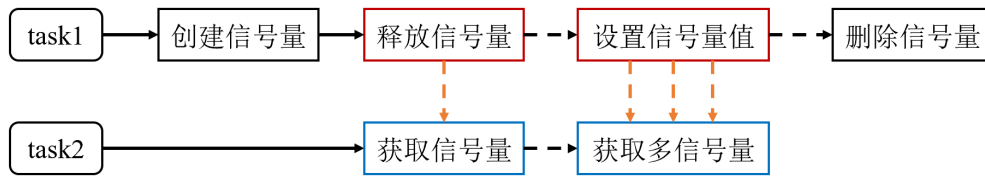


图 3-86-87 信号量实验设计

在代码 3-1-24 中，代码行 1 定义了一个 `task_semaphore` 函数，函数修改的内容与代码 3-1-15 中的 `task_scheduling` 函数内容相同，主要作用是创建两个任务，因此这里不再赘述；代码行 2 是定义一个宏定义，其中的标识符别定义为 -1；代码行 3 是设置一个信号量 ID。

接下来的代码行 5~24 和 26~38，分别实现了两个函数 `task1_func` 和 `task2_func`。在函数 `task1_func` 中，代码行 6 首先创建一个信号量；在代码行 7~18 中，循环五次，每次都释放一个信号量，并作了 1 秒延时（代码行 14~16）；当 `count` 为 2 时，重新对信号量进行设置（代码行 9~12）；当 `while` 循环结束后，延时 2 秒后删除创建的信号量（代码行 20~23）。函数 `task2_func` 的作用是获取 `task1_func` 释放的信号量（代码行 33）。总结函数 `task1_func` 和 `task2_func` 的关系是，`task1_func` 创建信号量，ID 为 `sem_id`，并向这个 ID 释放信号量；`task2_func` 获取 `task1_func` 释放的信号量，并用 `cnt` 记录获取次数。

代码 3-88-24 信号量实验代码

```

01 int task_semaphore(void);
02 #define WAIT_FOREVER -1
03 static int sem_id = -1;
04
05 void* task1_func(void* param) {
06     sem_id = UserSemaphoreCreate(0);
07     int count = 0;
08     while (count < 5) {
09         if(count == 2){
10             printf("set sem value as 5\n");
11             UserSemaphoreSetValue(sem_id, 3);
12         }
13
14         printf("task1 abandon a sem\n");
15         UserSemaphoreAbandon(sem_id);
16         UserTaskDelay(1000);

```

```

17         count++;
18     }
19
20     printf("task1 waiting for 2s then delete sem\n");
21     UserTaskDelay(2000);
22     UserSemaphoreDelete(sem_id);
23     sem_id = -1;
24 }
25
26 void* task2_func(void* param) {
27     int ret = 0;
28     int cnt = 0;
29     while(1) {
30         if(sem_id < 0){
31             break;
32         }
33         ret = UserSemaphoreObtain(sem_id, WAIT_FOREVER);
34         if(ret >= 0){
35             printf("task2 obtain sem %d times\n", cnt++);
36         }
37     }
38 }

```

系统编译、烧录、运行之后的界面如图 3-1-25 所示。前面两次都是 task1 释放一个信号量，然后 task2 接收一个信号量；当 count 为 2 时，信号量值被重新设置为 3，那么 task2 会连续收到 4 个信号量；最后两次同前面两次一样，都是 task1 释放一个信号量，task2 接收一个信号量。

另外，XiUOS 中也提供了信号量的查看命令 ShowSem，如图 3-1-26 和 3-1-27 所示。其中，图 3-1-26 和 3-1-27 分别显示的是信号量实验前和后的结果。在未进行信号量实验时，ShowSem 显示的有 3 条记录。在进行信号量实验之后，会多显示一条记录，其中的 SUSPEND_TASK 为 task2。这是因为 task2 在一直等待 task1 释放信号量，当其未接受到信号量时，系统将其挂起等待。

```

letter:/$ initialize letter-shell system success
task1 abandon a sem
task 2 obtain sem 0 times
task1 abandon a sem
task 2 obtain sem 1 times
set sem value as 5
task1 abandon a sem
task 2 obtain sem 2 times
task 2 obtain sem 3 times
task 2 obtain sem 4 times
task 2 obtain sem 5 times
task1 abandon a sem
task 2 obtain sem 6 times
task1 abandon a sem
task 2 obtain sem 7 times
task 1 waiting for 5s then delete sem

```

图 3-89-90 信号量实验结果

```

letter:/$ ShowSem
*****
SEM_ID  VALUE  SUSPEND_TASK
2       0      NONE
1       0      NONE
0       0      NONE
*****

```

图 3-91-92 未进行信号量实验的 ShowSem

```

letter:/$ ShowSem
*****
SEM_ID  VALUE  SUSPEND_TASK
3       0      :task_2 |
2       0      NONE
1       0      NONE
0       0      NONE
*****

```

图 3-93-94 进行信号量实验时的 ShowSem

(3) 互斥量实验:

互斥量实验设计过程如图 3-1-28 所示。该实验中首先创建了两个任务 task1 和 task2；任务 task1 创建了一个互斥量，然后最先获得该互斥量，获得后打印提示信息并释放互斥量；task1 的互斥量释放后，由 task2 获得，打印提示信息后继续释放；接下来再由 task1 获得互斥量，以此循环若干次，最后由 task1 删除互斥量。整个过程涉及到互斥量的创建、获取、释放、删除操作，具体实现过程如代码 3-1-25 所示。

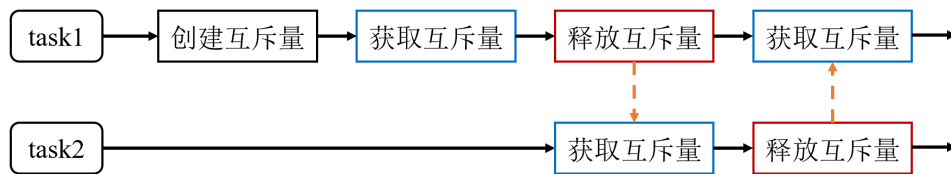


图 3-95-96 互斥量实验设计

代码 3-1-25 中的代码行 1 定义了一个 `task_mutex` 函数，主要作用是创建两个任务，`task_mutex` 函数与代码 3-2 中的 `task_scheduling` 函数内容相同；代码行 2 定义了一个互斥量 ID。代码行 4~22 和 24~34，分别实现了两个函数 `task1_func` 和 `task2_func`。在函数 `task1_func` 中，首先创建了一个互斥量，并判断是否创建成功（代码行 6~11）；接下来在 `while` 循环中，先获取一个信号量，打印提示信息后释放该信号量（代码行 12~18）；循环结束后，对创建的互斥量进行删除，并设置相应的信号量 ID（代码行 20~21）。在函数 `task2_func` 中，通过 `while` 循环不停地获取信号量，获取后打印提示信息并释放信号量（代码行 27~30）；当函数 `task1_func` 中将信号量删除之后，函数 `task2_func` 也会通过判断退出 `while` 循环（代码行 31~32）。

代码 3-97-25 `task_mutex` 函数代码

```

01 int task_mutex(void);
02 static int mtx_id = -1;
03
04 void* task1_func(void* param) {
05     int count = 0;
06     mtx_id = UserMutexCreate();
07     printf("mtx id = %d\n", mtx_id);
08     if(mtx_id < 0){
09         printf("Failed to create mutex\n");
10         return NULL;
11     }
12     while (count < 3) {
13         UserMutexObtain(mtx_id,-1);
14         printf("task1 obtains the mutex\n");
15         UserTaskDelay(1000);
16         UserMutexAbandon(mtx_id);
17         count++;
18     }
19

```

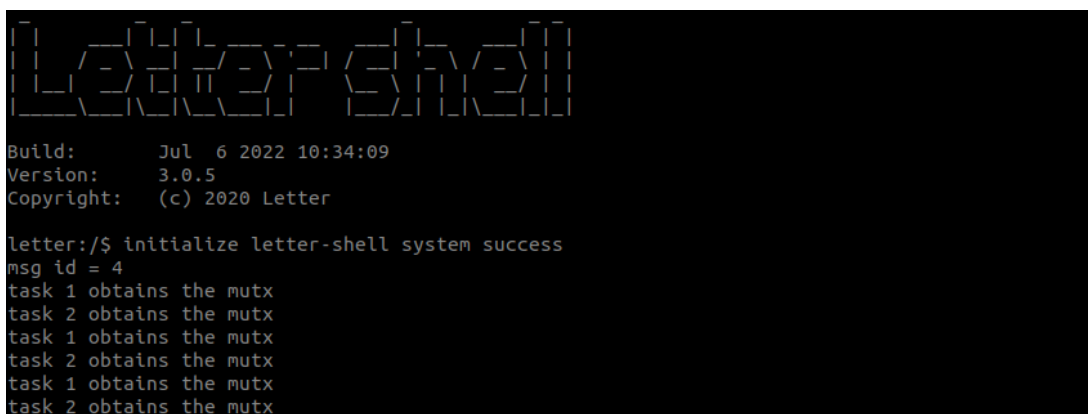
```

20     UserMutexDelete(mtx_id);
21     mtx_id = -1;
22 }
23
24 void* task2_func(void* param) {
25     while(1) {
26         if(mtx_id >= 0) {
27             UserMutexObtain(mtx_id,-1);
28             printf("task2 obtains the mutex\n");
29             UserTaskDelay(1000);
30             UserMutexAbandon(mtx_id);
31         } else
32             break;
33     }
34 }

```

系统编译、烧录、运行之后的界面如图 3-1-29 所示。可以看到，系统分配给新创建的信号量 ID 为 4；并从下面的打印信息可以看出，任务 task1 和 task2 交替地获取信号量。

XiUOS 中也提供了互斥量查看命令 ShowMutex。首先，在未进行互斥量实验时，使用该命令会显示图 3-1-30 中的结果，可以看到 4 条记录，均为空，互斥量 ID 是从 0~3。图 3-1-29 中显示的是进行互斥量实验时的结果，而且 ShowMutex 命令前后执行了两次。可以看到，图 3-1-31 中多了一个记录，互斥量的 ID 为 4。然后，前后两次命令显示中的互斥量 HOLDER 和 SUSPEND_TASK 均不相同，这是因为互斥量只能由一个任务持有。当任务 task1 是持有状态时（HOLDER），此时任务 task2 处于等待互斥量的挂起状态（SUSPEND）；同样，当任务 task2 持有互斥量时，那么任务 task1 是挂起状态。



```

letter-shell
Build:      Jul  6 2022 10:34:09
Version:    3.0.5
Copyright:  (c) 2020 Letter

letter:/$ initialize letter-shell system success
msg id = 4
task 1 obtains the mutex
task 2 obtains the mutex
task 1 obtains the mutex
task 2 obtains the mutex
task 1 obtains the mutex
task 2 obtains the mutex

```

图 3-98-99 互斥量实验结果

```
letter:/$ ShowMutex
*****
MUTEX_ID  HOLDER  RECRUSIVE_CNT  SUSPEND_TASK
3         NONE    0              NONE
2         NONE    0              NONE
1         NONE    0              NONE
0         NONE    0              NONE
*****
```

图 3-100-101 未进行互斥量实验的 ShowMutex

```
letter:/$ ShowMutex
*****
MUTEX_ID  HOLDER  RECRUSIVE_CNT  SUSPEND_TASK
4         task_2    1          task_1 |
3         NONE    0              NONE
2         NONE    0              NONE
1         NONE    0              NONE
0         NONE    0              NONE
*****

letter:/$ task 1 obtains the mutex
ShowMutex
*****
MUTEX_ID  HOLDER  RECRUSIVE_CNT  SUSPEND_TASK
4         task_1    1          task_2 |
3         NONE    0              NONE
2         NONE    0              NONE
1         NONE    0              NONE
0         NONE    0              NONE
*****
```

图 3-102-103 进行互斥量实验时的 ShowMutex

(4) 事件集实验：

事件集实验设计过程如图 3-1-32 所示。值得重提的是，XiUOS 中用 29 个位定义了 29 个事件，某一位被置为 1，就表明该位指向的事件有效。事件集实验中创建了两个任务 task 1 和 task2，其中，任务 task2 用于触发事件；任务 task1 接收相应的触发事件，根据触发条件决定是否执行任务。当任务 task2 触发事件 3，任务 task1 的 EVENT_OR 触发，执行相应的语句；任务 task2 继续触发事件 5 和事件 3，任务 task1 的 EVENT_AND 触发，执行相应的语句。事件集实验包含事件集的创建、处理、触发、删除等操作，具体实现过程如代码 3-1-26 所示。

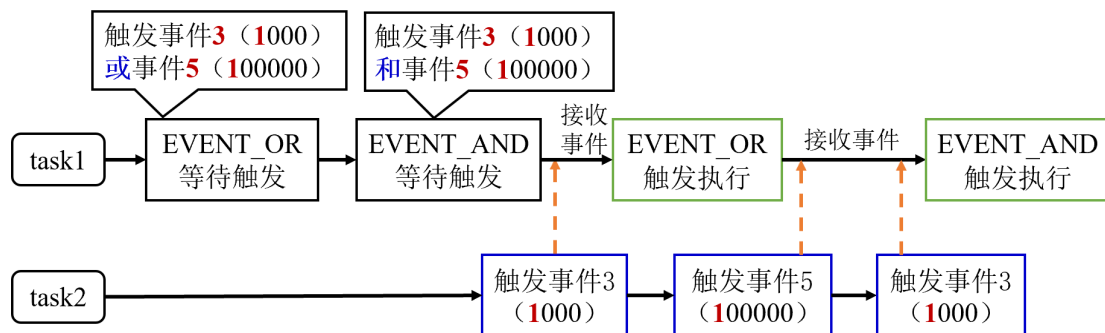


图 3-104-105 事件集实验设计

在代码 3-1-26 中，代码行 1 定义了一个 `task_eventSet` 函数，主要作用是创建两个任务，`task_eventSet` 函数与代码 3-2 中的 `task_scheduling` 函数内容相同；代码行 2 和 3 定义了两个宏 `EVENT_FLAG1` 和 `EVENT_FLAG2`，分别表示两个事件标识，例如 `EVENT_FLAG1` 的值是 1000，那么表示 XiUOS 中的第 3 个事件（编号从 0 开始）；在函数 `task1_func` 中，首先创建了一个事件集，并判断是否创建成功（代码行 8~11）；代码行 13~15 和 17~19 分别是两个事件处理函数，其触发条件各不相同。其中，代码行 13~15 的事件处理条件是 `EVENT_FLAG1 | EVENT_FLAG2`，触发方式是 `EVENT_OR | EVENT_AUTOCLEAN`，即逻辑或触发，只要接收到 `EVENT_FLAG1` 或 `EVENT_FLAG2` 的一个事件，条件满足，执行相应的打印语句；代码行 17~19 的事件处理条件是 `EVENT_FLAG1 | EVENT_FLAG2`，触发方式是 `EVENT_AND | EVENT_AUTOCLEAN`，即逻辑与触发，当 `EVENT_FLAG1` 和 `EVENT_FLAG2` 事件都触发之后，才条件成立并执行相应的打印语句；另外，任务 `task1_func` 中还负责最后的事件集删除和事件集 ID 重置（代码行 21~22）；任务 `task2_func` 主要负责触发事件，例如在代码行 29 中触发了事件 3（`EVENT_FLAG1`），接着在代码行 33 触发了事件 5（`EVENT_FLAG2`），最后在代码行 37 又触发了事件 3。以上触发的事件会被任务 `task2` 按顺序一一接收，根据条件触发相应的处理。

代码 3-106-26 事件集实验代码

```

01 int task_eventSet(void);
02 #define EVENT_FLAG1 (1 << 3) //1000, event 3 is set
03 #define EVENT_FLAG2 (1 << 5) //100000, event 5 is set
04 static int event_id = -1;
05
06 void* task1_func(void* param) {

```

```

07     uint32 event;
08     event_id = UserEventCreate(EVENT_AND);
09     if (event_id<0) {
10         printf("Failed to create  event.\n");
11     }
12
13     if (UserEventProcess(event_id, (EVENT_FLAG1 | EVENT_FLAG2), EVENT_OR |
14         EVENT_AUTOCLEAN, WAITING_FOREVER, &event) == 0){
15         printf("task1 receives %s, val = 0x%x, EVENT_OR is triggerred\n",
16             event == 0x8 ? "event3" : "fault",event);
17     }
18     UserTaskDelay(1000);
19     if (UserEventProcess(event_id, (EVENT_FLAG1 | EVENT_FLAG2), EVENT_AND
20         | EVENT_AUTOCLEAN, WAITING_FOREVER, &event) == 0){
21         printf("task1 receives %s, val = 0x%x, EVENT_AND is triggerred\n",
22             event == 0x28 ? "event3 and event5" : "fault",event);
23     }
24
25     UserEventDelete(event_id);
26     event_id = -1;
27 }
28
29 void* task2_func(void* param) {
30     UserTaskDelay(1000);
31     if(event_id >=0 ){
32         printf("task2 sends event3, val=0x8\n");
33         UserEventTrigger(event_id, EVENT_FLAG1);
34         UserTaskDelay(1000);
35
36         printf("task2 sends event5, val=0x20\n");
37         UserEventTrigger(event_id, EVENT_FLAG2);
38         UserTaskDelay(1000);
39
40         printf("task2: send event3\n");
41         UserEventTrigger(event_id, EVENT_FLAG1);
42     }
43 }

```

事件集实验运行的界面如图 3-1-33 所示。可以看过，任务 task2 逐个地触发事件 3、事件 5 和事件 3。当任务 task1 接收到事件 3 后，触发 EVENT_OR 条件；当任务 task1 接收到事件 5 和 3 之后，触发 EVENT_AND 条件。值得注意的是，当任务 task1 触发 EVENT_AN

D 时，事件标记的值是事件 5 和 3 的和，即 101000（0x28），可以看出，第 3 位和第 5 位均为 1。

XiUOS 中也提供了查看事件集的命令 ShowEvent。事件集实验运行过程中的 ShowEvent 结果如图 3-1-34 所示。可以看出，当前 EVENT_ID 为 0；OPTIONS 的值为 1，表示正在等待触发 EVENT_AND；EVENTS 的值为 0x20，表明当前触发的是事件 5；由于任务 task1 一直在等待触发，所以其当前处于挂起状态。

```

Letter-shell
Build:      Jul 13 2022 14:30:43
Version:    3.0.5
Copyright:  (c) 2020 Letter

letter:/ $ initialize letter-shell system success
task2 sends event3, val=0x8
task1 receives event3, val = 0x8, EVENT_OR is triggered
task2 sends event5, val=0x20
task2: send event3
task1 receives event3 and event5, val = 0x28, EVENT_AND is triggered
    
```

图 3-107-108 时间集实验结果

```

letter:/ $ ShowEvent
*****
EVENT_ID  OPTIONS  EVENTS    SUSPEND_TASK
  0        0x1    0x20      task_1_|
*****
    
```

图 3-109-110 事件集实验的 ShowEvent

1.2.4 任务隔离实验

在 XiUOS 中，通常情况下内核和应用层是不隔离的，如果需要任务隔离，则需要通过一系列的设置完成。首先，在控制台的 XiZi 目录下输入 make menuconfig，会显示如图 3-4 4 所示的 XiZi Project Configuration 界面。接着，分别选择 Kernel feature→Task feature→Enable task isolation，整个过程如图 3-1-35 至 3-1-37 所示。在图 3-1-37 中，通过输入“y”和“n”，分别选择使能任务隔离和不使能任务隔离。

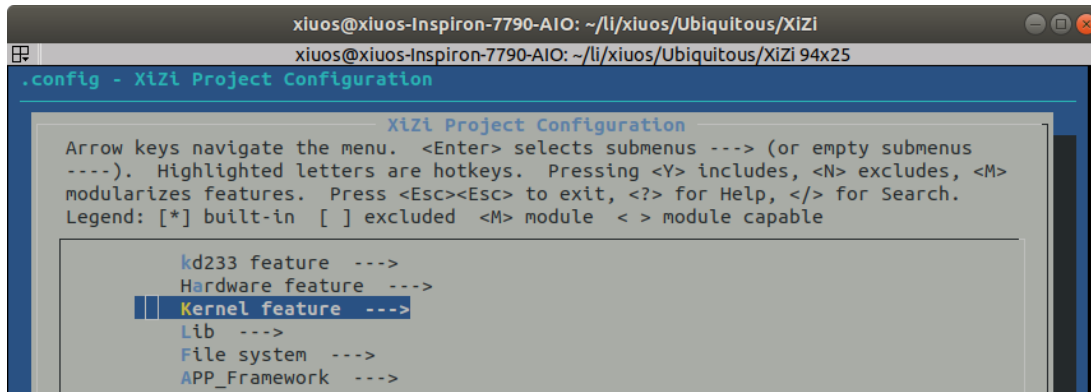


图 3-111-112 Kernel feature

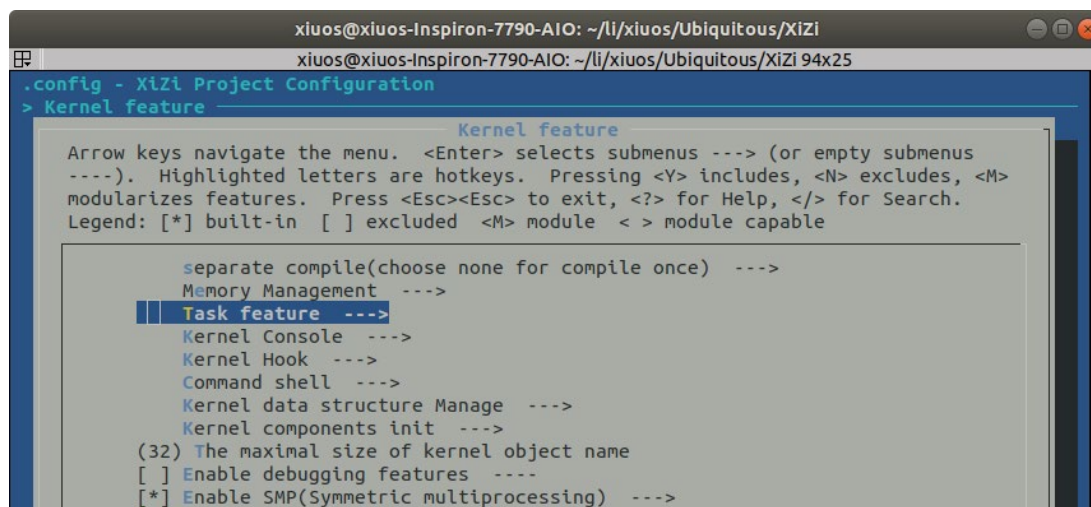


图 3-113-114 Task feature

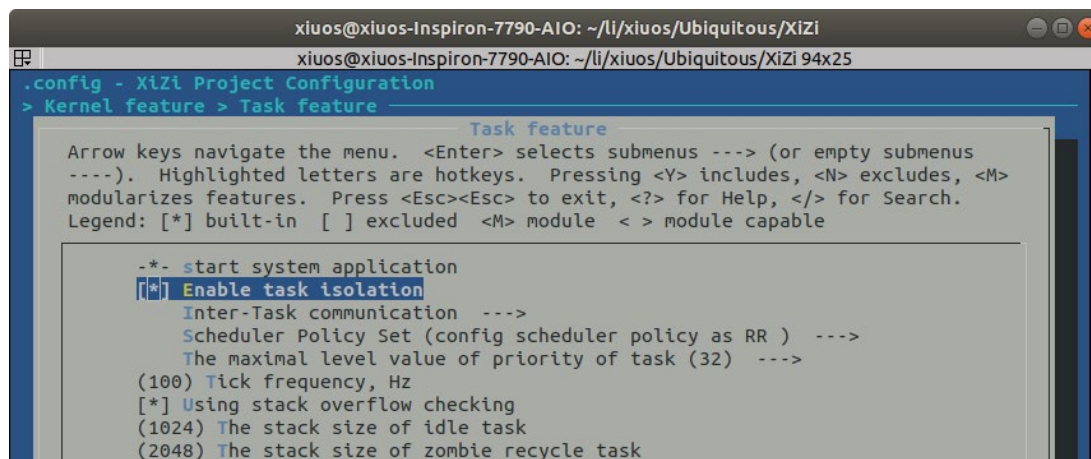


图 3-115-116 Enable task isolation

任务隔离设置完成之后，那么内核和应用运行在不同的地址空间。对系统进行编译（make 命令）之后，通过目录显示命令（ll build/）查看当前文件。如图 3-1-38 所示，可看到目录下有两个 bin 文件（app.bin 和 kernel.bin）。由于硬件厂商的限制，对于 K210 芯片而言，不能分段烧录两个 bin 文件。那么需要通过“python mergebin.py”命令，将这两个 bin 文件合并成一个 bin 文件。合并之后，在图 3-1-39 中可以看到此时多了一个 XiZi_edu-riscv64.bin 文件，即为合并的 bin 文件。

使用之前同样的方法可将 XiZi_edu-riscv64.bin 烧录至 K210 开发板，其运行的状态如图 3-1-40 所示。可以看到，系统提示内核和应用态已经隔离，而且应用态的地址空间范围是 0x80102000~0x80600000。值得注意的是，由于内核态与应用态已经隔离，所以使用应用态的任务去访问内核态的资源已经不能完成。这在系统编译时就提示错误，也就无法烧录至开发板上，所以任务隔离实验无法用代码显示其效果。

```
xiuos@xiuos-Inspiron-7790-AIO:~/li/xiuos/Ubiquitous/XiZi$ ll build/
总用量 3300
drwxrwxr-x 4 xiuos xiuos 4096 7月 14 09:20 ./
drwxrwxr-x 10 xiuos xiuos 4096 7月 14 09:20 ../
drwxrwxr-x 5 xiuos xiuos 4096 7月 14 09:20 APP_Framework/
-rw-rw-r-- 1 xiuos xiuos 220385 7月 14 09:20 make.dep
drwxrwxr-x 3 xiuos xiuos 4096 7月 14 09:20 Ubiquitous/
-rw-rw-r-- 1 xiuos xiuos 287594 7月 14 09:20 XiZi_app.map
-rwxrwxr-x 1 xiuos xiuos 178 7月 14 09:20 XiZi_kd233_app.bin*
-rwxrwxr-x 1 xiuos xiuos 32296 7月 14 09:20 XiZi_kd233_app.elf*
-rwxrwxr-x 1 xiuos xiuos 288256 7月 14 09:20 XiZi_kd233_kernel.bin*
-rwxrwxr-x 1 xiuos xiuos 1543352 7月 14 09:20 XiZi_kd233_kernel.elf*
-rw-rw-r-- 1 xiuos xiuos 977674 7月 14 09:20 XiZi_kd233.map
```

图 3-117-118 编译后生成应用和内核两个 bin 文件

```
xiuos@xiuos-Inspiron-7790-AIO:~/li/xiuos/Ubiquitous/XiZi$ ll build/
总用量 4332
drwxrwxr-x 4 xiuos xiuos 4096 7月 14 09:30 ./
drwxrwxr-x 10 xiuos xiuos 4096 7月 14 09:20 ../
drwxrwxr-x 5 xiuos xiuos 4096 7月 14 09:20 APP_Framework/
-rw-rw-r-- 1 xiuos xiuos 220385 7月 14 09:20 make.dep
drwxrwxr-x 3 xiuos xiuos 4096 7月 14 09:20 Ubiquitous/
-rw-rw-r-- 1 xiuos xiuos 287594 7月 14 09:20 XiZi_app.map
-rwxrwxr-x 1 xiuos xiuos 178 7月 14 09:20 XiZi_kd233_app.bin*
-rwxrwxr-x 1 xiuos xiuos 32296 7月 14 09:20 XiZi_kd233_app.elf*
-rw-rw-r-- 1 xiuos xiuos 1052850 7月 14 09:30 XiZi_kd233.bin
-rwxrwxr-x 1 xiuos xiuos 288256 7月 14 09:20 XiZi_kd233_kernel.bin*
-rwxrwxr-x 1 xiuos xiuos 1543352 7月 14 09:20 XiZi_kd233_kernel.elf*
-rw-rw-r-- 1 xiuos xiuos 977674 7月 14 09:20 XiZi_kd233.map
```

图 3-119-120 合成两个 bin 文件

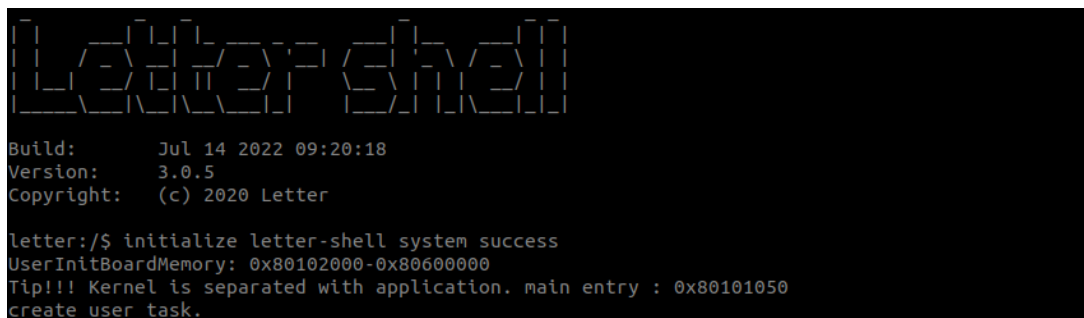


图 3-121-122 任务隔离运行后的 XiUOS 界面

第2章 内存管理

2.1 内存管理介绍

XiUOS 操作系统提供了独特的内存管理分配算法进行内存管理，通过静态内存管理和动态内存管理相结合，保证分配和释放内存的实时性，提高内存的使用率，有效地规避了内存碎片问题，同时增加了内存检索的速度。

2.1.1 内存堆管理

（1）静态内存划分：

静态内存包含 2 个链表，其中，每个链表都具有 `block_size`、`total_count`、`free_count` 和 `free_list` 这四个属性。其中，`block_size` 记录了当前链表中每个静态内存块的大小；`total_count` 记录了系统初始化之后分配给该链表中静态内存块的总个数；`free_count` 记录了该链表中还可以分配给用户静态内存块的个数；`free_list` 指向各个空闲静态内存块。

图 3-2-1 为静态内存链表的具体情况，图中包括两个静态链表 1 和 2。静态链表头 1 指向的内存池中存放的静态内存块的大小都是 32 字节，静态链表头 2 所指向的内存池中存放的静态内存块的大小都是 64 字节。此外，系统分别配置了静态链表头 1 和静态链表头 2 中静态内存块的 `total_count` 个数为 256 和 128。因此，静态链表头 1 最多可以响应用户 256 次的小于 32 字节的内存请求，静态链表头 2 最多可以响应用户 128 次的介于 33~64 字节之间的内存请求，一旦对应的静态内存块分配完了，系统会向动态内存区域寻求内存空间分配。

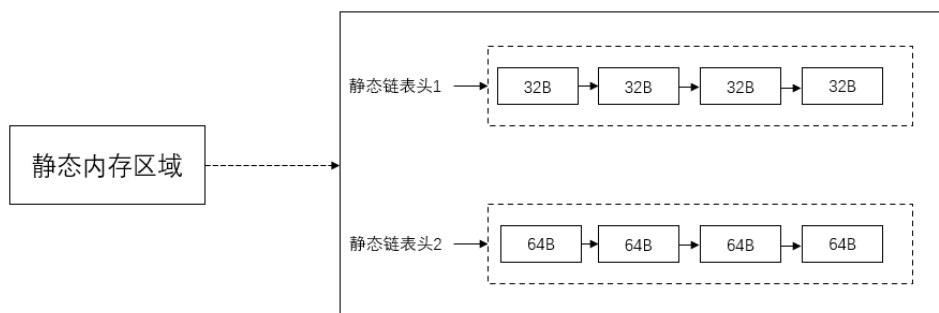


图 3-27-123 静态内存链表

（2） 静态内存分配：

在用户发起内存分配请求时，若用户请求分配的内存空间大小小于等于预设的静态内存阈值，且所述静态内存区域有相应空闲的内存块，则从静态内存区域分配相应大小的内存空间。从静态内存区域分配静态内存块的过程，包括三种情况。

- 当用户请求分配内存大小小于等于 32 字节，那么分系统获取静态链表头 1，从静态链表头 1 中获取一个静态内存块返回给用户；
- 当用户请求分配内存大小为 33~64 字节，那么系统获取静态链表头 2，从静态链表头 2 中获取一个静态内存块返回给用户；
- 若所获得的静态链表头是一个空的链表，即没有空闲的静态内存块可用，那么静态内存分配失败。

（3） 静态内存释放：

当用户请求释放内存，释放操作也包括三种情况。

- 要释放的内存地址是不合法的，则直接返回释放操作；
- 要释放的内存地址属于静态链表头 1 区域，则将内存块放置到链表头 1 的头部；
- 要释放的内存地址属于链表头 2 区域，则将内存块放置到链表头 2 的头部。

（4） 动态内存划分：

静态内存划分后，剩下的内存区域作为动态内存分配给动态内存区域。动态内存区域管理用到了三种重要的数据结构，分别是已分配动态内存段、空闲动态内存段和动态内存管理数据结构。图 3-2-2 显示了已分配动态内存段和空闲动态内存段这两种数据结构。其中，已分配动态内存包括元数据信息和用户的数据段（用户数据），元数据信息中的 `size` 属性记录了该动态内存段的内存大小，`prev_size` 属性则记录了该动态内存段前一个相邻动态内存段

的内存大小，从而可以获取前一个相邻动态内存段。用户数据则是分配给用户使用的内存空间。如图 3-2-2 所示，空闲动态内存段的元数据信息具有 size、prev_size、prev 和 next 这四个属性，其中，size 和 prev_size 属性与已分配动态内存段的对应属性表示的意义相同，prev 属性记录了前一个空闲动态内存段，最后一个 next 属性则记录了下一个空闲动态内存段。

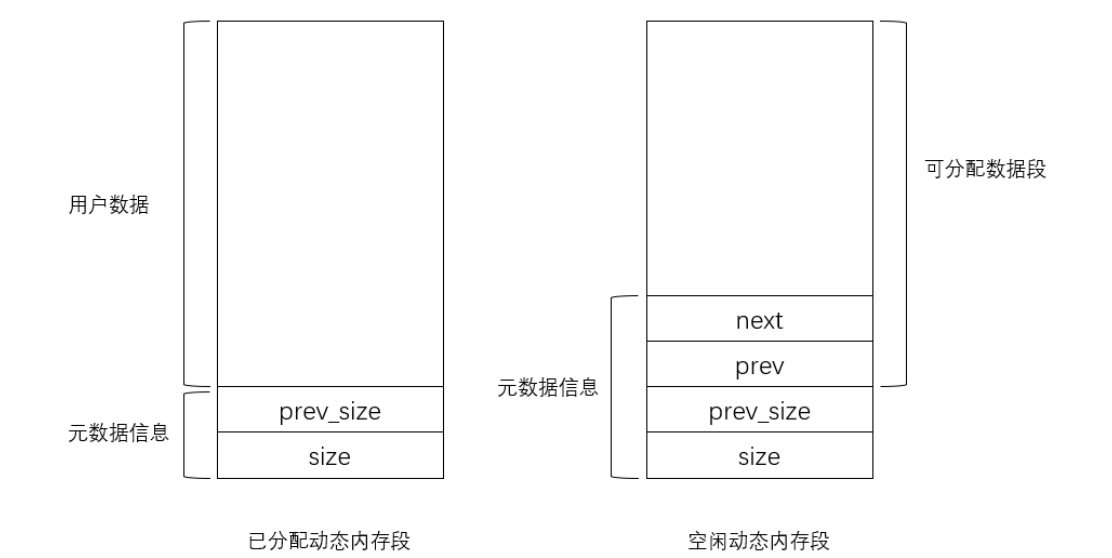


图 3-28-29 空闲动态内存段的元数据信息

图 3-2-3 展示了动态内存区域初始化之后的状态，系统将动态内存区域划分为三个动态内存段，分别是起始地址处已分配动态内存段和结尾已分配动态内存段，中间区域的内存用一个空闲动态内存段记录。

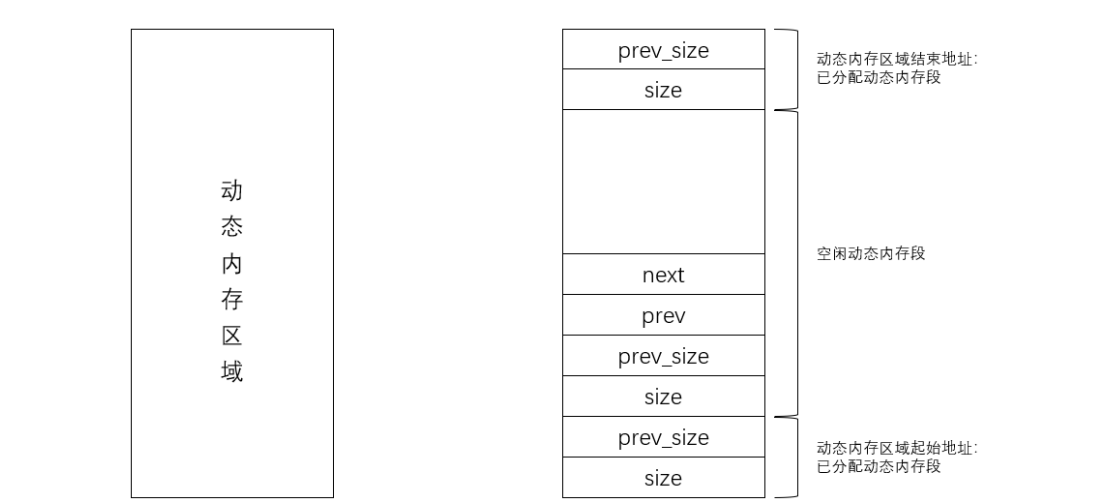


图 3-30-124 动态内存区域记录方式

动态内存管理数据结构是动态内存区域管理中的核心数据结构,用于组织所有的空闲动态内存段。所述动态内存管理数据结构具有 `total_size`、`dynamic_start`、`dynamic_end` 和 `freeLists` 这四个属性。其中, `total_size` 属性用于记录动态内存区域的大小, `dynamic_start` 属性用于记录起始地址处已分配动态内存段, `dynamic_end` 属性用于记录结尾处已分配动态内存段, `freeLists` 属性是一个多级链表,每一级链表记录了内存大小在指定范围的空闲动态内存段。根据空闲内存段的大小,系统判断空闲内存段所属动态链表,然后将空闲内存段插入不同的空闲链表中。

系统可配置空闲链表的个数为 10。当动态内存段大小在 1-31 字节之间,则插入到空闲链表 `freeList1` 中,当动态内存段大小在 32-63 字节之间时,则插入到空闲链表 `freeList2` 中,需要特别说明的是,当动态内存段大小大于或者等于 8192 字节时,都插入到空闲链表 `freeList10` 中。如下表所示:

表 3-31-2 动态链表分配

申请的内存范围	分配动态链表头
1 ~ 31 字节	<code>freeList1</code>
32 ~ 63 字节	<code>freeList2</code>
64 ~127 字节	<code>freeList3</code>
.....	
8192 ~MAX(MAX 由系统配置)	<code>freeList10</code>

(5) 动态内存分配:

系统根据要分配或者要释放的内存段大小计算要操作的内存段链表头,当内存段大小为 1 ~ 31 字节,那么计算之后将获取空闲动态链表头 `freeList1`,当内存段大小为 32 ~ 63 字节,那么计算之后将获取动态链表头 `freeList2`,依次类推。当用户请求从动态内存段申请内存时,有以下几种分配情况:

- 用户请求分配 `n` 个字节(对齐后的字节), `freeListK` 链表中有一个 `m` 字节内存大小可以满足用户的需求,之后进行内存块的分割操作,分成 `n` 字节和 `(n-m)` 字节,其中 `n-m` 小于 8 字节,因此将 `m` 字节内存返回给用户;

- 用户请求分配 n 个字节(对齐后的字节), `freeListK` 链表中有一个 m 字节内存大小可以满足用户的需求, 之后进行内存块的分割操作, 分成 n 字节和 $(n-m)$ 字节, 其中 $n-m$ 大于或等于 8 字节, 因此将 n 字节内存返回给用户并且将 $n-m$ 个字节重新插入动态内存管理结构;
- 用户请求分配 n 个字节(对齐后的字节), `freeListK` 链表中没有空闲内存大小可以满足用户的需求, 并且 `freeListK+1` 级链表存在, 则从 `freeListK+1` 链表中分配内存;
- 用户请求分配 n 个字节(对齐后的字节), `freeListK` 链表中没有空闲内存大小可以满足用户的需求, 并且 `freeListK+1` 级链表不存在, 则内存分配失败。

(6) 动态内存释放:

当用户请求释放内存, 系统释放内存区域到对应的动态内存段中。释放一块动态内存段, 下图提供了动态内存段释放和合并的三种情况。其中, 每一个动态内存段都包含了两个指针, 分别指向前一个相邻内存段和后一个相邻内存段, 下面介绍图 3-2-4 展示的释放和合并的三种情况:

- 如图中 (a) 所示, 四块动态内存段, 其中, A, B, D 三段为已分配动态内存段, C 为空闲动态内存段。当前, 系统要释放内存段 B, 动态内存管理模块检测到 B 内存段的后一个相邻 C 也是空闲内存段, 则系统会合并内存段 B, C, 然后释放 BC 合并之后的内存段。
- 如图中 (b) 所示, A, B, C, D 四块已分配内存段。系统要释放内存段 B, 动态内存管理模块检测到 B 的相邻内存段都是已分配内存段, 那么直接释放内存段 B。
- 如图中 (c) 所示, 四块动态内存段, 其中, A, C 是空闲内存段; B, D 是已分配内存段。系统要释放内存段 B, 动态内存管理模块检测到 B 的前一个相邻内存段 A 和后一个相邻内存段 C 都是空闲内存段, 那么先合并 A, B, C 三个内存段形成一个大的空闲内存段 ABC, 然后释放合并形成的 ABC 空闲内存段。

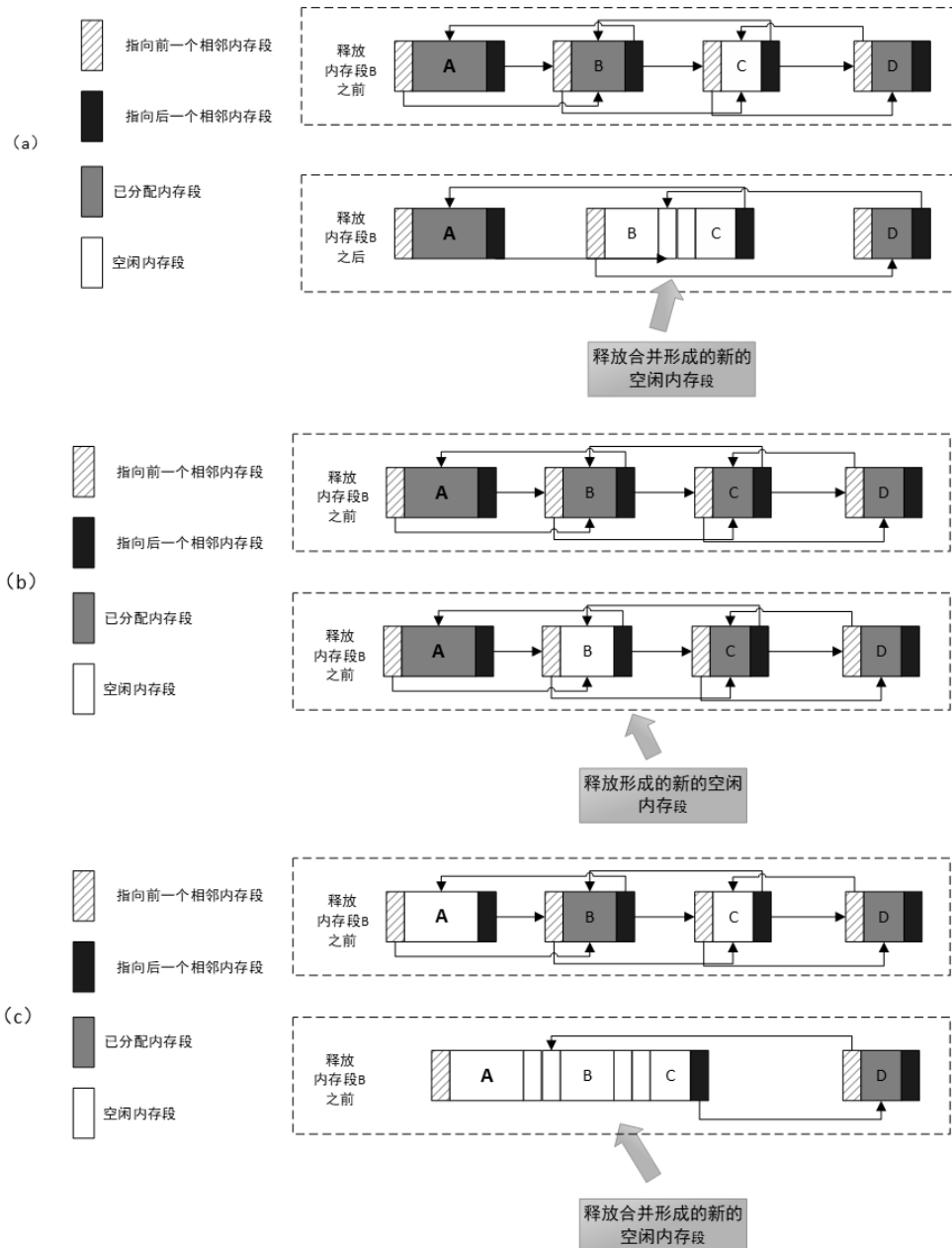


图 3-32-125 动态内存释放过程

2.1.2 内存池

内存堆管理器可以分配任意大小的内存块，非常灵活和方便。但其也存在明显的缺点：一是分配效率不高，在每次分配时，都要空闲内存块查找；二是容易产生内存碎片。为了提高内存分配的效率，并且避免内存碎片，XiUOS 提供了另外一种内存管理方法，内存池。

内存池在创建时先向系统预先申请分配一定数量、大小相等（一般情况下）的内存块留

作备用。当有新的内存需求时，就从内存池中分出一部分内存块，若内存块不够再继续申请新的内存。

2.2 内存堆管理接口

XiUOS 中定义一系列内存堆函数接口：

- `void InitBoardMemory(void *start_phy_address, void *end_phy_address)`，该函数用来初始化静态和动态内存。其中，参数 `start_phy_address` 表示内存开始的物理地址；`end_phy_address` 表示内存结束的物理地址。
- `void *x_malloc(xs_size_t size)`，该函数用来分配一块合适大小的内存，如果分配成功，则返回分配内存的首地址；否则返回 `NULL`。其中，参数 `size` 表示需要被分配的内存大小。
- `void *x_realloc(void *pointer, xs_size_t size)`，该函数用来重新分配一块内存，将原内存块中的数据保存到新的内存中去，并将原内存块释放。如果分配成功，则返回分配内存的首地址；否则返回 `NULL`。其中，参数 `pointer` 表示原内存块的指针；`size` 表示需要重新分配的内存大小。
- `void *x_calloc(xs_size_t count, xs_size_t size)`，该函数会分配多个内存块并将内存块中的数据初始化为 0。其中，参数 `count` 表示需要分配的内存块数量；`size` 表示单个内存块的大小。
- `void x_free(void *pointer)`，该函数用来释放内存。其中，参数 `pointer` 表示指向需要被释放的内存。
- `void MemoryInfo (uint32 *total_memory, uint32 *used_memory, uint32 *max_used_memory)`，该函数用于获取内存的统计信息。其中，参数 `total_memory` 表示内存总量；`used_memory` 表示已使用的内存；`max_used_memory` 表示最大分配内存。
- `void ShowMemory(void)`，该函数用于打印内存信息。
- `void ListBuddy(void)`，该函数用于打印动态内存中的空闲节点信息。

2.3 内存池管理接口

XiUOS 中也提供了一系列内存池管理的函数接口：

- `GatherMemType CreateMemGather(const char *gm_name, x_size_t block_number, x_size_t one_block_size)`，该函数用于创建一个内存池，如果创建成功，返回第一个内存块

的地址，否则返回 XS_NULL。其中，参数 gm_name 表示内存池名；block_number 表示内存池中的内存块数量；one_block_size 表示每个内存块的容量。

- x_err_t DeleteMemGather(GatherMemType gm_handler)，该函数用于删除由 xs_CreateMemGather 创建的内存池。其中，参数 gm_handler 需要被删除的内存池。
- x_err_t InitMemGather(struct MemGather *gm_handler, const char *gm_name, void *begin_address, x_size_t gm_size, x_size_t one_block_size)，该函数用于初始化内存池，用于静态内存管理模式。其中，参数 gm_handler 表示内存池对象；gm_name 表示内存池名称；begin_address 表示内存池起始地址；gm_size 表示内存池数据区大小；one_block_size 表示每个内存块的容量。
- x_err_t RemoveMemGather(struct MemGather *gm_handler)，该函数用于删除由 xs_MemGatherInit 创建的内存池。其中，参数 gm_handler 表示需要被移除的内存池。
- void *AllocBlockMemGather(GatherMemType gm_handler, int32 msec)，该函数将从指定的内存池中分配一个内存块。如果内存池中有可用的内存块，则从内存池的空闲块链表上取下一个内存块，将空闲内存块数目减 1 并返回这个内存块；如果内存池中已经没有空闲内存块，则判断超时时间设置：若超时时间设置为零，则立刻返回空内存块；如果等待时间大于零，则把当前线程挂起在该内存池对象上，直到内存池中有可用的自由内存块，或等待时间到达。其中，参数 gm_handler 表示内存池对象；wait_time 表示超时时间。
- void FreeBlockMemGather(void *data_block)，该函数用于释放指定的内存块，然后增加内存池对象的可用内存块数目，并把该被释放的内存块加入空闲内存块链表上。接着判断该内存池对象上是否有挂起的线程，如果有则唤醒挂起线程链表上的首线程。其中，参数 data_block 表示需要被释放的内存块指针。

2.4 内存管理及实验

前面介绍了 XiUOS 下的内存堆和内存池的管理方式。值得注意的是，在内存堆中的静态和动态内存管理，在内存申请时都是自动完成的。基本的原则是：如果需要分配的内存小于等于 32 字节，那么就从 32 字节的内存池中分配；如果申请的内存大小是 33~64 字节之间，那么就从 64 字节内存池中分配；如果需要大于 64 字节的内存，或者上述两种静态内存

已经分配完毕，那么就采用动态内存分配。另外，XiUOS 在 64 位 RISC-V 架构上采用 8 字节对齐原则，即使只申请 4 字节的内存，系统也会分配 8 字节内存。

2.4.1 内存堆分配释放实验

内存堆的内存分配和释放实验过程如代码 3-2-1 所示。代码行 1~6 定义了一些常量和变量，如代码行 1 是在该文件中引用 ShowMemory 命令；代码行 2~3 定义迭代次数和申请内存数量的常量；代码行 4 是申请的内存大小；代码行 5~6 分别定义了一个指针数组和无符号长整型数组，用于存放每次申请的内存地址和记录分配的内存大小。

代码行 8~32 定义了一个函数 memAllocFree，在其中实现内存申请和释放操作。首先是内存的申请，代码行 12 中的分配内存大小 size 是 $1 \sim 2^{22}$ 之间的一个随机数，其随机种子是在代码行 9 中产生；随后代码行 13 中采用 x_malloc 分配 size 大小的内存；代码行 14 中将该 size 记录在 arrsize 数组中；内存的申请可能成功，也可能不成功，所以在代码行 16~23 中进行内存申请状态的判断；以上内存的申请和状态判断循环 ITER_NUM 次，如代码行 11 所示；代码行 25 的 ShowMemory 命令在 ITER_NUM 次内存申请之后，显示当前的内存分配情况；代码行 26~29 通过 x_free 函数对申请的内存进行释放。

代码 3-33-34 内存堆分配释放实验代码

```

01  extern void ShowMemory(void);
02  #define ITER_NUM 10
03  #define MEM_NUM 23
04  unsigned long size;
05  char *ptr[ITER_NUM];
06  unsigned long arrsize[ITER_NUM];
07
08  void memAllocFree(){
09      srand(time(0));
10      int idx=0;
11      while (idx<ITER_NUM) {
12          size = 1 << (rand() % MEM_NUM);
13          ptr[idx] = x_malloc(size);
14          arrsize[idx]=size;
15
16          if (ptr[idx] != NULL) {
17              printf("Successfully get memory: %d byte\n",size);
18          } else {

```

```
19         printf("Failed to get %d byte memory!\n",size);
20         break;
21     }
22     idx++;
23 }
24
25 ShowMemory();
26 for(idx--; idx>=0; idx--) {
27     x_free(ptr[idx]);
28     printf("Free memory: %d byte\n",arrsize[idx]);
29 }
30
31 return;
32 }
```

内存堆分配释放实验的结果如图 3-2-5 至 3-2-7 所示。首先，图 3-2-5 是未进行内存堆实验的 ShowMemory 结果。可以看到，系统的内存总量（total memory）是 5996512 字节，已经使用的内存（used memory）是 23768 字节。总的缓存大小（total cache size）是 4096 字节，即之前所述的静态内存。其中，32 字节的静态内存有 64 个，已经使用了 7 个，还剩余 57 个；64 字节的静态内存 32 个，已经使用了 5 个，还剩余 27 个。

图 3-2-6 显示的内存堆分配和释放实验过程结果，以及 ShowMemory 结果。可以看到，总共申请了 10 次内存，并且是全部申请成功，此时已经使用的内存大小为 4648248 字节。其中，小于 32 字节的内存申请有 4 次，这些都先从 32 字节的静态内存中分配。所以，此时的 32 字节内存还剩余 53 个，相比于之前的 57 个，正好少了 4 个。另外，申请的内存中没有介于 33~64 字节的，所以 64 字节的静态内存剩余数量没有变化。

图 3-2-7 显示的是申请的内存被释放的过程。可以看到，内存被释放了十次，即之前分配的内存都已经被释放。当前使用的内存重新变成 23768 字节，并且相应的 32 字节静态内存数量也变为原来的 57 个。以上信息都表明，申请的内存都已经被释放成功。

```

Letter
Build:      Jul 19 2022 17:33:32
Version:    3.0.5
Copyright:  (c) 2020 Letter

letter:/$ initialize letter-shell system success
total memory: 5996512
used memory : 23768
total cache size: 4096, 57/64[32B],27/32[64B]

```

图 3-35-126 未进行内存堆实验的 ShowMemory 结果

```

Successfully get memory: 1 btye
Successfully get memory: 262144 btye
Successfully get memory: 32768 btye
Successfully get memory: 1 btye
Successfully get memory: 65536 btye
Successfully get memory: 32 btye
Successfully get memory: 4096 btye
Successfully get memory: 65536 btye
Successfully get memory: 4194304 btye
Successfully get memory: 1 btye
total memory: 5996512
used memory : 4648248
total cache size: 4096, 53/64[32B],27/32[64B]

```

图 3-36-127 内存堆分配的 ShowMemory 结果

```

Free memory: 1 byte
Free memory: 4194304 byte
Free memory: 65536 byte
Free memory: 4096 byte
Free memory: 32 byte
Free memory: 65536 byte
Free memory: 1 byte
Free memory: 32768 byte
Free memory: 262144 byte
Free memory: 1 byte
total memory: 5996512
used memory : 23768
total cache size: 4096, 57/64[32B],27/32[64B]

```

图 3-37-128 内存堆释放的 ShowMemory 结果

2.4.2 内存堆容量实验

内存堆容量实验的过程如代码 3-2-2 所示，该实验的思路是不断分配内存，直到将内存全部耗尽，相当于内存分配的压力测试。代码行 1~5 定义了不同 CPU 架构下的最大内存申请数量；代码行 6~8 的定义与代码 3-2-1 中相同。

代码行 10~34 定义了一个 memCapacity 函数，在其中的代码行 13~19 是循环申请内存，大小 size 是从 1 字节开始，每轮循环都增加 1 个字节。需要再次提醒的是，RISC-V 架构中内存申请的字节对齐机制。随着循环次数增加，size 也不断增大，当内存不够时，程序就转

到代码行 20~24 中。此时将 size 设置为较小的 32 字节，这样会继续进入代码行 14~19 中。以此往复，直到将内存消耗完毕。最后在代码行 28~31 中，将所有申请到的内存进行释放。

代码 3-38-39 内存堆容量实验代码

```
01  #ifdef ARCH_ARM
02  #define MEM_COUNT 1024
03  #else
04  #define MEM_COUNT 8192
05  #endif
06  extern void ShowMemory(void);
07  char *ptr[MEM_COUNT];
08  unsigned int arr[MEM_COUNT];
09
10  int memCapacity(){
11      int idx=0;
12      int size = 1;
13      while( idx< MEM_COUNT) {
14          ptr[idx] = x_malloc(size);
15          if (ptr[idx]) {
16              printf("\033[32;1m malloc memory [%d] ptr[%d] = 0x%x\033[0m\n",
size,idx,ptr[idx]);
17              arr[idx] = size;
18              idx++;
19              size++;
20          }else{
21              if (size == 32)
22                  break;
23              size=32;
24          }
25      }
26
27      ShowMemory();
28      for (idx--; idx>=0 ; idx--) {
29          x_free(ptr[idx]);
30          printf("free memory: %d byte\n",arr[idx]);
31      }
32
33      return 0;
34  }
```

内存堆容量实验的结果如图 3-2-8 所示。左图是 size 逐渐增大时的内存分配，当 size 为 3409 时，由于内存不足，无法为其分配内存。然后，根据实验代码逻辑，size 会被设置为 32 字节，并继续增加 size 和分配内存。这个内存分配过程中，可能分配的是动态内存，也可能分配的是 64 字节的静态内存。右图中发现，当 size 变为 56 之后，就无法继续分配更大的内存，而是 size 一直固定在 32 字节，并连续分配了若干次。这是因为此时的动态内存其实已经用完，而由于 32 字节的静态内存的分配条件，此时剩余了一些 32 字节的静态内存，通过代码行 20~24 的功能，可以将其耗尽。内存容量实验的 ShowMemory 结果如右图的下部分所示，此时总内存和使用的内存大小相同，32 字节和 64 字节的静态内存也都已经分配完毕。

```

xiuos@xiuos-Inspiron-7790
xiuos@xiuos-Inspiron-7790
alloc memory [3404] ptr[3403] = 0x805f05f0
alloc memory [3405] ptr[3404] = 0x805fc350
alloc memory [3406] ptr[3405] = 0x805fd0b0
alloc memory [3407] ptr[3406] = 0x805fde10
alloc memory [3408] ptr[3407] = 0x805feb70
alloc memory [32] ptr[3408] = 0x80060780
alloc memory [33] ptr[3409] = 0x80064990
alloc memory [34] ptr[3410] = 0x805ff8d0
alloc memory [35] ptr[3411] = 0x805ff920
alloc memory [36] ptr[3412] = 0x805ff970
alloc memory [37] ptr[3413] = 0x805ff9c0
alloc memory [38] ptr[3414] = 0x805ffa10
alloc memory [39] ptr[3415] = 0x805ffa60
alloc memory [40] ptr[3416] = 0x805ffab0
alloc memory [41] ptr[3417] = 0x805ffb00
alloc memory [42] ptr[3418] = 0x805ffb50
alloc memory [43] ptr[3419] = 0x805ffbba0
alloc memory [44] ptr[3420] = 0x805ffbf0
alloc memory [45] ptr[3421] = 0x805ffc40
alloc memory [46] ptr[3422] = 0x805ffc90
alloc memory [47] ptr[3423] = 0x805ffce0
alloc memory [48] ptr[3424] = 0x805ffd30
alloc memory [49] ptr[3425] = 0x805ffd80
alloc memory [50] ptr[3426] = 0x805ffdd0
alloc memory [51] ptr[3427] = 0x805ffe20
alloc memory [52] ptr[3428] = 0x805ffe70
alloc memory [53] ptr[3429] = 0x805ffec0
alloc memory [54] ptr[3430] = 0x805fff10
alloc memory [55] ptr[3431] = 0x805fff60
alloc memory [56] ptr[3432] = 0x805fffb0
alloc memory [32] ptr[3433] = 0x800607b0
alloc memory [32] ptr[3434] = 0x800607e0
alloc memory [32] ptr[3435] = 0x80060810

xiuos@xiuos-Inspiron-7790
xiuos@xiuos-Inspiron-7790
alloc memory [52] ptr[3428] = 0x805ffe70
alloc memory [53] ptr[3429] = 0x805ffec0
alloc memory [54] ptr[3430] = 0x805fff10
alloc memory [55] ptr[3431] = 0x805fff60
alloc memory [56] ptr[3432] = 0x805fffb0
alloc memory [32] ptr[3433] = 0x800607b0
alloc memory [32] ptr[3434] = 0x800607e0
alloc memory [32] ptr[3435] = 0x80060810
alloc memory [32] ptr[3436] = 0x80060840
alloc memory [32] ptr[3437] = 0x80060870
alloc memory [32] ptr[3438] = 0x800608a0
alloc memory [32] ptr[3439] = 0x800608d0
alloc memory [32] ptr[3440] = 0x80060900
alloc memory [32] ptr[3441] = 0x80060930
alloc memory [32] ptr[3442] = 0x80060960
alloc memory [32] ptr[3443] = 0x80060990
alloc memory [32] ptr[3444] = 0x800609c0
alloc memory [32] ptr[3445] = 0x800609f0
alloc memory [32] ptr[3446] = 0x80060a20
alloc memory [32] ptr[3447] = 0x80060a50
alloc memory [32] ptr[3448] = 0x80060a80
alloc memory [32] ptr[3449] = 0x80060ab0
alloc memory [32] ptr[3450] = 0x80060ae0
alloc memory [32] ptr[3451] = 0x80060b10
alloc memory [32] ptr[3452] = 0x80060b40
alloc memory [32] ptr[3453] = 0x80060b70
alloc memory [32] ptr[3454] = 0x80060ba0
alloc memory [32] ptr[3455] = 0x80060bd0
alloc memory [32] ptr[3456] = 0x80060c00
BigMemMalloc: allocation failed, size 32.
total memory: 5898208
used memory : 5898208
total cache size: 4096, 0/64[32B],0/32[64B]

```

图 3-40-129 内存堆容量实验结果

2.4.3 内存堆读写实验

内存堆的读写实验过程如代码 3-2-3 所示，整个实验过程由 memReadWrite 函数完成。首先代码行 3 中申请一个大小为 size 的内存；如果内存申请成功，那么在代码行 5~8 中，将 raw_content 的内容通过内存拷贝 memcpy 写到申请到的内存中，并通过 printf 函数读出来。同理，代码行 10~13 将 new_content 的内容写到内存中，并输出显示出来。

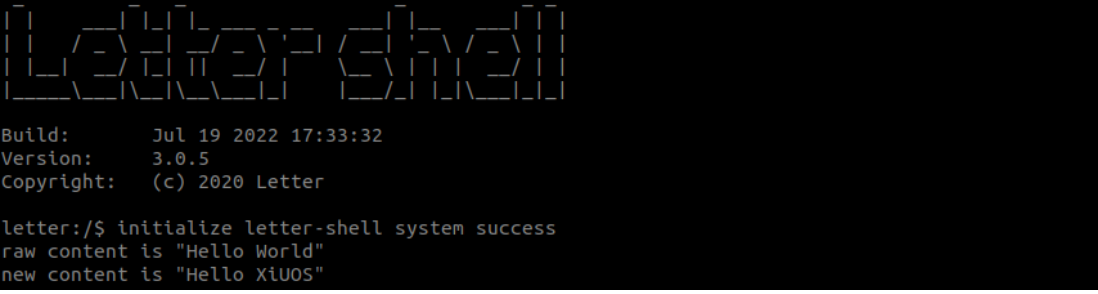
通过在 XiUOS/Applications/main.c 中执行 memReadWrite(16, "Hello world", "Hello XiUOS"), 运行结果如图 3-2-9 所示。可以看到，其结果显示正确，表明内存堆的读写成功。

代码 3-41-42 内存堆读写实验代码

```

01 void memReadWrite(x_size_t size, char *raw_content, char *new_content){
02     int endPos;
03     void *ptr = x_malloc(size);
04     if (ptr) {
05         memcpy(ptr,raw_content, size > strlen(raw_content) ? strlen(raw_content):
size);
06         endPos = size > strlen(raw_content) ? strlen(raw_content): size;
07         *((char*)ptr + endPos) = 0;
08         printf("raw content is \"%s\"\n",ptr);
09
10         memcpy(ptr,new_content, size > strlen(new_content)? strlen(new_content):
size);
11         endPos = size>strlen(new_content)? strlen(new_content): size;
12         *((char*)ptr + endPos) = 0;
13         printf("new content is \"%s\"\n",ptr);
14
15         x_free(ptr);
16     } else {
17         printf("x_malloc failure\n");
18     }
19 }

```



```

letter-shell
Build:      Jul 19 2022 17:33:32
Version:    3.0.5
Copyright:  (c) 2020 Letter

letter:/$ initialize letter-shell system success
raw content is "Hello World"
new content is "Hello XiUOS"

```

图 3-43-130 内存堆读写实验结果

2.4.4 内存池静态管理实验

内存池预先向系统申请若干数量、大小的内存块，当申请内存时，就将内存池中的内存块分配出去。内存池的管理分为静态内存管理和动态内存管理，两种内存管理的方式，使用的分配函数均不相同。

内存池的静态管理实验过程代码 3-2-4 所示。静态内存的管理方式是，先从内存中申请一个内存池，然后根据内存块的大小，将该内存池分为若干相同大小的块。后面申请的内存，从这些内存块中进行分配和使用。

代码行 1 引用了外部命令 ShowGatherMem，用于实验过程中显示内存池的状态。代码行 2~5 定义了若干变量，如代码行 2 定义了分配的块数量；代码行 3 定义了一个指针数组；代码行 4 定义了一个 1024 字节的内存池；代码行 5 定义了一个结构体变量，用于存放内存池对象。

内存池静态管理的过程如 StaticGatherMem 函数所示（代码行 7~36）。首先，代码行 9~10 中对指针数组进行初始化；代码行 12~14 中使用静态内存池管理的方式，通过 InitMemGather 函数将 1024 字节的内存池，按照每块大小 80 字节分割成若干块；代码行 16~22 主要使用 AllocBlockMemGather 函数申请 BLOCK_NUM 次内存块；代码行 25~31 中通过 FreeBlockMemGather 函数对申请到的内存块进行释放；最后，在代码行 34 处对申请的内存池进行删除。整个过程中多次调用 ShowGatherMem 函数，来显示内存池申请、分配、释放过程中的状态。

代码 3-44-45 内存池静态管理实验代码

```
01 extern long ShowGatherMem(void);
02 #define BLOCK_NUM 10
03 static uint8 *ptr[BLOCK_NUM];
04 static uint8 memPool[1024];
05 static struct MemGather gm_s;
06
07 void StaticGatherMem(){
08     int idx;
09     for (idx = 0; idx < BLOCK_NUM; idx ++){
10         ptr[idx] = NONE;
```

```

11
12     printf("Static create gatherblock.\n");
13     InitMemGather(&gm_s, "static", &memPool[0], sizeof(memPool), 80);
14     ShowGatherMem();
15
16     for (idx = 0 ; idx < BLOCK_NUM ; idx++){
17         if (ptr[idx] == NONE){
18             ptr[idx] = AllocBlockMemGather(&gm_s, WAITING_FOREVER);
19             if (ptr[idx] != NONE)
20                 printf(" Successflly allocate No.%d from static gatherblock\n", idx);
21         }
22     }
23
24     ShowGatherMem();
25     for (idx = 0; idx <BLOCK_NUM ; idx++){
26         if (ptr[idx] != NONE){
27             printf("release block %d\n", idx);
28             FreeBlockMemGather(ptr[idx]);
29             ptr[idx] = NONE;
30         }
31     }
32
33     ShowGatherMem();
34     RemoveMemGather(&gm_s);
35     return;
36 }

```

内存池的静态管理实验过程中的内存池状态如图 3-2-10 至 3-2-12 中所示。图 3-2-10 是内存池分块之后的结果，可以看到，分割的内存块大小 BLOCK 是代码行 13 中设置的 80 字节，总的块数是 11，目前空闲块数量也是 11。值得注意的是，静态内存池的大小是 1024，按 80 字节一块，可以分配出 12 块，而系统只分配了 11 个内存块。这是因为内存池中的 1024 字节，其中有一部分内存用于指针管理而占用，没有完全用于内存块，所以只能分配出 11 个内存块。图 3-2-11 是 10 次内存分配的过程以及分配后的内存池状态。可以看出，已经成功分配出 10 个内存，并且 FREE 标签下显示空闲的内存块从 11 变成了 1。图 3-2-12 是内存块的释放过程以及相应的内存池状态。可以看到，之前申请的 10 个内存块都能够内释放，而且空闲的内存块从 1 重新变成了 11。


```

Letter-shell
Build:      Jul 27 2022 09:01:17
Version:    3.0.5
Copyright:  (c) 2020 Letter

letter:/$ initialize letter-shell system success
Static create gatherblock.
*****
GATHERMEM BLOCK TOTAL FREE SUSPEND_TASK
static 80 11 11 NONE
*****

```

图 3-46-131 静态内存申请结果

```

Successfully allocate No.0 from static gatherblock
Successfully allocate No.1 from static gatherblock
Successfully allocate No.2 from static gatherblock
Successfully allocate No.3 from static gatherblock
Successfully allocate No.4 from static gatherblock
Successfully allocate No.5 from static gatherblock
Successfully allocate No.6 from static gatherblock
Successfully allocate No.7 from static gatherblock
Successfully allocate No.8 from static gatherblock
Successfully allocate No.9 from static gatherblock
*****
GATHERMEM BLOCK TOTAL FREE SUSPEND_TASK
static 80 11 1 NONE
*****

```

图 3-47-132 静态内存分配结果

```

release block 0
release block 1
release block 2
release block 3
release block 4
release block 5
release block 6
release block 7
release block 8
release block 9
*****
GATHERMEM BLOCK TOTAL FREE SUSPEND_TASK
static 80 11 11 NONE
*****

```

图 3-48-133 静态内存释放结果

2.4.5 内存池动态管理实验

内存池的动态管理实验过程代码 3-2-5 所示。不同于静态内存的管理方式，动态内存管理是根据用户申请的内存块数量和大小构建内存池。代码行 1 引用了外部命令 ShowGatherMem。代码行 2 定义了分配的块数量；代码行 3 定义了一个指针数组；代码行 4 定义了一个静态变量，用于存放内存池对象。

内存池动态管理的过程如 `DynamicGatherMem` 函数所示（代码行 6~38）。代码行 8~9 中对指针数组进行初始化；代码行 12 中使用动态内存池管理函数 `CreateMemGather`，创建一个内存池，共有 10 个内存块，每块大小为 80 字节；代码行 13~16 用于判断内存池是否创建成功；代码行 19~25 中主要通过 `AllocBlockMemGather` 函数申请 `BLOCK_NUM` 次内存块；代码行 28~34 中通过 `FreeBlockMemGather` 函数对申请到的内存块进行释放；最后，在代码行 37 处对申请的内存池进行删除。

代码 3-49-50 `DynamicGatherMem` 函数代码

```

01  extern long ShowGatherMem(void);
02  #define BLOCK_NUM 10
03  static uint8 *ptr[BLOCK_NUM];
04  static GatherMemType gm_d;
05
06  void DynamicGatherMem(){
07      int idx;
08      for (idx = 0; idx < BLOCK_NUM; idx ++){
09          ptr[idx] = NONE;
10
11          printf("Dynamically create gatherblock.\n");
12          gm_d = CreateMemGather("dynamic",10,80);
13          if(gm_d == NONE){
14              printf("%s: allocate failure.\n",__func__);
15              return;
16          }
17
18          ShowGatherMem();
19          for (idx = 0 ; idx < BLOCK_NUM ; idx++){
20              if (ptr[idx] == NONE){
21                  ptr[idx] = AllocBlockMemGather(gm_d, WAITING_FOREVER);
22                  if (ptr[idx] != NONE)
23                      printf("Successfly allocate No.%d from dynamic gatherblock\n", idx);
24              }
25          }
26
27          ShowGatherMem();
28          for (idx = 0; idx < BLOCK_NUM ; idx++){
29              if (ptr[idx] != NONE){
30                  printf("release block %d\n", idx);

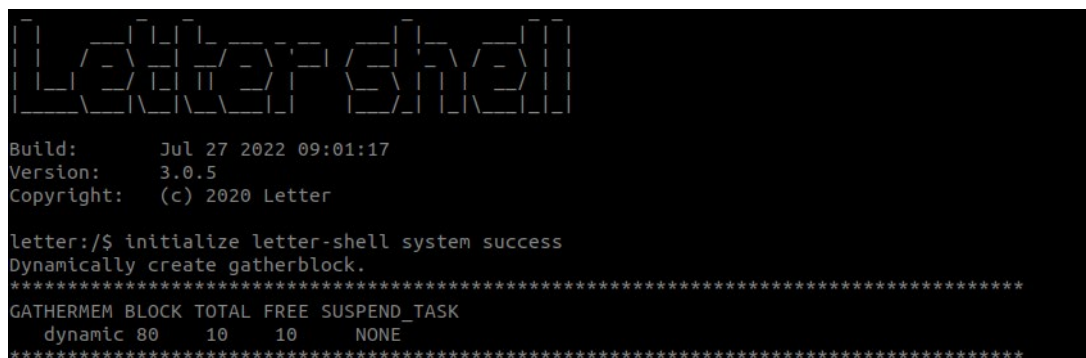
```

```

31      FreeBlockMemGather(ptr[idx]);
32      ptr[idx] = NONE;
33  }
34  }
35
36      ShowGatherMem();
37      DeleteMemGather(gm_d);
38  }

```

图 3-2-13 至 3-2-15 显示的是动态管理实验过程中的各阶段内存池状态。图 3-2-13 是内存池动态分配后的结果，实验在代码行 12 中申请了 10 块内存，每块大小 80 字节，那么动态内存分配中就直接做了相应的分配，且这 10 个内存块目前都处于空闲状态。图 3-2-14 是执行 BLOCK_NUM=10 次内存分配的过程以及分配后的内存池状态。可以看出，已经成功分配出 10 个内存，当前空闲的内存块数量变为了 0。图 3-2-15 是内存块的释放过程以及相应的内存池状态。内存块释放之后，空闲内存块数量重新变成了 10。



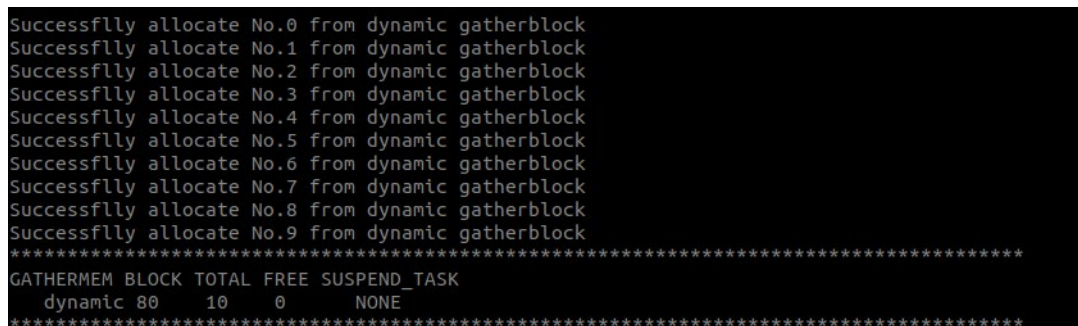
```

letter-shell
Build:      Jul 27 2022 09:01:17
Version:    3.0.5
Copyright:  (c) 2020 Letter

letter:/$ initialize letter-shell system success
Dynamically create gatherblock.
*****
GATHERMEM BLOCK TOTAL FREE SUSPEND_TASK
dynamic 80    10    10    NONE
*****

```

图 3-51-134 动态内存申请结果



```

Successfully allocate No.0 from dynamic gatherblock
Successfully allocate No.1 from dynamic gatherblock
Successfully allocate No.2 from dynamic gatherblock
Successfully allocate No.3 from dynamic gatherblock
Successfully allocate No.4 from dynamic gatherblock
Successfully allocate No.5 from dynamic gatherblock
Successfully allocate No.6 from dynamic gatherblock
Successfully allocate No.7 from dynamic gatherblock
Successfully allocate No.8 from dynamic gatherblock
Successfully allocate No.9 from dynamic gatherblock
*****
GATHERMEM BLOCK TOTAL FREE SUSPEND_TASK
dynamic 80    10    0    NONE
*****

```

图 3-52-135 动态内存分配结果

```

release block 0
release block 1
release block 2
release block 3
release block 4
release block 5
release block 6
release block 7
release block 8
release block 9
*****
GATHERMEM BLOCK TOTAL FREE SUSPEND_TASK
dynamic 80 10 10 NONE
*****

```

图 3-53-136 动态内存释放结果

第3章 中断管理及实验

3.1 中断管理介绍

中断是指计算机运行过程中，出现某些意外情况需要主机干预时，机器能自动停止正在运行的程序并转入处理新情况的程序，处理完毕后又返回原被暂停的程序继续运行。中断的触发一般是由外设或者 CPU 内部产生，由中断控制器进行处理再通知到 CPU 进行中断响应过程，其要点如下：

- 外设或者 CPU 内部产生的这个中断来源称为中断源；
- CPU 响应中断转入到的新的处理程序称为中断服务程序；
- 中断处理完成后，处理器需要恢复之前的现场，简称“恢复现场”；
- 当多个中断源请求中断的时候，需要通过优先级区分中断，便于 CPU 进行中断的处理；
- 低优先级可以被高优先级打断，这个过程称之为中断嵌套；
- 中断可以被屏蔽；
- 所有的中断源都有一个编号，称为“中断号”；
- 每一个中断号通过中断向量表与中断服务程序一一对应，中断向量表保存的是所有的中断服务程序的入口地址，该入口地址被称之为中断向量。

RISC-V 架构的中断和异常，从广义上来说，中断和异常属于一个概念。对于处理器而言，通常只区分为同步异常和异步异常。

- 同步异常，是指由于执行程序或者试图执行指令而产生的异常，例如，非法指令访问；
- 异步异常，最常见的异步异常是外部中断，例如外设触发一个外部中断。

RISC-V 架构定义的中断类型分为 4 种：

- 外部中断（External Interrupt），指来自处理器核外部的中断，例如 GPIO、UART 等产生的中断；
- 计时器中断（Timer Interrupt），指来自计时器的中断；
- 软件中断（Software Interrupt），指来自软件自己触发的中断；
- 调试中断（Debug Interrupt），专用于实现调试器（Debugger）。

RISC-V 中断控制器如图 3-3-1 所示。其中，提供两种中断控制器，即 CLINT 控制器和 PLIC 控制器。1）CLINT 模块生成计时器中断和软件中断，CLINT 的全称为处理器核局部中断控制器（Core Local Interrupts Controller），主要用于产生计时器中断（Timer Interrupt）和软件中断（Software Interrupt）；2）PLIC 管理多个外部中断，PLIC 全称为平台级别中断控制器（Platform Level Interrupt Controller），它是 RISC-V 架构标准定义的系统中断控制器，主要用于多个外部中断源的优先级仲裁。

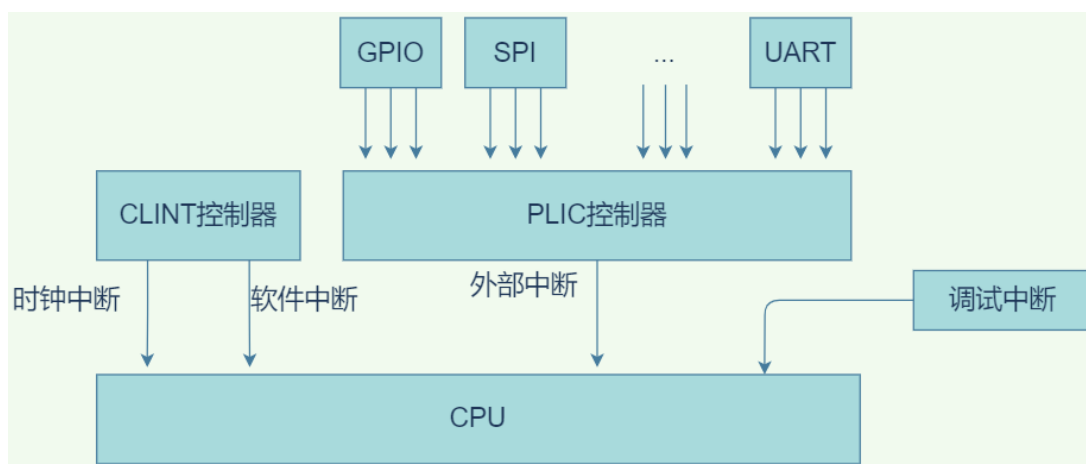


图 3-54-137 RISC-V 中断控制器

进入异常时，RISC-V 架构规定(以机器模式为例)：当前的程序执行流停止执行，直接跳转到 CSR 寄存器的 `mtvec` 定义的 PC 地址执行；硬件同时更新下列几个 CSR 寄存器。

- `mcause` (Machine Cause Register)：机器模式异常原因寄存
- `mepc` (Machine Exception Program Register)：机器模式异常 PC 寄存器
- `mtval` (Machine Trap Value Register)：机器模式异常值寄存器
- `mstatus` (Machine Status Register)：机器模式状态寄存器

`mcause` 寄存器的 Exception Code 域标识是何种异常或者何种中断。定义如表 3-3-1 所示：

表 3-55-138 Exception Code 域标识定义

Interrupt/Exception mcause[XLEN-1]	Exception Code mcause[XLEN-2:0]	Description
1	1	Supervisor software interrupt
1	3	Machine software interrupt
1	5	Supervisor timer interrupt
1	7	Machine timer interrupt
1	9	Supervisor external interrupt
1	11	Machine external interrupt
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	store address misaligned
0	7	Store access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	11	Environment call from M-mode
0	12	Instruction page fault
0	13	Load page fault
0	15	Store page fault

退出异常时，需要从异常服务程序中退出，并返回主程序。RISC-V 架构定义了一组专门的退出异常指令：MRET、SRET、URET，分别对应机器模式、监督模式、用户模式的退出。以 MRET 为例，当处理器执行 MRET 指令后，硬件行为如下：1) 当前程序执行流程停止执行，跳转到 mepc 的地址运行；2) 更新 mstatus 状态寄存器。

3.2 中断管理接口

XiUOS 在 IsrDone 结构体中定义了一系列中断函数，对外接口均由 isrManager 实现：

- isrManager.done->registerIrq(uint32 irq_num, IsrHandlerType handler, void *arg)，该函数用于注册一个中断，当产生中断时，将调用该硬件中断号相应的回调函数进行执行。

其中，参数 `irq_num` 表示硬件中断号；`handler` 表示中断处理回调函数；`arg` 表示中断处理回调函数的参数。

- `isrManager.done->freeIrq(uint32 irq_num)`，该函数用于释放一个中断。其中，参数 `irq_num` 表示硬件中断号。
- `isrManager.done->disableIrq(uint32 irq_num)`，该函数用于屏蔽一个中断。其中，参数 `irq_num` 表示硬件中断号。
- `isrManager.done->enableIrq(uint32 irq_num)`，该函数用于使能一个中断。其中，参数 `irq_num` 表示硬件中断号。
- `isrManager.done->getCounter()`，该函数用于统计进入中断的次数。

3.3 中断管理实验

中断管理过程中的函数调用过程如代码 3-3-1 所示。代码行 1 定义了一个中断号；代码行 3~5 定义了一个中断处理函数；代码行 7~13 是中断管理函数的调用方法，如代码行 9 是中断注册函数，中断处理函数 `IrqHandler` 作为了其中一个参数；代码行 10 用于使能该中断；代码行 11 用于屏蔽该中断；代码行 12 用于释放该中断。

需要解释的是，代码 3-3-1 是不能够编译运行的。中断功能的使用需要结合实际应用，所以在代码 3-3-1 中只做了相应的中断函数的调用方法。具体的中断管理和使用方法，我们将在第 10 章的驱动管理中进行详细的描述和直观的应用。

代码 3-56-57 中断管理实验代码

```

01  #define IRQ_NUM    100    /*define a IRQ number */
02
03  void IrqHandler(void *param){
04      do_something();    /* irq process */
05  }
06
07  void  irq_demo(){
08      void * param;
09      isrManager.done->registerIrq(IRQ_NUM, IrqHandler, param);
10      isrManager.done->enableIrq(IRQ_NUM);
11      isrManager.done->disableIrq(IRQ_NUM);
12      isrManager.done->freeIrq(IRQ_NUM);
13  }

```

(1) 基于 ARM 开发板的中断响应性能测试：

本实验中实际选择 PC11 作为外部中断的输入引脚，PD01 引脚作为回调函数的输出引脚。因而应当将开发板的 GND 与示波器的 GND 连接，PC11 连接至示波器的校准信号输出口，使用示波器双通道分别测量示波器的校准信号输出口和 PD01 输出引脚。通过测量校准信号方波的上升或下降沿与 PD01 的脉冲输出间的延迟来计算中断响应时间。

实验中的具体配置可见代码 3-3-2。代码行 1~5 和代码行 8~10 分别声明了与 GPIO 引脚配置参数和总线相关的结构体类型变量；代码 12~17 对总线配置接口的结构体变量进行了初始化；代码行 19~33 是将 PD1 引脚设置为输出模式，同时配置其输出电平为低电平。代码行 35~44 将 PC11 设置为输入模式；代码行 46~56 将 PC11 的触发模式配置为双边沿触发，并将 PinIrqIsr 配置为回调函数指针；代码行 58~65 对 PC11 外部中断进行了使能。

代码 3-58-59 中断配置代码

```

01  int RealtimeIrqTest()
02  {
03      struct PinParam testpin_1;
04      struct PinStat testpin_1_stat;
05      struct PinParam testpin_2;
06      int ret = 0;
07
08      struct BusConfigureInfo configure_info_1;
09      struct BusConfigureInfo configure_info_2;
10      struct BusBlockWriteParam write_param_1;
11
12      configure_info_1.configure_cmd = OPE_CFG;
13      configure_info_1.private_data = (void *)&testpin_1;
14      write_param_1.buffer = (void *)&testpin_1_stat;
15
16      configure_info_2.configure_cmd = OPE_CFG;
17      configure_info_2.private_data = (void *)&testpin_2;
18
19      /* config test pin 1 as output*/
20      testpin_1.cmd = GPIO_CONFIG_MODE;
21      testpin_1.pin = GPIO_D1;
22      testpin_1.mode = GPIO_CFG_OUTPUT;
23
24      ret = BusDrvConfigure(pin->owner_driver, &configure_info_1);

```

```
25     if (ret != EOK) {
26         KPrintf("config testpin_1   %d failed!\n", GPIO_D1);
27         return -ERROR;
28     }
29
30     /* set test pin 1 as high*/
31     testpin_1_stat.pin = GPIO_D1;
32     testpin_1_stat.val = GPIO_LOW;
33     BusDevWriteData(pin->owner_haldev, &write_param_1);
34
35     /* config test pin 2 as input*/
36     testpin_2.cmd = GPIO_CONFIG_MODE;
37     testpin_2.pin = GPIO_C11;
38     testpin_2.mode = GPIO_CFG_INPUT;
39
40     ret = BusDrvConfigure(pin->owner_driver, &configure_info_2);
41     if (ret != EOK) {
42         KPrintf("config testpin_2   %d input failed!\n", testpin_2.pin);
43         return -ERROR;
44     }
45
46     testpin_2.cmd = GPIO_IRQ_REGISTER;
47     testpin_2.pin = GPIO_C11;
48     testpin_2.irq_set.irq_mode = GPIO_IRQ_EDGE_BOTH;
49     testpin_2.irq_set.hdr = (void*)(void *)PinIrqIsr;
50     testpin_2.irq_set.args = NONE;
51
52     ret = BusDrvConfigure(pin->owner_driver, &configure_info_2);
53     if (ret != EOK) {
54         KPrintf("register testpin_2   %d   irq failed!\n", testpin_2.pin);
55         return -ERROR;
56     }
57
58     testpin_2.cmd = GPIO_IRQ_ENABLE;
59     testpin_2.pin = GPIO_C11;
60
61     ret = BusDrvConfigure(pin->owner_driver, &configure_info_2);
62     if (ret != EOK) {
63         KPrintf("enable testpin_2   %d   irq failed!\n", testpin_2.pin);
64         return -ERROR;
65     }
```

```

66
67     return 0;
68 }

```

PinIrqlsr 的中断函数可见代码 3-3-3。其主要功能为依次将 PD1 引脚拉高和拉低。为了减少回调函数中的代码运行时间开销，因而采用汇编代码实现。通过查询 HC32F4A0 芯片的用户手册，可以分别查找到 GPIO Port D 对应的通用置位寄存器(POSRx)地址为 0x40053838。通过 LDR、MOV、STR 汇编指令分别将 POSRD 寄存器的第 2 位置 1 即可使 PD1 引脚输出为高电平。对应的通用置位寄存器(PORRx)地址为 0x4005383A，再通过 LDR、MOV、STR 汇编指令分别将 PORRD 寄存器的第 2 位置 1 即可使 PD1 引脚输出为低电平。

代码 3-60-61 IO 电平控制代码

```

01 void PinIrqlsr(int vector,void *args)
02 {
03     /* 将 GPIO D1 置为高电平 */
04     asm volatile("LDR r2,=0x40053838"); // 测试代码
05     asm volatile("MOV r3,#0x0002");    // 测试代码
06     asm volatile("STR r3,[r2]");        // 测试代码
07
08     /* 将 GPIO D1 置为低电平 */
09     asm volatile("LDR r2,=0x4005383A"); // 测试代码
10     asm volatile("MOV r3,#0x0002");    // 测试代码
11     asm volatile("STR r3,[r2]");        // 测试代码
12 }

```

由于示波器的校准信号将重复触发外部中断，进而使 PD1 产生周期性的脉冲信号。将开发板的相关引脚与示波器对应连接完成后，对示波器的水平轴、垂直轴和触发电平等进行合适的设置，可以得到如图 3-3-2 所示的结果。从示波器的测试结果上来看，基于 XIUOS 的 HC32F4A0 开发板的中断响应时间在 1.02us 以内。

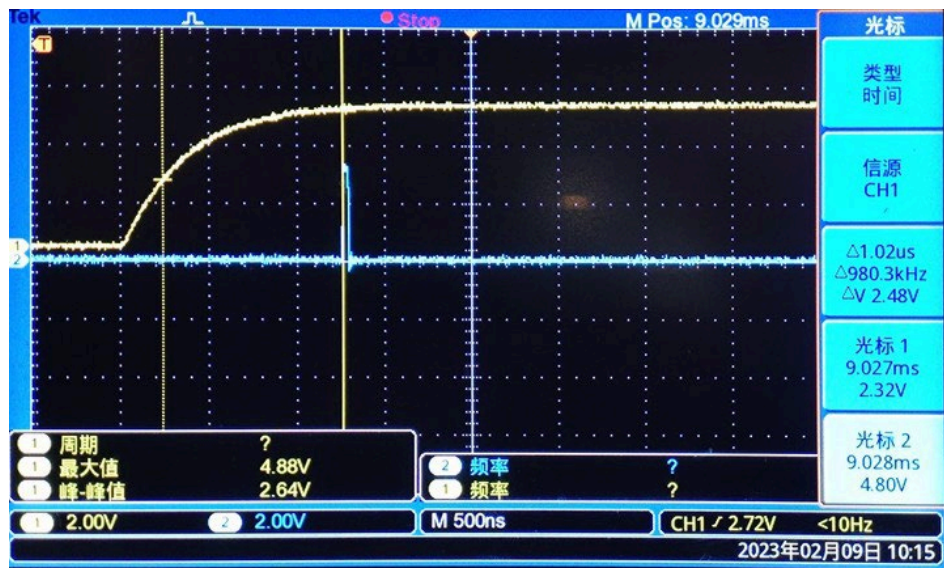


图 3-62-139 HC32F4A0 中断响应时间测量

(2) 基于 RISC-V 开发板的中断响应性能测试:

本实验中实际选择 IO35 作为外部中断的输入引脚, IO34 引脚作为回调函数的输出引脚。因而应当将开发板的 GND 与示波器的 GND 连接, IO35 连接至示波器的校准信号输出口, 使用示波器双通道分别测量示波器的校准信号输出口和 IO34 输出引脚。通过测量校准信号方波的上升或下降沿与 IO34 的脉冲输出间的延迟来计算中断响应时间。同时, 由于 K210 芯片的 FPIO 外设, 实验中将 IO34 分别映射为 GPIOHS5 和 GPIOHS6。

实验的具体实现代码可见代码 3-3-4,其具体逻辑与 ARM 开发板类似。代码行 3~5 和代码 7~9 分别声明了 GPIO 引脚参数和总线配置的结构体变量; 代码行 11~16 对总线配置的相关参数进行了配置。代码行 18~26 将 IO34 引脚配置为输出引脚并设置电平为低电平。代码行 28~43 将 IO35 配置引脚为输入引脚并使能中断源, 同时配置回调函数指针为 PinIrqIsr。

代码 3-63-64 中断响应测试硬件配置

```
01 int RealtimeIrqTest()
02 {
03     struct PinParam testpin_1;
04     struct PinStat testpin_1_stat;
05     struct PinParam testpin_2;
06
07     struct BusConfigureInfo configure_info_1;
08     struct BusConfigureInfo configure_info_2;
```

```
09     struct BusBlockWriteParam write_param_1;
10
11     configure_info_1.configure_cmd = OPE_CFG;
12     configure_info_1.private_data = (void *)&testpin_1;
13     write_param_1.buffer = (void *)&testpin_1_stat;
14
15     configure_info_2.configure_cmd = OPE_CFG;
16     configure_info_2.private_data = (void *)&testpin_2;
17
18     /* config GPIO18 as output and set as low */
19     testpin_1.cmd = GPIO_CONFIG_MODE;
20     testpin_1.pin = GPIO_34;
21     testpin_1.mode = GPIO_CFG_OUTPUT;
22     BusDrvConfigure(pin->owner_driver, &configure_info_1);
23
24     testpin_1_stat.pin = GPIO_34;
25     testpin_1_stat.val = GPIO_LOW;
26     BusDevWriteData(pin->owner_haldev, &write_param_1);
27
28     /* config GPIO18 as input */
29     testpin_2.cmd = GPIO_CONFIG_MODE;
30     testpin_2.pin = GPIO_35;
31     testpin_2.mode = GPIO_CFG_INPUT;
32     BusDrvConfigure(pin->owner_driver, &configure_info_2);
33
34     testpin_2.cmd = GPIO_IRQ_REGISTER;
35     testpin_2.pin = GPIO_35;
36     testpin_2.irq_set.irq_mode = GPIO_IRQ_EDGE_RISING;
37     testpin_2.irq_set.hdr = PinIrqIsr;
38     testpin_2.irq_set.args = NONE;
39     BusDrvConfigure(pin->owner_driver, &configure_info_2);
40
41     testpin_2.cmd = GPIO_IRQ_ENABLE;
42     testpin_2.pin = GPIO_35;
43     BusDrvConfigure(pin->owner_driver, &configure_info_2);
44
45     return 0;
46 }
```

PinIrqIsr 的中断函数可见代码 3-3-5。其主要功能为依次将 IO34 引脚拉高和拉低。为了减少回调函数中的代码运行时间开销，因而采用汇编代码实现。K210 实现 GPIOHS 电平控

制的对应寄存器地址为 0x3800100C。由于 IO34 通过 FPIO 映射到了 GPIOHS5，通过 0x3800100C 对应的寄存器与 0x00000005 进行或运算，即可配置 IO34 为高电平；通过 0x3800100C 对应的寄存器与 0x00000005 的反码进行与运算，即可配置 IO34 为低电平。

代码 3-65-66 外部中断回调函数

```

01 void PinIrqIsr(void *args)
02 {
03     /* 将 IO34 (GPIO5) 置为高电平 */
04     asm volatile ("lui a5, 0x38001"); // 测试代码
05     asm volatile ("addi a5, a5, 12"); // 测试代码
06     asm volatile ("lw a5, 0(a5)"); // 测试代码
07     asm volatile ("sext.w a4, a5"); // 测试代码
08     asm volatile ("lui a5, 0x38001"); // 测试代码
09     asm volatile ("addi a5, a5, 12"); // 测试代码
10     asm volatile ("ori a4, a4, 5"); // 测试代码
11     asm volatile ("sext.w a4, a4"); // 测试代码
12     asm volatile ("sw a4, 0(a5)"); // 测试代码
13
14     /* 将 IO34 (GPIO5) 置为低电平 */
15     asm volatile ("lui a5, 0x38001"); // 测试代码
16     asm volatile ("addi a5, a5, 12"); // 测试代码
17     asm volatile ("lw a5, 0(a5)"); // 测试代码
18     asm volatile ("sext.w a4, a5"); // 测试代码
19     asm volatile ("lui a5, 0x38001"); // 测试代码
20     asm volatile ("addi a5, a5, 12"); // 测试代码
21     asm volatile ("andi a4, a4, -6"); // 测试代码
22     asm volatile ("sext.w a4, a4"); // 测试代码
23     asm volatile ("sw a4, 0(a5)"); // 测试代码
24 }

```

由于示波器的校准信号将重复触发外部中断，进而使 IO34 产生周期性的脉冲信号。将开发板的相关引脚与示波器对应连接完成后，对示波器的水平轴、垂直轴和触发电平等进行合适的设置，可以得到如图 3-3-3 所示的结果。从示波器的测试结果上来看，基于 XIUOS 的 K210 开发板的中断响应时间在 1.80us 以内。



图 3-67-140 edu-riscv64 中断响应时间测量

第4章 节拍机制及实验

4.1 节拍机制介绍

每一个操作系统都需要一个时钟计数，提供“心跳”计数，该时钟可以提供系统处理所有和时钟相关的事件。XiUOS 的“心跳”通过芯片提供的硬件定时器产生的周期性中断进行计数，XiUOS 可以在 menuconfig 界面进行配置，配置单位为毫秒。配置界面如下：

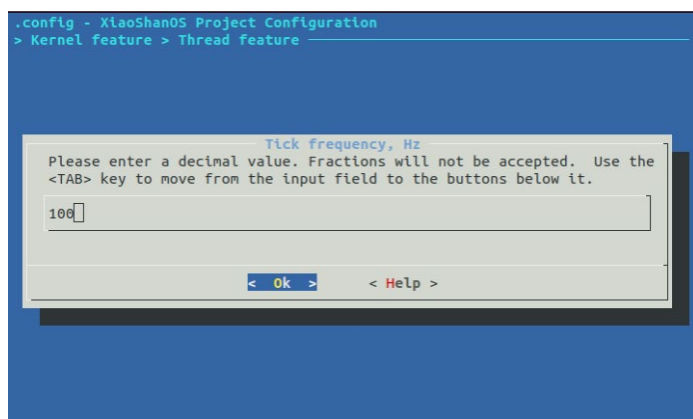


图 3-68-141 节拍计数配置界面

界面中的数值将配置给变量 `TICK_PER_SECOND`，该变量解释为每秒钟支持的节拍数（tick）。当 `TICK_PER_SECOND` 配置为 100，则 tick 节拍表示 10 毫秒，即一个时间片（timeslice）为 10 毫秒。在节拍配置成功后，系统支持的调度算法、软件定时器等依赖于该时钟周期服务的事件就可以正常工作了。例如，在 XiUOS 支持的时间片轮转算法当中，每进行的一次线程切换为 10 个 tick。

在每一个定时器中断到来时，系统进行常规的中断执行流，定时器按照每一个 tick 单位进行触发中断，在中断服务程序当中进行自增，并对当前运行线程的时间片进行计算，同步可处理定时器的计数时间。节拍计数的详细执行流程如下图所示：

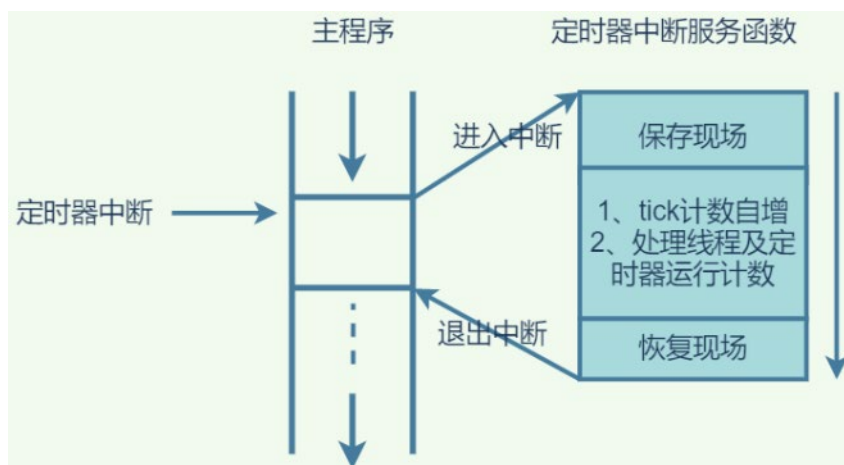


图 3-69-142 节拍计数的详细执行流程

4.2 节拍设置接口

节拍设置的函数接口包括：

- `x_ticks_t CalculteTickFromTimeMs(uint32 ms)`，该函数用于获取毫秒时间转化成节拍数。其中，参数 `ms` 表示毫秒时间。
- `uint32 CalculteTimeMsFromTick(x_ticks_t ticks)`，该函数用于获取节拍数转换成毫秒时间。其中，参数 `ticks` 表示节拍数。
- `x_ticks_t CurrentTicksGain(void)`，该函数用于获取当前的时钟节拍数。

另外，内核软件定时器模块根据系统的节拍心跳（tick）提供软件定时服务，用户可以设定固定的超时时间，当系统运行的节拍数到达用户设定的超时时间时，便执行用户定义的超时回调函数进行业务处理。软件定时器的存在可以解决硬件定时器数量不足的问题。

内核软件定时器结构定义及成员含义如下所示

代码 3-70-71 软件定时器结构定义

```

struct Timer {
    struct IdNode      id_node;          //软件定时器 ID，标识唯一的软件定时器
    char              name[NAME_NUM_MAX]; //软件定时器名称
    uint8             active_status;      //定时器的激活状态
    uint8             trigger_mode;       //定时器的触发模式
    void (*func_callback)(void *param);   //定时器超时回调函数
    void              *param;             //定时器超时回调函数的参数
    x_ticks_t         origin_timeslice;    //初始化的超时时间
    x_ticks_t         deadline_timeslice;  //截止时间
    DoubleLinklistType link;              //系统中所有定时器构成的链表
    DoubleLinklistType sortlist;          //处于激活态的定时器链表，并按时间排序
    uint8             prio;               //记录使用定时器处理回调的线程优先级
    struct Work        *t_work;           //定时器所用到的工作队列
    struct TimerDone    *done;            //定时器的函数操作集
};
    
```

另外，XiUOS 中还定义一系列消息队列函数接口：

- `int32 KCreateTimer(const char *name, void (*timeout)(void *parameter), void *parameter, x_ticks_t time, uint8 trigger_mode)`，该函数用于创建一个内核软件定时器，并返回创建成功的软件定时器的 ID，ID 默认范围 0-255，可配置。其中，参数 `name` 表示定时器名称；`timeout` 表示软件定时器回调函数；`parameter` 表示软件定时器回调函数参数；`time` 表示配置需要等待的超时时间，单位 tick；`trigger_mod` 表示触发方式，可以配置为宏 `TRIGGE_WAY_ONCE` 和 `TRIGGE_WAY_PERIODIC`。
- `void KTimerDelete(int32 timer_id)`，该函数用于删除一个软件定时器。其中，参数 `timer_id` 表示待删除的软件定时器 ID。
- `x_err_t KTimerStartRun(int32 timer_id)`，该函数用于启动一个软件定时器。其中，参数 `timer_id` 表示已创建且待运行的软件定时器 ID。

- `x_err_t KTimerQuitRun(int32 timer_id)`, 该函数用于停止一个软件定时器。其中, 参数 `timer_id` 表示待停止运行的软件定时器 ID。
- `x_err_t KTimerModify(int32 timer_id, x_tick_t ticks)`, 该函数用于修改一个软件定时器的超时时间。其中, 参数 `timer_id` 表示待修改超时时间的软件定时器 ID; `ticks` 表示超时时间。

4.3 节拍机制实验

节拍机制通过定时器来实现, 定时器的实验如代码 3-4-2 所示。代码行 1 引用了外部命令 `ShowTimer`, 用实验过程中查看定时器的状态。代码行 2~4 定义了一些基本的整形变量。代码行 6~14 和 16~21 分别定义了两个定时器实体函数。其中, `timerEntry1` 函数会根据定时器的设置, 周期性地打印信息, 在该实验中一共循环打印 10 次; 而 `timerEntry2` 函数只打印一次信息。这两个函数完成打印信息之后, 通过 `KTimerQuitRun` 和 `KDeleteTimer` 函数退出运行并删除定时器。

在执行函数 `softwareTimer` 中的代码行 24 处创建一个定时器 `timer1`。其中, `timerEntry1` 作为其中的回调函数; 定时设置为 9 个 ticks; 定时器触发模式为 `TIMER_TRIGGER_PERIODIC`, 即周期性触发。所以在函数 `timerEntry1` 中, 打印信息会被按固定时间周期性触发, 直到 `timer_cnt` 为 10 时才退出定时器运行。代码行 25~26 首先对创建的定时器 `timer1` 的是否创建成功进行判断, 如果成功, 则开始运行。

同理, 代码行 28 创建了定时器 `timer2`, `timerEntry2` 作为其中的回调函数; 定时设置为 100 个 ticks; 定时器触发模式为 `TIMER_TRIGGER_ONCE`, 即单次触发。根据定时设置可知, 在 `timer1` 循环执行 10 次之后, `timer2` 才触发执行。理所当然的是, `timer2` 的 ticks 可以任意设置, 此处这样设置完全是为了显示清晰的实验结果。代码行 29~30 对创建的定时器 `timer2` 的是否创建成功进行判断, 如果成功, 则开始运行。最后, 在代码行 32 使用 `ShowTimer` 命令显示当前定时器的状态。

代码 3-72-73 定时器实验代码

```
01 extern long ShowTimer(void);
02 static int32 timer1_id;
03 static int32 timer2_id;
04 static int timer_cnt = 0;
05
```

```

06 static void timerEntry1(void *parameter){
07     printf("Periodic timer is timeout %d\n", timer_cnt);
08
09     if (timer_cnt++>= 9){
10         KTimerQuitRun(timer1_id);
11         KDeleteTimer(timer1_id);
12         printf("Periodic timer was stopped! \n");
13     }
14 }
15
16 static void timerEntry2(void *parameter){
17     printf("Once timer is timeout\n");
18     KTimerQuitRun(timer2_id);
19     KDeleteTimer(timer2_id);
20     printf("Once timer was stopped! \n");
21 }
22
23 int softwareTimer(void){
24     timer1_id = KCreateTimer("timer1", timerEntry1,
NONE, 9, TIMER_TRIGGER_PERIODIC);
25     if (timer1_id >= 0)
26         KTimerStartRun(timer1_id);
27
28     timer2_id = KCreateTimer("timer2", timerEntry2,
NONE, 100, TIMER_TRIGGER_ONCE);
29     if (timer2_id >= 0)
30         KTimerStartRun(timer2_id);
31
32     ShowTimer();
33     return 0;
34 }

```

定时器实验结果如图 3-4-3 所示。首先可以看到，定时器 timer1 和 timer2 都已经创建成功，分配的 ID 分别为 0 和 1；两者的状态均为 active；定时周期分别为 9 和 100，是实验代码中的设置；timer1 和 timer2 的截止时间 DEADLINE 分别是 122 和 31，因为系统启动后的 ticks 是 22，对应与 PERIODIC 字段进行相加。另外，周期性定时器被调用了 10 次，而一次性定时器被调用了 1 次。并通过其它打印信息可知，创建的定时器可成功退出运行和删除。

```

Letter shell

Build:      Jul 28 2022 09:35:00
Version:    3.0.5
Copyright:  (c) 2020 Letter

letter:/$ initialize letter-shell system success
*****
TIMER_ID  NAME    STATUS  PERIODIC  DEADLINE
  1      timer2  active    100      122
  0      timer1  active     9       31
*****
Periodic timer is timeout 0
Periodic timer is timeout 1
Periodic timer is timeout 2
Periodic timer is timeout 3
Periodic timer is timeout 4
Periodic timer is timeout 5
Periodic timer is timeout 6
Periodic timer is timeout 7
Periodic timer is timeout 8
Periodic timer is timeout 9
Periodic timer was stopped!
Once timer is timeout
Once timer was stopped!

```

图 3-74-143 定时器实验结果

第5章 文件系统及实验

5.1 文件系统介绍

当前，Linux 和其他 RTOS 支持 Ext2，Ext3，Ext4，XFS，BtrFS，F2FS，RAMFS 等多种文件系统，因存储设备的硬件特征及其不同的应用场合，用户可以选择不同的文件系统。虚拟文件系统 VFS 的引入，使得这些不同的文件系统可以向外提供统一的编程接口和操作界面。

在物联网领域，主要使用的存储设备为 FLASH，所以可以选择 Jffs2 等适合 FLASH 特性的日志型文件系统；FATFS 由于其同时兼容 Linux 和 Windows，选择其作为文件系统便于在“端”、“边”、“云”之间以统一的格式交互和保存数据。Ext4 作为业内用户使用较多的文件系统，也可以将其轻量级的版本应用于物联网领域。

XiUOS 研发计划支持 FATFS，Jffs2，LWext4 三种文件系统，目前 XiUOS 1.0 版本只支持 FATFS。为了增强兼容性和支持 VFS 的特性，XiUOS 通过设计 IoT-VFS 的构件，为用户提供树形的文件组织结构。当前版本的 IoT-VFS 在功能上保证了 FATFS 的 VFS 特性，同时允许用户使用统一的 VFS 的接口来操作设备。IoT-VFS 具有以下特性：

- 统一采用 UNIX 风格路径格式，目录与文件层级清晰分明；

- 面向用户提供 POSIX 文件接口，降低应用程序移植难度；
- 可以在任意路径挂载不同文件系统，并且可以便捷地添加对新的文件系统的支持。

在 IoT-VFS 的支持下，应用程序开发者无须了解底层文件系统的接口细节，只需使用标准 POSIX 文件接口进行应用开发。对于已有的 POSIX 应用程序，也可以直接进行源代码级别的移植。使用 IoT-VFS 的应用程序架构如图 3-5-1 所示。

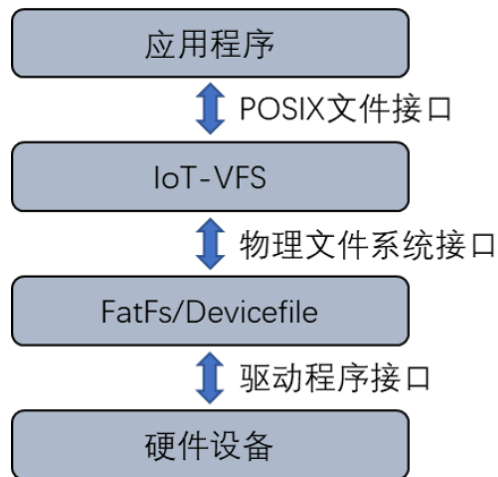


图 3-75-144 IoT-VFS 的应用程序架构

5.2 文件系统接口

XiUOS 中定义 2 个文件系统函数接口：

`int xs_MountFilesystem(const char *device_name, enum xs_FilesystemType fs_type, const char *path)`，该函数用于将指定设备上的物理文件系统挂载到指定路径，若挂载成功返回 0，失败返回对应错误码。其中，参数 `device_name` 表示待挂载的设备名；`fs_type` 表示设备上的物理文件系统类型，目前只支持 `FSTYPE_FATFS`；`path` 表示挂载路径。

`int xs_UnmountFileSystem(const char *path)`，该函数用于卸载指定路径上挂载的文件系统，若卸载成功返回 0，失败返回对应错误码。其中，参数 `path` 表示待卸载的路径。

另外，XiUOS 中定义了一系列 POSIX 文件接口：

- `int open(const char *path, int flags, ...)`，该函数用于打开一个文件。若文件打开成功则返回该文件的描述符（非负），后续读写该文件由该描述符标识，若不成功则返回 -1，此时 `errno` 会被置为对应错误码。在打开文件时可以指定打开选项，即 `flags` 参数。可选

的选项有打开时截断文件（O_TRUNC）和向末尾写入（O_APPEND）。其中，参数 `path` 表示打开文件的路径；`flags` 表示打开选项。

- `int close(int fd)`，该函数用于关闭一个已打开的文件。若文件关闭成功则返回 0，若不成功则返回-1，此时 `errno` 会被置为对应错误码。其中，参数 `fd` 表示待关闭文件描述符。
- `int read(int fd, void *buf, size_t len)`，该函数用于从文件读取数据。若读取成功则返回读取到的字节数，若不成功则返回-1，此时 `errno` 会被置为对应错误码。其中，参数 `fd` 表示待读取文件描述符；`buf` 表示存放读取数据的缓冲区首地址；`len` 表示缓冲区长度，单位为字节。
- `int write(int fd, const void *buf, size_t len)`，该函数用于向文件写入数据。若写入成功则返回读取到的字节数，若不成功则返回-1，此时 `errno` 会被置为对应错误码。其中，参数 `fd` 表示待写入文件描述符；`buf` 存放写入数据的缓冲区首地址；`len` 表示缓冲区长度，单位为字节。
- `off_t lseek(int fd, off_t offset, int whence)`，该函数用于重定位文件读写指针。若重定位成功则返回新的读写指针位置，即距离文件起始处的字节数，若不成功则返回-1，此时 `errno` 会被置为对应错误码。其中，参数 `fd` 表示待重定位的文件描述符；`offset` 表示新的文件读写指针位置；`whence` 表示 `offset` 参数的参照位置，可以为文件起始（SEEK_SET）、当前位置（SEEK_CUR）、文件末尾（SEEK_END）。
- `int rename(const char *from, const char *to)`，该函数用于重命名文件。若重命名成功则返回 0，若失败则返回-1，此时 `errno` 会被置为对应错误码。其中，参数 `from` 表示原文件路径；`to` 表示新文件路径，若与原路径属于不同目录则文件会被移动。
- `int unlink(const char *path)`，该函数用于删除一个文件。若删除成功则返回 0，若失败则返回-1，此时 `errno` 会被置为对应错误码。其中，参数 `path` 表示待删除文件路径。
- `int stat(const char *path, struct stat *buf)`，该函数用于获取文件的元数据。`stat` 结构与 POSIX 定义相同。若获取成功则返回 0，若失败则返回-1，此时 `errno` 会被置为对应错误码。其中，参数 `path` 表示待获取元数据的文件路径；`buf` 表示一个 `stat` 结构实例指针，用于存储元数据。
- `int fstat(int fd, struct stat *buf)`，该函数用于获取文件的元数据。`stat` 结构与 POSIX 定义相同。若获取成功则返回 0，若失败则返回-1，此时 `errno` 会被置为对应错误码。其

中，参数 `fd` 表示待获取元数据的文件描述符；`buf` 表示一个 `stat` 结构实例指针，用于存储元数据。

- `int fsync(int fd)`，该函数用于将指定文件数据写回存储设备，从而防止系统掉电数据丢失。若写回成功则返回 0，失败则返回-1，此时 `errno` 会被置为对应错误码。其中，参数 `fd` 表示待写回文件描述符。
- `int ftruncate(int fd, off_t length)`，该函数用于将文件截断至指定长度。若截断成功则返回 0，失败则返回-1，此时 `errno` 会被置为对应错误码。其中，参数 `fd` 表示待截断文件描述符；`length` 表示截断后的长度。
- `int mkdir(const char *path, mode_t mode)`，该函数用于创建一个目录。若创建成功则返回 0，若失败则返回-1，此时 `errno` 会被置为对应错误码。其中，参数 `path` 待创建目录的路径；`mode` 目前该参数无用。
- `DIR *opendir(const char *path)`，该函数用于打开一个目录。若打开成功则返回一个目录句柄，若失败则返回 `XS_NULL`。其中，参数 `path` 表示待打开目录的路径。
- `int closedir(DIR *dirp)`，该函数用于关闭一个已打开的目录。若关闭成功则返回 0，若失败则返回-1，此时 `errno` 会被置为对应错误码。其中，参数 `dirp` 表示待关闭目录句柄。
- `struct dirent *readdir(DIR *dirp)`，该函数用于读取目录的目录项。目录项由 `dirent` 结构表示，目前该结构仅包含文件或目录的类型（`d_kind`）与名称（`d_name`）。若读取成功则返回一个 `dirent` 指针，若读取失败则返回 `XS_NULL`，此时 `errno` 会被置为对应错误码。其中，参数 `dirp` 表示待读取目录句柄。
- `int rmdir(const char *path)`，该函数用于删除一个目录。若删除成功则返回 0，若失败则返回-1，此时 `errno` 会被置为对应错误码。其中，参数 `path` 表示待删除目录的路径。
- `int chdir(const char *path)`，该函数用于切换当前工作目录。若切换成功则返回 0，若失败则返回-1，此时 `errno` 会被置为对应错误码。其中，参数 `path` 表示新的工作目录路径。
- `char *getcwd(char *buf, size_t size)`，该函数用于获取当前的工作目录。若获取成功则返回工作目录字符串（与 `buf` 参数相同），若失败则返回 `XS_NULL`，此时 `errno` 会被置为对应错误码。其中，参数 `buf` 表示用于存储工作目录字符串的缓冲区首地址；`size` 表示缓冲区大小。

- `void rewinddir(DIR *dirp)`, 该函数用于重置目录的读取位置, 即下一次 `readdir` 将返回目录的第一个目录项。其中, 参数 `dirp` 表示待重置读取位置的目录句柄。
- `long telldir(DIR *dirp)`, 该函数用于获取目录的读取位置, 即下一个 `readdir` 返回目录项的位置。若获取成功则返回 0, 若失败则返回-1, 此时 `errno` 会被置为对应错误码。其中, 参数 `dirp` 表示待获取读取位置的目录句柄。
- `void seekdir(DIR *dirp, off_t offset)`, 该函数用于设置目录的读取位置, 即下一个 `readdir` 返回目录项的位置。若设置成功则返回 0, 若失败则返回-1, 此时 `errno` 会被置为对应错误码。其中, 参数 `dirp` 表示待设置读取位置的目录句柄; `offset` 表示新的读取位置, 应为某次 `telldir` 的返回值。

5.3 文件系统实验

在做文件系统实验之前, 需要在 K210 上安装一个 SD 卡。然后需要通过命令 `make menuconfig` 进行设置, 以识别插入的 SD 卡。整个设置过程如图 3-5-2 至 3-5-5 所示, 依次设置选择为 `edu-riscv feature`→`config default board resources`→`config board peripheral`→`mount cd card` 使能。这样设置之后, 待系统启动时, 才会去识别插入的 SD 卡。

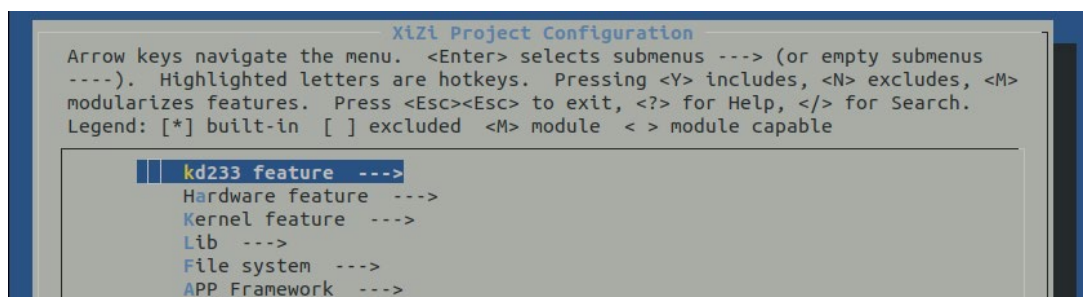


图 3-76-145 选择 edu-riscv feature

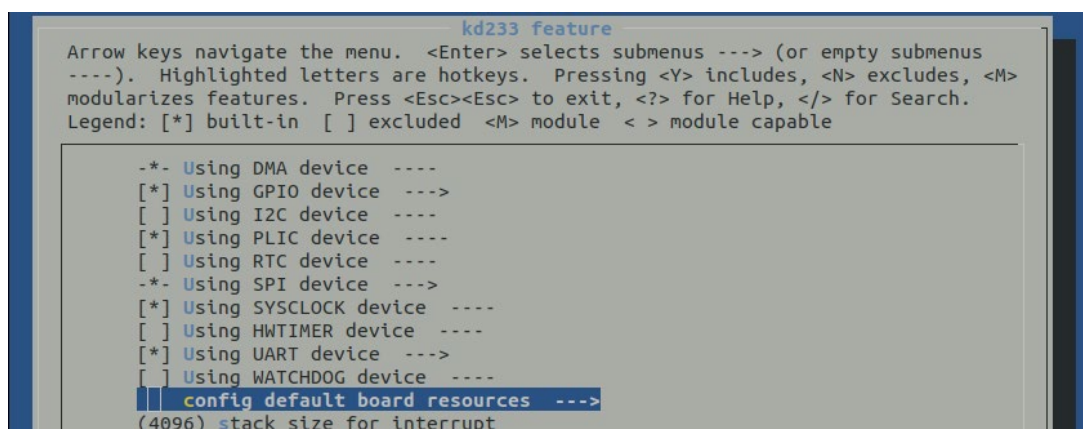


图 3-77-146 选择 config default board resources

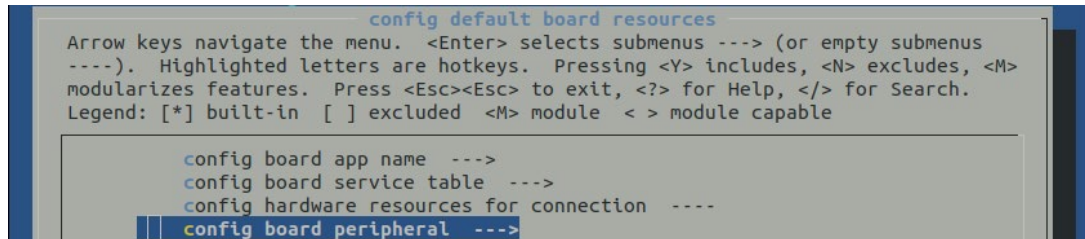


图 3-78-147 选择 config board peripheral

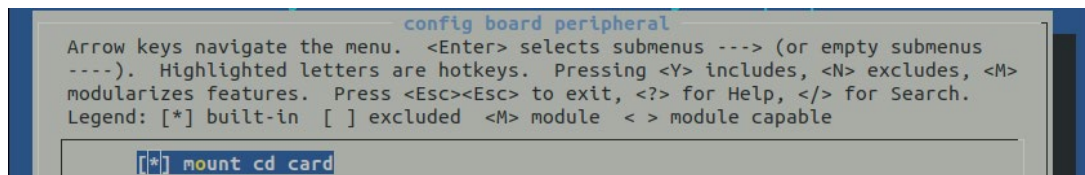


图 3-79-148 使能 mount cd card

文件系统实验如代码 3-5-1 所示，整个验证过程都在函数 `fileSystem` 中实现。代码行 2~4 定义了三个基本变量；代码行 3 通过 `open` 函数打开一个名为 `hello.txt` 的文件，参数 `O_RDWR | O_CREAT` 表示读写和无该文件时自动创建；代码行 6~12 中，首先判断文件是否打开成功，如果成功，则通过 `write` 函数进行写操作；代码行 13 在写操作结束后关闭文件；代码行 15~19 进行文件打开，然后进行读操作，最后关闭打开的文件。

文件系统的实验结果如图 3-5-6 所示，首先可以看到一些 SD 卡挂载的信息，如提示 SD 卡已插入、电源上电、卡的存储大小，以及工作条件等。在最后一行可以看到，写的信息可以被读出来。以上表明文件系统的 SD 卡挂载，文件读写功能均正常。

代码 3-80-81 文件系统实验代码

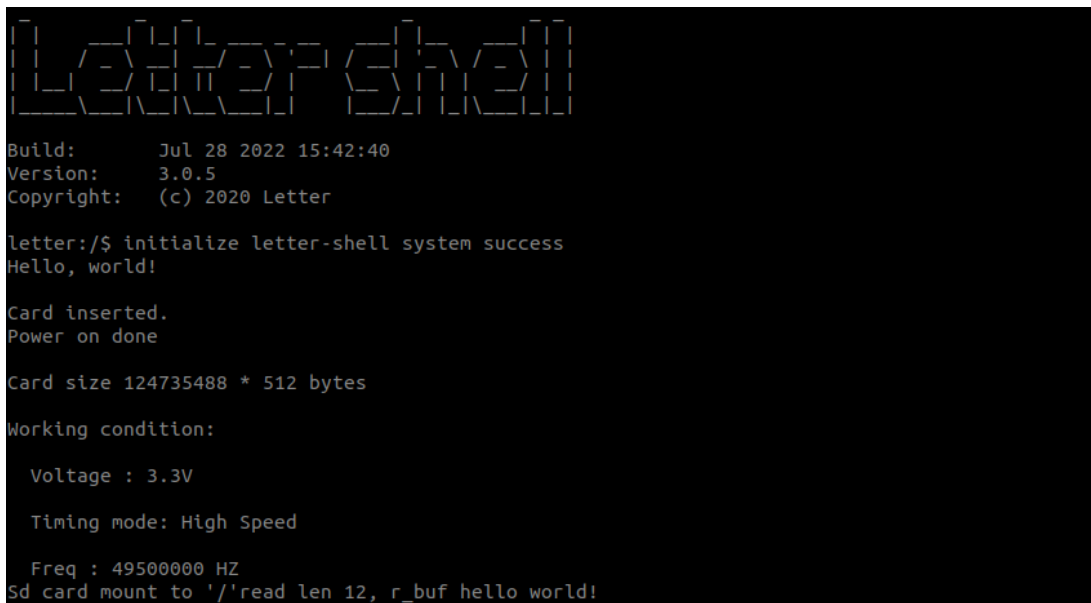
```
01 void* fileSystem () {
02     int ret = 0;
03     char w_buf[] = "hello world!";
04     char r_buf[20];
05
06     int fd_t = open("/hello.txt", O_RDWR | O_CREAT);
07     if(fd_t > 0) {
08         ret = write(fd_t, w_buf, strlen(w_buf));
09         if(ret < 0) {
10             printf("fd = %d write hello world failed.\n", fd_t);
```



```

11         return NULL;
12     }
13     close(fd_t);
14
15     fd_t = open("/hello.txt", O_RDONLY);
16     ret = read(fd_t, r_buf, 20);
17     if(ret > 0)
18         printf("read len %d, r_buf %s\n",ret,r_buf);
19     close(fd_t);
20 }
21 }

```



```

Build:      Jul 28 2022 15:42:40
Version:    3.0.5
Copyright:  (c) 2020 Letter

letter:/ $ initialize letter-shell system success
Hello, world!

Card inserted.
Power on done

Card size 124735488 * 512 bytes

Working condition:

Voltage : 3.3V

Timing mode: High Speed

Freq : 49500000 HZ
Sd card mount to '/' read len 12, r_buf hello world!

```

图 3-82-149 文件系统实验结果

第6章 驱动管理及实验

6.1 驱动管理介绍

多数嵌入式操作系统对驱动的抽象采用以物理外设实体为中心的方式，采用驱动(DRIVER)和设备(DEVICE)一一对应的模式开发驱动，每新增一个外设都需要相应地开发驱动 DRIVER 和设备 DEVICE 逻辑，涉及底层寄存器和应用层业务，开发难度和工作量相对较大，同时操作系统支持外设的数量和种类越多，工程中的驱动实现代码越多，部分同类驱动的代码存在冗余情况，不利于裁剪优化。

为解决以上问题，XiUOS 采用 BUS 总线驱动框架，利用一套 BUS 总线管理驱动 DRIVER 和设备 DEVICE，即同种设备总线管理下，多个设备 DEVICE 复用一個驱动 DRIVER，

数据结构上以 BUS 为基类，分发多种子类对应多种外设，统一管理驱动外设。以 SPI_BUS 总线为例，SPI_BUS 总线创建后，首先会注册 SPI_DRIVER 驱动相关实现，之后系统接入的所有 SPI_DEVICE 都会注册挂载到 SPI_BUS 总线上，DRIVER 驱动和 DEVICE 设备通过 bus_name 绑定上，实现设备和驱动相互对应的配置，这样即可实现用一套 BUS 数据结构管理其挂载的 DEVICE 设备和 DRIVER 驱动的目的。对于上层应用来说，通过获取 SPI_BUS 数据结构即可知道当前系统中挂载的 SPI_DEVICE 个数，利用 BUS 回调函数便可满足配置、数据交互需求，而不用担心具体的硬件细节，且底层新增或删减外设的操作由框架层统一管理，上层应用模块依旧可以复用，这套框架层次化管理界限清晰，驱动和应用解耦、复用功能凸显，应用层开发友好。

XiUOS 的驱动框架 resources/xxx/文件目录结构如下表所示：

表 3-150-83 驱动框架 resources/xxx/文件目录结构

文件名	描述
bus.c	bus 驱动框架实现，提供给上层应用调用
bus_xxx.c	xxx_bus 子类实现，如 spi_bus、i2c_bus、serial_bus 等
dev_xxx.c	xxx_dev 子类实现，如 spi_dev、i2c_dev、serial_dev 等
drv_xxx.c	xxx_drv 子类实现，如 spi_drv、i2c_drv、serial_drv 等

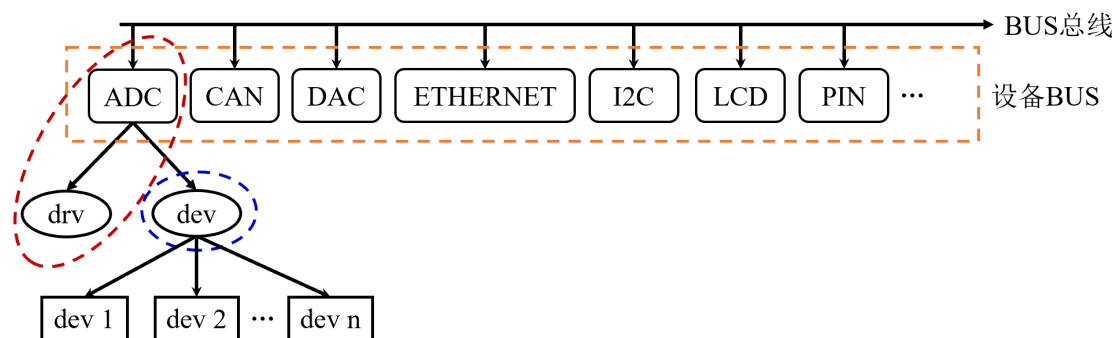


图 3-151-152 驱动框架结构

XiUOS 的驱动框架如图 3-6-1 所示。在设备驱动设置一条 BUS 总线，其实质就是一个链表，该总线上可挂接多个设备总线，比如 ADC 设备总线、CAN 设备总线、DAC 设备总线等等，即每个设备通过各自的总线与 BUS 总线连接，例如 ADC 设备通过 ADC_BUS 挂接到 BUS 总线上。每个设备总线都关联 drv（驱动）和 dev（设备）结构体。在实际场景中，

每种设备驱动都会关联多种物理设备，如图中的 ADC 设备驱动，不同的 ADC 物理设备 dev1、dev2 等，都在 ADC 的 dev 上进行统一管理。

6.2 驱动管理接口

XiUOS 中定义了一系列驱动管理的函数接口：

- `int BusRegister(struct Bus *bus)`，该函数用于注册 BUS。其中，参数 `bus` 是需要注册的总线所对应的结构体指针。
- `int BusUnregister(struct Bus *bus)`，该函数用于删除 BUS。其中，参数 `bus` 表示是需要删除的总线所对应的结构体指针。
- `int DriverRegisterToBus(struct Bus *bus, struct Driver *driver)`，该函数用于 DRIVER 注册并挂接到 BUS 函数。其中，参数 `bus` 是目标总线所对应的结构体变量指针；`driver` 是需要注册的驱动所对应的结构体指针。
- `int DriverDeleteFromBus(struct Bus *bus, struct Driver *driver)`，该函数用于将 DRIVER 从 BUS 中删除。其中参数 `bus` 参数 `bus` 是目标总线所对应的结构体变量指针；`driver` 是需要删除的驱动所对应的结构体指针。
- `int DeviceRegisterToBus(struct Bus *bus, struct HardwareDev *device)`，该函数用于将 DEVICE 注册并挂接到 BUS 函数。参数 `bus` 是目标总线所对应的结构体变量指针；`device` 是需要注册的设备所对应的结构体指针。
- `int DeviceDeleteFromBus(struct Bus *bus, struct HardwareDev *device)`，该函数用于将 DEVICE 从 BUS 中删除。其中参数 `bus` 是目标总线所对应的结构体变量指针；`device` 是需要删除的设备所对应的结构体指针。
- `BusType BusFind(const char *bus_name)`，该函数用于查找 BUS。其中，参数 `bus_name` 是需要查找的总线名称。
- `DriverType BusFindDriver(struct Bus *bus, const char *driver_name)`，该函数用于查找 DRIVER 函数。其中，参数 `bus` 表示从 `bus` 指向的结构体进行查找；`driver_name` 是需要查找的驱动。
- `HardwareDevType BusFindDevice(struct Bus *bus, const char *device_name)`，该函数用于查找 DEVICE 函数。其中，参数 `bus` 表示从 `bus` 指向的结构体进行查找；`device_name` 是需要查找的设备名称。

- uint32 BusDevOpen(struct HardwareDev *dev), 该函数用于打开 DEVICE。其中, 参数 dev 表示所需要进行打开操作的设备所对应的结构体指针。
- uint32 BusDevClose(struct HardwareDev *dev), 该函数用于关闭 DEVICE。其中, 参数 dev 表示所需要进行关闭操作的设备所对应的结构体指针。
- uint32 BusDevWriteData(struct HardwareDev *dev, struct BusBlockWriteParam *write_param), 该函数用于写 DEVICE。其中, 参数 dev 表示所需要进行写操作的设备所对应的结构体指针; write_param 表示进行写操作时所传入的参数。
- uint32 BusDevReadData(struct HardwareDev *dev, struct BusBlockReadParam *read_param), 该函数用于读 DEVICE。其中, 参数 dev 表示所需要进行读操作的设备所对应的结构体指针; write_param 表示进行读操作时所传入的参数。
- uint32 BusDrvConfigure(struct Driver *drv, struct BusConfigureInfo *configure_info), 该函数用于配置 DRIVER。其中, 参数 drv 表示所需要进行配置操作的设备所对应的 Driver 结构体指针; configure_info 表示配置信息, 包括命令类型和参数。

6.3 关键数据结构

代码 3-153-84 struct Bus 结构

```

struct Bus{
    int8 bus_name[NAME_NUM_MAX];
    enum BusType bus_type;
    enum BusState bus_state;

    int32 (*match)(struct Driver *driver, struct HardwareDev *device);
    int bus_lock;

    struct HardwareDev *owner_haldev;
    struct Driver *owner_driver;

    void *private_data;

    /*manage the drv of the bus*/
    uint8 driver_cnt;
    uint8 bus_drvlink_flag;
    DoubleLinklistType bus_drvlink;

```

```

/*manage the dev of the bus*/
uint8 haldev_cnt;
uint8 bus_devlink_flag;
DoubleLinklistType bus_devlink;

uint8 bus_cnt;
uint8 bus_link_flag;
DoubleLinklistType bus_link;
};

```

代码 3-154-85 enum BusType 枚举结构

```

enum BusType{
    TYPE_I2C_BUS = 0,
    TYPE_SPI_BUS,
    TYPE_HWTIMER_BUS,
    TYPE_USB_BUS,
    TYPE_CAN_BUS,
    TYPE_WDT_BUS,
    TYPE_SDIO_BUS,
    TYPE_TOUCH_BUS,
    TYPE_LCD_BUS,
    TYPE_PIN_BUS,
    TYPE_RTC_BUS,
    TYPE_SERIAL_BUS,
    TYPE_BUS_END,
};

```

代码 3-155-86 enum BusState 枚举结构

```

enum BusState{
    BUS_INIT = 0,
    BUS_INSTALL,
    BUS_UNINSTALL,
};

```

代码 3-156-87 struct HardwareDev 结构

```

struct HardwareDev{
    int8 dev_name[NAME_NUM_MAX];
    enum DevType dev_type;
    enum DevState dev_state;
};

```

```

const struct HalDevDone *dev_done;

int (*dev_recv_callback) (void *dev, x_size_t length);
int (*dev_block_control) (struct HardwareDev *dev, struct HalDevBlockParam
*block_param);

struct Bus *owner_bus;
void *private_data;

int32 dev_sem;

DoubleLinklistType dev_link;
};

```

代码 3-157-88 struct HalDevDone 结构

```

struct HalDevDone{
    uint32 (*open) (void *dev);
    uint32 (*close) (void *dev);
    uint32 (*write) (void *dev, struct BusBlockWriteParam *write_param);
    uint32 (*read) (void *dev, struct BusBlockReadParam *read_param);
};

```

HalDevDone 包含统一的、类似文件系统的 API，用于对具体外设进行实际的开关和数据读写。如在使用一个外设前后需要打开（open）/关闭（close）该外设，read、write 分别用于从外设接收数据与向外设发送数据。

代码 3-158-159 struct Driver 结构

```

struct Driver{
    int8 drv_name[NAME_NUM_MAX];
    enum DriverType driver_type;
    enum DriverState driver_state;

    uint32 (*configure)(void *drv, struct BusConfigureInfo *configure_info);

    struct Bus *owner_bus;
    void *private_data;

    DoubleLinklistType driver_link;
};

```

回调函数 configure 用于对具体外设进行实际的配置。

6.4 函数接口定义

代码 3-160-89 BUS 注册函数

```
int BusRegister(struct Bus *bus){
    x_err_t ret = EOK;
    NULL_PARAM_CHECK(bus);
    bus->match = BusMatchDrvDev;
    BusLinkInit(bus);
    bus->bus_lock = KMutexCreate();
    DoubleLinkedListInsertNodeAfter(&bus_linklist, &(bus->bus_link));
    return ret;
}
```

代码 3-161-90 BUS 删除函数

```
int BusUnregister(struct Bus *bus){
    NULL_PARAM_CHECK(bus);
    bus->bus_cnt--;
    DoubleLinkedListRmNode(&(bus->bus_link));
    return EOK;
}
```

代码 3-162-91 DRIVER 注册并挂接到 BUS 函数

```
int DriverRegisterToBus(struct Bus *bus, struct Driver *driver){
    NULL_PARAM_CHECK(bus);
    NULL_PARAM_CHECK(driver);
    driver->owner_bus = bus;
    bus->driver_cnt++;
    DoubleLinkedListInsertNodeAfter(&bus->bus_drvlink, &(driver->driver_link));
    return EOK;
}
```

代码 3-163-92 将 DRIVER 从 BUS 中删除函数

```
int DriverDeleteFromBus(struct Bus *bus, struct Driver *driver){
    NULL_PARAM_CHECK(bus);
    NULL_PARAM_CHECK(driver);
    bus->driver_cnt--;
    DoubleLinkedListRmNode(&(driver->driver_link));
    free(driver);
    return EOK;
}
```

}

代码 3-164-93 DEVICE 注册并挂接到 BUS 函数

```
int DeviceRegisterToBus(struct Bus *bus, struct HardwareDev *device){
    NULL_PARAM_CHECK(bus);
    NULL_PARAM_CHECK(device);
    device->owner_bus = bus;
    bus->haldev_cnt++;
    DoubleLinkListInsertNodeAfter(&bus->bus_devlink, &(device->dev_link));
    return EOK;
}
```

代码 3-165-94 DEVICE 从 BUS 中删除函数

```
int DeviceDeleteFromBus(struct Bus *bus, struct HardwareDev *device){
    NULL_PARAM_CHECK(bus);
    NULL_PARAM_CHECK(device);
    bus->haldev_cnt--;

    DoubleLinkListRmNode(&(device->dev_link));
    free(device);
    return EOK;
}
```

代码 3-166-95 查找 BUS 函数

```
BusType BusFind(const char *bus_name){
    struct Bus *bus = NONE;
    DoubleLinklistType *node = NONE;
    DoubleLinklistType *head = &bus_linklist;
    for (node = head->node_next; node != head; node = node->node_next){
        bus = SYS_DOUBLE_LINKLIST_ENTRY(node, struct Bus, bus_link);
        if(!strcmp(bus->bus_name, bus_name)) {
            return bus;
        }
    }
    KPrintf("BusFind cannot find the %s bus.return NULL\n", bus_name);
    return NONE;
}
```

代码 3-167-96 查找 DRIVER 函数

```

DriverType BusFindDriver(struct Bus *bus, const char *driver_name){
    NULL_PARAM_CHECK(bus);
    struct Driver *driver = NONE;
    DoubleLinklistType *node = NONE;
    DoubleLinklistType *head = &bus->bus_drvlink;
    for (node = head->node_next; node != head; node = node->node_next){
        driver = SYS_DOUBLE_LINKLIST_ENTRY(node, struct Driver, driver_link);
        if(!strcmp(driver->drv_name, driver_name)) {
            return driver;
        }
    }
    KPrintf("BusFindDriver cannot find the %s driver.return NULL\n", driver_name);
    return NONE;
}

```

代码 3-168-97 查找 DEVICE 函数

```

HardwareDevType BusFindDevice(struct Bus *bus, const char *device_name){
    NULL_PARAM_CHECK(bus);
    struct HardwareDev *device = NONE;
    DoubleLinklistType *node = NONE;
    DoubleLinklistType *head = &bus->bus_devlink;
    for (node = head->node_next; node != head; node = node->node_next){
        device = SYS_DOUBLE_LINKLIST_ENTRY(node, struct HardwareDev, dev_link);
        if(!strcmp(device->dev_name, device_name)) {
            return device;
        }
    }
    KPrintf("BusFindDevice cannot find the %s device.return NULL\n", device_name);
    return NONE;
}

```

代码 3-169-98 打开 DEVICE 函数

```

uint32 BusDevOpen(struct HardwareDev *dev){
    NULL_PARAM_CHECK(dev);
    x_err_t ret = EOK;
    if (dev->dev_done->open) {
        ret = dev->dev_done->open(dev);
        if(ret) {
            KPrintf("BusDevOpen error ret %u\n", ret);
            return ERROR;
        }
    }
}

```

```
    }  
}  
return ret;  
}
```

代码 3-170-99 关闭 DEVICE 函数

```
uint32 BusDevClose(struct HardwareDev *dev){  
    NULL_PARAM_CHECK(dev);  
    x_err_t ret = EOK;  
    if (dev->dev_done->close) {  
        ret = dev->dev_done->close(dev);  
        if(ret) {  
            KPrintf("BusDevClose error ret %u\n", ret);  
            return ERROR;  
        }  
    }  
    return ret;  
}
```

代码 3-171-100 DEVICE 写函数

```
uint32 BusDevWriteData(struct HardwareDev *dev, struct BusBlockWriteParam  
*write_param)  
{  
    NULL_PARAM_CHECK(dev);  
    if (dev->dev_done->write) {  
        return dev->dev_done->write(dev, write_param);  
    }  
    return EOK;  
}
```

代码 3-172-101 DEVICE 读函数

```
uint32 BusDevReadData(struct HardwareDev *dev, struct BusBlockReadParam *read_param){  
    NULL_PARAM_CHECK(dev);  
    if (dev->dev_done->read) {  
        return dev->dev_done->read(dev, read_param);  
    }  
    return EOK;  
}
```

代码 3-173-102 DRIVER 配置函数

```

uint32 BusDrvConfigure(struct Driver *drv, struct BusConfigureInfo *configure_info){
    NULL_PARAM_CHECK(drv);
    NULL_PARAM_CHECK(configure_info);
    x_err_t ret = EOK;
    if (drv->configure) {
        ret = drv->configure(drv, configure_info);
        if(ret) {
            KPrintf("BusDrvCfg error, ret %u\n", ret);
            return ERROR;
        }
    }
    return ret;
}

```

6.5 驱动模型实验

驱动管理的实验设计过程如图 3-6-2 所示。

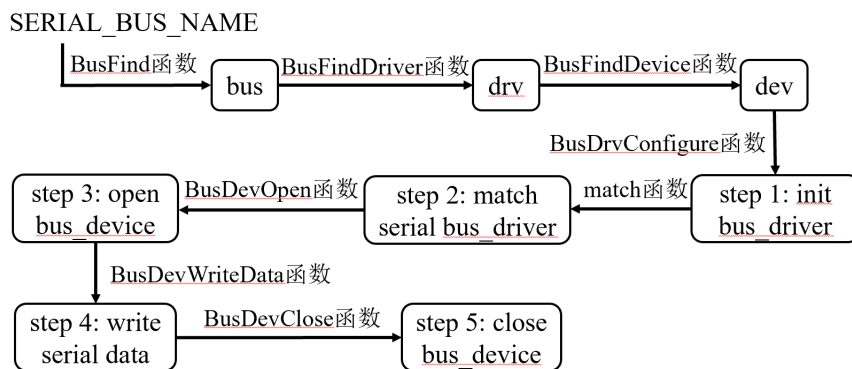


图 3-174-103 驱动管理实验设计

该图以完整的串口驱动使用过程为例。首先在获取外设数据前，要先获取外设 bus，匹配 driver 和 device 数据结构，若有需要应配置外设 driver，在外设 device 打开后对外设进行读写，使用完毕后，关闭设备。

代码 3-6-21 是驱动管理实验的具体实现。代码行 2~7 定义了一些基本变量和总线、驱动、设备结构体变量。代码行 9~14 通过 SERIAL_BUS_NAME 和调用 BusFind 函数生成串口总线指针。代码行 16~21 通过 BusFindDriver 函数找到串口驱动指针。代码行 23~28 通过 BusFindDevice 函数找到串口设备指针。然后，代码行 30~59 通过 5 个步骤，分别实现初始化、匹配、打开、写串口数据和关闭操作。需要说明的是，驱动管理种类繁多，该实验只针

对串口驱动的管理的案例，做了简单的介绍。更全面、详细的驱动管理相关实验，会在第四篇和第五篇介绍。

代码 3-175-104 驱动管理实验代码

```

01 int driverManagement(){
02     int ret = EOK;
03     char test_str[] = "Hello AIIT!\r\n";
04
05     struct Bus *bus;
06     struct Driver *drv;
07     struct HardwareDev *dev;
08
09     /* find the serial bus pointer */
10     bus = BusFind(SERIAL_BUS_NAME);
11     if (NONE == bus) {
12         KPrintf("BusFind  %s failed\n", SERIAL_BUS_NAME);
13         return ERROR;
14     }
15
16     /* find the serial driver pointer */
17     drv = BusFindDriver(bus, SERIAL_DRV_NAME);
18     if (NONE == drv) {
19         KPrintf("BusFindDriver  %s failed\n", SERIAL_DRV_NAME);
20         return ERROR;
21     }
22
23     /* find the serial device pointer */
24     dev = BusFindDevice(bus, SERIAL_DEVICE_NAME);
25     if (NONE == dev) {
26         KPrintf("BusFindDevice  %s failed\n", SERIAL_DEVICE_NAME);
27         return ERROR;
28     }
29
30     /*step 1: init bus_driver, change struct SerialCfgParam if necessary*/
31     struct SerialCfgParam serial_cfg;
32     memset(&serial_cfg, 0, sizeof(struct SerialCfgParam));
33     configure_info.configure_cmd = OPE_INT;
34     configure_info.private_data = &serial_cfg;
35     ret = BusDrvConfigure(drv, &configure_info);
36     if (EOK != ret) {

```

```
37         KPrintf("BusDrvConfigure  OPE_INT failed error code %d\n", ret);
38         return ret;
39     }
40
41     /*step 2: match serial bus_driver with bus_device*/
42     bus->match(drv, dev);
43
44     /*step 3: open bus_device, configure struct SerialDevParam if necessary*/
45     serial_dev_param->serial_set_mode = SIGN_OPER_INT_RX;
46     serial_dev_param->serial_stream_mode = SIGN_OPER_STREAM;
47     ret = BusDevOpen(dev);
48     if (EOK != ret) {
49         KPrintf("BusDevOpen failed error code %d\n", ret);
50         return ret;
51     }
52
53     /*step 4: write serial data, configure struct BusBlockWriteParam*/
54     struct BusBlockWriteParam write_param;
55     write_param.pos = 0;
56     write_param.buffer = (void *)test_str;
57     write_param.size = sizeof(test_str) - 1;
58     BusDevWriteData(bus_device, &write_param);
59
60     /*step 5: close bus_device*/
61     BusDevClose(bus_device);
62
63     return EOK;
64 }
```

第4篇 驱动开发实验-ARM 篇

第1章 GPIO 驱动

本章将介绍 HC32F4A0 的 GPIO 模块。HC32F4A0 拥有多个 GPIO（General Purpose Input/Output）端口，通过读取和控制信号来控制外部设备，例如控制 LED 灯的开关、读取按钮的状态等，大部分情况下使用的是数字信号，也有很少一部分情况下使用模拟信号。

1.1 GPIO 介绍

1.1.1 GPIO 基本介绍

在嵌入式开发中，GPIO（General Purpose Input/Output）是指通用输入输出引脚，它是一种可以被程序控制的数字信号引脚。GPIO 可以被用来连接各种外设，如 LED、按键、传感器等。通过控制 GPIO 的输出信号，我们可以控制外部设备的状态。GPIO 通常是通过处理器的 IO 口实现的。处理器的 IO 口可以被配置为输入模式或输出模式。当 IO 口被配置为输入模式时，它可以读取外部设备的状态；当 IO 口被配置为输出模式时，它可以控制外部设备的状态。

在程序中，我们可以通过操作 GPIO 的寄存器来控制其输入输出模式以及输出信号的状态。具体来说，我们可以使用一些特定的寄存器来进行配置，如 GPIO 方向寄存器（GPIO direction register）用于配置 GPIO 的输入输出模式，GPIO 数据寄存器（GPIO data register）用于设置 GPIO 的输出信号状态，GPIO 状态寄存器（GPIO status register）用于读取 GPIO 的输入信号状态等。

HC32F4A0 中的 GPIO 模块每组 Port 配有 16 个 I/O Pin，支持高中低三种驱动模式，支持外部中断的输入，同时支持 I/O 周边功能复用，一个 I/O Pin 最多可具有 64 个可选的复用功能，以提高引脚的利用率。比如，一个引脚可以同时用于控制 LED 灯和读取按钮状态。

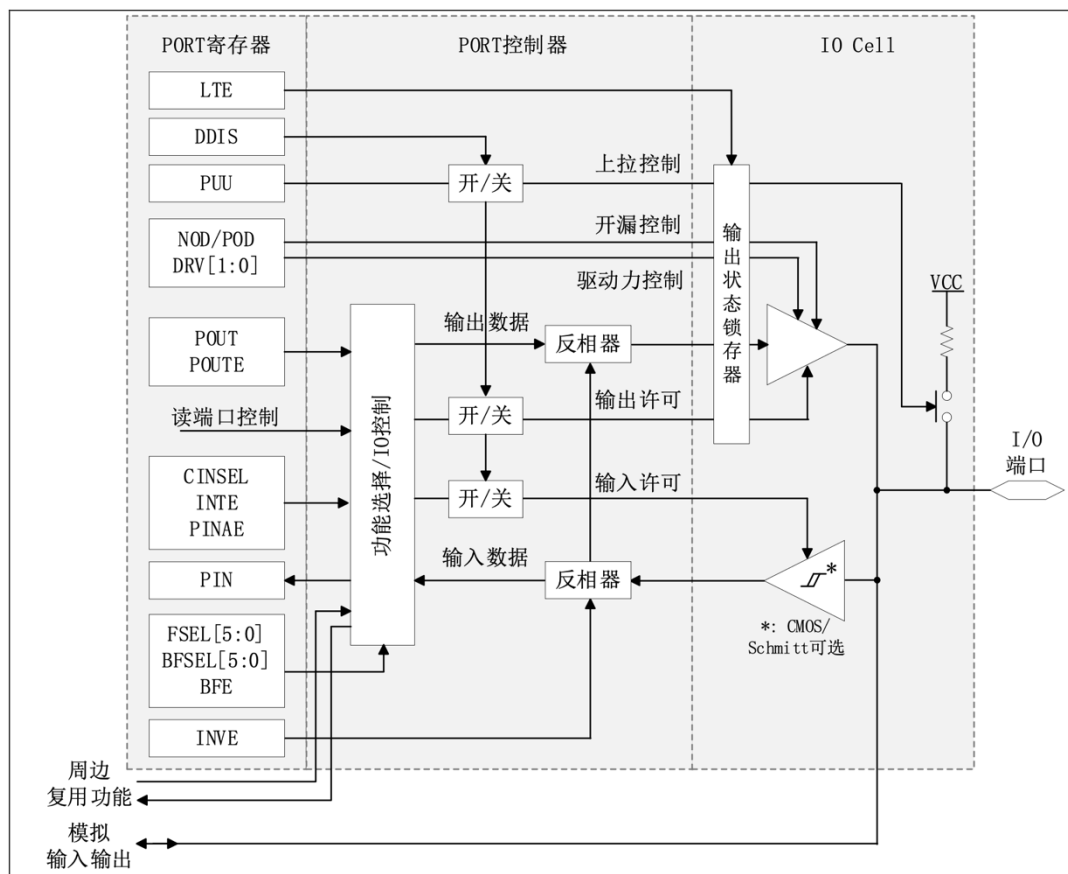


图 4-1-176 GPIO 使用框架图

各个 I/O 都具有通用输入 GPI 功能，且在数字功能禁止位 $PCR_{xy}.DDIS$ （对于 P_{xy} ， $x=A \sim I$ ， $y=0 \sim 15$ 表示单个 I/O 端口，比如 $PB10$ 表示 PB 组的第十个 I/O，所以 $PCR_{xy}.DDIS$ 表示 P_{xy} 端口的 PCR 寄存器的 DDIS 位）为 0 时，GPI 功能始终有效。可以通过访问端口输入数据寄存器 $PIDR_x$ （ $x=A \sim I$ ，表示这一组 16 个 I/O 端口）获取当前端口的状态，也可以通过端口控制寄存器 PCR_{xy} 的 PIN 位查询相应的单 I/O 端口状态。

除了输入专用的端口外，其他的 I/O 端口都具有通用输出 GPO 功能。通过设置端口功能选择寄存器 $PFSR_{xy}.FSEL[5:0]$ 为 0x0 可以有效 GPO 功能。GPO 功能有效时，可以通过设置通用输出许可寄存器 $POER_x$ 来允许或者禁止 I/O 的输出，通用输出数据寄存器 $PODR$ 来控制的输出值。使用下面的 3 个寄存器也可以控制 IO 的输出值：输出数据清零寄存器 $PORR_x$ ，输出数据置位寄存器 $POSRX$ ，输出数据翻转寄存器 $POTRX$ 。对上述寄存器中相应位写 1 可使对应 IO 输出 0、1、翻转写 0 时 IO 输出状态不改变。

1.1.2 中断说明

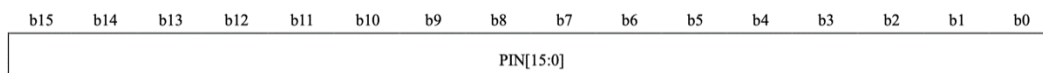
除 PI13 引脚外,每个 I/O 端口都具有外部中断输入功能。当 PCRxyINTE 位设为 1 时,此 IO 将作为外部中断源 EIRQy 被允许输入(如:PA0 对应 EIRQ0, PA2 对应 EIRQ2)。每个 EIRQy 可配置的 I/O 不止一个,使用时每个 EIRQy 不要同时允许多个 IO 输入。EIROY 输入功能与 PFSRXYFSEL 选择的周边功能(包括 GPIO)可同时有效。

1.1.3 寄存器说明

下面列出来 GPIO 模块的操作系统部分用到了的寄存器。

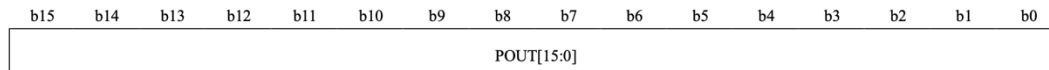
其中地址计算公式中 $x=A\sim I$, 对应的 $n=0\sim 8$ 。

一. 通用输入寄存器 (PIDRx)



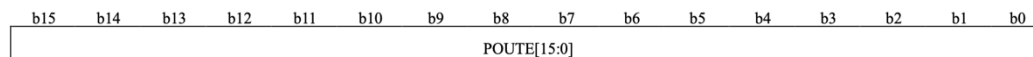
- 本寄存器为只读寄存器,写无效。在数字功能未被禁止 DDIS=0 时,通过读取寄存器可以获取端口的输入状态。

二. 通用输出寄存器



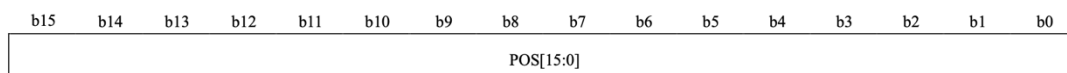
- 当 I/O 端口被设置成 GPO 功能时,改写此寄存器可以更改对应端口的输出状态。

三. 通用输出许可寄存器 (POERx)



- 为 0 时表示输出禁止,为 1 时表示输出许可。

四. 通用输出置位寄存器 (POSRx)



- 若是输入 0,则对应的 PODRx.POUT 无变化,若为 1,则对应的 PORTx.POUT 置 1。此寄存器的读出值始终为 0x0000。

五. 通用输出复位寄存器 (PORRx)

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
POR[15:0]															

- 若是输入 0，则对应的 PODRx.POUT 无变化，若为 1，则对应的 PORTx.POUT 置 0。此寄存器的读出值始终为 0x0000。

六. 通用输出翻转寄存器 (POTRx)

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
POT[15:0]															

- 若是输入 0，则对应的 PODRx.POUT 无变化，若为 1，则对应的 PORTx.POUT 取反。此寄存器的读出值始终为 0x0000。

七. 特殊控制寄存器 (PSPCR)

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	-	-	-	-	-	-	-	-	SPFE[4:0]				

- b15~b5: 作为保留位，读的时候为 0，写的时候写 0
- b4: 为 0 时 NJTRST 功能无效，为 1 时有效
- b3: 为 0 时 JTDI 功能无效，为 1 时有效
- b2: 为 0 时 JTDO_TRACESWO 功能无效，为 1 时有效
- b1: 为 0 时 JTMS_SWDIO 功能无效，为 1 时有效
- b0: 为 0 时 JTCK_SWCLK 功能无效，为 1 时有效

八. 公共控制寄存器 (PCCR)

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	RDWT[2:0]			-	-	-	-	-	-	BFSEL[5:0]					

- b15: 作为保留位，读的时候为 0，写的时候写 0
- b14~b12: 设置读寄存器 PIDRx, PCRxy 时插入的等待周期数
- 000 表示无等待，001 为默认值表示等待一周，以此类推 111 表示 7 周期
- b11~b6: 作为保留位，读的时候为 0，写的时候写 0
- b5~b0: 表示各端口的功能配置，具体配置请查看 HC32F4A0 数据手册中的引脚功能表

九. 输入控制寄存器 (PINAER)

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	-	-	-	-	PINA[8:0]								

- b15~b9: 作为保留位，读的时候为 0，写的时候写 0
- b8~b0: 值为 0 时表示输入 MOS 常开无效，输入 1 时表示有效

其中：

PINAE[0]控制 PA0~PA15

PINAE[1]控制 PBO~PB15

PINAE[2]控制 PCO~PC15

PINAE[3] 控制 PDO~PD15

PINAE[4]控制 PEO~PE15

PINAE[5] 控制 PFO~PF15

PINAE[6]控制 PGO~PG15

PINAE[7]控制 PHO~PH15

PINAE[8]控制 PIO~PI13

十. 写保护寄存器（PWPR）

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
WP[7:0]								-	-	-	-	-	-	-	WE

- b15~b8: 作为保留位，读的时候为 0，当 b15~b8 写入值为 0xA5 时，b0 值写入 WE，当写 0xA5 以外值时，WE 自动清零
- b7~b1: 作为保留位，读的时候为 0，写的时候写 0
- b0: 为 0 时表示 PSPCR, PCCR, PINAER, PCRxy, PFSRxy 寄存器写禁止，为 1 时表示 PSPCR, PCCR, PINAER, PCRxy, PFSRxy 寄存器写许可

十一. 通用控制寄存器（PCRxy）

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
DDIS	LTE	-	INTE	-	CINSEL	INVE	PIN	-	PUU	DRV[1:0]	-	NOD	POUTE	POUT	

- b15: 为 0 时表示数字功能有效，为 1 时表示数字功能禁止
- b14: 为 0 时表示输出锁存无效，为 1 时表示输出锁存有效
- b13: 作为保留位，读的时候为 0，写的时候写 0

- b12: 为 0 时表示外部中断输入禁止, 为 1 时表示外部中断输入许可
- b11: 作为保留位, 读的时候为 0, 写的时候写 0
- b10: 输入模式选择, 为 0 时表示 Schmitt 输入, 为 1 时表示 CMOS 输入
- b9: 反相许可, 为 0 时表示输入输出数据不反相, 为 1 时表示反相
- b8: 表示输入状态, 为 0 时表示输入状态为低电平, 为 1 时表示高电平
- b7: 作为保留位, 读的时候为 0, 写的时候写 0
- b6: 上拉许可, 为 0 时表示内部上拉电阻无效, 为 1 时表示有效
- b5~b4: 为 00 时表示低驱动模式, 01 时为中驱动模式, 10, 11 时为高驱动模式
- b3: b11: 作为保留位, 读的时候为 0, 写的时候写 0
- b2: 为 0 时表示正常 CMOS 输出模式, 为 1 时表示 NMOS 开漏输出
- b1: 输出许可, 0 表示输出禁止, 1 表示输出许可
- b0: 输出数据, 为 0 时表示输出低电平, 为 1 时表示输出高电平

十二. 功能选择寄存器 (PFSRxy)

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	-	-	-	-	BFE	-	-	FSEL[5:0]					

- b15~b9: 作为保留位, 读的时候为 0, 写的时候写 0
- b8: 副功能许可, 控制着 PCCR.BFSEL[5:0]的功能选择是否有效, 0 表示禁止, 1 表示有效
- b7~b6: 作为保留位, 读的时候为 0, 写的时候写 0
- b5~b0: 表示功能选择, 各个端口的功能配置请参考 HC32F4A0 数据手册中的引脚功能表

1.2 GPIO 驱动介绍

GPIO 设备通过 PIN BUS 挂接到 BUS 总线上, 其中 PIN BUS 的结构体在 bus_pin.h 文件中进行了定义。每个设备总线都关联一个 drv 和一个 dev, 其中, drv 结构体也在 bus_pin.h 中进行了定义, dev 结构体在 dev_pin.h 中进行了定义。另外, PIN 驱动及设备的注册流程在 connect_pin.c 中进行了实现。接下来, 针对这 3 个 PIN 相关的具体结构和实现流程进行详细介绍。

1.2.1 bus_pin.h 文件

bus_pin.h 文件中声明了 PIN 总线结构体 PinBus 和驱动结构体 PinDriver。PinBus 的结构体定义如下：

```
struct PinBus{
    struct Bus bus;
    void *private_data;
};
```

其中 bus 是通用管理总线；private_data 是传递参数的通用指针。

PinDriver 的结构体定义如下：

```
struct PinDriver{
    struct Driver driver;
    uint32 (*configure)(void *drv, struct BusConfigureInfo *configure_info);
};
```

其中 driver 表示通用管理驱动；configure 表示配置函数。

另外，在 bus_pin.h 文件中还定义了一些函数接口，具体均在 bus_pin.c 文件中实现：

- int PinBusInit(struct PinBus *pin_bus, const char *bus_name), 用于初始化 PIN 总线，其中 pin_bus 表示初始化的 PIN 总线；bus_name 表示总线名称。通过调用 BusRegister 函数将 PIN 总线注册到 BUS 管理总线链表。
- int PinReleaseBus(struct PinBus *pin_bus), 用于释放 LCD 总线，即将 LCD 总线从 BUS 管理总线链表中移除，其中 lcd_bus 表示需要释放的 LCD 总线。一般该接口函数与 LcdBusInit 函数配对使用，对 LCD 总线进行注册和释放。
- int PinDriverInit(struct PinDriver *pin_driver, const char *driver_name, void *data), 用于初始化 PIN 驱动，其中 pin_driver 表示需要初始化的 PIN 驱动；driver_name 表示驱动名称；data 是传递参数的通用指针。该接口通过调用 PinDriverRegister 函数将 PIN 驱动注册到 PIN 驱动总线。
- int PinDriverAttachToBus(const char *drv_name, const char *bus_name), 用于将 PIN 驱动挂接到 PIN BUS 总线，其中 drv_name 表示驱动名称；bus_name 表示总线名称。
- DriverType PinDriverFind(const char *drv_name, enum DriverType_e drv_type), 用于查找挂接的 PIN 驱动，其中 drv_name 表示驱动名称；drv_type 表示驱动类型。

1.2.2 dev_pin.h 文件

在 dev_pin.h 文件中声明了 PIN 设备结构体 PinHardwareDevice，其具体的定义如下：

```
struct PinHardwareDevice{
    struct HardwareDev haldev;
    const struct PinDevDone *dev_done;
    void *private_data;
};
```

其中 haldev 用于描述一个设备结构；dev_done 用于表示一个 PIN 设备的函数操作集；private_data 是传递参数的通用指针。

结构体 PinDevDone 的定义如下：

```
struct PinDevDone{
    uint32 (*open) (void *dev);
    uint32 (*close) (void *dev);
    uint32 (*write) (void *dev, struct BusBlockWriteParam *write_param);
    uint32 (*read) (void *dev, struct BusBlockReadParam *read_param);
};
```

可以看出，该结构体是用于描述 PIN 设备的打开、关闭、读写功能。

另外，在 dev_pin.h 文件中还定义了一些函数接口：

- int PinDeviceRegister(struct PinHardwareDevice *pin_device, void *pin_param, const char *device_name), 用于初始化并注册 PIN 设备到设备管理链表上，其中 pin_device 表示需要注册的 PIN 设备；pin_param 表示 PIN 参数；device_name 表示设备名称。
- int PinDeviceAttachToBus(const char *dev_name, const char *bus_name), 用于将 PIN 设备挂接到 PIN BUS 总线，其中 dev_name 表示设备名称；bus_name 表示总线名称。
- HardwareDevType PinDeviceFind(const char *dev_name, enum DevType dev_type), 用于查找挂接的 PIN 设备，其中 dev_name 表示设备名称；dev_type 表示设备类型。

1.2.3 connect_pin.c 文件

connect_pin.c 文件中实现了 PIN 驱动及设备的注册，该流程主要包含 3 个步骤：

- (1) 完成 PIN 的驱动和设备到 PIN 总线的挂接（或称为绑定）；
- (2) 实现 PIN 设备的打开、关闭及读写方法，其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植；

(3) 实现 PIN 设备的驱动配置，需结合 PIN 特性及操作系统的接口定义进行相应的开发。

步骤 (1) 的实现过程如代码 4-1-1 所示，该流程是完成一个 PIN 驱动及设备总线绑定的过程。代码行 5~6 定义了 1 个结果返回变量和 1 个 PinBus 结构体变量。代码行 8~12 使用 PinBusInit 函数对 PIN 总线进行初始化。代码行 18~22 使用 PinDriverInit 函数对驱动进行初始化。代码行 24~28 使用 PinDriverAttachToBus 函数将 PIN 驱动绑定到总线。代码行 34~38 通过 PinDeviceRegister 函数注册 PIN 设备。代码行 40~44 通过 PinDeviceAttachToBus 将 PIN 设备挂接到总线。

代码 4-1-105 GPIO 驱动及设备绑定

```

01 int HwGpioInit(void)
02 {
03     x_err_t ret = EOK;
04
05     static struct PinBus pin;
06     memset(&pin, 0, sizeof(struct PinBus));
07
08     ret = PinBusInit(&pin, PIN_BUS_NAME);
09     if (ret != EOK) {
10         KPrintf("gpio bus init error %d\n", ret);
11         return ERROR;
12     }
13
14     static struct PinDriver drv;
15     memset(&drv, 0, sizeof(struct PinDriver));
16     drv.configure = &Hc32GpioDrvConfigure;
17
18     ret = PinDriverInit(&drv, PIN_DRIVER_NAME, NONE);
19     if (ret != EOK) {
20         KPrintf("pin driver init error %d\n", ret);
21         return ERROR;
22     }
23
24     ret = PinDriverAttachToBus(PIN_DRIVER_NAME, PIN_BUS_NAME);
25     if (ret != EOK) {
26         KPrintf("pin driver attach error %d\n", ret);
27         return ERROR;

```

```

28     }
29
30     static struct PinHardwareDevice dev;
31     memset(&dev, 0, sizeof(struct PinHardwareDevice));
32     dev.dev_done = &dev_done;
33
34     ret = PinDeviceRegister(&dev, NONE, PIN_DEVICE_NAME);
35     if (ret != EOK) {
36         KPrintf("pin device register error %d\n", ret);
37         return ERROR;
38     }
39
40     ret = PinDeviceAttachToBus(PIN_DEVICE_NAME, PIN_BUS_NAME);
41     if (ret != EOK) {
42         KPrintf("pin device register error %d\n", ret);
43         return ERROR;
44     }
45
46     return ret;
47 }

```

步骤（2）的实现过程如代码 4-1-2 所示，该流程是一个实现 PIN 设备的打开、关闭及读写过程。根据结构体 PinDevDone 的定义，此时打开和关闭功能的值为 NONE，表示未启用该功能；读和写功能的值为 HC32PinWrite 和 HC32PinRead，表明读写功能是打开的。

代码 4-1-106 PIN 设备的打开、关闭及读写

```

01 static const struct PinDevDone dev_done =
02 {
03     .open = NONE,
04     .close = NONE,
05     .write = HC32PinWrite,
06     .read = HC32PinRead,
07 };

```

PIN 设备的驱动配置过程如代码 4-1-3 所示。在 HC32GpioDrvConfigure 函数中，代码行 2~3 进行输入参数 drv 和 configure_info 的检查。代码行 5~6 声明了 2 个变量。代码行 8~18 实现了具体的配置过程。若配置命令是 OPE_INT，则执行 HC32PinInit 函数进行初始化；若配置命令是 OPE_CFG，则执行 HC32PinConfigure 函数进行 PIN 配置。

代码 4-1-107 PIN 设备的驱动配置

```

01 static uint32 HC32GpioDrvConfigure(void *drv,
                                struct BusConfigureInfo *configure_info){
02     NULL_PARAM_CHECK(drv);
03     NULL_PARAM_CHECK(configure_info);
04
05     x_err_t ret = EOK;
06     struct PinParam *param;
07
08     switch (configure_info->configure_cmd){
09         case OPE_INT:
10             ret = HC32PinInit();
11             break;
12         case OPE_CFG:
13             param = (struct PinParam *)configure_info->private_data;
14             ret = HCPinConfigure(param);
15             break;
16         default:
17             break;
18     }
19
20     return ret;
21 }

```

1.3 GPIO 设备应用

GPIO 设备应用实现过程如代码 4-1-4 所示。在测试函数 TestGpio 中，代码行 2~6 用于打开一个 GPIO 设备。代码行 8~15 声明了两个结构体变量，并对其参数进行赋值。代码行 17~21 通过 IO 控制函数对打开的 GPIO 设备进行配置（输出）。代码行 23~30 先对结构体变量参数进行重新赋值，然后再用 IO 控制函数对打开的 GPIO 设备进行配置（输入）。代码行 39~43 对 GPIO 设备进行读操作。代码行 51~55 对 GPIO 设备进行写操作。

代码 4-1-108 GPIO 设备应用

```

01 void TestGpio(void){
02     int pin_fd = PrivOpen(GPIO_DEV_DRIVER, O_RDWR);
03     if(pin_fd<0){
04         printf("open pin fd error:%d\n",pin_fd);
05         return;
06     }

```

```
07
08     struct PinParam parameter;
09     parameter.cmd = GPIO_CONFIG_MODE;
10     parameter.pin = BSP_LED_PIN;
11     parameter.mode = GPIO_CFG_OUTPUT;
12
13     struct PrivIoctlCfg ioctl_cfg;
14     ioctl_cfg.ioctl_driver_type = PIN_TYPE;
15     ioctl_cfg.args = (void *)&parameter;
16
17     if (0 != PrivIoctl(pin_fd, OPE_CFG, &ioctl_cfg)) {
18         printf("ioctl pin fd error %d\n", pin_fd);
19         PrivClose(pin_fd);
20         return;
21     }
22
23     parameter.pin = BSP_KEY_PIN;
24     parameter.mode = GPIO_CFG_INPUT;
25
26     if (0 != PrivIoctl(pin_fd, OPE_CFG, &ioctl_cfg)) {
27         printf("ioctl pin fd error %d\n", pin_fd);
28         PrivClose(pin_fd);
29         return;
30     }
31
32     struct PinStat pin_led;
33     struct PinStat pin_key;
34
35     pin_led.pin = BSP_LED_PIN;
36     pin_key.pin = BSP_KEY_PIN;
37
38     while(1){
39         if(0>PrivRead(pin_fd,&pin_key,NULL_PARAMETER)){
40             printf("read pin fd error %d\n", pin_fd);
41             PrivClose(pin_fd);
42             return;
43         }
44
45         if(pin_key.val){
46             pin_led.val = GPIO_HIGH;
47         }else{
```

```
48         pin_led.val = GPIO_LOW;
49     }
50
51     if(0>PrivWrite(pin_fd,&pin_led,NULL_PARAMETER)){
52         printf("write pin fd error %d\n", pin_fd);
53         PrivClose(pin_fd);
54         return;
55     }
56
57 }
58 }
```

第2章 TIMER 驱动

这一章将介绍如何使用 HC32F4A0 的通用定时器。HC32F4A0 的定时器包括了 8 个多功能 PWM Timer (Timer6)，支持 16 路互补 PWM 输出，16 个高精度 PWM (HRPWM) 扩展了 Timer6 的 PWM 信号的分辨率，3 个电机 PWM Timer (Timer4) 支持 18 路互补 PWM 输出，12 个 16bit 通用 Timer (TimerA) 支持 6 路 3 相正交编码输入及 48 路 Duty 独立可设 PWM 输出，也包括了通用定时器 Timer0。本章将选择 Timer0 来介绍。

2.1 TIMER 介绍

2.1.1 TIMER0 基本介绍

在嵌入式开发中，TIMER（定时器）是一种常用的硬件模块，用于在设备上生成定时中断。定时中断是指定时器计数器到达预设的值时，会产生一个中断信号，以便 CPU 能够在中断服务程序中进行相应的处理。TIMER 通常由一个计数器和一些控制寄存器组成。计数器的值可以由 CPU 或外部时钟源进行读取和写入。控制寄存器用于配置 TIMER 的工作模式、计数器的初始值、中断使能等。

通用定时器 0 (Timer0) 是一个可以实现同步计数，异步计数方式的基本定时器，该定时器内含 2 个通道 (CH-A 和 CH-B)，可以在计数期间产生比较匹配事件。该事件可以触发中断，也可以作为事件输出来控制其他模块等。HC32F4A0 中搭载了两个单元的 Timer0。其基本框架如图 4-2-1 所示：

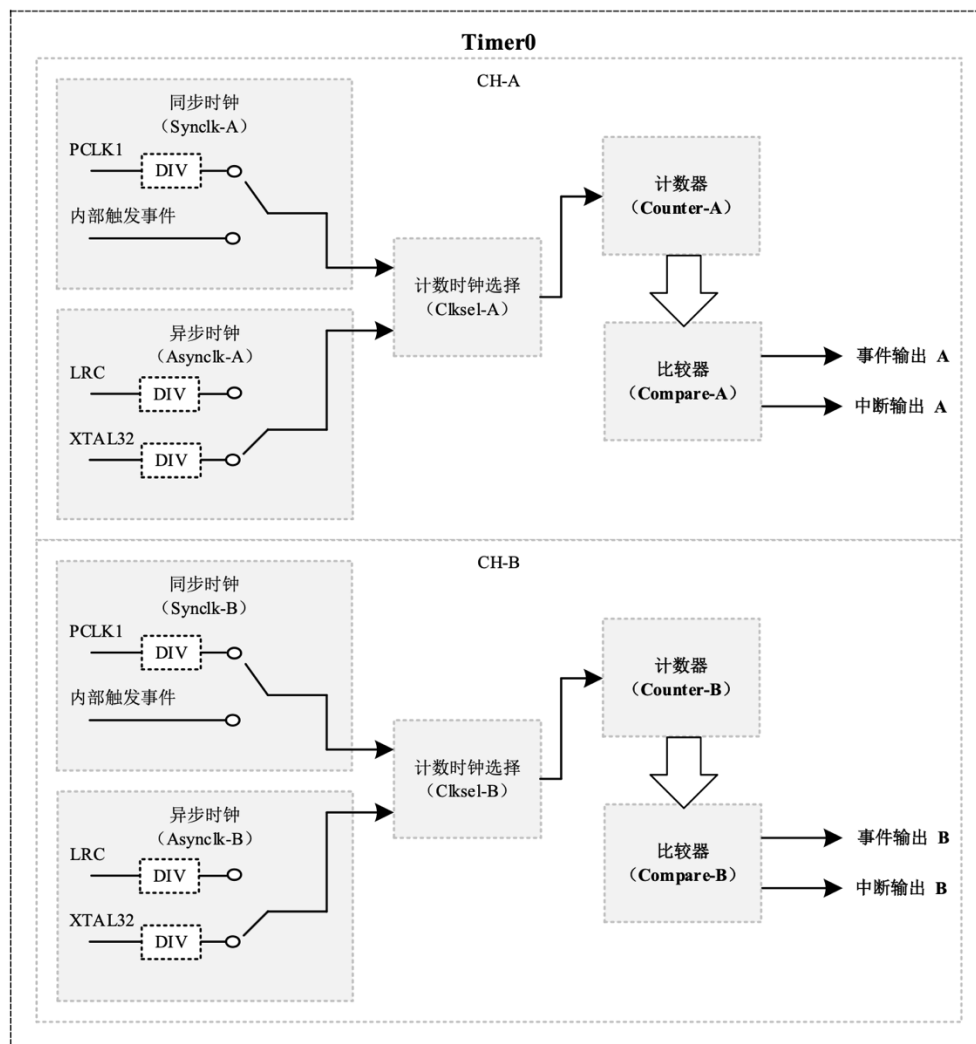


图 4-2-177 Timer0 基本框架图

Timer0 的计数方式可以选择同步计数方式或异步计数方式。同步计数方式是指定时器的计数时钟和总线访问时钟（通过寄存器读写操作时钟）有同步时序关系；异步计数方式是指定时器的计数时钟和总线访问时钟是非同步的时序关系。异步计数方式对寄存器读操作时，定时器等的状态可能正在发生变化，读出不可预期的状态。因此在异步计数方式时，寄存器读操作必须在计数停止状态下实现。

Timer0 的 2 个通道有一个共有的内部硬件触发源，可以通过基本控制寄存器（BCONR）的相关设定来控制定时器的状态（计数，启动，停止，清零）以及捕获输入动作等。该硬件触发源的源选择通过向硬件触发选择寄存器（HTSSR）中输入对应的编号来实现。使用

内部硬件触发功能时，需要先将功能时钟控制寄存器 0（PWC_FCG0）的外围电路触发功能位使能。

2.1.2 中断及事件说明

一个 Timer0 含有 2 个中断输出，分别是通道 A 和通道 B 的计数比较匹配中断或捕获输入中断。基准值寄存器（CMPAR，CMPBR）共两个，客分别与计数值寄存器（CNTAR，CNTBR）比较产生比较匹配有效信号。在这里，仅以通道 A 举例。计数比较匹配时，状态标志寄存器（STFLR）中的 STFLR.CMFA 位会被置 1。此时若设定基本控制寄存器（BCONR）的 BCONR.INTENA 位使能中断，则对应的中断请求也会被触发。

2.1.3 寄存器说明

一．计数值寄存器（TMR_CNTAR）

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
CNTA[15:0]															

- b31~b16: 作为保留位，读的时候为 0，是只读的部分
- b15~b0: 表示的是当前定时器的计数值，是可读可写的

二．基准值寄存器（TMR_CMPAR）

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
CMPA[15:0]															

- b31~b16: 作为保留位，读的时候为 0，是只读的部分
- b15~b0: 设定计数基准值，产生 Compare Match 事件，是可读可写的

三．基本控制寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
HICP B	HCLE B	HSTP B	HSTA B	-	ASYN CLKB	SYN CLKB	SYN SB	CKDIV B[3:0]				-	INT ENB	CAP MDB	CST B
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
HICP A	HCLE A	HSTP A	HSTA A	-	ASYN CLKA	SYN CLKA	SYN SA	CKDIV A[3:0]				-	INT ENA	CAP MDA	CST A

基本控制寄存器中的所有位都是可读可写的。

- b31~b16: 为控制通道 B 的位，功能同控制通道 A 相同
- b15: 为 0 时，条件匹配时，内部硬件触发事件无效，为 1 时则有效

- b14: 为 0 时, 条件匹配时, 定时器清零无效, 为 1 时则有效
- b13: 为 0 时, 条件匹配时, 定时器停止无效, 为 1 时则有效
- b12: 为 0 时, 条件匹配时, 定时器启动无效, 为 1 时则有效
- b11: 保留位, 可读可写, 读出时为 0, 写入时写 0
- b10: 异步计数时钟源选择, 为 0 时选择 LRC, 为 1 时选择 XTAL32
- b9: 同步计数时钟源选择, 为 0 时选择 PLCK1, 为 1 时表示内部硬件触发事件
- b8: 计数方式选择, 为 0 是表示同步计数方式, 为 1 时表示异步计数方式
- b7~b4: 计数时钟分频选择。设定的值的范围为: 0000~1010, 不能设定其他的值。表示的含义为时钟源处以 2 的设定的值的次方
- b3: 保留位, 可读可写, 读出时为 0, 写入时写 0
- b2: 计数匹配中断使能, 为 0 时表示 CMPAR 寄存器与计数值 (CNTAR) 相等时, 或者发生捕获输入事件时, 该中断无效, 为 1 时则改中段使能
- b1: 功能模式选择, 0 表示比较输出功能, 1 表示捕获输入功能
- b0: 为 0 时表示通道 A 定时器关闭, 为 1 时表示定时器启动

该寄存器中提到的内部硬件触发事件及异步计数时的 XTAL32 时钟源在 USART 模块的 TIMEOUT 功能有效时均由 USART 模块提供输入。

四. 触发选择寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
COMEN[1:0]		Reserved													
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved							TRGSEL[8:0]								

- b31: 公共触发使能 1, 0 表示禁止 AOS_COMTRG2 的公共触发事件触发 Timer0, 1 表示允许
- b30: 公共触发使能 0, 0 表示禁止 AOS_COMTRG1 的公共触发事件触发 Timer0, 1 表示允许
- b29~b9: 保留位, 读时读出 0, 写时写入 0
- b8~b0: 表示触发源选择, 将触发源写入之后可触发对应的中断

五. 状态标志寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															CMFB
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved															CMFA

- b31~b17: 作为保留位, 读的时候为 0, 写的时候写入 0
- b16: 通道 B 专用, 作用同 b0
- b15~b1: 作为保留位, 读的时候为 0, 写的时候写入 0
- b0: 计数匹配, 0 表示 CMPAR 寄存器的值与计数器 CNTAR 不相等且未发生捕获输入动作, 1 表示 CMPAR 寄存器的值与计数器 CNTAR 相等或发生捕获输入动作

2.2 TIMER 驱动介绍

2.2.1 bus_hwtimer.h 文件

bus_hwtimer.h 文件中声明了 Timer 总线结构体 HwtimerBus 和驱动结构体 HwtimerDriver。HwtimerBus 的结构体定义如下:

```
struct HwtimerBus{
    struct Bus bus;
    void *private_data;
};
```

其中 bus 是通用管理总线; private_data 是传递参数的通用指针。

HwtimerDriver 的结构体定义如下:

```
struct HwtimerDriver{
    struct Driver driver;
    uint32 (*configure)(void *drv, struct BusConfigureInfo *configure_info);
};
```

其中 driver 表示通用管理驱动; configure 表示配置函数。

另外, 在 bus_hwtimer.h 文件中还定义了一些函数接口, 具体均在 bus_hwtimer.c 文件中实现:

- int HwtimerBusInit(struct HwtimerBus *hwtimer_bus, const char *bus_name), 用于初始化 timer 总线, 其中 hwtimer_bus 表示初始化的 timer 总线; bus_name 表示总线名称。通过调用 BusRegister 函数将 TIMER 总线注册到 BUS 管理总线链表。

- `int HwtimerReleaseBus(struct HwtimerBus *hwtimer_bus)`, 用于释放 Timer 总线, 即将 Timer 总线从 BUS 管理总线链表中移除, 其中 `hwtimer_bus` 表示需要释放的 Timer 总线。一般该接口函数与 `HwtimerBusInit` 函数配对使用, 对 Timer 总线进行注册和释放。
- `int HwtimerDriverInit(struct HwtimerDriver *spi_driver, const char *driver_name)`, 用于初始化 Timer 驱动, 其中 `hwtimer_bus` 表示需要初始化的 Timer 驱动; `driver_name` 表示驱动名称。该接口通过调用 `HwtimerDriverRegister` 函数将 Timer 驱动注册到 Timer 驱动总线。
- `int HwtimerDriverAttachToBus(const char *drv_name, const char *bus_name)`, 用于将 Timer 驱动挂接到 Timer BUS 总线, 其中 `drv_name` 表示驱动名称; `bus_name` 表示总线名称。
- `DriverTypeHwtimerDriverFind(const char *drv_name, enum DriverType_e drv_type)`, 用于查找挂接的 Timer 驱动, 其中 `drv_name` 表示驱动名称; `drv_type` 表示驱动类型。

2.2.2 dev_hwtimer.h 文件

在 `dev_hwtimer.h` 文件中声明了 TIMER 设备结构体 `HwtimerHardwareDevice`, 其具体的定义如下:

```
struct HwtimerHardwareDevice{
    struct HardwareDev haldev;
    struct HwtimerDeviceParam hwtimer_param;
    const struct HwtimerDevDone *dev_done;
    void *private_data;
};
```

其中 `haldev` 用于描述一个设备结构; `hwtimer_param` 用于描述 TIMER 的参数; `dev_done` 用于表示一个 Timer 设备的函数操作集; `private_data` 是传递参数的通用指针。

结构体 `HwtimerDevDone` 的定义如下:

```
struct HwtimerDevDone{
    uint32 (*dev_open) (void *dev);
    uint32 (*dev_close) (void *dev);
    uint32 (*dev_write) (void *dev, struct BusBlockWriteParam *write_param);
    uint32 (*dev_read) (void *dev, struct BusBlockRead Param *read_param);
};
```

可以看出, 该结构体是用于描述 TIMER 设备的打开、关闭、读写功能。

另外，在 `dev_hwtimer.h` 文件中还定义了一些函数接口：

- `int Hwtimer DeviceRegister(struct Hwtimer HardwareDevice * hwtimer_device, void * hwtimer_param, const char *device_name)`，用于初始化并注册 Timer 设备到设备管理链表上，其中 `hwtimer_device` 表示需要注册的 Timer 设备；`hwtimer_param` 表示 Timer 参数；`device_name` 表示设备名称。
- `int HwtimerDeviceAttachToBus(const char *dev_name, const char *bus_name)`，用于将 Timer 设备挂接到 Timer BUS 总线，其中 `dev_name` 表示设备名称；`bus_name` 表示总线名称。
- `HardwareDevTypeHwtimerDeviceFind(const char *dev_name, enum DevType dev_type)`，用于查找挂接的 Timer 设备，其中 `dev_name` 表示设备名称；`dev_type` 表示设备类型。

2.2.3 connect_timer.c 文件

`connect_timer.c` 文件中实现了 timer 驱动及设备的注册，该流程主要包含 3 个步骤：

- （1）完成 timer 的驱动和设备到 timer 总线的挂接（或称为绑定）；
- （2）实现 timer 设备的打开、关闭及读写方法，其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植；
- （3）实现 timer 设备的驱动配置，需结合 timer 特性及操作系统的接口定义进行相应的开发。

步骤（1）的 timer 驱动及设备绑定过程由代码 4-2-1 中的 `BoardHwTimerBusInit` 函数和代码 4-2-2 中的 `BoardHwtimerDevBend` 函数共同完成。`HwTimerInit` 函数主要完成 Timer 驱动的绑定，代码行 10~14 使用 `BoardHwtimerBusInit` 函数对 Timer 总线进行初始化和驱动的绑定。代码行 16~20 使用 `BoardHwtimerDevBend` 函数对设备进行绑定。

`BoardHwtimerDevBend` 函数主要完成 Timer 设备的绑定，代码行 3~5 主要完成参数的声明及初始化；代码行 09~13 通过 `HwtimerDeviceRegister` 函数注册 Timer 设备。代码行 15~19 通过 `HwtimerDeviceAttachToBus` 将 Timer 设备挂接到总线。

代码 4-2-109 Timer 驱动绑定过程

```

01 int HwTimerInit(void)
02 {
03     x_err_t ret = EOK;

```

```

04     static struct HwtimerBus hwtimer_bus;
05     memset(&hwtimer_bus, 0, sizeof(struct HwtimerBus));
06
07     static struct HwtimerDriver hwtimer_driver;
08     memset(&hwtimer_driver, 0, sizeof(struct HwtimerDriver));
09
10     ret = BoardHwtimerBusInit(&hwtimer_bus, &hwtimer_driver);
11     if (EOK != ret) {
12         KPrintf("board_hwtimer_Init error ret %u\n", ret);
13         return ERROR;
14     }
15
16     ret = BoardHwtimerDevBend();
17     if (EOK != ret) {
18         KPrintf("board_hwtimer_Init error ret %u\n", ret);
19         return ERROR;
20     }
21
22     return ret;
23 }

```

代码 4-2-110 设备绑定过程

```

01 static int BoardHwtimerDevBend(void)
02 {
03     x_err_t ret = EOK;
04     static struct HwtimerHardwareDevice hwtimer_device_0;
05     memset(&hwtimer_device_0, 0, sizeof(struct HwtimerHardwareDevice));
06
07     hwtimer_device_0.dev_done = &dev_done;
08
09     ret = HwtimerDeviceRegister(&hwtimer_device_0, NONE,
HWTIMER_0_DEVICE_NAME_0);
10     if (EOK != ret) {
11         KPrintf("BoardHwtimerDevBend HwtimerDeviceRegister device %s
error %d\n", HWTIMER_0_DEVICE_NAME_0, ret);
12         return ERROR;
13     }
14
15     ret = HwtimerDeviceAttachToBus(HWTIMER_0_DEVICE_NAME_0,
HWTIMER_BUS_NAME_0);
16     if (EOK != ret) {

```

```

17      KPrintf("BoardHwtimerDevBend HwtimerDeviceAttachToBus device %s
error %d\n", HWTIMER_0_DEVICE_NAME_0, ret);
18      return ERROR;
19  }
20
21      return ret;
22  }

```

步骤（2）的实现过程如代码 4-2-3 所示，该流程是一个实现 Timer 设备的打开、关闭及读写过程。根据结构体 HwtimerDevDone 的定义，读和写的值均为 NONE，表示该功能未启用。而打开和关闭的值分别是 HwtimerOpen 和 HwtimerClose，表明这两个功能是打开的。

代码 4-2-111 Timer 设备的打开、关闭及读写

```

01  static const struct HwtimerDevDone dev_done =
02  {
03      .open = HwtimerOpen,
04      .close = HwtimerClose,
05      .write = NONE,
06      .read = NONE,
07  };

```

Timer 设备的驱动配置过程如代码 4-2-4 所示。在 HwtimerDrvConfigure 函数中，代码行 2~3 进行输入参数 drv 和 configure_info 的检查。代码行 9~28 根据配置命令进行选择，若配置命令为 OPE_INT，则执行 Timer 驱动初始化；若配置命令为 OPE_CFG，则根据 HwtimerDrvConfigure 函数进行 Timer 驱动配置。

代码 4-2-112 Timer 设备的驱动配置

```

01  static uint32 HwtimerDrvConfigure(void *drv, struct BusConfigureInfo *configure_info)
02  {
03      NULL_PARAM_CHECK(drv);
04      NULL_PARAM_CHECK(configure_info);
05
06      x_err_t ret = EOK;
07      __IO uint32_t *INTC_SELx;
08
09      switch (configure_info->configure_cmd)
10      {
11          case OPE_INT:

```

```

12         INTC_SELx = (__IO uint32_t*)(INTSEL_REG+ 4U *
(uint32_t)(TIMER0_IRQn));
13         WRITE_REG32(*INTC_SELx, EVT_SRC_TMR0_1_CMP_A);
14         callback_function = (void (*)(void *param))configure_info->private_data;
15         isrManager.done->registerIrq(TIMER0_IRQn+16,Timer0Callback,NULL);
16         isrManager.done->enableIrq(TIMER0_IRQn);
17         TMR0_IntCmd(TMR0x, TMR0_INT_CMP_A, ENABLE);
18         break;
19     case OPE_CFG:
20         TMR0_ClearStatus(TMR0x, TMR0_FLAG_CMP_A);
21         TMR0_SetCompareValue(TMR0x, TMR0_CH_x, *((int
*)configure_info->private_data));
22         /* Timer0 interrupt function Enable */
23         TMR0_SetCountValue(TMR0x, TMR0_CH_x, 0x0000);
24         TMR0_Start(TMR0x, TMR0_CH_x);
25         break;
26     default:
27         break;
28     }
29     return ret;
30 }

```

2.3 TIMER 设备应用

timer 设备的应用测试通过测试调用 timer 驱动来进行一次 led 灯的闪烁。代码行 5-9 和 11-15 分别是打开引脚和 timer 的描述符。代码行 17-21 进行 GPIO 引脚的设置。代码行 23-25 进行 IO 配置。代码行 33-39 进行 timer 驱动配置。代码行 55, 56 分别关闭引脚和 timer 的描述符。

代码 4-2-113 Timer 设备应用代码

```

01 void TestHwTimer(void)
02 {
03     x_ticks_t period = 100000;//uint:10ms
04
05     pin_fd = PrivOpen(HWTIMER_PIN_DEV_DRIVER, O_RDWR);
06     if(pin_fd<0){
07         printf("open pin fd error:%d\n",pin_fd);
08         return;
09     }
10

```

```
11     int timer_fd = PrivOpen(HWTIMER_TIMER_DEV_DRIVER, O_RDWR);
12     if(timer_fd<0){
13         printf("open timer fd error:%d\n",timer_fd);
14         return;
15     }
16
17     //config led pin in board
18     struct PinParam parameter;
19     parameter.cmd = GPIO_CONFIG_MODE;
20     parameter.pin = BSP_LED_PIN;
21     parameter.mode = GPIO_CFG_OUTPUT;
22
23     struct PrivIoctlCfg ioctl_cfg;
24     ioctl_cfg.ioctl_driver_type = PIN_TYPE;
25     ioctl_cfg.args = (void *)&parameter;
26
27     if (0 != PrivIoctl(pin_fd, OPE_CFG, &ioctl_cfg)) {
28         printf("ioctl pin fd error %d\n", pin_fd);
29         PrivClose(pin_fd);
30         return;
31     }
32
33     ioctl_cfg.ioctl_driver_type = TIMER_TYPE;
34     ioctl_cfg.args = (void *)&ledflip;
35     if (0 != PrivIoctl(timer_fd, OPE_INT, &ioctl_cfg)) {
36         printf("timer pin fd error %d\n", pin_fd);
37         PrivClose(pin_fd);
38         return;
39     }
40
41     ioctl_cfg.args = (void *)&period;
42     if (0 != PrivIoctl(timer_fd, OPE_CFG, &ioctl_cfg)) {
43         printf("timer pin fd error %d\n", pin_fd);
44         PrivClose(pin_fd);
45         return;
46     }
47
48     while(1){
49
50     }
51
```

```
52      // int32 timer_handle = KCreateTimer("LED on and off by  
1s",&ledflip,&pin_fd,period,TIMER_TRIGGER_PERIODIC);  
53  
54      // KTimerStartRun(timer_handle);  
55      PrivClose(pin_fd);  
56      PrivClose(timer_fd);  
57  }
```

第3章 RTC 驱动

3.1 RTC 介绍

3.1.1 RTC 基本介绍

实时时钟（RTC）是一个以BCD码格式保存时间信息的计数器。记录从00年到99年的具体日历时间。支持12/24小时两种时制，根据月份和年份自动计算日数28，29（闰年），30和31日。在嵌入式系统中，RTC通常是一个独立的芯片，具有自己的电池供电，这样即使系统断电，RTC也能继续工作。RTC可以通过I2C、SPI、串口等接口和主控芯片进行通信，主控芯片可以读取和写入RTC的寄存器来获取当前的时间和日期信息。除了提供时间和日期信息外，RTC还可以用于定时功能，例如定时唤醒系统、定时执行任务等。在实时控制系统中，RTC的精度非常重要，因为它直接影响到系统的稳定性和可靠性。

RTC 的计数时钟源可以有两个，一个是外部低俗振荡器（32.768kHz），另外一个 R TC 内部低俗振荡器（32.768kHz）。

RTC 的基本功能如下：

- BCD 码表示秒，分，时，日，周，月，年时间
- 软件启动或停止
- 12/24 时制可选，闰年自动识别
- 可编程闹钟
- 分布式/均匀式 1Hz 时钟输出
- 时钟误差补偿功能
- 时间戳功能
- 备份寄存器复位功能

RTC 的基本框架如图 4-3-1 所示。

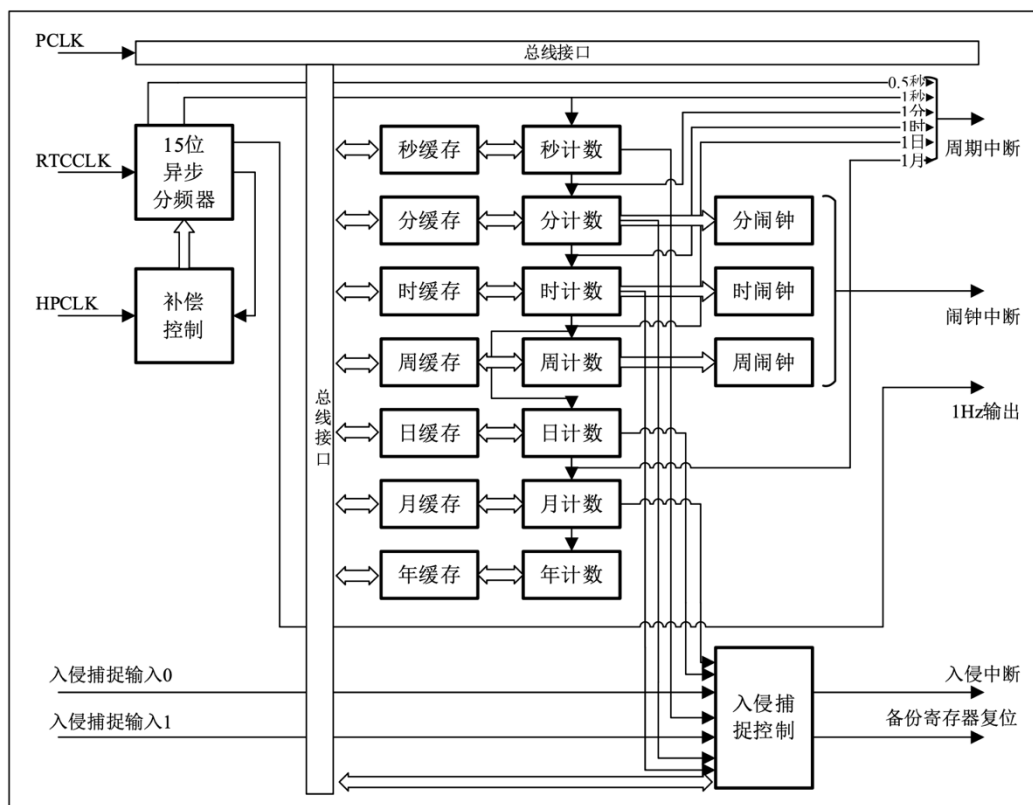


图 4-3-178 RTC 基本框架图

RTC 寄存器值上电复位后不定，上电复位后，必须先设定备份域复位寄存器 PWC VBATSTR 后，再设定控制寄存器 RTC CRORESET 位复位所有寄存器。然后设置控制寄存器，日历初始值、闹钟设置，启动 RTC。RTC 启动之后，其他外部各种复位请求都不能复位 RTC，RTC 会一直处于工作状态。可以设定控制寄存器的 RTC CR1.START 位为“0”停止 RTC 工作。RTC 在进行设定时，必须保证时钟源已稳定。

RTC 计数开始设定的过程如下：

- (1) 上电后，设定 RTC_CR0.RESET=0，确认 RESET 位为 0 后，设定 RTC_CR0.RESET=1，复位所有寄存器
- (2) 设定 RTC_CR1.START=0，停止计数
- (3) 设定系统时钟寄存器，打开外部低速振荡器，再设定 RTC_CR3，选择 RTC 计数时钟源
- (4) 设定 RTC_CR1，设定时制，周期，1Hz 时钟输出
- (5) 设定秒，分，时，周，日，年，月的日历计数寄存器

- (6) 需要进行时钟误差补偿时, 设定计数时钟误差补偿器 RTC_ERRCRL, RTC_ERRCRH
- (7) 清除寄存器 RTC_CR1, RTC_TPSR 中的标志寄存器位
- (8) 设定 RTC_CR1.START=1, 计数开始

由于外部低速晶振在各种温度条件下存在偏差, 在需要得到高精度的计数结果是, 需要对误差进行补偿, 具体方法为操作时钟误差补偿寄存器。

RTC 可输出 1Hz 时钟, 提供三种精度的输出方式。第一种是无时钟补偿的普通精度输出。第二种是每 32 秒内平均补偿的分布式补偿 1Hz 输出。第三种是每秒补偿的均匀式补偿 1Hz 输出。当时钟误差补偿有效 RTC_ERRCRH.COMPEN=1 时, 可选择分布式 1Hz 输出和均匀式补偿 1Hz 输出。

当入侵检测 RTC_TPCRn.TPEN(n-0,1)有效时, RTC 的外部管脚检测到有效的入侵事件后可分别触发入侵事件检测功能, 时间戳功能和备份寄存器复位功能。

入侵事件检测功能: 入侵事件发生时, 对应引脚的入侵标志会被置起, 当入侵事件连续发生时, 上溢标志会被置起。入侵事件 0 和 1 同时发生时, 入侵事件 0 标志置起后经过 1 个 RTC 计数时钟后入侵事件 1 标志再被置起, 同时上溢标志也被置起 RTC.TPCRn.TPIE(n-0,1)入侵事件中断使能时, 将发生入侵检测中断。

时间戳功能: RTC.TPCRn.TSTPE(n-0,1)时间戳功能有效时, 时间戳寄存器将记录入侵事件发生时的秒, 分, 时, 日, 月, 日历年时间。

备份寄存器复位: RTC.TPCRn.TPRSTE(n=0,1)备份寄存器复位使能时, 发生入侵事件后将复位备份寄存器。备份寄存器包括 32 个 32 位寄存器, 用于存储用户 128 字节的备份数据。

当入侵检测 RTC_TPCRn.TPEN(n-0,1)有效时, 且时间戳记录功能 RTC_TPCRn.TSTPE(n=0,1)有效时, RTC 的外部管脚检测到有效的入侵事件后将触发时间戳功能。时间戳寄存器将记录秒, 分, 时, 日, 月, 日历年时间。一组时间戳寄存器分别对应两个入侵事件输入管脚。当管脚在第一次发生入侵事件时, 入侵标志置起, 时间戳被记录, 再次发生入侵事件时, 则上溢标志位被置起, 但是时间戳寄存器仍然保持第一次的记录值。入侵标志寄存器被清除后, 入侵检测发生时, 时间戳才会再次被记录

3.1.2 中断说明

RTC 支持三种中断类型: 计时闹钟中断, 定周期中断, 入侵检测中断。

闹钟中断 RTC ALM，在控制寄存器 2(RTC CR2)的 ALMIE-1 并且控制寄存器 2(RTC CR2)的 ALME-1 时，若当前日历时间与分闹钟寄存器(RTC ALMMIN)、时闹钟寄存器(RTC ALMHOUR)、周闹钟寄存器(RTC ALMWEED)相等时，触发闹钟中断。闹钟配置独立的标志寄存器位 RTC CR2.ALMF,对 RTC CR2.ALMF 位写“0”清除闹钟标志。

定周期中断 RTC PRD，控制寄存器 2(RTC CR2)的 PRDIE-1 时，选择的周期发生后，触发定周期唤醒中断。定周期中断配置独立的标志寄存器 RTC CR2.PRDF，可通过对 RTC CR2.PRDF 位写“0”清除定周期标志。

当入侵检测有效 RTC TPCRn.TPEN(n=0,1)，且 RTC.TPCRn.TPIE(n=0,1)入侵事件中断使能时，RTC 的外部管脚检测到有效的入侵事件后，将发生入侵事件中断。由于两组入侵检测共用一个中断，可通过入侵事件检测中断标志位 RTC TPSR.TPFn(n=0,1)来区分。

3.1.3 寄存器说明

一. 控制寄存器 0

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved															RESET

- b31~b1：作为保留位，读的时候为 0，写的时候写 0
- b0：在写入状态时：0 初始化寄存器无效，1 表示初始化寄存器有效

在读出状态时：0 表示正常计数状态活 RTC 软件复位结束，1 表示正处于复位状态

二. 控制寄存器 1

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved								START	ONEHZSEL	ONEHZOE	-	AMPM	PRDS[2:0]		

- b31~b8：作为保留位，读的时候为 0，写的时候写 0
- b7：0 表示 RTC 计数停止，1 表示 RTC 计数开始
- b6：当 RTC_ERRCRH.COMPEN=1 时，该位设定有效。值为 0 时表示分布式补偿 1Hz 输出，为 1 时表示均匀式补偿 1Hz 输出
- b5：值为 0 式表示 1Hz 输出禁止，值为 1 式表示 1Hz 输出许可
- b4：作为保留位，读的时候为 0，写的时候写 0

- b3: 值为 0 式表示 12 小时制, 值为 1 式表示 24 小时制
- b2~b0: 表示周期选择设定。值为 000 时表示不选择周期, 值为 001 时表示是每 0.5 秒周期, 值为 010 时表示每 1 秒周期, 值为 011 时表示每 1 分周期, 值为 100 时表示每 1 时周期, 值为 101 时表示每 1 日周期, 1 值为 10 和 111 时表示每 1 月周期

三. 控制寄存器 2

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved								ALME	ALMIE	PRDIE	-	ALMF	PRDF	RWEN	RWREQ

- b31~b8: 作为保留位, 读的时候为 0, 写的时候写 0
- b7: 0 表示闹钟功能禁止, 1 表示闹钟功能许可
- b6: 0 表示闹钟中断禁止, 1 表示闹钟中断许可
- b5: 0 表示周期中断禁止, 1 表示周期中断许可
- b4: 作为保留位, 读的时候为 0, 写的时候写 0
- b3: 0 表示闹钟不匹配, 1 表示闹钟匹配
- b2: 0 表示周期不发生, 1 表示周期发生
- b1: 0 表示读出/写入禁止, 1 表示读出/写入许可
- b0: 0 表示正常计数模式, 1 表示读出/写入模式

四. 控制寄存器 3

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved								RCKSEL	-	-	LRCEN	-		-	

- b31~b8: 作为保留位, 读的时候为 0, 写的时候写 0
- b7: 0 表示选择外部低速发震器 XTAL32 时钟, 1 表示选择内部低速发震器 RTC LRC 时钟
- b6: 作为保留位, 读的时候为 0, 写的时候写 0
- b5: 作为保留位, 读的时候为 0, 写的时候写 0
- b4: 内部低速发震器使能, 0 表示内部低速发震器 RTCLRC 停止, 1 表示内部低速发震器 RTCLRC 工作

- b3~b1: 作为保留位, 读的时候为 0, 写的时候写 0
- b0: 作为保留位, 读的时候为 0, 写的时候写 0

五. 秒计数寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved								SECD[2:0]				SECU[3:0]			

- b31~b7: 作为保留位, 读的时候为 0, 写的时候写 0
- b6~b4: 秒十位计数值
- b3~b0: 秒个位计数值

六. 分计数寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved								MIND[2:0]				MINU[3:0]			

- b31~b7: 作为保留位, 读的时候为 0, 写的时候写 0
- b6~b4: 分十位计数值
- b3~b0: 分个位计数值

七. 时计数寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved								HOURLD[1:0]				HOURU[3:0]			

- b31~b7: 作为保留位, 读的时候为 0, 写的时候写 0
- b6~b4: 时十位计数值
- b3~b0: 时个位计数值

八. 日计数寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved								DAYD[1:0]				DAYU[3:0]			

- b31~b7: 作为保留位, 读的时候为 0, 写的时候写 0
- b6~b4: 日十位计数值

- b3~b0: 日个位计数值

九. 周计数寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved												WEEK[2:0]			

- b31~b7: 作为保留位, 读的时候为 0, 写的时候写 0
- b6~b4: 周十位计数值
- b3~b0: 周个位计数值

十. 月计数寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved											MON[4:0]				

- b31~b5: 作为保留位, 读的时候为 0, 写的时候写 0
- b4~b0: 月计数值

十一. 年计数寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved								YEAR[3:0]				YEARU[3:0]			

- b31~b7: 作为保留位, 读的时候为 0, 写的时候写 0
- b6~b4: 年十位计数值
- b3~b0: 年个位计数值

十二. 时钟误差补偿寄存器 (RTC_ERRCRH)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved								COMPEN	-	-	-	-	-	-	COMP[8]

- b31~b8: 作为保留位, 读的时候为 0, 写的时候写 0
- b7: 补偿使能, 0 表示时钟误差补偿无效, 1 表示时钟误差补偿有效
- b6~b1: 作为保留位, 读的时候为 0, 写的时候写 0
- b0: 和 RTC_ERRCRL 中的低 8 位一起设定补偿值

十三. 时钟误差补偿寄存器 (RTC_ERRCRL)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved								COMP[7:0]							

- b31~b8: 作为保留位，读的时候为 0，写的时候写 0
- b7~b0: 通过补偿值设定，可针对每秒进行 $\pm 0.96\text{ppm}$ 的精度补偿。补偿值为 9 位带小数点的 2 的补码，后五位为小数部分。可补偿范围为 $-275.\text{ppm}\sim +212.9\text{ppm}$ 。最小分辨率 0.96ppm 。具体补偿粒度和补偿值的设定请参考 HC32F4A0 系列参考手册。

十四. 入侵控制寄存器 0

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved								TPEN0	TSTPE0	TPIE0	TPRSTE0	TPNF0[1:0]	TPCT0[1:0]		

- b31~b8: 作为保留位，读的时候为 0，写的时候写 0
- b7: 入侵检测使能，0 表示 RTCIC0 输入引脚上入侵功能无效，1 表示 RTCIC0 输入引脚上入侵功能有效
- b6: 时间戳使能，0 表示时间戳功能禁止，1 表示时间戳功能允许
- b5: 入侵事件中断使能，0 表示入侵事件中断禁止，1 表示入侵事件中断许可
- b4: 入侵事件复位使能，0 表示备份寄存器复位禁止，1 表示备份寄存器复位许可
- b3~b2: 值为 00 或 01 时表示滤波功能无效，10 表示 RTCIC0 输入引脚检测按照计时时钟进行 3 次一致性检测滤波，11 表示 RTCIC0 输入引脚检测按照计时时钟 32 分频进行 3 此一致性检测滤波
- b1~b0: 值为 00 时表示 RTCIC0 输入引脚不检测，01 表示 RTCIC0 输入引脚检测到上升沿时有效，10 表示 RTCIC0 输入引脚检测到下降沿时有效，11 表示 RTCIC0 输入引脚检测到上升沿或者下降沿时有效

十五. 入侵控制寄存器 1

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved								TPEN1	TSTPE1	TPIE1	TPRSTE1	TPNF1[1:0]	TPCT1[1:0]		

- b31~b8: 作为保留位，读的时候为 0，写的时候写 0

- b7: 入侵检测使能, 0 表示 RTCIC1 输入引脚上入侵功能无效, 1 表示 RTCIC1 输入引脚上入侵功能有效
- b6: 时间戳使能, 0 表示时间戳功能禁止, 1 表示时间戳功能允许
- b5: 入侵事件中断使能, 0 表示入侵事件中断禁止, 1 表示入侵事件中断许可
- b4: 入侵事件复位使能, 0 表示备份寄存器复位禁止, 1 表示备份寄存器复位许可
- b3~b2: 值为 00 或 01 时表示滤波功能无效, 10 表示 RTCIC1 输入引脚检测按照计时时钟进行 3 次一致性检测滤波, 11 表示 RTCIC1 输入引脚检测按照计时时钟 32 分频进行 3 此一致性检测滤波
- b1~b0: 值为 00 时表示 RTCIC0 输入引脚不检测, 01 表示 RTCIC1 输入引脚检测到上升沿时有效, 10 表示 RTCIC1 输入引脚检测到下降沿时有效, 11 表示 RTCIC1 输入引脚检测到上升沿或者下降沿时有效

十六. 入侵状态寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved												TPOVF		TPF1	TPF0

- b31~b3: 作为保留位, 读的时候为 0, 写的时候写 0
- b2: 入侵状态上溢标志, 0 表示复位状态或未发生入侵状态上溢, 1 表示发生入侵上溢
- b1: 入侵状态标志 1, 0 表示 RTCIC1 输入引脚上未发生入侵事件, 1 表示输入引脚上发生入侵事件
- b0 入侵状态标志 0, 0 表示 RTCIC0 输入引脚上未发生入侵事件, 1 表示输入引脚上发生入侵事件

十七. 秒时间戳寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved									SECTPD[2:0]			SECTPU[3:0]			

- b31~b7: 作为保留位, 读的时候为 0, 写的时候写 0
- b6~b4: 秒时间戳十位计数值
- b3~b0: 秒时间戳个位计数值

十八. 分时间戳寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved								MINTPD[2:0]			MINTPU[3:0]				

- b31~b7: 作为保留位, 读的时候为 0, 写的时候写 0
- b6~b4: 分时间戳十位计数值
- b3~b0: 分时间戳个位计数值

十九. 时时间戳寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved								HOURTPD[1:0]			HOURTPU[3:0]				

- b31~b7: 作为保留位, 读的时候为 0, 写的时候写 0
- b6~b4: 时时间戳十位计数值
- b3~b0: 时时间戳个位计数值

二十. 日时间戳寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved								DAYTPD[1:0]			DAYTPU[3:0]				

- b31~b7: 作为保留位, 读的时候为 0, 写的时候写 0
- b6~b4: 日时间戳十位计数值
- b3~b0: 日时间戳个位计数值

二十一. 月时间戳寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved										MONTTP[4:0]					

- b31~b5: 作为保留位, 读的时候为 0, 写的时候写 0
- b4~b0: 时间戳计数值

3.2 RTC 驱动介绍

RTC 设备通过 RTC BUS 挂接到 BUS 总线上, 其中 RTC BUS 的结构体在 bus_rtc.h 文件中进行了定义。每个设备总线都关联一个 drv 和一个 dev, 其中, drv 结构体也在 bus_rtc.

h 中进行了定义，dev 结构体在 dev_rtc.h 中进行了定义。另外，RTC 驱动及设备的注册流程在 connect_rtc.c 中进行了实现。接下来，针对这 3 个 RTC 相关的具体结构和实现流程进行详细介绍。

3.2.1 bus_rtc.h 文件

bus_rtc.h 文件中声明了 RTC 总线结构体 RtcBus 和驱动结构体 RtcDriver。RtcBus 的结构体定义如下：

```
struct RtcBus{
    struct Bus bus;
    void *private_data;
};
```

其中 bus 是通用管理总线；private_data 是传递参数的通用指针。

RtcDriver 的结构体定义如下：

```
struct RtcDriver{
    struct Driver driver;
    uint32 (*configure)(void *drv, struct BusConfigureInfo *configure_info);
};
```

其中 driver 表示通用管理驱动；configure 表示配置函数。

另外，在 bus_rtc.h 文件中还定义了一些函数接口，具体均在 bus_rtc.c 文件中实现：

- int RtcBusInit(struct RtcBus *rtc_bus, const char *bus_name)，用于初始化 RTC 总线，其中 rtc_bus 表示初始化的 RTC 总线；bus_name 表示总线名称。通过调用 BusRegister 函数将 RTC 总线注册到 BUS 管理总线链表。
- int RtcReleaseBus(struct RtcBus *rtc_bus)，用于释放 RTC 总线，即将 RTC 总线从 BUS 管理总线链表中移除，其中 rtc_bus 表示需要释放的 RTC 总线。一般该接口函数与 RtcBusInit 函数配对使用，对 RTC 总线进行注册和释放。
- int RtcDriverInit(struct RtcDriver *rtc_driver, const char *driver_name)，用于初始化 RTC 驱动，其中 rtc_driver 表示需要初始化的 RTC 驱动；driver_name 表示驱动名称。该接口通过调用 RtcDriverRegister 函数将 RTC 驱动注册到 RTC 驱动总线。
- int RtcDriverAttachToBus(const char *drv_name, const char *bus_name)，用于将 RTC 驱动挂接到 RTC BUS 总线，其中 drv_name 表示驱动名称；bus_name 表示总线名称。

- `DriverType RtcDriverFind(const char *drv_name, enum DriverType_e drv_type)`, 用于查找挂接的 RTC 驱动, 其中 `drv_name` 表示驱动名称; `drv_type` 表示驱动类型。

3.2.2 dev_rtc.h 文件

在 `dev_rtc.h` 文件中声明了 RTC 设备结构体 `RtcHardwareDevice`, 其具体的定义如下:

```
struct RtcHardwareDevice{
    struct HardwareDev haldev;
    const struct RtcDevDone *dev_done;
    void *private_data;
};
```

其中 `haldev` 用于描述一个设备结构; `dev_done` 用于表示一个 RTC 设备的函数操作集; `private_data` 是传递参数的通用指针。

结构体 `RtcDevDone` 的定义如下:

```
struct RtcDevDone{
    uint32 (*open) (void *dev);
    uint32 (*close) (void *dev);
    uint32 (*write) (void *dev, struct BusBlockWriteParam *datacfg);
    uint32 (*read) (void *dev, struct BusBlockReadParam *datacfg);
};
```

可以看出, 该结构体是用于描述 RTC 设备的打开、关闭、读写功能。

另外, 在 `dev_rtc.h` 文件中还定义了一些函数接口:

- `int RtcDeviceRegister(struct RtcHardwareDevice *rtc_device, void *rtc_param, const char *device_name)`, 用于初始化并注册 RTC 设备到设备管理链表上, 其中 `rtc_device` 表示需要注册的 RTC 设备; `rtc_param` 表示 RTC 参数; `device_name` 表示设备名称。
- `int RtcDeviceAttachToBus(const char *dev_name, const char *bus_name)`, 用于将 RTC 设备挂接到 RTC BUS 总线, 其中 `dev_name` 表示设备名称; `bus_name` 表示总线名称。
- `HardwareDevType RtcDeviceFind(const char *dev_name, enum DevType dev_type)`, 用于查找挂接的 RTC 设备, 其中 `dev_name` 表示设备名称; `dev_type` 表示设备类型。

3.2.3 connect_rtc.c 文件

`connect_rtc.c` 文件中实现了 RTC 驱动及设备的注册, 该流程主要包含 3 个步骤:

- (1) 完成 RTC 的驱动和设备到 RTC 总线的挂接 (或称为绑定);

(2) 实现 RTC 设备的打开、关闭及读写方法，其具体实现需结合 SoC 厂商提供的 SD K 源码进行相应的移植；

(3) 实现 RTC 设备的驱动配置，需结合 RTC 特性及操作系统的接口定义进行相应的开发。

步骤 (1) 的实现过程如代码 4-3-1 所示，该流程是完成一个 RTC 驱动及设备总线绑定的过程。代码行 2~8 定义了结果标识符变量和总线及驱动结构体类型变量，并对结构体变量进行初始化。代码行 10 对 RTC 的配置函数进行封装，代码行 12~16 使用 BoardRtcBusInit 函数进行 RTC 驱动挂接到 RTC 总线，具体的实现过程详见代码 4-3-2。代码行 18~21 使用 BoardRtcDevBend 函数进行 RTC 设备挂接到 RTC 总线，具体的实现过程详见代码 4-3-3。代码行 23~33 对 RTC 硬件控制器进行配置初始化并使能。

代码 4-3-114 RTC 驱动及设备绑定

```

01  int HwRtcInit(void){
02      x_err_t ret = EOK;
03
04      static struct RtcBus rtc_bus;
05      memset(&rtc_bus, 0, sizeof(struct RtcBus));
06
07      static struct RtcDriver rtc_driver;
08      memset(&rtc_driver, 0, sizeof(struct RtcDriver));
09
10      rtc_driver.configure = &(RtcConfigure);
11
12      ret = BoardRtcBusInit(&rtc_bus, &rtc_driver);
13      if (EOK != ret) {
14          KPrintf("HwRtcInit error ret %u\n", ret);
15          return ERROR;
16      }
17
18      ret = BoardRtcDevBend();
19      if (EOK != ret) {
20          KPrintf("HwRtcInit error ret %u\n", ret);
21      }
22
23      stc_rtc_init_t stcRtcInit;
24      /* Configure structure initialization */

```

```

25     (void)RTC_StructInit(&stcRtcInit);
26
27     /* Configuration RTC structure */
28     stcRtcInit.u8ClockSrc      = RTC_CLK_SRC_XTAL32;
29     stcRtcInit.u8HourFormat    = RTC_HOUR_FMT_24H;
30     stcRtcInit.u8IntPeriod     = RTC_INT_PERIOD_PER_SEC;
31     (void)RTC_Init(&stcRtcInit);
32
33     RTC_Cmd(LL_RTC_ENABLE);
34
35     return ret;
36 }

```

在代码 4-3-2 的 BoardRtcBusInit 函数中，代码行 4~9 对 RTC 总线进行初始化；代码行 11~16 对 RTC 驱动进行初始化；代码行 18~23 将 RTC 驱动挂接到 RTC 总线上。在代码 4-3-3 的 BoardRtcDevBend 函数中，代码行 2~5 定义了 1 个结果返回变量和 1 个结构体变量，并对结构体变量进行初始化；代码行 9~13 中对 RTC 设备进行注册；代码行 15~19 将 RTC 设备挂接到 RTC 总线。

代码 4-3-115 RTC 驱动挂接函数

```

01 static int BoardRtcBusInit(struct RtcBus *rtc_bus, struct RtcDriver *rtc_driver){
02     x_err_t ret = EOK;
03
04     /*Init the rtc bus */
05     ret = RtcBusInit(rtc_bus, RTC_BUS_NAME);
06     if (EOK != ret) {
07         KPrintf("HwRtcInit RtcBusInit error %d\n", ret);
08         return ERROR;
09     }
10
11     /*Init the rtc driver*/
12     ret = RtcDriverInit(rtc_driver, RTC_DRV_NAME);
13     if (EOK != ret) {
14         KPrintf("HwRtcInit RtcDriverInit error %d\n", ret);
15         return ERROR;
16     }
17
18     /*Attach the rtc driver to the rtc bus*/
19     ret = RtcDriverAttachToBus(RTC_DRV_NAME, RTC_BUS_NAME);

```

```

20     if (EOK != ret) {
21         KPrintf("HwRtcInit RtcDriverAttachToBus error %d\n", ret);
22         return ERROR;
23     }
24
25     return ret;
26 }

```

代码 4-3-116 RTC 设备挂接过程

```

01 static int BoardRtcDevBend(void){
02     x_err_t ret = EOK;
03
04     static struct RtcHardwareDevice rtc_device;
05     memset(&rtc_device, 0, sizeof(struct RtcHardwareDevice));
06
07     rtc_device.dev_done = &(dev_done);
08
09     ret = RtcDeviceRegister(&rtc_device, NONE, RTC_DEVICE_NAME);
10     if (EOK != ret) {
11         KPrintf("HwRtcInit RtcDeviceInit device %s error %d\n",
12 RTC_DEVICE_NAME, ret);
13         return ERROR;
14     }
15
16     ret = RtcDeviceAttachToBus(RTC_DEVICE_NAME, RTC_BUS_NAME);
17     if (EOK != ret) {
18         KPrintf("HwRtcInit RtcDeviceAttachToBus device %s error %d\n",
19 RTC_DEVICE_NAME, ret);
20         return ERROR;
21     }
22     return ret;
23 }

```

步骤（2）的实现过程如代码 4-3-4 所示，该流程是一个实现 RTC 设备的打开、关闭及读写过程。根据结构体 RtcDevDone 的定义，此时打开、关闭和读写功能的值均为 NONE，表示未启用这些功能。

代码 4-3-117 RTC 设备的打开、关闭及读写

```

01 static const struct RtcDevDone dev_done =

```

```

02  {
03      .open = NONE,
04      .close = NONE,
05      .write = NONE,
06      .read = NONE,
07  };

```

RTC 设备的驱动配置过程如代码 4-3-5 所示。RtcConfigure 函数主要完成两个工作，在代码行 12~33 中，若 cmd 的值为 OPER_RTC_GET_TIME，则获取当前时钟的值；在代码行 34~55 中，若 cmd 的值为 OPER_RTC_SET_TIME，则设置当前的时钟值。

代码 4-3-118 RTC 设备的配置过程

```

01  static uint32 RtcConfigure(void *drv, struct BusConfigureInfo *configure_info){
02      NULL_PARAM_CHECK(drv);
03
04      struct RtcDriver *rtc_drv = (struct RtcDriver *)drv;
05      struct RtcDrvConfigureParam *drv_param =
06      (struct RtcDrvConfigureParam *)configure_info->private_data;
07
08      int cmd = drv_param->rtc_operation_cmd;
09      time_t *time = drv_param->time;
10
11      switch (cmd)
12      {
13          case OPER_RTC_GET_TIME:
14          {
15              struct tm ct;
16              stc_rtc_date_t rtc_date;
17              stc_rtc_time_t rtc_time;
18
19              // rtc_timer_get(&year, &month, &day, &hour, &minute, &second);
20              RTC_GetDate(RTC_DATA_FMT_DEC, &rtc_date);
21              RTC_GetTime(RTC_DATA_FMT_DEC, &rtc_time);
22
23              ct.tm_year = rtc_date.u8Year ;
24              ct.tm_mon = rtc_date.u8Month ;
25              ct.tm_mday = rtc_date.u8Day;
26              ct.tm_wday = rtc_date.u8Weekday;
27
28              ct.tm_hour = rtc_time.u8Hour;

```

```

28         ct.tm_min = rtc_time.u8Minute;
29         ct.tm_sec = rtc_time.u8Second;
30
31         *time = mktime(&ct);
32     }
33     break;
34     case OPER_RTC_SET_TIME:
35     {
36         struct tm *ct;
37         stc_rtc_date_t rtc_date;
38         stc_rtc_time_t rtc_time;
39         x_base lock;
40
41         lock = CriticalAreaLock();
42         ct = localtime(time);
43         rtc_date.u8Year = ct->tm_year ;
44         rtc_date.u8Month = ct->tm_mon ;
45         rtc_date.u8Day = ct->tm_mday;
46         rtc_date.u8Weekday = ct->tm_wday;
47         rtc_time.u8Hour = ct->tm_hour;
48         rtc_time.u8Minute = ct->tm_min;
49         rtc_time.u8Second = ct->tm_sec;
50         CriticalAreaUnLock(lock);
51
52         RTC_SetDate(RTC_DATA_FMT_DEC, &rtc_date);
53         RTC_SetTime(RTC_DATA_FMT_DEC, &rtc_time);
54     }
55     break;
56 }
57 return EOK;
58 }

```

3.3 RTC 设备应用

RTC 设备应用实现过程如代码 4-3-6 所示。代码行 1~3 需要导入 3 个包括标准库在内的头文件。在测试函数 TestRTC 中，代码行 6~10 通过 open 函数打开一个 RTC 设备，如果打开不成功，则直接 return 返回并打印错误提示。代码行 12~14 为解析命令参数，使得打印次数与第一个参数一致。命令行 16 至 24 定义并配置了 RtcDrvConfigureParam 结构体变量；代码行 26 通过 ioctl 函数（即 IO 控制函数）对打开的 RTC 设备调用配置接口对时间进行设

置。代码行 28~34 行从 RTC 设备间隔 5 秒获取当前时间，并打印获取的当前时间，结果以秒为单位。代码行 36~37 分别用于关闭 TRC 和函数返回。

代码 4-3-119 RTC 设备应用

```

01  #include <stdio.h>
02  #include <string.h>
03  #include <transform.h>
04
05  void TestRTC(int argc,char *argv[]){
06      int rtc_fd = PrivOpen(RTC_DEV_DRIVER, O_RDWR);
07      if(rtc_fd<0){
08          printf("open rtc fd error:%d\n",rtc_fd);
09          return;
10      }
11
12      if(argc>1){
13          int times = atoi(argv[1]);
14          printf("Time will be printf %d times\n",times);
15
16          struct RtcDrvConfigureParam rtc_para;
17          time_t my_time=0;
18
19          rtc_para.rtc_operation_cmd = OPER_RTC_SET_TIME;
20          rtc_para.time = &my_time;
21
22          struct PrivIoctlCfg ioctl_cfg;
23          ioctl_cfg.ioctl_driver_type = RTC_TYPE;
24          ioctl_cfg.args = (void *)&rtc_para;
25
26          PrivIoctl(rtc_fd,0,&ioctl_cfg);
27
28          rtc_para.rtc_operation_cmd = OPER_RTC_GET_TIME;
29          for(size_t i=0;i<times;i++){
30              PrivIoctl(rtc_fd,0,&ioctl_cfg);
31              printf("The time now is %d\n",*(rtc_para.time));
32              PrivTaskDelay(5000);
33          }
34      }
35
36      PrivClose(rtc_fd);

```

```
37     return;  
38 }
```

第4章 WATCHDOG 驱动

4.1 WATCHDOG 介绍

4.1.1 WATCHDOG 基本介绍

WATCHDOG是嵌入式开发中的一种重要机制，它用于监视系统的运行状态，并在系统出现故障或死锁时进行自动重启。WATCHDOG通常也是一个独立的硬件模块，它会定时向主控芯片发送一个计时器信号，如果主控芯片在一定时间内没有喂狗（即没有重置计时器），WATCHDOG就会认为系统出现了故障，然后自动重启系统。WATCHDOG可以有效地提高嵌入式系统的可靠性和稳定性。在一些关键应用场景下，比如飞行控制系统、医疗设备、工业自动化控制系统等，WATCHDOG的作用尤为重要。如果系统出现故障或死锁，WATCHDOG可以及时地重启系统，避免造成不可挽回的损失。

HC32F4A0 中看门狗计数器有两个，一种是计数时钟源为内部专用 RC 专用的 SWDT，另一种是计数时钟源为 PCLK3 的通用看门狗计数器（WDT）。专用看门狗和通用看门狗是 16 位递减计数器，用来检测由于外部干扰或不可预见的逻辑条件造成的应用程序背离正常的运行而产生的软件故障。两个看门狗都支持窗口功能，在计数开始前可预设窗口时间，计数值位于窗口区间时，可刷新计数器，计数重新开始。在这里仅对通用看门狗做介绍。

启动看门狗的方式有两种：硬件启动和软件启动。硬件启动是指启动时从主闪存区域读取看门狗计数器的设定信息（ICG0 寄存器），计数器自动开始计数；软件启动方式是指设定控制寄存器后，写刷新寄存器完成刷新动作，计数器开始计数。

ICG0 寄存器的位 16 和位 0 为 1 时，通过刷新寄存器的方式启动 WDT 为软件启动方式。复位后，设定 WDT_CR 寄存器中的计数时钟，窗口设定值，计数周期等等，然后执行刷新动作，计数器就开始计数。WDT_CR 设定只能 1 次，再次写入值无效。

4.1.2 中断说明

WDT 在计数器计数下溢或刷新错误时，可以选择产生中断请求或者复位请求。硬件启动模式下，通过 ICG0 的 WDTITS 位，来决定产生中断请求还是复位请求。软件启动模式下，通过设定 WDT_CR 寄存器 ITS 位，来决定产生中断还是复位请求。WDT 的中断复位

产生条件有两种，一种是计数器产生下溢，一种是允许区间之外执行刷新动作，产生刷新错误。

4.1.3 寄存器说明

一. 控制寄存器（WDT_CR）

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
ITS	-	-	-	-	-	-	-	-	-	-	-	-	-	-	SLPOFF
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	-	WDPT				CKS[3:0]				-	-	PERI[1:0]	

- b31：表示出现刷新错误时是进行中断请求还是复位请求，0 表示中断请求，1 表示复位请求。
- b30~b17：作为保留位，读的时候为 0，写的时候写 0
- b16：表示 WDT 在低功耗模式下计数是否禁止，1 表示 WDT 在 sleep 模式下计数许可，0 表示 WDT 在 sleep 模式下计数禁止
- b15~b12：作为保留位，读的时候为 0，写的时候写 0
- b11~b8：表示刷新允许区域计数值百分比
- b7~b4：表示计数时钟
- b3~b2：作为保留位，读的时候为 0，写的时候写 0
- b1~b0：表示计数周期，00 表示 256 周期，01 表示 4096 周期，10 表示 16384 周期，11 表示 65536 周期

二. 状态寄存器（WDT_SR）

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	-	-	-	-	-	-	REF	UDF
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
CNT[15:0]															

- b31~b18：作为保留位，读的时候为 0，写的时候写 0
- b17：刷新错误标志，0 表示没有刷新错误，1 表示发生刷新错误
- b16：计数下溢标志，0 表示没有计数下溢，1 表示发生计数下溢
- b15~b0：表示计数器当前计数值

三. 刷新寄存器（WDT_RR）

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
RF[15:0]															

- b31~b16: 作为保留位，读的时候为 0，写的时候写 0
- b4~b0: 需要一次写入 0x0123,0x3210 之后完成刷新动作，当寄存器写入 0x0123 h 后，读出寄存器的值为 0x00000001，其余情况读出来为 0x00000000

4.2 WATCHDOG 驱动介绍

WATCHDOG 设备通过 WATCHDOG BUS 挂接到 BUS 总线上，其中 WATCHDOG BUS 的结构体在 bus_wdt.h 文件中进行了定义。每个设备总线都关联一个 drv 和一个 dev，其中，drv 结构体也在 bus_wdt.h 中进行了定义，dev 结构体在 dev_wdt.h 中进行了定义。另外，SPI 驱动及设备的注册流程在 connect_wdt.c 中进行了实现。接下来，针对这 3 个 SPI 相关的具体结构和实现流程进行详细介绍。

4.2.1 bus_wdt.h 文件

bus_wdt.h 文件中声明了 WATCHDOG 总线结构体 WdtBus 和驱动结构体 WdtDriver。

WdtBus 的结构体定义如下：

```
struct WdtBus{
    struct Bus bus;
    void *private_data;
};
```

其中 bus 是通用管理总线；private_data 是传递参数的通用指针。

AdcDriver 的结构体定义如下：

```
struct WdtDriver{
    struct Driver driver;
    uint32 (*configure) (void *drv, struct BusConfigureInfo *ConfigureInfo);
    void *private_data;
};
```

其中 driver 表示通用管理驱动；configure 表示配置函数；private_data 是传递参数的通用指针。

另外，在 bus_wdt.h 文件中还定义了一些函数接口，具体均在 bus_wdt.c 文件中实现：

- `int WdtBusInit(struct WdtBus *wdt_bus, const char *bus_name)`, 用于初始化 Watchdog 总线, 其中 `wdt_bus` 表示初始化的 Watchdog 总线; `bus_name` 表示总线名称。通过调用 `BusRegister` 函数将 Watchdog 总线注册到 BUS 管理总线链表。
- `int WdtReleaseBus(struct WdtBus *wdt_bus)`, 用于释放 Watchdog 总线, 即将 Watchdog 总线从 BUS 管理总线链表中移除, 其中 `wdt_bus` 表示需要释放的 Watchdog 总线。一般该接口函数与 `WdtBusInit` 函数配对使用, 对 Watchdog 总线进行注册和释放。
- `int WdtDriverInit(struct WdtDriver *wdt_driver, const char *driver_name)`, 用于初始化 Watchdog 驱动, 其中 `wdt_driver` 表示需要初始化的 Watchdog 驱动; `driver_name` 表示驱动名称。该接口通过调用 `WdtDriverRegister` 函数将 Watchdog 驱动注册到 Watchdog 驱动总线。
- `int WdtDriverAttachToBus(const char *drv_name, const char *bus_name)`, 用于将 Watchdog 驱动挂接到 Watchdog BUS 总线, 其中 `drv_name` 表示驱动名称; `bus_name` 表示总线名称。
- `DriverType WdtDriverFind(const char *drv_name)`, 用于查找挂接的 Watchdog 驱动, 其中 `drv_name` 表示驱动名称; `drv_type` 表示驱动类型。

4.2.2 dev_wdt.h 文件

在 `dev_wdt.h` 文件中声明了 WDT 设备结构体 `WdtHardwareDevice`, 其具体的定义如下:

```
struct WdtHardwareDevice{
    struct HardwareDev haldev;
    const struct WdtDevDone *dev_done;
    void *private_data;
};
```

其中 `haldev` 用于描述一个设备结构; `WdtDevDone` 用于表示一个 Watchdog 设备的函数操作集; `private_data` 是传递参数的通用指针。

结构体 `WdtDevDone` 的定义如下:

```
struct WdtDevDone{
    uint32 (*open) (void *dev);
    uint32 (*close) (void *dev);
    uint32 (*write) (void *dev, struct BusBlockWriteParam *write_param);
    uint32 (*read) (void *dev, struct BusBlockReadParam *read_param);
};
```

可以看出，该结构体是用于描述 Watchdog 设备的打开、关闭、读写功能。

另外，在 `dev_wdt.h` 文件中还定义了一些函数接口：

- `int WdtDeviceRegister(struct WdtHardwareDevice *wdt_device, const char *device_name)`，用于初始化并注册 WDT 设备到设备管理链表上，其中 `wdt_device` 表示需要注册的 WDT 设备；`device_name` 表示设备名称。
- `int WdtDeviceAttachToBus(const char *dev_name, const char *bus_name)`，用于将 WDT 设备挂接到 WDT BUS 总线，其中 `dev_name` 表示设备名称；`bus_name` 表示总线名称。
- `HardwareDevType WdtDeviceFind(const char *dev_name, enum DevType dev_type)`，用于查找挂接的 WDT 设备，其中 `dev_name` 表示设备名称；`dev_type` 表示设备类型。

4.2.3 connect_wdt.c 文件

`connect_wdt.c` 文件中实现了 WDT 驱动及设备的注册，该流程主要包含 3 个步骤：

- （1）完成 Watchdog 的驱动和设备到 WDT 总线的挂接（或称为绑定）；
- （2）实现 Watchdog 设备的打开、关闭及读写方法，其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植；
- （3）实现 Watchdog 设备的驱动配置，需结合 Watchdog 特性及操作系统的接口定义进行相应的开发。

步骤（1）的实现过程如代码 4-4-1 所示，该流程是完成一个 Watchdog 驱动及设备总线绑定的过程。代码行 2 对结果返回值进行了相应的变量声明；代码 4 对 Watchdog 总线结构体变量进行了声明；代码行 6~10 对 Watchdog 总线结构体变量进行了初始化。代码行 12~13 对 Watchdog 驱动结构体变量进行了声明，同时将 Watchdog 的配置操作接口进行了配置；代码行 15~19 行对 Watchdog 的驱动结构体进行了初始化；代码行 21~25 将 Watchdog 驱动挂接到了 Watchdog 总线上。代码行 27~28 对 Watchdog 硬件设备结构体进行了声明，同时配置了相应的操作接口，具体可见代码 4-4-2；代码行 30~34、36~40 分别实现了硬件设备的注册和挂接。代码行 42 返回最终的初始化结果。

代码 4-4-120 Watchdog 驱动及设备绑定

```
01 int HwWdtInit(void){
```

```
02     x_err_t ret = EOK;
03
04     static struct WdtBus wdt0;
05
06     ret = WdtBusInit(&wdt0, WDT_BUS_NAME_0);
07     if (ret != EOK) {
08         KPrintf("Watchdog bus init error %d\n", ret);
09         return ERROR;
10     }
11
12     static struct WdtDriver drv0;
13     drv0.configure = WdtConfigure;
14
15     ret = WdtDriverInit(&drv0, WDT_DRIVER_NAME_0);
16     if (ret != EOK) {
17         KPrintf("Watchdog driver init error %d\n", ret);
18         return ERROR;
19     }
20
21     ret = WdtDriverAttachToBus(WDT_DRIVER_NAME_0, WDT_BUS_NAME_0);
22     if (ret != EOK) {
23         KPrintf("Watchdog driver attach error %d\n", ret);
24         return ERROR;
25     }
26
27     static struct WdtHardwareDevice dev0;
28     dev0.dev_done = &dev_done;
29
30     ret = WdtDeviceRegister(&dev0, WDT_0_DEVICE_NAME_0);
31     if (ret != EOK) {
32         KPrintf("Watchdog device register error %d\n", ret);
33         return ERROR;
34     }
35
36     ret = WdtDeviceAttachToBus(WDT_0_DEVICE_NAME_0,
WDT_BUS_NAME_0);
37     if (ret != EOK) {
38         KPrintf("Watchdog device register error %d\n", ret);
39         return ERROR;
40     }
41
```

```

42     return ret;
43 }

```

步骤（2）的实现过程如代码 4-4-2 所示，该结构体是实现 Watchdog 设备的打开、关闭及读写的对应接口。从上至下，分别对应了 Watchdog 设备的打开、关闭和读写操作。根据结构体 WdtDevDone 的定义，Watchdog 只有设备的开口接口，关闭设备以及读写操作的实现接口由于硬件特性定义为 None，即不执行操作。

代码 4-4-121 Watchdog 设备打开、关闭与读写配置

```

01 static const struct WdtDevDone dev_done = {
02     WdtOpen,
03     NONE,
04     NONE,
05     NONE,
06 };

```

Watchdog 设备的驱动配置过程如代码 4-4-3 所示。在 WdtDrvConfigure 函数中，代码行 2~3 进行输入参数 drv 和 args 的检查。代码行 5 声明了 Watchdog 初始化结构体变量。代码行 8~18 对应于设置超时参数命令。其中代码行 9~16 为 Watchdog 的初始化过程；代码行 17 是由于设备硬件特性，需要进行一次“喂狗”操作来实现 Watchdog 计时器的初次启动。代码行 20~24 对应于“喂狗”命令，当 Watchdog 计时器小于指定值时，进行“喂狗”操作以实现 Watchdog 计时器的重置。

代码 4-4-122 Watchdog 设备的驱动配置

```

01 static uint32 WdtConfigure(void *drv, struct BusConfigureInfo *args){
02     NULL_PARAM_CHECK(drv);
03     NULL_PARAM_CHECK(args);
04
05     stc_wdt_init_t stcWdtInit;
06     switch (args->configure_cmd)
07     {
08         case OPER_WDT_SET_TIMEOUT:
09             stcWdtInit.u32CountPeriod = WDT_CNT_PERIOD65536;
10             stcWdtInit.u32ClockDiv = WDT_CLK_DIV1024;
11             stcWdtInit.u32RefreshRange = WDT_RANGE_0TO25PCT;
12             stcWdtInit.u32LPMCount = WDT_LPM_CNT_STOP;
13             stcWdtInit.u32ExceptionType = WDT_EXP_TYPE_RST;

```

```

14         if (WDT_Init(&stcWdtInit) != 0) {
15             return ERROR;
16         }
17         WDT_FeedDog();
18         break;
19
20         case OPER_WDT_KEEPALIVE:
21             if (WDT_GetCountValue() < WDT_COUNT_CYCLE/4){
22                 WDT_FeedDog();
23             }
24             break;
25
26         default:
27             return ERROR;
28     }
29     return EOK;
30 }

```

4.3 WATCHDOG 设备应用

Watchdog 设备应用实现过程如代码 4-4-4 所示。代码行 1~3 需要导入 3 个包括标准库在内的头文件。在测试函数 TestWDT 中，代码行 6~11 为打开 Watchdog 设备，若打开失败将打印错误信息并返回。代码行 13~20 为对 Watchdog 进行初始化和启动；代码行 22~30 实现了间隔为 100ms 的 100 次“喂狗”操作，同时在每一次喂狗操作前打印提示信息。代码行 31 至 32 为关闭硬件设备并返回。当喂狗操作结束后，程序即模拟宕机情况。看门狗计时器清零后将触发 Reset 中断使得芯片重启。

代码 4-4-123 Watchdog 设备应用

```

01 #include <stdio.h>
02 #include <string.h>
03 #include <transform.h>
04
05 void TestWDT(int argc, char *argv[]){
06     int wdt_fd = PrivOpen(WDT0_DEV_DRIVER, O_RDWR);
07     if (wdt_fd < 0){
08         printf("open wdt_fd fd error:%d\n", wdt_fd);
09         return;
10     }
11     printf("hw watchdog open!\n");

```

```
12
13     struct PrivIoctlCfg ioctl_cfg;
14     ioctl_cfg.ioctl_driver_type = WDT_TYPE;
15
16     if (0 != PrivIoctl(wdt_fd, OPER_WDT_SET_TIMEOUT, &ioctl_cfg)){
17         printf("ioctl wdt fd error %d\n", wdt_fd);
18         PrivClose(wdt_fd);
19         return;
20     }
21
22     int test_counter = 100;
23
24     while (test_counter--){
25         if (1 == argc){
26             printf("dog is feed\n");
27             PrivIoctl(wdt_fd, OPER_WDT_KEEPAKIVE, &ioctl_cfg); // feed dog
28         }
29         PrivTaskDelay(100);
30     }
31     PrivClose(wdt_fd);
32     return;
33 }
```

第5章 UART 驱动

5.1 UART 介绍

5.1.1 UART 基本介绍

UART是一种串行通信接口，它在嵌入式系统中被广泛使用。UART可以将数据从一个设备传输到另一个设备，常见的应用场景包括与计算机进行通信、与传感器进行数据交换、与外设进行控制等。UART的原理很简单，它将数据位按照一定的格式（例如8位数据、无校验位、1位停止位）依次发送出去，并在接收端将数据位重新组装成一个完整的数据包。UART的通信速率由波特率来表示，通常支持的波特率范围从几十波特到数百万波特不等。在嵌入式系统中，UART通常是通过硬件实现的，主控芯片会集成UART硬件模块，通过配置寄存器来设置波特率、数据格式等参数。同时，主控芯片也可以通过中断或DMA等方式来进行UART的数据传输

HC32F4A0搭载通用串行收发器模块(USART)10 个单元。通用串行收发器模块

(USART)能够灵活地 与外部设备进行全双工数据交换;本 USART 支持通用异步串行通信接口(UART),时钟同步通信接口, 智能卡接口 (ISO/IEC7816-3)和 LIN 通信接口。支持调制解调器操作(CTS/RTS 操作), 多处理器操作。与 Timer0 模块配合支持 UART 接收 TIMEOUT 功能。USART_1 支持通过 RX 线唤醒STOP 模式功能。

UART主要特性:

- 数据长度可编程:8 位/9 位
- 校验功能可配置:奇校验/偶校验/无校验
- 停止位可配置:1 位/2 位
- 时钟源可选:内部时钟源(内部波特率生成器生成的时钟)/外部时钟源(USARTn_CK 管脚输入的时钟)
- 接收错误:校验错误, 帧错误, 上溢错误
- 支持多个处理器间通信
- 内置数字滤波器
- 支持接收数据 TIMEOUT 功能
- 单元 1 支持停止模式唤醒功能
- 支持全双工/半双工通信方式

5.1.2 中断说明

UART 的中断分为六种:接收错误中断,接收数据寄存器满中断,发送数据寄存器空中断,发送完成中断, TIMEOUT 中断和 RX 线唤醒停止模式中断。

5.1.3 寄存器说明

一. 状态寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	MPB
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	-	-	LBD	WKU P	RTO F	TXE	TC	RXN E	BE	ORE	-	FE	PE

- b31~b17: 作为保留位, 读的时候为 0, 写的时候写 0
- b16: 多处理器位标志, 0 表示当前接收数据位通信数据, 1 表示当前接收数据位

ID

- b15~b11: 作为保留位, 读的时候为 0, 写的时候写 0
- b10: LIN 间隔检出标志, 0 表示未检测到间隔段, 1 表示检测到间隔段
- b9: LIN Wakeup 信号标志, 0 表示检测到 Wakeup 信号, 1 表示检测到 Wakeup 信号
- b8: UART 接收 TIMEOUT 标志位, 0 表示无 UART 接收 TIMEOUT, 1 表示发生 UART 接收 TIMEOUT
- b7: 发送数据寄存器空标志位, 0 表示数据未传输到移位寄存器, 发送数据寄存器非空, 1 表示数据传输移位寄存器, 发送数据寄存器位空
- b6: 发送完成标志, 0 表示发送数据中, 1 表示发送数据完成
- b5: 接收数据寄存器不为空标志, 0 表示接收到数据, 1 表示准备好读取接收到的数据
- b4: LIN 总线检错误标志, 0 表示无总线错误, 1 表示有总线错误
- b3: 接收上溢错误标志, 0 表示无接收上溢错误, 1 表示发生接收上溢错误
- b2: 作为保留位, 读的时候为 0, 写的时候写 0
- b1: 接收帧错误标志位, 0 表示无接收帧错误, 1 表示发生帧错误
- b0: 接收数据校验, 0 表示无接收数据校验错误, 1 表示发生接收数据校验错误

二. 数据寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	RDR[8:0]								
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	-	-	-	MPI D	TDR[8:0]								

- b31~b25: 作为保留位, 读的时候为 0, 写的时候写 0
- b31~b17: 接收数据寄存器
- b15~b10: 作为保留位, 读的时候为 0, 写的时候写 0
- b9: 多处理器模式时, 发送通信数据后发送 ID 选择位, 0 表示发送数据, 1 表示发送 ID
- b8~b0: 发送数据寄存器

三. 波特率寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
DIV_Integer[7:0]								-	DIV_Fraction[6:0]						

- b31~b16: 作为保留位, 读的时候为 0, 写的时候写 0
- b15~b8: 整数分频寄存器
- b7: 作为保留位, 读的时候为 0, 写的时候写 0
- b6~b0: 小数分频寄存器

四. 控制寄存器 1

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
SBS	NFE	FBME	ML	-	-	-	MS	CLBD	CWKUP	CBE	CRTOF	CORE	-	CFE	CPE
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
OVER8	-	-	M	-	PCE	PS	-	TXEI	TCIE	RIE	SLME	TE	RE	RTOIE	RTOE

- b31: UART 模式接收数据时, 开始位检测方式设定位, 0 表示开始位检测方式为 RX 管脚低电平, 1 表示开始位检测方式为 RX 管脚下降沿
- b30: 数字滤波使能位
- b29: 小数波特率功能使能位
- b28: MSB/LSB 选择位, 0 表示 LSB 方式, 1 表示 MSB 方式
- b27~b25: 保留位, 读时读 0, 写时写 0
- b24: 通信模式选择位, 0 表示 UART 模式, 1 表示时钟同步模式
- b23: LBD 清零位, 0 表示不清零 LBD 标志, 1 表示清零 LBD 标志
- b22: WKUP 清零位, 0 表示不清零 WKUP 标志, 1 表示清零 WKUP 标志
- b21: BE 清零位, 0 表示不清零 BE 标志, 1 表示清零 BE 标志
- b20: RTOF 清零位, 0 表示不清零 RTOF 标志, 1 表示清零 RTOF 标志
- b19: ORE 标志清零位, 0 表示不清零 ORE 标志, 1 表示清零 ORE 标志
- b18: 保留位, 读时读 0, 写时写 0
- b17: FE 标志清零位, 0 表示不清零 FE 标志, 1 表示清零 FE 标志
- b16: PE 标志清零位, 0 表示不清零 PE 标志, 1 表示清零 PE 标志
- b15: UART 过采样模式设定, 0 表示 16 位, 1 表示 8 位
- b14~b13: 保留位, 读时读 0, 写时写 0

- b12: 数据长度设定位, 0 表示 8 位, 1 表示 9 位
- b11: 保留位, 读时读 0, 写时写 0
- b10: UART 模式时, 奇偶校验使能位, 0 表示无校验, 1 表示校验
- b9: UART 模式时, 奇偶校验选择位, 0 表示偶校验, 1 表示奇校验
- b8: 保留位, 读时读 0, 写时写 0
- b7: 发送数据寄存器空中断使能位, 0 表示 TI 中断请求无效, TI 中断不发生, 1 表示 TI 中断请求有效, TI 中断发生。
- b6: 发送完成中断使能位, 0 表示 TCI 中断请求无效, TCI 中断不发生, 1 表示 TCI 中断请求使能, TCI 中断发生
- b5: 接收中断使能位, 0 表示接收中断请求无效, RI 和 EI 中断不发生, 1 表示接收中断请求有效, RI 和 EI 中断发生
- b4: 多处理器操作时, 静默模式使能位, 0 表示正常模式, 1 表示静默模式
- b3: 发送器使能位, 0 表示发送器禁止, 1 表示发送器使能
- b2: 接收器使能位, 0 表示接受器禁止, 1 表示接受器使能
- b1: UART TIMEOUT 中断使能位, 0 表示 UART TIMEOUT 中断请求无效, RTOI 中断不发生, 1 表示 UART TIMEOUT 中断请求有效, RTOI 中断发生
- b0: UART TIMEOUT 功能使能位, 0 表示 UART TIMEOUT 禁止, 1 表示 UART TIMEOUT 功能使能

五. 控制寄存器 2

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	-	-	-	-	-	-	SBK M	SBK
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	LINE N	STO P	CLKC[1:0]	-	-	WKU PE	SBKL[1:0]	LBD L	LBDI E	BEE	BEIE	WKU PIE	MPE		

- b31~b28: 保留位, 读时读 0, 写时写 0
- b17: 间隔段发送模式选择 (仅对于 USART_5 和 USART_10 有效), 0 表示 SBK 位写 1 自动发送间隔段, 1 表示 SBK 位写 1 后, 写 0x00 到 USARTn_DR.TDR 寄存器, 开始发送间隔段

- b16: 间隔段发送使能位, 0 表示不发送间隔段, 1 表示根据 SBKM 位设定发送间隔段
- b15: 保留位, 读时读 0, 写时写 0
- b14: UART 模式时, LIN 功能使能位 (仅对于 USART_5 和 USART_10 有效), 0 表示禁止 LIN 功能, 1 表示使能 LIN 功能
- b13: UART 模式时, 停止位设定位, 0 表示 1 停止位, 1 表示 2 停止位
- b12~b11: 时钟控制位, 在 UART 模式下, 00 表示时钟源为内部波特率生成器生成的时钟, 时钟不输出到 USATRTn_CK 管脚, USARTn_CK 管脚可以当作普通 IO 使用, 01 表示时钟源为内部波特率生成器生成的时钟, 时钟输出到 USARTn_CK 管脚, 输出时钟频率和波特率相同, 10 或 11 时表示时钟源为外部输入时钟, 输入时钟的频率为波特率的 16 倍(OVER8=0)或者 8 倍(OVER8=1)
- b10~b9: 保留位, 读时读 1, 写时写 1
- b8: LIN 唤醒信号检测使能位, 0 表示禁止唤醒信号检测, 1 表示使能唤醒信号检测
- b7~b6: LIN 间隔段低电平宽度设定位, 00 表示 10 位, 01 表示 11 位, 10 表示 13 位, 11 表示 14 位
- b5: LIN 间隔段低电平宽度检测阈值设定位 (仅对于 USART_5 和 USART_10 有效)
- b4: LIN 间隔段检测中断使能位, 0 表示 LIN 间隔段检测中断请求无效, 不发生中断, 1 表示 LIN 间隔段检测中断请求有效, 发生中断
- b3: LIN 总线, 检测错误功能使能位, (仅对于 USART_5 和 USART_10 有效), 0 表示禁止 LIN 总线错误功能检测功能, 1 表示使能 LIN 总线错误功能检测功能
- b2: LIN 总线错误中断功能使能位, (仅对于 USART_5 和 USART_10 有效), 0 表示 LIN 总线错误中断请求无效, 1 表示使能 LIN 总线错误中断有效
- b1: LIN 唤醒信号检测中断使能位, (仅对于 USART_5 和 USART_10 有效), 0 表示 LIN 唤醒信号检测中断请求无效, 不发生中断, 1 表示 LIN 唤醒信号检测中断请求有效, 发生中断
- b0: UART 模式时, 多处理器功能使能位, 0 表示禁止, 1 表示允许

六. 控制寄存器 3

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	BCN[2:0]		-	-	-	-	-	-
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b15	b14
-	-	-	-	-	-	CTS E	RTS E	-	-	SCE N	LOO P	HDS EL	-	-	-

- b31~b24: 保留位, 读时读 0, 写时写 0
- b23~b21: 智能卡模式时, 设定一位数据传输期间的基本时钟数
- b20~b10: 保留位, 读时读 0, 写时写 0
- b9: CTS 功能使能位, 0 表示 CTS 功能无效, 1 表示 CTS 功能有效
- b8: RTS 功能使能位, 0 表示 RTS 功能无效, 1 表示 RTS 功能有效
- b7~b6: 保留位, 读时读 0, 写时写 0
- b5: 智能卡模式使能位, 0 表示禁止智能卡模式, 1 表示使能智能卡模式
- b4: LIN 回环模式使能位, 0 表示正常模式, 1 表示回环模式
- b3: UART 单线半双工模式使能位, 0 表示 UART 全双工模式, 1 表示 UART 半双工模式
- b2~b0: 保留位, 读时读 0, 写时写 0

七. 预分频寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	-	-	-	-	-	-	-	-	-	LBMPSC[1:0]		PSC[1:0]	

- b31~b4: 保留位, 读时读 0, 写时写 0
- b3~b2: LIN 波特率测量计数器时钟源选择, 00 表示 PLCK, 01 表示 PLCK/2, 10 表示 PLCK/4, 11 表示 PLCK/8
- b1~b0: 内部时钟源时, 预分频器分频值选择位, 00 表示 PLCK, 01 表示 PLCK/4, 10 表示 PLCK/16, 11 表示 PLCK/64

八. LIN 波特率测量计数寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
LBMC[15:0]															

- b31~b16: 保留位, 读时读 0, 写时写 0
- b15~b0: LIN 波特率测量结果计数器

九. USART1 滤波控制寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	-	-	-	-	-	-	-	-	-	-	USART1_NFE	USASRT1_NFS[1:0]	

- b31~b3: 保留位, 读时读 0, 写时写 0
- b2: USART1 滤波使能寄存器, 0 表示模拟滤波器关, 1 表示模拟滤波器开
- b1~b0: USART1 滤波选择寄存器, 00 表示滤波宽度档位 1, 01 表示滤波宽度档位 2, 10 表示滤波宽度档位 3, 11 表示滤波宽度档位 4

5.2 UART 驱动介绍

UART 设备通过 SERIAL BUS 挂接到 BUS 总线上, 其中 SERIAL BUS 的结构体在 bus_serial.h 文件中进行了定义。每个设备总线都关联一个 drv 和一个 dev, 其中, drv 结构体也在 bus_serial.h 中进行了定义, dev 结构体在 dev_serial.h 中进行了定义。另外, SERIAL 驱动及设备的注册流程在 connect_serial.c 中进行了实现。接下来, 针对这 3 个 SERIAL 相关的具体结构和实现流程进行详细介绍。

5.2.1 bus_serial.h 文件

bus_serial.h 文件中声明了 SERIAL 总线结构体 SerialBus 和驱动结构体 SerialDriver。SerialBus 的结构体定义如下:

```
struct SerialBus{
    struct Bus bus;
    void *private_data;
};
```

其中 bus 是通用管理总线; private_data 是传递参数的通用指针。

SerialDriver 的结构体定义如下:

```
struct SerialDriver{
    struct Driver driver;
    const struct SerialDrvDone *drv_done;
    uint32 (*configure)(void *drv, struct BusConfigureInfo *configure_info);
    void *private_data;
};
```

其中 `driver` 表示通用管理驱动；`drv_done` 结构体指针包含两个回调函数，用于配置串口波特率等参数和开关中断；`configure` 表示配置函数；`private_data` 是传递参数的通用指针。

另外，在 `bus_serial.h` 文件中还定义了一些函数接口，具体均在 `bus_serial.c` 文件中实现：

- `int SerialBusInit(struct SerialBus *serial_bus, const char *bus_name)`，用于初始化 SERIAL 总线，其中 `serial_bus` 表示初始化的 SERIAL 总线；`bus_name` 表示总线名称。通过调用 `SerialRegister` 函数将 SERIAL 总线注册到 BUS 管理总线链表。
- `int SerialReleaseBus(struct SerialBus *serial_bus)`，用于释放 SERIAL 总线，即将 SERIAL 总线从 BUS 管理总线链表中移除，其中 `serial_bus` 表示需要释放的 SERIAL 总线。一般该接口函数与 `SerialBusInit` 函数配对使用，对 SERIAL 总线进行注册和释放。
- `int SerialDriverInit(struct SerialDriver *serial_driver, const char *driver_name)`，用于初始化 SERIAL 驱动，其中 `serial_driver` 表示需要初始化的 SERIAL 驱动；`driver_name` 表示驱动名称。该接口通过调用 `SerialDriverRegister` 函数将 SERIAL 驱动注册到 SERIAL 驱动总线。
- `int SerialDriverAttachToBus(const char *drv_name, const char *bus_name)`，用于将 SERIAL 驱动挂接到 SERIAL BUS 总线，其中 `drv_name` 表示驱动名称；`bus_name` 表示总线名称。
- `DriverType SerialDriverFind(const char *drv_name, enum DriverType_e drv_type)`，用于查找挂接的 SERIAL 驱动，其中 `drv_name` 表示驱动名称；`drv_type` 表示驱动类型。

5.2.2 dev_serial.h 文件

在 `dev_serial.h` 文件中声明了 SERIAL 设备结构体 `SerialHardwareDevice`，其具体的定义如下：

```
struct SerialHardwareDevice{
    struct HardwareDev haldev;
    struct SerialHwDevDone *hwdev_done;
    struct SerialDataTransferParam serial_fifo;
    uint32 ext_serial_mode;
    const struct SerialDevDone *dev_done;
    void *private_data;
};
```

其中 `haldev` 用于描述一个设备结构；`hwdev_done` 结构体变量中的回调函数实体实现串口寄存器字符的输入和输出；`serial_fifo` 表示串口收发数据 FIFO；`ext_serial_mode` 表示是否为扩展串口；`dev_done` 用于表示一个 SERIAL 设备的函数操作集；`private_data` 是传递参数的通用指针。

结构体 `SerialDevDone` 的定义如下：

```
struct SerialDevDone{
    uint32 (*open) (void *dev);
    uint32 (*close) (void *dev);
    uint32 (*write) (void *dev, struct BusBlockWriteParam *datacfg);
    uint32 (*read) (void *dev, struct BusBlockReadParam *datacfg);
};
```

可以看出，该结构体是用于描述 SERIAL 设备的打开、关闭、读写功能。

另外，在 `dev_serial.h` 文件中还定义了一些函数接口：

- `int SerialDeviceRegister(struct SerialHardwareDevice *serial_device, void *serial_param, const char *device_name)`，用于初始化并注册 SERIAL 设备到设备管理链表上，其中 `serial_device` 表示需要注册的 SERIAL 设备；`serial_param` 表示 SERIAL 参数；`device_name` 表示设备名称。
- `int SerialDeviceAttachToBus(const char *dev_name, const char *bus_name)`，用于将 SERIAL 设备挂接到 SERIAL BUS 总线，其中 `dev_name` 表示设备名称；`bus_name` 表示总线名称。
- `HardwareDevType SerialDeviceFind(const char *dev_name, enum DevType dev_type)`，用于查找挂接的 SERIAL 设备，其中 `dev_name` 表示设备名称；`dev_type` 表示设备类型。

5.2.3 connect_serial.c 文件

`connect_serial.c` 文件中实现了 SERIAL 驱动及设备的注册，该流程主要包含 2 个步骤：

- （1）完成 SERIAL 的驱动和设备到 SERIAL 总线的挂接（或称为绑定）；
- （2）实现 SERIAL 设备的驱动配置，需结合 SERIAL 特性及操作系统的接口定义进行相应的开发。

步骤（1）的实现过程如代码 4-5-1 所示，该代码是从 `HwUsartInit` 函数中截取的一部分，是一个完整的 SERIAL 驱动及设备总线绑定的过程。代码行 2~9 定义了 3 个结构体变量，

并进行初始化。代码行 11~25 对 HwUsartInit 函数中的一些全局变量和局部变量进行赋值。代码行 18~26 根据串口工作模式对变量参数进行赋值。代码行 28-31 对中断进行配置。代码行 39~43 使用 BoardSerialBusInit 函数进行 SERIAL 驱动挂接到 SERIAL 总线，具体的实现过程详见代码 4-5-2。代码行 45~49 使用 BoardSerialDevBend 函数进行 SERIAL 设备挂接到 SERIAL 总线，具体的实现过程详见代码 4-5-3。

在代码 4-5-2 的 BoardSerialBusInit 函数中，代码行 6~10 对 SERIAL 总线进行初始化；代码行 12~17 对 SERIAL 驱动进行初始化；代码行 19~24 将 SERIAL 驱动挂接到 SERIAL 总线上。在代码 4-5-3 的 BoardSerialDevBend 函数中，代码行 5~9 中对 SERIAL 设备进行注册；代码行 11~15 将 SERIAL 设备挂接到 SERIAL 总线。

代码 4-5-124 UART 驱动及设备绑定

```

01  #ifndef BSP_USING_UART3
02      static struct SerialCfgParam serial_cfg_3;
03      memset(&serial_cfg_3, 0, sizeof(struct SerialCfgParam));
04
05      static struct SerialDevParam serial_dev_param_3;
06      memset(&serial_dev_param_3, 0, sizeof(struct SerialDevParam));
07
08      static struct UsartHwCfg serial_hw_cfg_3;
09      memset(&serial_hw_cfg_3, 0, sizeof(struct UsartHwCfg));
10
11      serial_driver_3.drv_done = &drv_done;
12      serial_driver_3.configure = SerialDrvConfigure;
13      serial_device_3.hwdev_done = &hwdev_done;
14
15      serial_cfg_3.data_cfg = data_cfg_init;
16
17      //default irq configure
18      serial_hw_cfg_3.uart_device = CM_USART3;
19      serial_hw_cfg_3.usart_clock = FCG3_PERIPH_USART3;
20      serial_hw_cfg_3.rx_err_irq.irq_config.irq_num
= BSP_UART3_RXERR_IRQ_NUM;
21      serial_hw_cfg_3.rx_err_irq.irq_config.irq_prio =
BSP_UART3_RXERR_IRQ_PRIO;
22      serial_hw_cfg_3.rx_err_irq.irq_config.int_src = INT_SRC_USART3_EI;
23

```

```

24     serial_hw_cfg_3.rx_irq.irq_config.irq_num = BSP_UART3_RX_IRQ_NUM;
25     serial_hw_cfg_3.rx_irq.irq_config.irq_prio = BSP_UART3_RX_IRQ_PRIO;
26     serial_hw_cfg_3.rx_irq.irq_config.int_src = INT_SRC_USART3_RI;
27
28     serial_hw_cfg_3.rx_err_irq.irq_callback = Uart3RxErrIrqHandler;
29     serial_hw_cfg_3.rx_irq.irq_callback = Uart3RxIrqHandler;
30
31     hc32_install_irq_handler(&serial_hw_cfg_3.rx_err_irq.irq_config,
                             serial_hw_cfg_3.rx_err_irq.irq_callback, 0);
32
33     serial_cfg_3.hw_cfg.private_data = (void *)&serial_hw_cfg_3;
34     serial_driver_3.private_data = (void *)&serial_cfg_3;
35
36     serial_dev_param_3.serial_work_mode = SIGN_OPER_INT_RX;
37     serial_device_3.haldev.private_data = (void *)&serial_dev_param_3;
38
39     ret = BoardSerialBusInit(&serial_bus_3, &serial_driver_3, SERIAL_BUS_NAME_3,
                             SERIAL_DRV_NAME_3);
40     if (EOK != ret) {
41         KPrintf("HwUartInit uart3 error ret %u\n", ret);
42         return ERROR;
43     }
44
45     ret = BoardSerialDevBend(&serial_device_3, (void *)&serial_cfg_3,
                             SERIAL_BUS_NAME_3,
                             SERIAL_3_DEVICE_NAME_0);
46     if (EOK != ret) {
47         KPrintf("HwUartInit uart3 error ret %u\n", ret);
48         return ERROR;
49     }
50 #endif

```

代码 4-5-125 SERIAL 驱动挂接过程

```

01 static int BoardSerialBusInit(struct SerialBus *serial_bus, struct SerialDriver
    *serial_driver, const char *bus_name, const char *drv_name)
02 {
03     x_err_t ret = EOK;
04
05     /*Init the serial bus */
06     ret = SerialBusInit(serial_bus, bus_name);

```

```

07     if (EOK != ret) {
08         KPrintf("HwUartInit SerialBusInit error %d\n", ret);
09         return ERROR;
10     }
11
12     /*Init the serial driver*/
13     ret = SerialDriverInit(serial_driver, drv_name);
14     if (EOK != ret) {
15         KPrintf("HwUartInit SerialDriverInit error %d\n", ret);
16         return ERROR;
17     }
18
19     /*Attach the serial driver to the serial bus*/
20     ret = SerialDriverAttachToBus(drv_name, bus_name);
21     if (EOK != ret) {
22         KPrintf("HwUartInit SerialDriverAttachToBus error %d\n", ret);
23         return ERROR;
24     }
25
26     return ret;
27 }

```

代码 4-5-126 SERIAL 设备挂接过程

```

01 static int BoardSerialDevBend(struct SerialHardwareDevice *serial_device, void.
    *serial_param, const char *bus_name, const char *dev_name)
02 {
03     x_err_t ret = EOK;
04
05     ret = SerialDeviceRegister(serial_device, serial_param, dev_name);
06     if (EOK != ret) {
07         KPrintf("HwUartInit SerialDeviceInit device %s error %d\n", dev_name, ret);
08         return ERROR;
09     }
10
11     ret = SerialDeviceAttachToBus(dev_name, bus_name);
12     if (EOK != ret) {
13         KPrintf("HwUartInit SerialDeviceAttachToBus device %s error %d\n",
14                                     dev_name, ret);
15         return ERROR;
16     }

```

```

16
17     return  ret;
18 }

```

步骤（2）中的 SERIAL 设备驱动配置过程如代码 4-5-4 所示。在 SerialDrvConfigure 函数中,代码行 3~4 进行输入参数 drv 和 configure_info 的检查。代码行 6~8 声明了 3 个变量。代码行 10~21 根据配置命令进行选择,若配置命令为 OPE_INT,则根据 SerialInit 函数执行串口初始化;若配置命令为 OPE_CFG,则根据 SerialConfigure 函数进行串口配置。

代码 4-5-127 SERIAL 设备的驱动配置

```

01 static uint32 SerialDrvConfigure(void *drv, struct BusConfigureInfo *configure_info)
02 {
03     NULL_PARAM_CHECK(drv);
04     NULL_PARAM_CHECK(configure_info);
05
06     x_err_t ret = EOK;
07     int serial_operation_cmd;
08     struct SerialDriver *serial_drv = (struct SerialDriver *)drv;
09
10     switch (configure_info->configure_cmd)
11     {
12         case OPE_INT:
13             ret = SerialInit(serial_drv, configure_info);
14             break;
15         case OPE_CFG:
16             serial_operation_cmd = *(int *)configure_info->private_data;
17             ret = SerialConfigure(serial_drv, serial_operation_cmd);
18             break;
19         default:
20             break;
21     }
22
23     return ret;
24 }

```

5.3 UART 设备应用

将控制台用到的总线设为 UART6,将板子与电脑连接,若是能打印出电脑信息,说明 UART 设备(总线)能够正常使用。由于需要使用 UART3 进行烧写,在测试时需要将被来

设定为 UART3 改为 UART6。具体位置在 include 文件夹下的 connect_usart.h 中。修改后的代码如下：

代码 4-5-128 UART 设备应用

```
#define KERNEL_CONSOLE_BUS_NAME      SERIAL_BUS_NAME_6
#define KERNEL_CONSOLE_DRV_NAME      SERIAL_DRV_NAME_6
#define KERNEL_CONSOLE_DEVICE_NAME   SERIAL_6_DEVICE_NAME_0
```

第6章 I2C 驱动

6.1 I2C 介绍

6.1.1 I2C 基本介绍

I2C是一种串行通信协议，在嵌入式系统中被广泛应用。它可以在多个设备之间进行通信，包括芯片（比如传感器、存储器、温度计等）、模块（比如LCD屏幕、电源管理模块等）和其他系统。

I2C通信协议由两根线组成：串行数据线（SDA）和串行时钟线（SCL）。设备之间通过这两根线进行双向通信，其中SDA线用于传输数据，SCL线用于传输时钟信号。

在I2C通信中，每个设备都有一个唯一的地址，主控芯片可以通过发送设备地址来选择需要进行通信的设备。通信过程中，主控芯片可以向设备发送数据，也可以从设备中接收数据。在数据传输过程中，每个数据位都由SCL线上的时钟信号控制，以保证数据的准确性。I2C通信协议还支持多主控芯片的应用场景，可以通过总线仲裁机制来协调多个主控芯片之间的通信。

HC32F4A0 中的 I2C 用作微控制器和 I2C 串行总线之间的接口。提供多主模式功能，可以控制所有 I2C 总线协议，仲裁。支持标准模式，快速模式。还支持 SMBus 总线。

HC32F4A0 的 I2C 的主要特性如下：

- I2C 总线方式、SMBUS 总线方式可选。主机模式、从机模式可选。自动确保与传送速率相对于的各种准备时间、保持时间和总线空闲时间。标准模式最大 100Kbps，快速模式最大 400Kbps。
- 自动生成开始条件、重新开始条件和停止条件，并能检测到总线的开始条件重新开始
- 开始条件和停止条件

- 可以设定 2 个从机模式地址。可同时设定 7 位地址格式和 10 位地址格式。能检测
- 到广播呼叫地址，SMBus 主机地址，SMBus 设备默认地址，SMBus 报警地址。发送时可以自动判定应答位。接收时可以自动发送应答位。
- 握手功能
- 仲裁功能
- 超时功能，可以检测 SCL 时钟长时间停止
- SCL 输入和 SDA 输入内置数字滤波器，滤波能力可编程
- 通信错误，接收数据满，发送数据空，一帧发送结束，地址匹配一致中断
- 当选择 I2C 总线时，SCL/SDA 输入电平为 CMOS 电平，当选择 SMBus 时 SCL/SDA 输入电平为 TTL 电平

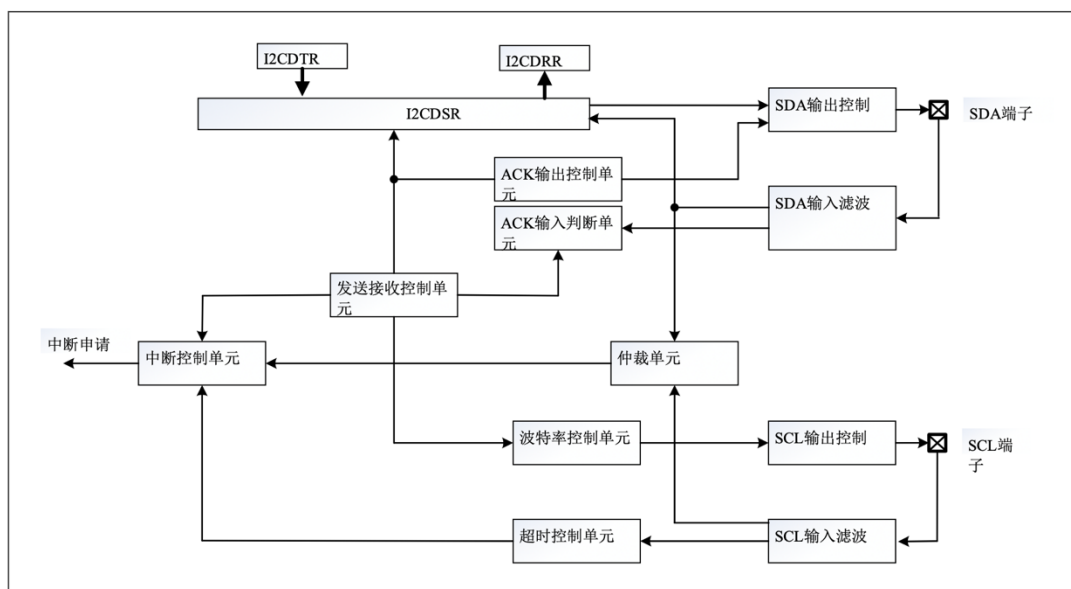


图 4-6-179 I2C 系统框架图

通常情况下一个完整的通信过程包括如下四个部分：开始条件、地址传送、数据传送和停止条件。

当总线上的主机都不驱动总线，总线进入空闲状态。SCL 和 SDA 都为高电平。总线空闲状态下总线上设备都可以通过发送开始条件启动通信。在 I2C_SR.BUSY 标志为“0”(总线空闲)的状态下如果将 START 位置“1”，就发行开始条件。如果检测到开始条件，就自

动将 I2C_SR.BUSY 标志和 I2C_SR.STARTF 标志置“1”，并且自动将 START 位置“0”。

此时，如果在 START 位为“1”的状态下发送的 SDA 信号和 SDA 线的信号状态相同，并且检测到开始条件，就视为通过 START 位正确地发行了开始条件，在将 I2C_SR.MSL 位和 I2C_SR.TRA 位自动置“1”后变为主控发送模式。另外，I2C_SR. EMPTYF 因 TRA 位为“1”而自动变为“1”。接下来把从机地址写入 I2C_DTR 寄存器，发送地址。开始条件或者重新开始条件后面的帧是地址帧，用于指定主机通信的对象地址。在发送停止条件之前，指定的从机一直有效。

地址匹配一致后，总线上的主机根据 R/W 定义的方向一帧一帧的传送数据。所有的地址帧后传送的数据都视为数据帧。即使是 10 位地址格式的低 8 位地址也视为数据帧。数据帧的长度是 8 位。SCL 的低电平 SDA 变化，SCL 的高电平 SDA 保持，每个时钟周期发送一位数据。数据帧后的第 9 个时钟是应答位，是接收方向发送方传送的握手信号。通过 I2C_CR1.STOP 位发行停止条件。在 I2C_SR.BUSY 标志为“1”（总线忙）并且 I2C_SR.MSL 位为“1”（主机模式）的状态下将 STOP 位置“1”，就发行停止条件。

在多主机模式中使用 I2C 总线时，有可能因和其他主机的竞争而使 SCL 时钟和 SCL 时钟发生冲突。如果 SCL 时钟发生冲突，主机就需要与 SCL 时钟同步，而且需要逐位进行 SCL 时钟的同步。当检测到 SCL 线的上升沿并且在对 I2C_CCR.SHIGHW 寄存器设定的高电平进行计数的过程中，如果因其他主机的 SCL 时钟输出而使 SCL 线下降，就在检测到 SCL 线的下降沿时中止高电平宽度的递增计数，并且在 SCL 线被驱动为低电平的同时开始对 I2C_CCR.SLOWW 设定的低电平宽度进行递增计数，在结束低电平宽度的计数时结束 SCL 线的低电平驱动，并释放 SCL 线。此时，如果其他主机的 SCL 时钟的低电平宽度大于 SLOWW 设定的低电平宽度，就延长 SCL 时钟的低电平宽度。当其他主机结束低电平输出时，释放 SCL 线并且 SCL 时钟上升。因此，在发生 SCL 时钟输出冲突时，SCL 时钟的高电平宽度与短时钟同步，低电平宽度与长时钟同步。

具有对通信模块进行复位的功能。有 2 种复位，1 种是对包括 ICCR2.BBSY 标志在内的全部寄存器进行初始化的复位，另一种是在保持各种设定值的状态下解除从属地址匹配状态以及对内部计数器进行初始化等的内部复位。复位后，必须将 I2C_CR1.SWRST 位置“0”。因为无论进行哪种复位都要解除 SCL 引脚/SDA 引脚的输出状态而变为高阻抗状态，所以也能用于解除总线意外停机状态。从属模式中的复位会引起与主控设备的不同

步，因此尽量避免使用。必须注意:在复位(I2C_CR1.PE 位和 I2C_CR1.SWRST 位为“01b”)过程中不能监视开始条件等的总线状态。

6.1.2 中断说明

I2C 具有 4 种中断和用于触发启动其他外围电路的事件输出供用户选择。包括:通信错误的发生(仲裁失败检测、NACK 检测、超时检测、开始条件检测、停止条件检测)、接收结束、发送数据空、发送结束。

6.1.3 寄存器说明

一. I2C 控制寄存器 1 (I2C_CR1)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
SWRST	-	-	-	-	ACK	STOP	START	RESTART	ENG C	-	SMB HOST EN	SMBD EFAU LTEN	SMB ALRT EN	SMB US	PE

- b31~b16: 作为保留位，读的时候为 0，写的时候写 0
- b15: 软件复位，0 表示解除复位，1 表示软件复位
- b34~b11: 作为保留位，读的时候为 0，写的时候写 0
- b9: 停止条件生成位，0 表示不生成停止条件，1 表示生成停止条件
- b8: 起始条件生成位，0 表示不生成起始条件，1 表示生成起始条件
- b7: 重复起始条件生成位，0 表示不生成重复起始条件，1 表示生成重复起始条件
- b6: 广播呼叫使能，0 表示广播地址检测无效，1 表示广播地址检测有效
- b5: 作为保留位，读的时候为 0，写的时候写 0
- b4: 0 表示禁止匹配 SMBUS 主机地址，1 表示允许匹配 SMBUS 主机地址
- b3: 0 表示禁止匹配 SMBUS 默认地址，1 表示允许匹配 SMBUS 默认地址
- b2: 0 表示禁止匹配 SMBUS 报警响应地址，1 表示允许匹配 SMBUS 报警响应地址
- b1: 0 表示 I2C 总线模式，1 表示 SMBUS 模式
- b0: 0 表示 I2C 功能禁止，1 表示 I2C 功能允许

二. I2C 控制寄存器 2 (I2C_CR2)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	SMB ALRT IE	SMB HOST IE	SMB DEFA ULTIE	GEN CALL IE	-	-	-	-

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	TMO UTIE	-	NAC KIE	-	-	ARL OIE	-	TEMP TYIE	RFU LLIE	-	STO PIE	TEN DIE	SLAD DR1IE	SLAD DR0IE	STA RTIE

- b31~b24: 作为保留位, 读的时候为 0, 写的时候写 0
- b23: 0 表示 SMBUS 报警响应地址匹配一致中断禁止, 1 表示允许
- b22: 0 表示 SMBUS 主机地址匹配一致中断禁止, 1 表示允许
- b21: 0 表示 SMBUS 默认地址匹配一致中断禁止, 1 表示允许
- b20: 0 表示广播呼叫地址匹配一致中断禁止, 1 表示允许
- b19~b15: 作为保留位, 读的时候为 0, 写的时候写 0
- b14: 0 表示超时中断禁止, 1 表示超时中断允许
- b13: 作为保留位, 读的时候为 0, 写的时候写 0
- b12: 0 表示接受到 NACK 中断禁止, 1 表示接受到 NACK 中断允许
- b11~b10: 作为保留位, 读的时候为 0, 写的时候写 0
- b9: 0 表示仲裁失败中断禁止, 1 表示仲裁失败中断允许
- b8: 作为保留位, 读的时候为 0, 写的时候写 0
- b7: 0 表示发送数据空中断禁止, 1 表示发送数据空中断允许
- b6: 0 表示接受数据满中断禁止, 1 表示接受数据满中断允许
- b5: 作为保留位, 读的时候为 0, 写的时候写 0
- b4: 0 表示检测到停止条件中断禁止, 1 表示检测到停止条件中断允许
- b3: 0 表示发送一帧数据结束中断禁止, 1 表示允许
- b2: 0 表示从机地址 1 匹配一致中断禁止, 1 表示允许
- b1: 0 表示从机地址 0 匹配一致中断禁止, 1 表示允许
- b0: 0 表示总线检测到开始条件中断禁止, 1 表示允许

三. I2C 控制寄存器 3 (I2C_CR3)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	-	-	-	-	-	FAC KEN	-	-	-	-	HTM OUT	LTM OUT	TMO UTEN

- b31~b8: 作为保留位, 读的时候为 0, 写的时候写 0
- b7: 0 表示在 SCL 时钟的第 9 个时钟上升时, 此位为 “1”, 1 表示在 SCL 时钟的第 8 个时钟上升时, 此位为 “1”
- b6~b3: 作为保留位, 读的时候为 0, 写的时候写 0
- b2: 0 表示在 SCL 线为高电平时, 禁止超时检测, 1 表示允许超时检测
- b1: 0 表示在 SCL 线为低电平时, 禁止超时检测, 1 表示允许超时检测
- b0: 0 表示检测 SCL 电平超时功能禁止, 1 表示允许

四. I2C 控制寄存器 4 (I2C_CR4)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	-	-	BUS WAIT	-	-	-	-	-	-	-	-	-	-

- b31~b11: 作为保留位, 读的时候为 0, 写的时候写 0
- b10: 总线等待位, 0 表示 I2C_DRR 接收满, I2C_DSR 为空时, 在第 9 个时钟和下次传送的第 1 个时钟之间不保持低电平, 继续接收下一个数据, 1 表示 I2C_DRR 接收满, I2C_DSR 为空时, 在第 9 个时钟和下次传送的第 1 个时钟之间不保持低电平, 通过读 I2C_DRR 寄存器来解除保持的低电平
- b9~b8: 作为保留位, 读的时候为 1, 写的时候写 1
- b7~b3: 作为保留位, 读的时候为 0, 写的时候写 0
- b2~b0: 作为保留位, 读的时候为 1, 写的时候写 1

五. I2C 从机地址寄存器 0 (I2C_SLR0)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
ADD			SLA			SLADDR0[9:0]									
RMO	-	-	DDR0	-	-										
D0			EN												

- b31~b16: 作为保留位，读的时候为 0，写的时候写 0
- b15: 0 表示选择 7 位地址，1 表示选择 10 位地址
- b14~b13: 作为保留位，读的时候为 0，写的时候写 0
- b12: 0 表示从机地址寄存器 0 设定值无效，1 表示有效
- b11~b10: 作为保留位，读的时候为 0，写的时候写 0
- b9~b8: 10 位从机地址的高位
- b7~b0: 7 位地址或者 10 位地址的低位

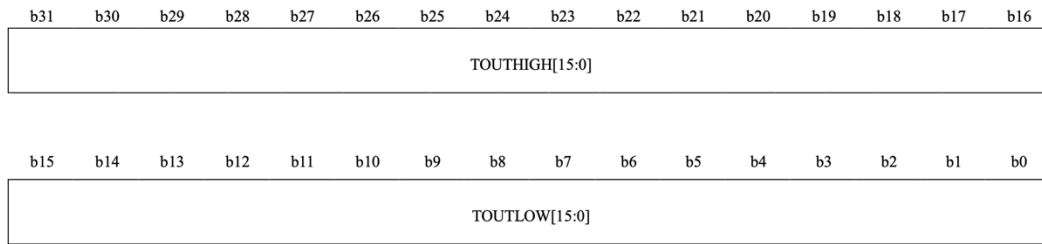
六. I2C 从机地址寄存器 1 (I2C_SLR01)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
ADD			SLA			SLADDR1[9:0]									
RMO	-	-	DDR1	-	-										
D1			EN												

- b31~b16: 作为保留位，读的时候为 0，写的时候写 0
- b15: 0 表示选择 7 位地址，1 表示选择 10 位地址
- b14~b13: 作为保留位，读的时候为 0，写的时候写 0
- b12: 0 表示从机地址寄存器 1 设定值无效，1 表示有效
- b11~b10: 作为保留位，读的时候为 0，写的时候写 0
- b9~b8: 10 位从机地址的高位
- b7~b0: 7 位地址或者 10 位地址的低位

七. I2C_SCL 电平超时控制寄存器 (I2C_SLTR)



- b31~b16: SCL 高电平超时周期
- b31~b16: SCL 低电平超时周期

八. I2C 状态控制寄存器 (I2C——SR)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	SMB ALRT F	SMB HOST F	SMB DEFA ULTF	GEN CALL F	-	TRA	BUS Y	MSL

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	TMO UTF	-	NAC KF	-	ACK RF	ARL OF	-	TEM PTYF	RFU LLF	-	STO PF	TEN DF	SLAD DR1F	SLAD DR0F	STA RTF

- b31~b24: 作为保留位, 读的时候为 0, 写的时候写 0
- b23: 0 表示未匹配到 SMBUS 报警相应地址, 1 表示检测到主机地址
- b22: 0 表示未匹配到 SMBUS 主机地址, 1 表示匹配到 SMBUS 主机地址
- b21: 0 表示未匹配到 SMBUS 默认地址, 1 表示匹配到 SMBUS 主机地址
- b20: 0 表示未匹配到广播呼叫地址, 1 表示匹配到广播呼叫地址
- b19: 作为保留位, 读的时候为 0, 写的时候写 0
- b18: 发送接收选择位, 0 表示接收数据, 1 表示发送数据
- b17: 总线忙标志位, 0 表示空闲状态, 总线上无通信, 1 表示占有状态, 总线上正在通信
- b16: 主从机选择位, 0 表示从机模式, 1 表示主机模式
- b15: 作为保留位, 读的时候为 0, 写的时候写 0
- b14: 超时标志位, 0 表示未检测到 SCL 电平超时, 1 表示 SCL 电平超时
- b13: 作为保留位, 读的时候为 0, 写的时候写 0
- b12: NACK 标志位, 0 表示未接受到 NACK, 1 表示接受到 NACK
- b11: 作为保留位, 读的时候为 0, 写的时候写 0

- b10: 接收应答位, 0 表示接受到应答位为 0, 1 表示接受到的应答位为 1
- b9: 仲裁失败标志位, 1 表示未发生仲裁失败, 0 表示仲裁失败
- b8: 作为保留位, 读的时候为 0, 写的时候写 0
- b7: 发送数据空标志位, 1 表示 I2C_DTR 寄存器满, 0 表示 I2C_DTR 寄存器空
- b6: 接受数据满标志位, 0 表示 I2C_寄存器空, 1 表示 I2C_DRR 寄存器满
- b5: 作为保留位, 读的时候为 0, 写的时候写 0
- b4: 停止条件标志位, 0 表示总线未检测到停止条件, 1 表示总线检测到停止条件
- b3: 发送数据标志位, 0 表示 I2C_DSR 寄存器发送中, 1 表示 I2C_DSR 寄存器发送结束
- b2: 从机地址寄存器 1 匹配一致标志, 0 表示未检测到从机地址寄存器 1 一致地址, 1 表示检测到从机地址寄存器 1 一致地址
- b1: 从机地址寄存器 0 匹配一致标志, 0 表示未检测到从机地址寄存器 0 一致地址, 1 表示检测到从机地址寄存器 0 一致地址
- b0: 开始条件/从新开始条件标志位, 0 表示总线未检测到开始条件, 1 表示总线检测到开始条件

九. I2C 状态清零寄存器 (I2C_CLR)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	SMB ALRT FCLR	SMB HOST FCLR	SMBD EFAUL TFCLR	GEN CALL FCLR	-	-	-	-

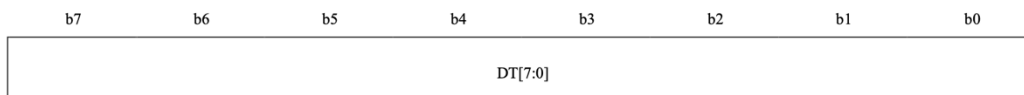
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	TMO UTFCL LR	-	NAC KFCLR	-	-	ARL OFCLR	-	TEM PTYF CLR	RFU LLFCL LR	-	STO PFCLR	TEN DFCLR	SLA DDR 1FCLR	SLA DDR 0FCLR	STA RTFCLR

此寄存器为只写寄存器。

- b31~b24: 作为保留位, 写的时候写 0
- b23: 写 1 表示清除 SMBALRTF 标志位
- b22: 写 1 表示清除 SMBHOSTF 标志位
- b21: 写 1 表示清除 SMBDEFAULTF 标志位
- b20: 写 1 表示清除 GENCALLF 标志位

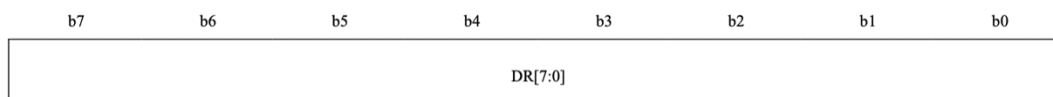
- b19~b15: 作为保留位, 写的时候写 0
- b14: 写 1 表示清除 TMOUTF 标志位
- b13: 作为保留位, 写的时候写 0
- b12: 写 1 表示清除 NACKF 标志位
- b11~b10: 作为保留位, 写的时候写 0
- b9: 写 1 表示清除 ARLOF 标志位
- b8: 作为保留位, 写的时候写 0
- b7: 写 1 表示清除 TEMPTYP 标志位
- b6: 写 1 表示清除 RFULLF 标志位
- b5: 作为保留位, 写的时候写 0
- b4: 写 1 表示清除 STOPF 标志位
- b3: 写 1 表示清除 TENDF 标志位
- b2: 写 1 表示清除 SLADDR1F 标志位
- b1: 写 1 表示清除 SLADDR0F 标志位
- b0: 写 1 表示清除 STARTF 标志位

十. I2C 数据发送寄存器 (I2C_DTR)



如果 I2C_DSR 寄存器为空, 就将写在 I2C_DTR 寄存器的发送数据传送到 I2C_DSR 寄存器, 发送模式时开始发送数据到 SDA 上。I2C_DSR 寄存器和 I2C_DTR 寄存器是双缓冲结构, 在 I2C_DSR 寄存器数据发送过程中, 如果预先写 I2C_DTR 寄存器的数据, 就能进行连续发送数据。I2C_DTR 寄存器可读可写。请在发送数据空中断要求发生时, 仅写一次 I2C_DTR 寄存器。

十一. I2C 数据接收寄存器 (I2C_DDR)



如果接收到 1 帧数据, 就能将接收数据从移位寄存器(I2C_DSR)转存到 I2C_DRR 寄存器, 进而可以进入到下一个数据接收状态。I2C_DSR 寄存器和 I2C_DRR 寄存器是双缓

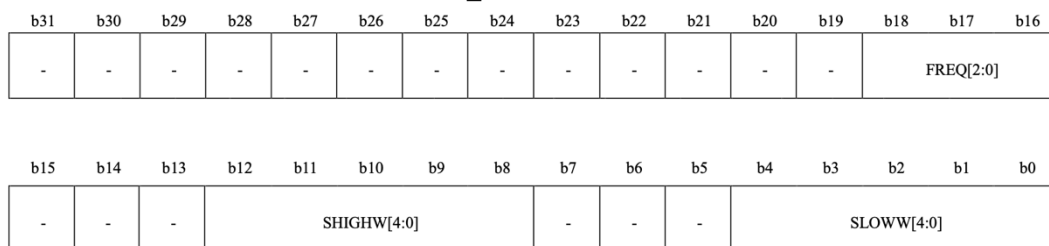
冲结构，在 I2C_DSR 寄存器数据接收过程中，如果读 取了 I2C_DRR 寄存器的数据，就能进行连续接收数据。禁止对 I2C_DRR 寄存器写。请在接收数据满中断要求发生时，仅读一次 I2C_DRR 寄存器。在 I2C_SR.RFULLF 标志位为“1”的状态下，如果不读取 I2C_DRR 寄存器的数据，而立即接收下一个数据，SCL 时钟就在下一次 RFULLF 标志位变“1”的前一个 SCL 时钟自动保持为低电平。

十二. I2C 数据移位寄存器 (I2C_DSR)



I2C_DSR 寄存器用于发送和接收数据的移位寄存器。I2C_DSR 寄存器不可读也不可写。在数据发送时，将发送数据从 I2C_DTR 寄存器传送到 I2C_DSR 寄存器，从 SDA 引脚发送数据。在数据接收时，一旦 1 帧数据接收结束，就将数据从 I2C_DSR 寄存器传送到 I2C_DRR 寄存器。

十三. I2C 时钟控制寄存器 (I2C_CCR)



- b31~b23: 作为保留位，写的时候写 0，读的时候读出来为 0
- b22~b19: 作为保留位，写的时候写 1，读的时候读出来为 1
- b18~b16: I2C 基准时钟频率设定位
- b15~b13: 作为保留位，写的时候写 1，读的时候读出来为 1
- b12~b8: 设定 SCL 高电平宽度位
- b7~b5: 作为保留位，写的时候写 1，读的时候读出来为 1
- b4~b0: 设定 SCL 低电平宽度位

十四. I2C 滤波控制寄存器 (I2C_FLTR)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	-	-	-	-	-	-	-	ANF EN	DNF EN	-	-	DNF[1:0]	

- b31~b6: 作为保留位，写的时候写 0，读的时候读出来为 0
- b5: 模拟滤波功能允许位，0 表示数字滤波功能禁止，1 表示数字滤波功能允许
- b3~b2: 作为保留位，写的时候写 0，读的时候读出来为 0
- b1~b0: 数字滤波器滤波能力选择，00 表示滤波能力 1 个 I2C 基准时钟周期，01 表示滤波能力 2 个 I2C 基准时钟周期，10 表示滤波能力 3 个 I2C 基准时钟周期，11 表示滤波能力 4 个 I2C 基准时钟周期

6.2 I2C 驱动介绍

I2C 设备通过 I2C BUS 挂接到 BUS 总线上，其中 I2C BUS 的结构体在 `bus_i2c.h` 文件中进行了定义。每个设备总线都关联一个 `drv` 和一个 `dev`，其中，`drv` 结构体也在 `bus_i2c.h` 中进行了定义，`dev` 结构体在 `dev_i2c.h` 中进行了定义。另外，I2C 驱动及设备的注册流程在 `connect_i2c.c` 中进行了实现。接下来，针对这 3 个 I2C 相关的具体结构和实现流程进行详细介绍。

I2C 设备通过 I2C BUS 挂接到 BUS 总线上，其中 I2C BUS 的结构体在 `bus_i2c.h` 文件中进行了定义。每个设备总线都关联一个 `drv` 和一个 `dev`，其中，`drv` 结构体也在 `bus_i2c.h` 中进行了定义，`dev` 结构体在 `dev_i2c.h` 中进行了定义。另外，I2C 驱动及设备的注册流程在 `connect_i2c.c` 中进行了实现。接下来，针对这 3 个 I2C 相关的具体结构和实现流程进行详细介绍。

6.2.1 bus_i2c.h 文件

`bus_i2c.h` 文件中声明了 I2C 总线结构体 `I2cBus` 和驱动结构体 `I2cDriver`。`I2cBus` 的结构体定义如下：

```
struct I2cBus{
    struct Bus bus;
    void *private_data;
};
```

其中 `bus` 是通用管理总线；`private_data` 是传递参数的通用指针。

I2cDriver 的结构体定义如下：

```
struct I2cDriver{
    struct Driver driver;
    uint32 (*configure) (void *drv, struct BusConfigureInfo *configure_info);
    void *private_data;
};
```

其中 driver 表示通用管理驱动；configure 表示配置函数；private_data 是传递参数的通用指针。

另外，在 bus_i2c.h 文件中还定义了一些函数接口，具体均在 bus_i2c.c 文件中实现：

- int I2cBusInit(struct I2cBus *i2c_bus, const char *bus_name), 用于初始化 I2c 总线，其中 i2c_bus 表示初始化的 I2C 总线；bus_name 表示总线名称。通过调用 BusRegister 函数将 I2C 总线注册到 BUS 管理总线链表。
- int I2cReleaseBus(struct I2cBus *i2c_bus), 用于释放 I2C 总线，即将 I2C 总线从 BUS 管理总线链表中移除，其中 i2c_bus 表示需要释放的 I2C 总线。一般该接口函数与 I2cBusInit 函数配对使用，对 I2C 总线进行注册和释放。
- int I2cDriverInit(struct I2cDriver *i2c_driver, const char *driver_name), 用于初始化 I2C 驱动，其中 i2c_driver 表示需要初始化的 I2C 驱动；driver_name 表示驱动名称。该接口通过调用 I2cDriverRegister 函数将 I2C 驱动注册到 I2C 驱动总线。
- int I2cDriverAttachToBus(const char *drv_name, const char *bus_name), 用于将 I2C 驱动挂接到 I2C BUS 总线，其中 drv_name 表示驱动名称；bus_name 表示总线名称。
- DriverType I2cDriverFind(const char *drv_name, enum DriverType_e drv_type), 用于查找挂接的 I2C 驱动，其中 drv_name 表示驱动名称；drv_type 表示驱动类型。

6.2.2 dev_i2c.h 文件

在 dev_i2c.h 文件中声明了 I2C 设备结构体 I2cHardwareDevice，其具体的定义如下：

```
struct I2cHardwareDevice{
    struct HardwareDev haldev;
    const struct I2cDevDone *i2c_dev_done;
    void *private_data;
};
```

其中 haldev 用于描述一个设备结构；i2c_dev_done 用于表示一个 I2C 设备的函数操作集；private_data 是传递参数的通用指针。

结构体 I2cDevDone 的定义如下：

```

struct I2cDevDone{
    uint32 (*dev_open) (struct I2cHardwareDevice *i2c_device);
    uint32 (*dev_close) (struct I2cHardwareDevice *i2c_device);
    uint32 (*dev_write) (struct I2cHardwareDevice *i2c_device, struct I2cDataStandard
    *msg);
    uint32 (*dev_read) (struct I2cHardwareDevice *i2c_device, struct I2cDataStandard
    *msg);
};

```

可以看出，该结构体是用于描述 I2C 设备的打开、关闭、读写功能。

另外，在 dev_i2c.h 文件中还定义了一些函数接口：

- int I2cDeviceRegister(struct I2cHardwareDevice *i2c_device, void *i2c_param, const char *device_name), 用于初始化并注册 I2C 设备到设备管理链表上，其中 i2c_device 表示需要注册的 I2C 设备；i2c_param 表示 I2C 参数；device_name 表示设备名称。
- int I2cDeviceAttachToBus(const char *dev_name, const char *bus_name), 用于将 I2C 设备挂接到 I2C BUS 总线，其中 dev_name 表示设备名称；bus_name 表示总线名称。
- HardwareDevType I2cDeviceFind(const char *dev_name, enum DevType dev_type), 用于查找挂接的 I2C 设备，其中 dev_name 表示设备名称；dev_type 表示设备类型。

6.2.3 connect_i2c.c 文件

connect_i2c.c 文件中实现了 I2C 驱动及设备的注册，该流程主要包含 3 个步骤：

- （1）完成 I2C 的驱动和设备到 I2C 总线的挂接（或称为绑定）；
- （2）实现 I2C 设备的打开、关闭及读写方法，其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植；
- （3）实现 I2C 设备的驱动配置。

步骤（1）的实现过程如代码 4-6-1 所示，该流程是完成一个 I2C 驱动及设备总线绑定的过程。代码行 3~6 定义了 1 个结果返回变量和 2 个结构体变量，并对结构体变量进行初始化。代码行 10 进行 I2C 的外部 GPIO 管脚的初始化。代码行 13 对 I2C 的驱动进行配置。代码行 17~21 使用 BoardI2cBusInit 函数进行 I2C 驱动挂接到 I2C 总线，具体的实现过程详见代码 4-6-2。代码行 23~27 使用 BoardI2cDevBend 函数进行 I2C 设备挂接到 I2C 总线，具体的实现过程详见代码 4-6-3。

代码 4-6-129 I2C 驱动及设备绑定

```
01 int HwI2cInit(void)
02 {
03     x_err_t ret = EOK;
04     static struct I2cBus i2c_bus;
05     memset(&i2c_bus, 0, sizeof(struct I2cBus));
06
07     static struct I2cDriver i2c_driver;
08     memset(&i2c_driver, 0, sizeof(struct I2cDriver));
09
10     I2cGpioInit();
11
12     /* Enable I2C Peripheral*/
13     FCG_Fcg1PeriphClockCmd(I2C_FCG_USE, ENABLE);
14
15     i2c_driver.configure = I2cDrvConfigure;
16
17     ret = BoardI2cBusInit(&i2c_bus, &i2c_driver);
18     if (ret != EOK) {
19         KPrintf("board_i2c_init error ret %u\n", ret);
20         return ERROR;
21     }
22
23     ret = BoardI2cDevBend();
24     if (EOK != ret) {
25         KPrintf("board_i2c_init error ret %u\n", ret);
26         return ERROR;
27     }
28
29     return ret;
30 }
```

在代码 4-6-2 的 BoardI2cBusInit 函数中，代码行 5~10 对 I2C 总线进行初始化；代码行 12~17 对 I2C 驱动进行初始化；代码行 19~24 将 I2C 驱动挂接到 I2C 总线上。在代码 4-6-3 的 BoardI2cDevBend 函数中，代码行 3~4 声明了两个变量 ret 和 i2c_device0，并对结构体变量 i2c_device0 进行初始化；代码行 9~13 中对 I2C 设备进行注册；代码行 15~19 将 I2C 设备挂接到 I2C 总线。

代码 4-6-130 I2C 驱动挂接过程

```
01 static int BoardI2cBusInit(struct I2cBus *i2c_bus, struct I2cDriver *i2c_driver)
02 {
03     x_err_t ret = EOK;
04
05     /* Init the i2c bus */
06     ret = I2cBusInit(i2c_bus, I2C_BUS_NAME_1);
07     if (EOK != ret) {
08         KPrintf("board_i2c_init I2cBusInit error %d\n", ret);
09         return ERROR;
10     }
11
12     /* Init the i2c driver*/
13     ret = I2cDriverInit(i2c_driver, I2C_DRV_NAME_1);
14     if (EOK != ret) {
15         KPrintf("board_i2c_init I2cDriverInit error %d\n", ret);
16         return ERROR;
17     }
18
19     /* Attach the i2c driver to the i2c bus*/
20     ret = I2cDriverAttachToBus(I2C_DRV_NAME_1, I2C_BUS_NAME_1);
21     if (EOK != ret) {
22         KPrintf("board_i2c_init I2cDriverAttachToBus error %d\n", ret);
23         return ERROR;
24     }
25
26     return ret;
27 }
```

代码 4-6-131 I2C 设备挂接过程

```
01 static int BoardI2cDevBend(void)
02 {
03     x_err_t ret = EOK;
04     static struct I2cHardwareDevice i2c_device0;
05     memset(&i2c_device0, 0, sizeof(struct I2cHardwareDevice));
06
07     i2c_device0.i2c_dev_done = &i2c_dev_done;
08 }
```

```

09     ret = I2cDeviceRegister(&i2c_device0, NONE, I2C_1_DEVICE_NAME_0);
10     if (EOK != ret) {
11         KPrintf("board_i2c_init I2cDeviceInit device %s error %d\n",
I2C_1_DEVICE_NAME_0, ret);
12         return ERROR;
13     }
14
15     ret = I2cDeviceAttachToBus(I2C_1_DEVICE_NAME_0, I2C_BUS_NAME_1);
16     if (EOK != ret) {
17         KPrintf("board_i2c_init I2cDeviceAttachToBus device %s error %d\n",
I2C_1_DEVICE_NAME_0, ret);
18         return ERROR;
19     }
20
21     return ret;
22 }

```

步骤（2）的实现过程如代码 4-6-4 所示，该流程是一个实现 I2C 设备的打开、关闭及读写过程。根据结构体 I2cDevDone 的定义，其中 .open、.close、.write 和 .read 分别表示 I2C 的打开、关闭、读、写。此时打开和关闭功能的值为 NONE，表示未启用该功能。I2cWriteData 和 I2cReadData 分别是实现写和读数据功能的具体函数。

代码 4-6-132 I2C 设备的打开、关闭及读写

```

01 static const struct I2cDevDone i2c_dev_done =
02 {
03     .dev_open = NONE,
04     .dev_close = NONE,
05     #ifdef I2C_DEVICE_SLAVE
06     .dev_write = I2cSlaveWriteData,
07     .dev_read = I2cSlaveReadData,
08     #else
09     .dev_write = I2cMasterWriteData,
10     .dev_read = I2cMasterReadData,
11     #endif
12 }

```

I2C 设备的驱动配置过程如代码 4-6-5 所示。在 I2cDrvConfigure 函数中，代码行 3~4 进行输入参数 drv 和 configure_info 的检查。代码行 6~7 声明了 2 个变量。代码行 9~16 实现了

具体的配置过程，其中，代码行 8 中 switch 语句的条件是获取输入参数 `configure_info` 的配置命令 `configure_cmd`，该命令有两种接口 `OPE_INT` 和 `OPE_CFG`；

代码 4-6-133 I2C 设备的驱动配置

```
01 static uint32 I2cDrvConfigure(void *drv, struct BusConfigureInfo *configure_info)
02 {
03     NULL_PARAM_CHECK(drv);
04     NULL_PARAM_CHECK(configure_info);
05
06     x_err_t ret = EOK;
07     struct I2cDriver *i2c_drv = (struct I2cDriver *)drv;
08
09     switch (configure_info->configure_cmd)
10     {
11         case OPE_INT:
12             ret = I2cInit(i2c_drv, configure_info);
13             break;
14         default:
15             break;
16     }
17
18     return ret;
19 }
```

6.3 I2C 设备应用

I2C 设备应用实现过程如代码 4-6-6 所示。代码行 2~6 打开一个 I2C 设备驱动。代码行 8 设置从设备的地址。代码行 10~17 并通过 IO 控制函数进行配置。代码行 19~28 使用 I2C 总线进行写和读。

代码 4-6-134 I2C 设备应用

```
01 void TestI2C(void){
02     int iic_fd = PrivOpen(I2C_DEV_DRIVER, O_RDWR);
03     if (iic_fd < 0){
04         printf("open iic_fd fd error:%d\n", iic_fd);
05         return;
06     }
07
08     uint16 iic_address = I2C_SLAVE_ADDRESS;
```

```
09
10     struct PrivIoctlCfg ioctl_cfg;
11     ioctl_cfg.ioctl_driver_type = I2C_TYPE;
12     ioctl_cfg.args = (void *)&iic_address;
13     if (0 != PrivIoctl(iic_fd, OPE_INT, &iioctl_cfg)){
14         printf("ioctl iic fd error %d\n", iic_fd);
15         PrivClose(iic_fd);
16         return;
17     }
18
19     char tmp_buff[100];
20     while (1){
21         PrivTaskDelay(1000);
22         PrivWrite(iic_fd, "Hello World!\n", sizeof("Hello World!\n"));
23         printf("msg send:%s\n", "Hello World!\n");
24         PrivTaskDelay(1000);
25         memset(tmp_buff, 0, sizeof(tmp_buff));
26         PrivRead(iic_fd, tmp_buff, sizeof(tmp_buff));
27         printf("msg recv:%s\n", tmp_buff);
28     }
29
30     PrivClose(iic_fd);
31     return;
32 }
```

第7章 SPI 驱动

7.1 SPI 介绍

7.1.1 SPI 基本介绍

SPI是一种串行通信协议，在嵌入式系统中被广泛应用。它可以在多个设备之间进行全双工通信，包括芯片（比如存储器、传感器、LCD屏幕等）和其他外设。SPI通信协议由四根线组成：串行数据输入线（MOSI）、串行数据输出线（MISO）、串行时钟线（SCK）和片选线（SS）。在SPI通信中，每个设备都有一个片选线，主控芯片可以通过选择某个片选线来选择需要进行通信的设备。

在 SPI 通信中，主控芯片可以向设备发送数据，也可以从设备中接收数据。数据的传输是通过 SCK 线上的时钟信号来控制的，数据的传输速率可以通过调整时钟频率来实现。同

时，SPI 协议还支持全双工通信，即主控芯片和设备可以同时发送和接收数据，以提高通信效率。

HC32F4A0 搭载 6 个通道的串行外设接口 SPI，支持高速全双工串行同步传输，方便地与外围设备进行数据交换。用户可以根据需要进行三线/四线，主机/从机及波特率范围的设置。HC32F4A0 中 SPI 有全双工串行通信和只发送串行通信两种通信方式，可通过 SPI 控制寄存器 SPI_CR1 的 TXMDS 位进行选择。

7.1.2 中断说明

SPI 的接收缓冲器满，发送缓冲器空，模式故障（过载，欠载，奇偶校验错误）和 SPI 空闲可以作为中断源也可以作为内部事件触发源，传送完成只可以作为内部触发源。过载，欠载和奇偶校验错误的中断被集成为 SPI 错误中断 SPEI，所以需要通过标志判断实际发生的中断源。一旦条件成立就产生相应的中断/内部触发请求。对于接收缓冲满和发送缓冲器空的中断源，需要通过数据传送改变缓冲器状态来清除。

7.1.3 寄存器说明

一. SPI 数据寄存器（SPI_DR）

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
SPD[31:16]															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
SPD[15:0]															

- b31~b0: SPI 存储数据

二. SPI 控制寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
PAE	PAOE	PATE	MODFE	IDIE	RXIE	TXIE	EIE	CSUSPE	SPE	SPLPBK2	SPLPBK	MSTR	—	TXMDS	SPIMDS

- b31~b16: 保留位，读的时候读出 0，写的时候写入 0
- b15: 奇偶校验允许，0 表示发送数据不附加奇偶校验位，接受数据不进行奇偶校验，1 表示发送数据附加奇偶校验位，接收数据进行奇偶校验
- b14: 0 表示选择偶校验进行发送和接收，1 表示选择奇校验进行发送和接收
- b13: 0 表示奇偶校验自诊断功能无效，1 表示奇偶校验自诊断功能有效
- b12: 0 表示禁止模式故障错误检测，1 表示允许模式故障错误检测

- b11: 0 表示禁止空闲中断请求产生, 1 表示允许空闲中断请求产生
- b10: 0 表示禁止 SPI 接收中断请求产生, 1 表示允许 SPI 接收中断请求产生
- b9: 0 表示禁止 SPI 发送中断请求产生, 1 表示允许 SPI 发送中断请求产生
- b8: 0 表示禁止 SPI 错误中断请求产生, 1 表示允许 SPI 错误中断请求产生
- b7: 0 表示通讯自动挂起功能无效, 1 表示通讯自动挂起功能有效
- b6: 0 表示 SPI 功能无效, 1 表示 SPI 功能有效
- b5: SPI 回环 2 位, 0 表示正常模式, 1 表示回环模式 (发送数据=接收数据)
- b4: SPI 回环位, 0 表示正常模式, 1 表示回环模式 (发送数据的反相=接收数据)
- b3: SPI 主从模式选择, 0 表示从机模式, 1 表示主机模式
- b2: 保留位, 读的时候读出 0, 写的时候写入 0
- b1: 通信模式选择, 0 表示全双工同步串行通信, 1 表示只进行发送串行通信
- b0: SPI 模式选择, 0 表示 SPI 运行 (4 线式), 1 表示时钟同步运行 (3 线式)

三. SPI 通信配置寄存器 1 (SPI_CFG1)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
	MIDI[2:0]			-	MSSDL[2:0]				MSSI[2:0]			-	-	-	
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	-	SS3P V	SS2P V	SS1P V	SS0P V	-	SPR DTD	-	-	-	-	FTHLV[1:0]	

- b31: 保留位, 读的时候读出 0, 写的时候写入 0
- b30~b28: 主机下次存取数据间隔闲置事件设定位
- b27: 保留位, 读的时候读出 0, 写的时候写入 0
- b26~b24: 主机 SS 无效延迟设定位
- b23: 保留位, 读的时候读出 0, 写的时候写入 0
- b22~b20: 主机 SS 闲置时间设定位
- b19~b12: 保留位, 读的时候读出 0, 写的时候写入 0
- b11: SS3 信号极性设定, 0 表示 SS3 信号的 Low 电平有效, 1 表示 SS3 信号的 High 电平有效
- b10: SS2 信号极性设定, 0 表示 SS2 信号的 Low 电平有效, 1 表示 SS2 信号的 High 电平有效

- b9: SS1 信号极性设定, 0 表示 SS1 信号的 Low 电平有效, 1 表示 SS1 信号的 High 电平有效
- b8: SS0 信号极性设定, 0 表示 SS0 信号的 Low 电平有效, 1 表示 SS0 信号的 High 电平有效
- b7: 保留位, 读的时候读出 0, 写的时候写入 0
- b6: 数据寄存器读取对象选择, 0 表示 SPI_DR 读接收缓冲器, 1 表示 SPI_DR 读发送缓冲器
- b5: 保留位, 读的时候读出 0, 写的时候写入 0
- b4: 保留位, 读的时候读出 1, 写的时候写入 1
- b3~b2: 保留位, 读的时候读出 0, 写的时候写入 0
- b1~b0: 帧数设定, 00 表示 1 帧, 01 表示 2 帧, 10 表示 3 帧, 11 表示 4 帧

四. SPI 状态寄存器 (SPI_SR)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	-	-	-	-	-	RDF F	—	TDE F	UDR ERF	PERF	MOD FERF	IDLN F	OVR ERF

- b31~b8: 保留位, 读的时候读出 0, 写的时候写入 0
- b7: 接收缓冲器满标志, 0 表示 SPI_DR 寄存器无数据, 1 表示 SPI_DR 寄存器有数据
- b6: 保留位, 读的时候读出 0, 写的时候写入 0
- b5: 发送缓冲器满标志, 0 表示发送寄存器有数据, 1 表示发送寄存器无数据
- b4: 欠载错误标志, 0 表示模式故障错误发生, 1 表示欠载错误发生
- b3: 奇偶校验错误标志, 0 表示未发生奇偶校验错误, 1 表示发生奇偶校验错误
- b2: 模式故障错误标志, 0 表示未发送模式故障错误, 1 表示发生故障模式错误
- b1: SPI 空闲标志, 0 表示 SPI 为空闲状态, 1 表示 SPI 为传送状态
- b0: 过载错误标志, 0 表示未发生过载错误, 1 表示发生过载错误

五. SPI 通信配置寄存器 2 (SPI_CFG2)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
MSSI E	MSS DLE	MIDI E	LSBF	DSIZE[3:0]				SSA[2:0]			MBR[2:0]			CPO L	CPH A

- b31~b16: 保留位，读的时候读出 0，写的时候写入 0
- b15: SCK 延迟允许，0 表示 SCK 延迟为 1 个 SCK，1 表示延迟为 SPI_CFG1 中的 MSSI 位的设定值
- b14: SCK 无效延迟允许，0 表示 SS 无效延迟为 1 个 SCK，1 表示延迟为 SPI_CFG1 中的 MSSDL 位的设定值
- b13: SPI 下次存取延迟允许，0 表示下次存取延迟为 1 个 SCK+2 个 PCLK1，1 表示下次存取延迟为 SPI_CFG1 中的 MIDI 位的设定值
- b12: SPI LSB first 位，0 表示 MSBfirst，1 表示 LSBfirst
- b11~b8: SPI 数据长度设定位
- b7~b5: SS 信号有效设定位
- b4~b2: 位速率分频设定位
- b1: SCK 极性设定位，0 表示空闲时的 SCK 为 Low 电平，1 表示空闲时的 SCK 为 High 电平
- b0: SCK 相位设定位，0 表示在奇数边沿进行数据采样，在偶数边沿数据发生变化，1 表示在奇数边沿数据发生变化，在偶数边沿进行数据采样

7.2 SPI 驱动介绍

SPI 设备通过 SPI BUS 挂接到 BUS 总线上，其中 SPI BUS 的结构体在 bus_spi.h 文件中进行了定义。每个设备总线都关联一个 drv 和一个 dev，其中，drv 结构体也在 bus_spi.h 中进行了定义，dev 结构体在 dev_spi.h 中进行了定义。另外，SPI 驱动及设备的注册流程在 connect_spi.c 中进行了实现。接下来，针对这 3 个 SPI 相关的具体结构和实现流程进行详细介绍。

7.2.1 bus_spi.h 文件

bus_spi.h 文件中声明了 SPI 总线结构体 SpiBus 和驱动结构体 SpiDriver。SpiBus 的结构体定义如下：

```

struct SpiBus{
    struct Bus bus;
    void *private_data;
};

```

其中 bus 是通用管理总线；private_data 是传递参数的通用指针。

SpiDriver 的结构体定义如下：

```

struct SpiDriver{
    struct Driver driver;
    uint32 (*configure) (void *drv, struct BusConfigureInfo *configure_info);
};

```

其中 driver 表示通用管理驱动；configure 表示配置函数。

另外，在 bus_spi.h 文件中还定义了一些函数接口，具体均在 bus_spi.c 文件中实现：

- int SpiBusInit(struct SpiBus *spi_bus, const char *bus_name), 用于初始化 SPI 总线，其中 spi_bus 表示初始化的 SPI 总线；bus_name 表示总线名称。通过调用 BusRegister 函数将 SPI 总线注册到 BUS 管理总线链表。
- int SpiReleaseBus(struct SpiBus *spi_bus), 用于释放 SPI 总线，即将 SPI 总线从 BUS 管理总线链表中移除，其中 spi_ bus 表示需要释放的 SPI 总线。一般该接口函数与 Spi BusInit 函数配对使用，对 SPI 总线进行注册和释放。
- int SpiDriverInit(struct SpiDriver *spi_driver, const char *driver_name), 用于初始化 SPI 驱动，其中 spi_ driver 表示需要初始化的 SPI 驱动；driver_name 表示驱动名称。该接口通过调用 SpiDriverRegister 函数将 SPI 驱动注册到 SPI 驱动总线。
- int SpiDriverAttachToBus(const char *drv_name, const char *bus_name), 用于将 SPI 驱动挂接到 SPI BUS 总线，其中 drv_name 表示驱动名称；bus_name 表示总线名称。
- DriverType SpiDriverFind(const char *drv_name, enum DriverType_e drv_type), 用于查找挂接的 SDIO 驱动，其中 drv_name 表示驱动名称；drv_type 表示驱动类型。

7.2.2 dev_spi.h 文件

在 dev_spi.h 文件中声明了 SPI 设备结构体 SpiHardwareDevice，其具体的定义如下：

```

struct SpiHardwareDevice{
    struct HardwareDev haldev;
    SpiDeviceParam spi_param;
    x_bool spi_dev_flag;
};

```

```
const struct SpiDevDone *spi_dev_done;
void *private_data;
};
```

其中 `haldev` 用于描述一个设备结构；`spi_param` 用于描述 SPI 的参数；`spi_dev_flag` 表示 SPI 设备标记；`dev_done` 用于表示一个 SPI 设备的函数操作集；`private_data` 是传递参数的通用指针。

结构体 `SpiDevDone` 的定义如下：

```
struct SpiDevDone{
    uint32 (*dev_open) (struct SpiHardwareDevice *dev);
    uint32 (*dev_close) (struct SpiHardwareDevice *dev);
    uint32 (*dev_write) (struct SpiHardwareDevice *dev, struct SpiDataStandard *msg);
    uint32 (*dev_read) (struct SpiHardwareDevice *dev, struct SpiDataStandard *msg);
};
```

可以看出，该结构体是用于描述 SPI 设备的打开、关闭、读写功能。

另外，在 `dev_spi.h` 文件中还定义了一些函数接口：

- `int SpiDeviceRegister(struct SpiHardwareDevice *spi_device, void *spi_param, const char *device_name)`，用于初始化并注册 SPI 设备到设备管理链表上，其中 `spi_device` 表示需要注册的 SPI 设备；`spi_param` 表示 SPI 参数；`device_name` 表示设备名称。
- `int SpiDeviceAttachToBus(const char *dev_name, const char *bus_name)`，用于将 SPI 设备挂接到 SPI BUS 总线，其中 `dev_name` 表示设备名称；`bus_name` 表示总线名称。
- `HardwareDevType SpiDeviceFind(const char *dev_name, enum DevType dev_type)`，用于查找挂接的 SPI 设备，其中 `dev_name` 表示设备名称；`dev_type` 表示设备类型。

7.2.3 connect_spi.c 文件

`connect_spi.c` 文件中实现了 SPI 驱动及设备的注册，该流程主要包含 3 个步骤：

- （1）完成 SPI 的驱动和设备到 SPI 总线的挂接（或称为绑定）；
- （2）实现 SPI 设备的打开、关闭及读写方法，其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植；
- （3）实现 SPI 设备的驱动配置，需结合 SPI 特性及操作系统的接口定义进行相应的开发。

步骤（1）的 SPI 驱动及设备绑定过程由代码 4-7-1 中的 BoardSpiBusInit 函数和代码 4-7-2 中的 HwSpiDeviceAttach 函数共同完成。BoardSpiBusInit 函数主要完成 SPI 驱动的绑定，代码行 4~8 使用 SpiBusInit 函数对 SPI 总线进行初始化。代码行 10~14 使用 SpiDriverInit 函数对驱动进行初始化。代码行 16~20 使用 SpiDriverAttachToBus 函数将 SPI 驱动绑定到总线。BoardSpiDevBend 函数主要完成 SPI 设备的绑定，代码行 7~19 主要完成参数的声明及初始化；代码行 21~25 通过 SpiDeviceRegister 函数注册 SPI 设备。代码行 27~31 通过 SpiDeviceAttachToBus 将 SPI 设备挂接到总线。

代码 4-7-135 SPI 驱动绑定过程

```

01 static int BoardSpiBusInit(struct SpiBus *spi_bus, struct SpiDriver *spi_driver, const char
    *bus_name, const char *drv_name)
02 {
03     x_err_t ret = EOK;
04
05     /*Init the spi bus */
06     ret = SpiBusInit(spi_bus, bus_name);
07     if (EOK != ret) {
08         KPrintf("Board_Spi_init SpiBusInit error %d\n", ret);
09         return ERROR;
10     }
11
12     /*Init the spi driver*/
13     ret = SpiDriverInit(spi_driver, drv_name);
14     if (EOK != ret) {
15         KPrintf("Board_Spi_init SpiDriverInit error %d\n", ret);
16         return ERROR;
17     }
18
19     /*Attach the spi driver to the spi bus*/
20     ret = SpiDriverAttachToBus(drv_name, bus_name);
21     if (EOK != ret) {
22         KPrintf("Board_Spi_init SpiDriverAttachToBus error %d\n", ret);
23         return ERROR;
24     }
25
26     return ret;
27 }

```

代码 4-7-136 SPI 设备绑定过程

```

01  /*Attach the spi device to the spi bus*/
02  static int BoardSpiDevBend(void)
03  {
04      x_err_t ret = EOK;
05
06  #ifdef SPI_1_DEVICE_NAME_0
07      static struct SpiHardwareDevice spi1_device0;
08      memset(&spi1_device0, 0, sizeof(struct SpiHardwareDevice));
09
10      static struct SpiSlaveParam spi1_slaveparam0;
11      memset(&spi1_slaveparam0, 0, sizeof(struct SpiSlaveParam));
12
13      spi1_slaveparam0.spi_slave_id = SPI1_DEVICE_SLAVE_ID_0;
14      spi1_slaveparam0.spi_cs_gpio_pin = SPI1_SS_PIN;
15      spi1_slaveparam0.spi_cs_gpio_port = SPI1_SS_PORT;
16
17      spi1_device0.spi_param.spi_slave_param = &spi1_slaveparam0;
18
19      spi1_device0.spi_dev_done = &(spi_dev_done);
20
21      ret = SpiDeviceRegister(&spi1_device0, (void *)&spi1_device0.spi_param,
                             SPI_1_DEVICE_NAME_0);
22      if (EOK != ret) {
23          KPrintf("BoardSpiDevBend SpiDeviceRegister device %s error %d\n",
                  SPI_1_DEVICE_NAME_0, ret);
24          return ERROR;
25      }
26
27      ret = SpiDeviceAttachToBus(SPI_1_DEVICE_NAME_0, SPI_BUS_NAME_1);
28      if (EOK != ret) {
29          KPrintf("BoardSpiDevBend SpiDeviceAttachToBus device %s error %d\n",
                  SPI_1_DEVICE_NAME_0, ret);
30          return ERROR;
31      }
32  #endif
33
34  #ifdef SPI_6_DEVICE_NAME_0
35      static struct SpiHardwareDevice spi6_device0;
36      memset(&spi6_device0, 0, sizeof(struct SpiHardwareDevice));

```

```

37
38     static struct SpiSlaveParam spi6_slaveparam0;
39     memset(&spi6_slaveparam0, 0, sizeof(struct SpiSlaveParam));
40
41     spi6_slaveparam0.spi_slave_id = SPI1_DEVICE_SLAVE_ID_0;
42     spi6_slaveparam0.spi_cs_gpio_pin = SPI6_SS_PIN;
43     spi6_slaveparam0.spi_cs_gpio_port = SPI6_SS_PORT;
44
45     spi6_device0.spi_param.spi_slave_param = &spi6_slaveparam0;
46
47     spi6_device0.spi_dev_done = &(spi_dev_done);
48
49     ret = SpiDeviceRegister(&spi6_device0, (void *)&(spi6_device0.spi_param),
SPI_6_DEVICE_NAME_0);
50     if (EOK != ret) {
51         KPrintf("BoardSpiDevBend SpiDeviceRegister device %s error %d\n",
                    SPI_6_DEVICE_NAME_0, ret);
52         return ERROR;
53     }
54
55     ret = SpiDeviceAttachToBus(SPI_6_DEVICE_NAME_0, SPI_BUS_NAME_6);
56     if (EOK != ret) {
57         KPrintf("BoardSpiDevBend SpiDeviceAttachToBus device %s error %d\n",
                    SPI_6_DEVICE_NAME_0, ret);
58         return ERROR;
59     }
60 #endif
61
62     return ret;
63 }

```

步骤（2）的实现过程如代码 4-7-3 所示，该流程是一个实现 SPI 设备的打开、关闭及读写过程。根据结构体 SpiDevDone 的定义，打开和关闭的值均为 NONE，表示该功能未启用。而读和写的值分别是 SpiWriteData 和 SpiReadData，表明这两个功能是打开的。

代码 4-7-137 SPI 设备的打开、关闭和读写

```

01 static const struct SpiDevDone spi_dev_done =
02 {
03     .dev_open = NONE,
04     .dev_close = NONE,

```

```

05     .dev_write = SpiWriteData,
06     .dev_read = SpiReadData,
07 };

```

SPI 设备的驱动配置过程如代码 4-7-4 所示。在 SpiDrvConfigure 函数中，代码行 2~3 进行输入参数 drv 和 configure_info 的检查。代码行 5~7 声明了 3 个变量。代码行 9~19 根据配置命令进行选择，若配置命令为 OPE_INT，则根据 SpiDrvInit 函数执行 SPI 驱动初始化；若配置命令为 OPE_CFG，则根据 SpiDrvConfigure 函数进行 SPI 驱动配置。

代码 4-7-138 SPI 设备的驱动配置

```

01 static uint32 SpiDrvConfigure(void *drv, struct BusConfigureInfo *configure_info){
02     NULL_PARAM_CHECK(drv);
03     NULL_PARAM_CHECK(configure_info);
04
05     x_err_t ret = EOK;
06     struct SpiDriver *spi_drv = (struct SpiDriver *)drv;
07     struct SpiMasterParam *spi_param;
08
09     switch (configure_info->configure_cmd){
10         case OPE_INT:
11             ret = SpiDrvInit(spi_drv);
12             break;
13         case OPE_CFG:
14             spi_param = (struct SpiMasterParam *)configure_info->private_data;
15             ret = SpiDrvConfigure(spi_drv, spi_param);
16             break;
17         default:
18             break;
19     }
20
21     return ret;
22 }

```

7.3 SPI 设备应用

SPI 设备应用的测试通过测试能否读写 lora 寄存器的方式来进行。代码 3~4 行进行相关变量的定义，6~7 行对写的寄存器地址进行定义，9 行打开设备总线，13 行对设备进行写，17 行读取设备里面的内容，19 行关闭设备，最后打印出读取到的数据，以测试与写进去的是否一致。

代码 4-7-139 SPI 设备应用

```
01 static void TestSpiRead(void)
02 {
03     struct BusBlockWriteParam write_param;
04     struct BusBlockReadParam read_param;
05
06     uint8 write_addr = 0x06 & 0x7F;
07     uint8 read_data = 0;
08
09     BusDevOpen(dev);
10
11     write_param.buffer = (void *)&write_addr;
12     write_param.size = 1;
13     BusDevWriteData(dev, &write_param);
14
15     read_param.buffer = (void *)&read_data;
16     read_param.size = 1;
17     BusDevReadData(dev, &read_param);
18
19     BusDevClose(dev);
20
21     KPrintf("read data from lora 0x06 register, receive data 0x%x\n", read_data);
22 }
23 SHELL_EXPORT_CMD(SHELL_CMD_PERMISSION(0)|SHELL_CMD_TYPE(
SHELL_TYPE_CMD_MAIN), TestSpiRead, TestSpiRead, read data from lora register);
```

第8章 QSPI flash 驱动

8.1 QSPI 介绍

8.1.1 QSPI 基本介绍

四线式串行外设接口（QSPI）是一个存储器控制模块，主要用于和带 SPI 兼容接口的串行 ROM 进行通信。其对象主要包括有串行闪存，串行 EEPROM 以及串行 FeRAM。

8.1.2 中断说明

QSPI 可以通过自动将 MCU 的外部 ROM 读取总线周期转换为 QSPI 总线周期来对串行闪存进行读取。但串行闪存还有很多不同的追加功能，诸如 ID 信息读取，擦除，写入及状态信息读取等。这些功能并没有一套标准的指令来进行设置，并且随着串行闪存新功能

的迅速增加，硬件层面上的对应变得愈发的困难。针对这种情况，QSPI 提供了直接通信模式，用户可通过软件直接对串行闪存进行控制。由此模式软件可以产生任意所需的 QSPI 总线周期。

当直接通信模式下检测到对 ROM 的读访问操作时，QSSR 寄存器的 RAER 位被置成 1，这时候 QSPI 将产生总线硬件错误中断。该中断请求将会一直被保留直到 RAER 位被清零。

8.1.3 寄存器说明

一. QSPI 控制寄存器 (QSCR)

b31	b30	b29	b28	b27	b26	b25	b24
—	—	—	—	—	—	—	—
b23	b22	b21	b20	b19	b18	b17	b16
—	—	DIV[5:0]					
b15	b14	b13	b12	b11	b10	b9	b8
—	—	DPRSL[1:0]		APRSL[1:0]		IPRSL[1:0]	
b7	b6	b5	b4	b3	b2	b1	b0
SPIMD3	XIPE	DCOME	PFSAE	PFE	MDSEL[2:0]		

- b31~b22: 保留位，读的时候读出 0，写的时候写入 0
- b21~b16: 基准时钟选择位
- b15~b14: 保留位，读的时候读出 0，写的时候写入 0
- b13~b12: 数据接收阶段 SPI 协议选择，00 表示扩展式 SPI 协议，01 表示二线式 SPI 协议，10 表示四线式 SPI 协议，11 表示设定禁止
- b11~b10: 数据发送阶段 SPI 协议选择，00 表示扩展式 SPI 协议，01 表示二线式 SPI 协议，10 表示四线式 SPI 协议，11 表示设定禁止
- b9~b8: 指令发送阶段 SPI 协议选择，00 表示扩展式 SPI 协议，01 表示二线式 SPI 协议，10 表示四线式 SPI 协议，11 表示设定禁止
- b7: SPI 模式选择，0 表示模式 0，1 表示模式 3
- b6: XIP 模式许可，0 表示 XIP 模式禁止，1 表示 XIP 模式许可
- b5: QSPI 总线通信模式选择，0 表示 ROM 访问模式，1 表示直接通信模式
- b4: 选择重置预读取动作的位置，0 表示当前预读取动作在字节边界中止，1 表示当前预读取动作即时中止

- b3: 预读取功能有效或者无效选择, 0 表示预读取功能无效, 1 表示预读取功能有效
- b2~b0: 串行接口读取模式选择, 000 表示标准读, 001 表示快速读, 010 表示二线式输出快速读, 011 表示二线式输入输出快速读, 100 表示四线式输出快速读, 101 表示四线式输入输出快速读, 110 表示自定义标准读, 111 表示自定义快速读

二. QSPI 片选控制寄存器 (QSCSCR)

b31	b30	b29	b28	b27	b26	b25	b24
—	—	—	—	—	—	—	—
b23	b22	b21	b20	b19	b18	b17	b16
—	—	—	—	—	—	—	—
b15	b14	b13	b12	b11	b10	b9	b8
—	—	—	—	—	—	—	—
b7	b6	b5	b4	b3	b2	b1	b0
—	—	SSNW[1:0]		SSHW[3:0]			

- b31~b6: 保留位, 读的时候读出 0, 写的时候写入 0
- b5~b4: QSPI 总线访问后 QSSN 有效时间延长功能选择, 00 表示不延长 QSSN 有效时间, 01 表示将 QSSN 有效时间延长 32 个 QSCK 周期, 10 表示将 QSSN 有效时间延长 128 哥 QSCK 周期, 11 表示将 QSSN 有效时间无限延长
- b3~b0: QSSN 信号最小无效时间选择, 0000 表示 1 个 QSCK 周期, 0001 表示 2 个 QSCK 周期, 依次类推, 1111 表示 16 个 QSCK 周期

三. QSPI 格式控制寄存器 (QSFCR)

b31	b30	b29	b28	b27	b26	b25	b24
—	—	—	—	—	—	—	—
b23	b22	b21	b20	b19	b18	b17	b16
—	—	—	—	—	—	—	—
b15	b14	b13	b12	b11	b10	b9	b8
DUTY	—	—	—	DMCYCN[3:0]			
b7	b6	b5	b4	b3	b2	b1	b0
—	WPOL	SSNLD	SSNHD	—	4BIC	AWSL[1:0]	

- b31~b16: 保留位, 读的时候读出 0, 写的时候写入 0
- b15: QSCK 输出波形占空比补正, 0 表示不进行占空比补正, 1 表示将 QSCK 的上升沿滞后 0.5 个 HCLK 周期
- b14~b12: 保留位, 读的时候读出 0, 写的时候写入 0
- b11~b8: 使用快读读指令时许你周期数量选择, 0000 表示 3 个 QSCK 周期, 0001 表示 4 个 QSCK 周期, 以此类推, 1111 表示 18 个 QSCK 周期
- b7: 保留位, 读的时候读出 0, 写的时候写入 0

- b6: WP 管脚电平设置, 0 表示低电平, 1 表示高电平
- b5: QSSN 信号输出时序选择, 0 表示比 QSCK 第一个上升沿提前 0.5 个 QSCK 输出 QSSN, 1 表示 QSCK 第一个上升沿提前 1.5 个 QSCK 输出 QSSN
- b4: QSSN 信号释放时序选择, 0 表示比 QSCK 第一个上升沿滞后 0.5 个 QSCK 释放 QSSN, 1 表示 QSCK 第一个上升沿滞后 1.5 个 QSCK 释放 QSSN
- b3: 保留位, 读的时候读出 0, 写的时候写入 0
- b2: 当地址宽度为 4 字节时的读指令代码选择, 0 表示不使用 4 字节地址读指令代码, 1 表示使用 4 字节地址读指令代码
- b1~b0: 串行接口地址宽度选择, 00 表示 1 字节, 01 表示 2 字节, 10 表示 3 字节, 11 表示 4 字节

四. QSPI 状态寄存器 (QSSR)

b31	b30	b29	b28	b27	b26	b25	b24
—	—	—	—	—	—	—	—
b23	b22	b21	b20	b19	b18	b17	b16
—	—	—	—	—	—	—	—
b15	b14	b13	b12	b11	b10	b9	b8
PFAN	PFFUL	—	PFNUM[4:0]				
b7	b6	b5	b4	b3	b2	b1	b0
RAER	XIPF	—	—	—	—	—	BUSY

- b31~b16: 保留位, 读的时候读出 0, 写的时候写入 0
- b15: 预读取动作状态信号, 0 表示预读取处于动作状态, 1 表示预读取处于停止状态
- b14: 预读取缓冲区状态信号, 0 表示预读取缓冲区有剩余空间, 1 表示预读取缓冲区数据已满
- b13: 保留位, 读的时候读出 0, 写的时候写入 0
- b12~b8: 预读取缓冲区已存储数据字节数显示, 00000 表示 0 字节, 以此类推, 一直到 10010 表示 18 个字节, 后面比 10010 大的值都为无效值
- b7: 直接通信模式下发生 ROM 访问的错误标志位, 0 表示未检测到发生 ROM 访问, 1 表示检测到发生 ROM 访问
- b6: XIP 模式状态信号, 0 表示非 XIP 模式, 1 表示 XIP 模式
- b5~b1: 保留位, 读的时候读出 0, 写的时候写入 0
- b0: 直接通信模式下 QSPI 总线工作状态标志位, 0 表示总线空闲, 无串行传输进程, 1 表示总线忙, 串行传输进程进行中

五. QSPI 直接通信指令寄存器 (QSDCOM)

b31	b30	b29	b28	b27	b26	b25	b24
—	—	—	—	—	—	—	—
b23	b22	b21	b20	b19	b18	b17	b16
—	—	—	—	—	—	—	—
b15	b14	b13	b12	b11	b10	b9	b8
—	—	—	—	—	—	—	—
b7	b6	b5	b4	b3	b2	b1	b0
DCOM[7:0]							

- b31~b8：保留位，读的时候读出 0，写的时候写入 0
- b7~b0：直接通信模式时的接口，通过 QSPI 总线进行直接通信。对该接口的读写访问会被转化成一个相应的 QSPI 总线周期。该接口只在直接通信模式下有效，ROM 访问模式下该接口禁止访问。

六. QSPI XIP 模式代码寄存器（QXCMD）

b31	b30	b29	b28	b27	b26	b25	b24
—	—	—	—	—	—	—	—
b23	b22	b21	b20	b19	b18	b17	b16
—	—	—	—	—	—	—	—
b15	b14	b13	b12	b11	b10	b9	b8
—	—	—	—	—	—	—	—
b7	b6	b5	b4	b3	b2	b1	b0
XIPMC [7:0]							

- b31~b8：保留位，读的时候读出 0，写的时候写入 0
- b7~b0：用于串行闪存的 XIP 模式代码

七. QSPI 系统配置寄存器（QSSR2）

b31	b30	b29	b28	b27	b26	b25	b24
—	—	—	—	—	—	—	—
b23	b22	b21	b20	b19	b18	b17	b16
—	—	—	—	—	—	—	—
b15	b14	b13	b12	b11	b10	b9	b8
—	—	—	—	—	—	—	—
b7	b6	b5	b4	b3	b2	b1	b0
RAERCLR	—	—	—	—	—	—	—

- b31~b8：保留位，读的时候读出 0，写的时候写入 0
- b7：写入 1 时表示将 QSSR 中的 RAER 位清零
- b6~b0：保留位，读的时候读出 0，写的时候写入 0

八. QSPI 外部扩展地址寄存器（QSEXAR）

b31	b30	b29	b28	b27	b26	b25	b24
EXADR[5:0]						—	—
b23	b22	b21	b20	b19	b18	b17	b16
—	—	—	—	—	—	—	—
b15	b14	b13	b12	b11	b10	b9	b8
—	—	—	—	—	—	—	—
b7	b6	b5	b4	b3	b2	b1	b0
—	—	—	—	—	—	—	—

- b31~b26: QSPI 外部地址高 6 位设置，配合 QSPI 的 ROM 访问窗口地址最大可访问 64MB*63 块外部 ROM 空间
- b25~b0: 保留位，读的时候读出 0，写的时候写入 0

8.2 QSPI flash 驱动介绍

QSPI 设备通过 SPI BUS 挂接到 BUS 总线上，其中 SPI BUS 的结构体在 `bus_spi.h` 文件中进行了定义。每个设备总线都关联一个 `drv` 和一个 `dev`，其中，`drv` 结构体也在 `bus_spi.h` 中进行了定义，`dev` 结构体在 `dev_spi.h` 中进行了定义。另外，QSPI flash 驱动及设备的注册流程在 `connect_flash.c` 和 `flash_spi.c` 中进行了实现。关于 `bus_spi.h` 和 `dev_spi.h` 的介绍可见本篇第 7 章，下面将围绕 QSPI 和 flash 驱动及设备的注册流程展开介绍。

8.2.1 flash_spi.c 文件

`flash_spi.c` 文件中主要实现了 FLASH 驱动及设备的注册。`flash` 驱动及设备的注册和功能实现需要在 SPI（或 QSPI）驱动实现的基础上，对 SPI（或 QSPI）的相关方法进行抽象，再根据 SFUD 协议对 FLASH 进行读写或配置等相关操作。

代码 4-8-1 是 flash 驱动及设备的初始化与注册的具体实现方法。其中参数 `bus_name`、`dev_name` 和 `drv_name` 分别对应了 SPI（或 QSPI）的总线名、设备名和驱动名称，`flash_name` 是所相应注册的 flash 设备名称。代码行 3 至 6 完成了对 `bus_name`、`dev_name`、`drv_name` 和 `bus_name` 参数的非空检查。代码行 10 至 21 通过 SPI 设备名称获取了 SPI 的设备结构体变量。代码行 23 至 39 完成了相应的 flash 设备结构体变量的声明与初始化。代码行 41 至 47 通过 SPI 的驱动名称获取了 SPI 的驱动结构体变量。代码行 49 至 61 对 flash 设备结构体变量进行了配置。代码行 63 至 69 对 flash 进行了硬件初始化并获取相关信息。代码行 71 至 73 根据从 flash 获取到的信息更新了内存扇区和块的大小。代码行 74 至 89 分别通过 `SpiDeviceRegister` 和 `SpiDeviceAttachToBus` 分别对 flash 设备进行了注册并绑定至对应的 SPI bus 总线上。

代码 4-8-140 flash 驱动及设备的初始化与注册

```

01 SpiFlashDeviceType SpiFlashInit(char *bus_name, char *dev_name,
                                char *drv_name, char *flash_name)
02 {
03     NULL_PARAM_CHECK(dev_name);
04     NULL_PARAM_CHECK(drv_name);
05     NULL_PARAM_CHECK(flash_name);
06     NULL_PARAM_CHECK(bus_name);
07
08     x_err_t ret;
09
10     HardwareDevType haldev = SpiDeviceFind(dev_name, TYPE_SPI_DEV);
11     if (NONE == haldev) {
12         KPrintf("SpiFlashInit find spi haldev %s error! \n", dev_name);
13         return NONE;
14     }
15
16     SpiFlashDeviceType spi_flash_dev = (SpiFlashDeviceType)malloc(
                                sizeof(struct SpiFlashDevice));
17     if (NONE == spi_flash_dev) {
18         KPrintf("SpiFlashInit malloc spi_flash_dev failed\n");
19         free(spi_flash_dev);
20         return NONE;
21     }
22
23     sfud_flash *sfud_flash_dev = (sfud_flash_t)malloc(sizeof(sfud_flash));
24     if (NONE == sfud_flash_dev) {
25         KPrintf("SpiFlashInit malloc sfud_flash_dev failed\n");
26         free(spi_flash_dev);
27         free(sfud_flash_dev);
28         return NONE;
29     }
30
31     memset(spi_flash_dev, 0, sizeof(struct SpiFlashDevice));
32     memset(sfud_flash_dev, 0, sizeof(sfud_flash));
33
34     spi_flash_dev->spi_dev = CONTAINER_OF(haldev,
                                struct SpiHardwareDevice, haldev);
35
36     spi_flash_dev->flash_dev.spi_dev_flag = RET_TRUE;

```

```

37     spi_flash_dev->flash_param.flash_private_data = (void *)sfud_flash_dev;
38     spi_flash_dev->flash_dev.haldev.dev_done = (struct HalDevDone *)&flash_done;
39     spi_flash_dev->flash_dev.haldev.dev_block_control = SpiFlashControl;
40
41     spi_flash_dev->spi_dev->haldev.owner_bus->owner_driver
                                = SpiDriverFind(drv_name, TYPE_SPI_DRV);
42     if (NONE == spi_flash_dev->spi_dev->haldev.owner_bus->owner_driver) {
43         KPrintf("SpiFlashInit find spi driver %s error! \n", drv_name);
44         free(spi_flash_dev);
45         free(sfud_flash_dev);
46         return NONE;
47     }
48
49     ret = FlashSpiConfigure(spi_flash_dev);
50     if (EOK != ret) {
51         KPrintf("SpiFlashInit configure failed\n");
52         free(spi_flash_dev);
53         free(sfud_flash_dev);
54         return NONE;
55     }
56
57     sfud_flash_dev->name = flash_name;
58     sfud_flash_dev->user_data = (void *)spi_flash_dev;
59
60     sfud_flash_dev->spi.name = dev_name;
61     sfud_flash_dev->spi.user_data = (void *)sfud_flash_dev;
62
63     ret = sfud_device_init(sfud_flash_dev);
64     if (SFUD_SUCCESS != ret) {
65         KPrintf("sfud_device_init failed %d\n", ret);
66         free(spi_flash_dev);
67         free(sfud_flash_dev);
68         return NONE;
69     }
70
71     spi_flash_dev->flash_param.flash_block_param.block_size
                                = sfud_flash_dev->chip.erase_gran;
72     spi_flash_dev->flash_param.flash_block_param.sector_bytes
                                = sfud_flash_dev->chip.erase_gran;
73     spi_flash_dev->flash_param.flash_block_param.sector_num
                                = sfud_flash_dev->chip.capacity / sfud_flash_dev->chip.erase_gran;

```

```

74
75     ret = SpiDeviceRegister(&spi_flash_dev->flash_dev, NONE, flash_name);
76     if (EOK != ret) {
77         KPrintf("SpiFlashInit SpiDeviceRegister device %s error %d\n", flash_name,
78 ret);
79         free(spi_flash_dev);
80         free(sfud_flash_dev);
81         return NONE;
82     }
83     ret = SpiDeviceAttachToBus(flash_name, bus_name);
84     if (EOK != ret) {
85         KPrintf("SpiFlashInit SpiDeviceAttachToBus
86                                     device %s error %d\n", flash_name, ret);
87         free(spi_flash_dev);
88         free(sfud_flash_dev);
89         return NONE;
90     }
91     spi_flash_dev->flash_lock = KMutexCreate();
92
93     return spi_flash_dev;
94 }

```

flash_spi.c 文件中还实现了 flash 设备的读写及配置方法。代码 4-8-2 是 flash 的读写方法的相关配置。对应实现 flash 设备的打开、关闭及读写过程。根据结构体 FlashDevDone 的定义，打开和关闭的值均为 NONE，表示该功能未启用。而读和写的值分别是 SpiFlashWrite 和 SpiFlashRead，表明这两个功能是打开的。

代码 4-8-141 Flash 设备的打开、关闭与读写

```

01 static const struct FlashDevDone flash_done =
02 {
03     .open = NONE,
04     .close = NONE,
05     .write = SpiFlashWrite,
06     .read = SpiFlashRead,
07 };

```

代码 4-8-3 对应为 flash 设备的配置过程。代码行 3 至 17 分别对所需的变量进行了声明和初始化。代码行 19 至 42 分别根据命令参数执行 if 条件下的语句。当命令为 OPER_BLK

_GETGEOME 将通过结构体指针赋值的形式将扇区和块的大小和数量传出；当命令为 OPER_BLK_ERASE 时，将执行块擦除命令。

代码 4-8-142 Flash 设备的驱动配置

```

01 static int SpiFlashControl(struct HardwareDev *dev,
                                struct HalDevBlockParam *block_param)
02 {
03     NULL_PARAM_CHECK(dev);
04     NULL_PARAM_CHECK(block_param);
05
06     x_err_t ret = EOK;
07     x_size_t size;
08     uint32_t addr;
09
10     sfud_flash *sfud_flash_dev;
11
12     SpiFlashDeviceType spi_flash_dev;
13     struct SpiHardwareDevice *flash_dev;
14
15     flash_dev = CONTAINER_OF(dev, struct SpiHardwareDevice, haldev);
16     spi_flash_dev = CONTAINER_OF(flash_dev, struct SpiFlashDevice, flash_dev);
17     sfud_flash_dev = (sfud_flash *)spi_flash_dev->flash_param.flash_private_data;
18
19     if (OPER_BLK_GETGEOME == block_param->cmd) {
20         block_param->dev_block.size_perbank =
21             spi_flash_dev->flash_param.flash_block_param.sector_bytes;
22         block_param->dev_block.block_size =
23             spi_flash_dev->flash_param.flash_block_param.block_size;
24         block_param->dev_block.bank_num =
25             spi_flash_dev->flash_param.flash_block_param.sector_num;
26     } else if (OPER_BLK_ERASE == block_param->cmd) {
27         NULL_PARAM_CHECK(block_param->dev_addr);
28         if (block_param->dev_addr->start > block_param->dev_addr->end) {
29             KPrintf("SpiFlashControl erase start %u > end %u addr\n",
30                 block_param->dev_addr->start, block_param->dev_addr->end);
31             return ERROR;
32         } else if (block_param->dev_addr->start == block_param->dev_addr->end) {
33             block_param->dev_addr->end++;
34         }
35     }
36 }

```

```

32         addr = block_param->dev_addr->start *
            spi_flash_dev->flash_param.flash_block_param.sector_bytes;
33         size = (block_param->dev_addr->end - block_param->dev_addr->start)*
            spi_flash_dev->flash_param.flash_block_param.sector_bytes;
34
35         ret = sfud_erase(sfud_flash_dev, addr, size);
36         if (SFUD_SUCCESS != ret) {
37             KPrintf("SpiFlashControl erase error %d
                        addr %p size %d\n", ret, addr, size);
38             return ERROR;
39         }
40     } else {
41         KPrintf("SpiFlashControl do not support the cmd 0x%x\n", block_param->cmd);
42     }
43
44     return ret;
45 }

```

8.2.2 connect_flash.c 文件

connect_flash.c 文件中实现了 QSPI 驱动及设备的注册，该流程主要包含 3 个步骤：

- （1）完成 QSPI 的驱动和设备到 SPI 总线的挂接（或称为绑定）；
- （2）实现 QSPI 设备的打开、关闭及读写方法，其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植；
- （3）实现 QSPI 设备的驱动配置，需结合 QSPI 特性及操作系统的接口定义进行相应的开发。

步骤（1）的 QSPI 驱动及设备绑定过程与 SPI 是完全一致的。具体可见代码 4-8-4。代码行 5~10 使用 SpiBusInit 函数对 SPI 总线进行初始化。代码行 12~17 使用 SpiDriverInit 函数对驱动进行初始化。代码行 19~24 使用 SpiDriverAttachToBus 函数将 SPI 驱动绑定到总线。

代码 4-8-143 SPI 驱动绑定过程

```

01 static int BoardSpiBusInit(struct SpiBus *spi_bus, struct SpiDriver *spi_driver,
                        const char *bus_name, const char *drv_name)
02 {
03     x_err_t ret = EOK;
04

```

```

05     /*Init the spi bus */
06     ret = SpiBusInit(spi_bus, bus_name);
07     if (EOK != ret) {
08         KPrintf("Board_Spi_init SpiBusInit error %d\n", ret);
09         return ERROR;
10     }
11
12     /*Init the spi driver*/
13     ret = SpiDriverInit(spi_driver, drv_name);
14     if (EOK != ret) {
15         KPrintf("Board_Spi_init SpiDriverInit error %d\n", ret);
16         return ERROR;
17     }
18
19     /*Attach the spi driver to the spi bus*/
20     ret = SpiDriverAttachToBus(drv_name, bus_name);
21     if (EOK != ret) {
22         KPrintf("Board_Spi_init SpiDriverAttachToBus error %d\n", ret);
23         return ERROR;
24     }
25
26     return ret;
27 }

```

BoardSpiDevBend 函数主要完成 SPI 设备的绑定，代码行 5~16 主要完成参数的声明及初始化；代码行 18~22 通过 SpiDeviceRegister 函数注册 SPI 设备。代码行 24~28 通过 SpiDeviceAttachToBus 将 SPI 设备挂接到总线。

代码 4-8-144 SPI 设备绑定过程

```

01 static int BoardQSpiDevBend(void)
02 {
03     x_err_t ret = EOK;
04
05     static struct SpiHardwareDevice qspi_device0;
06     memset(&qspi_device0, 0, sizeof(struct SpiHardwareDevice));
07
08     static struct SpiSlaveParam qspi_slaveparam0;
09     memset(&qspi_slaveparam0, 0, sizeof(struct SpiSlaveParam));
10
11     qspi_slaveparam0.spi_slave_id = QSPI_DEVICE_SLAVE_ID_0;

```

```

12     qspi_slaveparam0.spi_cs_gpio_pin = QSPI_CS_PIN;
13     qspi_slaveparam0.spi_cs_gpio_port = QSPI_CS_PORT;
14
15     qspi_device0.spi_param.spi_slave_param = &qspi_slaveparam0;
16     qspi_device0.spi_dev_done = &(qspi_dev_done);
17
18     ret = SpiDeviceRegister(&qspi_device0,
                            (void *)(&qspi_device0.spi_param), QSPI_DEVICE_NAME_0);
19     if (EOK != ret) {
20         KPrintf("BoardSpiDevBend SpiDeviceRegister
                            device %s error %d\n", QSPI_DEVICE_NAME_0, ret);
21         return ERROR;
22     }
23
24     ret = SpiDeviceAttachToBus(QSPI_DEVICE_NAME_0, QSPI_BUS_NAME);
25     if (EOK != ret) {
26         KPrintf("BoardSpiDevBend SpiDeviceAttachToBus
                            device %s error %d\n", QSPI_DEVICE_NAME_0, ret);
27         return ERROR;
28     }
29
30     return ret;
31 }

```

步骤（2）的实现过程如代码 4-8-6 所示，该流程是一个实现 QSPI 设备的打开、关闭及读写过程。根据结构体 QSpiDevDone 的定义，打开和关闭的值均为 NONE，表示该功能未启用。而读和写的值分别是 QSpiWriteData 和 QSpiReadData，表明这两个功能是打开的。

代码 4-8-145 SPI 设备的打开、关闭及读写

```

01 static const struct SpiDevDone qspi_dev_done =
02 {
03     .dev_open = NONE,
04     .dev_close = NONE,
05     .dev_write = QSpiWriteData,
06     .dev_read = QSpiReadData,
07 };

```

SPI 设备的驱动配置过程如代码 4-8-7 所示。在 SpiDrvConfigure 函数中，代码行 3~4 进行输入参数 drv 和 configure_info 的检查。代码行 6~8 声明了 3 个变量,分别是返回结果、SP

I 驱动结构体和配置参数变量。代码行 10~21 根据配置命令进行选择，若配置命令为 OPE_INT，则根据 QSpiSdkInit 和 QspiPinConfig 函数执行 QSPI 驱动重新初始化。

代码 4-8-146 QSPI 设备的驱动配置

```

01 static uint32 QSpiDrvConfigure(void *drv, struct BusConfigureInfo *configure_info)
02 {
03     NULL_PARAM_CHECK(drv);
04     NULL_PARAM_CHECK(configure_info);
05
06     x_err_t ret = EOK;
07     struct SpiDriver *spi_drv = (struct SpiDriver *)drv;
08     struct SpiMasterParam *spi_param;
09
10     switch (configure_info->configure_cmd)
11     {
12         case OPE_INT:
13             QSpiSdkInit(spi_drv);
14             QspiPinConfig();
15             break;
16         case OPE_CFG:
17             break;
18         default:
19             break;
20     }
21
22     return ret;
23 }

```

FLASH 设备的注册可见代码 4-8-8。代码行 3 调用 HwQSpiInit 首先完成 QSPI 设备驱动的注册。代码行 4~5 依次完成了 QSPI 的硬件初始化。代码行 6~8 完成了 FLASH 设备的注册，并将 QSPI 驱动绑定在 FLASH 驱动上。

代码 4-8-147 FLASH 设备的注册可见代码

```

01 int FlashW25qxxSpiDeviceInit(void)
02 {
03     HwQSpiInit();
04     QSpiSdkInit(NULL);
05     QspiPinConfig();
06     if (NONE == SpiFlashInit(QSPI_BUS_NAME, QSPI_DEVICE_NAME_0,

```

```

                                QSPI_DRV_NAME, QSPI_FLASH_DEV_NAME)) {
07         return ERROR;
08     }
09     return EOK;
10 }

```

8.3 QSPI flash 设备应用

flash 设备应用的测试通过测试依次读取和写入 flash 对应地址的存储空间来实现。代码 4-8-9 是对应的具体实现。代码行 1~3 对相关的头文件进行了包含；代码行 7~11 完成了对 flash 设备的打开；代码行 12~20 对读写操作所需要的结构体变量进行了声明和初始化；代码行 22~28 读取了 flash 地址 0x000000 处的 8 个字节长度的内存值，并通过 printf 函数进行输出显示，再对读入的缓存字节逐个加 1；代码行 30 将加 1 后的值重新写入至 flash 地址 0x000000 中。由于 flash 的硬件特性，该函数在开发板断电再重新上电后重新执行，读取到的对应地址的字节值后者应是前者值加 1

代码 4-8-148 flash 设备应用

```

01 #include <stdio.h>
02 #include <string.h>
03 #include <transform.h>
04
05 void TestFlash(void)
06 {
07     int fd = open(FLASH_DEV_DRIVER,O_RDWR);
08     if(fd<0){
09         printf("fs fd open error:%d\n",fd);
10         return;
11     }
12     struct BusBlockWriteParam flash_writer;
13     uint8_t read_buff[8] = {1,2,3,4,5,6,7,8};
14     flash_writer.pos = 0x000000L;
15     flash_writer.size = 8;
16     flash_writer.buffer = read_buff;
17     struct BusBlockReadParam flash_reader;
18     flash_reader.pos = 0x000000L;
19     flash_reader.size = 8;
20     flash_reader.buffer = read_buff;
21
22     PrivRead(fd,&flash_reader,NONE);

```

```
23     printf("Read data is:");
24     for(int i=0;i<flash_writer.size;i++){
25         printf("%02x ",read_buff[i]);
26         read_buff[i]++;
27     }
28     printf("\n");
29
30     PrivWrite(fd,&flash_writer,NONE);
31
32     return;
33 }
```

第9章 ADC 驱动开发

9.1 ADC 介绍

9.1.1 ADC 基本介绍

ADC是模拟信号转数字信号的模块，在嵌入式系统中被广泛应用。它可以将模拟量信号（如温度、光照强度、电压等）转换为数字信号，以便微处理器进行数据处理和分析。在嵌入式系统中，ADC通常是通过硬件实现的，主控芯片会集成ADC硬件模块，通过配置寄存器来设置采样率、参考电压等参数。同时，主控芯片也可以通过中断或DMA等方式来进行ADC的数据传输。ADC的转换精度通常由分辨率来表示，分辨率越高，转换精度越高。例如，一个12位的ADC可以将模拟信号转换为4096个数字量级，而一个16位的ADC可以将模拟信号转换为65536个数字量级。

12 位 ADC 是一种采用逐次逼近方式的模拟数字转换器。FC32F4A0 搭载 3 个 ADC 单元，单 1 和 2 支持 16 个通道，单元 3 支持 20 个通道，可以转换来自外部引脚、以及芯片内部的模拟信号。模拟输入通道可以任意组合成一个序列，一个序列可以进行单次扫描转换，或连续扫描转换。支持对任意指定通道进行连续多次转换并对转换结果进行平均。ADC 模块还搭载模拟看门狗功能，对任意指定通道的转换结果进行监视，检测其是否超出用户设定的范围。

9.1.2 中断说明

ADC 模块可以产生以下四种事件输出，每个事件发生时，若对应的中断许可寄存器设置为有效时，同时输出中断申请。

1. 序列 A 扫描结束 ADC_EOCA, 对应中断许可寄存器 ADC_ICR.EOCAIEN
2. 序列 B 扫描结束 ADC_EOCB, 对应中断许可寄存器 ADC_ICR.EOCBIEN
3. 模拟看门狗 0 ADC_CMP0, 对应中断许可寄存器 ADC_AWDCR.AWD0IEN
4. 模拟看门狗 1 ADC_CMP1, 对应中断许可寄存器 ADC_AWDCR.AWD1IEN

上述四种事件输出, 可以启动其他片内周边模块, 包括启动 DMA 传送。利用 DMA 传送可以连续读取 A/D 转换结果, 不需要软件干预, 完全由硬件实现, 降低 CPU 的负荷。事件信号输出和中断使能位的控制无关, 只要条件发生就会输出。

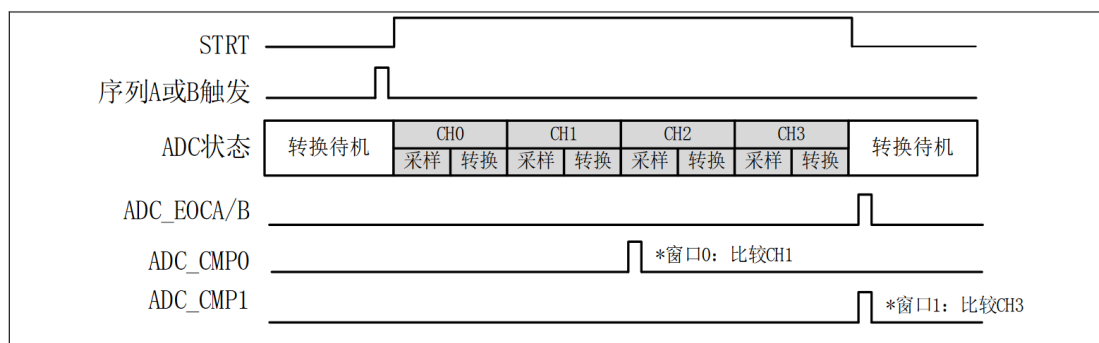


图 4-9-180 ADC 中断和事件输出时序

9.1.3 寄存器说明

一. A/D 启动寄存器

b7	b6	b5	b4	b3	b2	b1	b0
—	—	—	—	—	—	—	STRT

- B7~b1: 保留位, 读的时候读出 0, 写的时候写入 0
- b0: 0 表示停止转换, 1 表示开始转换

二. A/D 控制寄存器 0

b15	b14	b13	b12	b11	b10	b9	b8
—	—	—	—	—	AVCNT[2:0]		
b7	b6	b5	b4	b3	b2	b1	b0
DFMT	CLREN	ACCSEL[1:0]		—	—	MS[1:0]	

- b15~b12: 保留位, 读的时候读出 0, 写的时候写入 0
- b10~b8: 次数选择
- b7: 0 表示转换数据右对齐, 1 表示转换数据左对齐
- b6: 数据寄存器自动清除标志, 0 表示自动清除禁止, 1 表示自动清除许可
- b5: 分辨率选择
- b3~b2: 保留位, 读的时候读出 0, 写的时候写入 0

- b1~b0: 模式选择, 00 表示序列 A 单次扫描模式, 序列 B 无效, 01 表示序列 A 连续扫描模式, 序列 B 无效, 10 表示序列 A 单次扫描模式, 序列 B 单次扫描模式, 11 表示序列 A 连续扫描模式, 序列 B 单次扫描模式

三. A/D 控制寄存器 1

b15	b14	b13	b12	b11	b10	b9	b8
—	—	—	—	—	—	—	—
b7	b6	b5	b4	b3	b2	b1	b0
—	—	—	—	—	RSCHSEL	—	—

- b15~b3: 保留位, 读的时候读出 0, 写的时候写入 0
- b2: 0 表示被序列 B 中断后, 序列 A 重启时从被中断通道开始连续扫描
- b1~b0: 保留位, 读的时候读出 0, 写的时候写入 0

四. A/D 转换开始触发寄存器 ADC_TRGSR

b15	b14	b13	b12	b11	b10	b9	b8
TRGENB	—	—	—	—	—	TRGSELB[1]	TRGSELB[0]
b7	b6	b5	b4	b3	b2	b1	b0
TRGENA	—	—	—	—	—	TRGSELA[1]	TRGSELA[0]

- b15: 序列 B 触发使能, 0 表示序列 B 片内或外部引脚触发禁止, 1 表示许可
- b14~b10: 保留位, 读的时候读出 0, 写的时候写入 0
- b9~b8: 序列 B 触发条件选择
- b7: 序列 A 触发使能
- b6~b2: 保留位, 读的时候读出 0, 写的时候写入 0
- b1~b0: 序列 A 触发条件选择

五. A/D 中断状态寄存器 ADC_ISR

b7	b6	b5	b4	b3	b2	b1	b0
—	—	—	SASTPDF	—	—	EOCBF	EOCAF

- b7~b5: 保留位, 读的时候读出 0, 写的时候写入 0
- b4: 序列 A 被打断标志, 此位仅可读
- b3~b2: 保留位, 读的时候读出 0, 写的时候写入 0
- b1: 序列 B 转换完成标志, 此位仅可读
- b0: 序列 A 转换完成标志, 此位仅可读

六. A/D 中断许可寄存器 ADC_ICR

b7	b6	b5	b4	b3	b2	b1	b0
—	—	—	—	—	—	EOCBIEN	EOCAIEN

- b7~b2: 保留位, 读的时候读出 0, 写的时候写入 0
- b1: 序列 B 转换完成中断使能
- b0: 序列 A 转换完成中断使能

9.2 ADC 驱动介绍

ADC 设备通过 ADC BUS 挂接到 BUS 总线上, 其中 ADC BUS 的结构体在 `bus_adc.h` 文件中进行了定义。每个设备总线都关联一个 `drv` 和一个 `dev`, 其中, `drv` 结构体也在 `bus_adc.h` 中进行了定义, `dev` 结构体在 `dev_adc.h` 中进行了定义。另外, ADC 驱动及设备的注册流程在 `connect_adc.c` 中进行了实现。接下来, 针对这 3 个 ADC 相关的具体结构和实现流程进行详细介绍。

9.2.1 bus_adc.h 文件

`bus_adc.h` 文件中声明了 ADC 总线结构体 `AdcBus` 和驱动结构体 `AdcDriver`。 `AdcBus` 的结构体定义如下:

```
struct AdcBus{
    struct Bus bus;
    void *private_data;
};
```

其中 `bus` 是通用管理总线; `private_data` 是传递参数的通用指针。

`AdcDriver` 的结构体定义如下:

```
struct AdcDriver{
    struct Driver driver;
    uint32 (*configure)(void *drv, struct BusConfigureInfo *configure_info);
    void *private_data;
};
```

其中 `driver` 表示通用管理驱动; `configure` 表示配置函数; `private_data` 是传递参数的通用指针。

另外, 在 `bus_adc.h` 文件中还定义了一些函数接口, 具体均在 `bus_adc.c` 文件中实现:

- `int AdcBusInit(struct AdcBus *adc_bus, const char *bus_name)`, 用于初始化 ADC 总线, 其中 `adc_bus` 表示初始化的 ADC 总线; `bus_name` 表示总线名称。通过调用 `BusRegister` 函数将 ADC 总线注册到 BUS 管理总线链表。

- `int AdcReleaseBus(struct AdcBus *adc_bus)`, 用于释放 ADC 总线, 即将 ADC 总线从 BUS 管理总线链表中移除, 其中 `adc_bus` 表示需要释放的 ADC 总线。一般该接口函数与 `AdcBusInit` 函数配对使用, 对 ADC 总线进行注册和释放。
- `int AdcDriverInit(struct AdcDriver *adc_driver, const char *driver_name)`, 用于初始化 ADC 驱动, 其中 `adc_driver` 表示需要初始化的 ADC 驱动; `driver_name` 表示驱动名称。该接口通过调用 `AdcDriverRegister` 函数将 ADC 驱动注册到 ADC 驱动总线。
- `int AdcDriverAttachToBus(const char *drv_name, const char *bus_name)`, 用于将 ADC 驱动挂接到 ADC BUS 总线, 其中 `drv_name` 表示驱动名称; `bus_name` 表示总线名称。
- `DriverType AdcDriverFind(const char *drv_name, enum DriverType_e drv_type)`, 用于查找挂接的 ADC 驱动, 其中 `drv_name` 表示驱动名称; `drv_type` 表示驱动类型。

9.2.2 dev_adc.h 文件

在 `dev_adc.h` 文件中声明了 ADC 设备结构体 `AdcHardwareDevice`, 其具体的定义如下:

```
struct AdcHardwareDevice{
    struct HardwareDev haldev; ///< describe a device struct
    const struct AdcDevDone *adc_dev_done;
    void *private_data;
};
```

其中 `haldev` 用于描述一个设备结构; `adc_dev_done` 用于表示一个 ADC 设备的函数操作集; `private_data` 是传递参数的通用指针。

结构体 `AdcDevDone` 的定义如下:

```
struct AdcDevDone{
    uint32 (*open) (void *dev);
    uint32 (*close) (void *dev);
    uint32 (*write) (void *dev, struct BusBlockWriteParam *write_param);
    uint32 (*read) (void *dev, struct BusBlockReadParam *read_param);
};
```

可以看出, 该结构体是用于描述 ADC 设备的打开、关闭、读写功能。

另外, 在 `dev_adc.h` 文件中还定义了一些函数接口:

- `int AdcDeviceRegister(struct AdcHardwareDevice *adc_device, void *adc_param, const char *device_name)`, 用于初始化并注册 ADC 设备到设备管理链表上, 其中 `adc_device` 表示需要注册的 ADC 设备; `adc_param` 表示 ADC 参数; `device_name` 表示设备名称。

- `int AdcDeviceAttachToBus(const char *dev_name, const char *bus_name)`, 用于将 ADC 设备挂接到 ADC BUS 总线, 其中 `dev_name` 表示设备名称; `bus_name` 表示总线名称。
- `HardwareDevType AdcDeviceFind(const char *dev_name, enum DevType dev_type)`, 用于查找挂接的 ADC 设备, 其中 `dev_name` 表示设备名称; `dev_type` 表示设备类型。

9.2.3 connect_adc.c 文件

`connect_adc.c` 文件中实现了 ADC 驱动及设备的注册, 该流程主要包含 3 个步骤:

- (1) 完成 ADC 的驱动和设备到 ADC 总线的挂接 (或称为绑定);
- (2) 实现 ADC 设备的打开、关闭及读写方法, 其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植;
- (3) 实现 ADC 设备的驱动配置, 需结合 ADC 特性及操作系统的接口定义进行相应的开发。

步骤 (1) 的实现过程如代码 4-9-1 所示, 该流程是完成一个 ADC 驱动及设备总线绑定的过程。代码行 2~5 定义 4 个结构体变量, 分别用于 ADC 总线初始化、驱动初始化、设备注册和 ADC 配置。代码行 7~11 使用 `AdcBusInit` 函数对总线进行初始化。代码行 13~18 使用 `AdcDriverInit` 函数对驱动进行初始化。代码行 20~24 使用 `AdcDriverAttachToBus` 函数将 ADC 驱动绑定到总线。代码行 26~33 通过 `AdcDeviceRegister` 函数注册 ADC 设备。代码行 35~39 通过 `AdcDeviceAttachToBus` 将 ADC 设备挂接到总线。

代码 4-9-149 ADC 驱动及设备绑定

```

01  #ifndef BSP_USING_ADC1
02      static struct AdcBus adc1_bus;
03      static struct AdcDriver adc1_drv;
04      static struct AdcHardwareDevice adc1_dev;
05      static struct HwAdc adc1_cfg;
06
07      ret = AdcBusInit(&adc1_bus, ADC1_BUS_NAME);
08      if (ret != EOK) {
09          KPrintf("ADC1 bus init error %d\n", ret);
10          return ERROR;
11      }
12  
```

```

13     adc1_drv.configure = AdcDrvConfigure;
14     ret = AdcDriverInit(&adc1_drv, ADC1_DRIVER_NAME);
15     if (ret != EOK) {
16         KPrintf("ADC1 driver init error %d\n", ret);
17         return ERROR;
18     }
19
20     ret = AdcDriverAttachToBus(ADC1_DRIVER_NAME, ADC1_BUS_NAME);
21     if (ret != EOK) {
22         KPrintf("ADC1 driver attach error %d\n", ret);
23         return ERROR;
24     }
25
26     adc1_dev.adc_dev_done = &dev_done;
27     adc1_cfg.ADCx = ADC1;
28     adc1_cfg.adc_channel = 0;
29     ret = AdcDeviceRegister(&adc1_dev, (void *)&adc1_cfg, ADC1_DEVICE_NAME);
30     if (ret != EOK) {
31         KPrintf("ADC1 device register error %d\n", ret);
32         return ERROR;
33     }
34
35     ret = AdcDeviceAttachToBus(ADC1_DEVICE_NAME, ADC1_BUS_NAME);
36     if (ret != EOK) {
37         KPrintf("ADC1 device register error %d\n", ret);
38         return ERROR;
39     }
40 #endif

```

步骤（2）的实现过程如代码 4-9-2 所示，该流程是一个实现 ADC 设备的打开、关闭及读写过程。其中，AdcOpen、AdcClose、AdcRead 分别表示 ADC 的打开、关闭和读。根据结构体 AdcDevDone 的定义，此时写操作为 NONE。

代码 4-9-150 ADC 设备的打开、关闭及读写

```

01 static const struct AdcDevDone dev_done =
02 {
03     AdcOpen,
04     AdcClose,
05     NONE,
06     AdcRead,

```

07 };

ADC 设备的驱动配置过程如代码 4-9-3 所示。在 `AdcDrvConfigure` 函数中，代码行 2~3 进行输入参数 `drv` 和 `configure_info` 的检查。代码行 4~5 声明了 2 个变量。代码行 7~9 定义了 3 个结构体指针变量，其中，`adc_drv` 为 ADC 驱动指针变量，初始化为输入的 `drv`；`adc_dev` 为 ADC 设备结构体指针变量，通过强制转换对其进行初始化；`adc_cfg` 为实际硬件设备的结构体指针，通过强制转换使用 `adc_dev` 中的参数对其进行初始化。从此过程可以看出，代码行 7~9 实质上就是一层层地获取输入变量 `drv` 中的 `private_data` 参数。代码行 11~22 实现了具体的配置过程，其中，代码行 11 中 `switch` 语句的条件是获取输入参数 `configure_info` 的配置命令 `configure_cmd`，该命令有两种接口 `OPE_INT` 和 `OPE_CFG`；代码行 13 对 `adc_cfg` 变量的通道值进行赋值。

代码 4-9-151 ADC 设备的驱动配置

```

01 static uint32 AdcDrvConfigure(void *drv,
    struct BusConfigureInfo *configure_info){
02     NULL_PARAM_CHECK(drv);
03     NULL_PARAM_CHECK(configure_info);
04     x_err_t ret = EOK;
05     uint8 adc_channel;
06
07     struct AdcDriver *adc_drv = (struct AdcDriver *)drv;
08     struct AdcHardwareDevice *adc_dev =
        (struct AdcHardwareDevice *)adc_drv->driver.owner_bus->owner_haldev;
09     struct HwAdc *adc_cfg =
        (struct HwAdc *)adc_dev->haldev.private_data;
10
11     switch (configure_info->configure_cmd){
12         case OPE_CFG:
13             adc_cfg->adc_channel = *(uint8 *)configure_info->private_data;
14             if (adc_cfg->adc_channel > 18) {
15                 KPrintf("AdcDrvConfigure set adc channel(0-18) %u error!",
                    adc_cfg->adc_channel);
16                 adc_cfg->adc_channel = 0;
17                 ret = ERROR;
18             }
19             break;
20         default:

```

```

21         break;
22     }
23     return ret;
24 }

```

9.3 ADC 设备应用

ADC 设备应用实现过程如代码 4-9-4 所示。代码行 1 需要导入一个头文件，其中含有一系列的 read, close, ioctl 等函数。在测试函数 test_adc 中，代码行 4~7 声明一些变量。代码行 9~13 通过 open 函数打开一个 ADC 设备，如果打开不成功，则直接 return 返回。代码行 15~19 通过 ioctl 函数（即 IO 控制函数）对打开的 ADC 设备的通道进行配置，若配置不成功，则关闭 ADC 并返回。代码行 21 通过 read 函数读取 ADC 数据，此时读取 2 个字节的数据到 adc_sample。代码行 22~23 分别用于关闭 ADC 和函数返回。

代码 4-9-152 ADC 设备应用

```

01 #include <iot-vfs_posix.h>
02
03 void test_adc(){
04     int adc_fd;
05     uint8 adc_channel = 0x0;
06     uint16 adc_sample, adc_value_decimal = 0;
07     float adc_value;
08
09     adc_fd = open("/dev/adc1_dev", O_RDWR);
10     if (adc_fd < 0) {
11         printf("open adc fd error %d\n", adc_fd);
12         return;
13     }
14
15     if (0 != ioctl(adc_fd, OPE_CFG, &adc_channel)) {
16         printf("ioctl adc fd error %d\n", adc_fd);
17         close(adc_fd);
18         return;
19     }
20
21     read(adc_fd, &adc_sample, 2);
22     close(adc_fd);
23     return;
24 }

```

第10章 DAC 驱动

10.1 DAC 介绍

10.1.1 DAC 基本介绍

DAC是数字信号转模拟信号的模块，在嵌入式系统中被广泛应用。它可以将数字信号（如音频、视频、控制信号等）转换为模拟信号，以便驱动模拟电路和外设。在嵌入式系统中，DAC通常是通过硬件实现的，主控芯片会集成DAC硬件模块，通过配置寄存器来设置输出电压范围、分辨率等参数。同时，主控芯片也可以通过中断或DMA等方式来进行DAC的数据传输。

HC32F4A0 搭载了两个 12 位的数字-模拟电压转换器单元 DAC1 和 DAC2。每个 DAC 单元包含两个 D/A 转换通道，可以独立转换也可以通过转换数据的同步更新实现同步转换。每个转换通道配有一个输出放大器，可以在没有外部运放时直接驱动外部负载。独立管脚输入参考电压 VREFH 和 VREFL 可用来提高转换精度。

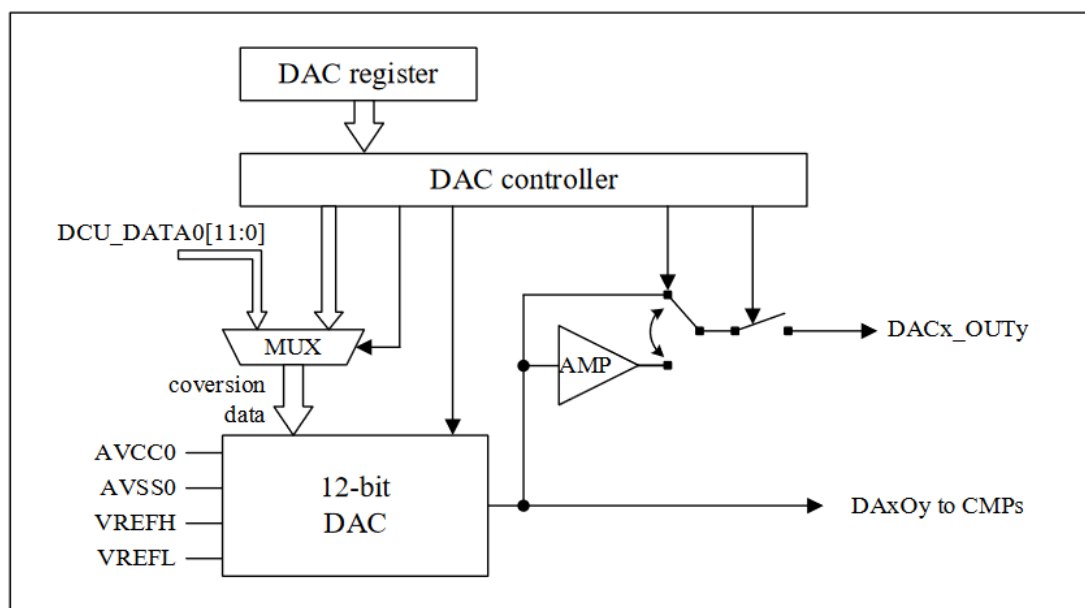


图 4-10-181 D/A 转换通道框图

10.1.2 中断说明

DAC 通常是同步操作，即 CPU 发送数字信号给 DAC，DAC 将信号转换为模拟信号输出，这个过程通常是有固定的时间延迟的，因此 CPU 可以根据 DAC 的转换速率和所需的精度来计算发送数字信号的时间间隔，而不需要中断告知 CPU 何时发送下一个数字信号。

10.1.3 DAC 寄存器说明

一. DAC 数据寄存器 (DADR_x_y x,y=1,2)

当 DACR.DPSEL=0 (数据右对齐) 时:

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	-	DADR[11:0]											

- b15~b12: 保留位, 读的时候读出 0, 写的时候写入 0
- b11~b0: 变换数据设定值

DACR.DPSEL=1 (数据左对齐) 时:

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
DADR[11:0]												-	-	-	-

- b15~b14: 变换数据设定值
- b11~b0: 保留位, 读的时候读出 0, 写的时候写入 0

二. DAC 控制寄存器 (DACR_x, x=1,2)

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	EXT DSL2	EXT SL1	DAA MP2	DAA MP1	DP SEL	-	-	-	-	-	DA2 E	DA1 E	DAE

- b15~b14: 保留位, 读的时候读出 0, 写的时候写入 0
- b12: 0 表示选择 DADR 作为通道 2 的转换数据, 1 表示选择外部数据作为通道 2 的转换数据
- b11: 0 表示选择 DADR 作为通道 1 的转换数据, 1 表示选择外部数据作为通道 1 的转换数据
- b10: 0 表示禁止通道 2 输出放大器, 1 表示启动通道 2 输出放大器
- b9: 0 表示禁止通道 1 输出放大器, 1 表示启动通道 1 输出放大器
- b8: 0 表示右对齐格式, 1 表示左对齐格式
- b7~b3: 保留位, 读的时候读出 0, 写的时候写入 0
- b2: 0 表示通道 2 禁止, 1 表示通道 2 使能
- b1: 0 表示通道 2 禁止, 1 表示通道 2 使能
- b0: 0 表示到 1, 2 禁止, 1 表示通道 1, 2 使能

三. DAC 模拟输出控制寄存器 (DAOCR_x, x=1,2)

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
DAO DIS2	DAO DIS1	-	-	-	-	-	-	-	-	-	-	-	-	-	-

- b15: 1 表示禁止 DACx_OUT2 端口输出模拟电压, 0 表示允许 DACx_OUT2 端口输出模拟电压
- b14: 1 表示禁止 DACx_OUT1 端口输出模拟电压, 0 表示允许 DACx_OUT1 端口输出模拟电压
- b13~b0: 保留位, 读的时候读出 0, 写的时候写入 0

四. A/D 转换优先控制寄存器 (DAADPCR_x, x=1,2)

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
ADP EN	-	-	-	-	-	DA2 SF	DA1 SF	-	-	-	-	-	ADC SL3	ADC SL2	ADC SL1

- b15: 1A/D 转换优先模式无效, 1 表示转换优先模式有效
- b14~b10: 保留位, 读的时候读出 0, 写的时候写入 0
- b9: 0 表示通道 2 数据更新完, 1 表示通道 2 数据更新中
- b8: 0 表示通道 1 数据更新完, 1 表示通道 1 数据更新中
- b7~b3: 保留位, 读的时候读出 0, 写的时候写入 0
- b2: 0 表示不选择 ADC_3 作为 A/D 转换优先对象, 1 表示选择 ADC_3 作为 A/D 转换优先对象
- b1: 0 表示不选择 ADC_2 作为 A/D 转换优先对象, 1 表示选择 ADC_2 作为 A/D 转换优先对象
- b0: 0 表示不选择 ADC_1 作为 A/D 转换优先对象, 1 表示选择 ADC_1 作为 A/D 转换优先对象

10.2 DAC 驱动介绍

DAC 设备通过 DAC BUS 挂接到 BUS 总线上, 其中 DAC BUS 的结构体在 `bus_dac.h` 文件中进行了定义。每个设备总线都关联一个 `drv` 和一个 `dev`, 其中, `drv` 结构体也在 `bus_dac.h` 中进行了定义, `dev` 结构体在 `dev_dac.h` 中进行了定义。另外, DAC 驱动及设备的注册流程在 `connect_dac.c` 中进行了实现。接下来, 针对这 3 个 DAC 相关的具体结构和实现流程进行详细介绍。

10.2.1 bus_dac.h 文件

bus_dac.h 文件中声明了 DAC 总线结构体 DacBus 和驱动结构体 DacDriver。DacBus 的结构体定义如下：

```
struct DacBus{
    struct Bus bus;
    void *private_data;
};
```

其中 bus 是通用管理总线；private_data 是传递参数的通用指针。

DacDriver 的结构体定义如下：

```
struct DacDriver{
    struct Driver driver;
    uint32 (*configure)(void *drv, struct BusConfigureInfo *configure_info);
    void *private_data;
};
```

其中 driver 表示通用管理驱动；configure 表示配置函数；private_data 是传递参数的通用指针。

另外，在 bus_dac.h 文件中还定义了一些函数接口，具体均在 bus_dac.c 文件中实现：

- int DacBusInit(struct DacBus *dac_bus, const char *bus_name)，用于初始化 DAC 总线，其中 dac_bus 表示初始化的 DAC 总线；bus_name 表示总线名称。通过调用 BusRegister 函数将 DAC 总线注册到 BUS 管理总线链表。
- int DacReleaseBus(struct DacBus *dac_bus)，用于释放 DAC 总线，即将 DAC 总线从 BUS 管理总线链表中移除，其中 dac_bus 表示需要释放的 DAC 总线。一般该接口函数与 DacBusInit 函数配对使用，对 DAC 总线进行注册和释放。
- int DacDriverInit(struct DacDriver *dac_driver, const char *driver_name)，用于初始化 DAC 驱动，其中 dac_driver 表示需要初始化的 DAC 驱动；driver_name 表示驱动名称。该接口通过调用 DacDriverRegister 函数将 DAC 驱动注册到 DAC 驱动总线。
- int DacDriverAttachToBus(const char *drv_name, const char *bus_name)，用于将 DAC 驱动挂接到 DAC BUS 总线，其中 drv_name 表示驱动名称；bus_name 表示总线名称。
- DriverType DacDriverFind(const char *drv_name, enum DriverType_e drv_type)，用于查找挂接的 DAC 驱动，其中 drv_name 表示驱动名称；drv_type 表示驱动类型。

10.2.2 dev_dac.h 文件

在 dev_dac.h 文件中声明了 DAC 设备结构体 DacHardwareDevice，其具体的定义如下：

```
struct DacHardwareDevice{
    struct HardwareDev haldev;
    const struct DacDevDone *dac_dev_done;
    void *private_data;
};
```

其中 haldev 用于描述一个设备结构；dac_dev_done 用于表示一个 DAC 设备的函数操作集；private_data 是传递参数的通用指针。

结构体 DacDevDone 的定义如下：

```
struct DacDevDone{
    uint32 (*open) (void *dev);
    uint32 (*close) (void *dev);
    uint32 (*write) (void *dev, struct BusBlockWriteParam *write_param);
    uint32 (*read) (void *dev, struct BusBlockReadParam *read_param);
};
```

可以看出，该结构体是用于描述 DAC 设备的打开、关闭、读写功能。

另外，在 dev_dac.h 文件中还定义了一些函数接口：

- int DacDeviceRegister(struct DacHardwareDevice *dac_device, void *dac_param, const char *device_name)，用于初始化并注册 DAC 设备到设备管理链表上，其中 dac_device 表示需要注册的 DAC 设备；dac_param 表示 DAC 参数；device_name 表示设备名称。
- int DacDeviceAttachToBus(const char *dev_name, const char *bus_name)，用于将 DAC 设备挂接到 DAC BUS 总线，其中 dev_name 表示设备名称；bus_name 表示总线名称。
- HardwareDevType DacDeviceFind(const char *dev_name, enum DevType dev_type)，用于查找挂接的 DAC 设备，其中 dev_name 表示设备名称；dev_type 表示设备类型。

10.2.3 connect_dac.c 文件

connect_dac.c 文件中实现了 DAC 驱动及设备的注册，该流程主要包含 3 个步骤：

- （1）完成 DAC 的驱动和设备到 DAC 总线的挂接；
- （2）实现 DAC 设备的打开、关闭及读写方法，其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植；

(3) 实现 DAC 设备的驱动配置。

步骤 (1) 的实现过程如代码 4-10-1 所示, 该流程是完成一个 DAC 驱动及设备绑定的过程。代码行 2 定义了一共结果返回变量。代码行 5~8 定义 4 个结构体变量, 分别用于 DAC 总线初始化、驱动初始化、设备注册和 DAC 配置。代码行 12 调用 DacDrvConfigure 函数对 DAC 驱动进行配置。代码行 12~16 使用 DacBusInit 函数对总线进行初始化。代码行 18~22 使用 DacDriverInit 函数对驱动进行初始化。代码行 24~28 使用 DacDriverAttachToBus 函数将 DAC 驱动绑定到总线。代码行 30~37 通过 DacDeviceRegister 函数注册 DAC 设备。代码行 39~43 通过 DacDeviceAttachToBus 函数将 DAC 设备挂接到 DAC 总线。

代码 4-10-153 DAC 驱动及设备绑定

```

01  int HwDacInit(void){
02      x_err_t ret = EOK;
03
04  #ifdef BSP_USING_DAC
05      static struct DacBus dac_bus;
06      static struct DacDriver dac_drv;
07      static struct DacHardwareDevice dac_dev;
08      static struct HwDac dac_cfg;
09
10      dac_drv.configure = DacDrvConfigure;
11
12      ret = DacBusInit(&dac_bus, DAC_BUS_NAME);
13      if (ret != EOK) {
14          KPrintf("DAC bus init error %d\n", ret);
15          return ERROR;
16      }
17
18      ret = DacDriverInit(&dac_drv, DAC_DRIVER_NAME);
19      if (ret != EOK) {
20          KPrintf("DAC driver init error %d\n", ret);
21          return ERROR;
22      }
23
24      ret = DacDriverAttachToBus(DAC_DRIVER_NAME, DAC_BUS_NAME);
25      if (ret != EOK) {
26          KPrintf("DAC driver attach error %d\n", ret);
27          return ERROR;

```

```

28     }
29
30     dac_dev.dac_dev_done = &dev_done;
31     dac_cfg.DACx = CM_DAC1;
32     dac_cfg.digital_data = 0;
33     ret = DacDeviceRegister(&dac_dev, (void *)&dac_cfg, DAC_DEVICE_NAME);
34     if (ret != EOK) {
35         KPrintf("DAC device register error %d\n", ret);
36         return ERROR;
37     }
38
39     ret = DacDeviceAttachToBus(DAC_DEVICE_NAME, DAC_BUS_NAME);
40     if (ret != EOK) {
41         KPrintf("DAC device register error %d\n", ret);
42         return ERROR;
43     }
44 #endif
45
46     return ret;
47 }

```

步骤（2）的实现过程如代码 4-10-2 所示，该流程是一个实现 DAC 设备的打开、关闭及读写过程。其中，DacOpen、DacClose、DacRead 分别表示 DAC 的打开、关闭和读。根据结构体 DacDevDone 的定义，此时写操作为 NONE。

代码 4-10-154 DAC 设备的打开、关闭及读写

```

01 static const struct DacDevDone dev_done =
02 {
03     DacOpen,
04     DacClose,
05     NONE,
06     DacRead,
07 };

```

DAC 设备的驱动配置过程如代码 4-10-3 所示。在 DacDrvConfigure 函数中，代码行 2~3 进行输入参数 drv 和 configure_info 的检查。代码行 5 声明了 1 个结果返回变量。代码行 7~9 定义了 3 个结构体指针变量，其中，dac_drv 为 DAC 驱动指针变量，初始化为输入的 drv；dac_dev 为 DAC 设备结构体指针变量，通过强制转换对其进行初始化；dac_cfg 为实际

硬件设备的结构体指针，通过强制转换使用 `dac_dev` 中的参数对其初始化。从此过程可以看出，代码行 7~9 实质上就是一层层地获取输入变量 `drv` 中的 `private_data` 参数。

代码行 11~18 实现了具体的配置过程，其中，代码行 11 中 `switch` 语句的条件是获取输入参数 `configure_info` 的配置命令 `configure_cmd`，该命令有两种接口 `OPE_INT` 和 `OPE_CFG`；代码行 13 对 `dac_cfg` 变量的通道值进行赋值。

代码 4-10-155 DAC 设备的驱动配置

```

01 static uint32 DacDrvConfigure(void *drv,
    struct BusConfigureInfo *configure_info){
02     NULL_PARAM_CHECK(drv);
03     NULL_PARAM_CHECK(configure_info);
04
05     x_err_t ret = EOK;
06
07     struct DacDriver *dac_drv = (struct DacDriver *)drv;
08     struct DacHardwareDevice *dac_dev =
    (struct DacHardwareDevice *)dac_drv->driver.owner_bus->owner_haldev;
09     struct HwDac *dac_cfg =
    (struct Stm32HwDac *)dac_dev->haldev.private_data;
10
11     switch (configure_info->configure_cmd){
12         case OPE_CFG:
13             dac_cfg->digital_data = *(uint16 *)configure_info->private_data;
14             DAC_SetChData(dac_cfg->DACx,DAC_CH1,dac_cfg->digital_data);
15             break;
16         default:
17             break;
18     }
19
20     return ret;
21 }

```

10.3 DAC 设备应用

DAC 设备应用实现过程如代码 4-10-4 所示。代码行 1 需要导入一个头文件，其中含有一系列的 `read`，`close`，`ioctl` 等函数。在测试函数 `test_dac` 中，代码行 4~7 声明一些变量。代码行 9~13 通过 `open` 函数打开一个 DAC 设备，如果打开不成功，则直接 `return` 返回。代码行 15~22 声明了一个结构体变量 `ioctl_cfg`，对其进行初始化，并通过 `ioctl` 函数（即 IO 控制

函数)对打开的 DAC 设备的通道进行配置,若配置不成功,则关闭 DAC 并返回。代码行 24 通过 read 函数读取 DAC 数据,此时读取 2 个字节的数据到 dac_sample。代码行 25~26 分别用于关闭 DAC 和函数返回。

代码 4-10-156 DAC 设备应用

```

01  #include <iot-vfs_posix.h>
02
03  void test_dac(){
04      int dac_fd;
05      uint16 dac_set_value = 800;
06      uint16 dac_sample, dac_value_decimal = 0;
07      float dac_value;
08
09      dac_fd = open(DAC_DEV_DRIVER, O_RDWR);
10      if (dac_fd < 0) {
11          printf("open dac fd error %d\n", dac_fd);
12          return;
13      }
14
15      struct ioctl_cfg;
16      ioctl_cfg.ioctl_driver_type = DAC_TYPE;
17      ioctl_cfg.args = &dac_set_value;
18      if (0 != ioctl(dac_fd, OPE_CFG, &ioctl_cfg)) {
19          printf("ioctl dac fd error %d\n", dac_fd);
20          close(dac_fd);
21          return;
22      }
23
24      read(dac_fd, &dac_sample, 2);
25      close(dac_fd);
26      return;
27  }

```

第11章 USB 驱动

11.1 USB 介绍

11.1.1 USB 基本介绍

USB (Universal Serial Bus) 是一种通用的数字通信接口,已经成为了现代计算机和嵌

入式系统中最常用的接口之一。USB接口分为主机和从设备两种类型，主机可以连接多个从设备，从设备则只能连接到一个主机。

在嵌入式系统中，USB 通常是通过硬件实现的，主控芯片会集成 USB 控制器和 USB PHY（物理层）模块。USB 控制器负责协调数据传输和设备管理，而 USB PHY 则负责将数字信号转换为模拟信号或将模拟信号转换为数字信号。

USB 全速（USBFS）控制器为便携式设备提供了一套 USB 通信解决方案。USBFS 控制器支持主机模式和设备模式，且芯片内部集成全速 PHY。主机模式下，USBFS 控制器支持全速（FS，12Mb/s）和低速（LS，1.5Mb/s）收发器，而设备模式下则仅支持全速（FS，12 Mb/s）收发器。USBFS 控制器支持 USB 2.0 协议所定义的所有四种传输方式（控制传输、批量传输、中断传输和同步传输）。

USBFS 决定当前工作模式有以下两种方法：

方法 1：根据 USB_ID 线的状态自动检测，检测到 USBFS_ID 线为高电平时，模块工作在设备模式，检测到 USBFS_ID 为低电平时，模块工作在主机模式。

方法 2：强制主机/设备模式，通过设定寄存器 USB_GUSBCFG 的 FDMOD 或者 FHMOD 位为 1，使得模块忽略 USBFS_ID 线的电平而强制工作在设备或主机模式。

11.1.1.2 中断说明

USBFS 中断有以下两类，SOTP 模式唤醒中断 USBFS_WKUP 和 USBFS 全局中断 USBFS_GLB。USBFS_WKUP 中断用于 STOP 模式下，通过 USBFS_DP 或者 USBFS_DM 唤醒系统的 STOP 模式，该中断使能位为 INT_WUPEN.USER_WUEN。使用 USBFS 唤醒系统 STOP 前，需要确保 USBFS 控制器处于挂起状态，并设定寄存器 USBFS_SYCTLREG 中的对应的滤波范围和使能滤波功能。USBFS_GLB 中断是软件需要处理的主要中断，全局中断的标志位可在 USBFS_GINTSTS 寄存器读取。

主机通道 x 中断寄存器 (HCINTx) 指示端点在出现 USB 和 AHB 相关事件时的状态。当中断寄存器中的主机通道中断位(USBFS_GINTSTS 中的 HCINT 位)置 1 时，应用程序必须读取这些寄存器以获得详细信息。在读取这些寄存器之前，应用程序必须先 读取主机全体通道中断 (HCAINT) 寄存器，以获取主机通道 x 中断寄存器的通道编号。应用程序必须将此寄存器中的相应位清零，才能将 HCAINT 和 GINTSTS 寄存器中的相应位清

零。USBFS_HCINTMSK_x 寄存器还□供每个通道各中断源的屏蔽位。主机模块□供以下状态检查和中断产生功能：

- 传输完成中断，指示应用程序 (AHB) 和 USB 端均已完成数据传输
- 通道因传输完成、USB 事务错误或应用程序发出禁止命令而停止
- 相关的发送 FIFO 为半空或全空状态(IN 端点) 接收到 ACK 响应
- 接收到 NAK 响应
- 接收到 STALL 响应
- 由于 CRC 校验失败、超时、位填充错误和错误的 EOP 导致 USB 事务错误
- 串扰错误
- 帧上溢
- 用于数据同步的翻转位出错

11.1.3 寄存器说明

一、系统控制寄存器(USB_SYCTLREG)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	-	-	-	-	USBHS_NFE	USBHS_NFS[1:0]	-	-	-	-	-	-	USBFS_NFE	USBFS_NFS[1:0]	-
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	-	-	-	-	USBHS_FSPHYE	USBHS_SOFEN	USBHS_DFB	-	-	-	-	-	-	USBFS_SOFEN	USBFS_DFB

- b31~b27: 保留位，必须保持为复位值（即必须保持为全 0）
- b26: USBHS 滤波使能寄存器，0 表示模拟滤波器关，1 表示模拟滤波器开
- b25~b24: 该寄存器用于控制 STOP 模式下，USBHS 片上全速 PHY DP/DM 模拟滤波器的滤波范围，00 表示滤波宽度档位 1，以此类推，11 表示滤波宽度档位 4
- b23~b19: 保留位，必须保持为复位值（即必须保持为全 0）
- b18: USBFS 滤波使能，0 表示模拟滤波器关，1 表示开
- b17~b16: 该寄存器用于控制 STOP 模式下，USBFS 片上全速 PHY DP/DM 模拟滤波器的滤波范围，00 表示滤波宽度档位 1，以此类推，11 表示滤波宽度档位 4

- b15~b11: 保留位, 必须保持为复位值 (即必须保持为全 0)
- b10: USBHS 全速 PHY 使能位, 0 表示 USBHS 片上全速 PHY 挂起, 去表示 USBHS 片上全速 PHY 使能
- b9: USBHS SOF 脉冲输出使能位, 0 表示 SOF 脉冲不输出, 1 表示 SOF 脉冲输出
- b8: USBHS VBUS/ID 管脚模块内部去抖动滤波器旁路使能位
- b7~b2: 保留位, 必须保持为复位值 (即必须保持为全 0)
- b1: USBFS SOF 脉冲输出使能位, 0 表示 SOF 脉冲不输出, 1 表示 SOF 脉冲输出
- b0: USBFS VBUS/ID 管脚模块内部去抖动滤波器旁路使能位

二. 配置寄存器(USBFS_GUSBCFG)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved	FDMOD	FHMOD	Reserved												
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved		TRDT[3:0]				Reserved			PHYSEL	Reserved			TOCAL[2:0]		

- b31: 保留位, 必须保持为复位值 (即必须保持为全 0)
- b30: 强制设备模式, 0 表示正常模式, 1 表示强制设备模式
- b29: 强制主机模式, 0 表示正常模式, 1 表示强制主机模式
- b28~b14: 保留位, 必须保持为复位值 (即必须保持为全 0)
- b13~b10: USB 周转时间, 以 PHY 时钟数位单位设置周转时间
- b9~b7: 保留位, 必须保持为复位值 (即必须保持为全 0)
- b6: 全速串行收发器选择
- b5~b3: 保留位, 必须保持为复位值 (即必须保持为全 0)
- b2~b0: FS 超时校准

三. 全局中断寄存器

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
WKUINT	VBUSVINT	DISCINT	CIDSCHG	LPMINT	PTXFE	HCINT	HPRINT	Reserved	DATAFSUSP	IPXFR/ INCOMPISSOUT	IISOIXFR	OEPIINT	IEPIINT	Reserved	Reserved
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
EOPF	ISOODRP	ENUMDNE	USBRST	USBSUSP	ESUSP	Reserved	Reserved	GONAKEFF	GINAKEFF	NPTXFE	RXFNE	SOF	Reserved	MMIS	CMOD

- b31: 检测到恢复原称唤醒中断, 在设备模式下, 当 USB 总线上检测到恢复信号时, 将触发该中断, 在主机模式下, 当 USB 上检测到原称唤醒时, 将触发该中断, 方式为通过软件向该位写 1
- b30: 设备模式下, 当检测到 USBFS_VBUS 管脚由低到高时, 将触发该中断, 方式为通过软件向该位写 1
- b29: 检测到设备断开链接时触发该中断, 方式为通过软件向该位写 1
- b28: 当连接器 ID 线状态发生改变时, 模块将该位置 1
- b27: 设备收到正确的 LPM 传输且有效应答时该位置 1
- b26: 当周期性发送 FIFO 为半空或者全空状态, 且周期性请求队列中存在可写入至少一个条目的空间时, 将触发该中断
- b25: 模块将该位置 1 时, 指示模块中一个通道上存在花期的中断 (主机模式)
- b24: 模块将该位置 1 时, 指示主机模式下 USBFS 控制器端口的状态发生变化
- b23: 保留位, 必须保持为复位值 (即必须保持为全 0)
- b22: 数据获取挂起
- b21: 在主机模式下, 如果存在仍处于挂起状态的未完成周期性事务, 而这些事务在当前帧期完成, 则该模块会讲该中断置 1
- b20: 模块将该中断置 1 时, 指示当前帧中至少有一个同步 IN 端点上的出阿叔未完成
- b19: 模块将该位置 1 时, 指示模块中一个 OUT 端点上存在挂起的中断 (设备模式下)
- b18: 模块将该位置 1 时, 指示模块中一个 IN 端点上存在挂起的中断 (设备模式下)
- b17~b16: 保留位, 必须保持为复位值 (即必须保持为全 0)
- b15: 指示当前帧已达到 USBFS——DCFG 寄存器中周期性帧间隔字段 (USBFS_DCFG 中的 PFIVL 位) 所指定的周期

- b14: 如果由于 RxFIFO 空间不足, 无法容纳同步 OUT 端点的最大数据包, 从而导致模块无法向 RxFIFO 写入同步 OUT 数据包, 模块会将该位置 1
- b13: 模块将该位置 1 时, 指示速度枚举已完成。应用程序必须读去 USBFS_DSTS 寄存器来获取枚举速度
- b12: 模块将该位置 1 时, 指示在 USB 上检测到复位信号
- b11: 模块将该位置 1 时, 指示在 USB 上检测到挂起状态
- b10: 模块将该位置 1 时, 指示已检测到 USB 处于空闲状态的时间达到 3ms
- b9~b8: 保留位, 必须保持为复位值 (即必须保持为全 0)
- b7: 指示 USBFS_DCTL 寄存器中应由应用程序设置的“将全局 OUT NAK 置 1”位 (USBFS_DCTL 中的 SGONAK 位) 已在模块中生效
- b6: 指示 USBFS_DCTL 寄存器中应由应用程序设置的“将全局 IN NAK 置 1”位 (USBFS_DCTL 中的 SGONAK 位) 已在模块中生效
- b5: 当非周期性 TxFIFO 为半空或全空状态, 且非周期性发送请求队列中至少存在可写入一个条目的空间时, 将触发该中断
- b4: 指示 RxFIFO 中至少有一恶搞数据包等待读取
- b3: 在主机模式下, 模块将该位置 1 时, 指示 USB 上已经发送一一个 SOF(FS)或 Keep-Alive (LS) 信号, 应用程序必须将此位置 1 才可清除该中断
- b2: 保留位, 必须保持为复位值 (即必须保持为全 0)
- b1: 模式不匹配中断, 当应用程序尝试在设备模式下访问主机模式寄存器, 或者在主机模式下访问设备模式寄存器时, 模块将该位置 1
- b0: 指示当前工作模式, 0 表示设备模式, 1 表示主机模式

11.2 USB 驱动介绍

USB 设备通过 USB BUS 挂接到 BUS 总线上, 其中 USB BUS 的结构体在 bus_usb.h 文件中进行了定义。每个设备总线都关联一个 drv 和一个 dev, 其中, drv 结构体也在 bus_usb.h 中进行了定义, dev 结构体在 dev_usb.h 中进行了定义。另外, USB 驱动及设备的注册流程在 connect_usb.c 中进行了实现。接下来, 针对这 3 个 USB 相关的具体结构和实现流程进行详细介绍。

11.2.1 bus_usb.h 文件

bus_usb.h 文件中声明了 USB 总线结构体 UsbBus 和驱动结构体 UsbDriver。UsbBus 的结构体定义如下：

```
struct UsbBus{
    struct Bus bus;
    void *private_data;
};
```

其中 bus 是通用管理总线；private_data 是传递参数的通用指针。

UsbDriver 的结构体定义如下：

```
struct SdioDriver{
    struct Driver driver;
    uint32 (*configure)(void *drv, struct BusConfigureInfo *configure_info);
    void *private_data;
};
```

其中 driver 表示通用管理驱动；configure 表示配置函数。

另外，在 bus_sdio.h 文件中还定义了一些函数接口，具体均在 bus_sdio.c 文件中实现：

- int UsbBusInit(struct UsbBus *usb_bus, const char *bus_name), 用于初始化 USB 总线，其中 usb_bus 表示初始化的 USB 总线；bus_name 表示总线名称。通过调用 BusRegister 函数将 USB 总线注册到 BUS 管理总线链表。
- int UsbReleaseBus(struct UsbBus *usb_bus), 用于释放 USB 总线，即将 USB 总线从 BUS 管理总线链表中移除，其中 usb_bus 表示需要释放的 SDIO 总线。一般该接口函数与 UsbBusInit 函数配对使用，对 USB 总线进行注册和释放。
- int UsbDriverInit(struct UsbDriver *usb_driver, const char *driver_name), 用于初始化 USB 驱动，其中 usb_driver 表示需要初始化的 USB 驱动；driver_name 表示驱动名称。该接口通过调用 UsbDriverRegister 函数将 USB 驱动注册到 USB 驱动总线。
- int UsbDriverAttachToBus(const char *drv_name, const char *bus_name), 用于将 USB 驱动挂接到 USB BUS 总线，其中 drv_name 表示驱动名称；bus_name 表示总线名称。
- DriverType UsbDriverFind(const char *drv_name), 用于查找挂接的 USB 驱动，其中 drv_name 表示驱动名称。

11.2.2 dev_usb.h 文件

在 dev_usb.h 文件中声明了 USB 设备结构体 `UsbHardwareDevice`，其具体的定义如下：

```
struct UsbHardwareDevice{
    struct HardwareDev haldev;
    const struct UsbDevDone *dev_done;
    void *private_data;
};
```

其中 `haldev` 用于描述一个设备结构；`dev_done` 用于表示一个 USB 设备的函数操作集；`private_data` 是传递参数的通用指针。

结构体 `SdioDevDone` 的定义如下：

```
struct UsbDevDone{
    uint32 (*open) (void *dev);
    uint32 (*close) (void *dev);
    uint32 (*write) (void *dev, struct BusBlockWriteParam *write_param);
    uint32 (*read) (void *dev, struct BusBlockReadParam *read_param);
};
```

可以看出，该结构体是用于描述 SDIO 设备的打开、关闭、读写功能。

另外，在 dev_usb.h 文件中还定义了一些函数接口：

- `int USBDeviceRegister(struct UsbHardwareDevice *usb_device, void *usb_param , const char *device_name)`，用于初始化并注册 USB 设备到设备管理链表上，其中 `usb_device` 表示需要注册的 USB 设备；`usb_param` 表示 USB 参数；`device_name` 表示设备名称。
- `int USBDeviceAttachToBus(const char *dev_name, const char *bus_name)`，用于将 USB 设备挂接到 USB BUS 总线，其中 `dev_name` 表示设备名称；`bus_name` 表示总线名称。
- `HardwareDevType USBDeviceFind(const char *dev_name)`，用于查找挂接的 USB 设备，其中 `dev_name` 表示设备名称。

11.2.3 connect_usb.c 文件

`connect_usb.c` 文件中实现了 USB 驱动及设备的注册，该流程主要包含 3 个步骤：

- （1）完成 USB 的驱动和设备到 USB 总线的挂接（或称为绑定）；
- （2）实现 USB 设备的打开、关闭及读写方法，其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植；

(3) 实现 USB 设备的驱动配置，需结合 USB 特性及操作系统的接口定义进行相应的开发。

步骤 (1) 的实现过程如代码 4-11-1 所示，该流程是完成一个 USB 驱动及设备总线绑定的过程。代码行 5~8 定义了 2 个结构体变量。代码行 11~15 使用 BoardUsbBusInit 函数对 USB 总线进行初始化。代码行 17~21 使用 BoardUsbDevBend 函数对驱动进行初始化。

代码 4-11-157 USB 驱动及设备绑定

```

01 int HwUsbHostInit(void)
02 {
03     x_err_t ret = EOK;
04
05     static struct UsbBus usb_bus;
06     memset(&usb_bus, 0, sizeof(struct UsbBus));
07
08     static struct UsbDriver usb_driver;
09     memset(&usb_driver, 0, sizeof(struct UsbDriver));
10
11     ret = BoardUsbBusInit(&usb_bus, &usb_driver);
12     if (EOK != ret) {
13         KPrintf("BoardUsbBusInit error ret %u\n", ret);
14         return ERROR;
15     }
16
17     ret = BoardUsbDevBend();
18     if (EOK != ret) {
19         KPrintf("BoardUsbDevBend error ret %u\n", ret);
20         return ERROR;
21     }
22
23     return ret;
24 }

```

步骤 (2) 的实现过程如代码 4-11-2 所示，该流程是一个实现 USB 设备的打开、关闭及读写过程。根据结构体 UsbDevDone 的定义，打开、关闭、读和写的值分别是 UsbHostOpen、UsbHostClose、UsbHostWrite 和 UsbHostRead，表明所有功能是打开的。

代码 4-11-158 USB 设备的打开、关闭及读写

```

01 static const struct UsbDevDone dev_done =
02 {
03     .open = UsbHostOpen,
04     .close = UsbHostClose,
05     .write = UsbHostWrite,
06     .read = UsbHostRead,
07 };

```

USB 设备的驱动配置过程如代码 4-11-3 所示。代码行 05~09 中对结构体变量 block_param 的 dev_block 中的成员 size_perbank、block_size 和 bank_num 进行赋值。

代码 4-11-159 USB 设备的驱动配置

```

01 static int UsbControl(struct HardwareDev *dev, struct HalDevBlockParam *block_param)
02 {
03     NULL_PARAM_CHECK(dev);
04
05     if (OPER_BLK_GETGEOME == block_param->cmd) {
06         block_param->dev_block.size_perbank = USB_SINGLE_BLOCK_SIZE;
07         block_param->dev_block.block_size = USB_SINGLE_BLOCK_SIZE;
08         block_param->dev_block.bank_num =
09         USB_HOST_MSC_Param.MSC_Capacity;
10     }
11     return EOK;
12 }

```

11.3 USB 设备应用

USB 设备应用实现过程如代码 4-11-4 所示，主要通过验证 U 盘的打开、关闭和读写功能。代码行 2~6 打开一个文件。代码行 12~27 首先用 read 函数读文件内容，然后用 write 函数写文件，最后用 close 函数关闭文件。代码行 29~44 首先用 open 函数重新打开文件，然后再进行读和关闭文件。

代码 4-11-160 USB 设备应用

```

01 void TestFs(void){
02     int fd = open(FPATH,O_RDWR|O_CREAT);
03     if(fd<0){
04         printf("fs fd open error:%d\n",fd);

```

```
05         return;
06     }
07
08     char filewords[MAX_READ_LENGTH];
09     memset(filewords,0,MAX_READ_LENGTH);
10     const char *input_words = "these words are going to write in fs\n";
11
12     int err_flag = read(fd,filewords,MAX_READ_LENGTH);
13     if(err_flag<0){
14         printf("read failed,error:%d\n",err_flag);
15         return;
16     }
17     printf("read data is \n%s\n",filewords);
18     err_flag = write(fd,input_words,strlen(input_words));
19     if(err_flag<0){
20         printf("write failed,error:%d\n",err_flag);
21         return;
22     }
23     err_flag = close(fd);
24     if(err_flag<0){
25         printf("close failed,error %d\n",err_flag);
26         return ;
27     }
28
29     fd = open(FPATH,O_RDWR);
30     if(fd<0){
31         printf("fs fd open error:%d\n",fd);
32         return;
33     }
34     err_flag = read(fd,filewords,MAX_READ_LENGTH);
35     if(err_flag<0){
36         printf("read failed,error:%d\n",err_flag);
37         return;
38     }
39     printf("read data is \n%s\n",filewords);
40     err_flag = close(fd);
41     if(err_flag<0){
42         printf("close failed,error:%d\n",err_flag);
43         return;
44     }
45
```

```
46     return;  
47 }
```

第12章 SDIO 驱动

12.1 SDIO 介绍

12.1.1 SDIO 基本介绍

SDIO是Secure Digital Input/Output的缩写，是一种用于嵌入式系统中的高速存储接口。它可以通过SD卡、MMC卡等存储设备实现高速数据存储和传输。在嵌入式系统中，SDIO通常是通过硬件实现的，主控芯片会集成SDIO控制器和SDIO PHY（物理层）模块。SDIO控制器负责协调数据传输和设备管理，而SDIO PHY则负责将数字信号转换为模拟信号或将模拟信号转换为数字信号。

SDIO最大的优点是它的传输速率非常快，可以达到几十MB/s的速度。此外，SDIO还支持热插拔和自动识别设备的功能，因此非常适合用于嵌入式系统中的高速数据存储和传输。在嵌入式系统中，SDIO通常用于连接各种存储设备，如SD卡、MMC卡等。此外，SDIO还可以用于连接其他类型的设备，如Wi-Fi模块、蓝牙模块、GPS模块等。

在 HC32F4A0 中，SDIOC 提供了一个 SD 主机接口和一个 MMC 主机接口，用于和支持 SD2.0 协议的 SD 卡，SDIO 设备以及支持 eMMC4.2 协议的 MMC 设备进行通信。用户可以通过读写 SDIOC 寄存器的方式启动 SDIOC 同片外 SD/MMC 设备进行通信。SDIOC 的数据缓冲寄存器用于 CPU/DMA 等主机设备和 SD 设备之间进行数据交换。SD 时钟由主机生成并输出给设备用于两者间的通信。

12.1.2 中断说明

一. SD 中断

SDIOC 提供两类中断，普通中断(normal interrupt)和错误中断(error interrupt)。普通中断是 SDIOC 和 SD/MMC 卡通信的过程中产生各种事件时发生的中断。错误中断是 SDIOC 动作通信过程中发生各种错误时产生的中断。普通中断和错误中断分别具有各自的中断状态寄存器，中断状态使能寄存器，以及中断信号使能寄存器。中断状态寄存器用于表示中断产生的要因，中断状态使能寄存器用于使能中断状态寄存器的各个状态位，中断状态寄存器的

各个状态位需要在中断状态使能寄存器使能位使能时有效,中断信号使能寄存器用于是否允许各个中断要因向 CPU 申请中断。

二. SDIO 中断

SDIO 设备在传输空闲时可以向主机发送卡中断(card interrupt)请求。使用 SDIO 中断时需要将中断状态使能寄存器(NORINTSTEN)的卡中断(CINTEN)使能,如果需要向 CPU 申请中断,还需将中断信号使能寄存器(NORINTSGEN)的卡中断(CINTEN)使能。SDIO 设备通过将 SDIOx_D1(x=1~2)数据线拉低向主机申请中断。SDIOC 检测到 SDIOx_D1(x=1~2)被拉低后将中断状态寄存器(NORINTST)的卡中断(CINT)置位,并根据设定向 CPU 申请中断。SDIO 设备判断中断处理完成后将释放 SDIOx_D1(x=1~2)。

三. DMA 中断

SDIOC 在通信中的读写数据能够通过 DMA 来完成。使用 DMA 传送向设备写数据时,将 DMA 的一个通道的启动源设置为 SDIOC 的写请求,再将 DMA 的传输目标地址设置为数据缓冲寄存器(BUF0, BUF1)且为固定地址。当 SDIOC 发送完写数据命令后,如果此时数据 FIFO 空,则向 DMA 发出写请求信号,启动 DMA 向数据缓冲寄存器(BUF0, BUF1)写入数据,这些数据将依次进入数据 FIFO 中。当 FIFO 中数据达到数据长度寄存器(BLKSIZE)的设定值或 512 字节时,SDIOC 将通过 SDIOx_Dy(x=1~2) (y=0~7)发送数据。如果是多块写命令,SDIOC 将同时继续向 DMA 发送启动请求写入数据。使用 DMA 传送从设备读数据时,将 DMA 的另一个通道的启动源设置为 SDIOC 的读请求,再将 DMA 的传输源地址设置为数据缓冲寄存器(BUF0, BUF1)且为固定地址。当 SDIOC 发送完读数据命令后,设备将通过 SDIOx_Dy(x=1~2) (y=0~7)发送数据,SDIOC 的数据 FIFO 开始接收数据,当 FIFO 中数据达到数据长度寄存器(BLKSIZE)的设定值或 512 字节时,SDIOC 向 DMA 发出读请求信号,启动 DMA 从数据缓冲寄存器(BUF0, BUF1)读出数据。如果是多块读命令,SDIOC 将同时继续从设备读取下一个数据块。

12.1.3 寄存器说明

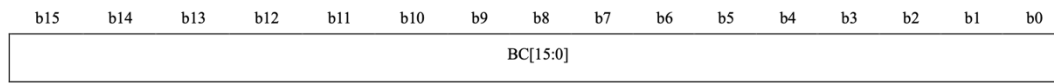
一. 数据块长度寄存器(BLKSIZE)

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved				TBS[11:0]											

- b15~b12: 保留位,读的时候读出 0,写的时候写入 0

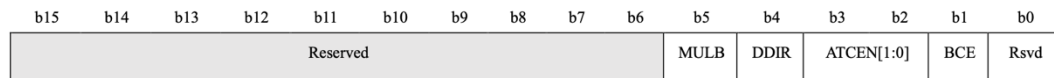
- b11~b0: 数据块长度

二. 数据块计数器(BLKCNT)



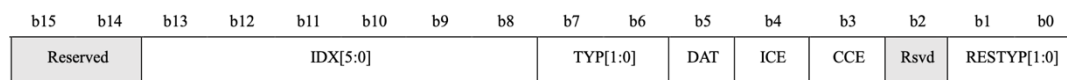
- b15~b0: 数据块计数

三. 传输模式寄存器(TRANSMODE)



- b15~b6: 保留位, 读的时候读出 0, 写的时候写入 0
- b5: 多数据块标志, 0 表示当前传输为单数据块, 1 表示为多数据块
- b4: 数据传输方向, 0 表示主机发送数据, 1 表示主机接收数据
- b3: 自动命令使能, 00 表示不发送自动命令, 01 表示多块传输结束后自动发送 CMD1
2, 10 和 11 表示禁止设定
- b1: 数据块计数使能, 0 表示禁止数据块计数使能, 1 表示许可
- b0: 保留位, 读的时候读出 0, 写的时候写入 0

四. 命令寄存器(CMD)



- b15~b14: 保留位, 读的时候读出 0, 写的时候写入 0
- b13~b8: 发送命令的编号
- b7~b6: 命令的类型
- b5: 0 表示当前命令仅仅使用 SDIOx_CMD 命令线, 1 表示需要使用数据线
- b4: 0 表示不检查应答中的命令编号, 1 表示检查
- b3: 0 表示不检查应答中的 CRC 校验码, 1 表示检查
- b2: 保留位, 读的时候读出 0, 写的时候写入 0
- b1~b0: 应答类型, 00 表示无应答, 10 表示应答长度为 136bit, 10 表示 48bit 应答, 11 表示命令具有长度为 48bit 并且带有忙状态的应答

五. 主机状态寄存器(PSTAT)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved							CMDL	DATL[3:0]			WPL	CDL	CSS	CIN	

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved				BRE	BWE	RTA	WTA	Reserved					DA	CID	CIC

- b31~b25: 保留位, 读的时候读出 0, 写的时候写入 0
- b24: 命令线的状态
- b23~b20: 数据线的状态
- b19: 写保护线的状态
- b18: 卡识别线的状态
- b17: 设备稳定状态
- b16: 设备插入的状态, 0 表示无设备插入, 1 表示有设备插入
- b15~b12: 保留位, 读的时候读出 0, 写的时候写入 0
- b11: 数据缓冲满标志
- b10: 数据缓冲空标志
- b9: 读操作状态, 0 表示没有正在进行的读操作, 1 表示有
- b8: 写操作状态, 0 表示没有正在进行的写操作, 1 表示有
- b7~b3: 保留位, 读的时候读出 0, 写的时候写入 0
- b2: 数据线传输状态, 0 表示数据线空闲, 1 表示正在传输数据
- b1: 带数据命令抑制, 0 表示允许发送带数据命令, 1 表示禁止
- b0: 命令抑制, 0 表示允许发送命令, 1 表示禁止

六. 主机控制寄存器(HOSTCON)

b7	b6	b5	b4	b3	b2	b1	b0
CDSS	CDTL	EXDW	Reserved		HSEN	DW	Rsvd

- b7: 卡识别线选择
- b6: 卡识别测试信号状态
- b5: 扩展数据位宽
- b4~b3: 保留位, 读的时候读出 0, 写的时候写入 0
- b2: 高速模式使能

- b1: 数据位宽选择, 0 表示数据位宽为 1, 1 表示数据位宽为 4
- b0: 保留位, 读的时候读出 0, 写的时候写入 0

12.2 SDIO 驱动介绍

SDIO 设备通过 SDIO BUS 挂接到 BUS 总线上, 其中 SDIO BUS 的结构体在 `bus_sdio.h` 文件中进行了定义。每个设备总线都关联一个 `drv` 和一个 `dev`, 其中, `drv` 结构体也在 `bus_sdio.h` 中进行了定义, `dev` 结构体在 `dev_sdio.h` 中进行了定义。另外, SDIO 驱动及设备的注册流程在 `connect_sdio.c` 中进行了实现。接下来, 针对这 3 个 SDIO 相关的具体结构和实现流程进行详细介绍。

12.2.1 bus_sdio.h 文件

`bus_sdio.h` 文件中声明了 SDIO 总线结构体 `SdioBus` 和驱动结构体 `SdioDriver`。`SdioBus` 的结构体定义如下:

```
struct SdioBus{
    struct Bus bus;
    void *private_data;
};
```

其中 `bus` 是通用管理总线; `private_data` 是传递参数的通用指针。

`SdioDriver` 的结构体定义如下:

```
struct SdioDriver{
    struct Driver driver;
    uint32 (*configure)(void *drv, struct BusConfigureInfo *configure_info);
};
```

其中 `driver` 表示通用管理驱动; `configure` 表示配置函数。

另外, 在 `bus_sdio.h` 文件中还定义了一些函数接口, 具体均在 `bus_sdio.c` 文件中实现:

- `int SdioBusInit(struct SdioBus *sdio_bus, const char *bus_name)`, 用于初始化 SDIO 总线, 其中 `sdio_bus` 表示初始化的 SDIO 总线; `bus_name` 表示总线名称。通过调用 `BusRegister` 函数将 SDIO 总线注册到 BUS 管理总线链表。
- `int SdioReleaseBus(struct SdioBus *sdio_bus)`, 用于释放 SDIO 总线, 即将 SDIO 总线从 BUS 管理总线链表中移除, 其中 `sdio_bus` 表示需要释放的 SDIO 总线。一般该接口函数与 `SdioBusInit` 函数配对使用, 对 SDIO 总线进行注册和释放。

- `int SdioDriverInit(struct SdioDriver *sdio_driver, const char *driver_name)`, 用于初始化 SDIO 驱动, 其中 `sdio_driver` 表示需要初始化的 SDIO 驱动; `driver_name` 表示驱动名称。该接口通过调用 `SdioDriverRegister` 函数将 SDIO 驱动注册到 SDIO 驱动总线。
- `int SdioDriverAttachToBus(const char *drv_name, const char *bus_name)`, 用于将 SDIO 驱动挂接到 SDIO BUS 总线, 其中 `drv_name` 表示驱动名称; `bus_name` 表示总线名称。
- `DriverType SdioDriverFind(const char *drv_name)`, 用于查找挂接的 SDIO 驱动, 其中 `drv_name` 表示驱动名称。

12.2.2 dev_sdio.h 文件

在 `dev_sdio.h` 文件中声明了 SDIO 设备结构体 `SdioHardwareDevice`, 其具体的定义如下:

```
struct SdioHardwareDevice{
    struct HardwareDev haldev;
    struct SdioDevDone *dev_done;
    void *private_data;
};
```

其中 `haldev` 用于描述一个设备结构; `dev_done` 用于表示一个 SDIO 设备的函数操作集; `private_data` 是传递参数的通用指针。

结构体 `SdioDevDone` 的定义如下:

```
struct SdioDevDone{
    uint32 (*open) (void *dev);
    uint32 (*close) (void *dev);
    uint32 (*write) (void *dev, struct BusBlockWriteParam *write_param);
    uint32 (*read) (void *dev, struct BusBlockReadParam *read_param);
};
```

可以看出, 该结构体是用于描述 SDIO 设备的打开、关闭、读写功能。

另外, 在 `dev_sdio.h` 文件中还定义了一些函数接口:

- `int SdioDeviceRegister(struct SdioHardwareDevice *sdio_device, const char *device_name)`, 用于初始化并注册 SDIO 设备到设备管理链表上, 其中 `sdio_device` 表示需要注册的 SDIO 设备; `sdio_param` 表示 SDIO 参数; `device_name` 表示设备名称。

- `int SdioDeviceAttachToBus(const char *dev_name, const char *bus_name)`, 用于将 SDIO 设备挂接到 SDIO BUS 总线, 其中 `dev_name` 表示设备名称; `bus_name` 表示总线名称。
- `HardwareDevType SdioDeviceFind(const char *dev_name)`, 用于查找挂接的 SDIO 设备, 其中 `dev_name` 表示设备名称。

12.2.3 connect_sdio.c 文件

`connect_sdio.c` 文件中实现了 SDIO 驱动及设备的注册, 该流程主要包含 3 个步骤:

- (1) 完成 SDIO 的驱动和设备到 SDIO 总线的挂接 (或称为绑定);
- (2) 实现 SDIO 设备的打开、关闭及读写方法, 其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植;
- (3) 实现 SDIO 设备的驱动配置, 需结合 SDIO 特性及操作系统的接口定义进行相应的开发。

步骤 (1) 的实现过程如代码 4-12-1 所示, 该流程是完成一个 SDIO 驱动及设备总线绑定的过程。代码行 5~7 定义了 3 个结构体变量。代码行 13 通过 `SD_Init` 函数初始化 SD 卡的硬件资源。代码行 15~19 使用 `SdioBusInit` 函数对 SDIO 总线进行初始化。代码行 21~25 使用 `SdioDriverInit` 函数对驱动进行初始化。代码行 26~29 使用 `SdioDriverAttachToBus` 函数将 SDIO 驱动绑定到总线。代码行 32~38 通过 `SdioDeviceRegister` 函数注册 SDIO 设备。代码行 39~43 通过 `SdioDeviceAttachToBus` 将 SDIO 设备挂接到总线。

代码 4-12-161 SDIO 驱动及设备绑定

```

01  int HwSdioInit(void)
02  {
03      x_err_t ret = EOK;
04
05      static struct SdioBus sdio_bus;
06      static struct SdioDriver sdio_drv;
07      static struct SdioHardwareDevice sdio_dev;
08
09      memset(&sdio_bus, 0, sizeof(struct SdioBus));
10      memset(&sdio_drv, 0, sizeof(struct SdioDriver));
11      memset(&sdio_dev, 0, sizeof(struct SdioHardwareDevice));
12

```

```

13     SdCardConfig();
14
15     ret = SdioBusInit(&sdio_bus, SDIO_BUS_NAME);
16     if (ret != EOK) {
17         KPrintf("Sdio bus init error %d\n", ret);
18         return ERROR;
19     }
20
21     ret = SdioDriverInit(&sdio_drv, SDIO_DRIVER_NAME);
22     if (ret != EOK) {
23         KPrintf("Sdio driver init error %d\n", ret);
24         return ERROR;
25     }
26     ret = SdioDriverAttachToBus(SDIO_DRIVER_NAME, SDIO_BUS_NAME);
27     if (ret != EOK) {
28         KPrintf("Sdio driver attach error %d\n", ret);
29         return ERROR;
30     }
31
32     sdio_dev.dev_done = &dev_done;
33     sdio_dev.haldev.dev_block_control = SdioControl;
34     ret = SdioDeviceRegister(&sdio_dev, SDIO_DEVICE_NAME);
35     if (ret != EOK) {
36         KPrintf("Sdio device register error %d\n", ret);
37         return ERROR;
38     }
39     ret = SdioDeviceAttachToBus(SDIO_DEVICE_NAME, SDIO_BUS_NAME);
40     if (ret != EOK) {
41         KPrintf("Sdio device register error %d\n", ret);
42         return ERROR;
43     }
44
45     return ret;
46 }

```

步骤（2）的实现过程如代码 4-12-2 所示，该流程是一个实现 SDIO 设备的打开、关闭及读写过程。根据结构体 SdioDevDone 的定义，打开、关闭、读和写的值分别是 SdioOpen、SdioClose、SdioWrite 和 SdioRead，表明所有功能是打开的。

代码 12-162 SDIO 设备的打开、关闭及读写

```

01 static struct SdioDevDone dev_done =
02 {
03     SdioOpen,
04     SdioClose,
05     SdioWrite,
06     SdioRead,
07 };

```

SDIO 设备的驱动配置过程如代码 4-12-3 所示。在 SdioConfigure 函数中，代码行 3~4 进行输入参数 drv 和 configure_info 的检查。代码行 6~13 对 SD 卡进行配置，如代码行 7 中对参数 configure_info->private_data 进行检查；代码行 8 声明一个结构体变量 args，并将其初始化为 configure_info->private_data；代码行 10~12 中对结构体变量 args 中的成员 size_perbank、block_size 和 bank_num 进行赋值。

代码 12-163 SDIO 设备的驱动配置

```

01 static uint32 SdioConfigure(void *drv, struct BusConfigureInfo *configure_info)
02 {
03     NULL_PARAM_CHECK(drv);
04     NULL_PARAM_CHECK(configure_info);
05
06     if (configure_info->configure_cmd == OPER_BLK_GETGEOME) {
07         NULL_PARAM_CHECK(configure_info->private_data);
08         struct DeviceBlockArrange *args =
09 (struct DeviceBlockArrange *)configure_info->private_data;
10
11         args->size_perbank = gSdHandle.stcSdCardInfo.u32BlockSize;
12         args->block_size = gSdHandle.stcSdCardInfo.u32BlockSize;
13         args->bank_num = gSdHandle.stcSdCardInfo.u32BlockNum;
14     }
15     return EOK;
16 }

```

12.3 SDIO 设备应用

SDIO 设备应用实现过程如代码 4-12-4 所示，主要通过验证 SD 卡的打开、关闭和读写功能。代码行 2~6 打开一个文件。代码行 12~27 首先用 read 函数读文件内容，然后用 write

函数写文件，最后用 `close` 函数关闭文件。代码行 29~44 首先用 `open` 函数重新打开文件，然后再进行读和关闭文件。

代码 4-12-164 SDIO 设备应用

```
01 void TestFs(void){
02     int fd = open(FPATH,O_RDWR|O_CREAT);
03     if(fd<0){
04         printf("fs fd open error:%d\n",fd);
05         return;
06     }
07
08     char filewords[MAX_READ_LENGTH];
09     memset(filewords,0,MAX_READ_LENGTH);
10     const char *input_words = "these words are going to write in fs\n";
11
12     int err_flag = read(fd,filewords,MAX_READ_LENGTH);
13     if(err_flag<0){
14         printf("read failed,error:%d\n",err_flag);
15         return;
16     }
17     printf("read data is \n%s\n",filewords);
18     err_flag = write(fd,input_words,strlen(input_words));
19     if(err_flag<0){
20         printf("write failed,error:%d\n",err_flag);
21         return;
22     }
23     err_flag = close(fd);
24     if(err_flag<0){
25         printf("close failed,error %d\n",err_flag);
26         return ;
27     }
28
29     fd = open(FPATH,O_RDWR);
30     if(fd<0){
31         printf("fs fd open error:%d\n",fd);
32         return;
33     }
34     err_flag = read(fd,filewords,MAX_READ_LENGTH);
35     if(err_flag<0){
36         printf("read failed,error:%d\n",err_flag);
```

```

37     return;
38 }
39 printf("read data is \n%s\n",filewords);
40 err_flag = close(fd);
41 if(err_flag<0){
42     printf("close failed,error:%d\n",err_flag);
43     return;
44 }
45
46 return;
47 }

```

第13章 CAN 驱动

13.1 CAN 介绍

13.1.1 CAN 基本介绍

CAN（Controller Area Network）总线是一种可以在无主机情况下实现微处理器或者设备之间相互通信的总线标准。CAN 总线控制器可以处理总线上的数据收发。CAN2.0B 控制器中应用程序可通过 1 个高优先级的主发送缓冲器（Primary Transmit Buffer，以下简称 PTB）和 3 个辅发送缓冲器（Secondary Transmit Buffer，以下简称 STB）将发送数据送至总线，由发送调度器决定邮箱发送顺序。通过 8 个接收缓冲器（Receive Buffer，以下简称 RB）获取总线数据。3 个 STB 以及 8 个 RB 可以理解为一个 3 级 FIFO 和一个 8 级 FIFO，FIFO 完全由硬件控制。下图 4-13-1 为 CAN 系统的结构框图。

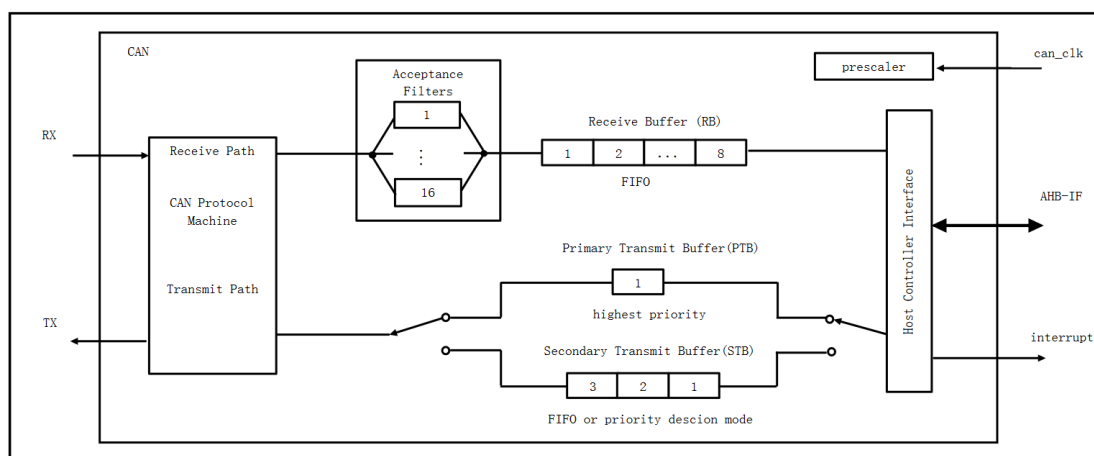


图 4-13-182 CAN 系统框图

13.1.2 中断说明

表 4-13-1 罗列了 CAN 控制器的中断标志及其描述。

表 4-13-3 CAN 中断表

符号	中断标志	描述
CAN_1_HOST CAN_2_HOST	RIF	接收中断
	ROIF	接收上溢中断
	RFIF	接收 BUF 满中断
	RAFIF	接收 BUF 将满中断
	TPIF	PTB 发送中断
	TSIF	STB 发送中断
	EIF	错误中断
	AIF	取消发送中断
	EPIE	错误被动中断
	ALIF	仲裁失败中断
	BEIF	总线错误中断
	WTIF	触发看门中断
	TEIF	触发错误中断
	TTIF	时间触发中断

13.1.3 寄存器说明

一. CAN 配置和状态寄存器 (CAN_CFG_STAT)

b7	b6	b5	b4	b3	b2	b1	b0
RESET	LBME	LBMI	TPSS	TSSS	RACTIVE	TACTIVE	BUSOFF

- b7: 复位请求标志, 0 表示不请求局部复位, 1 表示请求局部复位
- b6: 外部回环模式使能位, 0 表示禁止外部回环模式, 1 表示使能外部回环模式
- b5: 内部回环模式使能位, 0 表示禁止内部回环模式, 1 表示使能内部回环模式
- b4: PTB 单次传输模式, 0 表示禁止 PTB 单次传输模式, 1 表示使能 PTB 单次传输模式
- b3: STB 单次传输模式, 0 表示禁止 STB 单次传输模式, 1 表示使能 STB 单次传输模式
- b2: 接收中状态信号标志, 0 表示非接收中, 1 表示接收中
- b1: 发送中状态信号标志, 0 表示非发送中, 1 表示发送中
- b0: 总线关闭状态标志, 0 表示总线有效状态, 1 表示总线无效状态

二. CAN 发送控制寄存器 (CAN_TCTRL)

b7	b6	b5	b4	b3	b2	b1	b0
-	TSNEXT	TSMODE	TTTBM	-	-	TSSTAT[1:0]	

- b7: 保留位, 必须保持复位值 (即必须为 0)
- b6: 下一个 STB 标志, 0 表示无动作, 1 表示当前 STB SLOT 已经填充, 指向下一个 SLOT
- b5: STB 发送模式标志, 0 表示 FIFO 模式, 1 表示优先级模式
- b4: TTCAN BUF 模式标志, 0 表示 TSMODE 决定 PTB 和 STB, 1 表示通过 TBPTR 和 TTPTR 设定
- b3~b2: 保留位, 必须保持复位值 (即必须为 0)
- b1~b0: STB 状态标志, 00 表示 STB 空, 01 表示 STB 小于等于半满, 10 表示 STB 大于半满, 11 表示 STB 满

三. CAN 接收控制寄存器 (CAN_RCTRL)

b7	b6	b5	b4	b3	b2	b1	b0
SACK	ROM	ROV	RREL	RBALL	-	RSTAT[1:0]	

- b7: 自应答标志位, 0 表示无字应答, 1 表示使能自应答功能
- b6: 接收 BUF 上溢模式设定位, 0 表示最早接收的数据被覆盖, 1 表示接受到的数据不被存储
- b5: 接收 BUF 上溢标志位, 0 表示无上溢, 1 表示发生上溢, 最少有一个数据丢失
- b4: 释放接收 BUF 标志位, 0 表示不释放, 1 表示该接收 BUF 已经被读取过, RBUF 寄存器只想下一个 RB SLOT
- b3: 接收 BUF 数据存储所有的数据帧标志, 0 表示正常模式, 1 表示存储所有的数据包有错误的帧
- b2: 保留位, 必须保持复位值 (即必须为 0)
- b1~b0: 接收 BUF 状态, 00 表示 RBUF 为空, 01 表示非空但是小于 AFWL 的值, 10 表示 RBUF 大于等于 AFWL 的值, 但是没有满, 11 表示满

四. CAN 接收和发送中断使能寄存器 (CAN_RTIE)

b7	b6	b5	b4	b3	b2	b1	b0
RIE	ROIE	RFIE	RAFIE	TPIE	TSIE	EIE	TSFF

- b7: 接收中断使能
- b6: 接收上溢中断使能
- b5: 接收 BUF 满中断使能
- b4: 接收 BUF 将满中断使能
- b3: PTB 发送中断使能
- b2: STB 发送中断使能
- b1: 错误中断使能
- b0: 发送 BUF 满标志

五. CAN 接收和发送中断状态寄存器 (CAN_RTIF)

b7	b6	b5	b4	b3	b2	b1	b0
RIF	ROIF	RFIF	RAFIF	TPIF	TSIF	EIF	AIF

- b7: 接收中断标志, 0 表示为收到数据帧, 1 表示收到有效的数据帧或者远程帧
- b6: 接收上溢中断标志
- b5: 接收 BUF 满中断标志
- b4: 接收 BUF 将满中断标志
- b3: PTB 发送中断标志
- b2: STB 发送中断标志
- b1: 错误中断标志
- b0: 取消发送中断标志

13.2 CAN 驱动介绍

CAN 设备通过 CAN BUS 挂接到 BUS 总线上, 其中 CAN BUS 的结构体在 `bus_can.h` 文件中进行了定义。每个设备总线都关联一个 `drv` 和一个 `dev`, 其中, `drv` 结构体也在 `bus_can.h` 中进行了定义, `dev` 结构体在 `dev_can.h` 中进行了定义。另外, CAN 驱动及设备的注册流程在 `connect_can.c` 中进行了实现。接下来, 针对这 3 个 CAN 相关的具体结构和实现流程进行详细介绍。

13.2.1 bus_can.h 文件

bus_can.h 文件中声明了 CAN 总线结构体 CanBus 和驱动结构体 CanDriver。CanBus 的结构体定义如下：

```
struct CanBus{
    struct Bus bus;
    void *private_data;
};
```

其中 bus 是通用管理总线；private_data 是传递参数的通用指针。

CanDriver 的结构体定义如下：

```
struct CanDriver{
    struct Driver driver;
    uint32 (*configure) (void *drv, struct BusConfigureInfo *configure_info);
    void *private_data;
};
```

其中 driver 表示通用管理驱动；configure 表示配置函数；private_data 是传递参数的通用指针。

另外，在 bus_can.h 文件中还定义了一些函数接口，具体均在 bus_can.c 文件中实现：

- int CanBusInit(struct CanBus *can_bus, const char *bus_name)，用于初始化 CAN 总线，其中 can_bus 表示初始化的 CAN 总线；bus_name 表示总线名称。通过调用 BusRegister 函数将 CAN 总线注册到 BUS 管理总线链表。
- int CanReleaseBus(struct CanBus *can_bus)，用于释放 CAN 总线，即将 CAN 总线从 BUS 管理总线链表中移除，其中 can_bus 表示需要释放的 CAN 总线。一般该接口函数与 CanBusInit 函数配对使用，对 CAN 总线进行注册和释放。
- int CanDriverInit(struct CanDriver *can_driver, const char *driver_name)，用于初始化 CAN 驱动，其中 can_driver 表示需要初始化的 CAN 驱动；driver_name 表示驱动名称。该接口通过调用 CanDriverRegister 函数将 CAN 驱动注册到 CAN 驱动总线。
- int CanDriverAttachToBus(const char *drv_name, const char *bus_name)，用于将 CAN 驱动挂接到 CAN BUS 总线，其中 drv_name 表示驱动名称；bus_name 表示总线名称。

- `DriverType CanDriverFind(const char *drv_name, enum DriverType_e drv_type)`, 用于查找挂接的 CAN 驱动, 其中 `drv_name` 表示驱动名称; `drv_type` 表示驱动类型。

13.2.2 dev_can.h 文件

在 `dev_can.h` 文件中声明了 CAN 设备结构体 `CanHardwareDevice`, 其具体的定义如下:

```
struct CanHardwareDevice{
    struct HardwareDev haldev;
    const struct CanDevDone *dev_done;
    void *private_data;
};
```

其中 `haldev` 用于描述一个设备结构; `dev_done` 用于表示一个 CAN 设备的函数操作集; `private_data` 是传递参数的通用指针。

结构体 `CanDevDone` 的定义如下:

```
struct CanDevDone{
    uint32 (*open) (void *dev);
    uint32 (*close) (void *dev);
    uint32 (*write) (void *dev, struct BusBlockWriteParam *write_param);
    uint32 (*read) (void *dev, struct BusBlockReadParam *read_param);
};
```

可以看出, 该结构体是用于描述 CAN 设备的打开、关闭、读写功能。

另外, 在 `dev_can.h` 文件中还定义了一些函数接口:

- `int CanDeviceRegister(struct CanHardwareDevice *can_device, void *can_param, const char *device_name)`, 用于初始化并注册 CAN 设备到设备管理链表上, 其中 `can_device` 表示需要注册的 CAN 设备; `can_param` 表示 CAN 参数; `device_name` 表示设备名称。
- `int CanDeviceAttachToBus(const char *dev_name, const char *bus_name)`, 用于将 CAN 设备挂接到 CAN BUS 总线, 其中 `dev_name` 表示设备名称; `bus_name` 表示总线名称。
- `HardwareDevType CanDeviceFind(const char *dev_name, enum DevType dev_type)`, 用于查找挂接的 CAN 设备, 其中 `dev_name` 表示设备名称; `dev_type` 表示设备类型。

13.2.3 connect_can.c 文件

`connect_can.c` 文件中实现了 CAN 驱动及设备的注册, 与 ADC 的流程相同, CAN 的流程也包含 3 个步骤:

- (1) 完成 CAN 的驱动和设备到 CAN 总线的挂接（或称为绑定）；
- (2) 实现 CAN 设备的打开、关闭及读写方法；
- (3) 实现 CAN 设备的驱动配置。

步骤（1）的实现过程如代码 4-13-1 所示，该流程是完成一个 CAN 驱动及设备总线绑定的过程。代码行 5 到 6 对定义变量 `can_bus` 进行结构体类型填充，实现 CAN 总线初始化。代码行 13~17 使用 `BoardCanBusInit` 函数进行 CAN 驱动挂接到 CAN 总线，具体的实现过程详见代码 4-13-2。代码行 19~23 使用 `BoardCanDevBend` 函数进行 CAN 设备挂接到 CAN 总线，具体的实现过程详见代码 4-13-3。

在代码 4-13-2 的 `BoardCanBusInit` 函数中，代码行 5~10 对 CAN 总线进行初始化；代码行 12~17 对 CAN 驱动进行初始化；代码行 18~23 将 CAN 驱动挂接到 CAN 总线上。

在代码 4-13-3 的 `BoardCanDevBend` 函数中，代码行 3~5 中声明了变量 `can_device0`，并对结构体变量 `can_device0` 进行类型填充；代码行 9~13 中对 CAN 设备进行注册；代码行 15~19 将 CAN 设备挂接到 CAN 总线。

代码 4-13-165 CAN 驱动及设备绑定

```

01 int HwCanInit(void)
02 {
03     x_err_t ret = EOK;
04
05     static struct CanBus can_bus;
06     memset(&can_bus, 0, sizeof(struct CanBus));
07
08     static struct CanDriver can_driver;
09     memset(&can_driver, 0, sizeof(struct CanDriver));
10
11     can_driver.configure = &(CanDrvConfigure);
12
13     ret = BoardCanBusInit(&can_bus, &can_driver);
14     if (EOK != ret) {
15         KPrintf(" can_bus_init %s error ret %u\n", CAN_BUS_NAME_2, ret);
16         return ERROR;
17     }
18
19     ret = BoardCanDevBend();

```

```

20     if (EOK != ret) {
21         KPrintf("board_can_init error ret %u\n", ret);
22         return ERROR;
23     }
24     return EOK;
25 }

```

代码 4-13-166 CAN 驱动挂接过程

```

01 static int BoardCanBusInit(struct CanBus *can_bus, struct CanDriver *can_driver)
02 {
03     x_err_t ret = EOK;
04
05     /*Init the can bus */
06     ret = CanBusInit(can_bus, CAN_BUS_NAME_2);
07     if (EOK != ret) {
08         KPrintf("Board_can_init canBusInit error %d\n", ret);
09         return ERROR;
10     }
11
12     /*Init the can driver*/
13     ret = CanDriverInit(can_driver, CAN_DRIVER_NAME_2);
14     if (EOK != ret) {
15         KPrintf("Board_can_init canDriverInit error %d\n", ret);
16         return ERROR;
17     }
18     /*Attach the can driver to the can bus*/
19     ret = CanDriverAttachToBus(CAN_DRIVER_NAME_2, CAN_BUS_NAME_2);
20     if (EOK != ret) {
21         KPrintf("Board_can_init CanDriverAttachToBus error %d\n", ret);
22         return ERROR;
23     }
24
25     return ret;
26 }

```

代码 4-13-167 CAN 设备挂接过程

```

01 static int BoardCanDevBend(void)
02 {
03     x_err_t ret = EOK;
04     static struct CanHardwareDevice can_device0;

```

```

05     memset(&can_device0, 0, sizeof(struct CanHardwareDevice));
06
07     can_device0.dev_done = &can_dev_done;
08
09     ret = CanDeviceRegister(&can_device0, NONE, CAN_2_DEVICE_NAME_1);
10     if (EOK != ret) {
11         KPrintf("board_can_init CanDeviceInit device %s error %d\n",
CAN_2_DEVICE_NAME_1, ret);
12         return ERROR;
13     }
14
15     ret = CanDeviceAttachToBus(CAN_2_DEVICE_NAME_1, CAN_BUS_NAME_2);
16     if (EOK != ret) {
17         KPrintf("board_can_init CanDeviceAttachToBus device %s error %d\n",
CAN_2_DEVICE_NAME_1, ret);
18         return ERROR;
19     }
20
21     return ret;
22 }

```

步骤（2）的实现过程如代码 4-13-4 所示，该流程是一个实现 CAN 设备的打开、关闭及读写过程。根据结构体 CanDevDone 的定义，其中.open、.close、.write 和.read 分别表示 CAN 的打开、关闭、读、写。此时打开和关闭功能的值为 NONE，表示未启用该功能。CanSendMsg 和 CanRecvMsg 分别是实现写和读数据功能的具体函数。

代码 13-168 CAN 设备的打开、关闭及读写

```

01 static struct CanDevDone dev_done =
02 {
03     .open  = NONE,
04     .close = NONE,
05     .write = CanWriteData,
06     .read  = CanReadData
07 };

```

CAN 设备的驱动配置过程如代码 4-13-5 所示。在 CanDrvConfigure 函数中，代码行 4~5 进行输入参数 drv 和 configure_info 的检查。代码行 6 开始从接收的参数 configure_info 结构体里的 configure_cmd 字段来决定工作模式。若为 OPE_INT 则执行 CanInit 对 Can 设备进行初始化。若为 OPE_CFG，则执行 CanConfig 函数。

代码 4-13-169 CAN 设备的驱动配置

```

01 static uint32 CanDrvConfigure(void *drv, struct BusConfigureInfo *configure_info)
02 {
03     x_err_t ret = EOK;
04     NULL_PARAM_CHECK(drv);
05     NULL_PARAM_CHECK(configure_info);
06     struct CanDriverConfigure *can_drv_config;
07     switch (configure_info->configure_cmd)
08     {
09         case OPE_INT: // can basic init
10             can_drv_config = (struct CanDriverConfigure
11 *)configure_info->private_data;
12             CanInit(can_drv_config);
13             break;
14         case OPE_CFG:
15             CanConfig(configure_info->private_data);
16             break;
17         default:
18             break;
19     }
20     return ret;
21 }

```

13.3 CAN 设备应用

在测试函数 test_can 中，代码行 3 到 9 通过 open 函数打开一个 CAN 设备，如果打开不成功，则直接 return 返回。代码行 15-20 定义 can 配置结构体，并进行初始化。代码行 24 到 30 通过 ioctl 函数（即 IO 控制函数）对打开的 CAN 设备的通道进行配置，若配置不成功，则关闭 CAN 并返回。代码行 45 开始的 while 循环中，开始调用接口对 can 设备进行读写。代码行 57 和 58 分别用于关闭 CAN 和函数返回。

代码 4-13-170 CAN 设备应用

```

01 void TestCAN(void)
02 {
03     // config CAN pin(SCL:34.SDA:35) in menuconfig
04     int can_fd = PrivOpen(CAN_DEV_DRIVER, O_RDWR);
05     if (can_fd < 0)
06     {

```

```
07     printf("open can_fd fd error:%d\n", can_fd);
08     return;
09 }
10     printf("CAN open successful!\n");
11
12     struct PrivIoctlCfg ioctl_cfg;
13     ioctl_cfg.ioctl_driver_type = CAN_TYPE;
14
15     struct CanDriverConfigure can_config;
16     can_config.brp = 8U;
17     can_config.tbs1 = 1U + 8U;
18     can_config.tbs2 = 4U;
19     can_config.tsjw = 4U;
20     can_config.mode = 0U;
21
22     ioctl_cfg.args = (void *)&can_config;
23
24     if (0 != PrivIoctl(can_fd, OPE_INT, &ioctl_cfg))
25     {
26         printf("init can fd error %d\n", can_fd);
27         PrivClose(can_fd);
28         return;
29     }
30     printf("CAN configure successful!\n");
31
32     uint8_t data_buff[64u] = {1,2,3,4,4,3,2,1};
33     struct CanSendConfigure frame_send;
34     frame_send.ide=0;
35     frame_send.stdid = 0x55;
36     frame_send.rtr=0;
37     frame_send.data_lenth=8;
38     frame_send.data = data_buff;
39
40     struct CanSendConfigure frame_rcv;
41     uint8_t rcv_buff[65U] = {0};
42     frame_rcv.data = rcv_buff;
43
44     // CAN write
45     while (1)
46     {
47         PrivTaskDelay(500);
```

```

48     PrivWrite(can_fd, &frame_send, NONE);
49     PrivTaskDelay(500);
50     PrivRead(can_fd, &frame_recv, NONE);
51     // if any data has received,Then printf message
52     if(frame_recv.data_lenth > 0){
53         printf("ID %08x:%s\n",frame_recv.exdid,frame_recv.data);
54     }
55 }
56
57 PrivClose(can_fd);
58 return;
59 }

```

第14章 ETH 驱动

14.1 以太网 MAC 控制器介绍

14.1.1 以太网 MAC 控制器基本介绍

以太网 MAC 控制器（ETHMAC）用于在以太网网络中按照 IEEE802.3-2002 标准发送和接收数据，有多种应用领域，如交换机、网络接口卡等。该 MAC 控制器支持与外部物理层（PHY）相连的两个工业标准接口：介质独立接口（MII）和简化介质独立接口（RMII）。

图 4-14-1 为以太网 MAC 控制器的架构框图。

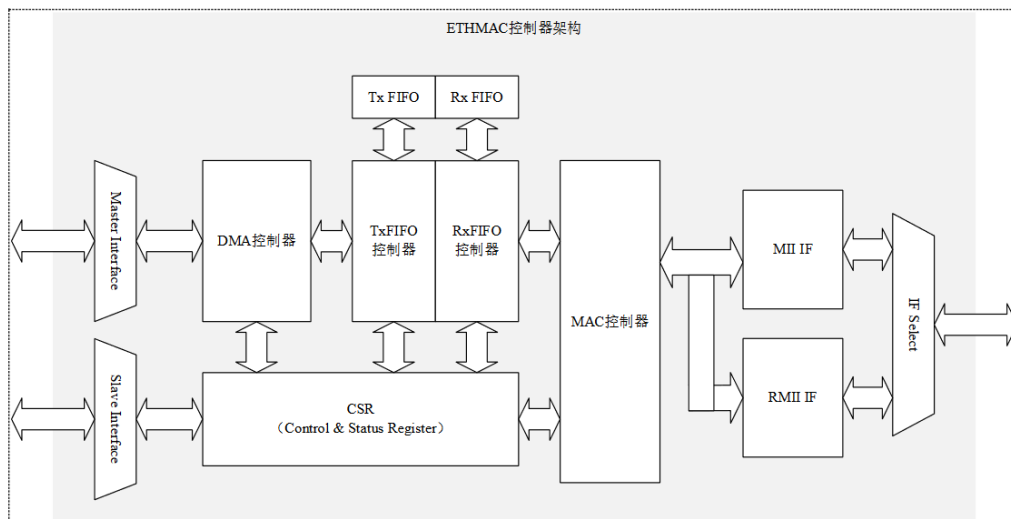


图 4-14-183 以太网 MAC 控制器架构框图

图中 AHB Master 接口用于数据传输，AHB Slave 接口用于该控制器的基本寄存器访问配置等。在发送数据时，首先将数据从系统存储器由 DMA 控制器送至 Tx FIFO 进行缓冲，经 MAC 控制器的一系列处理（COE 控制、PTP 控制、MMC 控制等）后通过 MII 接口或 RMII 接口送至外部 PHY。在接收数据时，经过 MI 接口或 RMII 接口接收进来的数据，经 MAC 控制器的一系列处理（COE 控制、PTP 控制、MMC 控制等）后送至 Rx FIFO，DMA 控制器再将 Rx FIFO 的数据传送到系统存储器。

14.1.2 中断说明

图 4-14-2 是以太网 MAC 控制器中的中断实现方案。从图中可以看出，ETHMAC 将 DMA 的正常事件（NIS）、DMA 的异常事件（AIS）、PMT 事件（PMTIS）、MMC 事件（MMC**IS）、PTP 事件（TSPIS）等通过中断的使能或屏蔽控制汇总成一个中断事件输出至中断模块，所以系统在中断响应时，需要通过读取相应的状态信息来确定是哪种类型的事件产生了中断。同时，ETHMAC 的 PMT 模块还有一个非屏蔽的中断输出。在 PMT 事件产生时，不管中断屏蔽寄存器（ETH_MAC_INTMSKR）的 PMTIM 位是否置 1，都会产生 PMT 非屏蔽中断。

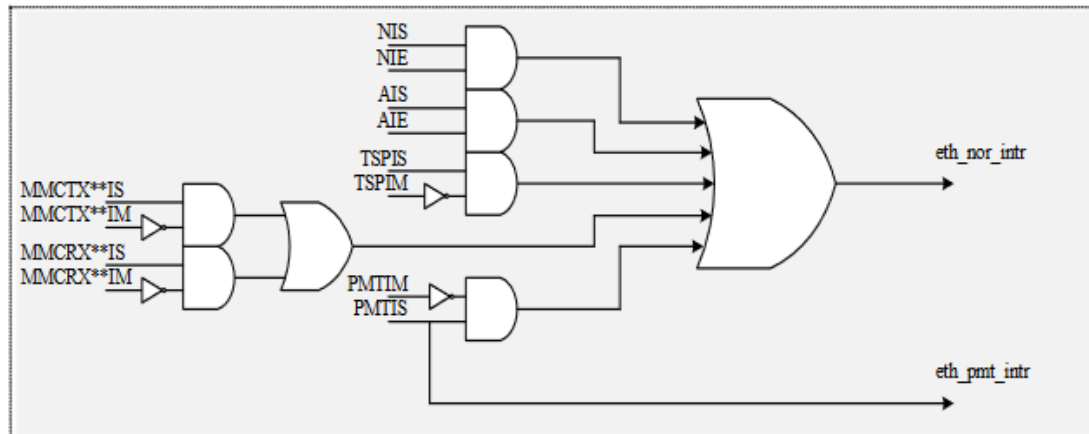


图 4-14-184 ETHMAC 中断方案

一. DMA 中断

DMA 在发送数据和接收数据过程中产生的各种正常事件会触发普通中断、各种异常事件会触发异常中断。如图 4-13-3 所示。各具体事件的中断控制可通过 DMA 中断使能寄存器的各个位控制。

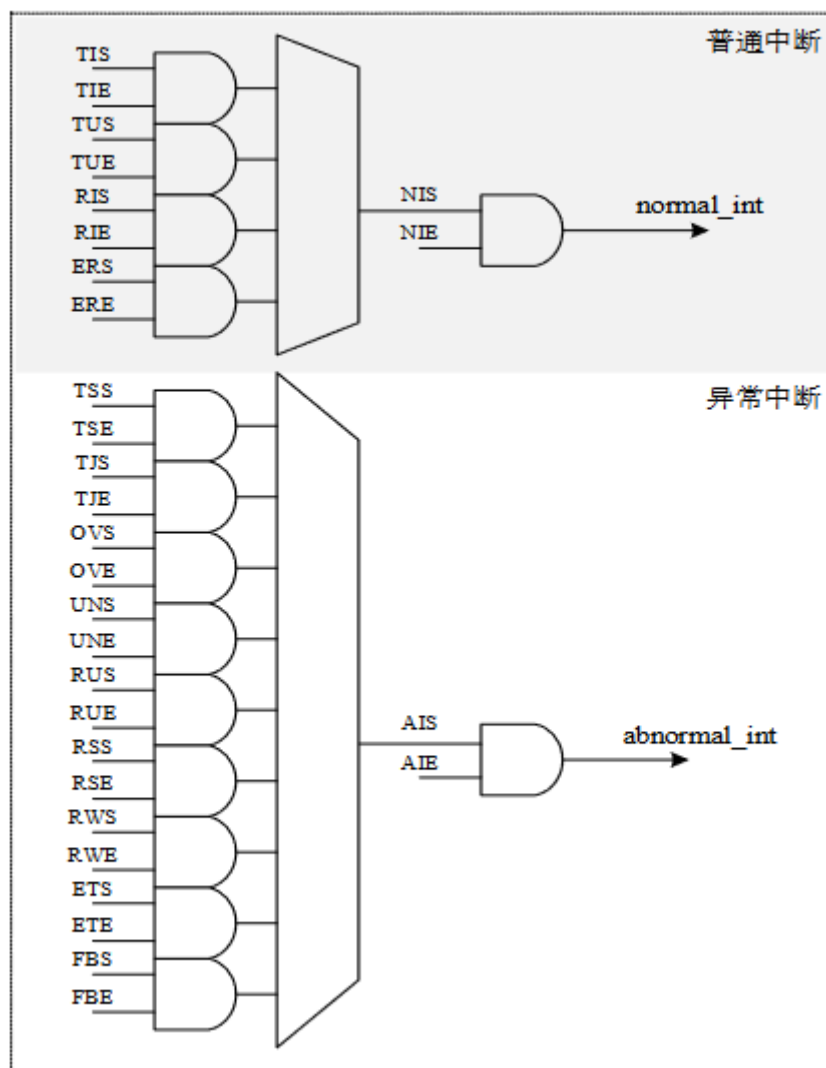


图 4-14-185 DMA 中断组成

二. PMT 中断

MAC 在接收到远程唤醒帧和魔术数据包时可触发生成 PMT 中断（可屏蔽中断和非屏蔽中断）。其中可屏蔽中断的生成由 MAC 中断屏蔽寄存器（ETH_MAC_INTMSKR）的 PMTIM 位控制。

三. PTP 中断

在 PTP 功能有效时，系统时间到达时间戳目标时间寄存器 0~1 中的设定值时可触发生成 PTP 中断。该中断由 PPS 输出控制寄存器（ETH_MAC_PPSCCLR）的 TT0SEL、TT1SEL 位和 MAC 中断屏蔽寄存器（ETH_MAC_INTMSKR）的 TSPIM 位控制。

四. MMC 中断

在 MMC 中各项发送帧统计数和接收帧统计数溢出时可触发生成 MMC 中断。该中断由 MMC 的 Rx 中断控制寄存器（ETH_MMC_RITCTLR）和 Tx 中断控制寄存器（ETH_MMC_TITCTLR）的各个位控制。

14.1.3 以太网 MAC 寄存器说明

一. 接口选择寄存器(ETH_MAC_IFCONFR)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved										TCK INV	RCK INV	-	-	-	IF SEL

- b31~b6: 保留位，作为保留位，读的时候为 0，写的时候写 0
- b5: TX 时钟取反标志
- b4: RX 时钟取反标志
- b3~b1: 保留位，作为保留位，读的时候为 0，写的时候写 0
- b0: 接口选择位，0 表示选择 MII 接口，1 表示选择 RMII 接口

二. MAC 配置寄存器(ETH_MAC_CONFIGR)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
-	SAIRC[2:0]			-	-	CST	-	MWD	MJB	-	-	IFG[2:0]		DCRS	
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
-	FES	DO	LM	DM	IPCO	DRTY	-	ACS	BL[1:0]		DC	TE	RE	-	-

- b31: 保留位，作为保留位，读的时候为 0，写的时候写 0
- b30~b28: 源地址插入及替换，010 表示将 MAC 地址寄存器 0 中的地址值作为 SA 地址插入到发送帧中 R/W，110 表示将 MAC 地址寄存器 1 中的地址值作为 SA 地址插入到发送帧中，011 表示将 MAC 地址寄存器 0 中的地址值作为 SA 地址替换发送帧中的 SA 字段，111 表示将 MAC 地址寄存器 1 中的地址值作为 SA 地址替换发送帧中的 SA 字段，x0x 表示发送帧中 SA 字段地址的处理由 Tx 述符 TDES1 的 SAIRC 位控制
- b27~b26: 保留位，作为保留位，读的时候为 0，写的时候写 0
- b25: TYPE 帧 FCS 去除，0 表示该功能无效，1 表示在在将帧转发到应用之前，去除并丢弃所有 TYPE(类型字段大 于 0x0600)帧的最后 4 个字节(FCS 字段)
- b24: 保留位，作为保留位，读的时候为 0，写的时候写 0

- b23: 屏蔽看门狗标志
- b22: 屏蔽 Jabber 标志
- b21~b20: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0
- b19~b17: 发送期间帧间的最小间隙
- b16: 屏蔽载波侦听标志
- b15: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0
- b14: 速度选择标志, 0 表示 10Mibps, 1 表示 100Mibps
- b13: 屏蔽接收自身包标志
- b12: Loopback 模式标志
- b11: 全双工模式选择标志, 0 表示半双工模式, 1 表示全双工模式
- b10: Checksum Offload 功能
- b9: 屏蔽重试
- b8: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0
- b7: 自动去吃 PAD/FCS
- b6~b5: 后退限制
- b4: 延迟检查使能
- b3: 发送使能
- b2: 接收使能
- b1~b0: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0

三. MAC 状态寄存器(ETH_MAC_MACSTSR)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved						TFF	TFNE	-	TFWA	TFRS[1:0]		MTP	MTS[1:0]		MTEA
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved						RFFL[1:0]		-	RFRS[1:0]		RFWA	-	MRS[1:0]		MREA

- b31~b26: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0
- b25: Tx FIFO 满标志, 0 表示 Tx FIFO 未滿, 1 表示滿
- b24: Tx FIFO 空标志, 0 表示 Tx FIFO 空, 1 表示非空
- b23: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0

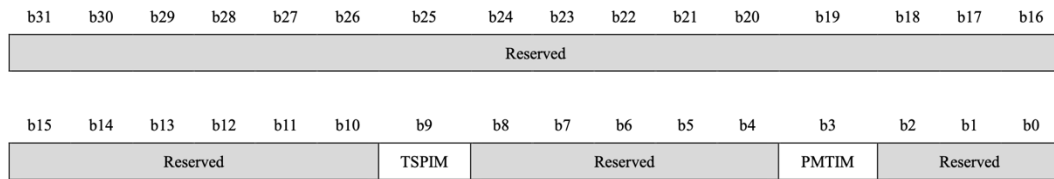
- b22: Tx FIFO 写有效标志
- b21~b20: Tx FIFO 读状态
- b19: MAC 发送器暂停
- b18~b17: MAC 发送器状态
- b16: MAC MII 发送引擎状态
- b15~b10: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0
- b9~b8: Rx FIFO 填充级别
- b7: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0
- b6~b5: Rx FIFO 读状态
- b4: Rx FIFO 写有效标志
- b3: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0
- b2~b1: MAC 接收器状态
- b0: MAC MII 接收引擎状态

四. 中断状态寄存器(ETH_MAC_INTSTSR)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
Reserved															
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
Reserved						TSPIS	-	-	MMC TXIS	MMC RXIS	MMC IS	PMTIS	-	-	-

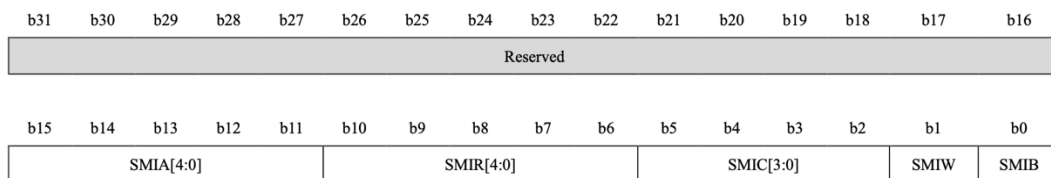
- b31~b10: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0
- b9: PPT 时间戳中断
- b8~b7: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0
- b6: MMC 发送统计中断
- b5: MMC 接收统计中断
- b4: MMC 统计中断
- b3: PMT 中断
- b2~b0: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0

五. 中断屏蔽寄存器(ETH_MAC_INITMSKR)



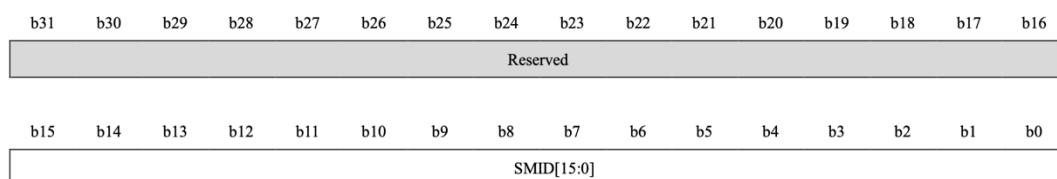
- b31~b10: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0
- b9: PPT 时间戳中断屏蔽
- b8~b4: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0
- b3: PMT 中断屏蔽
- b2~b0: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0

六. SMI 地址寄存器(ETH_MAC_SMIADDR)



- b31~b16: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0
- b15~b11: 物理层地址
- b10~b6: PHY 寄存器
- b5~b2: MDC 时钟
- b1: SMI 读写标志, 0 表示 SMI 读操作, 1 表示写操作
- b0: SMI 访问忙标志

七. SMI 数据寄存器(ETH_MAC_SMIDATR)



- b31~b16: 保留位, 作为保留位, 读的时候为 0, 写的时候写 0
- b15~b0: SMI 数据, 站点管理读操作之后从 PHY 中读取的 16 位数据值, 或者站点管理写操作之前要写入 PHY 的 16 位数据值

14.2 ETH 驱动介绍

14.2.1 ethernetif.c 文件

ETH 底层通过 lwip 来实现通信，主要功能在 ethernetif.c 文件中实现，底层调用 lwip，实现 socket 通信接口。

ETH 的初始化函数如代码 4-14-1 所示，代码行 3~6 对主机名称进行初始化，由于使用的是 lwip，所以初始化为“lwip”；代码行对主机名称进行初始化，代码行 10~17 对网络输出端口及相关硬件进行初始化；代码行 20 对底层模块和硬件进行始化。

代码 4-14-171 ETH 初始化

```

01  err_t ethernetif_init(struct netif *netif)
02  {
03  #if LWIP_NETIF_HOSTNAME
04      /* Initialize interface hostname */
05      netif->hostname = "lwip";
06  #endif /* LWIP_NETIF_HOSTNAME */
07      netif->name[0] = IFNAME0;
08      netif->name[1] = IFNAME1;
09
10  #ifndef ETHERNET_LOOPBACK_TEST
11      /* We directly use etharp_output() here to save a function call.
12       * You can instead declare your own function an call etharp_output()
13       * from it if you have to do some checks before sending (e.g. if link
14       * is available...) */
15      netif->output = &etharp_output;
16      netif->linkoutput = &low_level_output;
17  #endif
18
19      /* initialize the hardware */
20      return low_level_init(netif);
21  }

```

14.3 ETH 设备应用

ETH 设备应用分为服务端和客户端，两者的操作类似。这里我们使用回环进行测试。代码行 7~12 进行管脚初始化，15 行对网络进行初始化，17~20 行对缓冲区进行初始化，23 行创建一个线程，调用 EthLoopBackTask 函数进行测试。30 行启动这个线程。35 行为 EthLoopBackTask 函数。这个函数主要包含一个 while 循环，使用 user 的管脚电平的高低来控制信息的收发，每按下一次 user 按键，进行一次收发。

代码 4-14-172 ETH 设备应用

```

01 static void EthLoopBackTest(void)
02 {
03     x_err_t ret = EOK;
04
05     stc_gpio_init_t stcGpioInit;
06
07     /* KEY initialize */
08     (void)GPIO_StructInit(&stcGpioInit);
09     stcGpioInit.u16PinState = PIN_STAT_RST;
10     stcGpioInit.u16PinDir = PIN_DIR_IN;
11     (void)GPIO_Init(USER_KEY_PORT, USER_KEY_PIN, &stcGpioInit);
12     GPIO_ResetPins(USER_KEY_PORT, USER_KEY_PIN);
13
14     /* Configure the Ethernet */
15     (void)ethernetif_init(&testnetif);
16
17     /* fill data to txPbuf */
18     txPbuf.next = NULL;
19     txPbuf.payload = txBuf;
20     txPbuf.len = strlen(txBuf);
21
22     int eth_loopback_task = 0;
23     eth_loopback_task = KTaskCreate("eth_loopback"
24                                     , EthLoopBackTask, NONE, 2048, 8);
25     if(eth_loopback_task < 0) {
26         KPrintf("eth_loopback_task create failed ...
27 %s %d.\n", __FUNCTION__, __LINE__);
28         return;
29     }
30     StartupKTask(eth_loopback_task);
31
32     return;
33 }
34
35 static void EthLoopBackTask(void *parameter)
36 {
37     while (1) {

```

```
38         if (RESET == GPIO_ReadInputPins(USER_KEY_PORT, USER_KEY_PIN)) {
39             KPrintf("ready to send eth data\n");
40             if (LL_OK != low_level_output(&testnetif, &txPbuf)) {
41                 KPrintf("eth send data error! \r\n");
42             }
43         }
44
45         //KPrintf("ready to receive eth loop back data\n");
46         /* Read a received packet */
47         ethernetif_input(&testnetif);
48         /* Handle periodic timers */
49         EthernetIF_PeriodicHandle(&testnetif);
50     }
51 }
```

第5篇 驱动开发实验-RISC-V 篇

第1章 GPIO 驱动

1.1 GPIO 介绍

1.1.1 GPIO 基本介绍

芯片上的引脚一般分为 4 类：电源、时钟、控制与 I/O，I/O 口在使用模式上又分为 General Purpose Input Output（通用输入/输出），简称 GPIO，与功能复用 I/O（如 SPI/I2C/UART 等）。大多数 MCU 的引脚都不止一个功能，不同引脚内部结构不一样，拥有的功能也不一样。可以通过不同的配置，切换引脚的实际功能。通用 I/O 口主要特性如下：

（1）可编程控制中断：中断触发模式可配置，一般有以下图 5-1-1 所示 5 种中断触发模式：

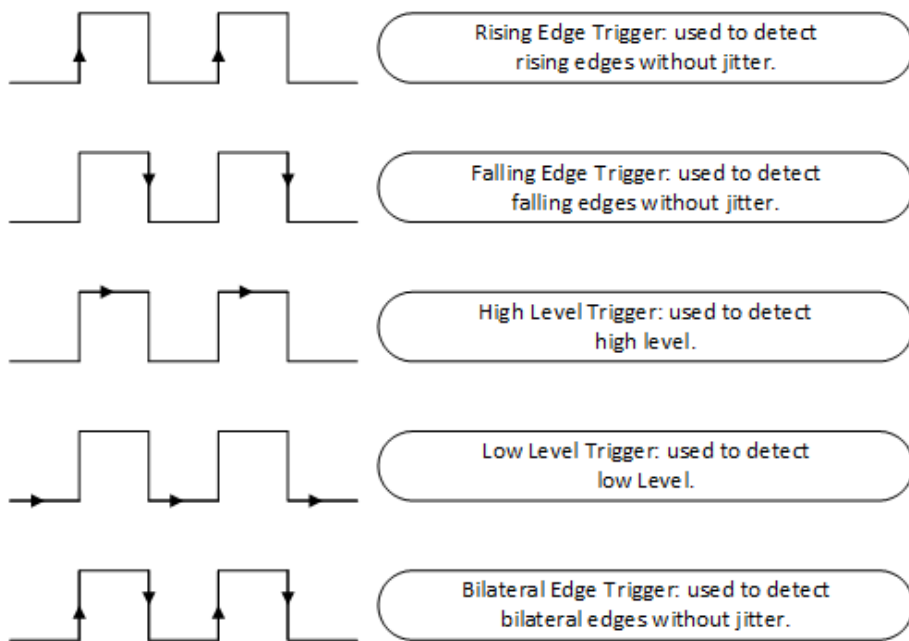


图 5-1-186 中断触发模式

（2）输入输出模式可控制：输出模式一般包括：推挽、开漏、上拉、下拉。引脚为输出模式时，可以通过配置引脚输出的电平状态为高电平或低电平来控制连接的外围设备。输入模式一般包括：浮空、上拉、下拉、模拟。引脚为输入模式时，可以读取引脚的电平状态，即高电平或低电平。

RISC-V 砒璜硬件终端搭载的 K210 芯片上有高速 GPIO 和通用 GPIO。高速 GPIO 为 GPIOHS，共 32 个。

1.1.2 中断说明

高速 GPIOHS 中断具有如下特点：

- (1) 可配置输入输出信号；
- (2) 每个 IO 具有独立中断源；
- (3) 中断支持边沿触发和电平触发；
- (4) 每个 IO 可以分配到 FPIOA 上 48 个管脚之一；
- (5) 可配置上下拉，或者高阻。

通用 GPIO 共 8 个，具有如下特点：

- (1) 8 个 IO 使用一个中断源；
- (2) 可配置输入输出信号；
- (3) 可配置触发 IO 总中断，边沿触发和电平触发；
- (4) 每个 IO 可以分配到 FPIOA 上 48 个管脚之一；
- (5) 可配置上下拉，或者高阻。

1.1.3 寄存器说明

以下将对 GPIO 使用的寄存器进行介绍。

一. GPIO_SWPORTA_DR:

位	位名	读写	功能
b31-b0	GPIOSWPORTADR	R/W	存储端口 A 输出值

端口 A 设置为输出模式且对应控制位为软件模式时，写入该寄存器的值将从端口 A 的 I/O 口输出。

二. GPIO_SWPORTA_DDR:

位	位名	读写	功能
b31-b0	GPIOSWPORTADDR	R/W	端口 A 数据方向控制

写入该寄存器的值独立控制端口 A 输入输出方向，0x0 代表输入，0x1 代表输出，可通过 GPIODFLTDIR_A 参数配置默认输入输出方向。

三. GPIO_INTEN:

位	位名	读写	功能
b31-b0	GPIO_INTEN	R/W	端口 A 中断配置

默认情况下，中断的生成被禁用。每当 1 被写入该寄存器的一位时，它将端口 a 上的相应位配置为中断，否则端口 A 作为正常的 GPIO 信号进行操作。

四. GPIO_INTMASK:

位	位名	读写	功能
b31-b0	GPIO_INTMASK	R/W	端口 A 中断屏蔽配置

控制端口 A 上的中断是否可以通过不屏蔽为中断控制器创建中断。默认情况下，所有中断位都不屏蔽。每当 1 被写入该寄存器中的一位时，它就屏蔽了该信号的中断生成能力，否则允许中断通过。

五. GPIO_INTTYPE_LEVEL:

位	位名	读写	功能
b31-b0	GPIOINTTYPELEVEL	R/W	端口 A 中断类型配置

控制端口 A 上可能发生的中断类型。每当 0 被写入该寄存器的一位时，它会将中断类型配置为电平灵敏，否则为边沿灵敏。

六. GPIO_INT_POLARITY:

位	位名	读写	功能
b31-b0	GPIOINTPOLARITY	R/W	端口 A 中断极性配置

控制端口 A 输入端可能出现的边沿或电平灵敏度的极性。每当 0 被写入该寄存器的一位时，它将中断类型配置为下降沿或低电平灵敏，否则它是上升沿或高电平灵敏。

七. GPIO_INTSTATUS:

位	位名	读写	功能
b31-b0	GPIO_INTSTATUS	R	端口 A 中断状态配置

控制端口 A 的中断状态，0x0 代表 inactive，0x1 代表 active。

八. GPIO_RAW_INTSTATUS:

位	位名	读写	功能
---	----	----	----

b31-b0	GPIORAWINTSTATUS	R	端口 A 状态初始中断
--------	------------------	---	-------------

配置端口 A 状态的原始中断，0x0 代表 inactive，0x1 代表 active。

九. GPIO_DEBOUNCE:

位	位名	读写	功能
b31-b0	GPIO_DEBOUNCE	R/W	去抖动使能位

控制作为中断源的外部信号是否需要去抖动以消除任何伪故障。将 1 写入该寄存器中的位使能去抖动电路。

十. GPIO_PORTA_EOI:

位	位名	读写	功能
b31-b0	GPIOPORTAEOI	W	端口 A 边沿中断清空

控制从端口 A 清除边沿中断。当 1 被写入该寄存器的相应位时，中断被清除。当端口 A 没有为中断配置时，所有中断都被清除。

十一. GPIO_EXT_PORTA:

位	位名	读写	功能
b31-b0	GPIOEXTPORTA	R	外部端口 A 信号

此寄存器反映外部端口 A 上的信号值。

十二. GPIO_LS_SYNC:

位	位名	读写	功能
b31-b1	RSVDGPIO_LS_SYNC	R	保留位
b0	GPIOLSSYNC	R/W	中断同步控制位

将 1 写入此寄存器会导致所有级别敏感的中断同步到 pclk intr。

十三. GPIO_ID_CODE:

位	位名	读写	功能
b31-b0	GPIOIDCODE	R	用户代码

这是用户指定的代码，系统可以读取。它可以用于芯片识别等。

十四. GPIO_INT_BOTHEDGE:

位	位名	读写	功能
---	----	----	----

b31-b0	GPIOINTBOTHEDGE	R/W	边沿中断使能
--------	-----------------	-----	--------

控制端口 A 上可能发生的边沿中断。每当特定位被编程为 1 时，它能够在与端口 A 上的该位对应的外部输入信号的上升沿和下降沿上产生中断。当该寄存器的对应位设置为 1 时，不考虑在寄存器 GPIO_INTTYPE_LEVEL 和 GPIO_INT_POLARITY 中为该特定位编程的值。

十五. GPIO_VER_ID_CODE:

位	位名	读写	功能
b31-b0	GPIOVERID_CODE	R	版本号

版本中每个数字的 ASCII 值，后跟*。例如，323132_2A 表示版本 2.12*。

十六. GPIO_CONFIG_REG2:

位	位名	读写	功能
b31-b20	RSVDGPIOCONFIG_REG2	R	保留位
b19-b15	ENCODEDIDPWIDTH_D	R	寄存器值源自 GPIOP WIDTHD 配置参数
b14-b10	ENCODEDIDPWIDTH_C	R	寄存器值源自 GPIOP WIDTHC 配置参数
b9-b5	ENCODEDIDPWIDTH_B	R	寄存器值源自 GPIOP WIDTHB 配置参数
b4-b0	ENCODEDIDPWIDTH_A	R	寄存器值源自 GPIOP WIDTHA 配置参数

保留位读取为 0，其他位的值根据相应配置参数决定，0x0 代表 8，0x1 代表 16，0x2 代表 32，0x3 代表 Reserved。

十七. GPIO_CONFIG_REG1:

位	位名	读写	功能
b31-b22	RSVDGPIOCONFIG_REG1	R	保留位

b21	INTERRUPTBOTHEDGE _TYPE	R	寄存器值源自 GPIOIN TBOTH_EDGE 配置 参数
b20-b16	ENCODEDIDWIDTH	R	寄存器值源自 GPIOI DWIDTH 配置参数
b15	GPIO_ID	R	寄存器值源自 GPIOI D 配置参数
b14	ADDENCODEDPARAMS	R	寄存器值源自 GPIOA DDENCODED_PARA MS 配置参数
b13	DEBOUNCE	R	寄存器值源自 GPIO_ DEBOUNCE 配置参数
b12	PORTA_INTR	R	寄存器值源自 GPIOP ORTAINTR 配置参数
b11	HW_PORTD	R	寄存器值源自 GPIOH WPORTD 配置参数
b10	HW_PORTC	R	寄存器值源自 GPIOH WPORTC 配置参数
b9	HW_PORTB	R	寄存器值源自 GPIOH WPORTB 配置参数
b8	HW_PORTA	R	寄存器值源自 GPIOH WPORTA 配置参数
b7	PORTDSINGLECTL	R	寄存器值源自 GPIOP ORTDSINGLE_CTL 配置参数

b6	PORTCSINGLECTL	R	寄存器值源自 GPIOPORTCSINGLE_CTL 配置参数
b5	PORTBSINGLECTL	R	寄存器值源自 GPIOPORTBSINGLE_CTL 配置参数
b4	PORTASINGLECTL	R	寄存器值源自 GPIOPORTASINGLE_CTL 配置参数
b3-b2	NUM_PORTS	R	寄存器值源自 GPIONUM_PORTS 配置参数
b1-b0	APBDATAWIDTH	R	寄存器值源自 GPIOAPBDATAWIDTH 配置参数

除以上 GPIO 的寄存器外，GPIOHS 同样包含一系列寄存器，每个寄存器均为 32 位，每一位代表相应的 IO 号，共 32 个 GPIOHS。寄存器种类如下：

一. Input_val:

位	位名	读写	功能
b31-b0	input_val	R/W	输入值

二. Input_en:

位	位名	读写	功能
b31-b0	input_en	R/W	输入使能

三. Output_en:

位	位名	读写	功能
b31-b0	output_en	R/W	输出使能

四. Output_val:

位	位名	读写	功能
---	----	----	----

b31-b0	output_val	R/W	输出值
--------	------------	-----	-----

五. Pullup_en:

位	位名	读写	功能
b31-b0	pullup_en	R/W	内部上拉使能

六. Drive:

位	位名	读写	功能
b31-b0	drive	R/W	驱动力加强

七. Rise_ie:

位	位名	读写	功能
b31-b0	rise_ie	R/W	上升沿中断使能

八. Rise_ip:

位	位名	读写	功能
b31-b0	rise_ip	R/W	上升沿中断挂起

九. Fall_ie:

位	位名	读写	功能
b31-b0	fall_ie	R/W	下降沿中断使能

十. Fall_ip:

位	位名	读写	功能
b31-b0	fall_ip	R/W	下降沿中断挂起

十一. High_ie:

位	位名	读写	功能
b31-b0	high_ie	R/W	高电平中断使能

十二. High_ip:

位	位名	读写	功能
b31-b0	high_ip	R/W	高电平中断挂起

十三. Low_ie:

位	位名	读写	功能
---	----	----	----

b31-b0	low_ie	R/W	低电平中断使能
--------	--------	-----	---------

十四. Low_ip:

位	位名	读写	功能
b31-b0	low_ip	R/W	低电平中断挂起

十五. Iof_en:

位	位名	读写	功能
b31-b0	iof_en	R/W	IO 特殊功能使能

十六. Iof_sel:

位	位名	读写	功能
b31-b0	iof_sel	R/W	IO 特殊功能启用

十七. Output_xor:

位	位名	读写	功能
b31-b0	output_xor	R/W	输出翻转

1.2 GPIO 驱动程序

GPIO 设备通过 PinBus 挂接到 Bus 总线上, 其中 PinBus 的结构体在 bus_pin.h 文件中进行了定义。每个设备总线都关联一个 drv 和一个 dev, 其中, drv 结构体在 drv_pin.h 中进行了定义, dev 结构体在 dev_pin.h 中进行了定义。RISC-V 板的 GPIO 与 ARM 板的 GPIO 共用 bus_pin.h、drv_pin.h 和 dev_pin.h 文件, 读者可参考第 4 篇的第 1 章。另外, GPIO 驱动及设备的注册流程在 connect_gpio.c 中进行了实现, 主要包含 3 个步骤:

(1) 实现 GPIO 设备的读写方法, 其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植;

(2) 实现 GPIO 的驱动配置, 需结合 GPIO 特性及操作系统的接口定义进行相应的开发;

(3) 完成 GPIO 的驱动和设备到 GPIO 总线的挂接 (或称为绑定)。

步骤 (1) 中 GPIO 设备的读写方法实现见代码 5-1-1、代码 5-1-2、代码 5-1-3。结构体 PinDevDone 封装了 GPIO 设备的打开、关闭及读写函数, 其中打开、关闭的值为 NONE,

表示未启用该功能；写功能值为 PinWrite，读功能值为 PinRead，表明读写功能是打开的。

读函数的具体实现见代码 5-1-2，写函数的具体实现见代码 5-1-3。

代码 5-1-173 GPIO 设备的打开、关闭及读写

```
01 static const struct PinDevDone dev_done =
02 {
03     .open  = NONE,
04     .close = NONE,
05     .write = PinWrite,
06     .read  = PinRead,
07 };
```

代码 5-1-174 GPIO 设备的读函数

```
01 uint32 PinRead(void *dev, struct BusBlockReadParam *read_param)
02 {
03     NULL_PARAM_CHECK(dev);
04     NULL_PARAM_CHECK(read_param);
05     struct PinStat *pinstat = (struct PinStat *)read_param->buffer;
06
07     int pin_channel = GetPinChannel(pinstat->pin);
08     if (pin_channel == -1) {
09         SYS_ERR("pin %d not set mode", pinstat->pin);
10         return ERROR;
11     }
12
13     if (gpiohs_get_pin(pin_channel) == GPIO_PV_HIGH){
14         pinstat->val = GPIO_HIGH;
15         return GPIO_HIGH;
16     } else {
17         pinstat->val = GPIO_LOW;
18         return GPIO_LOW;
19     }
20 }
```

代码 5-1-175 GPIO 设备的写函数

```
01 uint32 PinWrite(void *dev, struct BusBlockWriteParam *write_param)
02 {
03     NULL_PARAM_CHECK(dev);
04     NULL_PARAM_CHECK(write_param);
05     struct PinStat *pinstat = (struct PinStat *)write_param->buffer;
06
```

```

07  int pin_channel = GetPinChannel(pinstat->pin);
08  if (pin_channel == -1) {
09      SYS_ERR("pin %d not set mode", pinstat->pin);
10      return ERROR;
11  }
12  gpiohs_set_pin(pin_channel, pinstat->val == GPIO_HIGH ? GPIO_PV_HIGH
: GPIO_PV_LOW);
13  }

```

步骤（2）中 GPIO 的驱动配置实现见代码 5-1-4。其中参数 drv 是驱动指针，configure_info 是 GPIO 驱动配置信息结构体指针。代码行 2 到 3 检查了输入函数的两个参数指针是否是空指针；代码行 8 到 16，判断了配置命令是否为 OPE_CFG，即配置操作模式，如果是，在代码行 10 到 13 按照配置参数对 GPIO 进行具体配置。

代码 5-1-176 GPIO 驱动的配置

```

01  static uint32 GpioDrvConfigure(void *drv, struct BusConfigureInfo *configure_i
nfo){
02      NULL_PARAM_CHECK(drv);
03      NULL_PARAM_CHECK(configure_info);
04
05      x_err_t ret = EOK;
06      struct PinParam *param;
07
08      switch (configure_info->configure_cmd)
09      {
10          case OPE_CFG:
11              param = (struct PinParam *)configure_info->private_data;
12              ret = PinConfigure(param);
13              break;
14          default:
15              break;
16      }
17
18      return ret;
19  }

```

步骤（3）中 GPIO 的驱动和设备到 GPIO 总线的挂接实现见代码 5-1-5。代码行 6 至 7 是将已分配的 GPIO 引脚的记录表进行清空；代码行 9 至 13 完成了 Bus 总线的初始化；代码行 15 至 22、24 至 28 分别完成了驱动的初始化及绑定；代码行 30 到 37、39 到 43 分别完成了设备的注册及绑定。

代码 5-1-177 GPIO 的初始化函数

```
01 int HwGpioInit(void){
02     x_err_t ret = EOK;
03
04     static struct PinBus pin;
05
06     memset(pin_alloc_table, -1, sizeof pin_alloc_table);
07     free_pin = GPIO_ALLOC_START;
08
09     ret = PinBusInit(&pin, PIN_BUS_NAME);
10     if (ret != EOK) {
11         KPrintf("gpio bus init error %d\n", ret);
12         return ERROR;
13     }
14
15     static struct PinDriver drv;
16     drv.configure = &GpioDrvConfigure;
17
18     ret = PinDriverInit(&drv, PIN_DRIVER_NAME, NONE);
19     if (ret != EOK) {
20         KPrintf("pin driver init error %d\n", ret);
21         return -ERROR;
22     }
23
24     ret = PinDriverAttachToBus(PIN_DRIVER_NAME, PIN_BUS_NAME);
25     if (ret != EOK) {
26         KPrintf("pin driver attach error %d\n", ret);
27         return -ERROR;
28     }
29
30     static struct PinHardwareDevice dev;
31     dev.dev_done = &dev_done;
32
33     ret = PinDeviceRegister(&dev, NONE, PIN_DEVICE_NAME);
34     if (ret != EOK) {
35         KPrintf("pin device register error %d\n", ret);
36         return -ERROR;
37     }
38
39     ret = PinDeviceAttachToBus(PIN_DEVICE_NAME, PIN_BUS_NAME);
40     if (ret != EOK) {
```

```

41         KPrintf("pin device register error %d\n", ret);
42         return -ERROR;
43     }
44
45     return ret;
46 }

```

1.3 GPIO 应用实例

GPIO 设备应用实现过程如代码 5-1-6 GPIO 设备应用所示。在测试函数 TestGpio 中，代码行 2~6 用于打开一个 GPIO 设备；代码行 8~15 声明了两个结构体变量，并对其参数进行赋值；代码行 17~21 通过 IO 控制函数对打开的 GPIO 设备进行配置（输出）；代码行 23~30 先对结构体变量参数进行重新赋值，然后再用 IO 控制函数对打开的 GPIO 设备进行配置（输入）；代码行 39~43 对 GPIO 设备进行读操作；代码行 51~55 对 GPIO 设备进行写操作。

代码 5-1-178 GPIO 设备应用

```

01 void TestGpio(void){
02     int pin_fd = PrivOpen(GPIO_DEV_DRIVER, O_RDWR);
03     if(pin_fd<0){
04         printf("open pin fd error:%d\n",pin_fd);
05         return;
06     }
07
08     struct PinParam parameter;
09     parameter.cmd = GPIO_CONFIG_MODE;
10     parameter.pin = BSP_LED_PIN;
11     parameter.mode = GPIO_CFG_OUTPUT;
12
13     struct PrivIoctlCfg ioctl_cfg;
14     ioctl_cfg.ioctl_driver_type = PIN_TYPE;
15     ioctl_cfg.args = (void *)&parameter;
16
17     if (0 != PrivIoctl(pin_fd, OPE_CFG, &ioctl_cfg)) {
18         printf("ioctl pin fd error %d\n", pin_fd);
19         PrivClose(pin_fd);
20         return;
21     }
22
23     parameter.pin = BSP_KEY_PIN;

```

```
24     parameter.mode = GPIO_CFG_INPUT;
25
26     if (0 != PrivIoctl(pin_fd, OPE_CFG, &iocctl_cfg)) {
27         printf("ioctl pin fd error %d\n", pin_fd);
28         PrivClose(pin_fd);
29         return;
30     }
31
32     struct PinStat pin_led;
33     struct PinStat pin_key;
34
35     pin_led.pin = BSP_LED_PIN;
36     pin_key.pin = BSP_KEY_PIN;
37
38     while(1){
39         if(0>PrivRead(pin_fd,&pin_key,NULL_PARAMETER)){
40             printf("read pin fd error %d\n", pin_fd);
41             PrivClose(pin_fd);
42             return;
43         }
44
45         if(pin_key.val){
46             pin_led.val = GPIO_HIGH;
47         }else{
48             pin_led.val = GPIO_LOW;
49         }
50
51         if(0>PrivWrite(pin_fd,&pin_led,NULL_PARAMETER)){
52             printf("write pin fd error %d\n", pin_fd);
53             PrivClose(pin_fd);
54             return;
55         }
56
57     }
58 }
```

第2章 TIMER 驱动

2.1 TIMER 介绍

2.1.1 TIMER 基本介绍

硬件定时器，可以用来定时触发任务或者处理任务，设定时间到了后可以触发中断（调用回调函数），精度比软件定时器高。硬件定时器一般有 2 种工作模式，定时器模式和计数器模式。不管是工作在哪一种模式，实质都是通过内部计数器模块对脉冲信号进行计数。下面是定时器的一些重要概念。

- （1）计数器模式：对外部输入引脚的外部脉冲信号计数；
- （2）定时器模式：对内部脉冲信号计数。定时器常用作定时时钟，以实现定时检测，定时响应、定时控制；
- （3）计数器：计数器可以递增计数或者递减计数。16 位计数器的最大计数值为 65535，32 位的最大值为 4294967295；
- （4）计数频率：定时器模式时，计数器单位时间内的计数次数，由于系统时钟频率是定值，所以可以根据计数器的计数值计算出定时时间， $\text{定时时间} = \text{计数值} / \text{计数频率}$ 。例如计数频率为 1MHz，计数器计数一次的时间则为 $1/1000000$ ，也就是每经过 1 微秒计数器加 1（或减 1），此时 16 位计数器的最大定时能力为 65535 微秒，即 65.535 毫秒。

RISC-V 矽琇硬件终端搭载的 K210 芯片上有 3 个 TIMER 模块，它们有如下特性：

- （1）32 位计数器宽度；
- （2）可配置的向上/向下时基计数器：增加/减少；
- （3）时钟独立可配；
- （4）每个中断的可配置极性；
- （5）单个或组合中断输出标志可配置；
- （6）每个定时器有读/写一致性寄存器；
- （7）定时器切换输出，每当定时器计数器重新加载时切换；
- （8）定时器切换输出的脉冲宽度调制(PWM)，0%到 100%占空比。

2.1.2 中断说明

TIMER 模块的中断具备如下特性：

- （1）每个中断可配置极性；
- （2）单个或组合中断输出标志可配置。

TIMER 可以触发报警，报警则会引发重新加载触发中断，如报警寄存器。TIMER1LOADCOUNT 的值等于当前定时器的值，则报警触发。为解决寄存器设置过晚，计数器值超过报警值的问题，当前定时器值高于（适用于向上定时器）或低于（适用于向下定时器）当前报警值也会触发报警。在这种情况下，使能报警功能会马上触发报警。

2.1.3 寄存器说明

以下将对 TIMER 使用的寄存器进行介绍。

一. TIMER1LOADCOUNT:

位	位名	读写	功能
b31-b0	Timer1LoadCount	R/W	计时器 1 初始计数值

要加载到计时器 1 中的值。这是开始计数的值，写入该寄存器的任何值都会加载到相关的计时器中。

二. TIMER1CURRENTVAL:

位	位名	读写	功能
b31-b0	Timer1CurrentValue	R/W	计时器 1 当前值

当 TIM_NEWMODE=0 时，只有 timer_1_clk 与 pclk 同步，才支持此寄存器。当使用独立时钟时读取此寄存器会导致未定义的值。当 TIM_NEWMODE=1 时，不适用任何限制。

三. TIMER1CONTROLREG:

位	位名	读写	功能
b31-b5	RSVD_Timer1ControlReg	R	保留字段
b4	RSVD_TIMER_ON100PWMMEN	R	保留字段
b3	TIMER_PWM	R/W	timer_1_toggle 输出的脉宽调制。
b2	TIMER_INTERRUPT_MASK	R/W	计时器 1 中断屏蔽
b1	TIMER_MODE	R/W	计时器 1 计时模式
b0	TIMER_ENABLE	R/W	计时器 1 使能

b31-b4 均为保留字段；b3 仅在启用 TIM_NEWMODE 时出现，0x1 代表 timer_1_toggle 输出脉宽调制开启，0x1 代表关闭；b2 为计时器 1 的中断屏蔽位；b1 选择计时器的模式，0x1 代表用户指定，0x0 代表自由运行；b0 为计时器使能标志。

四. TIMER1EOI:

位	位名	读写	功能
b31-b1	RSVD_Timer1EOI	R	保留字段
b0	Timer1EOI	R	中断清空

从该寄存器读取将返回零并清除计时器 1 的所有中断。

五. TIMER1INTSTAT:

位	位名	读写	功能
b31-b1	RSVD_Timer1IntStatus	R	保留字段
b0	Timer1IntStatus	R	计时器 1 中断状态

b0 位为 0x0 代表中断未开启，0x1 代表中断开启。

六. TIMER2LOADCOUNT:

位	位名	读写	功能
b31-b0	Timer2LoadCount	R/W	计时器 2 初始计数值

要加载到计时器 2 中的值。这是开始计数的值。写入该寄存器的任何值都会加载到相关的计时器中。

七. TIMER2CURRENTVAL:

位	位名	读写	功能
b31-b0	Timer2CurrentValue	R/W	计时器 2 当前值

当 TIM_NEWMODE=0 时，只有 timer_2_clk 与 pclk 同步，才支持此寄存器。当使用独立时钟时读取此寄存器会导致未定义的值。当 TIM_NEWMODE=1 时，不适用任何限制。

八. TIMER2CONTROLREG:

位	位名	读写	功能
b31-b5	RSVD_Timer2ControlReg	R	保留字段
b4	RSVD_TIMER	R	保留字段

	_ON100PWMEN		
b3	TIMER_PWM	R/W	timer_2_toggle 输出的脉宽调制。
b2	TIMER_INTERRUPT_M ASK	R/W	计时器 2 中断屏蔽
b1	TIMER_MODE	R/W	计时器 2 计时模式
b0	TIMER_ENABLE	R/W	计时器 2 使能

b31-b4 均为保留字段；b3 仅在启用 TIM_NEWMODE 时出现，0x1 代表 timer_2_toggle 输出脉宽调制开启，0x0 代表关闭；b2 为计时器 2 的中断屏蔽位；b1 选择计时器的模式，0x1 代表用户指定，0x0 代表自由运行；b0 为计时器使能标志。

九. TIMER2EOI:

位	位名	读写	功能
b31-b1	RSVD_Timer2EOI	R	保留字段
b0	Timer2EOI	R	中断清空

从该寄存器读取将返回零并清除计时器 2 的所有中断。

十. TIMER2INTSTAT:

位	位名	读写	功能
b31-b1	RSVD_Timer2IntStatus	R	保留字段
b0	Timer2IntStatus	R	计时器 2 中断状态

b0 位为 0x0 代表中断未开启，0x1 代表中断开启。

十一. TIMER3LOADCOUNT:

位	位名	读写	功能
b31-b0	Timer3LoadCount	R/W	计时器 3 初始计数值

要加载到计时器 3 中的值。这是开始计数的值，写入该寄存器的任何值都会加载到相关的计时器中。

十二. TIMER3CURRENTVAL:

位	位名	读写	功能
---	----	----	----

b31-b0	Timer3CurrentValue	R/W	计时器 3 当前值
--------	--------------------	-----	-----------

当 TIM_NEWMODE=0 时，只有 timer_3_clk 与 pclk 同步，才支持此寄存器。当使用独立时钟时读取此寄存器会导致未定义的值。当 TIM_NEWMODE=1 时，不适用任何限制。

十三. TIMER3CONTROLREG:

位	位名	读写	功能
b31-b5	RSVD_Timer3ControlReg	R	保留字段
b4	RSVD_TIMER_ON100PWMEN	R	保留字段
b3	TIMER_PWM	R/W	timer_3_toggle 输出的脉宽调制。
b2	TIMER_INTERRUPT_MASK	R/W	计时器 3 中断屏蔽
b1	TIMER_MODE	R/W	计时器 3 计时模式
b0	TIMER_ENABLE	R/W	计时器 3 使能

b31-b4 均为保留字段；b3 仅在启用 TIM_NEWMODE 时出现，0x1 代表 timer_3_toggle 输出脉宽调制开启，0x0 代表关闭；b2 为计时器 1 的中断屏蔽位；b1 选择计时器的模式，0x1 代表用户指定，0x0 代表自由运行；b0 为计时器使能标志。

十四. TIMER3EOI:

位	位名	读写	功能
b31-b1	RSVD_Timer3EOI	R	保留字段
b0	Timer3EOI	R	中断清空

从该寄存器读取将返回零并清除计时器 1 的所有中断。

十五. TIMER3INTSTAT:

位	位名	读写	功能
b31-b1	RSVD_Timer3IntStatus	R	保留字段
b0	Timer3IntStatus	R	计时器 3 中断状态

b0 位为 0x0 代表中断未开启，0x1 代表中断开启。

十六. TIMER4LOADCOUNT:

位	位名	读写	功能
b31-b0	Timer4LoadCount	R/W	计时器 4 初始计数值

要加载到计时器 4 中的值。这是开始计数的值。写入该寄存器的任何值都会加载到相关的计时器中。

十七. TIMER4CURRENTVAL:

位	位名	读写	功能
b31-b0	Timer4CurrentValue	R/W	计时器 4 当前值

当 TIM_NEWMODE=0 时, 只有 timer_4_clk 与 pclk 同步, 才支持此寄存器。当使用独立时钟时读取此寄存器会导致未定义的值。当 TIM_NEWMODE=1 时, 不适用任何限制。

十八. TIMER4CONTROLREG:

位	位名	读写	功能
b31-b5	RSVD_Timer4ControlReg	R	保留字段
b4	RSVD_TIMER_ON100PWMEN	R	保留字段
b3	TIMER_PWM	R/W	timer_1_toggle 输出的脉宽调制。
b2	TIMER_INTERRUPT_MASK	R/W	计时器 4 中断屏蔽
b1	TIMER_MODE	R/W	计时器 4 时模式
b0	TIMER_ENABLE	R/W	计时器 4 使能

b31-b4 均为保留字段; b3 仅在启用 TIM_NEWMODE 时出现, 0x1 代表 timer_4_toggle 输出脉宽调制开启, 0x0 代表关闭; b2 为计时器 1 的中断屏蔽位; b1 选择计时器的模式, 0x1 代表用户指定, 0x0 代表自由运行; b0 为计时器使能标志。

十九. TIMER4EOI:

位	位名	读写	功能
b31-b1	RSVD_Timer4EOI	R	保留字段

b0	Timer4EOI	R	中断清空
----	-----------	---	------

从该寄存器读取将返回零并清除计时器 4 的所有中断。

二十. TIMER4INTSTAT:

位	位名	读写	功能
b31-b1	RSVD_Timer4IntStatus	R	保留字段
b0	Timer4IntStatus	R	计时器 4 中断状态

b0 位为 0x0 代表中断未开启，0x1 代表中断开启。

二十一. TimersIntStatus:

位	位名	读写	功能
b31-b4	RSVD_TimersIntStatus	R	保留字段
b3-b0	TimersIntStatus	R	所有计时器中断状态

如果 b3-b0 中的某一位为 0x0，则相应的定时器中断未激活，相应的中断可能在 timer_intr 总线或 timer_intr_n 总线上，具体取决于选择的中断极性。类似地，如果这个寄存器的一个位是 0x1，则相应的中断位已经被设置在相关的中断总线中。在这两种情况下，报告的状态都是应用中断掩码后的状态。从该寄存器读取不会清除任何活动中断。

二十二. TimersEOI:

位	位名	读写	功能
b31-b4	RSVD_TimersEOI	R	保留字段
b3-b0	TIMERSEOI	R	所有计时器中断清空

读取此寄存器将返回所有零（0）并清除所有活动中断。

二十三. TimersRawIntStatus:

位	位名	读写	功能
b31-b4	RSVD_TIMERSRAWINT STAT	R	保留字段
b3-b0	TIMERSRAWINTSTAT	R	所有计时器中断状态

该寄存器包含组件中所有定时器的未屏蔽中断状态。

二十四. TIMERS_COMP_VERSION:

位	位名	读写	功能
b31-b0	TIMERSCOMPVERSION	R	当前版本号

该寄存器保存定时器组件版本信息。

二十五. TIMER1LOADCOUNT2:

位	位名	读写	功能
b31-b0	TIMER1LOADCOUNT2	R/W	toggle 输出的 HIGH 周期宽度

该寄存器保存当 timer_1_toggle 输出从 0 变为 1 时，加载到 Timer1 中的值。该值确定 timer_1_toggle 输出的 HIGH 周期的宽度。

二十六. TIMER2LOADCOUNT2:

位	位名	读写	功能
b31-b0	TIMER2LOADCOUNT2	R/W	toggle 输出的 HIGH 周期宽度

该寄存器保存当 timer_2_toggle 输出从 0 变为 1 时，加载到 Timer2 中的值。该值确定 timer_2_toggle 输出的 HIGH 周期的宽度。

二十七. TIMER3LOADCOUNT2:

位	位名	读写	功能
b31-b0	TIMER1LOADCOUNT2	R/W	toggle 输出的 HIGH 周期宽度

该寄存器保存当 timer_3_toggle 输出从 0 变为 1 时，加载到 Timer3 中的值。该值确定 timer_3_toggle 输出的 HIGH 周期的宽度。

二十八. TIMER4LOADCOUNT2:

位	位名	读写	功能
b31-b0	TIMER4LOADCOUNT2	R/W	toggle 输出的 HIGH 周期宽度

该寄存器保存当 timer_4_toggle 输出从 0 变为 1 时，加载到 Timer4 中的值。该值确定 timer_4_toggle 输出的 HIGH 周期的宽度。

2.2 TIMER 驱动程序

Timer 通过 HwtimerBus 挂接到 Bus 总线上，其中 HwTimerBus 结构体在 bus_hwtimer.h 文件中进行了定义。每个设备总线都关联一个 drv 和一个 dev，其中，drv 结构体也在 bus_hwtimer.h 中进行了定义，dev 结构体在 dev_hwtimer.h 中进行了定义。RISC-V 板的 Timer 与 ARM 板的 Timer 共用 bus_hwtimer.h 和 dev_hwtimer.h 文件，读者可参考第 4 篇的第 2 章。Timer 驱动及设备的注册流程在 connect_hwtimer.c 中进行了实现。此外，在 hardware_hwtimer.c 中还实现了 Timer 相关控制方法。下面对 RISC-V 架构中 Timer 的驱动及设备注册流程进行介绍。

connect_hwtimer.c 函数实现了 bus、dev 和 drv 的初始化过程，其具体实现见代码 5-2-1 Timer 初始化函数，其中代码行 3 到 7 为创建 bus 和 drv 结构体变量；代码行 9 到 13 实现了 bus 和 drv 的初始化和绑定；代码行 15 至 19 中完成了设备的注册和绑定。

代码 5-2-179 Timer 初始化函数

```

01 int HwTimerInit(void){
02     x_err_t ret = EOK;
03     static struct HwtimerBus hwtimer_bus;
04     memset(&hwtimer_bus, 0, sizeof(struct HwtimerBus));
05
06     static struct HwtimerDriver hwtimer_driver;
07     memset(&hwtimer_driver, 0, sizeof(struct HwtimerDriver));
08
09     ret = BoardHwtimerBusInit(&hwtimer_bus, &hwtimer_driver);
10     if (EOK != ret) {
11         KPrintf("board_hwtimer_Init error ret %u\n", ret);
12         return ERROR;
13     }
14
15     ret = BoardHwtimerDevBend();
16     if (EOK != ret) {
17         KPrintf("board_hwtimer_Init error ret %u\n", ret);
18         return ERROR;
19     }
20
21     return ret;
22 }

```

Timer 总线和驱动初始化和绑定操作的具体实现见代码 5-2-2。代码行 4 到 9、11 到 17 分别实现了总线和驱动的初始化；代码行 19 到 24 完成了驱动到总线的绑定。

代码 5-2-180 Timer 总线和驱动的初始化与绑定

```

01 static int BoardHwtimerBusInit(struct HwtimerBus *hwtimer_bus,
    struct HwtimerDriver *hwtimer_driver){
02     x_err_t ret = EOK;
03
04     /*Init the hwtimer bus */
05     ret = HwtimerBusInit(hwtimer_bus, HWTIMER_BUS_NAME_1);
06     if (EOK != ret) {
07         KPrintf("board_hwtimer_init HwtimerBusInit error %d\n", ret);
08         return ERROR;
09     }
10
11     /*Init the hwtimer driver*/
12     hwtimer_driver->configure = NONE;
13     ret = HwtimerDriverInit(hwtimer_driver, HWTIMER_DRIVER_NAME_1);
14     if (EOK != ret) {
15         KPrintf("board_hwtimer_init HwtimerDriverInit error %d\n", ret);
16         return ERROR;
17     }
18
19     /*Attach the hwtimer driver to the hwtimer bus*/
20     ret = HwtimerDriverAttachToBus(HWTIMER_DRIVER_NAME_1,
        HWTIMER_BUS_NAME_1);
21     if (EOK != ret) {
22         KPrintf("board_hwtimer_init USEDriverAttachToBus error %d\n", ret);
23         return ERROR;
24     }
25
26     return ret;
27 }

```

Timer 设备的注册与绑定的具体实现见代码 5-2-3。其中代码行 3 和 4 完成了 dev 结构体变量的声明；在代码行 6 完成了设备动作的赋值；代码行 8 至 12 实现了设备的注册；代码行 14 至 18 完成了设备到总线的绑定。

代码 5-2-181 Timer 设备的注册与绑定

```

01 static int BoardHwtimerDevBend(void){

```

```

02     x_err_t ret = EOK;
03     static struct HwtimerHardwareDevice hwtimer_device_0;
04     memset(&hwtimer_device_0, 0, sizeof(struct HwtimerHardwareDevice));
05
06     hwtimer_device_0.dev_done = &dev_done;
07
08     ret = HwtimerDeviceRegister(&hwtimer_device_0, NONE,
    HWTIMER_1_DEVICE_NAME_1);
09     if (EOK != ret) {
10         KPrintf("BoardHwtimerDevBend HwtimerDeviceRegister device %s
    error %d\n", HWTIMER_1_DEVICE_NAME_1, ret);
11         return ERROR;
12     }
13
14     ret = HwtimerDeviceAttachToBus(HWTIMER_1_DEVICE_NAME_1,
    HWTIMER_BUS_NAME_1);
15     if (EOK != ret) {
16         KPrintf("BoardHwtimerDevBend HwtimerDeviceAttachToBus device %s
    error %d\n", HWTIMER_1_DEVICE_NAME_1, ret);
17         return ERROR;
18     }
19
20     return ret;
21 }

```

2.3 TIMER 应用实例

Timer 设备应用实现过程如代码 5-2-4 所示。TestHwTimer 函数通过调用 timer 驱动来进行一次 led 灯的闪烁。代码行 5-9 和 11-15 分别是打开引脚和 timer 的描述符；代码行 17-21 进行 GPIO 引脚的设置；代码行 23-25 进行 IO 配置；代码行 33-39 进行 timer 驱动配置；代码行 55, 56 分别关闭引脚和 timer 的描述符。

代码 5-2-182 Timer 设备应用代码

```

01 void TestHwTimer(void)
02 {
03     x_ticks_t period = 100000;//uint:10ms
04
05     pin_fd = PrivOpen(HWTIMER_PIN_DEV_DRIVER, O_RDWR);
06     if(pin_fd<0){
07         printf("open pin fd error:%d\n",pin_fd);
08         return;

```

```
09     }
10
11     int timer_fd = PrivOpen(HWTIMER_TIMER_DEV_DRIVER, O_RDWR);
12     if(timer_fd<0){
13         printf("open timer fd error:%d\n",timer_fd);
14         return;
15     }
16
17     //config led pin in board
18     struct PinParam parameter;
19     parameter.cmd = GPIO_CONFIG_MODE;
20     parameter.pin = BSP_LED_PIN;
21     parameter.mode = GPIO_CFG_OUTPUT;
22
23     struct PrivIoctlCfg ioctl_cfg;
24     ioctl_cfg.ioctl_driver_type = PIN_TYPE;
25     ioctl_cfg.args = (void *)&parameter;
26
27     if (0 != PrivIoctl(pin_fd, OPE_CFG, &ioctl_cfg)) {
28         printf("ioctl pin fd error %d\n", pin_fd);
29         PrivClose(pin_fd);
30         return;
31     }
32
33     ioctl_cfg.ioctl_driver_type = TIMER_TYPE;
34     ioctl_cfg.args = (void *)&ledflip;
35     if (0 != PrivIoctl(timer_fd, OPE_INT, &ioctl_cfg)) {
36         printf("timer pin fd error %d\n", pin_fd);
37         PrivClose(pin_fd);
38         return;
39     }
40
41     ioctl_cfg.args = (void *)&period;
42     if (0 != PrivIoctl(timer_fd, OPE_CFG, &ioctl_cfg)) {
43         printf("timer pin fd error %d\n", pin_fd);
44         PrivClose(pin_fd);
45         return;
46     }
47
48     while(1){
49
```

```
50     }  
51  
52     // int32 timer_handle = KCreateTimer("LED on and off by 1s",&ledflip,&pin_f  
d,period,TIMER_TRIGGER_PERIODIC);  
53  
54     // KTimerStartRun(timer_handle);  
55     PrivClose(pin_fd);  
56     PrivClose(timer_fd);  
57 }
```

第3章 RTC 驱动

3.1 RTC 介绍

3.1.1 RTC 基本介绍

RTC (Real-Time Clock) 实时时钟可以提供精确的实时时间，它可以用于产生年、月、日、时、分、秒等信息。目前实时时钟芯片大多采用精度较高的晶体振荡器作为时钟源。有些时钟芯片为了在主电源掉电时还可以工作，会外加电池供电，使时间信息一直保持有效。

RISC-V 砂璜硬件终端搭载的 K210 芯片上有 RTC 模块，在设置时间后具备计时功能：

- (1) 可使用外部高频晶振进行计时；
- (2) 可配置外部晶振频率与分频；
- (3) 支持万年历配置，可配置的项目包含世纪、年、月、日、时、分、秒与星期；
- (4) 可按秒进行计时，并查询当前时刻；
- (5) 支持设置一组闹钟，可配置的项目包含年、月、日、时、分、秒，闹钟到达时触发中断；
- (6) 中断可配置，支持每日、每时、每分、每秒触发中断；
- (7) 可读出小于 1 秒的计数器计数值，最小刻度单位为外部晶振的单个周期；
- (8) 上电/复位后数据清零。

3.1.2 中断说明

RTC 模块的中断具备以下特性：

- (1) 支持设置一组闹钟，可配置的项目包含年、月、日、时、分、秒，闹钟到达时触发中断；

(2) 中断可配置，支持每日、每时、每分、每秒触发中断。

3.1.3 寄存器说明

以下将对 RTC 使用的寄存器进行介绍。

一. Data Register:

位	位名	读写	功能
b191-b160	year	R/W	年计数
b159-b128	month	R/W	月计数
b127-b96	resv1	R/W	保留位
b95-b64	day	R/W	日计数
b63-b32	resv0	R/W	保留位
b31-b0	week	R/W	周计数

二. Time Register:

位	位名	读写	功能
b191-b160	resv2	R/W	保留位
b159-b128	hour	R/W	时计数
b127-b96	resv1	R/W	保留位
b95-b64	minute	R/W	分计数
b63-b32	second	R/W	秒计数
b31-b0	resv0	R/W	保留位

三. Alarm_date Register:

位	位名	读写	功能
b191-b160	year	R/W	年时间戳
b159-b128	month	R/W	月时间戳
b127-b96	resv1	R/W	保留位
b95-b64	day	R/W	日时间戳
b63-b32	resv0	R/W	保留位
b31-b0	week	R/W	周时间戳

四. Alarm_time Register:

位	位名	读写	功能
b191-b160	resv2	R/W	保留位
b159-b128	hour	R/W	时时间戳
b127-b96	resv1	R/W	保留位
b95-b64	minute	R/W	分时间戳
b63-b32	second	R/W	秒时间戳
b31-b0	resv0	R/W	保留位

五. Initial_count Register:

位	位名	读写	功能
b31-b0	count	R/W	计数值

六. Current_count Register:

位	位名	读写	功能
b31-b0	count	R/W	计数值

七. Interrupt_ctrl Register:

位	位名	读写	功能
b159-b128	alarm_compare_mask	R/W	Alarm compare 中断屏蔽
b127-b96	resv	R/W	保留位
b95-b64	tick_int_mode	R/W	tick 中断使能
b63-b32	alarm_enable	R/W	定时使能
b31-b0	tick_enable	R/W	Tick 使能

八. Register_ctrl Register:

位	位名	读写	功能
b255-b224	Rev1	R/W	保留位
b223-b192	interrupt_register_mask	R/W	RTC 中断屏蔽

b191-b160	initial_count_mask	R/W	RTC 计数器初始值 屏蔽
b159-b128	alarm_mask	R/W	RTC Alarm 屏蔽
b127-b96	TimerMask	R/W	RTC 时间屏蔽
b95-b64	Resv0	R/W	保留位
b63-b32	write_enable	R/W	写使能位
b31-b0	read_enable	R/W	读使能位

九. Reserved0 Register:

位	位名	读写	功能
b31-b0	resv	R/W	保留位

十. Reserverd1 Register:

位	位名	读写	功能
b31-b0	resv	R/W	保留位

十一. Extend Register:

位	位名	读写	功能
b95-b64	resv	R/W	保留位
b63-b32	leap_year	R/W	闰年判断
b31-b0	century	R/W	纪计数

3.2 RTC 驱动程序

RTC 通过 RtcBus 挂接到 Bus 总线上，其中 RtcBus 的结构体在 bus_rtc.h 文件中进行了定义。每个设备总线都关联一个 drv 和一个 dev，其中，drv 结构体也在 bus_rtc.h 中进行了定义，dev 结构体在 dev_rtc.h 中进行了定义。RISC-V 板的 RTC 与 ARM 板的 RTC 共用 bus_rtc.h 和 dev_rtc.h 文件，读者可参考第 4 篇的第 3 章。RTC 驱动及设备的注册流程在 connect_rtc.c 中进行了实现。下面对 RISC-V 架构中 RTC 的驱动及设备注册流程进行介绍。

RTC 的硬件初始化函数见代码 5-3-1。代码行 4 到 8 是对总线和驱动的声明与置零；代码行 10 对 rtc 驱动的配置进行赋值，主要过程在 RtcConfigure(*drv, *configure_info)中实现；代码行 12 到 16 是对 Bus 和 Drv 结构体变量的初始化，主要过程在 BoardRtcBusInit(&rtc_b

us, &rtc_driver)中完成；代码行 18 到 21 进行了设备注册并将设备绑定到总线上，其主要过程在 BoardRtcDevBend()函数中完成；代码行 23 调用 rtc_init()完成对片上 RTC 的硬件初始化。

代码 5-3-183 RTC 的初始化函数

```

01  int HwRtcInit(void){
02      x_err_t ret = EOK;
03
04      static struct RtcBus rtc_bus;
05      memset(&rtc_bus, 0, sizeof(struct RtcBus));
06
07      static struct RtcDriver rtc_driver;
08      memset(&rtc_driver, 0, sizeof(struct RtcDriver));
09
10      rtc_driver.configure = &(RtcConfigure);
11
12      ret = BoardRtcBusInit(&rtc_bus, &rtc_driver);
13      if (EOK != ret) {
14          KPrintf("HwRtcInit error ret %u\n", ret);
15          return ERROR;
16      }
17
18      ret = BoardRtcDevBend();
19      if (EOK != ret) {
20          KPrintf("HwRtcInit error ret %u\n", ret);
21      }
22
23      rtc_init();
24
25      return ret;
26  }

```

RTC 的总线和驱动初始化函数见代码 5-3-2。其中代码行 4 到 9 是对 RTC 总线初始化；代码行 11 到 16 是对 RTC 驱动初始化；代码行 18 到 23 将 RTC 驱动绑定在 BUS 总线上。

代码 5-3-184 RTC 的总线及设备初始化

```

01  static int BoardRtcBusInit(struct RtcBus *rtc_bus, struct RtcDriver *rtc_driver){
02      x_err_t ret = EOK;
03
04      /*Init the rtc bus */

```

```

05     ret = RtcBusInit(rtc_bus, RTC_BUS_NAME);
06     if (EOK != ret) {
07         KPrintf("HwRtcInit RtcBusInit error %d\n", ret);
08         return ERROR;
09     }
10
11     /*Init the rtc driver*/
12     ret = RtcDriverInit(rtc_driver, RTC_DRV_NAME);
13     if (EOK != ret) {
14         KPrintf("HwRtcInit RtcDriverInit error %d\n", ret);
15         return ERROR;
16     }
17
18     /*Attach the rtc driver to the rtc bus*/
19     ret = RtcDriverAttachToBus(RTC_DRV_NAME, RTC_BUS_NAME);
20     if (EOK != ret) {
21         KPrintf("HwRtcInit RtcDriverAttachToBus error %d\n", ret);
22         return ERROR;
23     }
24
25     return ret;
26 }

```

RTC 设备的注册与绑定的具体实现见代码 5-3-3。其中代码行 4 到 5 是对 RTC 设备的声明和初始化；代码行 7 完成了设备动作的赋值；代码行 9 到 13 完成设备的注册；代码行 15 到 19 将注册的设备绑定至设备总线。

代码 5-3-185 RTC 设备绑定函数

```

01 static int BoardRtcDevBend(void){
02     x_err_t ret = EOK;
03
04     static struct RtcHardwareDevice rtc_device;
05     memset(&rtc_device, 0, sizeof(struct RtcHardwareDevice));
06
07     rtc_device.dev_done = &(dev_done);
08
09     ret = RtcDeviceRegister(&rtc_device, NONE, RTC_DEVICE_NAME);
10     if (EOK != ret) {
11         KPrintf("RtcDeviceInit device %s error %d\n", RTC_DEVICE_NAME,
ret);
12         return ERROR;

```

```

13     }
14
15     ret = RtcDeviceAttachToBus(RTC_DEVICE_NAME, RTC_BUS_NAME);
16     if (EOK != ret) {
17         KPrintf("RtcDeviceAttachToBus device %s error %d\n", RTC_DEVICE_
NAME, ret);
18         return ERROR;
19     }
20
21     return ret;
22 }

```

RTC 驱动的配置具体实现见代码 5-3-4。其中代码行 4 到 8 为对局部变量的声明，并传入了驱动和配置信息指针；代码行 10 到 55 根据配置信息中的配置命令类型进行了区分：当命令为获取时钟时，对应代码行 12 到 31，通过 `rtc_timer_get(&year, &month, &day, &hour, &minute, &second)` 获取时钟，并在之后进行了数值转换。当命令为设定时钟时，对应代码行 32 到 54。代码行 38 到 41 通过原子操作获取本地时钟。代码行 43 到 48 为对系统时钟和 RTC 的置零。代码行 50 到 53 是对 RTC 数值的具体设置并反馈设置结果。

代码 5-3-186 RTC 硬件配置函数

```

01 static uint32 RtcConfigure(void *drv, struct BusConfigureInfo *configure_info){
02     NULL_PARAM_CHECK(drv);
03
04     struct RtcDriver *rtc_drv = (struct RtcDriver *)drv;
05     struct RtcDrvConfigureParam *drv_param =
        (struct RtcDrvConfigureParam *)configure_info->private_data;
06
07     int cmd = drv_param->rtc_operation_cmd;
08     time_t *time = drv_param->time;
09
10     switch (cmd)
11     {
12         case OPER_RTC_GET_TIME:
13         {
14             struct tm ct;
15             int year,month,day,hour,minute,second;
16             memset(&ct,0,sizeof(struct tm));
17
18             rtc_timer_get(&year, &month, &day, &hour, &minute, &second);

```

```

19
20         ct.tm_year = year - 1900;
21         ct.tm_mon = month - 1;
22         ct.tm_mday = day;
23         ct.tm_wday = GetWeekDay(year, month, day);
24
25         ct.tm_hour = hour;
26         ct.tm_min = minute;
27         ct.tm_sec = second;
28
29         *time = mktime(&ct);
30     }
31     break;
32 case OPER_RTC_SET_TIME:
33     {
34         struct tm *ct;
35         struct tm tm_new;
36         x_base lock;
37
38         lock = CriticalAreaLock();
39         ct = localtime(time);
40         memcpy(&tm_new, ct, sizeof(struct tm));
41         CriticalAreaUnLock(lock);
42
43         sysctl_reset(SYSCTL_RESET_RTC);
44         sysctl_clock_enable(SYSCTL_CLOCK_RTC);
45         rtc_protect_set(0);
46         rtc_timer_set_clock_frequency(SysctlClockGetFreq
                                     (SYSCTL_CLOCK_IN0));
47         rtc_timer_set_clock_count_value(1);
48         rtc_timer_set_mode(RTC_TIMER_RUNNING);
49
50         if (rtc_timer_set(tm_new.tm_year+1900,
                           tm_new.tm_mon+1,tm_new.tm_mday,
                           tm_new.tm_hour,tm_new.tm_min,tm_new.tm
                           _sec)==-1)
51             return ERROR;
52     }
53     break;
54 }
55 return EOK;

```

56 }

3.3 RTC 应用实例

RTC 设备应用实现过程如代码 5-3-5 所示。其中代码行 2~6 用于打开一个 RTC 设备驱动；代码行 9~18 用于设置系统时间，因为芯片里时钟初始的时间需要重新设置；代码行 20~24 用于循环读取时间，每隔 5 秒读一次。

代码 5-3-187 RTC 设备应用

```

01 void TestRTC(int argc,char *argv[]){
02     int rtc_fd = PrivOpen(RTC_DEV_DRIVER, O_RDWR);
03     if(rtc_fd<0){
04         printf("open rtc fd error:%d\n",rtc_fd);
05         return;
06     }
07
08     if(argc>1){
09         int times = atoi(argv[1]);
10         printf("Time will be printf %d times\n",times);
11         struct RtcDrvConfigureParam rtc_para;
12         rtc_para.rtc_operation_cmd = OPER_RTC_SET_TIME;
13         *(rtc_para.time) = 0;
14
15         struct PrivIoctlCfg ioctl_cfg;
16         ioctl_cfg.ioctl_driver_type = RTC_TYPE;
17         ioctl_cfg.args = (void *)&rtc_para;
18         PrivIoctl(rtc_fd,0,&ioctl_cfg);
19
20         rtc_para.rtc_operation_cmd = OPER_RTC_GET_TIME;
21         for(size_t i=0;i<times;i++){
22             PrivIoctl(rtc_fd,0,&ioctl_cfg);
23             printf("The time now is %d\n",*(rtc_para.time));
24             PrivTaskDelay(5000);
25         }
26     }
27
28     PrivClose(rtc_fd);
29     return;
30 }

```

第4章 WATCHDOG 驱动

4.1 WATCHDOG 介绍

4.1.1 WATCHDOG 基本介绍

硬件看门狗（watchdog timer）是一个定时器，其定时输出连接到电路的复位端。当看门狗启动后，计数器开始自动计数，在计数器溢出前如果没有被复位，计数器溢出就会对 CPU 产生一个复位信号使系统重启（俗称“被狗咬”）。系统正常运行时，需要在看门狗允许的时间间隔内对看门狗计数器清零（俗称“喂狗”），不让复位信号产生。RISC-V 砂璜硬件终端搭载的 K210 芯片上，WDT 是 APB 的一种从外设，并且也是“同步化硬件组件设计”的组成部分。具有两个 WDT，分别为 WDT0、WDT1，看门狗定时器主要包含模块有：

- (1) 一个 APB 从接口；
- (2) 一个当前计数器同步的寄存器模块；
- (3) 一个随着计数器递减的中断/系统重置模块和逻辑控制电路；
- (4) 一个同步时钟域来为异步时钟同步做支持。

看门狗定时器支持如下设置：

- (1) APB 总线宽度可配置为 8、16 和 32 位；
- (2) 时钟计数器从某一个设定的值递减到 0 来指示时间的计时终止；
- (3) 可选择的外部时钟使能信号，用于控制计数器的计数速率；
- (4) 一个时钟超时 WDT 可以执行以下任务：
 - 1) 产生一个系统复位信号；
 - 2) 首先产生一个中断，即使该位是否已经被中断服务清除，其次它会产生一个系统复位信号。
- (5) 占空比可编程调节；
- (6) 可编程和硬件设定计数器起始值；
- (7) 计数器重新计时保护；
- (8) 暂停模式，仅当使能外部暂停信号时；
- (9) WDT 偶然禁用保护；

(10) 测试模式，用来进行计数器功能测试（递减操作）；

(11) 外部异步时钟支持。当该项功能启用时，将会产生时钟中断和系统重置信号，即使 APB 总线时钟关闭的情况下。

4.1.2 中断说明

WDT 在计数器计数下溢或刷新错误时，可以选择产生中断请求或者复位请求。芯片内部有两个看门狗模块 WDT0、WDT1，均可复位除看门狗模块之外的所有模块。

4.1.3 寄存器说明

WDT 需要复位和时钟寄存器中的模块时钟使能寄存器、模块复位控制寄存器和时钟分频设置寄存器 6 来控制，这三个寄存器的名称和地址如下：

名称	描述	地址
SYS_CLK_EN	模块时钟使能寄存器	0x5044002C
SYS_RST_CTRL	模块复位控制寄存器	0x50440034
SYS_CLK_TH6	时钟分频设置寄存器 6	0x50440050

其中，模块时钟使能寄存器中的第 24 位和第 25 位，分别用于对 WDT0 时钟和 WDT1 时钟使能，具体如下表所示：

位	参数名	描述	复位值	访问类型
b24	wdt0_clk_en	WDT0 时钟使能	0	R / W
b25	wdt1_clk_en	WDT1 时钟使能	0	R / W

模块复位控制寄存器中的第 24 位和第 25 位，分别用于对 WDT0 时钟和 WDT1 时钟进行复位控制，具体如下表所示：

位	参数名	描述	复位值	访问类型
b24	wdt0_reset	WDT0 复位控制，置 1 表示复位，置 0 表示复位结束	0	R / W
b25	wdt1_reset	WDT1 复位控制，置 1 表示复位，置 0 表示复位结束	0	R / W

时钟分频设置寄存器 6 中的第 0~7 位控制 wdt0 以 IN0 输出时钟为基准时钟基础上分频，第 8~15 位控制 wdt1 以 IN0 输出时钟为基准时钟基础上分频，具体如下表所示：

位	参数名	描述	复位值	访问类型
b7-b0	wdt0_clk_threshold	以 IN0 输出时钟为基准时钟基础上分频 0: clk/2 1: clk/4 2: clk/6	0	R / W

		3: clk/8 ... 255: clk/512		
b15-b8	wdt1_clk_threshold	以 IN0 输出时钟为基准时钟基础上分频 0: clk/2 1: clk/4 2: clk/6 3: clk/8 ... 255: clk/512	0	R / W

4.2 WATCHDOG 驱动程序

WATCHDOG 通过 WdtBus 挂接到 Bus 总线上，其中 WdtBus 结构体在 bus_wdt.h 文件中进行了定义。每个设备总线都关联一个 drv 和一个 dev，其中，drv 结构体也在 bus_wdt.h 中进行了定义，dev 结构体在 dev_wdt.h 中进行了定义。RISC-V 板的 WATCHDOG 与 ARM 板的 WATCHDOG 共用 bus_wdt.h 和 dev_wdt.h 文件，读者可参考第 4 篇的第 4 章。WATCHDOG 驱动及设备的注册流程在 connect_wdt.c 中进行了实现，主要包含 3 个步骤：

- （1）完成 WATCHDOG 的驱动和设备到 WATCHDOG 总线的挂接（或称为绑定）；
- （2）实现 WATCHDOG 设备的打开，其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植；
- （3）实现 WATCHDOG 的驱动配置。

步骤（1）的具体实现见代码 5-4-1，WATCHDOG 的初始化与设备驱动绑定的过程根据所选择的 WATCHDOG 实现外设对象的不同，在实现步骤上有所不同。当选择 WDT0 时，即对应代码行 6 到 49；当选择 WDT1 时，对应代码行 51 到 96。

在具体实现上以 WDT0 为例，代码行 8 到 14 完成了对 WATCHDOG 总线的声明和初始化；代码行 16 到 25 完成了 WATCHDOG 驱动的声明和初始化；代码行 27 到 31 将驱动绑定在 WATCHDOG 总线上；代码行 33 到 47 完成了 WATCHDOG 具体设备的声明、初始化、注册和绑定。

代码 5-4-188 WATCHDOG 的初始化及驱动和设备的绑定

```

01 int HwWdtInit(void){
02     wdt_device_number_t id;
03

```

```
04     x_err_t ret = EOK;
05
06 #ifdef BSP_USING_WDT0
07     {
08         static struct WdtBus wdt0;
09
10         ret = WdtBusInit(&wdt0, WDT_BUS_NAME_0);
11         if (ret != EOK) {
12             KPrintf("Watchdog bus init error %d\n", ret);
13             return ERROR;
14         }
15
16         static struct WdtDriver drv0;
17         drv0.configure = WdtConfigure;
18         id = WDT_DEVICE_0;
19         drv0.private_data = &id;
20
21         ret = WdtDriverInit(&drv0, WDT_DRIVER_NAME_0);
22         if (ret != EOK) {
23             KPrintf("Watchdog driver init error %d\n", ret);
24             return ERROR;
25         }
26
27         ret = WdtDriverAttachToBus(WDT_DRIVER_NAME_0,
                                   WDT_BUS_NAME_0);
28         if (ret != EOK) {
29             KPrintf("Watchdog driver attach error %d\n", ret);
30             return ERROR;
31         }
32
33         static struct WdtHardwareDevice dev0;
34         dev0.dev_done = &dev_done;
35         dev0.private_data = &id;
36
37         ret = WdtDeviceRegister(&dev0, WDT_0_DEVICE_NAME_0);
38         if (ret != EOK) {
39             KPrintf("Watchdog device register error %d\n", ret);
40             return ERROR;
41         }
42
43         ret = WdtDeviceAttachToBus(WDT_0_DEVICE_NAME_0,
```

```

                                WDT_BUS_NAME_0);

44         if (ret != EOK) {
45             KPrintf("Watchdog device register error %d\n", ret);
46             return ERROR;
47         }
48     }
49 #endif
50
51 #ifdef BSP_USING_WDT1
52     {
53         static struct WdtBus wdt1;
54
55         ret = WdtBusInit(&wdt1, WDT_BUS_NAME_1);
56         if (ret != EOK) {
57             KPrintf("Watchdog bus init error %d\n", ret);
58             return ERROR;
59         }
60
61         static struct WdtDriver drv1;
62         drv1.configure = WdtConfigure;
63         id = WDT_DEVICE_1;
64         drv1.private_data = &id;
65
66         ret = WdtDriverInit(&drv1, WDT_DRIVER_NAME_1);
67         if (ret != EOK) {
68             KPrintf("Watchdog driver init error %d\n", ret);
69             return ERROR;
70         }
71
72         ret = WdtDriverAttachToBus(WDT_DRIVER_NAME_1,
WDT_BUS_NAME_1);
73         if (ret != EOK) {
74             KPrintf("Watchdog driver attach error %d\n", ret);
75             return ERROR;
76         }
77
78         static struct WdtHardwareDevice dev1;
79         dev1.dev_done = &dev_done;
80         dev1.private_data = &id;
81
82         ret = WdtDeviceRegister(&dev1, WDT_1_DEVICE_NAME_1);

```

```

83         if (ret != EOK) {
84             KPrintf("Watchdog device register error %d\n", ret);
85             return ERROR;
86         }
87
88         ret = WdtDeviceAttachToBus(WDT_1_DEVICE_NAME_1,
                                     WDT_BUS_NAME_1);
89
90         if (ret != EOK) {
91             KPrintf("Watchdog device register error %d\n", ret);
92             return ERROR;
93         }
94     #endif
95
96     return ret;
97 }

```

WATCHDOG 设备的打开见代码 5-4-2。在该实现中，将超时时间默认设置为 4095 毫秒。

代码 5-4-189 WATCHDOG 设备的打开

```

01 static uint32 WdtOpen(void *dev){
02     NULL_PARAM_CHECK(dev);
03
04     wdt_device_number_t id;
05     struct WdtHardwareDevice *wdt = (struct WdtHardwareDevice *)dev;
06     id = *(wdt_device_number_t *)wdt->private_data;
07
08     wdt_init(id, 4095, NONE, NONE);
09     return EOK;
10 }

```

WATCHDOG 的驱动配置实现见代码 5-4-3。其驱动配置的具体实现需要根据实际的配置命令来加以区分。代码行 10 到 14 对应于设置超时时间；代码行 15 到 17 对应于“喂狗”操作。

代码 5-4-190 WATCHDOG 的驱动配置

```

01 static uint32 WdtConfigure(void *drv, struct BusConfigureInfo *args){
02     NULL_PARAM_CHECK(drv);
03     NULL_PARAM_CHECK(args);
04

```

```

05     struct WdtDriver *wdt = (struct WdtDriver *)drv;
06     wdt_device_number_t id = *(wdt_device_number_t *)wdt->private_data;
07
08     switch (args->configure_cmd)
09     {
10         case OPER_WDT_SET_TIMEOUT:
11             if (wdt_init(id, (uint64_t)*(int *)args->private_data, NONE, NONE) == 0){
12                 return ERROR;
13             }
14             break;
15         case OPER_WDT_KEEPAIVE:
16             wdt_feed(id);
17             break;
18         default:
19             return ERROR;
20     }
21     return EOK;
22 }

```

4.3 WATCHDOG 应用实例

WATCHDOG 设备应用实现过程如代码 5-4-4 所示。代码行 2~6 用于打开一个 WATCHDOG 设备驱动；代码行 8~17 用于设置 WATCHDOG 的类型、时间，并通过 IO 控制函数进行配置；代码行 19~26 用于喂狗，循环 100 次。

代码 5-4-191 WATCHDOG 设备应用

```

01 void TestWDT(int argc, char *argv[]){
02     int wdt_fd = PrivOpen(WDT0_DEV_DRIVER, O_RDWR);
03     if(wdt_fd<0){
04         printf("open wdt_fd fd error:%d\n",wdt_fd);
05         return;
06     }
07
08     int wdt_time = 1000;
09     struct PrivIoctlCfg ioctl_cfg;
10     ioctl_cfg.ioctl_driver_type = WDT_TYPE;
11     ioctl_cfg.args = (void *)&wdt_time;
12
13     if (0 != PrivIoctl(wdt_fd, OPER_WDT_SET_TIMEOUT, &ioctl_cfg)) {
14         printf("ioctl wdt fd error %d\n", wdt_fd);

```

```
15     PrivClose(wdt_fd);
16     return;
17 }
18
19 int test_counter=100;
20 while(test_counter--){
21     if(1 == argc){
22         printf("dog is feed\n");
23         PrivIoctl(wdt_fd, OPER_WDT_KEEPALIVE, &iocctl_cfg);
24     }
25     PrivTaskDelay(100);
26 }
27
28 PrivClose(wdt_fd);
29 return;
30 }
```

第5章 UART 驱动

5.1 UART 介绍

5.1.1 UART 基本介绍

UART (Universal Asynchronous Receiver/Transmitter) 即通用异步收发传输器。UART 作为异步串口通信协议的一种,工作原理是将传输数据的每个字符一位接一位地顺序传输,只要 2 根传输线就可以实现双向通信,一根线发送数据的同时用另一根线接收数据。UART 串口通信有几个重要的参数,分别是波特率、起始位、数据位、停止位和奇偶检验位,对于两个使用 UART 串口通信的端口,这些参数必须匹配,否则通信将无法正常工作。RISC-V 矽璞硬件终端搭载的 K210 芯片上有高速 UART 和通用 UART。

高速 UART 为 UARHS(UART0), 它们有如下特性:

- (1) 通信速率可达 5Mbps;
- (2) 8 字节发送和接收 FIFO;
- (3) 可编程式 THRE 中断;
- (4) 不支持硬件流控制或其他调制解调器控制信号, 或同步串行数据转换器。

通用 UART 为 UART1、UART2 和 UART3，支持异步通信（RS232 和 RS485 和 IRDA），通信速率可达到 5Mbps。UART 支持 CTS 和 RTS 信号的硬件管理以及软件流控(XON 和 XOFF)。3 个接口均可被 DMA 访问或者 CPU 直接访问。它们有如下特性：

- (1) 8 字节发送和接收 FIFO；
- (2) 异步时钟支持：为了应对 CPU 对于数据同步的对波特率的要求，UART 可以单独配置数据时钟，全双工模式能保证两个时钟域中数据的同步；
- (3) RS485 接口支持：UART 可以配置为软件可编程式 RS485 模式，默认为 RS232 模式；
- (4) 可编程式 THRE 中断：用 THRE 中断模式来提升串口性能。当 THRE 模式和 FIFO 模式被选择之后，如果 FIFO 中少于阈值便触发 THRE 中断。

5.1.2 中断说明

K210 芯片中 UART 的中断分为四种：接收器线路状态中断，这是优先级最高的中断，可通过 IER 寄存器的 ELSI 位使能；接收数据可用中断，这是第二高的优先中断，可通过 IER 寄存器的 ERBFI 位使能；发送保持寄存器空中断，这是第三高的优先中断，可通过 IER 寄存器的 ETBEI 位使能；调制解调器状态中断，这是第四高的优先中断，可通过 IER 寄存器的 EDSSI 位使能。

5.1.3 寄存器说明

UART1、UART2 和 UART3 的基地址分别为：UART1_BASE_ADDR (0x50210000)、UART2_BASE_ADDR (0x50220000)、UART3_BASE_ADDR (0x50230000)，下面以 UART1 为例，对相关的寄存器进行介绍。

一. RBR:

RBR 为接收缓冲寄存器，它的偏移地址为 0x0，具体如下表所示：

位	参数名	描述	访问类型
b31-b8	RSVD_RBR	保留位	R
b7-b0	RBR	接收缓冲寄存器。复位值：0	R

二. DLL:

DLL 为除数锁（低）寄存器，它的偏移地址为 0x0，具体如下表所示：

位	参数名	描述	访问类型
b31-b8	RSVD_DLL_31to8	DLL 31 to 8 保留位，并且读为零	R

b7-b0	DLL	分频锁存（低）。波特率 = （串行时钟频率） / （ 16 * 除数）。复位值：0	R
-------	-----	--	---

三. THR:

THR 为发送保持寄存器，它的偏移地址为 0x0，具体如下表所示：

位	参数名	描述	访问类型
b31-b8	RSVD_THR_31to8	THR 31 to 8 保留位，并且读为零	R
b7-b0	THR	发送保持寄存器。复位值：0	R

四. DLH:

DLH 为除数锁存（高）寄存器，它的偏移地址为 0x0，具体如下表所示：

位	参数名	描述	访问类型
b31-b8	RSVD_DLH	DLR 31 to 8 保留位，并且读为零	R
b7-b0	DLH	16 位读 / 写， Divisor Latch 寄存器的高 8 位包含 UART 的波特率除数。输出波特率等于串行时钟 （如果是，则为 pclk ）设计	R

五. IER:

IER 为中断使能寄存器，它的偏移地址为 0x4，具体如下表所示：

位	参数名	描述	访问类型
b31-b4	RSVD_IER_31to4	IER 31 to 4 保留位，并且读为零	R/W
b3	EDSSI	启用调制解调器状态中断。这是第四高优先级中断。复位值：0，取值：0：禁用 Modem Status 中断；1：启用 Modem Status 中断	R/W
b2	ELSI	启用接收器线路状态中断。这是优先级最高的中断。复位值：0，取值：0：禁用 Receiver Line Status 中断；1：启用 Receiver Line Status 中断	R/W
b1	ETBEI	使能发送保持寄存器空中断。这是第三高优先中断。复位值：0，取值：0：禁用 Transmit empty 中断；1：启用 Transmit empty 中断	R/W
b0	ERBFI	启用接收数据可用中断。这是第二高的优先中断。复位值：0，取值：0：禁用 THRE 中断；1：启用 THRE 中断	R/W

六. LCR:

LCR 为线控寄存器，它的偏移地址为 0xc，具体如下表所示：

位	参数名	描述	访问类型
b31-b4	RSVD_LCR_31to8	LCR 31 to 4 保留位，并且读为零	R/W
b7	DLAB	该位用于启用读取和写入 Divisor Latch 寄存器，（DLL 和 DLH / LPDLL 和 LPDLH）用于设置 UART 的波特率。按顺序设置初始波特率后，必须清除该位	R/W

		访问其他寄存器。复位值：0，取值：0：除数锁存寄存器只有当 UART 不忙时才能写入；1:Divisor Latch 寄存器始终是可读和可写	
b6	BC	这用于导致传输中断条件接收设备。复位值：0，取值：0：为数据发布串行输出传输； 1：串行输出被强制为间隔状态	R/W
b5	SP	该位用于强制奇偶校验值。当 PEN，EPS 和 Stick Parity 设置为 1 时，奇偶校验位被发送并检查为逻辑 0。如果 PEN 和 Stick Parity 设置为 1，EPS 为逻辑 0，然后是奇偶校验位作为逻辑 1 发送和检查。如果该位设置为 0，Stick Parity 已禁用。复位值：0，取值： 0：禁用奇偶校验； 1：启用奇偶校验	R/W
b4	EPS	这用于在两者之间进行选择偶数和奇数奇偶校验。复位值：0，取值：0：发送奇数奇偶校验； 1：发送偶数奇偶校验	R/W
b3	PEN	奇偶校验控制复位值：0，取值：0：禁用奇偶校验；1：启用奇偶校验	R/W
b2	STOP	这用于选择外设将发送和接收的每个字符的停止位数。如果设置为零，则在串行数据中发送一个停止位。如果设置为 1 并且数据位设置为 5（LCR [1：0] 设置为零），则发送一个半停止位。否则，发送两个停止位。请注意，无论选择多少个停止位，接收器都只会检查第一个停止位。复位值：0，取值：0:1 个停止位；1：当 DLS（LCR [1：0]）为零时， 1.5 个停止位，否则为 2 个停止位	R/W
b1-b0	DLS	当 LCR_EXT 中的 DLS_E 设置为 0 时，该寄存器用于选择外设将发送和接收的每个字符的数据位数。复位值：0，取值：0：每个字符 5 个数据位；1：每个字符 6 个数据位；2：每个字符 7 个数据位；3：每个字符 8 个数据位；	

七. MCR:

MCR 为调制解调器控制寄存器，它的偏移地址为 0x10，具体如下表所示：

位	参数名	描述	访问类型
b31-b2	RSVD_MCR_31to2	MCR31to2 保留位，并且读为零	R/W
b1	RTS	请求发送这用于直接控制 Request to Send（rts_n）输出。当 MCR [1] 设置为低电平时， rts_n 信号将被置低。请注意，在环回模式 MCR [4] 设置为 1）时， rts_n 输出保持非高电平，而此位置的值在内部循环回输入。复位值： 0，取值：0：请求发送 rts_	R/W

		n 取消断言（逻辑 1）；1: 请求发送 rts_n 断言（逻辑 0）	
b0	DTR	数据终端就绪这用于直接控制数据终端就绪（dtr_n）输出。复位值：0，取值：0:dtr_n 取消断言（logic1） 1:dtr_n 断言（逻辑 0）	R/W

八. LSR:

LSR 为线路状态寄存器，它的偏移地址为 0x14，具体如下表所示：

位	参数名	描述	访问类型
b31-b8	RSVD_LSR_31to8	LSR31to8 保留位，并且读为零	R
b7	RFE	接收器 FIFO 错误位。复位值：0，取值：0:RX FIFO 没有错误； 1:RX FIFO 有错误	R
b6	TEMT	发送器空位。复位值：1，取值：0: 发射器不空；1: 发射器空	R
b2	PE	奇偶校验错误位。复位值：1，取值：0: 无奇偶校验错误；1: 有奇偶校验错误	R
b1	OE	溢出错误位。复位值：1，取值：0: 无溢出错误；1: 有溢出错误	R
b0	DR	数据就绪位。复位值：1，取值：0: 数据尚未准备好；1: 数据准备好	R

九. TCR:

TCR 为收发器控制寄存器，它的偏移地址为 0xac，具体如下表所示：

位	参数名	描述	访问类型
b31-b5	RSVD_TCR_31to5	TCR31to5 保留位，并且读为零	R
b4-b3	RFE	传输模式。0: 在此模式下，发送和接收可以同时进行。用户可以在任何时候启用 DEEN、REEN。在此模式下，编程中设置的转换时间不适用。1: 在此模式下，DE 和 RE 是互斥的。编程中只能启用 DE 或 RE 中的一个。硬件将在从 RE 到 DE 或 DE 到 RE 切换时考虑程序中设置的转换时间。在传输时，硬件将等待中止接收任何传输，然后才开始发送。2: 在此模式下，DE 和 RE 是互斥的。一旦启用了 DEEN/REEN，缺省情况下“re”将被启用，DWapbuart 控制器将准备好接收数据。如果用户将 TX FIFO 与数据一起编程，则 DWapbuart 在确保没有接收正在进行时，将禁用“re”并启用“de”信号。一旦 TX FIFO 变空，“re”信号将被启用，而“de”信号将被禁用。在此操作模式下，硬件将在从 RE 到 DE 或 DE 到 RE 切换时考虑程序中设置的转换时间。“de”和“re”信号在此模式下严格	R/W

		互补。复位值：0x0。存在：始终。内存访问：“（UARTS485INTERFACE_EN==1）？”读-写：“只读”	
b2	DE_POL	驱动程序启用极性。1：DE 信号高电平有效 0：DE 信号低电平有效，复位值：UART_DE_POL	R/W
b1	RE_POL	接收器启用极性。1：RE 信号为高电平有效 0：RE 信号为低电平有效，复位值：UART_RE_POL	R/W
b0	RS485_EN	RS485 传输使能。0：在此模式下，传输仍处于 RS232 模式。该寄存器中的所有其他字段都被保留并注册，DE_EN / RE_EN / TAT 也是保留的。1：在此模式下，传输将在 RS485 模式下进行。该寄存器的所有其他字段均适用。复位值：0x0	R/W

十. DE_EN:

DE_EN 为驱动输出使能寄存器，它的偏移地址为 0xb0，具体如下表所示：

位	参数名	描述	访问类型
b31-b1	RSVD_DE_EN_31to1	DE_EN31to1 保留位，并且读为零	R
b0	DE_Enable	DE 启用控制。- 0：不启用 ‘de’ 信号 - 1：启用 ‘de’，信号复位值：0x0	R/W

十一. RE_EN:

RE_EN 为驱动输入使能寄存器，它的偏移地址为 0xb4，具体如下表所示：

位	参数名	描述	访问类型
b31-b1	RSVD_RE_EN_31to1	RE_EN31to1 保留位，并且读为零	R
b0	RE_Enable	RE 启用控制。- 0：不启用 ‘re’ 信号 - 1：启用 ‘re’，信号复位值：0x0	R/W

UARTS 的基地址为：UARTS_BASE_ADDR (0x38000000)，下面对 UARTS 的相关寄存器进行介绍。

(1) txdata:

txdata 为发送数据寄存器，它的偏移地址为 0x00，具体如下表所示：

位	参数名	描述	访问类型
b31	full	1：数据没发完	R
b30-b8	zero	保留位，读数为 0	R
b7-b0	data	发送数据	W

(2) rxdata:

rxdata 为接收数据寄存器，它的偏移地址为 0x04，具体如下表所示：

位	参数名	描述	访问类型
b31	empty	1 : 没有数据	R
b30-b8	zero	保留位, 读出为 0	R
b7-b0	data	接收数据	R

(3) txctrl:

txctrl 为发送控制寄存器, 它的偏移地址为 0x08, 具体如下表所示:

位	参数名	描述	访问类型
b31-b19	resv1	保留位, 读出为 0	读
b18-b16	txcnt	中断触发阈值	写
b15-b2	resv0	保留位, 读出为 0	读
b1	nstop	0 表示一个停止位, 1 表示两个停止位	写
b0	txen	控制 TX 通道是否处于活动状态	写

(4) rxctrl:

rxctrl 为接收控制寄存器, 它的偏移地址为 0x0c, 具体如下表所示:

位	参数名	描述	访问类型
b31-b19	resv1	保留位, 读出为 0	读
b18-b16	rxcnt	中断触发阈值	写
b15-b1	resv0	保留位, 读出为 0	读
b0	rxen	控制 RX 通道是否处于活动状态	写

(5) ie:

ie 为中断控制寄存器, 它的偏移地址为 0x10, 具体如下表所示:

位	参数名	描述	访问类型
b31-b2	resv1	保留位, 读出为 0	R
b1	rxwm	当接收超过 rxcnt 时触发中断, 0: 关闭中断; 1 : 打开中断	W
b0	txwm	当发送数据少于 txcnt 时触发中断, 0: 关闭中断; 1: 打开中断	W

(6) ip:

ip 为中断状态寄存器, 它的偏移地址为 0x14, 具体如下表所示:

位	参数名	描述	访问类型
b31-b2	resv1	保留位, 读出为 0	R
b1	rxwm	当接收超过 rxcnt 时触发中断, 0: 无中断; 1: 有中断	R
b0	txwm	当发送数据少于 txcnt 时触发中断, 0: 无中断; 1: 有中断	R

(7) div:

div 为波特率分频寄存器，它的偏移地址为 0x18，具体如下表所示：

位	参数名	描述	访问类型
b31-b16	resv1	保留位，读出为 0	R
b15-b0	div	波特率的分频系数， $\text{div} = \text{freq} / \text{baud} - 1$	R

5.2 UART 驱动程序

UART 通过 SerialBus 挂接到 Bus 总线上，其中 SerialBus 的结构体在 bus_serial.h 文件中进行了定义。每个设备总线都关联一个 drv 和一个 dev，其中，drv 结构体也在 bus_serial.h 中进行了定义，dev 结构体在 dev_serial.h 中进行了定义。RISC-V 板的 UART 与 ARM 板的 UART 共用 bus_serial.h 和 dev_serial.h 文件，读者可参考第 4 篇的第 5 章。UART 驱动及设备的注册流程在 connect_uart.c 中进行了实现，主要包含 3 个步骤：

- （1）完成 UART 的驱动和设备到 UART 总线的挂接（或称为绑定）；
- （2）实现 UART 设备的驱动配置，需结合 UART 特性及操作系统的接口定义进行相应的开发。

步骤（1）的具体实现见代码 5-5-1，该流程是 HwUartInit 函数中的一个部分，以此介绍 UART 驱动及设备总线绑定的过程。代码行 2~15 定义了一些结构体变量并进行初始化；代码行 17~28 对结构体变量进行赋值；代码行 30~34 使用 BoardSerialBusInit 函数完成 UART 总线和驱动的初始化，并实现 UART 驱动挂接到 UART 总线，具体的实现过程详见代码 5-5-3；代码行 36~40 使用 BoardSerialDevBend 函数实现 UART 设备的注册并挂接到 UART 总线，具体的实现过程详见代码 5-5-4。

代码 5-5-192 UART 驱动及设备绑定

```

01  #ifdef BSP_USING_UART_HS
02      static struct SerialBus serial_bus_hs;
03      memset(&serial_bus_hs, 0, sizeof(struct SerialBus));
04
05      static struct SerialDriver serial_driver_hs;
06      memset(&serial_driver_hs, 0, sizeof(struct SerialDriver));
07
08      static struct SerialHardwareDevice serial_device_hs;
09      memset(&serial_device_hs, 0, sizeof(struct SerialHardwareDevice));
10
11      static struct SerialCfgParam serial_cfg_hs;
12      memset(&serial_cfg_hs, 0, sizeof(struct SerialCfgParam));

```

```

13
14 static struct SerialDevParam serial_dev_param_hs;
15 memset(&serial_dev_param_hs, 0, sizeof(struct SerialDevParam));
16
17 serial_driver_hs.drv_done = &drv_done_hs;
18 serial_driver_hs.configure = &SerialHsDrvConfigure;
19 serial_device_hs.hwdev_done = &hwdev_done_hs;
20
21 serial_cfg_hs.data_cfg = data_cfg_init;
22
23 serial_cfg_hs.hw_cfg.serial_register_base = UARTHS_BASE_ADDR;
24 serial_cfg_hs.hw_cfg.serial_irq_interrupt = IRQN_UARTHS_INTERRUPT;
25 serial_driver_hs.private_data = (void *)&serial_cfg_hs;
26
27 serial_dev_param_hs.serial_work_mode = SIGN_OPER_INT_RX;
28 serial_device_hs.haldev.private_data = (void *)&serial_dev_param_hs;
29
30 ret = BoardSerialBusInit(&serial_bus_hs, &serial_driver_hs,
31                          SERIAL_BUS_NAME_0, SERIAL_DRV_NAME_0);
32 if (EOK != ret) {
33     KPrintf("hw_serial_init uarths error ret %u\n", ret);
34     return ERROR;
35 }
36
37 ret = BoardSerialDevBend(&serial_device_hs, (void *)&serial_cfg_hs,
38                          SERIAL_BUS_NAME_0, SERIAL_0_DEVICE_NAME_0);
39 if (EOK != ret) {
40     KPrintf("hw_serial_init uarths error ret %u\n", ret);
41     return ERROR;
42 }
43 #endif

```

代码 5-5-193 UART 驱动挂接过程

```

01 static int BoardSerialBusInit(struct SerialBus *serial_bus
    , struct SerialDriver *serial_driver, const char *bus_name, const char *drv_name)
me){
02     x_err_t ret = EOK;
03

```

```

04     ret = SerialBusInit(serial_bus, bus_name);
05     if (EOK != ret) {
06         KPrintf("hw_serial_init SerialBusInit error %d\n", ret);
07         return ERROR;
08     }
09
10     ret = SerialDriverInit(serial_driver, drv_name);
11     if (EOK != ret) {
12         KPrintf("hw_serial_init SerialDriverInit error %d\n", ret);
13         return ERROR;
14     }
15
16     ret = SerialDriverAttachToBus(drv_name, bus_name);
17     if (EOK != ret) {
18         KPrintf("hw_serial_init SerialDriverAttachToBus error %d\n", ret);
19         return ERROR;
20     }
21
22     return ret;
23 }

```

BoardSerialDevBend 函数的实现见代码 5-5-3，代码行 4~8 对 UART 设备进行注册；代码行 10~14 将 UART 设备挂接到 UART 总线。

代码 5-5-194 UART 设备挂接过程

```

01 static int BoardSerialDevBend(struct SerialHardwareDevice *serial_device,
02     void *serial_param, const char *bus_name, const char *dev_name){
03
04     ret = SerialDeviceRegister(serial_device, serial_param, dev_name);
05     if (EOK != ret) {
06         KPrintf("hw_serial_init SerialDeviceInit device %s error %d\n", dev_na
me, ret);
07         return ERROR;
08     }
09
10     ret = SerialDeviceAttachToBus(dev_name, bus_name);
11     if (EOK != ret) {
12         KPrintf("SerialDeviceAttachToBus %s error %d\n", dev_name, ret);
13         return ERROR;
14     }

```

```

15
16     return ret;
17 }

```

步骤（2）的具体实现见代码 5-5-4。在 SerialDrvConfigure 函数中，代码行 2~3 进行输入参数 drv 和 configure_info 的检查；代码行 5~7 声明了 3 个变量；代码行 9~20 根据配置命令进行选择，若配置命令为 OPE_INT，则根据 SerialInit 函数执行串口初始化；若配置命令为 OPE_CFG，则根据 SerialConfigure 函数进行串口配置。

代码 5-5-195 UART 设备的驱动配置

```

01 static uint32 SerialDrvConfigure(void *drv, struct BusConfigureInfo *configure_i
nfo){
02     NULL_PARAM_CHECK(drv);
03     NULL_PARAM_CHECK(configure_info);
04
05     x_err_t ret = EOK;
06     int serial_operation_cmd;
07     struct SerialDriver *serial_drv = (struct SerialDriver *)drv;
08
09     switch (configure_info->configure_cmd)
10     {
11         case OPE_INT:
12             ret = SerialInit(serial_drv, configure_info);
13             break;
14         case OPE_CFG:
15             serial_operation_cmd = *(int *)configure_info->private_data;
16             ret = SerialConfigure(serial_drv, serial_operation_cmd);
17             break;
18         default:
19             break;
20     }
21
22     return ret;
23 }

```

5.3 UART 应用实例

UART 设备应用实现过程如代码 5-5-5 所示，以测试 LoRa 设备的读写为例。代码行 2~3 声明一个缓存，并对其初始化；代码行 5 和 6 分别用于打开一个 GPIO 驱动和一个串口驱动；代码行 8~17 定义了一些结构体变量，并对其进行赋值；代码行 19~21 通过 IO 控制

函数对打开的 GPIO 驱动进行配置；代码行 23~29 重新设置 GPIO 口和模式之后，再通过 IO 控制函数对打开的 GPIO 驱动进行配置；代码行 31~36 重新设置 GPIO 口之后，再通过 IO 控制函数对打开的 GPIO 驱动进行配置；代码行 38~49 声明一个结构体变量，并对该变量参数进行初始化和赋值；代码行 54 通过 IO 控制函数对打开的串口驱动进行配置；代码行 60~69 通过 sendbuff 配置指令，并执行写和读操作；代码行 76~85 实现串口的写和读。

代码 5-5-196 UART 设备应用

```

01 void TestLora(int argc, char *argv[]){
02     char uart_recvbuff[100];
03     memset(uart_recvbuff, 0, sizeof(uart_recvbuff));
04
05     int pin_fd = PrivOpen(LORA_PIN_DEV_DRIVER, O_RDWR);
06     int uart_fd = PrivOpen(LORA_UART_DEV_DRIVER, O_RDWR);
07
08     struct PinStat pin_m0;
09     struct PinStat pin_m1;
10     struct PinStat pin_key;
11     pin_m0.pin = BSP_E220_M0_PIN;
12     pin_m1.pin = BSP_E220_M1_PIN;
13
14     struct PrivIoctlCfg ioctl_cfg;
15     struct PinParam pin_param;
16     pin_param.cmd = GPIO_CONFIG_MODE;
17     pin_param.mode = GPIO_CFG_INPUT;
18
19     ioctl_cfg.ioctl_driver_type = PIN_TYPE;
20     ioctl_cfg.args = (void *)&pin_param;
21     PrivIoctl(pin_fd, OPE_CFG, &ioctl_cfg);
22
23     pin_param.mode = GPIO_CFG_OUTPUT;
24     pin_param.pin = BSP_E220_M0_PIN;
25     if (0 != PrivIoctl(pin_fd, OPE_CFG, &ioctl_cfg)){
26         printf("ioctl pin fd error %d\n", pin_fd);
27         PrivClose(pin_fd);
28         return;
29     }
30
31     pin_param.pin = BSP_E220_M1_PIN;
32     if (0 != PrivIoctl(pin_fd, OPE_CFG, &ioctl_cfg)){

```

```
33     printf("ioctl pin fd error %d\n", pin_fd);
34     PrivClose(pin_fd);
35     return;
36 }
37
38 struct SerialDataCfg uart_cfg;
39 memset(&uart_cfg, 0, sizeof(struct SerialDataCfg));
40
41 uart_cfg.serial_baud_rate = BAUD_RATE_9600;
42 uart_cfg.serial_data_bits = DATA_BITS_8;
43 uart_cfg.serial_stop_bits = STOP_BITS_1;
44 uart_cfg.serial_parity_mode = PARITY_NONE;
45 uart_cfg.serial_bit_order = BIT_ORDER_LSB;
46 uart_cfg.serial_invert_mode = NRZ_NORMAL;
47 uart_cfg.serial_buffer_size = SERIAL_RB_BUFSZ;
48 uart_cfg.serial_timeout = 1000;
49 uart_cfg.is_ext_uart = 0;
50
51 ioctl_cfg.ioctl_driver_type = SERIAL_TYPE;
52 ioctl_cfg.args = (void *)&uart_cfg;
53
54 PrivIoctl(uart_fd, OPE_INT, &ioctl_cfg);
55
56 GPIOSET(pin_fd, pin_m0, GPIO_HIGH);
57 GPIOSET(pin_fd, pin_m1, GPIO_HIGH);
58 printf("lora configure into sleep(configure) mode\n");
59
60 char sendbuff[] = {0xC0, 0x00, 0x05, 0x19, 0x49, 0xE6, 0x00, 0x17};
61
62 PrivTaskDelay(2000);
63 printf("Sending lora configure information(SIZE:%d)\n", sizeof(sendbuff));
64 PrivWrite(uart_fd, sendbuff, sizeof(sendbuff));
65 printf("lora configure information send\n");
66
67 PrivTaskDelay(2000);
68 PrivRead(uart_fd, uart_rcvbuff, sizeof(sendbuff));
69 printf("lora configure success\n");
70
71 uart_cfg.serial_baud_rate = BAUD_RATE_115200;
72 PrivIoctl(uart_fd, OPE_INT, &ioctl_cfg);
73
```

```

74     GPIOSET(pin_fd, pin_m0, GPIO_LOW);
75     GPIOSET(pin_fd, pin_m1, GPIO_LOW);
76     while (1){
77         PrivTaskDelay(500);
78         PrivWrite(uart_fd, "Hello_World!", sizeof("Hello_World!"));
79         printf("Data Send:\n%s\n", "Hello_World!");
80
81         PrivTaskDelay(500);
82         memset(uart_recvbuff, 0, sizeof(uart_recvbuff));
83         PrivRead(uart_fd, uart_recvbuff, sizeof(uart_recvbuff));
84         printf("Receive Data is :\n%s\n", uart_recvbuff);
85     }
86 }

```

第6章 I2C 驱动

6.1 I2C 介绍

6.1.1 I2C 基本介绍

I2C（Inter Integrated Circuit）总线是 PHILIPS 公司开发的一种半双工、双向二线制同步串行总线。I2C 总线传输数据时只需两根信号线，一根是双向数据线 SDA（serial data），另一根是双向时钟线 SCL（serial clock）。RISC-V 矽琇硬件终端搭载的 K210 芯片上共有 3 个 I2C 总线接口，根据用户的配置，总线接口可以用作 I2C MASTER 或 SLAVE 模式。同时，I2C 接口支持：

- (1) 标准模式（0 到 100Kb/s）；
- (2) 快速模式（<= 400Kb/s）；
- (3) 7-位/10-位寻址模式；
- (4) 批量传输模式；
- (5) 中断或轮询模式操作。

6.1.2 中断说明

I2C 总线控制器中有以下五种中断：

(1) I2C 总线错误中断：当总线上出现错误时，如设备未响应、总线竞争等，会产生该中断；

- (2) I2C 接收完毕中断：当 I2C 控制器接收到指定数量的数据后，会产生该中断；
- (3) I2C 发送完毕中断：当 I2C 控制器发送完指定数量的数据后，会产生该中断；
- (4) I2C 总线超时中断：当 I2C 控制器在指定时间内未收到设备响应时，会产生该中断；
- (5) I2C 传输完成中断：当 I2C 控制器完成一次完整的数据传输后，会产生该中断。

6.1.3 寄存器说明

I2C0、I2C1 和 I2C2 的基地址分别为： I2C0_BASE_ADDR (0x50280000)、 I2C1_BASE_ADDR (0x50290000)、 I2C2_BASE_ADDR (0x502A0000)，下面以 I2C0 为例，对相关的寄存器进行介绍。

一. IC_CON：

IC_CON 为 I2C 控制寄存器，该寄存器只能在 DWapbi2c 被禁用时进行写入，它的偏移地址为 0x0，具体如下表所示：

位	参数名	描述	访问类型
b31-b20	RSVDICCON_2	ICCON2 保留位	R
b19	RSVDSMBUSPERSISTENTSLVADDR_EN	SMBUSPERSISTENTSLVADDREN 保留位	R
b18	RSVDSMBUSARP_EN	SMBUSARPEN 保留位	R
b17	RSVDSMBUSSLAVEQUICKEN	SMBUSSLAVEQUICK_EN 保留位	R
b16	RSVDOPTIONALSAR_CTRL	OPTIONALSARCTRL 保留位	R
b15-b12	RSVDICCON_1	ICCON1 保留位	R
b11	RSVD-BUSCLEARFEATURECTRL	BUSCLEARFEATURE_CTR 保留位	R
b10	STOPDETIFMASTERRACTIVE	在主模式下：1' b1：仅当主设备处于活动状态时发出 STOPDET 中断。1' b0：无论主设备是否处于活动状态都发出 STOPDET。重置值：0x0，值：0x1（ENABLED）：主设备仅在主设备处于激活状态时发出 STOPDET 中断，0x0（DISABLED）：无论主控设备是否处于激活状态，主设备都发出 STOPDET 中断	R
b9	RXFIFOFULLHLDCTRL	此位控制当 Rx FIFO 物理上已满到其 RXBUFFER DEPTH 时，DWapbi2c 是否应保持总线。重置值：	R

		0x0, 值: 0x1 (ENABLED): 当 RXFIFO 满时保持总线, 0x0 (DISABLED): Rx_FIFO 满时溢出	
b8	TXEMPTYCTRL	这一位控制 TXEMPTY 中断的生成。重置值: 0x0。值: 0x1 (ENABLED): TXEMPTY 中断的受控生成, 0x0 (DISABLED): TX_EMPTY 中断默认行为	R
b7	STOPDETIFADDRESSED	在从机模式下: 1' b1: 仅在被寻址时发出 STOPDET 中断。1' b0: 无论是否被寻址都发出 STOPDET。重置值: 0x0 注意: 在常规调用地址期间, 如果 STOPDETIFADDRESSED=1' b1, 即使从机通过生成 ACK 来响应常规调用地址, 该从机也不会发出 STOPDET 中断。STOPDET 中断仅在传输的地址与从机地址 (SAR) 匹配时生成。值: 0x1 (ENABLED): 从机仅在寻址, 0x0 (DISABLED) 时发出 STOPDET intr: 从机始终发出 STOP_DET intr	R
b6	ICSLAVEDISABLE	该位控制 I2C 是否禁用其从机, 这意味着一旦施加了预置信号, 则该位采用配置参数 ICSLAVEDISABLE 的值。在应用重置后, 您可以选择启用或禁用从属设备, 这意味着软件不必配置从属设备。默认情况下, 从设备始终处于启用状态 (也处于重置状态)。如果您需要在重置后禁用它, 请将此位设置为 1。如果设置了此位 (从设备被禁用), DWapbi2c 仅起到主设备的作用, 不执行任何需要从设备的操作。重置值: ICSLAVEDISABLE 配置参数 注意: 软件应确保如果此位用 0 写入, 则位 0 也应用 0 写入。值: 0x1 (SLAVEDISABLED): 禁用从模式, 0x0 (SLAVEENABLED): 启用从模式	R
b5	ICRESTARTEN	确定在充当主机时是否可以发送 RESTART 条件。一些较旧的从设备不支持处理 RESTART 条件; 然而, 在几个 DWapbi2c 操作中使用了 RESTART 条件。当 RESTART 被禁用时, 主机被禁止执行以下功能: 发送 START BYTE 执行任何高速模式操作 高速模式操作在组合格式模式中执行方向更改使用 10 位地址执行读取操作通过将 RESTART 条件替换为 STOP 和随后的 START 条件, 拆分操作被分解为多个 DWapbi2c 传输。如果执行上述操作, 将导致设置 ICRAWINTRSTAT 寄存器的第 6 位 (TXABRT)。重置值: ICRESTARTEN 配置参数 值: 0x1 (ENABLED): 启用主机重新启动, 0x0 (DISABLED): 禁用主机重新启动	R

b4	IC10BITADDRMASTER	如果 I2CDYNAMICARUPDATE 配置参数设置为“否”（0），则此位被命名为 IC10BITADDRMASTER，并控制 DWapbi2c 在充当主机时是以 7 位还是 10 位寻址模式开始传输。如果 I2CDYNAMICARUPDATE 设置为“是”（1），则此位的功能由 IC_TAR 寄存器的第 12 位处理，并成为只读副本，称为 IC10BITADDRMASTERONLY。0:7 位寻址 1:10 位寻址重置值：IC10BITADDRMASTER 配置参数值：0x1（ADDR10BITS）：主 10 位寻址模式，0x0（ADDR_7BITS）：主 7 位寻址模式	R
b3	IC10BITADDRSLAVE	当充当从设备时，此位控制 DWapbi2c 是否响应 7 位或 10 位地址。0:7 位寻址。DWapbi2c 忽略涉及 10 位寻址的事务；对于 7 位寻址，只比较 ICSAR 寄存器的低 7 位。1:10 位寻址。DWapbi2c 只响应与 ICSAR 寄存器的全部 10 位匹配的 10 位寻址传输。重置值：IC10BITADDRSLAVE 配置参数值：0x1（ADDR10BITS）：从设备 10 位寻址，0x0（ADDR7BITS）：从属设备 7 位寻址	R
b2-b1	SPEED	这些位控制 DWapbi2c 的操作速度；只有当在主模式下操作 DWapbi2c 时，其设置才是相关的。硬件可以防止由软件编程的非法值。这些位也必须针对从模式进行适当编程，因为它用于根据速度模式捕获尖峰滤波器的正确值。此寄存器应仅使用 1 至 IC_MAXSPEEDMODE 范围内的值进行编程；否则硬件使用 IC_MAXSPEEDMODE 的值更新此寄存器。 1：标准模式（100 kbit/s）2：快速模式（≤400 kb/s）或快速模式加（≤1000Kbit/s）3：高速模式（3.4 Mbit/s）注意：当 IC_ULTRAFASTMODE=1 重置值：IC_MAXSPEEDMODE 配置值：0x1（standard）：标准速度操作模式 0x2（fast）：快速或快速加操作模式 0x3（high）时，此字段不适用：高速操作模式	R
b0	MASTER_MODE	此位控制是否启用 DWapbi2c 主机。重置值：IC_MASTERMODE 配置参数 注意：软件应确保如果此位用“1”写入，则位 6 也应用“1”写入。值：0x1（ENABLED）：启用主模式 0x0（DISABLED）：禁用主模式	R

二. IC_TAR:

IC_TAR 为 I2C 目标地址寄存器，它的偏移地址为 0x4，具体如下表所示：

位	参数名	描述	访问类型
---	-----	----	------

b31-b17	RSVDICTAR_2	ICTAR2 保留位	R
b16	RSVDSM-BUSQUICK_CMD	SMBUSQUICKCM 保留位	R
b15-b14	RSVDSMBUSARP_EN	ICTAR1 保留位	R
b13	RSVDDEVICEID	DEVICE_ID 保留位	R
b12	RSVDOPTIONALSAR_CTRL	IC10BITADDRMASTE 保留位	R
b11	SPECIAL	此位指示软件是否执行设备 ID 或常规呼叫或 START BYTE 命令。0:忽略第 10 位 GCORSTART 并正常使用 ICTAR 1:执行设备 ID 中指定的特殊 I2C 命令或 GCORSTART 位 重置值: 0x0 值: 0x1 (ENABLED): 启用 GENERALCALL 或 STARTBYTE 传输编程, 0x0 (DISABLED): 禁用 GENERALCALL 或 STARTBYTE 传输编程	R/W
b10	GCORSTART	如果位 11 (SPECIAL) 设置为 1, 位 13 (设备 ID) 设置为 0, 则此位指示 DWapbi2c 是否要执行常规呼叫或 START 字节命令。0:常规呼叫地址。发出常规呼叫后, 只能执行写入。试图发出读取命令会导致设置 ICRAWINRSTAT 寄存器的第 6 位 (TX ABRT)。DWapbi2c 保持在 General Call (常规调用) 模式, 直到 SPECIAL 位值 (位 11) 被清除。1: START BYTE 重置值: 0x0 值: 0x1 (START BYTE): START 字节传输, 0x0 (GENERALCALL): General_Call 字节传输	R/W
b9-b0	IC_TAR	这是所有主事务的目标地址。当传输普通呼叫时, 这些位将被忽略。要生成 START BYTE, CPU 只需要向这些位写入一次。如果 ICTAR 和 ICSAR 相同, 则存在环回, 但 FIFO 在主设备和从设备之间共享, 因此完全环回是不可行的。仅支持单向环回模式 (单工), 不支持双工。主设备不能向自身传输; 它只能传输到从设备。重置值: ICDEFAULTARSLAVEADDR 配置参数	R/W

三. IC_SAR:

IC_SAR 为 I2C 从设备地址寄存器, 它的偏移地址为 0x8, 具体如下表所示:

位	参数名	描述	访问类型
b31-b10	RSVDICSAR	IC_SAR 保留位	R
b9-b0	IC_SAR	当 I2C 作为从设备运行时, ICSAR 保存从设备地址。对于 7 位寻址, 仅使用 ICSAR[6:0]。只有当 I2C 接口被禁用时, 才能写入此寄存器, 这对应于 IC	R/W

		ENABLE[0]寄存器被设置为 0。其他时间的写入无效。注意：默认值不能是任何保留地址位置：即 0x00 到 0x07，或 0x78 到 0x7f。如果您将 ICSAR 或 ICTAR 编程为保留值，则无法保证设备的正确操作。重置值：ICDEFAULTSLAVEADDR 配置参数	
--	--	---	--

四. IC_DATA_CMD:

IC_DATA_CMD 为 I2C Rx/Tx 数据缓冲和命令寄存器，它的偏移地址为 0x10，具体如下表所示：

位	参数名	描述	访问类型
b31-b12	RSVDICDATA_CMD	ICDATACMD 保留位	R
b11	RSVDFIRSTDATA_BYTE	FIRSTDATABYTE 保留位	R
b10	RSVD_RESTART	RESTART 保留位	R
b9	RSVD_STOP	STOP 保留位	R
b8	CMD	该位控制是执行读取还是执行写入。当 DWapbi2c 充当从设备时，该位不控制方向。当它充当主设备时，它只控制方向。当在 TX FIFO 中输入命令时，此位区分写入和读取命令。在从接收器模式下，此位为“不在乎”，因为不需要写入此寄存器。在从发射机模式下，“0”表示要传输 ICDATACMD 中的数据。在对该位进行编程时，您应该记住以下几点：在发送 General Call 命令后尝试执行读取操作会导致 TXABRT 中断（ICRAWIN/STAT 寄存器的第 6 位），除非 ICTAR 寄存器中的第 11 位（SPECIAL）已被清除。如果在接收到 RDREQ 中断后向该位写入“1”，则发生 TX_ABRT 中断。重置值：0x0 值：0x1（READ）：主读取命令，0x0（WRITE）：主机写入命令	W
b7-b0	DAT	该寄存器包含要在 I2C 总线上发送或接收的数据。如果您正在写入此寄存器并希望执行读取，DWapbi2c 将忽略位 7:0（DAT）。但是，当您读取此寄存器时，这些位返回在 DWapbi2c 接口上接收的数据值。重置值：0x0	R/W

五. IC_SS_SCL_HCNT:

IC_SS_SCL_HCNT 为标准速度 I2C 时钟 SCL 高计数寄存器，它的偏移地址为 0x14，具体如下表所示：

位	参数名	描述	访问类型
---	-----	----	------

b31-b16	RSVDICSSSCLHIG H_COUNT	RSVDICSSSCLLOW_COUNT 保留位	R
b15-b0	ICSSSCL_LCNT	必须在发生任何 I2C 总线事务之前设置此寄存器，以确保正确的 I/O 定时。此寄存器设置标准速度的 SCL 时钟低周期计数。只有当 I2C 接口被禁用时，才能写入此寄存器，该接口对应于 ICENABLE[0]寄存器被设置为 0。其他时间的写入无效。最小有效值为 8；硬件阻止写入小于此值的值，如果尝试，则会导致设置 8。对于 APBDATAWIDTH=8 的设计，编程顺序对于确保 DWapbi2c 的正确操作非常重要。必须先对低位字节进行编程，然后对高位字节进行编程。当配置参数 ICHCCOUNTVALUES 设置为 1 时，此寄存器为只读。重置值：ICSSSCLLOW_COUNT 配置参数	R/W

六. IC_SS_SCL_LCNT:

IC_SS_SCL_LCNT 为标准速度 I2C 时钟 SCL 低计数寄存器，它的偏移地址为 0x18，具体如下表所示：

位	参数名	描述	访问类型
b31-b16	RSVDICSSSCLLOW _COUNT	ICSSSCL_HCNT 保留位	R
b15-b0	ICSSSCL_HCNT	必须在发生任何 I2C 总线事务之前设置此寄存器，以确保正确的 I/O 定时。此寄存器设置标准速度的 SCL 时钟高周期计数。只有当 I2C 接口被禁用时，才能写入此寄存器，该接口对应于 ICENABLE[0]寄存器被设置为 0。其他时间的写入无效。最小有效值为 6；硬件阻止写入小于此值的值，如果尝试，则会导致设置 6。对于 APBDATAWIDTH=8 的设计，编程顺序对于确保 DWapbi2c 的正确操作非常重要。必须首先对低位字节进行编程。然后对高位字节进行编程。当配置参数 ICHCCOUNTVALUES 设置为 1 时，此寄存器为只读。注意：此寄存器不得编程为高于 65525 的值，因为当此计数器达到 ICSSSCLHCNT+10 的值时，DWapbi2c 使用 16 位计数器来标记 I2C 总线空闲状态。重置值：ICSSSCLHIGHCOUNT 配置参数	R/W

七. IC_INTR_STAT:

IC_INTR_STAT 为 I2C 中断状态寄存器，该寄存器中的每个位都有一个对应的屏蔽位在 ICINTRMASK 中，它的偏移地址为 0x2c，具体如下表所示：

位	参数名	描述	访问类型
b31-b15	RSVDICINTR_STAT	ICINTRSTAT 保留位	R
b14	RSVDRSCLSTUCK AT_LOW	RSCLSTUCKATLOW 寄存器字段保留位 重置值： 0x0 值：0x1 (ACTIVE)；RSCLSTUCKATLOW 中断处于活动状态，0x0 (INACTIVE)；RS CLST UCKATLOW 中断不处于活动状态	R
b13	RMASTERON_HOL D	重置值：0x0 值：0x1 (ACTIVE)；RMASTERON HOLD 中断激活，0x0 (INACTIVE)；RMASTER ONHOLL 中断未激活	R
b12	RRESTARTDET	重置值：0x0 值：0x1 (ACTIVE)；RRESTARTD ET 中断激活，0x0 (INACTIVE)；RRESTART_ DET 中断未激活	R
b11	RGENCALL	重置值：0x0 值：0x1 (ACTIVE)；RGENCALL 中断激活，0x0 (INACTIVE)；RGEN_CALL 中断 未激活	R
b10	RSTARTDET	重置值：0x0 值：0x1 (ACTIVE)；RSTARTDET 中断激活，0x0 (INACTIVE)；RSTART_DET 中 断未激活	R
b9	RSTOPDET	重置值：0x0 值：0x1 (ACTIVE)；RSTOPDET 中断激活，0x0 (INACTIVE)；RSTOP_DET 中断 未激活	R
b8	R_ACTIVITY	重置值：0x0 值：0x1 (ACTIVE)；RACTIVITY 中断激活，0x0 (INACTIVE)；RACTIVITY 中断 未激活	R
b7	RRXDONE	重置值：0x0 值：0x1 (ACTIVE)；RRXDONE 中 断激活，0x0 (INACTIVE)；RRX_DONE 中断未 激活	R
b6	RTXABRT	重置值：0x0 值：0x1 (ACTIVE)；RTXABRT 中 断激活，0x0 (INACTIVE)；RTX_ABRT 中断未 激活	R
b5	RRDREQ	重置值：0x0 值：0x1 (ACTIVE)；RRDREQ 中 断激活，0x0 (INACTIVE)；RRD_REQ 中断未激 活	R
b4	RTXEMPTY	重置值：0x0 值：0x1 (ACTIVE)；RTXEMPTY 中断激活，0x0 (INACTIVE)；RTX_EMPTY 中断 未激活	R
b3	RTXOVER	重置值：0x0 值：0x1 (ACTIVE)；RTXOVER 中 断激活，0x0 (INACTIVE)；RTX_OVER 中断未 激活	R

b2	RRXFULL	重置值：0x0 值：0x1 (ACTIVE)：RRXFULL 中断激活，0x0 (INACTIVE)：RRX_FULL 中断未激活	R
b1	RRXOVER	重置值：0x0 值：0x1 (ACTIVE)：RRXOVER 中断激活，0x0 (INACTIVE)：RRX_OVER 中断未激活	R
b0	RRXUNDER	重置值：0x0 值：0x1 (ACTIVE)：RRXOVER 中断激活，0x0 (INACTIVE)：RRX_OVER 中断未激活	R

八. IC_INTR_MASK:

IC_INTR_MASK 为 I2C 中断屏蔽寄存器，它的偏移地址为 0x30，具体如下表所示：

位	参数名	描述	访问类型
b31-b15	RSVDICINTR_STAT	ICINTRSTAT 保留位	R
b14	RSVDMSCLSUCK AT_LOW	MSCLSUCKATLOW 寄存器字段保留位	R
b13	MMASTERONHOLD	此 MMAASTERONHOLD 只读位屏蔽 ICINTRSTAT 寄存器中的 RMASTERONHOLD 中断。重置值：0x0 值：0x1 (DISABLED)：MASTERONHOLD 中断未屏蔽，0x0 (ENABLED)：MASTERONHOLD 中断被屏蔽	R
b12	MRESTARTDET	此 MRESTARTDET 只读位屏蔽 ICINTRSTAT 寄存器中的 RRESTARTDET 中断。重置值：0x0 值：0x1 (DISABLED)：RESTARTDET 中断未屏蔽，0x0 (ENABLED)：RESTARTDET 中断被屏蔽	R
b11	MGENCALL	这一位屏蔽 ICINTRSTAT 寄存器中的 RGENCALL 中断。重置值：0x0 值：0x1 (DISABLED)：GENCALL 中断未屏蔽，0x0 (ENABLED)：GENCALL 中断被屏蔽	R/W
b10	MSTARTDET	这一位屏蔽 ICINTRSTAT 寄存器中的 RSTARTDET 中断。重置值：0x0 值：0x1 (DISABLED)：STARTDET 中断未屏蔽，0x0 (ENABLED)：STARTDET 中断被屏蔽	R/W
b9	MSTOPDET	这一位屏蔽 ICINTRSTAT 寄存器中的 RSTOPDET 中断。重置值：0x0 值：0x1 (DISABLED)：STOPDET 中断未屏蔽，0x0 (ENABLED)：STOPDET 中断被屏蔽	R/W
b8	M_ACTIVITY	这一位屏蔽 ICINTRSTAT 寄存器中的 RACTIVITY 中断。重置值：0x0 值：0x1 (DISABLED)：ACTIVITY 中断未屏蔽，0x0 (ENABLED)：ACTIVITY 中断被屏蔽	R/W

b7	MRXDONE	这一位屏蔽 ICINTRSTAT 寄存器中的 RRXDONE 中断。重置值：0x0 值：0x1 (DISABLED)：RX DONE 中断未屏蔽，0x0 (ENABLED)：RXDONE 中断被屏蔽	R/W
b6	MTXABRT	这一位屏蔽 ICINTRSTAT 寄存器中的 RTXABRT 中断。重置值：0x0 值：0x1 (DISABLED)：TX ABRT 中断未屏蔽，0x0 (ENABLED)：TXABRT 中断被屏蔽	R/W
b5	MRDREQ	这一位屏蔽 ICINTRSTAT 寄存器中的 RRDREQ 中断。重置值：0x0 值：0x1 (DISABLED)：RDREQ 中断未屏蔽，0x0 (ENABLED)：RDREQ 中断被屏蔽	R/W
b4	MTXEMPTY	这一位屏蔽 ICINTRSTAT 寄存器中的 RTXEMPTY 中断。重置值：0x0 值：0x1 (DISABLED)：TXEMPTY 中断未屏蔽，0x0 (ENABLED)：TXEMPTY 中断被屏蔽	R/W
b3	MTXOVER	这一位屏蔽 ICINTRSTAT 寄存器中的 RTXOVER 中断。重置值：0x0 值：0x1 (DISABLED)：TXOVER 中断未屏蔽，0x0 (ENABLED)：TXOVER 中断被屏蔽	R/W
b2	MRXFULL	这一位屏蔽 ICINTRSTAT 寄存器中的 RRXFULL 中断。重置值：0x0 值：0x1 (DISABLED)：RXFULL 中断未屏蔽，0x0 (ENABLED)：RXFULL 中断被屏蔽	R/W
b1	MRXOVER	这一位屏蔽 ICINTRSTAT 寄存器中的 RRXOVER 中断。重置值：0x0 值：0x1 (DISABLED)：RXOVER 中断未屏蔽，0x0 (ENABLED)：RXOVER 中断被屏蔽	R/W
b0	MRXUNDER	这一位屏蔽 ICINTRSTAT 寄存器中的 RRXUNDER 中断。重置值：0x0 值：0x1 (DISABLED)：RXUNDER 中断未屏蔽，0x0 (ENABLED)：RXUNDER 中断被屏蔽	R/W

九. IC_RAW_INTR_STAT:

IC_RAW_INTR_STAT 为 I2C 原始中断状态寄存器，它的偏移地址为 0x34，具体如下表所示：

位	参数名	描述	访问类型
b31-b15	RSVDICRAWINTRSTAT	ICRAWINTR_STAT 保留位	R

b14	RSVDSCLSTUCKAT LOW	SCLSTUCKAT_LO 寄存器字段保留位	R
b13	MASTERONHOLD	指示主设备是否保持总线，且 TX FIFO 是否为空。仅当 I2CDYNAMICARUPDATE=1 且 ICEMPTIFIFOOLDMASTEREN=1 时启用。重置值：0x0 值：0x1 (ACTIVE)：MASTERONHOLD 中断处于激活状态，0x0 (INACTIVE)：MASTERON_HOLD 中断处于未激活状态	R
b12	RESTART_DET	指示当 DWapbi2c 在从设备模式下操作并且正在寻址从机时，I2C 接口上是否出现 RESTART 条件。仅当 ICSLVRESTARTDETEN=1 时启用。注意：在高速模式或 START BYTE 传输期间，根据 I2C 协议，RESTART 位于地址字段之前。在这种情况下，发出 RESTART 时，从设备不是被寻址的从设备，因此 DWapbi2c 不会生成 RESTARTDET 中断。重置值：0x0 值：0x1 (ACTIVE)：RESTARTDET 中断为激活，0x0 (INACTIVE)：RESTART_DET 中断为未激活	R
b11	GEN_CALL	仅当收到一般调用地址并确认时置位。它保持置位，直到通过禁用 DWapbi2c 或 CPU 读取 ICCLRG ENCALL 寄存器的位 0 将其清除。DWapbi2c 将接收到的数据存储在 Rx 缓冲区中。重置值：0x0 值：0x1 (ACTIVE)：GENCALL 中断为激活，0x0 (INACTIVE)：GEN_CALL 中断为未激活	R
b10	START_DET	指示 I2C 接口上是否出现 START 或 RESTART 条件，无论 DWapbi2c 是在从模式还是主模式下运行。重置值：0x0 值：0x1 (ACTIVE)：STARTDET 中断处于激活状态，0x0 (INACTIVE)：START DET 中断处于未激活状态	R
b9	STOP_DET	指示无论 DWapbi2c 是在从模式还是主模式下操作，I2C 接口上是否发生了 STOP 条件。在从模式下：如果 ICCON[7]=1'b1 (STOPDETIFADDRESED)，则仅当对从设备进行寻址时才会发出 STOPDET 中断。注意：在通用调用地址期间，如果 STOPDETIF ADDRESED=1'b1，则该从设备不会发出 STOPDEST 中断，即使从设备通过生成 ACK 来响应一般调用地址。只有当传输的地址与从地址 (SAR) 匹配时，才会生成 STOPDET 中断。如果 ICCON[7]=1'b0 (STOPDETIFADDRESED)，则无论是否正在寻址，都会发出 STOPDET 打断。在主模式中：如果 ICCON[10]=1'b1 (STOPDIFMASTERACTIV	R

		E)，则仅当主设备处于激活状态时才会发出 STOP DEST 中断，STOPDET 中断将被发出，而不管主设备是否处于激活状态。重置值：0x0 值：0x1（活动）：STOPDET 中断处于激活状态，0x0（不活动）：STOP_DET 中断处于未激活状态	
b8	ACTIVITY	此位捕获 DWapbi2c 活动并保持置位，直到清除为止。有四种方法可以清除它：禁用 DWapbi2c，读取 ICCLRACTIVITY 寄存器，读取 ICCLRINTR 寄存器，系统复位。一旦设置了此位，除非使用四种方法中的一种来清除它，否则它将保持置位状态。即使 DWapbi2c 模块空闲，此位仍保持置位状态，直到清除，指示总线上有活动。重置值：0x0 值：0x1	R
b7	RX_DONE	当 DWapbi2c 用作从发送器时，如果主发送器没有确认发送的字节，则该比特被设置为 1。这发生在传输的最后一个字节，表示传输已完成。重置值：0x0 值：0x1（ACTIVE）：RXDONE 中断处于激活状态，0x0（INACTIVE）：RXDONE 中断处于未激活状态	R
b6	TX_ABRT	该位指示作为 I2C 发送器的 DWapbi2c 是否无法完成对发送 FIFO 内容的预期操作。这种情况既可能发生在 I2C 主设备上，也可能发生在从设备上，被称为“传输中止”。当此位设置为 1 时，ICTXBRTSOURCE 寄存器会指示发生传输中止的原因。注意：每当 ICTXABRTSOURCE 寄存器跟踪的任何事件导致传输中止时，DWapbi2c 只刷新/重置/清空 TXFIFO。Tx FIFO 保持这种刷新状态，直到寄存器 ICCLRTXABRT 被读取。一旦执行此读取，Tx FIFO 就准备好接受来自 APB 接口的更多数据字节。由于 TXABRT 而导致的 RX FIFO 刷新由 CORECONSULTANT 参数 ICAVOIDRXFIFOFLUSHONTXBART 控制。重置值：0x0 值：0x1（ACTIVE）：TXABRT 中断处于激活状态，0x0（INACTIVE）：TXABRT 中断不处于激活状态	R
b5	RD_REQ	当 DWapbi2c 充当从设备并且另一个 I2C 主设备试图从 DWapbi2c 读取数据时，该位被设置为 1。DWapbi2c 将 I2C 总线保持在等待状态（SCL=0），直到该中断得到服务，这意味着从设备已被要求传输数据的远程主设备寻址。处理器必须对此中断作出响应，然后将请求的数据写入 ICDATACMD 寄存器。此位在处理器读取 ICCLRRDREQ 寄存器后立即设置为 0。重置值：0x0 值：0x1（ACTIVE）：R	R

		DREQ 中断处于激活状态，0x0 (INACTIVE)：RD_REQ 中断处于未激活状态	
b4	TX_EMPTY	TXEMPTY 中断状态的行为根据 ICCON 寄存器中的 TXEMPTYCTRL 选择而不同。当 TXEMPTYCTRL=0 时：当传输缓冲区处于或低于 ICTXTL 寄存器中设置的阈值时，此位设置为 1 来自内部移位寄存器的最近弹出命令的地址/数据已完成。当缓冲区级别超过阈值时，硬件会自动清除。当 ICENABLE[0] 设置为 0 时，TX FIFO 被刷新并保持在复位状态。在那里，TX FIFO 看起来里面没有数据，所以这个位被设置为 1，前提是主状态机或从状态机中有活动。当不再有任何活动时，在 icen=0 的情况下，此位设置为 0。重置值：0x0。值：0x1 (ACTIVE)：TXEMPTY 中断处于激活状态，0x0 (INACTIVE)：TX_EMPTY 中断不处于激活状态	R
b3	TX_OVER	如果传输缓冲区被填充到 ICTXBUFFERDEPTH，并且处理器试图通过写入 ICDATACMD 寄存器来发出另一个 I2C 命令，则在传输期间设置。当模块被禁用时，此位保持其电平，直到主状态机或从状态机进入空闲状态，当 icen 变为 0 时，此中断被清除。重置值：0x0 值：0x1 (ACTIVE)：TXOVER 中断处于激活状态，0x0 (INACTIVE)：TX_OVER 中断不处于激活状态	R
b2	RX_FULL	当接收缓冲区达到或超过 ICRXTL 寄存器中的 RXTL 阈值时设置。当缓冲区级别低于阈值时，它会被硬件自动清除。如果模块被禁用 (ICENABLE[0]=0)，RX FIFO 被刷新并保持在复位状态；因此 RX FIFO 不是满的。因此，一旦 ICENABLE 位 0 被编程为 0，则该位将被清除，无论活动是否继续。重置值：0x0 值：0x1 (ACTIVE)：RXFULL 中断处于激活状态，0x0 (INACTIVE)：RXFULL 中断不处于激活状态	R
b1	RX_OVER	如果接收缓冲区完全填充到 ICRXBUFFERDEPTH，并且从外部 I2C 设备接收到额外的字节，则置位。DWapbi2c 确认这一点，但 FIFO 满后接收到的任何数据字节都会丢失。如果模块被禁用 (ICENABLE[0]=0)，该位将保持其电平，直到主或从状态机进入空闲状态，并且当 icen 变为 0 时，该中断被清除。注意：如果配置参数 ICRXFULLHLDBUSEN 被启用，并且 ICCON 寄存器的位 9 (RXXFIFOFULLHDLTRL) 被编程为 HIGH，则 RXOVER 中断永远	R

		不会发生，因为 Rx FIFO 从不溢出。重置值：0x0 值：0x1 (ACTIVE)：RXOVER 中断处于激活状态，0x0 (INACTIVE)：XOVER 中断处于未激活状态	
b0	RX_UNDER	如果处理器试图在接收缓冲区为空时通过从 ICDAT ACMD 寄存器读取，则置位。如果模块被禁用 (IC ENABLE[0]=0)，此位将保持其电平，直到主或从状态机进入空闲状态，当 icen 变为 0 时，此中断被清除。重置值：0x0 值：0x1 (ACTIVE)：RXUNDER 中断处于激活状态，0x0 (INACTIVE)：RXUNDER 中断不处于激活状态	R

十. IC_RX_TL:

IC_RX_TL 为 I2C 接收 FIFO 阈值寄存器，它的偏移地址为 0x38，具体如下表所示：

位	参数名	描述	访问类型
b31-b8	RSVDICRX_TL	ICRX_TL 保留位	R
b7-b0	RX_TL	接收 FIFO 阈值电平。控制触发 RXFULL 中断 (IC RAWINSTRAT 寄存器中的第 2 位) 的条目 (或以上) 的电平。有效范围为 0-255，但硬件不允许将此值设置为大于缓冲区深度的值。如果尝试这样做，实际设置的值将是缓冲区的最大深度。值 0 设置 1 条目的阈值，值 255 设置 256 条目的阈值。重置值：ICRX_TL 配置参数	R/W

十一. IC_TX_TL:

IC_TX_TL 为 I2C 传输 FIFO 阈值寄存器，它的偏移地址为 0x3c，具体如下表所示：

位	参数名	描述	访问类型
b31-b8	RSVDICTX_TL	ICTX_TL 保留位	R
b7-b0	TX_TL	传输 FIFO 阈值电平。控制触发 TXEMPTY 中断的条目 (或以下) 的电平 (ICRAWINRSTAT 寄存器中的第 4 位)。有效范围为 0-255，但有一个额外的限制，即不能将其设置为大于缓冲区深度的值。如果尝试这样做，实际设置的值将是缓冲区的最大深度。值 0 设置 0 条目的阈值，值 255 设置 255 条目的阈值。重置值：ICTX_TL 配置参数	R/W

十二. IC_CLR_INTR:

IC_CLR_INTR 为清除组合和独立中断寄存器，它的偏移地址为 0x40，具体如下表所示：

位	参数名	描述	访问类型
b31-b1	RSVDICCLR_INTR	CLR_INTR 保留位	R

b0	CLR_INTR	读取此寄存器以清除组合中断、所有单独中断和 IC TXABRTSOURCE 寄存器。该位不清除硬件可清除中断，而是清除软件可清除中断。有关清除 ICTXA BRT_SOURCE 的异常情况，请参阅 ICTXBRTSOU RCE 寄存器的第 9 位。重置值：0x0	R
----	----------	---	---

十三. IC_CLR_RX_UNDER:

IC_CLR_RX_UNDER 为清除 RX_UNDER 中断寄存器，它的偏移地址为 0x44，具体如下表所示：

位	参数名	描述	访问类型
b31-b1	RSVDICLRRXUN DER	ICLRRX_UNDER 保留位	R
b0	CLRRXUNDER	读取此寄存器以清除 ICRAWINRSTAT 寄存器的 R XUNDER 中断（位 0）。重置值：0x0	R

十四. IC_CLR_RX_OVER:

IC_CLR_RX_OVER 为清除 RX_OVER 中断寄存器，它的偏移地址为 0x48，具体如下表所示：

位	参数名	描述	访问类型
b31-b1	RSVDICLRRXOV ER	ICLRRX_OVER 保留位	R
b0	CLRRXOVER	读取此寄存器以清除 ICRAWINRSTAT 寄存器的 R XOVER 中断（第 1 位）。重置值：0x0	R

十五. IC_CLR_TX_OVER:

IC_CLR_TX_OVER 为清除 TX_OVER 中断寄存器，它的偏移地址为 0x4c，具体如下表所示：

位	参数名	描述	访问类型
b31-b1	RSVDICLRTXOVE R	ICLRTX_OVER 保留位	R
b0	CLRTXOVER	读取此寄存器以清除 ICRAWINRSTAT 寄存器的 T XOVER 中断（第 3 位）。重置值：0x0	R

十六. IC_CLR_RD_REQ:

IC_CLR_RD_REQ 为清除 RD_REQ 中断寄存器，它的偏移地址为 0x50，具体如下表所示：

位	参数名	描述	访问类型
b31-b1	RSVDICLRRDREQ	ICLRRD_REQ 保留位	R

b0	CLRTXOVER	读取此寄存器以清除 ICRAWINRSTAT 寄存器的 R DREQ 中断（第 5 位）。重置值：0x0	R
----	-----------	--	---

十七. IC_CLR_TX_ABRT:

IC_CLR_TX_ABRT 为清除 TX_ ABRT 中断寄存器，它的偏移地址为 0x54，具体如下表所示：

位	参数名	描述	访问类型
b31-b1	RSVDICCLRTXABRT	ICCLRTX_ABRT 保留位	R
b0	CLRTXABRT	读取此寄存器以清除 ICRAWINRSTAT 寄存器和 IC TXABRTSOURCE 寄存器的 TXABRT 中断（第 6 位）。这也将 TX FIFO 从刷新/重置状态释放，允许对 TX FIFO 进行更多写入。有关清除 ICTXABRT_SOURCE 的异常情况，请参阅 ICTXBRTSOURCE 寄存器的第 9 位。重置值：0x0	R

十八. IC_CLR_RX_DONE:

IC_CLR_RX_DONE 为清除 RX_ DONE 中断寄存器，它的偏移地址为 0x58，具体如下表所示：

位	参数名	描述	访问类型
b31-b1	RSVDICCLRRXDONE	ICCLRRX_DONE 保留位	R
b0	CLRTXABRT	读取此寄存器以清除 ICRAWINRSTAT 寄存器的 R XDONE 中断（第 7 位）。重置值：0x0	R

十九. IC_CLR_ACTIVITY:

IC_CLR_ACTIVITY 为清除 ACTIVITY 中断寄存器，它的偏移地址为 0x5c，具体如下表所示：

b31-b1	RSVDICCLR_ACTIVITY	ICCLRACTIVITY 保留位	R
b0	CLR_ACTIVITY	如果 I2C 不再处于激活状态，读取此寄存器将清除活动中断。如果 I2C 模块在总线上仍处于激活状态，则继续设置 ACTIVITY（活动）中断位。如果模块被禁用并且总线上没有进一步的活动，它会被硬件自动清除。从该寄存器读取的值，用于获取 IC RAWINT_STAT 寄存器的 ACTIVITY 中断（第 8 位）的状态。重置值：0x0	R

二十. IC_CLR_STOP_DET:

IC_CLR_STOP_DET 为清除 STOP_DET 中断寄存器，它的偏移地址为 0x60，具体如下表所示：

位	参数名	描述	访问类型
b31-b1	RSVDICCLRSTOPDET	保留位	R
b0	CLRSTOPDET	读取该寄存器以清除 ICRAWINTRSTAT 寄存器中的 STOPDET 中断（第 9 位）。复位值：0x0	R

二十一. IC_CLR_START_DET:

IC_CLR_START_DET 为清除 START_DET 中断寄存器，它的偏移地址为 0x64，具体如下表所示：

位	参数名	描述	访问类型
b31-b1	RSVDICCLRSTARTDET	保留位	R
b0	CLRSTARTDET	读取该寄存器以清除 ICRAWINTRSTAT 寄存器中的 STARTDET 中断（第 10 位）。复位值：0x0	R

二十二. IC_CLR_GEN_CALL:

IC_CLR_GEN_CALL 为清除 GEN_CALL 中断寄存器，它的偏移地址为 0x68，具体如下表所示：

位	参数名	描述	访问类型
b31-b1	RSVDICCLRGENCALL	保留位	R
b0	CLRGENCALL	读取该寄存器以清除 ICRAWINTRSTAT 寄存器中的 GENCALL 中断（第 11 位）。复位值：0x0	R

二十三. IC_ENABLE:

IC_ENABLE 为 I2C 使能寄存器，它的偏移地址为 0x6c，具体如下表所示：

位	参数名	描述	访问类型
b31-b19	RSVDICENABLE_2	IC_ENABLE 保留位	R
b18	RSVDSMBUSALERT_EN	SMBUSALERTEN 寄存器字段保留位	R
b17	RSVDSMBUSSUSPEND_EN	SMBUSSUSPENDEN 寄存器字段保留位	R
b16	RSVDSMBUSCLK_RESET	SMBUSCLKRESET 寄存器字段保留位	R
b15-b4	RSVDICENABLE_1	RSVDICENABLE_1 保留位	R
b3	RSVDSASTUCKRECOVERY_EN	SDASTUCKRECOVERY_ENABLE 保留位	R

	ERYENABLE		
b2	TXCMDBLOCK	在主模式下：1'b1：即使 Tx FIFO 有数据要传输，也会阻止在 I2C 总线上传输数据。1'b0：当 Tx FIFO 中有第一批数据可用时，数据在 I2C 总线上自动开始传输。注意：要阻止执行主命令，请仅在 Tx FIFO 为空（ICSTATUS [2] == 1）且主机处于空闲状态（ICSTATUS [5] == 0）时设置 TXCMDBLOCK 位。在取消设置 TXCMDBLOCK 位之前，放入 Tx FIFO 中的任何进一步命令都不会被执行。重置值：ICTXCMDBLOCKDEFAULT 值：0x1（BLOCKED）：Tx 命令执行被阻止，0x0（NOT_BLOCKED）：Tx 命令执行未被阻止	R/W
b1	ABORT	当设置时，控制器启动传输中止。0：未启动 ABORT 或完成 ABORT；1：正在进行 ABORT 操作。软件可以通过设置此位在主模式下中止 I2C 传输。软件只能在 ENABLE 已经设置时设置此位；否则，控制器将忽略对 ABORT 位的任何写操作。一旦设置，软件无法清除 ABORT 位。响应于中止操作，控制器在完成当前传输后发出 STOP 并清空 Tx FIFO，然后设置 TX_ABORT 中断。ABORT 位在中止操作后自动清除。复位值为 0x0。值：0x1（已启用）：正在进行 ABORT 操作；0x0（已禁用）：未进行 ABORT 操作。	R/W
b0	ENABLE	该寄存器控制 DWapbi2c 是否启用。0：禁用 DWapbi2c（TX 和 RX FIFO 保持在擦除状态）1：启用 DWapbi2c。软件可以在其处于活动状态时禁用 DWapbi2c。然而，重要的是要注意确保 DWapbi2c 被正确地禁用。推荐的禁用过程在“禁用 DWapbi2c”中描述。当 DWapbi2c 被禁用时，发生以下情况：TX FIFO 和 RX FIFO 被刷新。ICINTRSTAT 寄存器中的状态位仍然是活动的，直到 DWapbi2c 进入空闲状态。如果模块正在传输，则在当前传输完成后停止并删除传输缓冲区的内容。如果模块正在接收，则 DWapbi2c 在当前字节结束时停止当前传输，并且不确认传输。在异步 pclk 和 icclk 的系统中，当 ICCLKTYPE 参数设置为异步（1）时，启用或禁用 DWapbi2c 时会有两个 icclk 延迟。复位值：0x0 值：0x1（已启用）：启用 I2C，0x0（已禁用）：禁用 I2C	R/W

二十四. IC_STATUS:

IC_STATUS 为 I2C 状态寄存器，用于指示当前的传输状态，它的偏移地址为 0x70，具体如下表所示：

位	参数名	描述	访问类型
b31-b21	RSVDICSTATUS_2	IC_STATUS 保留位	R
b20	RSVDSM-BUSALERT_STATUS	SMBUSALERTSTATUS 寄存器字段保留位	R
b19	RSVDSM-BUSSUSPEND_STATU S	SMBUSSUSPENDSTATUS 寄存器字段保留位	R
b18	RSVDSM-BUSSLAVEADDRRESO LVED	SMBUSSLAVEADDR_RESOLVED 寄存器字段保留位	R
b17	RSVDSM-BUSSLAVEADDRVALI D	SMBUSSLAVEADDR_VALID 寄存器字段保留位	R
b16	RSVDSMBUSQUICKC MDBIT	SMBUSQUICKCMD_BIT 寄存器字段保留位	R
b15-b12	RSVDICSTATUS_1	RSVDICSTATUS_1 保留位	R
b11	RSVDSASTUC-KNOTRECOVERED	SDASTUCKNOT_RECOVERED 寄存器字段保留位	R
b10	RSVD-SLVHOLDRXFIFO_FU LL	SLVHOLDRXFIFOFULL 寄存器字段保留位	R
b9	RSVD-SLVHOLDTXFIFO_EM PTY	SLVHOLDTXFIFOEMPTY 寄存器字段保留位	R
b8	RSVDMSTHOLDRXFIF O_FULL	MSTHOLDRXFIFOFULL 寄存器字段保留位	R
b7	RSVDMSTHOLDTXFIF O_EMPTY	MSTHOLDTXFIFOEMPTY 寄存器字段保留位	R
b6	SLV_ACTIVITY	从机 FSM 活动状态。当从机有限状态机（FSM）不处于 IDLE 状态时，此位被设置为 1。0：从机 FSM 处于 IDLE 状态，因此 DWapbi2c 的从机部分处于非活动状态。1：从机 FSM 不处于 IDLE 状态，因此 DWapbi2c 的从机部分处于活动状态。复位值：0x0，值：0x1（活动）：从机非空闲，0x0（IDLE）：从机空闲	R
b5	MST_ACTIVITY	主机 FSM 活动状态。当主机有限状态机（FSM）不处于 IDLE 状态时，此位被设置为 1，表示主部分的 DWapbi2c 处于活动状态；当主 FSM 处于空	R

		闲状态时，此位被设置为 0，表示主部分的 DWap bi2c 处于非活动状态。注意：ICSTATUS [0] -即 ACTIVITY 位-是 SLVACTIVITY 和 MST_ACTIVITY 位的 OR。重置值为 0x0。值为 0x1（活动）：主机不空闲；值为 0x0（空闲）：主机空闲	
b4	RFF	接收 FIFO 完全满。当接收 FIFO 完全充满时，将设置此位。当接收 FIFO 包含一个或多个空位置时，该位将被清除。0：接收 FIFO 未滿；1：接收 FIFO 已滿；重置值：0x0；值：0x1（FULL）：Rx FIFO 已滿；0x0（NOT_FULL）：Rx FIFO 未滿	R
b3	RFNE	接收 FIFO 非空。当接收 FIFO 中包含一个或多个条目时，该位被设置；当接收 FIFO 为空时，该位被清除。0：接收 FIFO 为空，1：接收 FIFO 非空，重置值：0x0，值：0x1（非空）：Rx FIFO 非空，0x0（空）：Rx FIFO 为空	R
b2	TFE	发送 FIFO 完全为空。当发送 FIFO 完全为空时，此位被设置。当它包含一个或多个有效条目时，此位被清除。此位域不请求中断。0：发送 FIFO 不为空；1：发送 FIFO 为空。重置值：0x1。值：0x1（空）：Tx FIFO 为空；0x0（非空）：Tx FIFO 不为空	R
b1	TFNF	发送 FIFO 未滿。当发送 FIFO 包含一个或多个空位置时设置，当 FIFO 已滿时清除。0：发送 FIFO 已滿，1：发送 FIFO 未滿。复位值为 0x1。值：0x1（未滿）：Tx FIFO 未滿，0x0（已滿）：Tx FIFO 已滿	R
b0	ACTIVITY	I2C 活动状态。复位值为 0x0。取值：0x1 (ACTIVE):I2C 正在运行，0x0 (INACTIVE):I2C 处于空闲状态	R

二十五. IC_TXFLR:

IC_TXFLR 为 I2C 发送 FIFO 级别寄存器，该寄存器包含发送 FIFO 中有效数据条目的数量，它的偏移地址为 0x74，具体如下表所示：

位	参数名	描述	访问类型
b31-b4	RSVD_TXFLR	保留位	R
b3-b0	TXFLR	传输 FIFO 级别。包含传输 FIFO 中有效数据条目的数量。复位值：0x0	R

二十六. IC_RXFLR:

IC_RXFLR 为 I2C 接收 FIFO 级别寄存器，该寄存器包含接收 FIFO 中有效数据条目的数量，它的偏移地址为 0x78，具体如下表所示：

位	参数名	描述	访问类型
b31-b4	RSVD_RXFLR	保留位	R
b3-b0	RXFLR	接收 FIFO 级别。包含接收 FIFO 中有效数据条目的数量。复位值：0x0	R

二十七. IC_SDA_HOLD:

IC_SDA_HOLD 为 I2C SDA 保持时间长度寄存器，该寄存器的[15:0]位用于控制保持时间长度，它的偏移地址为 0x7c，具体如下表所示：

位	参数名	描述	访问类型
b31-b24	RSVDICSDA_HOLD	保留位	R
b23-b16	ICSDARX_HOLD	设置了当 DWapbi2c 作为接收器时，所需的 SDA 保持时间，单位为 icclk 周期。复位值：ICDEFAULT SDAHOLD [23:16]	R/W
b15-b0	ICSDATX_HOLD	设置了当 DWapbi2c 作为发送器时，所需的 SDA 保持时间，单位为 icclk 周期。复位值：ICDEFAULT SDAHOLD[15:0]。	R/W

二十八. IC_TX_ABRT_SOURCE:

IC_TX_ABRT_SOURCE 为 I2C 传输中止源寄存器，该寄存器有 32 位，用于指示 TX_ABRT 的来源，它的偏移地址为 0x80，具体如下表所示：

位	参数名	描述	访问类型
b31-b23	TXFLUSHCNT	该字段指示由于 TX_ABRT 中断而刷新的 Tx FIFO 数据命令的数量。每当 I2C 被禁用时，它都会被清除。重置值：0x0	R
b22-b21	RSVDICTXABRTSOURCE	ICTXABRT_SOURCE 保留位	R
b20-b18	RSVDABRTDEVICE_WRITE	ABRTDEVICEWRITE 寄存器字段保留位	R
b17	RSVDABRTSDASTUCKAT_LOW	ABRTSDASTUCKATLOW 寄存器字段保留位	R
b16	ABRTUSERABRT	这是一个只有主模式才有的位。主机检测到传输中止(ICENABLE[1])的复位值为 0x0，DWapbi2c 的角色是主机发送器，值为 0x1(ABRTUSERABRTGENERATED)表示主机检测到了传输中止。0x0(ABRTUSERABRT_VOID)表示主机检测到了传输中止-场景不存在	R

b15	ABRTSLVRDINTX	1: 当处理器端响应从机模式请求以将数据传输给远程主机, 并且用户在 ICDATACMD 寄存器的 CMD (第 8 位) 中写入 1 时。复位值: 0x0。DWapbi2c 的作用: 从机-发射器。值: 0x1 (ABRTSLVRDINTXGENERATED): 从机尝试在读模式下向远程主机发送。0x0 (ABRTSLVRDINTXVOID): 从机尝试在读模式下向远程主机发送-场景不存在	R
b14	ABRTSLVARBLOST	该字段表示从设备在向远程主设备传输数据时, 丢失了总线控制权。同时, ICTXABRTSOURCE[12] 也被设置。注意: 即使从设备从未拥有过总线控制权, 总线上仍可能出现故障。这是一种故障安全检查。例如, 在 SCL 从低到高的转换期间进行数据传输时, 如果数据总线上的内容与应该传输的内容不一致, 则从设备就不再拥有总线控制权。复位值: 0x0。DWapbi2c 的作用是从设备发送器。值: 0x1 (ABRTSLVARBLOSTGENERATED) 表示从设备丢失了向远程主设备的仲裁; 0x0 (ABRTSLVARBLOST_VOID) 表示从设备丢失了向远程主设备的仲裁-场景不存在	R
b13	ABRTSLVFLUSHTXFIFO	这个字段指定从设备已收到读取指令并且 TX FIFO 中存在一些数据, 因此从设备发出 TXABRT 中断以刷新 TX FIFO 中的旧数据。复位值为: 0x0。DWapbi2c 的作用: 从设备发送器。值: 0x1 (ABRTSLVFLUSHTXFIFOGENERATED): 从设备在收到读取命令时刷新 TX-FIFO 中已有的数据。值为: 0x0 (ABRTSLVFLUSHTXFIFO_VOID): 从设备在收到读取命令时刷新 TX-FIFO 中已有的数据-场景不存在	R
b12	ARB_LOST	这个字段指示主设备丢失仲裁, 或者如果 ICCTXABRTSOURCE[14] 也被设置, 则从设备发送器已经丢失了仲裁。重置值为 0x0。DWapbi2c 的角色: 主设备发送器或从设备发送器。取值: 0x1 (ABRTLOSTGENERATED): 主设备或从设备发送器丢失了仲裁; 0x0 (ABRTLOST_VOID): 主设备或从设备发送器丢失了仲裁-场景不存在	R
b11	ABRTMASTERDIS	这个字段表示用户试图在禁用主模式的情况下启动主操作。重置值为 0x0。DWapbi2c 的角色为主发射器或主接收器。值: 0x1 (ABRTMASTERDISGENERATED) 表示用户在禁用主模式时发起主操作; 0x0 (ABRTMASTERDISVOID) 表示用户在禁用主模式时发起主操作-场景不存在	R

b10	ABRT10BRD_NORSTR	该字段表示重启被禁用（ICCON [5]的 ICRESTARTEN 位= 0），主机以 10 位寻址模式发送读命令。复位值：0x0。DWapbi2c 的角色：主机接收器。值：0x1（ABRT10BRDGENERATED）：当禁用重启时，主机试图在 10 位寻址模式下读取；0x0（ABRT10BRD_VOID）：当禁用重启时，主机不尝试使用 10 位寻址模式读取	R
b9	ABRTSBYTENORSTR	要清除第 9 位，必须先修复 ABRTSBYTENORSTR 的来源；必须启用重新启动（ICCON [5] = 1），必须清除 SPECIAL 位（ICTAR [11]），或者必须清除 GCORSTART 位（ICTAR [10]）。一旦修复了 ABRTSBYTENORSTR 的来源，就可以与该寄存器中其他位相同的方式清除该位。如果在尝试清除此位之前未修复 ABRTSBYTENORSTR 的来源，则第 9 位会清除一次，然后重新断言。当将此字段设置为 1 时，将禁用重新启动（ICRESTARTEN 位（ICCON [5]）= 0），用户正在尝试发送 START 字节。复位值：0x0。DWapbi2c 的作用：主控制器。值：0x1（ABRTSBYTENORSTRGENERATED）：用户在禁用重新启动时尝试发送 START 字节，0x0（ABRTSBYTENORSTR_VOID）：用户在禁用重新启动时尝试发送 START 字节-场景不存在	R
b8	ABRTHSNORSTR	该字段指示重新启动被禁用（ICCON [5]的 ICRESTARTEN 位= 0），用户正在尝试使用主设备在高速模式下传输数据。复位值：0x0。DWapbi2c 的角色：主发射器或主接收器。值：0x1（ABRTHSNORSTRGENERATED）：用户尝试在禁用重新启动时将主设备切换到 HS 模式，0x0（ABRTHSNORSTR_VOID）：用户尝试在禁用重新启动时将主设备切换到 HS 模式-场景不存在	R
b7	ABRTSBYTEACKDET	这个字段表示主控发送了一个 START 字节，并且该字节被确认了（错误行为）。重置值：0x0。DWapbi2c 的作用：主控制器。取值：0x1（ABRTSBYTEACKDETERGENERATED）表示检测到了 START 字节的 ACK；0x0（ABRTSBYTEACKDET_VOID）表示未出现检测到 START 字节的 ACK 的情况	R
b6	ABRTHSACKDET	这个字段表示主设备处于高速模式，并且高速主设备代码已被确认（错误的行为）。重置值：0x0。DWapbi2c 的作用：主控制器。值：0x1（ABRTHSACKGENERATED）：在 HS 模式下 ACK 了 HS 主	R

		设备代码。0x0 (ABRTHSACKVOID)：在 HS 模式下 ACK 了 HS 主设备代码-场景不存在	
b5	ABRTGCALLREAD	该字段表示在主模式下，DWapbi2c 已发送一个广播调用，但用户将跟随广播调用的字节编程为从总线读取 (ICDATACMD [9]设置为 1)。复位值：0x0，DWapbi2c 的作用：主机-发送器，值：0x1 (ABRTGCALLREADGENERATED)：GCALL 后跟从总线读取，0x0 (ABRTGCALLREADVOID)：GCALL 后跟从总线读取-场景不存在	R
b4	ABRTGCALLNOACK	该字段表示 DWapbi2c 在主机模式下发送了一个广播呼叫，但总线上没有从机响应该广播呼叫。复位值：0x0，DWapbi2c 的作用：主机-发送器，值：0x1 (ABRTGCALLNOACKGENERATED)：广播呼叫未被任何从机确认，0x0 (ABRTGCALLNOACKVOID)：广播呼叫未被任何从机确认-场景不存在	R
b3	ABRTTXDATANOACK	这个字段表示仅主模式。当主机收到地址的确认信号，但在发送地址后的数据字节时，它没有收到远程从机的确认信号。复位值为 0x0。DWapbi2c 的角色：主机-发送器。值：0x1 (ABRTTXDATANOACKGENERATED)：已发送的数据未被寻址的从机确认；0x0 (ABRTTXDATANOACKVOID)：已发送的数据未被寻址的从机确认-场景不存在	R
b2	ABRT10ADDR2NOACK	该字段指示主设备处于 10 位地址模式，并且 10 位地址的第二个字节未被任何从设备确认。复位值为 0x0。DWapbi2c 的作用为主设备-发送器或主设备-接收器。值为 0x1 (激活)：10 位地址的第二个字节未被任何从设备确认；值为 0x0 (不激活)：未生成中止信号	R
b1	ABRT10ADDR1NOACK	这个字段表示主设备处于 10 位地址模式，并且第一个 10 位地址字节没有被任何从设备确认。复位值为 0x0。DWapbi2c 的作用为主设备-发送器或主设备-接收器。取值为：0x1 (激活)：10 位地址的第一字节没有被任何从设备确认；0x0 (不激活)：不会生成此中止	R
b0	ABRT7BADDR_NOACK	这个字段表示主设备处于 7 位寻址模式，并且发送的地址未被任何从设备确认。复位值为 0x0。DWapbi2c 的作用：主发送器或主接收器值：0x1 (激活)：由于 7 位地址的 NOACK 而生成此中止信号。0x0 (非激活)：不会生成此中止信号	R

二十九. IC_DMA_CR:

IC_DMA_CR 为 DMA 控制寄存器，该寄存器仅在 DWapbi2c 配置 DMA 时有效，它的偏移地址为 0x88，具体如下表所示：

位	参数名	描述	访问类型
b31-b2	RSVDICDMACR2_3 1	保留位	R
b1	TDMAE	传输 DMA 使能。此位用于启用/禁用传输 FIFO 的 DMA 通道。复位值：0x0。取值：0x1（已启用）：传输 FIFO 的 DMA 通道已启用；0x0（已禁用）：传输 FIFO 的 DMA 通道已禁用	R/W
b0	RDMAE	接收 DMA 使能。此位用于启用/禁用接收 FIFO 的 DMA 通道。复位值：0x0。取值：0x1（已启用）：接收 FIFO DMA 通道已启用，0x0（已禁用）：接收 FIFO DMA 通道已禁用	R/W

三十. IC_DMA_TDLR:

IC_DMA_TDLR 为 DMA 传输数据级别寄存器，它的偏移地址为 0x8c，具体如下表所示：

位	参数名	描述	访问类型
b31-b3	RSVDDMAT-DLR	保留位	R
b2-b0	DMATDL	传输数据等级。这个位域控制着传输逻辑发出 DMA 请求的等级。它处于水位线级别；也就是说，当传输 FIFO 中的有效数据条目数等于或低于这个字段的值时，dmatxreq 信号会被产生，并且 TDMAE = 1。重置值：0x0	R/W

三十一. IC_DMA_RDLR:

IC_DMA_RDLR 为 I2C 接收数据级别寄存器，它的偏移地址为 0x90，具体如下表所示：

位	参数名	描述	访问类型
b31-b3	RSVD-DMARDLR	保留位	R
b2-b0	DMARDL	接收数据等级。这个位域控制着接收逻辑发出 DMA 请求的等级。水位线=DMARDL+1；也就是说，当接收 FIFO 中有效数据条目的数量等于或超过此字段值+1，并且 RDMAE=1 时，将生成 dmarxreq。例如，当 DMARDL 为 0 时，当接收 FIFO 中存在 1 个或多个数据条目时，将触发 dmarxreq。复位值：0x0	R/W

三十二. IC_SDA_SETUP:

IC_SDA_SETUP 为 I2C SDA 设置寄存器，该寄存器控制延迟时间（以 ic_clk 数量为单位），它的偏移地址为 0x94，具体如下表所示：

位	参数名	描述	访问类型
b31-b8	RSVDICSDA_SETUP	保留位	R
b7-b0	SDA_SETUP	SDA 设置。建议如果所需延迟为 1000ns，则对于 ic_clk 频率为 10 MHz，ICSDASETUP 应编程为 11。ICSDASETUP 必须编程为最小值 2。复位值为 0x64，但可以通过设置 ICDEFAULTSDASETUP 配置参数来硬编码。	R/W

三十三. IC_ACK_GENERAL_CALL:

IC_ACK_GENERAL_CALL 为 I2C ACK 通用呼叫寄存器，该寄存器控制 DWapbi2c 是否响应 ACK 或 NACK，它的偏移地址为 0x98，具体如下表所示：

位	参数名	描述	访问类型
b31-b1	RSVDICACKGEN1_31	保留位	R
b0	ACKGENCALL	ACK 通用调用。当设置为 1 时，DWapbi2c 在接收到通用调用时会通过断言 icdataoe 来回复 ACK。否则，DWapbi2c 会通过否定 icdataoe 回复 NACK。复位值为 0x1，但可以通过设置 ICDEFAULTACKGENERALCALL 配置参数来硬编码。值：0x1（启用）：为通用调用生成 ACK；0x0（禁用）：为通用调用生成 NACK。	R/W

三十四. IC_ENABLE_STATUS:

IC_ENABLE_STATUS 为 I2C 使能状态寄存器，该寄存器用于报告 DWapbi2c 的硬件状态，它的偏移地址为 0x9c，具体如下表所示：

位	参数名	描述	访问类型
b31-b3	RSVDICENABLE_STATUSR	保留位	R
b2	SLVRXDATA_LOST	从机接收到的数据丢失。该位表示是否由于将 ICENABLE 位从 1 设置为 0 而中止了至少一个数据字节的 I2C 传输的从机接收操作。当读取为 1 时，DWapbi2c 被认为已经积极参与了中止的 I2C 传输（具有匹配地址），并且已经进入了 I2C 传输的数据阶段，即使数据字节已经响应了 NACK。注意：如果远程 I2C 主机在 DWapbi2c 有机会 NACK 传输之前使用 STOP 条件终止传输，并且 ICENABLE	R

		[0]已设置为 0，则此位也设置为 1。当读取为 0 时，DWapbi2c 被认为已被禁用，未积极参与从机接收传输的数据阶段。注意：当 IC_EN（位 0）读取为 0 时，CPU 可以安全地读取此位。复位值：0x0，值：0x1（活动）：从机 RX 数据丢失 0x0（非活动）：从机 RX 数据未丢失	
b1	SLVDISABLEDWHILE_BUS Y	当忙于传输或接收时，从设备被禁用。该位表示是否由于将 ICENABLE 寄存器的位 0 从 1 设置为 0 而中止了潜在或活动的从设备操作。当 CPU 在以下情况下写入 0 到 ICENABLE 寄存器时，设置该位：(a) DWapbi2c 正在从远程主机接收从设备-发送器操作的地址字节；或(b)从远程主机接收从设备-接收器操作的地址和数据字节。当读取为 1 时，DWapbi2c 被认为在 I2C 传输的任何部分强制 NACK，无论 I2C 地址是否与 DWapbi2c 中设置的从设备地址匹配(ICSAR 寄存器)，或者在 ICENABLE 设置为 0 之前完成了传输但未生效。注意：如果远程 I2C 主机在 DWapbi2c 有机会 NACK 传输之前以 STOP 条件终止传输，并且 ICENABLE[0]已设置为 0，则该位也将设置为 1。当读取为 0 时，DWapbi2c 被认为在主机活动时或 I2C 总线处于空闲状态时已禁用。注意：当 ICEN(位 0)读取为 0 时，CPU 可以安全地读取此位。复位值：0x0，值：0x1 (活动)：从设备在活动时被禁用，0x0 (非活动)：从设备在空闲时被禁用	R
b0	IC_EN	icen 状态。该位始终反映 icen 输出端口上驱动的值。当读取为 1 时，DWapbi2c 被视为处于启用状态。当读取为 0 时，DWapbi2c 被视为完全不活动。注意：CPU 可以随时安全地读取此位。当该位被读取为 0 时，CPU 可以安全地读取 SLVRXDATALOST（位 2）和 SLVDISABLEDWHILEBUSY（位 1）。复位值：0x0。取值：0x1（启用）：I2C 启用；0x0（禁用）：I2C 禁用	R

三十五. IC_FS_SPKLEN:

IC_FS_SPKLEN 为 I2C SS, FS 或 FM+抑制脉冲限制寄存器，用于存储测量到的持续时间，它的偏移地址为 0xa0，具体如下表所示：

位	参数名	描述	访问类型
b31-b8	RSVDICFS_SPKLEN	保留位	R

b7-b0	ICFSSPKLEN	在进行任何 I2C 总线事务之前，必须设置此寄存器以确保稳定操作。该寄存器设置了在 icclk 周期中测量的 SCL 或 SDA 线上最长的尖峰，这些尖峰将被尖峰抑制逻辑过滤掉的持续时间。只有在禁用 I2C 接口时才能写入此寄存器，这对应于将 ICENABLE[0]寄存器设置为 0。其他时间的写入没有效果。最小有效值为 1；硬件防止写入小于此值的值，并且如果尝试会导致设置为 1。复位值：ICDEFAULTFS_SPKLEN 配置参数	R/W
-------	------------	--	-----

三十六. IC_COMP_PARAM_1:

IC_COMP_PARAM_1 为组件参数寄存器 1，它的偏移地址为 0xf4，具体如下表所示：

位	参数名	描述	访问类型
b31-b24	RSVDICCOMPPARAM1	保留位	R
b23-b16	TXBUFFERDEPTH	该寄存器的值是从 ICTXBUFFER_DEPTH coreConsultant 参数中获取的。0x00=保留，0x01=2，0x02=3，...，0xFF=256	R
b15-b8	RXBUFFERDEPTH	该寄存器的值是从 ICRXBUFFER_DEPTH coreConsultant 参数中获取的。0x00=保留，0x01=2，0x02=3，...，0xFF=256	R
b7	ADDENCODEDPARAMS	该寄存器的值是从 ICADDENCODED_PARAMS coreConsultant 参数中获取的。在这个位上读取 1 表示已经包含了通过软件读取这些编码参数的能力。否则，无论其他编码在位中的任何其他参数的设置如何，整个寄存器都为 0。值：0x1（已启用）：启用读取编码参数的能力；0x0（已禁用）：禁用读取编码参数的能力	R
b6	HAS_DMA	该寄存器的值是从 ICHASDMA coreConsultant 参数中获取的。取值为 0x1（启用）：DMA 握手信号已启用；0x0（禁用）：DMA 握手信号已禁用	R
b5	INTR_IO	该寄存器的值是从 ICINTRIO coreConsultant 参数中获取的。取值为 0x1（COMBINED）时，表示合并中断输出；取值为 0x0（INDIVIDUAL）时，表示独立中断输出	R
b4	HCCOUNTVALUES	该寄存器的值是从 ICHCCOUNT_VALUES coreConsultant 参数中获取的。取值为 0x1（启用）：每种模式的计数值是硬编码的。取值为 0x0（禁用）：每种模式的计数值可以编程	R
b3-b2	MAXSPEEDMODE	该寄存器的值是从 ICMAXSPEED_MODE coreConsultant 参数中获取的。0x0：保留，值：0x1（标	R

		准)：IC MAX SPEED 是标准模式，0x2 (快速)：IC MAX SPEED 是快速模式，0x3 (高)：IC MAX SPEED 是高模式	
b1-b0	APBDATAWIDTH	这个寄存器的值是从 APBDATAWIDTH coreConsul tant 参数中获取的。值：0x0 (APB08BITS)：APB 数据总线宽度为 08 位，0x1 (APB16BITS)：APB 数据总线宽度为 16 位，0x2 (APB_32BITS)：APB 数据总线宽度为 32 位，0x3 (保留)：保留位	R

三十七. IC_COMP_VERSION:

IC_COMP_VERSION 为 I2C 组件版本寄存器，它的偏移地址为 0xf8，具体如下表所示：

位	参数名	描述	访问类型
b31-b0	ICCOMPVERSION	该寄存器的具体值在 DWapbi2c 发布说明的发布表中描述	R

三十八. IC_COMP_TYPE:

IC_COMP_TYPE 为 I2C 组件类型寄存器，它的偏移地址为 0xfc，具体如下表所示：

位	参数名	描述	访问类型
b31-b0	ICCOMPTYPE	设计件类型编号为 0x445701_40。这个分配的唯一十六进制值是固定的，并且由两个 ASCII 字母“DW”后跟一个 16 位无符号数派生而来	R

6.2 I2C 驱动程序

I2C 通过 I2cBus 挂接到 Bus 总线上，其中 I2cBus 的结构体在 bus_i2c.h 文件中进行了定义。每个设备总线都关联一个 drv 和一个 dev，其中，drv 结构体也在 bus_i2c.h 中进行了定义，dev 结构体在 dev_i2c.h 中进行了定义。RISC-V 板的 I2C 与 ARM 板的 I2C 共用 bus_i2c.h 和 dev_i2c.h 文件，读者可参考第 4 篇的第 6 章。I2C 驱动及设备的注册流程在 connect_i2c.c 中进行了实现，主要包含 3 个步骤：

- (1) 完成 I2C 的驱动和设备到 I2C 总线的挂接（或称为绑定）；
- (2) 实现 I2C 设备的打开、关闭及读写方法，其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植；
- (3) 实现 I2C 设备的驱动配置。

步骤 (1) 的具体实现见代码 5-6-1，该流程是完成一个 I2C 驱动及设备总线绑定的过程。代码行 2~6 定义了 1 个结果返回变量和 2 个结构体变量，并对结构体变量进行初始化；代码行 9 进行 I2C 的外部 GPIO 管脚的初始化；代码行 10 对 I2C 的驱动进行配置；代码行

12~16 使用 BoardI2cBusInit 函数完成 I2C 总线和驱动的初始化，并实现 I2C 驱动挂接到 I2C 总线，具体的实现过程详见代码 5-6-2；代码行 18~22 使用 BoardI2cDevBend 函数实现 I2C 设备的注册并挂接到 I2C 总线，具体的实现过程详见代码 5-6-3；代码行 24 对 I2C 总线进行重置。

代码 5-6-197 I2C 驱动及设备绑定

```

01 int HwI2cInit(void){
02     x_err_t ret = EOK;
03     static struct I2cBus i2c_bus;
04     memset(&i2c_bus, 0, sizeof(struct I2cBus));
05     static struct I2cDriver i2c_driver;
06     memset(&i2c_driver, 0, sizeof(struct I2cDriver));
07
08     #ifdef BSP_USING_I2C1
09         I2cGpioInit(&i2c_bus_param);
10         i2c_driver.configure = I2cDrvConfigure;
11
12         ret = BoardI2cBusInit(&i2c_bus, &i2c_driver);
13         if (EOK != ret) {
14             KPrintf("board_i2c_Init error ret %u\n", ret);
15             return ERROR;
16         }
17
18         ret = BoardI2cDevBend();
19         if (EOK != ret) {
20             KPrintf("board_i2c_Init error ret %u\n", ret);
21             return ERROR;
22         }
23
24         I2cBusReset(&i2c_bus_param);
25     #endif
26     return ret;
27 }

```

BoardI2cBusInit 函数的实现见代码 5-6-2，代码行 5~9 对 I2C 总线进行初始化；代码行 11~16 对 I2C 驱动进行初始化；代码行 18~22 将 I2C 驱动挂接到 I2C 总线上。

代码 5-6-198 I2C 驱动挂接过程

```

01 static int BoardI2cBusInit(struct I2cBus *i2c_bus, struct I2cDriver *i2c_driver){
02     x_err_t ret = EOK;

```

```

03     i2c_bus->private_data = (void *)&i2c_hal_drv_done;
04
05     ret = I2cBusInit(i2c_bus, I2C_BUS_NAME_1);
06     if (EOK != ret) {
07         KPrintf("board_i2c_init I2cBusInit error %d\n", ret);
08         return ERROR;
09     }
10
11     i2c_driver->private_data = (void *)&i2c_hal_drv_done;
12     ret = I2cDriverInit(i2c_driver, I2C_DRV_NAME_1);
13     if (EOK != ret) {
14         KPrintf("board_i2c_init I2cDriverInit error %d\n", ret);
15         return ERROR;
16     }
17
18     ret = I2cDriverAttachToBus(I2C_DRV_NAME_1, I2C_BUS_NAME_1);
19     if (EOK != ret) {
20         KPrintf("board_i2c_init I2cDriverAttachToBus error %d\n", ret);
21         return ERROR;
22     }
23
24     return ret;
25 }

```

BoardI2cDevBend 函数的实现见代码 5-6-3，代码行 2~4 声明了两个变量 ret 和 i2c_device0，并对结构体变量 i2c_device0 进行初始化；代码行 7~11 中对 I2C 设备进行注册；代码行 13~17 将 I2C 设备挂接到 I2C 总线。

代码 5-6-199 I2C 设备挂接过程

```

01 static int BoardI2cDevBend(void){
02     x_err_t ret = EOK;
03     static struct I2cHardwareDevice i2c_device0;
04     memset(&i2c_device0, 0, sizeof(struct I2cHardwareDevice));
05     i2c_device0.i2c_dev_done = &i2c_dev_done;
06
07     ret = I2cDeviceRegister(&i2c_device0, NONE, I2C_1_DEVICE_NAME_0);
08     if (EOK != ret) {
09         KPrintf("DeviceInit %s error %d\n", I2C_1_DEVICE_NAME_0, ret);
10         return ERROR;
11     }
12

```

```

13     ret = I2cDeviceAttachToBus(I2C_1_DEVICE_NAME_0, I2C_BUS_NAME_
14 );
15     if (EOK != ret) {
16         KPrintf("DeviceAttachToBus %s error %d\n", I2C_1_DEVICE_NAME_
17 0, ret);
18         return ERROR;
19     }
20     return ret;

```

步骤（2）的具体实现见代码 5-6-4，该流程是一个实现 I2C 设备的打开、关闭及读写过程。根据结构体 I2cDevDone 的定义，其中.open、.close、.write 和.read 分别表示 I2C 的打开、关闭、读、写。此时打开和关闭功能的值为 NONE，表示未启用该功能。I2cWriteData 和 I2cReadData 分别是实现写和读数据功能的具体函数。

代码 5-6-200 I2C 设备的打开、关闭及读写

```

01 static const struct I2cDevDone i2c_dev_done =
02 {
03     .dev_open = NONE,
04     .dev_close = NONE,
05     .dev_write = I2cWriteData,
06     .dev_read = I2cReadData,
07 };

```

步骤（3）的具体实现见代码 5-6-5。在 I2cDrvConfigure 函数中，代码行 2~3 进行输入参数 drv 和 configure_info 的检查；代码行 5~6 声明了 2 个变量；代码行 8~14 实现了具体的配置过程，其中，代码行 8 中 switch 语句的条件是获取输入参数 configure_info 的配置命令 configure_cmd，该命令有两种接口 OPE_INT 和 OPE_CFG，如果是 OPE_INT 命令，代码行 10 通过 I2cInit 函数对 I2C 的驱动进行配置。

代码 5-6-201 I2C 设备的驱动配置

```

01 static uint32 I2cDrvConfigure(void *drv, struct BusConfigureInfo *configure_inf
02 o){
03     NULL_PARAM_CHECK(drv);
04     NULL_PARAM_CHECK(configure_info);
05     x_err_t ret = EOK;
06     struct I2cDriver *i2c_drv = (struct I2cDriver *)drv;

```

```

07
08     switch (configure_info->configure_cmd){
09         case OPE_INT:
10             ret = I2cInit(i2c_drv, configure_info);
11             break;
12         default:
13             break;
14     }
15
16     return ret;
17 }

```

6.3 I2C 应用实例

I2C 设备应用实现过程如代码 5-6-6 所示。代码行 2~6 打开一个 I2C 设备驱动；代码行 8 设置从设备的地址；代码行 10~17 通过 IO 控制函数进行配置；代码行 19~28 使用 I2C 总线进行写和读。

代码 5-6-202 I2C 设备应用

```

01 void TestI2C(void){
02     int iic_fd = PrivOpen(I2C_DEV_DRIVER, O_RDWR);
03     if (iic_fd < 0){
04         printf("open iic_fd fd error:%d\n", iic_fd);
05         return;
06     }
07
08     uint16 iic_address = I2C_SLAVE_ADDRESS;
09
10     struct PrivIoctlCfg ioctl_cfg;
11     ioctl_cfg.ioctl_driver_type = I2C_TYPE;
12     ioctl_cfg.args = (void *)&iic_address;
13     if (0 != PrivIoctl(iic_fd, OPE_INT, &ioctl_cfg)){
14         printf("ioctl iic fd error %d\n", iic_fd);
15         PrivClose(iic_fd);
16         return;
17     }
18
19     char tmp_buff[100];
20     while (1){
21         PrivTaskDelay(1000);
22         PrivWrite(iic_fd, "Hello World!\n", sizeof("Hello World!\n"));

```

```
23     printf("msg send:%s\n", "Hello World!\n");
24     PrivTaskDelay(1000);
25     memset(tmp_buff, 0, sizeof(tmp_buff));
26     PrivRead(iic_fd, tmp_buff, sizeof(tmp_buff));
27     printf("msg recv:%s\n", tmp_buff);
28 }
29
30 PrivClose(iic_fd);
31 return;
32 }
```

第7章 TOUCH 驱动

7.1 TOUCH 介绍

7.1.1 TOUCH 基本介绍

Touch（触摸芯片）是 UI 设计中进行人机交互重要的一部分，一个完整的 UI 设计应该包括输入信息和输出信息，LCD 等屏幕设备负责显示输出，那么 Touch 设备就负责触点信息采集作为信息输入。Touch 设备与主机通讯一般都是采用 I2C 总线协议来进行数据交互，所以一个 Touch 设备，就是一个标准的 I2C 从设备，而且为了提高接收 Touch 数据的实时性，触摸芯片都会提供中断支持，当有触摸事件(抬起，按下，移动)发生时，会触发中断通知 MCU 有触摸事件。主机可以通过中断回调函数去读取触摸点信息。RISC-V 矽琇硬件终端搭载的 TOUCH 芯片型号为 GSL2038，具体固件移植可以参考：<https://github.com/arovak/GSL2038>。

7.1.2 中断说明

GSL2038 由单电源供电，范围从 2.6V~3.3V，GSL3680 有片内上电复位（POR）电路。上电后，GSL2038 在 5 毫秒内进入正常工作模式，为了使 POR 正常工作，VDD 必须下降到 1.6V 以下。GSL2038 还具有一个 RESET 引脚，RESET 引脚置低至少 100 ns 器件回到复位状态，释放 RESET 后 GSL2038 在 5 毫秒内进入正常工作模式。

芯片经历一个上电复位后，便通过 RQ 引脚向主机发出中断信号，中断标志表明是上电复位消息。该功能可用于检测任何意外断电突破事件，因此允许主机采取任何必要的措施，

例如重新配置。软件复位命令可以用来通过 I2C 复位芯片，软件复位仅需要 1 毫秒，软件复位或 RESET 引脚复位事件将不会触发上电复位中断。

7.1.3 寄存器说明

GSL2038 芯片和 CPU 之间的通信是使用 I2C 总线完成的。目前已知的寄存器是：

- 0x00-0x7F：这些寄存器用于加载部分固件；
- 0x80：包含屏幕中的触摸次数。如果为 0，则用户没有触摸屏幕，无触发点；如果为 1，则只有一个触发点；如果为 2，则有两个触发点；
- 0x84-0x87：包含第一次触摸的坐标；
- 0x88-0x8B：包含第二次触摸的坐标；
- 0x8C-0xAB：包含第三个、第四个等等（某些设备最多五个，其他设备最多十个）触摸的坐标，格式与之前的相同（每个触摸四个字节）；
- 0xE0：状态寄存器；
- 0xE4、0xBC-0xBF：某种控制寄存器。需要上传固件和软重置芯片；
- 0xF0：页寄存器。包含当前映射到 0x00-0x7F 寄存器中的内存页码。

7.2 TOUCH 驱动程序

Touch 通过 TouchBus 挂接到 Bus 总线上，其中 TouchBus 结构体在 bus_touch.h 文件中进行了定义。每个设备总线都关联一个 drv 和一个 dev，其中，drv 结构体也在 bus_touch.h 中进行了定义，dev 结构体在 dev_touch.h 中进行了定义，具体介绍详见 7.2.1 和 7.2.2。Touch 驱动及设备的注册流程在 connect_touch.c 中进行了实现，具体介绍详见 7.2.3。

7.2.1 bus_touch.h

bus_touch.h 文件中声明了 touch 总线结构体 TouchBus 和驱动结构体 TouchDriver。TouchBus 的结构体定义如下：

代码 5-7-203 TouchBus 的结构体定义

```
struct TouchBus{
    struct Bus bus;
    void *private_data;
};
```

其中 bus 是通用管理总线；private_data 是传递参数的通用指针。

TouchDriver 的结构体定义如下：

代码 5-7-204 TouchDriver 的结构体定义

```

struct TouchDriver{
    struct Driver driver;
    void *private_data;
};

```

其中 driver 表示通用管理驱动；private_data 是传递参数的通用指针。

另外,在 bus_touch.h 文件中还定义了一些函数接口,具体均在 bus_touch.c 文件中实现:

- int TouchBusInit(struct TouchBus *touch_bus, const char *bus_name), 用于初始化 Touch 总线, 其中 touch_bus 表示初始化的 touch 总线; bus_name 表示总线名称。该接口通过调用 BusRegister 函数将 touch 总线注册到 BUS 管理总线链表。
- int TouchReleaseBus(struct TouchBus *touch_bus), 用于释放 touch 总线, 即将 touch 总线从 BUS 管理总线链表中移除, 其中 touch_bus 表示需要释放的 touch 总线。一般该接口函数与 TouchBusInit 函数配对使用, 对 touch 总线进行注册和释放。
- int TouchDriverInit(struct TouchDriver *touch_driver, const char *driver_name), 用于初始化 touch 驱动, 其中 touch_driver 表示需要初始化的 touch 驱动; driver_name 表示驱动名称。该接口通过调用 TouchDriverRegister 函数将 touch 驱动注册到 touch 驱动总线。
- int TouchDriverAttachToBus(const char *drv_name, const char *bus_name), 用于将 touch 驱动挂接到 touch BUS 总线, 其中 drv_name 表示驱动名称; bus_name 表示总线名称。
- DriverType TouchDriverFind(const char *drv_name, enum DriverType_e drv_type), 用于查找挂接的 touch 驱动, 其中 drv_name 表示驱动名称; drv_type 表示驱动类型。

7.2.2 dev_touch.h

dev_touch.h 文件中声明了 touch 设备结构体 TouchHardwareDevice, 其具体的定义如下:

代码 5-7-205 TouchHardwareDevice 结构体定义

```

struct TouchHardwareDevice{
    struct HardwareDev haldev;
    const struct TouchDevDone *dev_done;
    void *private_data;
};

```

其中 `haldev` 用于描述一个设备结构；`dev_done` 用于表示一个 `touch` 设备的函数操作集；`private_data` 是传递参数的通用指针。

结构体 `TouchDevDone` 的定义如下：

代码 5-7-206 `TouchDevDone` 结构体的定义

```
struct TouchDevDone{
    uint32 (*open) (void *dev);
    uint32 (*close) (void *dev);
    uint32 (*write) (void *dev, struct BusBlockWriteParam *write_param);
    uint32 (*read) (void *dev, struct BusBlockWriteParam *read_param);
};
```

可以看出，该结构体是用于描述 `touch` 设备的读写功能。

另外，在 `dev_touch.h` 文件中还定义了一些函数接口：

- `int TouchDeviceRegister(struct TouchHardwareDevice *touch_device, void *touch_param, const char *device_name)`，用于初始化并注册 `touch` 设备到设备管理链表上，其中 `touch_device` 表示需要注册的 `touch` 设备；`touch_param` 表示 `touch` 参数；`device_name` 表示设备名称。
- `int TouchDeviceAttachToBus(const char *dev_name, const char *bus_name)`，用于将 `touch` 设备挂接到 `TOUCH` `BUS` 总线，其中 `dev_name` 表示设备名称；`bus_name` 表示总线名称。
- `HardwareDevType TouchDeviceFind(const char *dev_name, enum DevType dev_type)`，用于查找挂接的 `touch` 设备，其中 `dev_name` 表示设备名称；`dev_type` 表示设备类型。

7.2.3 `connect_touch.c` 文件

`connect_touch.c` 文件中实现了 `Touch` 驱动及设备的注册，主要包含 3 个步骤：

- （1）完成 `Touch` 的驱动和设备到 `TOUCH` 总线的挂接（或称为绑定）；
- （2）实现 `Touch` 设备的打开、关闭及读写方法，其具体实现需结合 `SoC` 厂商提供的 `SDK` 源码进行相应的移植；

步骤（1）的具体实现见代码 5-7-5。代码行 4 到 12 是对局部变量的声明和初始化；代码行 14 到 17 为对 `Touch` 总线和驱动的初始化；代码行 19 是对 `Touch` 设备变量的接口进行了初始化；代码行 21 到 25 是将 `Touch` 设备和驱动挂接到 `Touch` 总线上。

代码 5-7-207 `Touch` 驱动及设备绑定

```

01 int HwTouchInit(void){
02     x_err_t ret = EOK;
03
04     static struct TouchBus touch_bus;
05     static struct TouchDriver touch_driver;
06     static struct TouchHardwareDevice touch_dev;
07
08     memset(&touch_bus, 0, sizeof(struct TouchBus));
09     memset(&touch_driver, 0, sizeof(struct TouchDriver));
10     memset(&touch_dev, 0, sizeof(struct TouchHardwareDevice));
11
12     touch_driver.configure = TouchConfigure;
13
14     ret = BoardTouchBusInit(&touch_bus, &touch_driver,
                                TOUCH_BUS_NAME, TOUCH_DRV_NAME
E);
15     if (EOK != ret) {
16         return -ERROR;
17     }
18
19     touch_dev.dev_done = &touch_dev_done;
20
21     ret = BoardTouchDevBend(&touch_dev, NONE,
                                TOUCH_BUS_NAME, TOUCH_DEVICE_NAME);
22     if (EOK != ret) {
23         KPrintf("board_touch_Init error ret %u\n", ret);
24         return -ERROR;
25     }
26
27     return EOK;
28 }

```

代码 5-7-6 实现了驱动到 Touch 总线的挂接。代码行 4 到 9、11 到 16 分别是 Touch 总线和驱动的初始化过程；代码 18 到 23 实现了将驱动挂接到设备总线。

代码 5-7-208 Touch 总线绑定函数

```

01 static int BoardTouchBusInit(struct TouchBus* touch_bus, struct TouchDriver* t
ouch_driver, const char* bus_name, const char* drv_name){
02     x_err_t ret = EOK;
03
04     /*Init the touch bus */

```

```

05     ret = TouchBusInit(touch_bus, bus_name);
06     if (EOK != ret) {
07         KPrintf("Board_touch_init touchBusInit error %d\n", ret);
08         return -ERROR;
09     }
10
11     /*Init the touch driver*/
12     ret = TouchDriverInit(touch_driver, drv_name);
13     if (EOK != ret) {
14         KPrintf("Board_touch_init touchDriverInit error %d\n", ret);
15         return -ERROR;
16     }
17
18     /*Attach the touch driver to the touch bus*/
19     ret = TouchDriverAttachToBus(drv_name, bus_name);
20     if (EOK != ret) {
21         KPrintf("Board_touch_init TouchDriverAttachToBus error %d\n", ret);
22         return -ERROR;
23     }
24
25     return ret;
26 }

```

代码 5-7-7 实现了将 Touch 设备挂接到总线。代码行 4 到 8 为设备注册的过程，代码行 10 到 14 实现了将设备绑定至 Touch 总线。

代码 5-7-209 Touch 绑定设备函数

```

01 static int BoardTouchDevBend(struct TouchHardwareDevice* touch_device,
02 void* param, const char* bus_name, const char* dev_name){
03
04     ret = TouchDeviceRegister(touch_device, param, dev_name);
05     if (EOK != ret) {
06         KPrintf("TouchDeviceRegister device %s error %d\n", dev_name, ret);
07         return -ERROR;
08     }
09
10     ret = TouchDeviceAttachToBus(dev_name, bus_name);
11     if (EOK != ret) {
12         KPrintf("TouchDeviceAttachToBus device %s error %d\n", dev_name,
ret);

```

```

13         return -ERROR;
14     }
15
16     return ret;
17 }
18

```

步骤（2）的具体实现见代码 5-7-8，该流程是一个实现 Touch 设备的打开、关闭及读写过程。根据结构体 TouchDevDone 的定义，其中.open、.close、.write 和.read 分别表示 Touch 的打开、关闭、读、写。此时，TouchOpen 和 TouchClose 分别是实现打开和关闭设备功能的具体函数。读功能的值为 NONE，表示未启用该功能，TouchRead 是实现读数据功能的具体函数。

代码 5-7-210 Touch 设备的打开、关闭及读写

```

01 struct TouchDevDone touch_dev_done =
02 {
03     .open = TouchOpen,
04     .close = TouchClose,
05     .write = NONE,
06     .read = TouchRead
09 };

```

7.3 TOUCH 应用实例

Touch 设备应用实现过程如代码 5-7-9 所示。代码行 2~6 打开一个 Touch 设备驱动；代码行 8~12 打开一个 LCD 设备驱动；代码行 14~17 定义了两个变量，并对其初始化和赋值；代码行 19~28 配置 LCD 的像素信息，并将其写入到 LCD 设备驱动；代码行 30~49 首先通过打开的 TOUCH 感应触摸的位置，然后在 LCD 上画出相应的位置。

代码 5-7-211 TOUCH 设备应用

```

01 void TestTouch(void){
02     int touch_fd = PrivOpen(TOUCH_DEV_DRIVER, O_RDWR);
03     if (touch_fd < 0){
04         printf("open touch fd error:%d\n", touch_fd);
05         return;
06     }
07
08     int lcd_fd = PrivOpen(TOUCH_LCD_DEV_DRIVER, O_RDWR);
09     if (lcd_fd < 0){
10         printf("open lcd fd error:%d\n", lcd_fd);

```

```

11         return;
12     }
13
14     struct TouchDataStandard touch_pixel;
15     memset(&touch_pixel,0,sizeof(touch_pixel));
16     LcdWriteParam graph_param;
17     graph_param.type = LCD_DOT_TYPE;
18
19     uint16_t back_color[LCD_SIZE];
20     memset(back_color,0x00,sizeof(back_color));
21     for (int i = 0; i < LCD_SIZE; i++){
22         graph_param.pixel_info.pixel_color = &back_color;
23         graph_param.pixel_info.x_startpos = 0;
24         graph_param.pixel_info.y_startpos = i;
25         graph_param.pixel_info.x_endpos = LCD_SIZE -1;
26         graph_param.pixel_info.y_endpos = i;
27         PrivWrite(lcd_fd, &graph_param, NULL_PARAMETER);
28     }
29
30     uint16 color_select[LCD_SIZE];
31     memset(color_select,0xff,sizeof(color_select));
32     graph_param.pixel_info.pixel_color = &color_select;
33     while(1){
34         if(0 > PrivRead(touch_fd, &touch_pixel, NULL_PARAMETER)){
35             printf("read touch error\n");
36             return;
37         }
38         printf("touch pixel position x:%d,y:%d\n",touch_pixel.x,touch_pixel.y);
39         graph_param.pixel_info.x_startpos = touch_pixel.x-10>0?touch_pixel.x-1
0:0;
40         graph_param.pixel_info.y_startpos = touch_pixel.y;
41         graph_param.pixel_info.x_endpos = touch_pixel.x+10;
42         graph_param.pixel_info.y_endpos = touch_pixel.y;
43         PrivWrite(lcd_fd, &graph_param, NULL_PARAMETER);
44         graph_param.pixel_info.x_startpos = touch_pixel.x;
45         graph_param.pixel_info.y_startpos = touch_pixel.y-10>0?touch_pixel.y-1
0:0;
46         graph_param.pixel_info.x_endpos = touch_pixel.x;
47         graph_param.pixel_info.y_endpos = touch_pixel.y+10;
48         PrivWrite(lcd_fd, &graph_param, NULL_PARAMETER);
49     }

```

```
50
51     PrivClose(touch_fd);
52 }
```

第8章 SPI 驱动

RISC-V 的 SPI 驱动开发流程与 ARM 的类似。首先，通过 SPI BUS 挂接到 BUS 总线上，其中 SPI BUS 的结构体在 `bus_spi.h` 文件中进行了定义。然后，每个设备总线都关联一个 `drv` 和一个 `dev`，其中，`drv` 结构体也在 `bus_spi.h` 中进行了定义，`dev` 结构体在 `dev_spi.h` 中进行了定义。另外，SPI 驱动及设备的注册流程在 `connect_spi.c` 中进行了实现。RISC-V 的 SPI 与 ARM 的 SPI 共用 `bus_spi.h` 和 `dev_spi.h` 文件，读者可参考第四篇的第 7 章。下面 8.1 介绍了 SPI 子系统，8.2 对 RISC-V 架构中 `connect_spi.c` 文件的驱动及设备注册流程进行介绍。

8.1 SPI 介绍

8.1.1 SPI 基本介绍

RISC-V 的 SPI (Serial Peripheral Interface) 子系统是一个用于与 SPI 设备进行通信的硬件接口，它由多个组件组成，包括 SPI 主控制器、SPI 从设备接口和 SPI Flash 控制器。

SPI 主控制器是 SPI 中最核心的组件，它负责控制 SPI 总线的时序、数据传输、主设备选中和从设备选中等操作。SPI 主控制器还可以支持多个从设备的并行操作，同时可以设置传输模式和时钟频率等参数。

SPI 从设备接口是连接 SPI 从设备的接口，它可以连接多个 SPI 从设备，并提供从设备选中、数据传输和时序等功能。在 SPI 从设备接口中，每个从设备都有一个片选信号，通过控制片选信号的高低来选中不同的从设备。SPI 从设备接口还可以支持多种 SPI 数据传输模式，如全双工、半双工和单向传输等。

SPI Flash 控制器是一个特殊的 SPI 从设备接口，用于连接 SPI Flash 芯片。SPI Flash 控制器支持多种 SPI Flash 芯片，包括 Nor Flash 和 Nand Flash 等。SPI Flash 控制器还可以提供 Flash 读取、写入、擦除和保护等操作，以及支持多种 SPI Flash 芯片的特殊功能。

K210 的串行外设接口有 4 组 SPI 接口，其中 SPI0、SPI1、SPI3 只能工作在 MASTER 模式，SPI2 只能工作在 SLAVE 模式，他们有如下特性：

- 支持 1/2/4/8 线全双工模式；
- SPI0、SPI1、SPI2 可支持 25MHz 时钟；
- SPI3 最高可支持 80MHz 时钟；
- 支持 32 位宽、32BYTE 深的 FIFO；
- 独立屏蔽中断 - 主机冲突，发送 FIFO 溢出，发送 FIFO 空，接收 FIFO 满，接收 FIFO 下溢，接收 FIFO 溢出中断都可以被屏蔽独立；
- 支持 DMA 功能；
- 支持双沿的 DDR 传输模式；

8.1.2 中断说明

SPI 的接收缓冲器满，发送缓冲器空，模式故障（过载，欠载，奇偶校验错误）和 SPI 空闲可以作为中断源也可以作为内部事件触发源，传送完成只可以作为内部触发源。过载，欠载和奇偶校验错误的中断被集成为 SPI 错误中断 SPEI，所以需要通过标志判断实际发生的中断源，一旦条件成立就产生相应的中断/内部触发请求。对于接收缓冲满和发送缓冲器空的中断源，需要通过数据传送改变缓冲器状态来清除。

8.1.3 寄存器说明

一. CTRLR0 (0x0):

位	位名	读写	功能
b31-b25	RSVD_CTRLR0	R	SSTE 保留位
b24	SSTE	R/W	选择开启
b23	RSVD_CTRLR0_2	R	CTRLR0_23 保留位
b22-b21	SPI_FRF	R/W	SPI 帧格式
b20-b16	DFS_32	R/W	32 位传输大小模式下的数据帧大小
b15-b12	CFS	R/W	控制框架尺寸
b11	SRL	R/W	移位寄存器循环
b10	RSVD_SLV_OE	R	SLV_OE 保留字段
b9-b8	TMOD	R/W	传输模式
b7	SCPOL	R/W	时钟极性
b6	SCPH	R/W	时钟相位
b5-b4	FRF	R/W	帧格式

b3-b0	DFS	R	数据帧大小
-------	-----	---	-------

二. CTRLR1 (0x4):

位	位名	读写	功能
b31-b16	RSVD_CTRLR1	R	CTRLR1 保留位
b15-b0	NDF	R/W	数据帧的数量

三. SSINER (0x8):

位	位名	读写	功能
b31-b1	RSVD_SSIENR	R	SSIENR 保留位
b0	SSI-EN	R/W	开启 SSI

四. MWCR (0xc):

位	位名	读写	功能
b31-b3	RSVD_MWCR	R	MWCR 保留位
b2	MHS	R/W	Microwire 握手
b1	MDD	R/W	Microwire 控制
b0	MWMOD	R/W	Microwire 传输模式

五. SER (0x10):

位	位名	读写	功能
b31-b1	RSVD_SER	R	SER 保留位
b0	SER	R/W	开启标志

六. TXFTLR (0x18):

位	位名	读写	功能
b31-b3	RSVD_TXFTLR	R	TXFTLR 保留位
b2-b0	TFT	R/W	传输 FIFO 阈值

七. SR (0x28):

位	位名	读写	功能
b31-b7	RSVD_SR	R	SR 保留位
b6	DCOL	R	数据冲突错误
b5	RSVD_TXE	R	TXE 保留字段
b4	RFF	R	接收 FIFO 已满
b3	RFNE	R	接收 FIFO 不为空
b2	TFE	R	传输 FIFO 为空

b1	TFNF	R	传输 FIFO 未滿
b0	BUSY	R	SSI 忙标志

八. IMR (0x2c):

位	位名	读写	功能
b31-b6	RSVD_IMR	R	IMR 保留位
b5	MSTIM	R/W	多主机争用中断掩码
b4	RXFIM	R/W	接收 FIFO 完全中断掩码
b3	RXOIM	R/W	接收 FIFO 溢出中断掩码
b2	RXUIM	R/W	接收 FIFO 欠流中断掩码
b1	TXOIM	R/W	传输 FIFO 溢出中断掩码
b0	TXEIM	R/W	传输 FIFO 空中断掩码

九. ISR (0x30):

位	位名	读写	功能
b31-b6	RSVD_ISR	R	ISR 保留位
b5	MSTIS	R	多主机争用中断状态
b4	RXFIS	R	接收 FIFO 完全中断状态
b3	RXOIS	R	接收 FIFO 溢出中断状态
b2	RXUIS	R	接收 FIFO 欠流中断状态
b1	TXOIS	R	传输 FIFO 溢出中断状态
b0	TXEIS	R	传输 FIFO 空中断状态

十. TXOICR (0x38):

位	位名	读写	功能
b31-b1	RSVD_TX-OICR	R	TXOICR 保留位
b0	TXOICR	R	清除传输 FIFO 溢出中断

十一. ICR (0x48):

位	位名	读写	功能
b31-b1	RSVD_ICR	R	ICR 保留位
b0	ICR	R	清除中断

十二. DMACR (0x4c):

位	位名	读写	功能
b31-b2	RSVD_DMACR	R	DMACR 保留位

b1	TDMAE	R/W	启用传输 DMA
b0	RDMAE	R/W	启用接收 DMA

十三. DMATDLR (0x50):

位	位名	读写	功能
b31-b3	RSVD_DMATDLR	R	DMATDLR 保留位
b2-b0	DMATDL	R/W	传输数据级别

十四. IDR (0x58):

位	位名	读写	功能
b31-b0	IDCODE	R	识别码

十五. SSI_VERSION_ID (0x5c):

位	位名	读写	功能
b31-b0	SSI_COMP_VERSION	R	包含 Synopsys 组件版本的十六进制表示

8.2 SPI 驱动程序

8.2.1 connect_spi.c 文件

connect_spi.c 文件中实现了 SPI 驱动及设备的注册，该流程主要包含 3 个步骤：

- (1) 完成 SPI 的驱动和设备到 SPI 总线的挂接（或称为绑定）；
- (2) 实现 SPI 设备的打开、关闭及读写方法，其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植；
- (3) 实现 SPI 设备的驱动配置，需结合 SPI 特性及操作系统的接口定义进行相应的开发。

步骤（1）的 SPI 驱动及设备绑定过程由代码 5-8-1 中所示。在 HwSpiInit 函数中，代码行 2~3 声明了 1 个结果返回变量和 1 个结构体变量，并对结构体变量进行初始化。代码行 7~10 声明 2 个结构体变量，并对其进行初始化。代码行 12~14 对结构体变量进行赋值。代码行 16~21 使用 BoardSpiBusInit 函数进行 SPI 驱动挂接到 SPI 总线，具体的实现过程详见代码 5-8-2。代码行 23~27 使用 BoardSpiDevBend 函数进行 SPI 设备挂接到 SPI 总线，具体的实现过程详见代码 5-8-3。

在代码 5-8-2 的 BoardSpiBusInit 函数中，代码行 4~8 对 SPI 总线进行初始化；代码行 10~14 对 SPI 驱动进行初始化；代码行 16~20 将 SPI 驱动挂接到 SPI 总线上。代码 5-8-3 是

BoardSpiDevBend 函数中的一部分，作为一个例子讲述 SPI 设备挂接过程，代码行 2~12 声明了 2 个结构体变量，并对其进行初始化和赋值；代码行 14~18 中对 SPI 设备进行注册；代码行 20~34 将 SPI 设备挂接到 SPI 总线。

代码 5-8-212 SPI 驱动绑定过程

```

01 int HwSpiInit(void){
02     x_err_t ret = EOK;
03     static struct SpiDmaParam spi_initparam;
04     memset(&spi_initparam, 0, sizeof(struct SpiDmaParam));
05
06 #ifdef BSP_USING_SPI1
07     static struct SpiBus spi_bus;
08     memset(&spi_bus, 0, sizeof(struct SpiBus));
09     static struct SpiDriver spi_driver;
10     memset(&spi_driver, 0, sizeof(struct SpiDriver));
11
12     spi_initparam.spi_master_id = SPI_DEVICE_1;
13     spi_initparam.spi_dmac_txchannel = DMAC_CHANNEL1;
14     spi_initparam.spi_dmac_rxchannel = DMAC_CHANNEL2;
15
16     spi_driver.configure = &(SpiDrvConfigure);
17     ret = BoardSpiBusInit(&spi_bus, &spi_driver);
18     if (EOK != ret) {
19         KPrintf("Board_Spi_Init error ret %u\n", ret);
20         return ERROR;
21     }
22
23     ret = BoardSpiDevBend(&spi_initparam);
24     if (EOK != ret) {
25         KPrintf("Board_Spi_Init error ret %u\n", ret);
26         return ERROR;
27     }
28 #endif
29     return ret;
30 }

```

代码 5-8-213 SPI 驱动挂接过程

```

01 static int BoardSpiBusInit(struct SpiBus *spi_bus, struct SpiDriver *spi_driver){
02     x_err_t ret = EOK;
03

```

```

04     ret = SpiBusInit(spi_bus, SPI_BUS_NAME_1);
05     if (EOK != ret) {
06         KPrintf("Board_Spi_init SpiBusInit error %d\n", ret);
07         return ERROR;
08     }
09
10     ret = SpiDriverInit(spi_driver, SPI_1_DRV_NAME);
11     if (EOK != ret) {
12         KPrintf("Board_Spi_init SpiDriverInit error %d\n", ret);
13         return ERROR;
14     }
15
16     ret = SpiDriverAttachToBus(SPI_1_DRV_NAME, SPI_BUS_NAME_1);
17     if (EOK != ret) {
18         KPrintf("Board_Spi_init SpiDriverAttachToBus error %d\n", ret);
19         return ERROR;
20     }
21
22     return ret;
23 }

```

代码 5-8-214 SPI 设备挂接过程

```

01  #ifdef BSP_SPI1_USING_SS0
02     static struct SpiHardwareDevice spi_device0;
03     memset(&spi_device0, 0, sizeof(struct SpiHardwareDevice));
04     static struct SpiSlaveParam spi_slaveparam0;
05     memset(&spi_slaveparam0, 0, sizeof(struct SpiSlaveParam));
06
07     spi_slaveparam0.spi_slave_id = SPI_DEVICE_SLAVE_ID_0;
08     spi_slaveparam0.spi_cs_gpio_pin = SPI1_CS0_PIN;
09     spi_slaveparam0.spi_cs_select_id = SPI_CHIP_SELECT_0;
10     spi_device0.spi_param.spi_dma_param = spi_initparam;
11     spi_device0.spi_param.spi_slave_param = &spi_slaveparam0;
12     spi_device0.spi_dev_done = &(spi_dev_done);
13
14     ret = SpiDeviceRegister(&spi_device0, (void *)(&spi_device0.spi_param),
SPI_1_DEVICE_NAME_0);
15     if (EOK != ret) {
16         KPrintf("SpiDeviceInit %s error %d\n", SPI_1_DEVICE_NAME_0, re
t);
17         return ERROR;

```

```

18     }
19
20     ret = SpiDeviceAttachToBus(SPI_1_DEVICE_NAME_0, SPI_BUS_NAME_
1);
21     if (EOK != ret) {
22         KPrintf("AttachToBus %s error %d\n", SPI_1_DEVICE_NAME_0, ret);
23         return ERROR;
24     }
25 #endif

```

步骤（2）的实现过程如代码 5-8-4 所示，该流程是一个实现 SPI 设备的打开、关闭及读写过程。根据结构体 SpiDevDone 的定义，打开和关闭的值均为 NONE，表示该功能未启用。而读和写的值分别是 SpiWriteData 和 SpiReadData，表明这两个功能是打开的。

代码 5-8-215 SPI 设备的打开、关闭及读写

```

01 static const struct SpiDevDone spi_dev_done =
02 {
03     .dev_open = NONE,
04     .dev_close = NONE,
05     .dev_write = SpiWriteData,
06     .dev_read = SpiReadData,
07 };

```

SPI 设备的驱动配置过程如代码 5-8-5 所示。在 SpiDrvConfigure 函数中，代码行 2~3 进行输入参数 drv 和 configure_info 的检查。代码行 5~7 声明了 3 个变量。代码行 9~20 根据配置命令进行选择，若配置命令为 OPE_INT，则根据 SpiSdkInit 函数执行 SPI 驱动初始化；若配置命令为 OPE_CFG，则根据 SpiSdkCfg 函数进行 SPI 驱动配置。

代码 5-8-216 SPI 设备的驱动配置

```

01 static uint32 SpiDrvConfigure(void *drv, struct BusConfigureInfo *configure_inf
0){
02     NULL_PARAM_CHECK(drv);
03     NULL_PARAM_CHECK(configure_info);
04
05     x_err_t ret = EOK;
06     struct SpiDriver *spi_drv = (struct SpiDriver *)drv;
07     struct SpiMasterParam *spi_param;
08
09     switch (configure_info->configure_cmd)
10     {

```

```
11         case OPE_INT:
12             ret = SpiSdkInit(spi_drv);
13             break;
14         case OPE_CFG:
15             spi_param = (struct SpiMasterParam *)configure_info->private_data;
16             ret = SpiSdkCfg(spi_drv, spi_param);
17             break;
18         default:
19             break;
20     }
21
22     return ret;
23 }
```

第9章 USB 驱动开发

9.1 USB 介绍

9.1.1 USB 基本介绍

USB (Universal Serial Bus) 是一种通用的数字通信接口,已经成为了现代计算机和嵌入式系统中最常用的接口之一。USB 接口分为主机和从设备两种类型,主机可以连接多个从设备,从设备则只能连接到一个主机。

在嵌入式系统中,USB 通常是通过硬件实现的,主控芯片会集成 USB 控制器和 USB PHY (物理层) 模块。USB 控制器负责协调数据传输和设备管理,而 USB PHY 则负责将数字信号转换为模拟信号或将模拟信号转换为数字信号。

K210 的 USB 驱动采用 CH376 是文件管理控制芯片,用于单片机系统读写 U 盘或者 SD 卡中的文件。CH376 支持 USB 设备方式和 USB 主机方式,并且内置了 USB 通讯协议的基本固件,内置了处理海量存储设备的专用通讯协议的固件,内置了 SD 卡的通讯接口固件,内置了 FAT16 和 FAT32 以及 FAT12 文件系统的管理固件,支持常用的 USB 存储设备 (包括 U 盘/USB 硬盘/USB 闪存盘/USB 读卡器) 和 SD 卡 (包括标准容量 SD 卡和高容量 HG-S D 卡以及协议兼容的 MMC 卡和 TF 卡)。

CH376 支持三种通讯接口：8 位并口、SPI 接口或者异步串口，单片机/DSP/MCU/MPU 等控制器可以通过上述任何一种通讯接口控制 CH376 芯片，存取 U 盘或者 SD 卡中的文件或者与计算机通讯。

9.1.2 中断说明

CH376 芯片有 3 个中断源：

1. USB 中断：当 USB 总线上有事务需要处理时，USB 模块会产生中断。此中断由 INT0 引脚触发，中断服务程序需要读取 USBIRQ 寄存器确定中断源，然后进行相应处理；

2. 串口中断：当串口接收到一个字节数据或发送一个字节数据完成时，串口模块会产生中断。此中断由 INT1 引脚触发，中断服务程序需要读取 UARTIR 寄存器确定中断源，然后进行相应处理；

3. 定时器中断：定时器计数到设置的值时会产生中断。此中断由 INT2 引脚触发，中断服务程序需要读取 TOIR 寄存器确定中断源，然后进行相应处理。

9.1.3 寄存器说明

一. CMD_SET_BAUDRATE:

该命令寄存器用于设置 CH376 的串口通讯波特率。在 CH376 工作于串口通讯方式时，复位后默认的通讯波特率 BZ/D4、SCK/D5、SD1/D6 三个引脚的电平组合设定，这些引脚都悬空时默认是 9600bps。如果单片机支持较高的通讯速度，可以通过该命令动态调整串口通讯波特率。

二. CMD_ENTER_SLEEP:

该命令使 CH376 芯片进入低功耗睡眠挂起状态。进入低功耗状态后，CH376 芯片的时钟停振，从而节约电能，直到检测到以下两种情况之一才退出低功耗状态：一是检测到 USB 总线有信号（例如 USB 主机启动传输或者 USB 设备插拔事件）；二是单片机向 CH376 芯片写入新的命令（没有输入数据的命令，例如 CMD_GET_IC_VER 或 CMD_ABORT_NAK）。对于 SPI 通讯接口方式，SCS 片选有效也将导致 CH376 退出低功耗状态，所以单片机在发出 CMD_ENTER_SLEEP 命令后，应该立即使 SCS 片选无效。

通常情况下，CH37 芯片从低功耗状态退出并恢复到正常工作状态需要几毫秒的唤醒时间，当完全恢复到正常工作状态后，CH376 将产生 USB_INT_WAKE_UP 事件中断。

三. CMD_RESET_ALL:

该命令使 CH376 执行硬件复位。通常情况下，硬件复位在 35ms 时间之内完成。对于并口通讯方式，硬件复位通常在 1ms 时间之内完成。

四. CMD_CHECK_EXIST:

该命令用于测试通讯接口和工作状态，以检查 CH376 是否正常工作。该命令需要输入 1 个数据，可以是任意数据，如果 CH376 正常工作，那么 CH376 的输出数据是输入数据的按位取反。例如，输入数据是 57H，则输出数据是 A8H。另外，对于并口通讯方式的 CH376，在其复位后未收到任何命令之前，从其并口通常可以读出数据 00H。

五. CMD_SET_SDO_INT:

该命令用于设置 SPI 接口的 SDO 引脚的中断方式。该命令首先需要输入 1 个数据 16H，然后需要输入新的中断方式。中断方式有两种：10H 禁止 SDO 引脚用于中断输出，其在 SCS 片选无效时三态输出禁止，便于与其它设备共享单片机的 SPI 总线；90H 设置 SDO 引脚一直处于输出状态，在 SCS 片选无效时兼做中断请求输出，等效于 INT#引脚，供单片机查询中断请求状态。

六. CMD_GET_FILE_SIZE:

该命令用于获取当前文件的长度，即字节数。该命令需要输入 1 个数据 68H，输出为当前正在打开的文件的长度，该长度是以低字节在前的 4 个字节表示的双字数据（32 位）。

七. CMD_SET_FILE_SIZE:

该命令用于获取当前文件的长度，即字节数。该命令需要输入 1 个数据 68H，输出为当前正在打开的文件的长度，该长度是以低字节在前的 4 个字节表示的双字数据（32 位）。

八. CMD_SET_USB_MODE:

该命令用于设置当前文件的长度，即字节数。该命令首先需要输入 1 个数据 68H，然后需要输入新的文件长度，该长度是以低字节在前的 4 个字节表示的双字数据（32 位）。

该命令只是修改 CH376 内存中的文件长度变量，只有当 CMD_FILE_CLOSE 等命令执行之后才会真正更新 USB 存储设备或者 SD 卡中的文件的长度。

九. CMD_GET_STATUS:

该命令用于获取 CH376 的中断状态并通知 CH376 取消中断请求。当 CH376 向单片机请求中断后，单片机通过该命令获取中断状态，分析中断原因并处理。

十. CMD_RD_USB_DATAO:

该命令用于从当前 USB 中断的端点缓冲区或者主机端点的接收缓冲区读取数据块。首先读取的输出数据是数据块长度，也就是后续数据流的字节数。数据块长度的有效值对于文件读写是 0 至 255，对于 USB 底层传输是 0 至 64，如果长度不为 0，则单片机必须将后续数据从 CH376 逐个读取完。

十一. CMD_WR_HOST_DATA:

该命令用于向 USB 主机端点的发送缓冲区写入数据块。首先写入的输入数据是数据块长度，也就是后续数据流的字节数。数据块长度的有效值是 0 至 64，如果长度不为 0，则单片机必须将后续数据逐个写入 CH376。

十二. CMD_WR_REQ_DATA:

该命令用于向内部指定缓冲区写 CH376 请求的数据块。首先读取的输出数据是数据块长度，也就是 CH376 请求单片机写入的后续数据流的字节数。数据块长度的有效值对于文件读写是 0 至 255，对于 USB 底层传输是 0 至 64，如果长度不为 0，则单片机必须将后续数据逐个写入 CH376。

十三. CMD_WR_OFS_DATA:

该命令用于向内部缓冲区指定偏移地址写入数据块。首先写入的输入数据是偏移地址（内部缓冲区起始地址加上该偏移地址得到该命令数据块的写入起始地址），接着写入的输入数据是数据块长度，也就是后续数据流的字节数。数据块长度的有效值是 0 至 32，并且偏移地址加上数据块长度的和不能大于 32，如果数据块长度不为 0，则单片机必须将后续数据逐个写入 CH376。

十四. CMD_DISK_CONNECT:

该命令用于检查磁盘是否连接，不支持 SD 卡。在 USB 主机方式下，该命令可以随时查询磁盘是否连接，CH376 在命令执行完成后向单片机请求中断。如果操作状态是 USB_INT_SUCCESS，那么说明有磁盘或者 USB 设备连接。

十五. CMD_DISK_MOUNT:

该命令用于初始化磁盘并测试磁盘是否就绪。新连接的 USB 存储设备或者 SD 卡必须通过该命令进行初始化后，然后才能进行文件操作，有些 USB 存储设备可能需要多次初始化才能返回操作成功状态 USB_INT_SUCCESS。另外，在文件操作的过程中，该命令也可以随时用于测试磁盘是否就绪。

首次执行 `CMD_DISK_MOUNT` 命令如果中断状态是 `USB_INT_SUCCESS`。那么可以由 `CMD_RD_USB_DATAO` 命令获取数据，数据通常是 36 个字节，包括 USB 存储设备的特性以及厂商和产品的识别信息等。

十六. `CMD_FILE_CREATE`:

该命令用于新建文件，如果文件已经存在那么先删除再新建。在新建文件命令之前，应该先通过 `CMD_SET_FILE_NAME` 命令设置将要新建的文件的文件名，格式与 `CMD_FILE_OPEN` 命令相同，但不支持通配符。如果存在同名文件，那么该同名文件将首先被删除，然后再新建文件。如果不希望已有文件被删除，那么应该事先通过 `CMD_FILE_OPEN` 命令确认文件不存在后再新建。新建文件的文件日期和时间默认为 2004 年 1 月 1 日 0 时 0 分 0 秒，文件默认长度为 1，如果需要修改这些信息，那么可以通过 `CMD_DIR_INFO_READ` 和 `CMD_DIR_INFO_SAVE` 命令实现。

十七. `CMD_FILE_ERASE`:

该命令用于删除文件，如果文件已经被打开那么将被直接删除，否则，对于文件会自行先打开再删除，而对于子目录则必须先被打开。

对于普通文件，删除步骤如下：

1. 确认之前的文件或者目录已经关闭，否则将被直接删除，而不受步骤 2 的影响；
2. 通过 `CMD_SET_FILE_NAME` 命令设置将要被删除的文件名，不支持通配符；
3. 通过 `CMD_FILE_ERASE` 命令自行打开文件并删除。

对于子目录（或者文件）必须参照以下步骤删除：

1. 对于子目录，必须事先删除子目录中的所有文件以及下级子目录；
2. 通过 `CMD_SET_FILE_NAME` 命令设置将要被删除的子目录名，不支持通配符；
3. 通过 `CMD_FILE_OPEN` 命令打开子目录名（或者文件名）；
4. 通过 `CMD_FILE_ERASE` 命令删除步骤②中已经打开的子目录（或者文件）。

十八. `CMD_FILE_CLOSE`:

该命令用于关闭当前已经打开的文件或者目录（文件夹）。该命令需要 1 个输入数据，表明是否允许更新文件长度，为 0 则禁止更新文件长度，为 1 则允许自动更新文件长度。

打开文件或者目录（文件夹）读写完毕后，应该关闭文件。对于根目录的操作，关闭文件是必须的。对于普通文件的读操作，关闭文件是可选操作。对于普通文件的写操作，关闭文件的同时，可以选择是否由 CH376 自动更新文件长度。

十九. CMD_BYTE_LOCATE:

该命令用于以字节为单位移动当前文件指针。该命令需要输入偏移字节数，该偏移字节数是以低字节在前的 4 个字节表示的双字数据（32 位）。如果命令执行完成后中断状态是 USB_INT_SUCCESS，那么可以由 CMD_RD_USB_DATA 命令获取当前文件指针对应的绝对线性扇区号 LBA(以低字节在前的 4 个字节表示的 32 位双字数据)，如果已到文件末尾，那么该值为 0FFFFFFFFH。

文件被新建或者被重新打开时，当前文件指针都为 0，移动当前文件指针，通常用于从指定位置开始读写数据。例如，单片机希望跳过文件的前 158 字节再读写数据，那么可以用 CMD_BYTE_LOCATE 命令带上参数 158 作为偏移字节数，命令执行成功后，紧接在后面的读写操作将从第 158 字节开始。

对于写操作，如果单片机准备在原文件的尾部继续添加数据，而不希望影响前面的原有数据，那么可以指定很大的字节偏移，例如 0FFFFFFFFH，将文件指针移动原文件的末尾，以便追加数据。

二十. CMD_DISK_CAPACITY:

该命令用于查询磁盘物理容量，支持 USB 存储设备或者 SD 卡。如果命令执行完成后中断状态是 USB_INT_SUCCESS，那么可以由 CMD_RD_USB_DATA 命令获取磁盘物理容量，即总扇区数，该容量是以低字节在前的 4 个字节表示的双字数据（32 位），如果再乘以扇区大小 512，那么可以得到以字节为单位的物理总容量。

二十一. CMD_DISK_QUERY:

该命令用于查询磁盘空间信息，包括剩余空间和文件系统类型。如果命令执行完成后中断状态是 USB_INT_SUCCESS，那么可以由 CMD_RD_USB_DATA0 命令依次获取逻辑盘的总扇区数（以低字节在前的 4 个字节表示的 32 位双字数据），当前逻辑盘的剩余扇区数（以低字节在前的 4 个字节表示的 32 位双字数据），逻辑盘的 FAT 文件系统类型。

二十二. CMD_DIR_CREATE:

该命令用于新建子目录（文件夹）并打开，如果子目录已经存在那么直接打开，只支持第一级子目录，参考 EXAM9 例子可以支持新建多级子目录。

在新建子目录命令之前，应该先通过 `CMD_SET_FILE_NAM` 命令设置将要新建的子目录的目录名，格式与 `CMD_FILE_CREATE` 命令相同。如果存在同名的普通文件，那么中断状态为 `ERR_FOUND_NAME`；

如果子目录新建成功或者打开事先已经存在的子目录，那么中断状态为 `USB_INT_SUCCESS`。新建子目录的文件日期和时间等信息 `CMD_FILE_CREATE` 命令新建文件时相同，修改方法也相同，只是文件属性为 `ATTR_DIRECTORY`，并且文件长度始终为 0。

二十三. `CMD_SEC_LOCATE`:

该命令用于以扇区为单位移动当前文件指针，不支持 SD 卡。该命令需要输入偏移扇区数，该偏移扇区数是以低字节在前的 4 个字节表示的双字数据（32 位）。如果命令执行完成后中断状态是 `USB_INT_SUCCESS`，那么可以由 `CMD_RD_USB_DATAO` 命令获取当前文件指针对应的绝对线性扇区号 LBA（以低字节在前的 4 个字节表示的 32 位双字数据），如果已到文件末尾，那么该值为 `0FFFFFFFFH`。

文件被新建或者被重新打开时，当前文件指针都为 0，移动当前文件指针，通常用于从指定位置开始读写数据。例如，单片机希望跳过文件的前 18 个扇区再读写数据，那么可以用 `CMD_SEC_LOCATE` 命令带上参数 18 作为偏移扇区数，命令执行成功后，紧接在后面的读写操作将从第 18 扇区开始。对手写操作，如果单片机准备在原文件的尾部继续添加数据，而不希望影响前面的原有数据，那么可以指定很大的字节偏移，例如 `0FFFFFFFFH`，将文件指针移动原文件的末尾，以便追加数据。

二十四. `CMD_DISK_BOC_CMD`:

该命令用于对 USB 存储设备执行 BulkOnly 传输协议的命令。在执行该命令之前，单片机必须先通过 `CMD_WR_HOST_DATA` 命令向 CH376 写入相应的 CBW 包，CH376 在命令执行完成后向单片机请求中断，如果中断状态是 `USB_INT_SUCCESS` 则说明命令执行成功，对于有返回数据的操作，可以由 `CMD_RD_USB_DATAO` 命令获取返回数据。

9.2 USB 驱动程序

CH376 是一款文件管理控制芯片，RSIC-V 通过对 CH346 的驱动控制可以实现 USB 接口。其基本开发流程是，硬件初始化-硬件打开-硬件读/写操作-...-硬件关闭，其具体实现代

码在 `connect_ch376.c` 文件中。此外在过程中，为实现 USB 设备的直接管理，初始化过程中首先初始化 USB 总线，然后将 CH376 设备和驱动挂接在 USB 总线上，具体实现在 `bus_usb.c`；USB 的设备、驱动和总线结构体定义在 `dev_usb.h` 和 `bus_usb.h` 中，详见第 4 篇第 11 章。

9.2.1 connect_ch376.c 文件

在 `connect_ch376.c` 实现了 CH376 的设备初始化，其具体实现过程可根据是否使用 SDIO 或者 USB 进行。在代码 5-9-1 中是 CH376 进行初始化的过程，代码行 2 是对错误标志位的声明，为公共使用部分。

代码行 4 到 40 对应 CH376 在 SDIO 模式的初始化过程。代码行 5 到 7 分别是声明初始化过程中所需要使用到的 SDIO 总线、驱动和设备结构体变量。代码行 9 到 13、15 到 20、28 到 33 分别对应为 SDIO 总线的初始化，SDIO 驱动的初始化和 SDIO 设备的注册。代码行 22 到 26、35 到 39 分别将 SDIO 的驱动和设备绑定到 SDIO 总线上。

代码行 42 到 76 对应 CH376 在 USB 模式的初始化过程，其基本过程和在 SDIO 模式下相似。代码行 43 到 45 声明初始化 USB 总线、驱动和设备结构体变量。代码行 47 到 51、53 到 58、65 到 70 分别 USB 总线结构体初始化，USB 驱动结构体初始化和 USB 设备的注册。代码行 59 到 63 和 71 到 75 分别是将 USB 的驱动和设备绑定到先前声明的总线上。

代码 5-9-217 CH376 的初始化过程

```

01  int HwCh376Init(void){
02      x_err_t ret = EOK;
03
04  #ifdef BSP_USING_SDIO
05      static struct SdioBus sdio_bus;
06      static struct SdioDriver sdio_drv;
07      static struct SdioHardwareDevice sdio_dev;
08
09      ret = SdioBusInit(&sdio_bus,SDIO_BUS_NAME );
10      if (ret != EOK) {
11          KPrintf("Sdio bus init error %d\n", ret);
12          return -ERROR;
13      }
14
15      sdio_drv.configure = &Ch376Configure;
16      ret = SdioDriverInit(&sdio_drv, SDIO_DRIVER_NAME);

```

```
17  if (ret != EOK) {
18      KPrintf("Sdio driver init error %d\n", ret);
19      return -ERROR;
20  }
21
22  ret = SdioDriverAttachToBus( SDIO_DRIVER_NAME, SDIO_BUS_NAME);
23  if (ret != EOK) {
24      KPrintf("Sdio driver attach error %d\n", ret);
25      return -ERROR;
26  }
27
28  sdio_dev.dev_done = &sdio_done;
29  ret = SdioDeviceRegister(&sdio_dev, SDIO_DEVICE_NAME);
30  if (ret != EOK) {
31      KPrintf("Sdio device register error %d\n", ret);
32      return -ERROR;
33  }
34
35  ret = SdioDeviceAttachToBus(SDIO_DEVICE_NAME, SDIO_BUS_NAME);
36  if (ret != EOK) {
37      KPrintf("Sdio device register error %d\n", ret);
38      return -ERROR;
39  }
40  #endif
41
42  #ifdef BSP_USING_USB
43      static struct UsbBus usb_bus;
44      static struct UsbDriver usb_drv;
45      static struct UsbHardwareDevice usb_dev;
46
47      ret = UsbBusInit(&usb_bus, USB_BUS_NAME);
48      if (ret != EOK) {
49          KPrintf("USB bus init error %d\n", ret);
50          return -ERROR;
51      }
52
53      usb_drv.configure = &Ch376Configure;
54      ret = UsbDriverInit(&usb_drv, USB_DRIVER_NAME);
55      if (ret != EOK) {
56          KPrintf("USB driver init error %d\n", ret);
57          return -ERROR;
```

```

58     }
59     ret = UsbDriverAttachToBus(USB_DRIVER_NAME, USB_BUS_NAME);
60     if (ret != EOK) {
61         KPrintf("USB driver attach error %d\n", ret);
62         return -ERROR;
63     }
64
65     usb_dev.dev_done = &usb_done;
66     ret = USBDeviceRegister(&usb_dev, NONE, USB_DEVICE_NAME);
67     if (ret != EOK) {
68         KPrintf("USB device register error %d\n", ret);
69         return -ERROR;
70     }
71     ret = USBDeviceAttachToBus(USB_DEVICE_NAME, USB_BUS_NAME);
72     if (ret != EOK) {
73         KPrintf("USB device register error %d\n", ret);
74         return -ERROR;
75     }
76 #endif
77     return ret;
78 }

```

SDIO 和 USB 模式下配置过程中使用了不同结构体类型变量 `struct SdioDevDone sdio_done`、`struct UsbDevDone usb_done`，分别表示对应设备的可执行动作的函数接口。其在文件 `connect_ch376.c` 进行了 `static` 类型的声明，其声明过程中的初始化如下。

代码 5-9-218 sdio_done 的初始化过程

```

01 #ifdef BSP_USING_SDIO
02 static struct SdioDevDone sdio_done =
03 {
04     Ch376Open,
05     Ch376Close,
06     Ch376Write,
07     Ch376Read,
08 };
09 #endif

```

代码 5-9-219 usb_done 的初始化过程

```

01 #ifdef BSP_USING_USB
02 static struct UsbDevDone usb_done =
03 {

```

```

04     Ch376Open,
05     Ch376Close,
06     Ch376Write,
07     Ch376Read,
08 };
09 #endif

```

CH376 的配置是 RISC-V 通过发送总线数据来对芯片进行配置。其对应接口函数是 Ch376Configure，对应参数有 drv 和 BusConfigureInfo *configure_info，分别对应空指针 void* 和结构体 struct BusConfigureInfo *类型。

代码 5-9-220 CH376 的配置

```

01 static uint32 Ch376Configure(void *drv, struct BusConfigureInfo *configure_inf
o){
02     NULL_PARAM_CHECK(drv);
03     NULL_PARAM_CHECK(configure_info);
04
05     uint8 temp[2];
06     temp[0] = 0x57;
07     temp[1] = 0xab;
08     MdelayKTask(100);
09
10     struct BusBlockWriteParam write_param;
11     write_param.pos = 0;
12     write_param.buffer = (void *)temp;
13     write_param.size = 2;
14     BusDevWriteData(ch376.dev, &write_param);
15
16     write_param.pos = 0;
17     write_param.buffer = &configure_info->configure_cmd;
18     write_param.size = 1;
19     BusDevWriteData(ch376.dev, &write_param);
20
21     return EOK;
22 }

```

9.3 USB 设备应用

USB 设备应用实现过程如代码 5-9-5 所示，主要通过 USB 连接 U 盘，然后验证 U 盘内文件的打开、关闭和读写功能。代码行 2~6 打开一个文件。代码行 12~27 首先用 read 函

数读文件内容，然后用 write 函数写文件，最后用 close 函数关闭文件。代码行 29~44 首先用 open 函数重新打开文件，然后再进行读和关闭文件。

代码 5-9-221 USB 设备应用

```
01 void TestFs(void){
02     int fd = open(FPATH,O_RDWR|O_CREAT);
03     if(fd<0){
04         printf("fs fd open error:%d\n",fd);
05         return;
06     }
07
08     char filewords[MAX_READ_LENGTH];
09     memset(filewords,0,MAX_READ_LENGTH);
10     const char *input_words = "these words are going to write in fs\n";
11
12     int err_flag = read(fd,filewords,MAX_READ_LENGTH);
13     if(err_flag<0){
14         printf("read failed,error:%d\n",err_flag);
15         return;
16     }
17     printf("read data is \n%s\n",filewords);
18     err_flag = write(fd,input_words,strlen(input_words));
19     if(err_flag<0){
20         printf("write failed,error:%d\n",err_flag);
21         return;
22     }
23     err_flag = close(fd);
24     if(err_flag<0){
25         printf("close failed,error %d\n",err_flag);
26         return ;
27     }
28
29     fd = open(FPATH,O_RDWR);
30     if(fd<0){
31         printf("fs fd open error:%d\n",fd);
32         return;
33     }
34     err_flag = read(fd,filewords,MAX_READ_LENGTH);
35     if(err_flag<0){
36         printf("read failed,error:%d\n",err_flag);
37         return;
```

```
38     }
39     printf("read data is \n%s\n",filewords);
40     err_flag = close(fd);
41     if(err_flag<0){
42         printf("close failed,error:%d\n",err_flag);
43         return;
44     }
45
46     return;
47 }
```

第10章 TF 卡驱动

10.1 TF 卡介绍

10.1.1 TF 卡基本介绍

TF 卡也叫作 T-Flash 卡或 Micro SD 卡，是一种小型的可擦除式存储卡，广泛应用于移动设备和嵌入式系统中。下面是对 TF 卡的详细介绍：

1、物理特性：TF 卡尺寸大多约为 15*11*1mm，体积小、重量轻，易于携带和使用。

TF 卡和 SD 卡一样，拥有铜制接点，支持高速数据传输；

2、存储容量：TF 卡的存储容量通常有 128MB、256MB、512MB、1GB、2GB、4GB、8GB 等多种规格，已经有一些厂家生产的 TF 卡容量已经达到 TB 级别；

3、I/O 接口：TF 卡通常采用 SPI 或者 SDIO 等标准的接口进行数据交互。SPI 接口相对简单，只需要三个信号线（SCLK、MOSI、MISO），数据传输速度相对较慢。而 SDIO 接口是更快的接口，支持多主机等功能，但是使用的信号线更多，相对复杂；

4、文件系统：TF 卡支持实现 FAT16、FAT32、NTFS、exFAT 等多种文件系统。由于 TF 卡容量较小，一般使用较小的 FAT 系列文件系统，因为它占用的空间较小，支持的设备驱动程序也较多；

5、应用范围：TF 卡广泛应用于移动设备、数码相机、嵌入式系统、高清录像等领域。其中，由于其体积小、重量轻、存储容量大、读写速度快等优点，嵌入式系统中广泛使用 TF 卡进行数据存储和管理。

总的来说，TF 卡是一种小巧、轻便、存储容量大、读写速度快的存储卡，使用方便，广泛应用于移动设备和嵌入式系统中。需要注意的是，在使用 TF 卡时，需要根据具体的应用场景选择适合的容量和文件系统，并遵循文件系统的使用规范进行数据存储和管理，以保证数据的安全和可靠性。

10.1.2 中断说明

TF 卡驱动和 USB 驱动都采取的是 CH376 文件管理控制芯片，具体内容见 9.1.2 USB 驱动中断说明。

10.1.3 寄存器说明

TF 卡驱动和 USB 驱动都采取的是 CH376 文件管理控制芯片，具体内容见 9.1.3 USB 驱动寄存器说明。

10.2 TF 卡驱动程序

与 ARM 平台不同，RISC-V 平台的 TF 卡驱动的实现主要是通过软件模拟 SPI 协议来实现的，与硬件 SPI 等共享一类总线和设备类型。首先，通过 SPI BUS 挂接到 BUS 总线上，其中 SPI BUS 的结构体在 `bus_spi.h` 文件中进行了定义。然后，每个设备总线都关联一个 `drv` 和一个 `dev`，其中，`drv` 结构体也在 `bus_spi.h` 中进行了定义，`dev` 结构体在 `dev_spi.h` 中进行了定义。另外，SPI 驱动及设备的注册流程在 `connect_spi.c` 中进行了实现。RISC-V 的 SPI 与 ARM 的 SPI 共用 `bus_spi.h` 和 `dev_spi.h` 文件，读者可参考第 4 篇第 7 章。下面对 RISC-V 架构中 `connect_spi.c` 文件的驱动及设备注册流程进行介绍。SPI 读写 TF 卡的实现方法在 `sd_spi.c` 文件下，具体协议内容读者可自行查阅资料。

10.2.1 connect_soft_spi.c 文件

`connect_soft_spi.c` 文件中实现了软件模拟 SPI 驱动及设备的注册，该流程主要包含 3 个步骤：

- (1) 完成软件模拟 SPI 的驱动和设备到 SPI 总线的挂接（或称为绑定）；
- (2) 实现软件模拟 SPI 设备的打开、关闭及读写方法，其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植；
- (3) 实现软件模拟 SPI 设备的驱动配置，需结合软件模拟 SPI 特性及操作系统的接口定义进行相应的开发。

步骤（1）的软件模拟 SPI 驱动及设备绑定过程由代码 5-10-1 中所示。在 HwSoftSPIInit 函数中，代码行 2~6 声明了 1 个结果返回变量和 2 个结构体变量，并对结构体变量进行初始化。代码行 8 实现了软件模拟 SPI 的驱动配置。代码行 10~14 使用 BoardSoftSpiBusInit 函数进行软件模拟 SPI 驱动挂接到 SPI 总线，具体的实现过程详见代码 5-10-2。代码行 16~20 使用 BoardSoftSpiDevBend 函数进行软件模拟 SPI 设备挂接到 SPI 总线，具体的实现过程详见代码 5-10-3。

在代码 5-10-2 的 BoardSoftSpiBusInit 函数中，代码行 4~8 对 SPI 总线进行初始化；代码行 10~14 对软件模拟 SPI 驱动进行初始化；代码行 16~20 将软件模拟 SPI 驱动挂接到 SPI 总线上。在代码 5-10-3 的 BoardSoftSpiDevBend 函数中，代码行 2~6 声明了 1 个结果返回变量和 2 个结构体变量，并对结构体变量进行初始化；代码行 8~14 对结构体变量进行参数赋值；代码行 16~20 中对软件模拟 SPI 设备进行注册；代码行 22~26 将软件模拟 SPI 设备挂接到 SPI 总线。

代码 5-10-222 软件模拟 SPI 驱动绑定过程

```

01 int HwSoftSPIInit(void){
02     x_err_t ret = EOK;
03     static struct SpiBus spi_bus;
04     memset(&spi_bus, 0, sizeof(struct SpiBus));
05     static struct SpiDriver spi_driver;
06     memset(&spi_driver, 0, sizeof(struct SpiDriver));
07
08     spi_driver.configure = &(softSpiDrvConfigure);
09
10     ret = BoardSoftSpiBusInit(&spi_bus, &spi_driver);
11     if (EOK != ret) {
12         KPrintf("Board_Spi_Init error ret %u\n", ret);
13         return ERROR;
14     }
15
16     ret = BoardSoftSpiDevBend();
17     if (EOK != ret) {
18         KPrintf("Board_Spi_Init error ret %u\n", ret);
19         return ERROR;
20     }
21
22     return ret;

```

23 }

代码 5-10-223 软件模拟 SPI 驱动挂接过程

```

01 static int BoardSoftSpiBusInit(struct SpiBus *spi_bus, struct SpiDriver *spi_driver){
02     x_err_t ret = EOK;
03
04     ret = SpiBusInit(spi_bus, SOFT_SPI_BUS_NAME);
05     if (EOK != ret) {
06         KPrintf("Board_Spi_init SpiBusInit error %d\n", ret);
07         return ERROR;
08     }
09
10     ret = SpiDriverInit(spi_driver, SOFT_SPI_DRV_NAME);
11     if (EOK != ret) {
12         KPrintf("Board_Spi_init SpiDriverInit error %d\n", ret);
13         return ERROR;
14     }
15
16     ret = SpiDriverAttachToBus(SOFT_SPI_DRV_NAME, SOFT_SPI_BUS_NAME);
17     if (EOK != ret) {
18         KPrintf("Board_Spi_init SpiDriverAttachToBus error %d\n", ret);
19         return ERROR;
20     }
21
22     return ret;
23 }

```

代码 5-10-224 软件模拟 SPI 设备挂接过程

```

01 static int BoardSoftSpiDevBend(void){
02     x_err_t ret = EOK;
03     static struct SpiHardwareDevice spi_device0;
04     memset(&spi_device0, 0, sizeof(struct SpiHardwareDevice));
05     static struct SpiSlaveParam spi_slaveparam0;
06     memset(&spi_slaveparam0, 0, sizeof(struct SpiSlaveParam));
07
08     spi_slaveparam0.spi_slave_id = SOFT_SPI_DEVICE_SLAVE_ID;
09     spi_slaveparam0.spi_cs_gpio_pin = SOFT_SPI_CS0_PIN;
10     spi_slaveparam0.spi_cs_select_id = SOFT_SPI_CHIP_SELECT;
11
12     spi_device0.spi_param.spi_dma_param = NONE;

```

```

13     spi_device0.spi_param.spi_slave_param = &spi_slaveparam0;
14     spi_device0.spi_dev_done = &(soft_spi_dev_done);
15
16     ret = SpiDeviceRegister(&spi_device0,
(void *)(&spi_device0.spi_param), SOFT_SPI_DEVICE_NAME);
17     if (EOK != ret) {
18         KPrintf("DeviceInit %s error %d\n", SOFT_SPI_DEVICE_NAME, ret);
19         return ERROR;
20     }
21
22     ret = SpiDeviceAttachToBus(SOFT_SPI_DEVICE_NAME,
SOFT_SPI_BUS_NAME);
23     if (EOK != ret) {
24         KPrintf("AttachToBus %s error %d\n", SOFT_SPI_DEVICE_NAME, re
t);
25         return ERROR;
26     }
27
28     return ret;
29 }

```

步骤(2)的实现过程如代码 5-10-4 所示,该流程是一个实现软件模拟 SPI 设备的打开、关闭及读写过程。根据结构体 SpiDevDone 的定义,打开和关闭的值均为 NONE,表示该功能未启用。而读和写的值分别是 softSpiReadData 和 softSpiWriteData,表明这两个功能是打开的。

代码 5-10-225 软件模拟 SPI 设备的打开、关闭及读写

```

01 const struct SpiDevDone soft_spi_dev_done={
02     .dev_close = NONE,
03     .dev_open = NONE,
04     .dev_read = softSpiReadData,
05     .dev_write = softSpiWriteData
06 };

```

软件模拟 SPI 设备的驱动配置过程如代码 5-10-5 所示。在 softSpiDrvConfigure 函数中,代码行 2~3 进行输入参数 drv 和 configure_info 的检查。代码行 5~7 声明了 3 个变量。代码行 9~20 根据配置命令进行选择,若配置命令为 OPE_INT,则根据 softSPIinit 函数执行 SPI 驱动初始化。

代码 5-10-226 软件模拟 SPI 设备的驱动配置

```

01  static uint32 softSpiDrvConfigure(void *drv, struct BusConfigureInfo *configure
_info){
02      NULL_PARAM_CHECK(drv);
03      NULL_PARAM_CHECK(configure_info);
04
05      x_err_t ret = EOK;
06      struct SpiDriver *spi_drv = (struct SpiDriver *)drv;
07      struct SpiMasterParam *spi_param;
08
09      switch (configure_info->configure_cmd)
10      {
11      case OPE_INT:
12          softSPIinit(spi_drv,configure_info);
13          break;
14
15      case OPE_CFG:
16          break;
17      default:
18
19          break;
20      }
21
22      return ret;
23  }

```

10.3 TF 卡设备应用

参考第 9 章第 3 节 USB 设备应用中文件系统测试应用。

第11章 ETH 驱动

ETH (Ethernet) 驱动是一种网络通信子系统，它主要负责控制处理器与网络设备之间的通信，以通过以太网接口与其他网络设备进行数据传输。下面 11.1 是对 ETH 的详细介绍；11.2 介绍了 ETH 驱动的实现过程，在 RISC-V 上的 ETH（即以太网）开发方法和 ARM 类似，其功能主要在 connect_w5500.c 文件中实现。K210 芯片通过 SPI 通信协议连接 w5500 芯片，实现了 socket 通信接口；11.3 介绍了 ETH 设备应用。

11.1 ETH 介绍

11.1.1 ETH 基本介绍

1、ETH 驱动架构：ETH 驱动主要包含以太网 MAC（Media Access Control）、PHY（Physical）和 DMA（Direct Memory Access）三个核心模块。其中，以太网 MAC 负责处理数据链路层协议，PHY 则负责与物理层传输信号，DMA 则负责数据的直接内存访问；

2、以太网 MAC：以太网 MAC 模块实现了 IEEE802.3 标准，支持数据链路层协议的标准化操作，比如数据包大小限制、传输速率和 CRC 校验等。在 ETH 驱动通过以太网 MAC 模块将数据发送到网络的其他节点时，也支持多队列和多种优先级的数据包处理来提高网络性能；

3、PHY 模块：PHY 模块负责 ETH 驱动器和物理层传输信号之间的物理接口，实现了 IEEE802.3ab 标准，并支持 100 Mbps 和 10 Mbps 两种传输速率；

4、DMA 模块：DMA 模块负责数据包的直接内存访问，使数据包传输更加快速和高效，因为不需要 CPU 的介入，从而提高了处理器的效率；

5、驱动程序：为了实现网络数据的传输和通信，我们需要为其编写以太网驱动程序。ETH 驱动程序需要实现数据包的接收和传输、DMA 的配置和控制、PHY 接口和 MAC 端口的配置和控制等多个功能。

11.1.2 中断说明

W5500 中有以下 5 种中断：

1、当 SEND 完成时，产生 SENDOK 中断；

2、当 ARPTO 或者 TCPTO 超时被触发时，产生 TIMEOUT 中断；

3、无论何时，只要接收到了对方数据都会产生 RECV 中断；

4、当接收到对方的 FIN 或 FIN/ACK 包时会产生 DISCON 中断；

5、当成功与对方建立连接，且 Sn_SR 变为 SOCK_ESTABLISHED 状态时，会产生 CON 中断。

11.1.3 寄存器说明

W5500 有 1 个通用寄存器，8 个 Socket 寄存器区，以及对应每个 Socket 的收/发缓存区，每一个 Socket 通过 Socket n 寄存器区控制（ $0 \leq n \leq 7$ ）。

一．模式寄存器 Mode Register：

地址范围：0x0000 该寄存器用于 S/W 复位，ping block 模式和 PPPoE 模式。

位	参数名	描述	访问类型
b7	RST	如果该位(bit)为'1', 内部寄存器将被初始化, 它会在复位后自动清零。	R/W
b6	Reserved	保留位	R/W
b5	WOL	0: 关闭网络唤醒; 1: 开启网络唤醒;	R/W
b4	PB	0: 关闭 Ping block; 1: 启用 Ping block;	R/W
b3	PPPoE	0: 关闭 PPPoE 模式; 1: 启用 PPPoE 模式;	R/W
b2	Reserved	保留位	R/W
b1	FARP	0: 关闭强迫 ARP 模式; 1: 启用强迫 ARP 模式;	R/W
b0	Reserved	保留位	R/W

二. 网关 IP 地址寄存器:

地址范围: 0x0001–0x0004, 初始化值: 0x00, 该寄存器用来设置默认网关地址。

三. 子网掩码寄存器:

地址范围: 0x0005–0x0008, 初始化值: 0x00, 该寄存器用来设置子网掩码地址。

四. 源 MAC 地址寄存器:

地址范围: 0x0009–0x000E, 初始化值: 0x00, 该寄存器用来设置源 MAC 地址。

五. 源 IP 地址寄存器:

地址范围: 0x000F – 0x0012, 初始化值: 0x00, 该寄存器用来设置源 IP 地址。

六. 中断寄存器:

地址范围: 0x0015, 初始化值: 0x00, 中断寄存器 (IR) 指明了中断的状态。某一位为 '1' 时表示发生中断, 主机对该位写入 '1' 时, IR 的该位被清 '0'。如果 IR 不等于 '0x00', INT_n 引脚将会被拉低。直到其变为 '0x00' 时, INT_n 才会被拉高。

位	参数名	描述	访问类型
b7	CONFLICT	在收到 APR 请求时, 发现发送方 IP 与本地 IP 重复, 该位将被置 '1'	R/W
b6	UNREACH	当接收到 ICMP (目的端口不可达) 包后, 该位置 '1'	R/W
b5	PPPoE	当 PPPoE 模式下, PPPoE 连接断开时, 该位生效	R/W
b4	MP	当网络唤醒模式启用并通过 UDP 接收到 Magic Packet 网络唤醒包时, 该位生效	R/W
b3-b0	Reserved	保留位	R/W

七. 中断屏蔽寄存器:

地址范围：0x0016 初始化值：0x00，用来屏蔽中断源。每个中断屏蔽位对应中断寄存器（IR）中的一个位。

位	参数名	描述	访问类型
b7	IM_IR7	0: 关闭 IP 冲突中断 1: 启用 IP 冲突中断	R/W
b6	IM_IR6	0: 关闭目的地址不能抵达 1: 开启目的地址不能抵达	R/W
b5	IM_IR5	0: 关闭 PPPoE 关闭中断 1: 开启 PPPoE 关闭中断	R/W
b4	IM_IR4	0: 关闭 Magic Packet 中断 1: 开启 Magic Packet 中断	R/W
b3-b0	Reserved	保留位	R/W

八. Socket 中断寄存器:

地址范围：0x0017，初始化值：0x00，SIR 指明了 Socket 的中断状态。SIR 被置 1 后将一直保持为‘1’，直到 Sn_IR 被主机写‘1’清零。

位	参数名	描述	访问类型
b7-b0	Sn_INT	当 Socket n 中断触发，则 SIR 寄存器对应位变为‘1’	R/W

九. Socket 中断屏蔽寄存器:

地址范围：0x0018 初始化值：0x00，SIMR 寄存器中的每一位都对应 SIR 的相应位。当 SIMR 的一位为‘1’，而 SIR 的对应位为‘1’时，中断将被触发。

位	参数名	描述	访问类型
b7-b0	Sn_IMR	0: 关闭 Socket n 中断 1: 启用 Socket n 中断	R/W

十. 重试时间值寄存器:

地址范围：0x0019 – 0x001A，初始化值：0x07D0，RTR 配置了重传超时的时间值。

十一. 重试计数寄存器:

地址范围：0x001B，初始化值：0x08，该寄存器是设置重新传送的次数。

十二. PPP 连接控制协议请求定时寄存器:

地址范围：0x001C 初始化值：0x0028，PTIMER 设置发送 LCP Echo 请求的时间，单位时间是 25 毫秒(ms)。

十三. PPP 连接控制协议幻数寄存器:

地址范围：0x001D，初始化值：0x00，该寄存器配置了用于 LCP 回应请求的 4 字节幻数(Magic number)。

十四. W5500 PHY 配置寄存器：

地址范围：0x002E，初始化值：0b10111XXX，PHYCFGR 配置 PHY 的工作模式及 PHY 重启。此外，PHYCFGR 指明了 PHY 的状态，比如：全/半双工，速度，链接状态。

位	参数名	描述	访问类型
b7	RST	当该位为 ‘0’时，内部 PHY 重启。在 PHY 重启后，该位须设置为‘1’	R/W
b6	OPMD	1：通过 PHYCFGR 的 OPMDC[2:0]位配置； 0：通过硬件引脚配置(PMODE[2:0])；	R/W
b5-b3	OPMDC	工作模式配置位	R/W
b2	DPX	1：全双工 0：半双工	R/W
b1	SPD	1: 100Mbps based 0: 10Mbps based	R/W
b0	LNK	1: 已连接 0: 连接断开	R/W

十五. W5500 芯片版本寄存器：

只读，地址范围：0x0039 初始化值：0x04，W5500 的版本寄存器默认为 0x04。

十六. Sn_MR (Socket n 模式寄存器)：

地址范围：0x0000，初始化值：0x00，该寄存器用于配置所有 SOCKET 的选项或协议类型。

位	参数名	描述	访问类型
b7	MULTI/MFEN	UDP 多播模式 在 MACRAW 模式下开启 MAC 地址过滤	R/W
b6	BCASTB	MACRAW 和 UDP 模式下的网络阻塞	R/W
b5	ND/MC/MMB	使用无延时 ACK 多播 MACRAW 模式下的多播阻塞	R/W
b4	UCASTB/MIP6B	0: 关闭 Ping block; 1: 启用 Ping block;	R/W
b3-b0	P3-P0	该位配置 Socket n 使用的协议模式	R/W

十七. Sn_IR (Socket n 中断寄存器)：

Sn_IR 寄存器用于提供给 Socket n 中断类型信息

位	参数名	描述	访问类型
b7-b5	MULTI/MFEN	UDP 多播模式 在 MACRAW 模式下开启 MAC 地址过滤	R/W
b6	BCASTB	MACRAW 和 UDP 模式下的网络阻塞	R/W

b5	ND/MC/MMB	使用无延时 ACK 多播 MACRAW 模式下的多播阻塞	R/W
b4	UCASTB/MIP6B	0: 关闭 Ping block; 1: 启用 Ping block;	R/W
b3-b0	P3-P0	该位配置 Socket n 使用的协议模式	R/W

十八. Sn_PORT (Socket n 源端口寄存器):

地址范围: 0x0004-0x0005 初始化值: 0x0000, 该寄存器配置了 Socket n 的源端口号。

当 Socket n 工作在 TCP 或 UDP 模式下, 该寄存器生效。

十九. Sn_DHAR(Socket n 目的 MAC 地址寄存器):

地址范围: 0x0006-0x000B, 初始化值: 0xFFFFFFFFFFFF, Sn_DHAR 寄存器指示: UDP 模式下, 使用 Send_MAC 配置命令, 配置 Socket n 的目标主机 MAC 地址; 或者 CONNECT/SEND 配置命令, ARP 过程获取到的 MA 地址。

二十. Sn_DIPR (Socket 目标 IP 地址寄存器):

地址范围: 0x000C-0x000F, 初始化值: 0x00000000, Sn_DIPR 配置或指示的为 Socket n 的目标主机 IP 地址, 在 TCP/UDP 模式下生效。

二十一. Sn_DPORT (Socket n 目标端口寄存器):

地址范围: 0x0010-0x0011, 初始化值: 0x00, Sn_DPORT 配置或指示了 Socket n 的目标主机端口号, 在 TCP/UDP 模式下生效。

二十二. Sn_RXBUF_SIZE (Socket n 接收缓存大小寄存器):

地址范围: 0x001E, 初始化值: 0x02, Sn_RXBUF_SIZE 配置了 Socket n 的接收缓存大小。Socket n 接收缓存区大小可以配置为 1, 2, 4, 8 和 16Kbytes。如果配置为其他大小, 则 W5500 不能正常的从对方主机接收数据。

二十三. Sn_TXBUF_SIZE (Socket n 发送缓存大小寄存器):

地址范围: 0x001F, 初始化值: 0x02, Sn_TXBUF_SIZE 配置了 Socket n 的发送缓存大小。Socket n 发送缓存区大小可以配置为 1, 2, 4, 8 和 16Kbytes。如果配置为其他大小, 则 W5500 不能正常给对方主机发送数据。

二十四. Sn_TX_FSR (Socket n 空闲发送缓存寄存器):

只读，地址范围：0x0020-0x0021，初始化值：0x0800，Sn_TX_FSR 显示了 Socket n 发送缓存的空闲空间大小。该寄存器初始化配置为 Sn_TXBUF_SIZE 大小。如果要发送的数据长度大于 Sn_Tx_FSR 的值，在发送缓存中还没被发送的数据将会被覆盖，因此数据长度大于 Sn_Tx_FSR 的值时，W5500 将不允许写入发送缓存。

二十五. Sn_RX_RSR (Socket n 空闲接收缓存寄存器):

只读，地址范围：0x0026-0x0027，初始化值：0x0000，Sn_RX_RSR 显示了 Socket n 接收缓存中已接收和保存的数据大小。Sn_RX_RSR 不会超过 n_RXBUF_SIZE 大小，且计算的为 Socket n 接收写指针(Sn_RX_WR)和 Socket n 接收读指针之间的空间大小。

11.2 ETH 驱动程序

ETH 的硬件初始化函数如代码 5-11-1 所示，其中 w5500 是芯片型号，Wiznet 是芯片的制造商。代码行 2 对硬件重置；代码行 4~6 对 w5500 的 SPI 初始化；代码行 8 实现硬件设备、中断函数等注册；代码行 10~12 对芯片配置初始化；代码行 14 对网络初始化。

代码 5-11-227 ETH 硬件初始化

```

01 int HwWiznetInit(void) {
02     wiz_reset();
03
04     if (EOK != w5500_spi_init()) {
05         return ERROR;
06     }
07
08     wiz_spi_handler_reg();
09
10     if (EOK != wiz_chip_cfg_init()) {
11         return ERROR;
12     }
13
14     network_init();
15
16     return EOK;
17 }

```

11.3 ETH 设备应用

ETH 设备应用分为服务端和客户端，两者的操作类似。接下来以服务端的操作为例，其实现过程如代码 5-11-2 所示。代码行 4 根据不同的当 socket 状态值 getSn_SR(sn)，执行相

应的操作，若值为 SOCK_ESTABLISHED，则进行设置中断、发送数据或接收数据操作（代码行 5~29）；若值为 SOCK_CLOSE_WAIT，则执行 socket 关闭等待操作；若值为 SOCK_INIT，则执行 socket 初始化操作；若值为 SOCK_CLOSED，在执行 socket 关闭操作。

代码 5-11-228 ETH 设备应用

```

01 static int32_t wiz_server_op(uint8_t sn, uint8_t *buf, uint32_t buf_size,
02   uint16_t port, enum TCP_OPTION opt) {
03     int32_t ret = 0;
04     uint16_t size = 0, sentsize = 0;
05     switch (getSn_SR(sn)) {
06       case SOCK_ESTABLISHED:
07         if (getSn_IR(sn) & Sn_IR_CON) {
08           printf("%d:Connected\r\n", sn);
09           setSn_IR(sn, Sn_IR_CON);
10         }
11         if (opt == SEND_DATA) {
12           uint32_t sent_size = 0;
13           memset(buf,0,buf_size);
14           strcpy(buf,"The message has been recved");
15           ret = wiz_sock_send(sn, buf, buf_size);
16           if (ret < 0) {
17             wiz_sock_close(sn);
18             return ret;
19           }
20         } else if (opt == RECV_DATA) {
21           uint32_t size = 0;
22           if ((size = getSn_RX_RSR(sn)) > 0) {
23             if (size > buf_size) size = buf_size;
24             memset(buf,0,buf_size);
25             ret = wiz_sock_recv(sn, buf, size);
26             printf("Recv message: %s\n",buf);
27             return ret;
28           }
29         }
30       case SOCK_CLOSE_WAIT:
31         printf("%d:CloseWait\r\n", sn);
32         if ((ret = wiz_sock_disconnect(sn)) != SOCK_OK) return ret;
33         printf("%d:Closed\r\n", sn);
34         break;

```

```
35     case SOCK_INIT:
36         printf("%d:Listen, port [%d]\r\n", sn, port);
37         if ((ret = wiz_sock_listen(sn)) != SOCK_OK) return ret;
38         break;
39     case SOCK_CLOSED:
40         printf("%d:LBTStart\r\n", sn);
41         if ((ret = wiz_socket(sn, Sn_MR_TCP, port, 0x00)) != sn) return ret;
42         printf("%d:Opened\r\n", sn);
43         break;
44     default:
45         break;
46 }
47 return 0;
48 }
```

第12章 LCD 驱动

12.1 LCD 介绍

12.1.1 LCD 基本介绍

LCD 是一种用于控制液晶显示器像素呈现的控制器。它将输出信号转换为电压信号，从而控制液晶显示器中各个像素的颜色和亮度，实现图像的显示。液晶显示器广泛应用于计算机显示器、移动设备显示器、电视等领域。下面是对 LCD 的详细介绍：

1、基本工作原理：液晶显示器驱动器根据输入的视频信号通过液晶阵列切换电场，控制液晶的方向，从而控制液晶显示器的亮暗和颜色变化。一般而言，液晶显示器由像素点、行、列分别组成的阵列，液晶显示器的驱动器需要对信号进行处理，在合适的时间将像素点传输到正确的地址，以达到图像的需求；

2、显示控制器：液晶显示器驱动器通常包含一个显示控制器，负责控制各个像素的地址和控制数据，发送给液晶显示器。显示控制器操作过程通常与硬件特性相关联，涉及到电路、电源、信号在显示出的实际效果；

3、显示接口：液晶显示器使用各种显示接口连接。常见的显示接口包括 TFT、RGB、DVI、VGA 等，每种的特性和电气特性不同。液晶显示器驱动器需要与液晶显示器连接并且支持正确的接口以满足显示器的工作需求；

4、驱动程序：为了实现图像和信息的传输，程序员通常会编写液晶显示器驱动程序，控制液晶光栅数组的像素数据，配置显卡或者微处理器的外围设备控制器（PLD 或 FPGA 等）逐行更新像素，并且根据显得控制器的工作现状接受用户输入，完成与用户操作的交互；

5、规格、分辨率、大小：液晶显示器驱动的功能状态需要根据具体液晶的像素数量调整。一般而言，驱动器需要能够支持液晶显示器的分辨率大小，同时能够准确响应用户需要，保证实现高质量的图像渲染和图像动态效果。

Kendryte K210 支持 MCU 接口的 LCD，接口的数据总线宽度为 8 位，需要接到专用管脚 SPI0_D0 到 SPI0_D7 上，固定为 1.8V 电平。

12.1.2 中断说明

LCD 使用 SPI 总线接口实现，中断可参考第 5 篇第 8 章 SPI 驱动中中断说明描述。

12.1.3 操作说明

不同于多数外设，对于 LCD 的操作不用通过读写寄存器进行控制。对 LCD 的操作需要通过 SPI 传输命令的方式进行，LCD 操作命令均为 8 位长度，下表列出了对 LCD 的操作命令以及输入输出数据。

代码	命令名称	输入数据	输出数据	功能描述
0x00	NO_OPERATION	-	-	空命令
0x01	SOFTWARE_RESET	-	-	初始化
0x04	READ_ID	-	ID	获取 LCD 信息
0x09	READ_STATUS	-	状态信息	获取 LCD 状态
0x11	SLEEP_OFF	-		恢复到正常工作模式
0x2A	HORIZONTAL_ADDRESS_SET	x1 高八位	-	设置水平方向起止点
		x1 低八位		
		x2 高八位		
		x2 低八位		
0x2B	VERTICAL_ADDRESS_SET	y1 高八位	-	设置垂直方向起止点
		y1 低八位		
		y2 高八位		
		y2 低八位		
0x2C	MEMORY_WRITE	n 字节数据	-	将数据从内存写入显示缓存
0x2E	MEMORY_READ	-	n 字节数据	将数据从显示缓存读取到内存
0x3A	PIXEL_FORMAT_SET	像素数据格式		设置像素格式，0x55 表示 RGB565

LCD 使用 SPI0 进行通信，并且开启了 DMA 功能，在初始化 LCD 时需要通过配置 `sysctl` 控制寄存器将 1.8V IO 口配置为 SPI0，相关寄存器地址为：0x50440054，将其中第 10 位置 1，即可完成该配置工作。

在传输数据之前还需要对 SPI 通道号、SPI 工作模式和 SPI 数据格式进行配置。由于数据通过 DMA 传输，故还需对 DMA 进行配置，参数有：DMA 通道号、SPI 通道号、设备片选引脚、发送数据起始地址、发送数据长度和传输宽度。

12.2 LCD 驱动程序

首先，通过 LCD BUS 挂接到 BUS 总线上，其中 LCD BUS 的结构体在 `bus_lcd.h` 文件中进行了定义。然后，每个设备总线都关联一个 `drv` 和一个 `dev`，其中，`drv` 结构体也在 `bus_lcd.h` 中进行了定义，`dev` 结构体在 `dev_lcd.h` 中进行了定义。另外，LCD 驱动及设备的注册流程在 `connect_lcd.c` 中进行了实现。

12.2.1 bus_lcd.h

`bus_lcd.h` 文件中声明了 lcd 总线结构体 `LcdBus` 和驱动结构体 `LcdDriver`。`LcdBus` 的结构体定义如下：

代码 5-12-229 LcdBus 的结构体定义

```
struct LcdBus{
    struct Bus bus;
    void *private_data;
};
```

其中 `bus` 是通用管理总线；`private_data` 是传递参数的通用指针。

`LcdDriver` 的结构体定义如下：

代码 5-12-230 LcdDriver 的结构体定义

```
struct LcdDriver{
    struct Driver driver;
    uint32 (*configure) (void *drv, struct BusConfigureInfo *configure_info);
    void *private_data;
};
```

其中 `driver` 表示通用管理驱动；`configure` 表示配置函数；`private_data` 是传递参数的通用指针。

另外，在 `bus_lcd.h` 文件中还定义了一些函数接口，具体均在 `bus_lcd.c` 文件中实现：

- `int LcdBusInit(struct LcdBus *lcd_bus, const char *bus_name)`, 用于初始化 Lcd 总线, 其中 `lcd_bus` 表示初始化的 LCD 总线; `bus_name` 表示总线名称。通过调用 `BusRegister` 函数将 `lcd` 总线注册到 BUS 管理总线链表。
- `int LcdReleaseBus(struct LcdBus *lcd_bus)`, 用于释放 `lcd` 总线, 即将 `lcd` 总线从 BUS 管理总线链表中移除, 其中 `lcd_bus` 表示需要释放的 `lcd` 总线。一般该接口函数与 `LcdBusInit` 函数配对使用, 对 `lcd` 总线进行注册和释放。
- `int LcdDriverInit(struct LcdDriver *lcd_driver, const char *driver_name)`, 用于初始化 `lcd` 驱动, 其中 `lcd_driver` 表示需要初始化的 `lcd` 驱动; `driver_name` 表示驱动名称。该接口通过调用 `LcdDriverRegister` 函数将 `lcd` 驱动注册到 `lcd` 驱动总线。
- `int LcdDriverAttachToBus(const char *drv_name, const char *bus_name)`, 用于将 `lcd` 驱动挂接到 `lcd` BUS 总线, 其中 `drv_name` 表示驱动名称; `bus_name` 表示总线名称。
- `DriverType LcdDriverFind(const char *drv_name, enum DriverType_e drv_type)`, 用于查找挂接的 `lcd` 驱动, 其中 `drv_name` 表示驱动名称; `drv_type` 表示驱动类型。

12.2.2 dev_lcd.h

在 `dev_lcd.h` 文件中声明了 `lcd` 设备结构体 `LcdHardwareDevice`, 其具体的定义如下:

代码 5-12-231 LcdHardwareDevice 的结构体定义

```
struct LcdHardwareDevice{
    struct HardwareDev haldev;
    const struct LcdDevDone *dev_done;
    void *private_data;
};
```

其中 `haldev` 用于描述一个设备结构; `dev_done` 用于表示一个 `lcd` 设备的函数操作集; `private_data` 是传递参数的通用指针。

结构体 `LcdDevDone` 的定义如下:

代码 5-12-232 LcdDevDone 的结构体定义

```
struct LcdDevDone{
    uint32 (*open) (void *dev);
    uint32 (*close) (void *dev);
    uint32 (*write) (void *dev, struct BusBlockWriteParam *write_param);
    uint32 (*read) (void *dev, struct BusBlockWriteParam *read_param);
};
```

可以看出，该结构体是用于描述 lcd 设备的读写功能。

另外，在 dev_lcd.h 文件中还定义了一些函数接口：

- `int LcdDeviceRegister(struct LcdHardwareDevice *lcd_device, void *lcd_param, const char *device_name)`，用于初始化并注册 lcd 设备到设备管理链表上，其中 `lcd_device` 表示需要注册的 lcd 设备；`lcd_param` 表示 lcd 参数；`device_name` 表示设备名称。
- `int LcdDeviceAttachToBus(const char *dev_name, const char *bus_name)`，用于将 lcd 设备挂接到 lcd BUS 总线，其中 `dev_name` 表示设备名称；`bus_name` 表示总线名称。
- `HardwareDevType LcdDeviceFind(const char *dev_name, enum DevType dev_type)`，用于查找挂接的 lcd 设备，其中 `dev_name` 表示设备名称；`dev_type` 表示设备类型。

12.2.3 connect_lcd.c 文件

下面对 RISC-V 架构中 connect_lcd.c 文件的驱动及设备注册流程进行介绍。

connect_lcd.c 文件中实现了 LCD 驱动及设备的注册，该流程主要包含 3 个步骤：

- （1）完成 LCD 的驱动和设备到 LCD 总线的挂接（或称为绑定）；
- （2）实现 LCD 设备的打开、关闭及读写方法，其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植；
- （3）实现 LCD 设备的驱动配置，需结合 LCD 特性及操作系统的接口定义进行相应的开发。

步骤（1）的实现过程如代码 5-12-5 所示，该流程是完成一个 LCD 驱动及设备总线绑定的过程。代码行 2~4 定义了 1 个结果返回变量合 1 个结构体变量，并对结构体变量进行初始化。代码行 6~10 给变量 `lcd_dev` 分配内存，并初始化。代码行 12~15 分别对四个管脚进行 GPIO 映射。代码行 17~22 对 LCD 显示进行设置。代码行 24~26 对时钟进行设置。代码行 28~34 使用 `BoardLcdBusInit` 函数进行 LCD 驱动挂接到 LCD 总线，具体的实现过程详见代码 5-49。代码行 39~45 使用 `BoardLcdDevBend` 函数进行 LCD 设备挂接到 LCD 总线，具体的实现过程详见代码 5-12-7。代码行 47~48 设置 GPIO 模式。代码行 50 进行驱动 LCD 初始化。

在代码 5-12-6 的 `BoardLcdBusInit` 函数中，代码行 4~8 对 LCD 总线进行初始化；代码行 10~14 对 LCD 驱动进行初始化；代码行 16~20 将 LCD 驱动挂接到 LCD 总线上。在代码

5-12-7 的 BoardLcdDevBend 函数中，代码行 4~8 中对 LCD 设备进行注册；代码行 10~14 将 LCD 设备挂接到 LCD 总线。

代码 5-12-233 LCD 驱动及设备绑定

```

01  int HwLcdInit(void){
02      x_err_t ret = EOK;
03      static struct LcdDriver lcd_driver;
04      memset(&lcd_driver, 0, sizeof(struct LcdDriver));
05
06      Lcd8080DeviceType lcd_dev =
          (Lcd8080DeviceType)x_malloc(sizeof(struct Lcd8080Device));
07      if (!lcd_dev) {
08          return -ERROR;
09      }
10      memset(lcd_dev, 0, sizeof(struct Lcd8080Device));
11
12      FpioaSetFunction(BSP_LCD_DC_PIN, FUNC_GPIOHS9);
13      FpioaSetFunction(BSP_LCD_BL_PIN, FUNC_GPIOHS10);
14      FpioaSetFunction(BSP_LCD_CS_PIN, FUNC_SPI0_SS0);
15      FpioaSetFunction(BSP_LCD_WR_PIN, FUNC_SPI0_SCLK);
16
17      lcd_dev->cs = SPI_CHIP_SELECT_0;
18      lcd_dev->dc_pin = 9;
19      lcd_dev->dma_channel = DMAC_CHANNEL0;
20      lcd_dev->spi_channel = SPI_DEVICE_0;
21      lcd_dev->lcd_info.bits_per_pixel = 16;
22      lcd_dev->lcd_info.pixel_format = PIXEL_FORMAT_BGR565;
23
24      sysctl_set_power_mode(SYSCTL_POWER_BANK6, SYSCTL_POWER_V1
8);
25      sysctl_set_power_mode(SYSCTL_POWER_BANK7, SYSCTL_POWER_V1
8);
26      sysctl_set_spi0_dvp_data(1);
27
28      lcd_driver.configure = &drv_lcd_control;
29      ret = BoardLcdBusInit(&lcd_dev->lcd_bus, &lcd_driver,
          LCD_BUS_NAME, LCD_DRV_NAME);
30      if (EOK != ret){
31          KPrintf("Board_lcd_Init error ret %u\n", ret);
32          x_free(lcd_dev);
33          return -ERROR;

```

```

34     }
35
36     static struct LcdHardwareDevice lcd_device;
37     memset(&lcd_device, 0, sizeof(struct LcdHardwareDevice));
38
39     lcd_device.dev_done = &(lcd_dev_done);
40     ret = BoardLcdDevBend(&lcd_device, NONE,
                           LCD_BUS_NAME, LCD_DEVICE_NAME);
41     if (EOK != ret){
42         KPrintf("BoardLcdDevBend error ret %u\n", ret);
43         x_free(lcd_dev);
44         return -ERROR;
45     }
46
47     gpiohs_set_drive_mode(10, GPIO_DM_OUTPUT);
48     gpiohs_set_pin(10, GPIO_PV_HIGH);
49     KPrintf("LCD driver initd ...\r\n");
50     DrvLcdInit(lcd_dev);
51
52     return ret;
53 }

```

代码 5-12-234 LCD 驱动挂接过程

```

01 static int BoardLcdBusInit(struct LcdBus * lcd_bus, struct LcdDriver * lcd_driver,
02 const char *bus_name, const char *drv_name){
03
04     ret = LcdBusInit( lcd_bus, bus_name);
05     if (EOK != ret) {
06         KPrintf("Board_lcd_init LcdBusInit error %d\n", ret);
07         return -ERROR;
08     }
09
10     ret = LcdDriverInit( lcd_driver, drv_name);
11     if (EOK != ret) {
12         KPrintf("Board_LCD_init LcdDriverInit error %d\n", ret);
13         return -ERROR;
14     }
15
16     ret = LcdDriverAttachToBus(drv_name, bus_name);
17     if (EOK != ret) {

```

```

18      KPrintf("Board_LCD_init LcdDriverAttachToBus error %d\n", ret);
19      return -ERROR;
20  }
21
22      return ret;
23  }

```

代码 5-12-235 LCD 设备挂接过程

```

01  static int BoardLcdDevBend(struct LcdHardwareDevice *lcd_device, void *param,
const char *bus_name, const char *dev_name){
02      x_err_t ret = EOK;
03
04      ret = LcdDeviceRegister(lcd_device, NONE, dev_name);
05      if (EOK != ret) {
06          KPrintf("Board_LCD_init LcdDeviceInit device %s error %d\n", dev_name,
ret);
07          return -ERROR;
08      }
09
10      ret = LcdDeviceAttachToBus(dev_name, bus_name);
11      if (EOK != ret) {
12          KPrintf("B LcdDeviceAttachToBus %s error %d\n", dev_name, ret);
13          return -ERROR;
14      }
15
16      return ret;
17  }

```

步骤（2）的实现过程如代码 5-12-8 所示，该流程是一个实现 LCD 设备的打开、关闭及读写过程。根据结构体 LcdDevDone 的定义，此时打开、关闭和读功能的值为 NONE，表示未启用该功能；读功能值为 LcdWrite，表明该功能是打开的。

代码 5-12-236 LCD 设备的打开、关闭及读写

```

01  const struct LcdDevDone lcd_dev_done =
02  {
03      .open = NONE,
04      .close = NONE,
05      .write = LcdWrite,
06      .read = NONE
07  };

```

LCD 设备的驱动配置过程如代码 5-12-9 所示。在 `drv_lcd_control` 函数中，代码行 2~5 进行结构体变量声明和参数检查。代码行 7~35 根据配置信息中的配置命令，执行相应的操作。如配置命令为 `GRAPHIC_CTRL_RECT_UPDATE` 时，则使用 `DrvLcdRectUpdate` 函数设置 LCD 的矩形区域参数进行更新。

代码 5-12-237 LCD 设备的驱动配置

```

01 static uint32 drv_lcd_control(void* drv, struct BusConfigureInfo *configure_info){
02     x_err_t ret = EOK;
03     struct LcdDriver *lcd_drv = (struct LcdDriver *)drv;
04     struct DeviceRectInfo* rect_info;
05     NULL_PARAM_CHECK(drv);
06
07     switch (configure_info->configure_cmd)
08     {
09         case GRAPHIC_CTRL_RECT_UPDATE:
10             rect_info = (struct DeviceRectInfo*)configure_info->private_data;
11             if(!rect_info){
12                 SYS_ERR("GRAPHIC_CTRL_RECT_UPDATE error args");
13                 return -ERROR;
14             }
15             DrvLcdRectUpdate(rect_info->x, rect_info->y,
rect_info->width, rect_info->height);
16             break;
17         case GRAPHIC_CTRL_POWERON:
18             ret = -ENONESYS;
19             break;
20         case GRAPHIC_CTRL_POWEROFF:
21             ret = -ENONESYS;
22             break;
23         case GRAPHIC_CTRL_GET_INFO:
24             *(struct DeviceLcdInfo *)configure_info->private_data =
aiit_lcd->lcd_info;
25             break;
26         case GRAPHIC_CTRL_SET_MODE:
27             ret = -ENONESYS;
28             break;
29         case GRAPHIC_CTRL_GET_EXT:
30             ret = -ENONESYS;
31             break;
32         default:

```

```

33         SYS_ERR("drv_lcd_control cmd: %d", configure_info->configure_cm
d);
34         break;
35     }
36
37     return ret;
38 }

```

12.3 LCD 设备应用

LCD 设备应用实现过程如代码 5-12-10 所示。代码行 2~6 用于打开一个 LCD 设备驱动。在代码行 8~18 中，首先声明一个 LCD 写参结构体变量 `graph_param`；然后在 9~17 行中对 `graph_param` 参数进行赋值，目的是在 LCD 屏上设置的位置上显示“hello_world”；最好调用 `PrivWrite` 函数将上述参数写入打开的 LCD 设备。代码行 21~29 是实现像素级的 LCD 显示，循环 5 次，相当于每次都画了一条线，同样使用 `PrivWrite` 函数进行写入到 LCD 设备。

代码 5-12-238 LCD 设备应用

```

01 void TestLcd(void){
02     int lcd_fd = PrivOpen(EDU_LCD_DEV_DRIVER, O_RDWR);
03     if (lcd_fd < 0){
04         printf("open lcd fd error:%d\n", lcd_fd);
05         return;
06     }
07
08     LcdWriteParam graph_param;
09     graph_param.type = LCD_STRING_TYPE;
10     graph_param.string_info.x_pos = 0;
11     graph_param.string_info.y_pos = 0;
12     graph_param.string_info.width = 250;
13     graph_param.string_info.height = 24;
14     graph_param.string_info.font_size = 24;
15     graph_param.string_info.back_color = 0xFFFF;
16     graph_param.string_info.font_color = 0x0000;
17     graph_param.string_info.addr = "hello_world!";
18     PrivWrite(lcd_fd, &graph_param, NULL_PARAMETER);
19
20     uint16 color_select = 0xF800;
21     for (int i = 0; i < 5; i++){
22         graph_param.type = LCD_DOT_TYPE;

```

```
23     graph_param.pixel_info.x_startpos = 0;
24     graph_param.pixel_info.y_startpos = 50 * i;
25     graph_param.pixel_info.x_endpos = 320;
26     graph_param.pixel_info.y_endpos = 50 * i;
27     graph_param.pixel_info.pixel_color = &color_select;
28     PrivWrite(lcd_fd, &graph_param, NULL_PARAMETER);
29 }
30
31 PrivClose(lcd_fd);
32 }
```

第13章 CAMERA 驱动

13.1 CAMERA 介绍

13.1.1 CAMERA 基本介绍

CAMERA 是内核中负责与相机设备连接、控制和通信的模块。该模块为 XiUOS 提供了支持各种不同类型的 CAMERA 设备的能力。下面是对 CAMERA 的详细介绍：

1、CAMERA 架构: K210 CAMERA 的设计基于一组标准化的电路和协议，包括一个 MIPI CSI2 接口，一个 DMA 控制器，以及一个图像处理单元 (VIP)；

2、MIPI CSI2 接口: CAMERA 通过 MIPI CSI2 接口与相机设备进行连接和控制。MIPI CSI2 是一种用于控制相机设备的标准，并提供了高速传输和低功耗的特性；

3、DMA 控制器: DMA(Direct Memory Access) 控制器负责控制数据流向和数据的传输，以提高数据流的吞吐量和降低处理器的负载。CAMERA 的 DMA 控制器支持多通道数据传输和数据压缩，以提高处理大量数据的能力；

4、VIP: VIP (Vision Image Processing) 单元负责对于通过相机捕获的原始数据进行预处理和优化。VIP 处理器由 KPU 和 APU 两个部分组成，KPU 通过训练模型实现图像识别和分类功能，APU 负责支持不同的图像处理算法，并支持高达 1080p 的视频编解码能力；

5、支持的 CAMERA 设备: Kendryte K210 CAMERA 支持多种不同类型的相机设备，包括 MIPI CSI2 相机、标准 USB 摄像头以及红外相机等。

13.1.2 中断说明

DVP 有以下两个中断：

- 1、检测到帧开始信号后拉起 FRAME_START_INT_EN 中断；
- 2、一帧传输完成后拉起 FRAME_FINISH_INT_EN 中断。

13.1.3 寄存器说明

DVP 模块中有以下寄存器：

名称	描述	地址
DVP_CFG	DVP 配置	0x50430000
DVP_R_ADDR	输出到 AI 的数据中 R 分量的起始地址寄存器	0x50430004
DVP_G_ADDR	输出到 AI 的数据中 G 分量的起始地址寄存器	0x50430008
DVP_B_ADDR	输出到 AI 的数据中 B 分量的起始地址寄存器	0x5043000C
DVP_CMOS	CMOS 控制寄存器	0x50430010
SCCB_CFG	SCCB 控制寄存器	0x50430014
SCCB_W	SCCB 写数据寄存器	0x50430018
DVP_AXI	AXI 接口寄存器	0x5043001C
DVP_STS	DVP 状态寄存器	0x50430020
DVP_DISPLAY_ADDR	显示输出地址寄存器	0x50430028

每个寄存器位含义如下：

一. DVP_CFG(0x50430000):

位	参数名	描述	访问类型
b31-b30	RSRV	保留	-
b29-b20	RLINE_NUM	一帧图像里行的个数	R/W
b19-b12	RHREF_BURST_SIZE	一行图像对应的 burst 传输个数。	R/W
b11	RSRV	保留	-
b10	RCMOS_Y_OUT	RCMOS_YUV_RGB 为 1, RCMOS_Y_OUT 为 1 表示 DVP 会输出 Y 分量的图像, RCMOS_Y_OUT 为 0 表示 DVP 把 YUV 格式转成 RGB 格式输出。	R/W
b9	RCMOS_YUV_RGB	1 表示输入图像为 YUV 格式, 0 表示输入图像为 RGB 格式。	R/W
b8	RBURST_SIZE	DVP 输出数据的 DMA burst 方式, 1 表示 4 beats burst. 0 表示 1 beats burst。	R/W
b7-b5	RSRV	保留	-
b4	RDVP_AUTO	1 表示 1 帧图像传输完成后会自动开始开始下一帧的传输, 需要在两帧传输之间的时间间隔里配置下一帧的 DMA 地址	R/W
b3	RDVP_DISPLAY_FORMAT_EN	1 表示输出图像会按照显示格式送到显示内存地址	R/W

位	参数名	描述	访问类型
b2	RDVP_AI_FORMAT_EN	1 表示输出图像会按照 AI 格式送到 AI 内存地址，DVP 支持同时输出显示格式和 AI 格式的图像到显示和 AI 内存地址	R/W
b1	FRAME_START_INT_EN	1 表示检测到帧开始信号后拉起中断，CPU 应写 DVP_FRAME_START 来清除中断	R/W
b0	FRAME_FINISH_INT_E	1 表示一帧传输完成后拉起中断，CPU 应写 DVP_FRAME_FINISH 来清除中断	R/W

二. DVP_R_ADDR(0x50430004):

位	参数名	描述	访问类型
b31-b0	RDMA_ADDR_R	RGB 模式下 R 分量的基地址，或 YUV 只输出 Y 分量模式下的 Y 分量的基地址	R/W

三. DVP_G_ADDR(0x50430008):

位	参数名	描述	访问类型
b31-b0	RDMA_ADDR_G	G 分量的基地址。	R/W

四. DVP_B_ADDR(0x5043000C):

位	参数名	描述	访问类型
b31-b0	RDMA_ADDR_B	B 分量的基地址。	R/W

五. DVP_CMOS(0x504300010):

位	参数名	描述	访问类型
b31-b25	RSRV	保留	-
b24	CMOS_PWND		R/W
b23-b17	RSRV	保留	-
b16	CMOS_RST	CMOS 的 RST 控制信号，0 表示复位	R/W
b15-b9	RSRV	保留	-
b8	CMOS_XCLK_EN	1 表示输出 XCLK 给摄像头	R/W
b7-b0	CMOS_XCLK_DIV	XCLK 的分频系数，0 表示 PCLK 的 2 分频，1 表示 PCLK 的 4 分频，255 表示 PCLK 的 512 分频	R/W

六. SCCB_CFG(0x504300014):

位	参数名	描述	访问类型
b31-b24	SCCB_RDATA	SCCB 接口从摄像头返回的读数。	R/W
b23-b16	SCCB_HIGH	SCCB 时钟的高电平持续周期，单位是 APB clock。	R/W
b15-b8	SCCB_LOW	SCCB 时钟的低电平持续周期，单位是 APB clock。	R/W
b7-b2	RSRV	保留	-

位	参数名	描述	访问类型
b1-b0	SCCB_BYTE_NUM	写操作的 byte 数。1 表示 2 个 byte, 2 表示 3 个 byte, 3 表示 4 个 byte	R/W

七. DVP_AXI(0x50430001C):

位	参数名	描述	访问类型
b31-b15	RSRV	保留	-
b14-b12	GM_MPROT	AXI 接口的 WPROT 即保护类型, 推荐用复位值。	R/W
b11-b8	GM_MCACHE	AXI 接口的 WCACHE 即 cache 类型, 推荐用复位值。	R/W
b7-b0	GM_MLEN	AXI 接口的 burst 长度设置, 设成 0 表示 burst 1 传输, 设成 3 表示 burst 4 传输, 推荐设成 3	R/W

八. DVP_STS(0x504300020):

位	参数名	描述	访问类型
b31-b26	RSRV	保留	-
b25	SCCB_STS_WE	SCCB_STS 的写使能位, 写 SCCB_STS 时 SCCB_STS_WE 也应同时写 1。	R/W
b24	SCCB_STS	写 1 开启 SCCB 的传输, 传输过程中保持为 1, 传输完成后自动清零	R/W
b23-b18	RSRV	保留	-
b17	DVP_EN_WE	DVP_EN 的写使能位, 写 DVP_EN 时 DVP_EN_WE 也应同时写 1	R/W
b16	DVP_EN	写 1 开启 DVP 的图像采集和传输, 如果 RDVP_AUTO 为 1, 则 DVP_EN 会保持为 1 并自动采集每一帧的图像, 如果 RDVP_EN 为 0, 则 DVP_EN 会在在一帧图像传输完成后清零	
b15-b10	RSRV	保留	-
b9	DVP_FRAME_FINISH_WE	DVP_FRAME_FINISH 的写使能位, 写 DVP_FRAME_FINISH 时 DVP_FRAME_FINISH_WE 也应同时写 1	R/W
b8	DVP_FRAME_FINISH	DVP 在一帧传输完成后 DVP_FRAME_FINISH 置 1, CPU 对这个 bit 写 1 来将其清零	R/W
b7-b2	RSRV	保留	-
b1	DVP_FRAME_START_WE	写使能位, 写 DVP_FRAME_START 时 DVP_FRAME_START_WE 也应同时写 1	R/W
b0	DVP_FRAME_START	DVP 在检测到一帧开始后 DVP_FRAME_START 置 1, CPU 对这个 bit 写 1 来将其清零	R/W

九. DVP_DISPLAY_ADDR(0x504300028):

位	参数名	描述	访问类型
b31-b0	RDMA_ADDR_DISPLAY	显示内存的基地址	R/W

13.2 CAMERA 驱动程序

CAMERA 设备通过 CAMERA BUS 挂接到 BUS 总线上，其中 CAMERA BUS 的结构体在 bus_camera.h 文件中进行了定义。每个设备总线都关联一个 drv 和一个 dev，其中，drv 结构体也在 bus_camera.h 中进行了定义，dev 结构体在 dev_camera.h 中进行了定义。另外，CAMERA 驱动及设备的注册流程在 connect_dvp.c 中进行了实现。接下来，针对这 3 个 CAMERA 相关的具体结构和实现流程进行详细介绍。

13.2.1 bus_camera.h 文件

bus_camera.c 文件中声明了 CAMERA 总线结构体 CameraBus 和驱动结构体 CameraDriver。CameraBus 的结构体定义如下：

代码 5-13-239 CameraBus 的结构体定义

```
struct CameraBus{
    struct Bus bus;
    void *private_data;
};
```

其中 bus 是通用管理总线；private_data 是传递参数的通用指针。

CameraDriver 的结构体定义如下：

代码 5-13-240 CameraDriver 的结构体定义

```
struct CameraDriver{
    struct Driver driver;
    uint32 (*configure) (void *drv, struct BusConfigureInfo *configure_info);
    void *private_data;
};
```

其中 driver 表示通用管理驱动；configure 表示配置函数；private_data 是传递参数的通用指针。

另外，在 bus_camera.h 文件中还定义了一些函数接口，具体均在 bus_camera.c 文件中实现：

- int CameraBusInit(struct CameraBus *camera_bus, const char *bus_name)，用于初始化 CAMERA 总线，其中 camera_bus 表示初始化的 CAMERA 总线；bus_name 表示总线名称。通过调用 BusRegister 函数将 CAMERA 总线注册到 BUS 管理总线链表。

- `int CameraReleaseBus(struct CameraBus *camera_bus)`, 用于释放 CAMERA 总线, 即将 CAMERA 总线从 BUS 管理总线链表中移除, 其中 `camera_bus` 表示需要释放的 CAMERA 总线。一般该接口函数与 `CameraBusInit` 函数配对使用, 对 CAMERA 总线进行注册和释放。
- `int CameraDriverInit(struct CameraDriver *camera_driver, const char *driver_name)`, 用于初始化 CAMERA 驱动, 其中 `camera_driver` 表示需要初始化的 CAMERA 驱动; `driver_name` 表示驱动名称。该接口通过调用 `CameraDriverRegister` 函数将 CAMERA 驱动注册到 CAMERA 驱动总线。
- `int CameraDriverAttachToBus(const char *drv_name, const char *bus_name)`, 用于将 CAMERA 驱动挂接到 CAMERA BUS 总线, 其中 `drv_name` 表示驱动名称; `bus_name` 表示总线名称。
- `DriverType CameraDriverFind(const char *drv_name, enum DriverType_e drv_type)`, 用于查找挂接的 CAMERA 驱动, 其中 `drv_name` 表示驱动名称; `drv_type` 表示驱动类型。

13.2.2 dev_camera.h 文件

在 `dev_camera.h` 文件中声明了 CAMERA 设备结构体 `CameraHardwareDevice`, 其具体的定义如下:

代码 5-13-241 CameraHardwareDevice 的结构体定义

```
struct CameraHardwareDevice{
    struct HardwareDev haldev;
    const struct CameraDevDone *camera_dev_done;
};
```

其中 `haldev` 用于描述一个设备结构; `camera_dev_done` 用于表示一个 CAMERA 设备的函数操作集。

结构体 `CameraDevDone` 的定义如下:

代码 5-13-242 CameraDevDone 的结构体定义

```
struct CameraDevDone{
    uint32 (*dev_open) (void *camera_device);
    uint32 (*dev_close) (void *camera_device);
    uint32 (*dev_write) (void *camera_device, struct BusBlockWriteParam *msg);
    uint32 (*dev_read) (void *camera_device, struct BusBlockReadParam *msg);
```

```
};
```

可以看出，该结构体是用于描述 CAMERA 设备的打开、关闭、读写功能。

另外，在 dev_camera.h 文件中还定义了一些函数接口：

- `int CameraDeviceRegister(struct CameraHardwareDevice *camera_device, void *camera_param, const char *device_name)`，用于初始化并注册 CAMERA 设备到设备管理链表上，其中 `camera_device` 表示需要注册的 CAMERA 设备；`camera_param` 表示 CAMERA 参数；`device_name` 表示设备名称。
- `int CameraDeviceAttachToBus(const char *dev_name, const char *bus_name)`，用于将 CAMERA 设备挂接到 CAMERA BUS 总线，其中 `dev_name` 表示设备名称；`bus_name` 表示总线名称。
- `HardwareDevType CameraDeviceFind(const char *dev_name, enum DevType dev_type)`，用于查找挂接的 CAMERA 设备，其中 `dev_name` 表示设备名称；`dev_type` 表示设备类型。

13.2.3 connect_dvp.c 文件

`connect_dvp.c` 文件中实现了 CAMERA 驱动及设备的注册，该流程主要包含 3 个步骤：

- （1）完成 CAMERA 的驱动和设备到 CAMERA 总线的挂接（或称为绑定）；
- （2）实现 CAMERA 设备的打开、关闭及读写方法，其具体实现需结合 SoC 厂商提供的 SDK 源码进行相应的移植；
- （3）实现 CAMERA 设备的驱动配置，需结合 CAMERA 特性及操作系统的接口定义进行相应的开发。

步骤（1）的 CAMERA 驱动及设备绑定过程由代码 5-13-5 中所示。在 `HwDvpInit` 函数中，代码行 2~3 声明了 1 个结果返回变量和 2 个结构体变量，并对结构体变量进行初始化。代码行 8 对 CAMERA 设备驱动进行配置。代码行 10~14 使用 `BoardCameraBusInit` 函数进行 CAMERA 驱动挂接到 CAMERA 总线，具体的实现过程详见代码 5-13-6。代码行 16~20 使用 `BoardCameraDevBend` 函数进行 CAMERA 设备挂接到 CAMERA 总线，具体的实现过程详见代码 5-13-7。

在代码 5-13-6 的 `BoardCameraBusInit` 函数中，代码行 4~9 对 CAMERA 总线进行初始化；代码行 11~16 对 CAMERA 驱动进行初始化；代码行 18~22 将 CAMERA 驱动挂接到 C

AMERA 总线上。在代码 5-56 的 BoardCameraDevBend 函数中，代码行 2~5 声明了 1 个结果返回变量和 2 个结构体变量，并对结构体变量进行初始化；代码行 7~11 中对 CAMERA 设备进行注册；代码行 13~17 将 CAMERA 设备挂接到 CAMERA 总线。

代码 5-13-243 CAMERA 驱动绑定过程

```
01 int HwDvpInit(void){
02     x_err_t ret = EOK;
03     static struct CameraBus camera_bus;
04     memset(&camera_bus, 0, sizeof(struct CameraBus));
05     static struct CameraDriver camera_driver;
06     memset(&camera_driver, 0, sizeof(struct CameraDriver));
07
08     camera_driver.configure = dvpDrvConfigure;
09
10     ret = BoardCameraBusInit(&camera_bus, &camera_driver);
11     if (EOK != ret){
12         KPrintf("board_camera_Init error ret %u\n", ret);
13         return ERROR;
14     }
15
16     ret = BoardCameraDevBend();
17     if (EOK != ret){
18         KPrintf("board_camera_Init error ret %u\n", ret);
19         return ERROR;
20     }
21
22     return ret;
23 }
```

代码 5-13-244 CAMERA 驱动挂接过程

```
01 static int BoardCameraBusInit(struct CameraBus *camera_bus,
02 struct CameraDriver *camera_driver){
03
04     camera_bus->private_data = (void *)&camera_dev_done;
05     ret = CameraBusInit(camera_bus, CAMERA_BUS_NAME);
06     if (EOK != ret){
07         KPrintf("board_camera_init CameraBusInit error %d\n", ret);
08         return ERROR;
09     }
```

```

10
11     camera_driver->private_data = (void *)&camera_dev_done;
12     ret = CameraDriverInit(camera_driver, CAMERA_DRV_NAME);
13     if (EOK != ret){
14         KPrintf("board_camera_init CameraDriverInit error %d\n", ret);
15         return ERROR;
16     }
17
18     ret = CameraDriverAttachToBus(CAMERA_DRV_NAME,
CAMERA_BUS_NAME);
19     if (EOK != ret){
20         KPrintf("board_camera_init CameraDriverAttachToBus error %d\n", re
t);
21         return ERROR;
22     }
23
24     return ret;
25 }

```

代码 5-13-245 CAMERA 设备挂接过程

```

01 static int BoardCameraDevBend(void){
02     x_err_t ret = EOK;
03     static struct CameraHardwareDevice camera_device;
04     memset(&camera_device, 0, sizeof(struct CameraHardwareDevice));
05     camera_device.camera_dev_done = &camera_dev_done;
06
07     ret = CameraDeviceRegister(&camera_device,
NONE, CAMERA_DEVICE_NAME);
08     if (EOK != ret){
09         KPrintf("CameraDeviceInit %s error %d\n", CAMERA_DEVICE_NAM
E, ret);
10         return ERROR;
11     }
12
13     ret = CameraDeviceAttachToBus(CAMERA_DEVICE_NAME,
CAMERA_BUS_NAME);
14     if (EOK != ret){
15         KPrintf("AttachToBus %s error %d\n", CAMERA_DEVICE_NAME, re
t);
16         return ERROR;
17     }

```

```

18
19     return ret;
20 }

```

步骤（2）的实现过程如代码 5-13-8 所示，该流程是一个实现 CAMERA 设备的打开、关闭及读写过程。根据结构体 CameraDevDone 的定义，设备写功能为 NONE，表示未启用。而打开、关闭和读的值分别是 dvpOpen、dvpClose 和 dvpRead，表明这三个功能是打开的。

代码 5-13-246 CAMERA 设备的打开、关闭及读写

```

01 static const struct CameraDevDone camera_dev_done =
02     {
03         .dev_open = dvpOpen,
04         .dev_close = dvpClose,
05         .dev_write = NONE,
06         .dev_read = dvpRead,
07     }

```

SPI 设备的驱动配置过程如代码 5-13-9 所示。在 DvpDrvConfigure 函数中，代码行 4~23 根据配置命令进行选择，若配置命令为 OPE_CFG，则进行驱动配置；若配置命令为 REG_SCCB_READ，则进行读寄存器操作；若配置命令为 REG_SCCB_WRITE，则进行写寄存器操作。

代码 5-13-247 CAMERA 设备的驱动配置

```

01 static uint32 DvpDrvConfigure(void *drv, struct BusConfigureInfo *args){
02     x_err_t ret = EOK;
03
04     int cmd_type = args->configure_cmd;
05     struct CameraCfg* tmp_cfg;
06     switch (cmd_type)
07     {
08     case OPE_INT:
09         break;
10     case OPE_CFG:
11         tmp_cfg = (struct CameraCfg *)args->private_data;
12         sensorConfigure(tmp_cfg);
13         dvp_set_image_size(tmp_cfg->output_w, tmp_cfg->output_h);
14         break;
15     case REG_SCCB_READ:
16         readDvpReg(drv, (struct DvpRegConfigureInfo *)args->private_data);
17         break;

```

```

18     case REG_SCCB_WRITE:
19         writeDvpReg(drv, (struct DvpRegConfigureInfo *)args->private_data);
20         break;
21     default:
22         break;
23     }
24
25     return ret;
26 }

```

13.3 CAMERA 设备应用

CAMERA 设备应用实现过程如代码 5-13-10 所示。在测试函数 TestCamera 中，代码行 2~6 是根据传入命令的参数设置从 CAMERA 刷新图像的帧数。代码行 8 和 9 分别打开了 CAMERA 和 LCD 屏幕的设备文件（打开文件失败的处理部分已被省略）。代码行 11~24 通过 PrivIoctl 函数（即 IO 控制函数）对打开的 CAMERA 设备进行配置。代码行 26 至 28 通过 PrivRead 函数设置 CAMERA 缓存地址并开始读取，读取过程在中断中自动执行。代码行 30 至 53 分别实现了屏幕的初始化并将 CAMERA 缓存数据输送到 LCD 屏幕上。代码行 55~57 分别用于关闭 CAMERA 和 LCD 及打印输出。

代码 5-13-248 CAMERA 设备应用

```

01 void TestCamera(int argc, char *argv[]){
02     int frame_counter = 10000;
03     if (argc > 1){
04         frame_counter = atoi(argv[1]);
05     }
06     printf("This test will refresh %d frames\n", frame_counter);
07
08     int camera_fd = PrivOpen(CAMERA_DEV_DRIVER, O_RDWR);
09     int lcd_fd = PrivOpen(CAMERA_LCD_DEV_DRIVER, O_RDWR);
10
11     struct PrivIoctlCfg ioctl_cfg;
12     ioctl_cfg.ioctl_driver_type = CAMERA_TYPE;
13     struct CameraCfg camera_cfg ={
14         .gain_manu_enable = 0,
15         .gain = 0x00,
16         .window_w = 800,
17         .window_h = 600,
18         .output_w = IMAGE_WIDTH,

```

```

19         .output_h = IMAGE_HEIGHT,
20         .window_xoffset = 0,
21         .window_yoffset = 0
22     };
23     ioctl_cfg.args = &camera_cfg;
24     PrivIoctl(camera_fd, OPE_CFG, &ioctl_cfg);
25
26     ioctl_cfg.args = (void *)image_buff;
27     PrivRead(camera_fd, image_buff, NULL_PARAMETER);
28     printf("address buff is %x\n", image_buff);
29
30     LcdWriteParam graph_param;
31     graph_param.type = LCD_DOT_TYPE;
32
33     uint16_t back_color[LCD_SIZE];
34     memset(back_color, 0, sizeof(back_color));
35     for (int i = 0; i < LCD_SIZE; i++){
36         graph_param.pixel_info.pixel_color = &back_color;
37         graph_param.pixel_info.x_startpos = 0;
38         graph_param.pixel_info.y_startpos = i;
39         graph_param.pixel_info.x_endpos = LCD_SIZE - 1;
40         graph_param.pixel_info.y_endpos = i;
41         PrivWrite(lcd_fd, &graph_param, NULL_PARAMETER);
42     }
43
44     while (frame_counter--){
45         for (int i = 0; i < IMAGE_HEIGHT; i++){
46             graph_param.pixel_info.pixel_color = image_buff + i * IMAGE_
WIDTH;
47             graph_param.pixel_info.x_startpos = 0;
48             graph_param.pixel_info.y_startpos = i + (LCD_SIZE - IMAGE_H
EIGHT) / 2;
49             graph_param.pixel_info.x_endpos = IMAGE_WIDTH - 1;
50             graph_param.pixel_info.y_endpos = i + (LCD_SIZE - IMAGE_HE
IGHT) / 2;
51             PrivWrite(lcd_fd, &graph_param, NULL_PARAMETER);
52         }
53     }
54
55     PrivClose(lcd_fd);
56     PrivClose(camera_fd);

```

```
57     printf("The camera test is finished successfully\n");  
58 }
```

第6篇 感联知控框架开发

根据功能分别对应划分为四个应用框架，分别为“全面感知”、“泛在互联”、“实时认知”、“精准控制”，每个应用框架相对独立且和其他模块解耦。

“全面感知”框架可以帮助用户更方便采集外接传感器数据。传统的嵌入式开发在应用开发时无法绕过传感器设备的多样性，进而增加了传感器应用的开发难度和周期。该模块可以以用户为中心，采用了以物理量为中心的抽象方式，在开发应用时只需要考虑所需感知的物理量种类，无需把重点放在实际采集物理量的传感器设备。这种抽象方式有效地隐藏了底层传感器设备的硬件细节，对外提供统一的数据采集接口，可以简化传感器应用与驱动的开发。

“泛在互联”框架以网络适配器为中心的抽象方式，发送与接收数据时，只需要像 `Linux socket` 网络通信一样，仅关注发送与接收的数据，而不用把重点放在所使用的具体协议上。这种抽象方式有效屏蔽了种类繁多的网络协议细节，对外提供统一的网络接口，简化了基于嵌入式操作系统的网络开发。一个网络协议设备，被抽象为一个 `Adapter`，采用类似面向对象的方法，针对不同的网络协议，扩展其数据成员与接口，从而为种类繁多的网络协议实现统一的管理架构。

“实时认知”框架的关键是将现有的深度学习等智能算法映射到算力/存储能力/通信能力有限的设备上去执行，并达到实时认知识别的要求。针对运行 `RTOS` 嵌入式设备来说，在硬件资源有限的情况下，提供一个简易部署的子框架尤为重要，可以快速地应用在轻量化的 `AI` 场景中。同时针对运行 `Linux` 的嵌入式设备来说，目前通常都有其硬件加速，例如 `Hi3559A`、`MLU220` 等芯片，通常情况需要在云端将特定场景应用的模型训练好利用各个芯片厂商的 `SDK` 部署到边缘计算设备上，达到边缘硬件加速的目的。针对工业不同场景应用，可将场景归类，模型归类，建立相应的模型库，再在边缘计算端侧，抽象统一的利用硬件加速部署模型的软件接口，屏蔽底层不同硬件不同厂商带来的问题，从而达到软件复用、模型复用、场景复用的效果。

“精准控制”框架从“控制需求”本身出发，同时面向专业用户和非专业用户，通过对“控制需求”本身和复杂的工业控制流程进行深入分析，通过软件定义的方式，提出以“控制元”为核心的控制流程。具体地，本设计通过解耦复杂的工业控制流程，将工业生产中的

各种复制工业控制流程分解为各种类型的“控制元”命令，这些“控制元”的命令以软件 API 的形式交互给用户使用，从而屏蔽了以 PLC 为中心的各种控制器的巨大差异。

第1章 “全面感知”实践

1.1 “全面感知” 框架介绍

多数嵌入式操作系统对传感器的抽象采用以物理传感器设备为中心的方式，在应用开发时无法绕过传感器设备的多样性，进而增加了传感器应用的开发难度和周期。这种抽象带来的缺陷在面对一些可以同时采集多种物理量的传感器（如温湿度传感器）时尤为突出，因为应用开发者不得不考虑每种传感器设备的采集数据的能力。

XiUOS 的“全面感知”框架以用户为中心，采用了以物理量为中心的抽象方式，在开发应用时只需要考虑所需感知的物理量种类，无需把重点放在实际采集物理量的传感器设备。这种抽象方式有效地隐藏了底层传感器设备的硬件细节，对外提供统一的数据采集接口，可以简化传感器应用与驱动的开发。为实现以物理量为中心的抽象，对传感器设备进行两层抽象：

（1）一个物理传感器测量一种物理量的能力（ability）被抽象为一个 SensorQuantity 结构；

（2）一个物理传感器本身被抽象为一个 SensorDevice 结构。

其中 SensorQuantity 被设计为可以采用类似面向对象的方法、针对不同物理量扩展其数据成员与接口，从而为众多不同性质的物理量实现统一的管理架构。在应用开发的过程中只需要使用对应物理量的 SensorQuantity 实例，无需关心传感器的硬件细节，从而实现数据采样功能与底层硬件的解耦。

从关联关系上来看，一个 SensorQuantity 对应一个 SensorDevice，一个 SensorDevice 对应一个或多个 SensorQuantity。例如，对于一个可以测量温度与湿度的传感器设备，该设备唯一对应一个 SensorDevice 结构，而该设备测量温度与湿度的能力分别对应一个 SensorQuantity 结构。框架图如下：

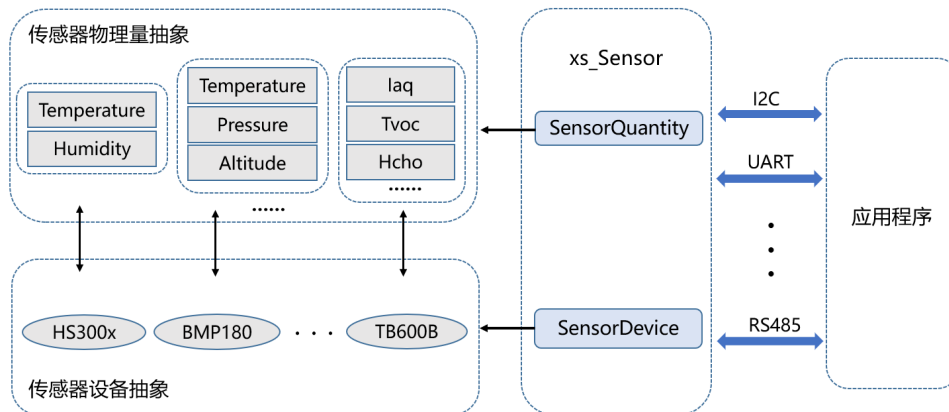


图 6-187-1 “全面感知”框架结构示意图

下面介绍 XiUOS 的“全面感知”框架关键数据结构定义和解析，具体详见 sensor.h 文件。主要介绍 SensorQuantity 和 SensorDevice 结构体的定义，以及一些接口函数。

1.1.1 SensorQuantity 结构体

代码 6-1-1 SensorQuantity 结构体定义

```
struct SensorQuantity {
    char *name;
    enum SensorQuantityType type;
    struct SensorQuantityValue value;
    struct SensorDevice *sdev;
    int32_t (*ReadValue)(struct SensorQuantity *quant);
    int32_t (*ReadDecimalPoint)(struct SensorQuantity *quant);
    struct DoublelistNode quant_link;
    struct DoublelistNode link;
};
```

其中，name 成员是一个可读的名字，用于唯一标识一个 SensorQuantity 结构。type 成员表示该 SensorQuantity 可测量的物理量，用一个枚举变量表示：

代码 6-1-2 SensorQuantityType 枚举变量

```
enum SensorQuantityType {
    SENSOR_QUANTITY_CO2 = 0,
    SENSOR_QUANTITY_TEMP,
    SENSOR_QUANTITY_HUMI,
    SENSOR_QUANTITY_HCHO,
    SENSOR_QUANTITY_CO,
    SENSOR_QUANTITY_PM,
    SENSOR_QUANTITY_VOICE,
    SENSOR_QUANTITY_CH4,
```

```

    SENSOR_QUANTITY_IAQ,
    SENSOR_QUANTITY_TVOC,
    SENSOR_QUANTITY_WINDSPEED,
    SENSOR_QUANTITY_WINDDIRECTION,
    SENSOR_QUANTITY_ALTITUDE,
    SENSOR_QUANTITY_O3,
    SENSOR_QUANTITY_O2,
    SENSOR_QUANTITY_NO2,
    SENSOR_QUANTITY_SO2,
    SENSOR_QUANTITY_NH3,
    SENSOR_QUANTITY_CH20,
    SENSOR_QUANTITY_C2H5OH,
    SENSOR_QUANTITY_AQS,
    SENSOR_QUANTITY_PM1,
    /* ..... */
    SENSIR_QUANTITY_INTEGRATION,
    SENSOR_QUANTITY_END,
};

```

SensorQuantity 结构体中的 value 成员记录与该 SensorQuantity 相关的值，包括结果的小数位数、采集历史的最大值和最小值、国家标准的最大值和最小值、最近一次采集的值。value 的结构体类型 SensorQuantityValue 定义如下：

代码 6-1-3 SensorQuantityValue 结构体定义

```

struct SensorQuantityValue {
    uint8_t decimal_places;
    uint32_t last_value;
    uint32_t min_value;
    uint32_t max_value;
    uint32_t min_std;
    uint32_t max_std;
};

```

SensorQuantity 结构体中的 sdev 成员表示该 SensorQuantity 所属的 SensorDevice 结构。ReadValue 指向 SensorQuantity 的解析函数，根据每个传感器模块的协议不同而不同，解析结果会存入 value 成员的 last_value 中。最后，在系统中每种物理量的 SensorQuantity 会注册到两个链表中，quant_link 注册到所属的 SensorDevice 下，link 注册到对应的物理量链表，如二氧化碳浓度 SensorQuantity 链表、温度 SensorQuantity 链表等。

1.1.2 SensorDevice 结构体

代码 6-1-4 SensorDevice 结构体定义

```

struct SensorDevice {
    char *name;                /* Name of sensor */
    struct SensorProductInfo *info; /* Sensor model info */
    struct SensorDone *done;
    int fd;                    /* File descriptor */
    int status;                /* Sensor work mode */
    uint8 buffer[SENSOR_RECEIVE_BUFFSIZE]; /* Buffer for read data */
    int ref_cnt;               /* Reference count */
    DoubleLinklistType quant_list; /* Sensor quantity link */
    struct SysDoubleLinklistNode link; /* Sensors link node */
};

```

其中，name 成员记录传感器设备在系统中的名字，用于唯一标识一个 SensorDevice 结构。info 成员记录传感器设备的一些属性信息，包括传感器的采集能力 ability、厂家名 vendor name 与产品型号 model name，其中 ability 用一个位图表示该传感器设备可以测量的物理量：

代码 6-1-5 传感器的采集能力 ability

```

#define SENSOR_ABILITY_CO2      ((uint32_t)(1 << SENSOR_QUANTITY_CO2))
#define SENSOR_ABILITY_TEMP    ((uint32_t)(1 << SENSOR_QUANTITY_TEMP))
#define SENSOR_ABILITY_HUMI    ((uint32_t)(1 << SENSOR_QUANTITY_HUMI))
#define SENSOR_ABILITY_HCHO    ((uint32_t)(1 << SENSOR_QUANTITY_HCHO))
#define SENSOR_ABILITY_CO      ((uint32_t)(1 << SENSOR_QUANTITY_CO))
#define SENSOR_ABILITY_PM      ((uint32_t)(1 << SENSOR_QUANTITY_PM))
#define SENSOR_ABILITY_VOICE   ((uint32_t)(1 << SENSOR_QUANTITY_VOICE))
/* ..... */

struct SensorProductInfo {
    uint32_t ability;          /* Bitwise OR of sensor ability */
    const char *vendor_name;
    const char *model_name;
};

```

SensorDevice 结构体中的 done 成员包含统一的、类似文件系统的 API，用于对传感器进行实际的数据读写。在使用一个传感器前后需要打开（open）/关闭（close）该传感器，read、write 分别用于从传感器接收数据与向传感器发送数据，ioctl 用于配置传感器工作模式（如主动上报或一问一答）。涉及的 SensorDone 结构体如下：

代码 6-1-6 SensorDone 结构体

```

struct SensorDone {
    int (*open)(struct SensorDevice *sdev);
    int (*close)(struct SensorDevice *sdev);
    x_size_t (*read)(struct SensorDevice *sdev, size_t len);
    int (*write)(struct SensorDevice *sdev, const void *buf, size_t len);
    int (*ioctl)(struct SensorDevice *sdev, int cmd);
};

```

SensorDevice 结构体中的 fd 成员表示用于与传感器进行通信的设备的文件描述符。status 成员标识传感器的工作模式，具体如下：

代码 6-1-7 传感器的工作模式

```

/* Sensor quantity report mode */
#define SENSOR_DEVICE_PASSIVE    0x00
#define SENSOR_DEVICE_ACTIVE    0x01

```

SensorDevice 结构体中的 buffer 成员用来存放从传感器中读取到的数据。ref_cnt 成员记录 SensorDevice 被打开的次数，当该值为 0 时传感器将被关闭。quant_list 链表中注册了该传感器支持采集的物理量。最后，系统中所有注册过的传感器设备被组织成一个双链表，即 link 成员。

1.1.3 感知框架接口函数

另外，在 sensor.h 文件中还定义了一些函数接口：

- struct SensorQuantity *SensorQuantityFind(const char *name, enum SensorQuantityType type)，用于查找一个具体的传感器物理量，其中 name 是物理量的名称；type 是物理量的类型。
- int SensorQuantityRegister(struct SensorQuantity *quant)，用于注册传感器物理量，将其加入到链表中，其中 quant 表示需要操作的物理量指针。
- int SensorQuantityUnregister(struct SensorQuantity *quant)，用于注销一个传感器物理量，其中 quant 表示需要操作的物理量指针。
- int SensorQuantityOpen(struct SensorQuantity *quant)，用于打开一个传感器物理量，其中 quant 表示需要操作的物理量指针。
- int SensorQuantityClose(struct SensorQuantity *quant)，用于关闭一个传感器物理量，其中 quant 表示需要操作的物理量指针。

- `int SensorQuantityReadValue(struct SensorQuantity *quant)`, 用于读取一个传感器物理量的数据, 其中 `quant` 表示需要操作的物理量指针。
- `int SensorQuantityReadDecimalPoint(struct SensorQuantity *quant)`, 用于读取小数点所在位数, 其中 `quant` 表示需要操作的物理量指针。
- `int SensorQuantityControl(struct SensorQuantity *quant, int cmd)`, 用于配置传感器物理量, 其中 `quant` 表示需要操作的物理量指针; `cmd` 表示配置命令。

1.2 ARM 架构的传感器

1.2.1 空气质量传感器

空气质量传感器用于对空气中的挥发性有机化合物进行监控, 同时也收集温度、湿度等信息。TB600B-IAQ-10 (设备型号) 传感器采用电化学领域体积很小的德国 EC Sense 固态聚合物传感器, 通过 UART 数字式信号输出。具体的空气质量采集过程体现在 `tb600b_iaq.c` 和 `iaq_tb600b_iaq10.c` 文件中, 其中 `tb600b_iaq.c` 实现了 TB600B-IAQ-10 传感器设备初始化, 包括打开、读、写等操作, 并将其注册到 XiUOS 的感框架; `iaq_tb600b_iaq10.c` 用于查找到空气质量传感器, 并进行数据读取。

在使用空气质量传感器之前, 需要进行一些配置, 在终端输入 “`make BOARD=hc32f4a0 menuconfig`” 命令, 进入编译配置菜单, 如图 6-1-2 所示。

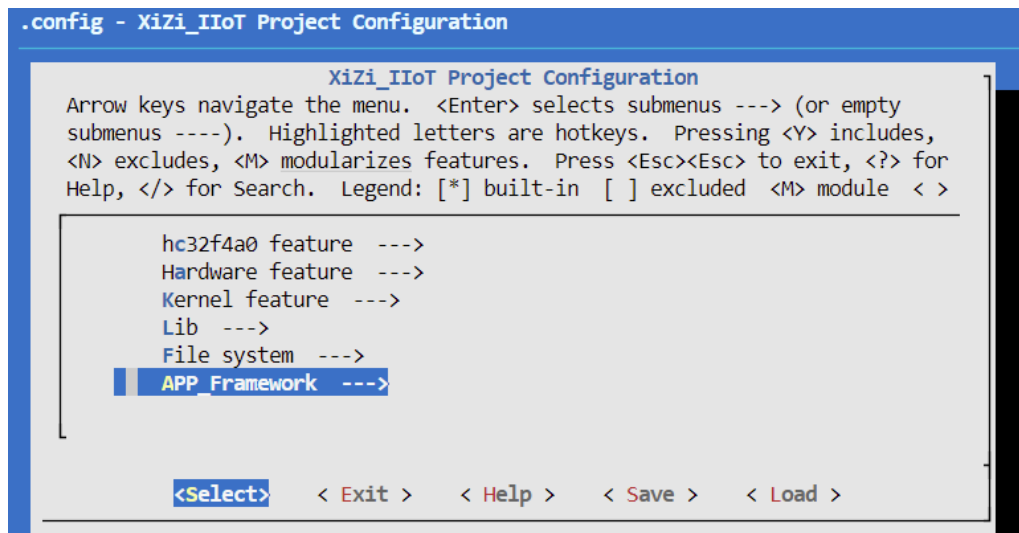


图 6-1-2 空气质量传感器相关配置

首先, 配置 APP_Framework 中的 Framework 模块。按照 APP_Framework->Framework ->support sensor framework 顺序选中 support sensor framework; 键入回车键, 进入 support

sensor framework 配置菜单，选中 Using IAQ sensor device；键入回车键，进入 Using IAQ sensor device 配置菜单，选中 Using TB600B IAQ10，配置 IAQ 传感器设备信息，具体如图 6-1-3 所示：

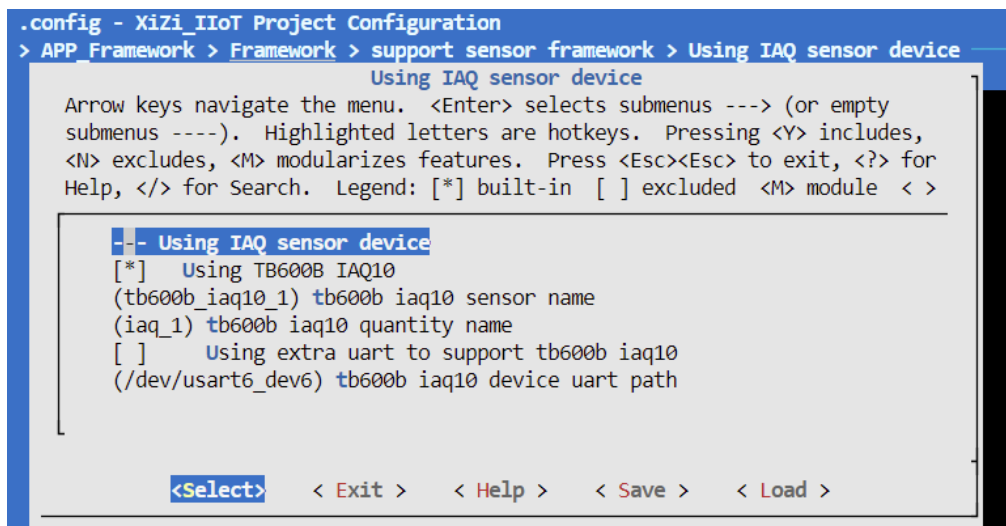


图 6-1-3 空气质量传感器相关配置

然后，配置 APP_Framework 中的 Applications 模块。按照 APP_Framework->Applications->sensor app->Using sensor apps 顺序选中 Using sensor apps；键入回车键，进入 Using sensor apps 配置菜单；选中 Using sensor IAQ apps 并进入其配置菜单；选中 Using sensor TB600B_IAQ10 apps，最后界面如图 6-1-4 所示：

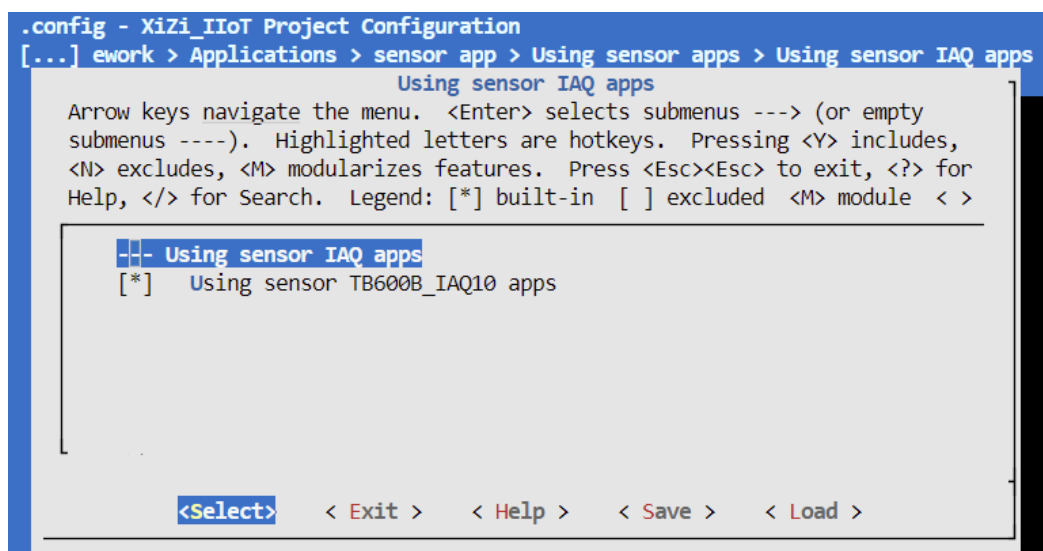


图 6-1-4 空气质量传感器相关配置

最后，配置 TB600B-IAQ-10 传感器所使用的 usart 接口。回到主界面，按照 hc32f4a0 feature->Using UART device 顺序选中 Using UART device；键入回车键，进入 Using UART device 配置菜单；选中 Enable USART6 并进入其配置菜单，具体配置如图 6-1-5 所示：

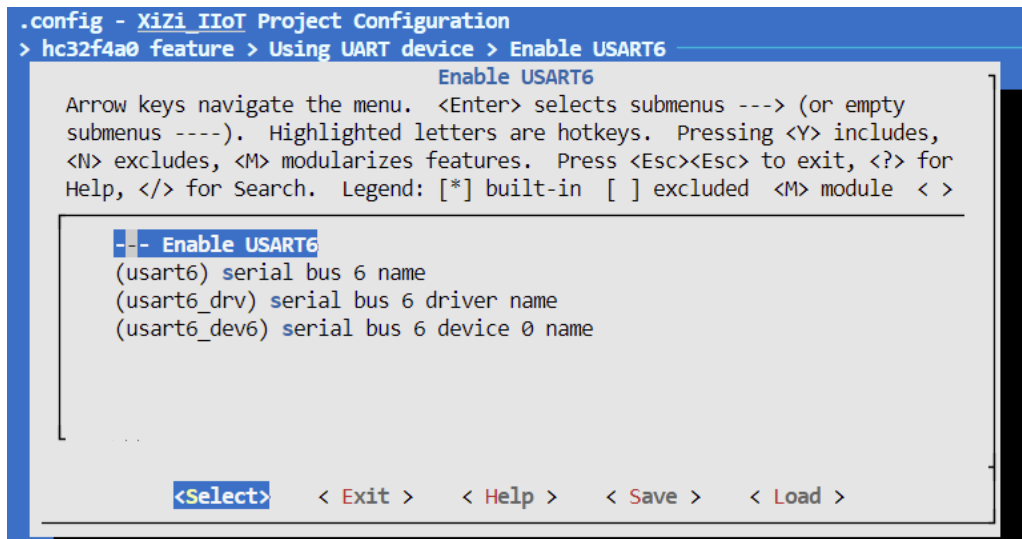


图 6-1-5 空气质量传感器相关配置

在所有配置完成后，选择 Save 保存配置文件并退出。

tb600b_iaq.c 中的空气质量传感器初始化过程如代码 6-1-8 所示。代码行 3 对传感器设备进行初始化。代码行 5~14 对结构体类型为 SensorQuantity 的变量 tb600b_iaq10_iaq 参数进行赋值。代码行 16 将 tb600b_iaq10_iaq 注册到传感器物理量列表中。

代码 6-1-8 TB600B-IAQ-10 空气质量传感器初始化

```
01 int Tb600bIaq10IaqInit(void)
02 {
03     SensorDeviceTb600bIaq10Init();
04
05     tb600b_iaq10_iaq.name = SENSOR_QUANTITY_TB600B_IAQ;
06     tb600b_iaq10_iaq.type = SENSOR_QUANTITY_IAQ;
07     tb600b_iaq10_iaq.value.decimal_places = 0;
08     tb600b_iaq10_iaq.value.max_std = SENSOR_QUANTITY_VALUE_ERROR;
09     tb600b_iaq10_iaq.value.min_std = SENSOR_QUANTITY_VALUE_ERROR;
10     tb600b_iaq10_iaq.value.last_value = SENSOR_QUANTITY_VALUE_ERROR;
11     tb600b_iaq10_iaq.value.max_value = SENSOR_QUANTITY_VALUE_ERROR;
12     tb600b_iaq10_iaq.value.min_value = SENSOR_QUANTITY_VALUE_ERROR;
13     tb600b_iaq10_iaq.sdev = &tb600b_iaq10;
14     tb600b_iaq10_iaq.ReadValue = QuantityRead;
15 }
```

```
16    SensorQuantityRegister(&tb600b_iaq10_iaq);
17
18    return 0;
19 }
```

空气质量传感器数据读取过程如代码 6-1-9 所示。代码行 3~4 用于查找空气质量传感器，然后将其打开；代码行 6~10 用于循环读取空气质量传感器数据；最后，代码行 11 关闭传感器。需要注意的是，打开传感器设备有一定的响应时延，在实验过程中可以通过 PrivTaskDelay 函数调高等待时延，以防止传感器读不到数据。

代码 6-1-9 TB600B-IAQ-10 空气质量传感器数据读取

```
01 void IaqTb600bIaq10(void)
02 {
03     struct SensorQuantity *iaq =
SensorQuantityFind(SENSOR_QUANTITY_TB600B_IAQ, SENSOR_QUANTITY_IAQ);
04     SensorQuantityOpen(iaq);
05     int32_t result = 0;
06     for (int i = 0; i < 5; i++) {
07         PrivTaskDelay(1000);
08         result = SensorQuantityReadValue(iaq);
09         printf("Gas concentration is : %dppb\n", result);
10     }
11     SensorQuantityClose(iaq);
12 }
```

TB600B-IAQ-10 空气质量传感器数据读取如图 6-1-6 所示。一共读取了 5 次数据，显示当前室内挥发性有机化合物浓度在 200~500ppb 之间。

```

Device tb600b_iaq10_1 open success.
Gas concentration is : 481ppb
The temperature is : 25.63℃
The humidity is : 29.16rh%
Gas concentration is : 481ppb
Gas concentration is : 514ppb
The temperature is : 25.59℃
The humidity is : 28.87rh%
Gas concentration is : 514ppb
Gas concentration is : 479ppb
The temperature is : 25.59℃
The humidity is : 28.67rh%
Gas concentration is : 479ppb
Gas concentration is : 228ppb
The temperature is : 25.58℃
The humidity is : 28.47rh%
Gas concentration is : 228ppb
Gas concentration is : 0ppb
The temperature is : 25.53℃
The humidity is : 28.7rh%
Gas concentration is : 0ppb
tb600b_iaq10_1 successfully closed.

```

图 6-1-6 空气质量传感器 TB600B-IAQ-10 数据读取显示

1.2.2 电表传感器

电表传感器用于精确测量电能计量中正向有功电能，收集电流、电压、功率等信息。三相四线导轨式电能表 DTSD342（设备型号）的通信端口均为 RS485，协议使用 MODBUS-RTU。具体的电表采集过程体现在 test_485.c 及 test_485_ReadElectry.c 文件中，其中 test_485.c 实现了 485 端口配置，test_485_ReadElectry.c 中实现了电表数据的读取。

使用电表传感器之前，需要进行一些配置，如图 6-11 所示。首先在终端输入“make BOARD=xidatong-arm32 menuconfig”命令，进入编译配置菜单，如图 6-1-7 所示。

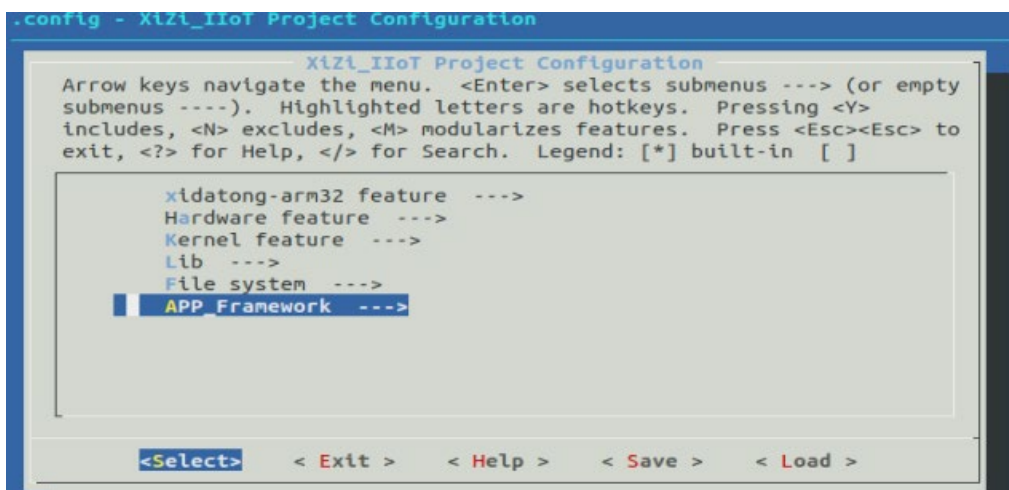


图 6-1-7188 电表传感器相关配置

首先，配置 APP_Framework 中的 Applications 模块。按照 APP_Framework->Applications->test_app 顺序选中 Enable application test function；键入回车键，进入 Enable application test function 配置菜单，选中 Config test uart(RS485)；键入回车键，进入 Config test uart(RS485)配置菜单，配置 RS485 信息，具体如图 6-12 所示：

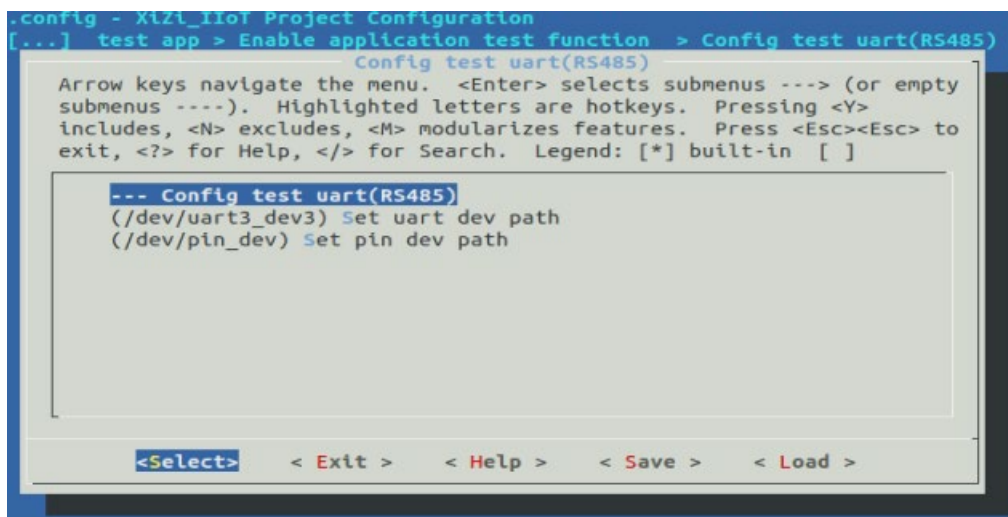


图 6-1-8 电表传感器相关配置

然后，配置电表传感器所使用的 usart 接口。回到主界面，按照 xidatong-arm32 feature->Using UART device 顺序选中 Using UART device；键入回车键，进入 Using UART device 配置菜单；选中 Enable USART3 并进入其配置菜单，具体配置如图 6-13 所示。

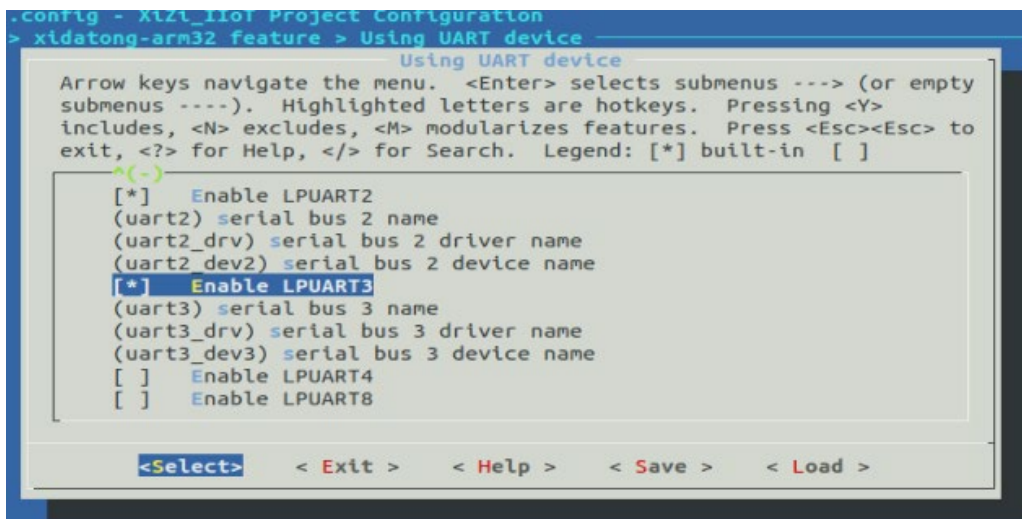


图 6-1-9 电表传感器相关配置

test_485.c 中的 485 通信端口配置过程如代码 6-5 所示，代码行 3~20 定义了一些结构体变量，并对其进行赋值。代码行 24~25 通过 IO 控制函数分别对打开的串口驱动和 GPIO 驱动进行配置。

代码 6-1-10 电表传感器初始化

```

01 void Set485()
02 {
03     struct PinParam pin_parameter;
04     pin_parameter.cmd = GPIO_CONFIG_MODE;
05     pin_parameter.pin = BSP_485_DIR_PIN;
06     pin_parameter.mode = GPIO_CFG_OUTPUT;
07     struct PrivIoctlCfg ioctl_cfg;
08     ioctl_cfg.ioctl_driver_type = PIN_TYPE;
09     ioctl_cfg.args = (void *)&pin_parameter;
10
11     struct SerialDataCfg uart_cfg;
12     uart_cfg.serial_baud_rate = BAUD_RATE_9600;
13     uart_cfg.serial_data_bits = DATA_BITS_8;
14     uart_cfg.serial_stop_bits = STOP_BITS_1;
15     uart_cfg.serial_parity_mode = PARITY_NONE;
16     uart_cfg.serial_bit_order = BIT_ORDER_LSB;
17     uart_cfg.serial_invert_mode = NRZ_NORMAL;
18     uart_cfg.serial_buffer_size = SERIAL_RB_BUFSZ;
19     uart_cfg.serial_timeout = 1000;
20     uart_cfg.is_ext_uart = 0;
21
22     ioctl_cfg.ioctl_driver_type = SERIAL_TYPE;
23     ioctl_cfg.args = (void *)&uart_cfg;
24     PrivIoctl(pin_fd, OPE_CFG, &ioctl_cfg);
25     PrivIoctl(uart_fd, OPE_INT, &ioctl_cfg);
26 }

```

test_485_ReadElectry.c 中给电表发送指令过程如代码 6-6 所示，代码行 3~10 为指令 Instruction 并赋值，作为传递给电表的功能码。代码行 12~15 定义了串口方向结构体，并将串口置为高位，控制 GPIO 驱动方向为写入，代码行 16 将指令通过 485 端口发送给电表。三相四线导轨式电能表及 DTSD342 的功能码格式存在一定差别。

代码 6-1-11 电表传感器读取指令传输

```

01 void WriteElectry()
02 {
03     Instruction[0] = 0x01;
04     Instruction[1] = 0x04;
05     Instruction[2] = 0x00;
06     Instruction[3] = 0x00;

```



```
07     Instruction[4] = 0x00;
08     Instruction[5] = 0x36;
09     Instruction[6] = 0x70;
10     Instruction[7] = 0x1C;
11
12     struct PinStat pin_dir;
13     pin_dir.pin = BSP_485_DIR_PIN;
14     pin_dir.val = GPIO_HIGH;
15     PrivWrite(pin_fd,&pin_dir,0);
16     PrivWrite(uart_fd,Instruction,sizeof(Instruction));
17     PrivTaskDelay(100);
18 }
```

对于三相四线导轨式电能表，以功能码“01 04 00 00 00 36 70 1C”为例，其含义为从 01 表读取输入寄存器的输出，寄存器地址从 0000 读至 0036，701C 则为根据 modbus 协议生成的校验码。

而对于 DTSD342，其功能码格式如图 6-14。以功能码“01 03 20 00 00 08 4F CC”，其含义为从 01 电表读取地址 0020 至 0800 的数据，并根据 modbus 协议计算得到校验码 4F CC。

设备通信地址 (0~247)	功能代码 (03H)	数据域起始地址 高字节	数据域起始地址 低字节	数据域长度 高字节	数据域长度 低字节	CRC 校验低 字节	CRC 校验高 字节
-------------------	---------------	----------------	----------------	--------------	--------------	---------------	---------------

图 6-1-10190 DTSD342 功能码

代码 6-7 从电表读取数据。代码行 3~4 将串口置为低位，控制 GPIO 驱动方向为读取。代码行 05~07 从 485 端口接收电表传输的数据。

代码 6-1-12 电表传感器数据读取

```
01 void ReadElectry()
02 {
03     pin_dir.val = GPIO_LOW;
04     PrivWrite(pin_fd,&pin_dir,0);
05     uint8 buff[255];
06     memset(buff,0,sizeof(buff));
07     PrivRead(uart_fd,buff,20);
08     int32_t result = 0;
09     result = ElectryReadValue(buff);
10 }
```

三相四线导轨式电表数据每个参数由 4 个字节构成，将每两个字节低位与高位进行交换后得到真实值，解码出的参数依次为 ABC 三相的电压、电流、有功功率、无功功率、视载功率、电网频率、运行状态字、组合有功总电量、剩余电量、透支电量。

DTSD342 电表数据每个参数同样由 4 个字节构成，但无需低位与高位进行交换，顺序读取即可得到相应的参数，包括 ABC 三相的有功电能及总有功电能。

三相四线导轨式电能表读取出的数据如图 6-15 所示，在未进行测量时除 C 相电压为 255V 外，其他数据均为 0。

```

A相电压:0.06V   B相电压:0.80V   C相电压:235.52V
A相电流:0.000A   B相电流:0.000A   C相电流:0.000A
A相有功功率:0.0W   B相有功功率:0.0W   C相有功功率:0.0W   总有功功率:0.0W
A相无功功率:0.0Var   B相无功功率:0.0Var   C相无功功率:0.0Var   总无功功率:0.0Var
A相视在功率:0.0VA   B相视在功率:0.0VA   C相视在功率:0.0VA   总视在功率:0.0VA
A相功率因数:0.000   B相功率因数:0.000   C相功率因数:0.000   总功率因数:0.000
电网频率:0.00Hz
电表运行状态字:0x0
组合有功总电量:0.00kwh
剩余电量:0.00kwh
透支电量:0.00kwh
Return: 0, 0x00000000

```

图 6-1-11 三相四线导轨式电能表数据读取显示

DTSD342 电能表在未进行测量时读取出的数据如图 6-16 所示，有功电能此时基本均为 0。

```

A相有功电能:0.04kwh
B相有功电能:0.00kwh
C相有功电能:0.00kwh
总有功电能:0.00kwh

```

图 6-191-12 DTSD342 电能表数据读取显示

1.3 RISC-V 架构下的传感器

1.3.1 温度传感器

温度传感器用于采集环境的温度，根据实时数据对环境的温度进行调节和控制。HS300X（设备型号）温度传感器的端口和协议均采用 I2C。具体的温度采集过程体现在 hs300x_temp.c 和 temperature_hs300x.c 文件中，其中 hs300x_temp.c 实现了 HS300X 温度传感器设备初始化；temperature_hs300x.c 用于查找到温度传感器，并进行数据读取。

在使用温度传感器之前，需要进行一些配置，如图 2.1 所示。首先，在终端输入“make BOARD=edu-riscv64 menuconfig”命令，进入编译配置菜单；然后进入 APP_Framework->Framework->选中 support sensor framework；

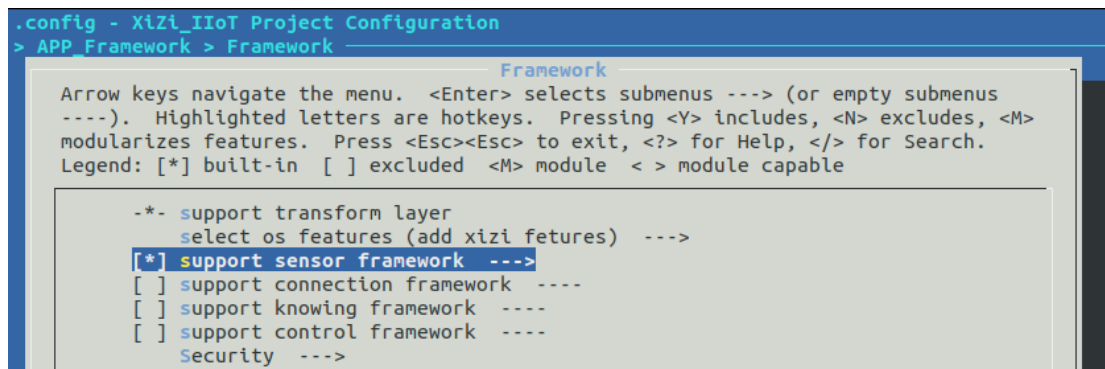


图 6-1-13192 温度传感器配置过程

输入回车键，进入 support sensor framework 配置菜单，选中 using temperature sensor device;

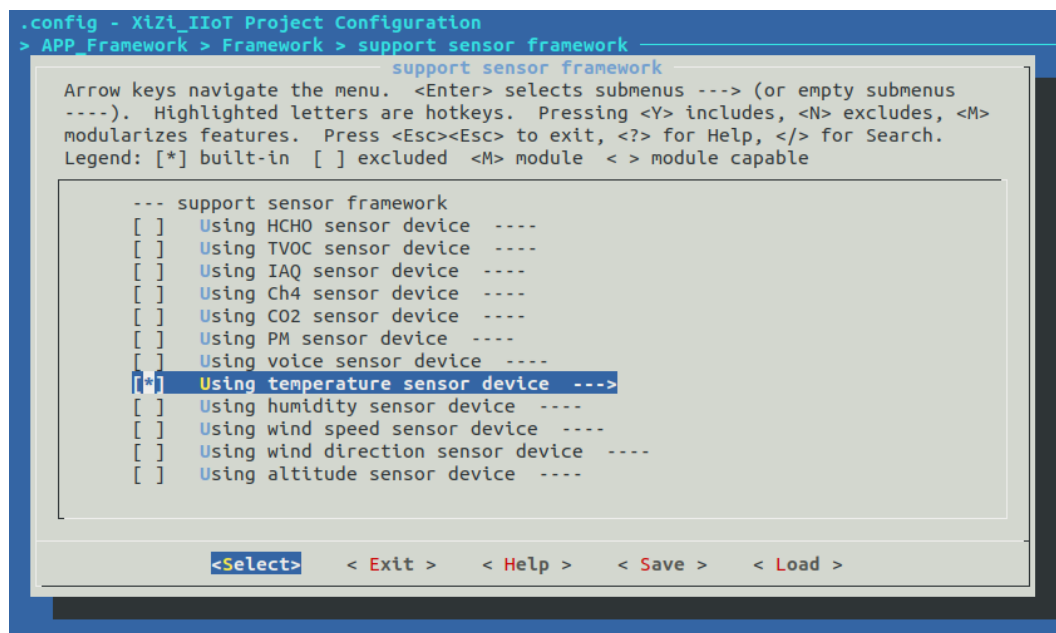


图 6-1-14 温度传感器配置过程

键入回车键，进入 using temperature sensor device 配置菜单，选中 using HS300x for temperature;

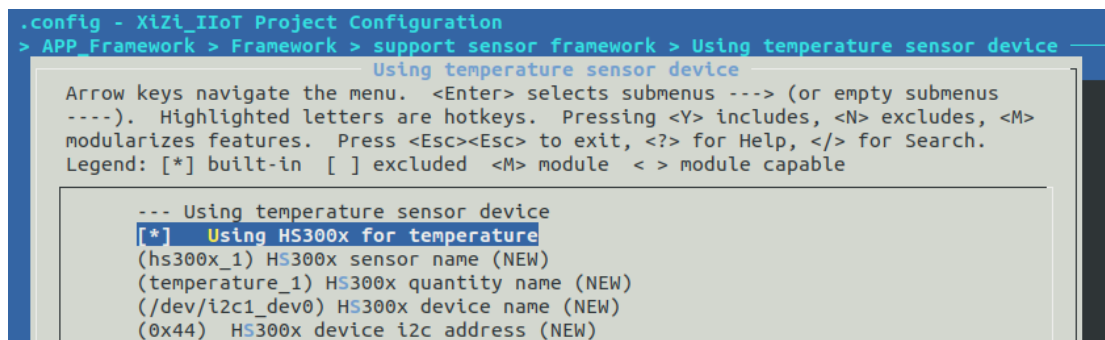


图 6-1-15 温度传感器配置过程

键入三次 ESC 按钮，回到主菜单，进入 Application 配置菜单；进入 Application 配置菜单后，进入 sensor_app 配置菜单，再选中 Using sensor apps；

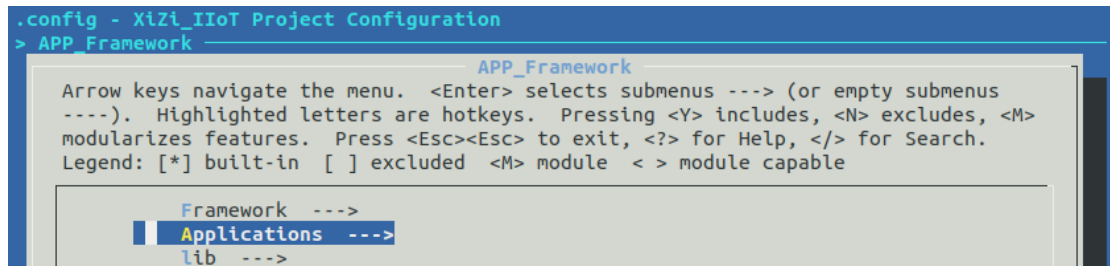


图 6-1-16 温度传感器配置过程

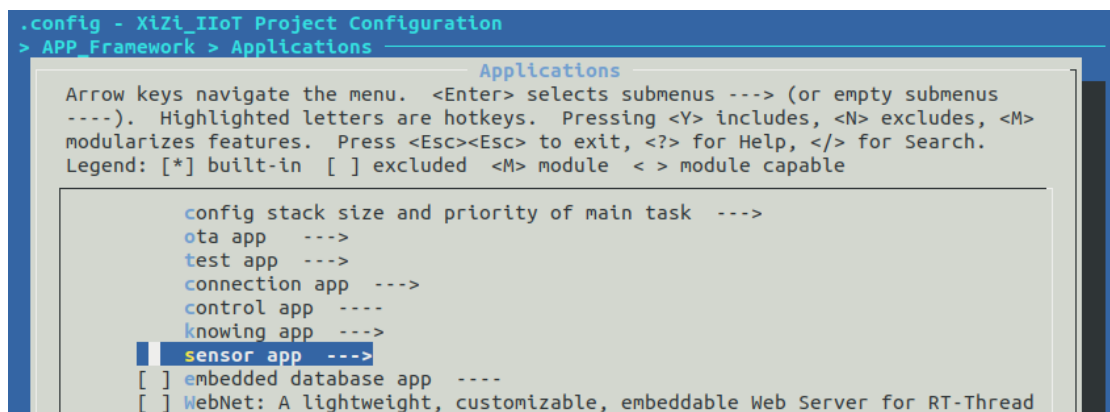


图 6-1-17 温度传感器配置过程

继续进入 Using sensor apps 的配置菜单，选中 Using sensor temperature apps 并进入其配置菜单；

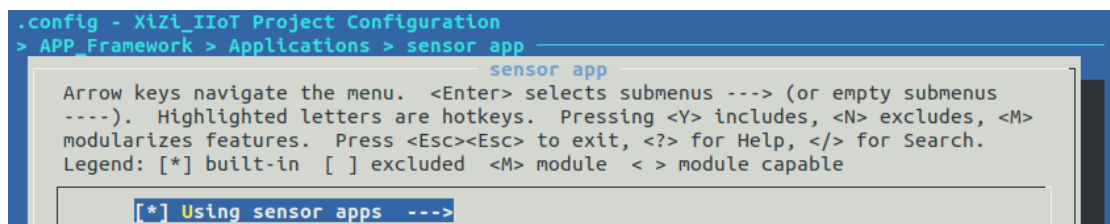


图 6-1-18 温度传感器配置过程

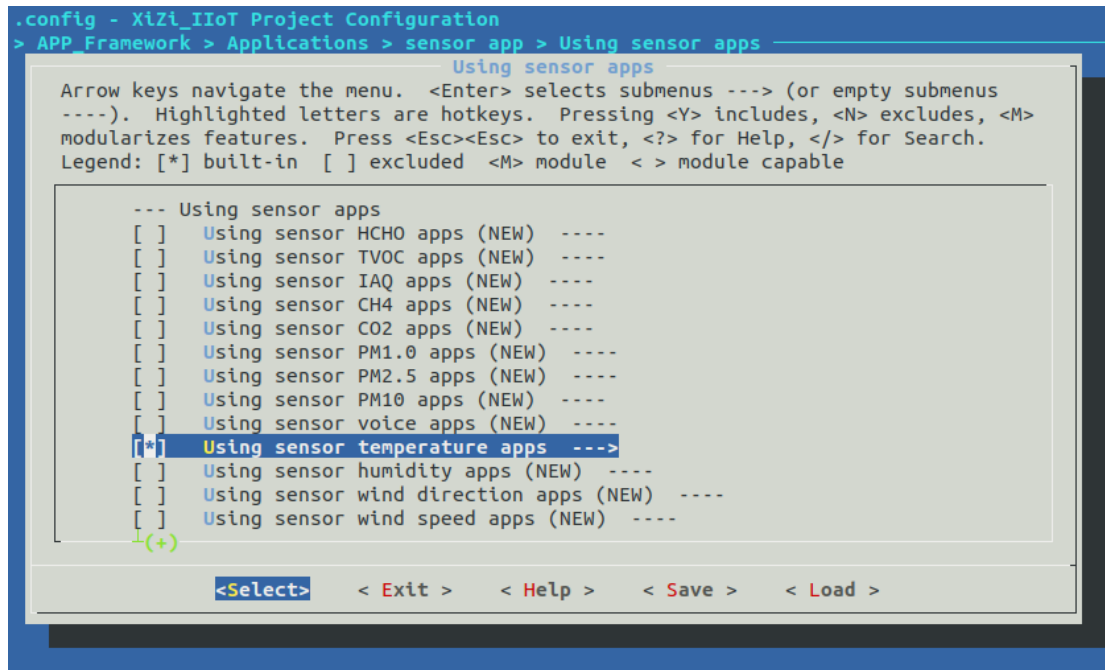


图 6-1-19 温度传感器相关配置

选中 Using sensor Hs300x apps。

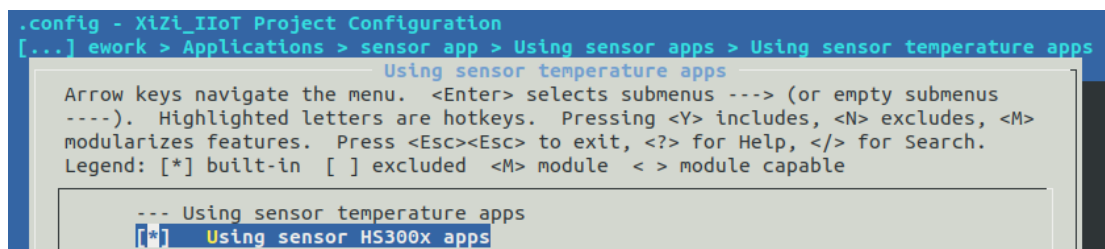


图 6-1-20 温度传感器配置过程

回到主界面，进入 edu-riscv64 feature 配置菜单，选中 Using I2C device 并进入其配置菜单。

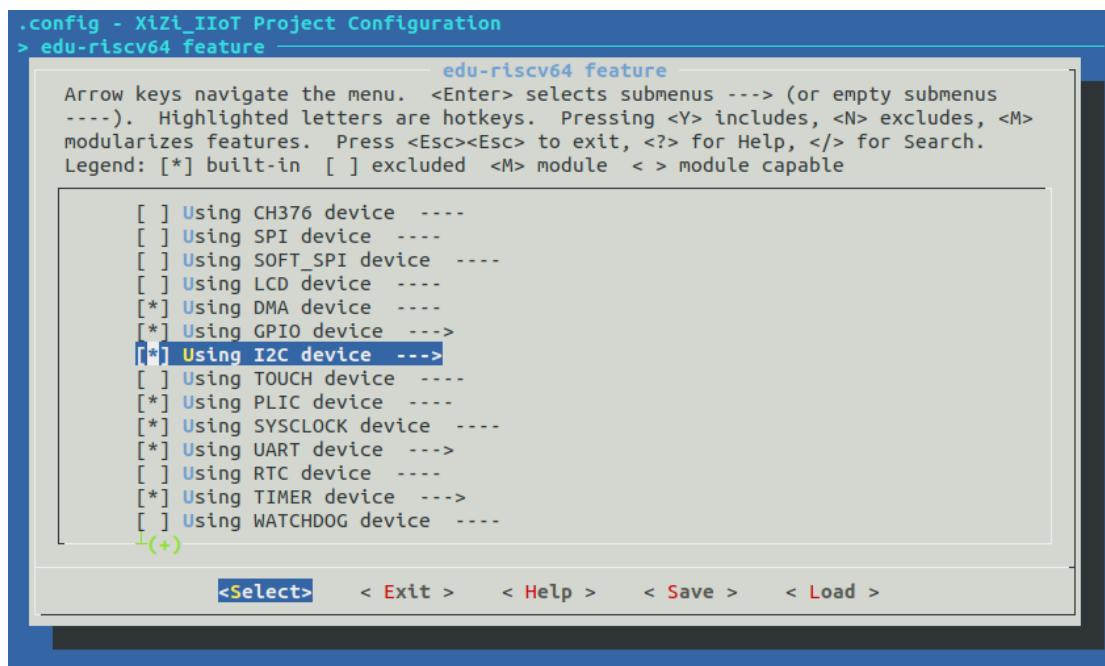


图 6-1-21 温度传感器配置过程

修改 SDA pin number for I2C、SCL pin number for I2C 分别为 34、35（即分别对应外置 IO34、IO35）；选择 Save 保存配置文件并退出。

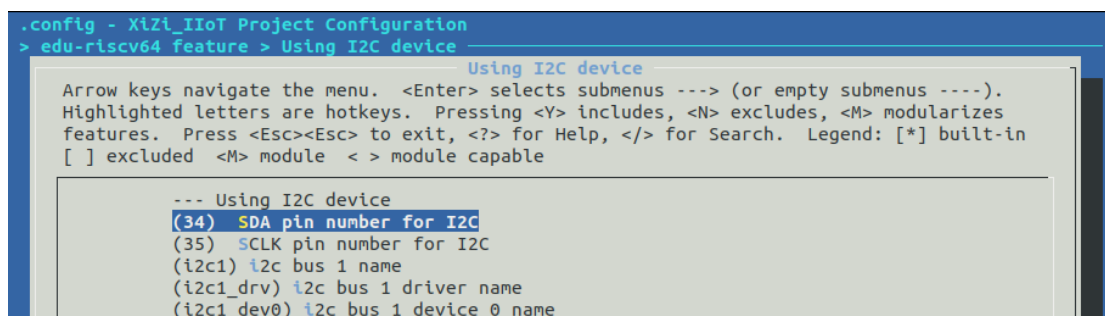


图 6-1-22 温度传感器配置过程

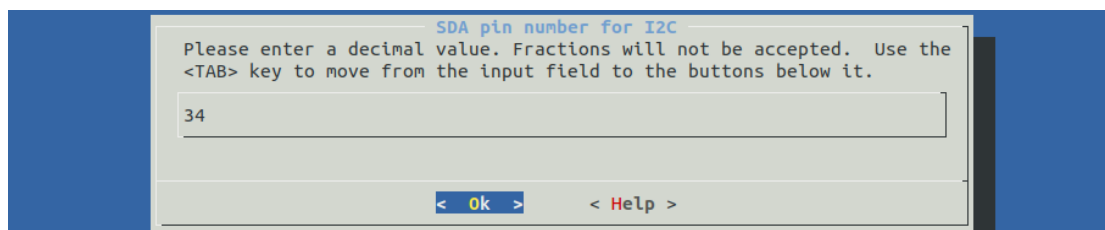


图 6-1-23 温度传感器配置过程

hs300x_temp.c 中的温度传感器初始化过程如代码 6-5 所示。代码行 2 对传感器设备进行初始化。代码行 4~13 对结构体类型为 SensorQuantity 的变量 hs300x_temperature 参数进行赋值。代码行 15 将 hs300x_temperature 注册到传感器列表中。

代码 6-1-13 HS300X 温度传感器初始化

```

01 int Hs300xTemperatureInit(void){
02     SensorDeviceHs300xInit();
03
04     hs300x_temperature.name = SENSOR_QUANTITY_HS300X_TEMPERATURE;
05     hs300x_temperature.type = SENSOR_QUANTITY_TEMP;
06     hs300x_temperature.value.decimal_places = 1;
07     hs300x_temperature.value.max_std = 1000;
08     hs300x_temperature.value.min_std = 0;
09     hs300x_temperature.value.last_value = SENSOR_QUANTITY_VALUE_ERROR;
10     hs300x_temperature.value.max_value = SENSOR_QUANTITY_VALUE_ERROR;
11     hs300x_temperature.value.min_value = SENSOR_QUANTITY_VALUE_ERROR;
12     hs300x_temperature.sdev = &hs300x;
13     hs300x_temperature.ReadValue = ReadTemperature;
14
15     SensorQuantityRegister(&hs300x_temperature);
16
17     return 0;
18 }

```

温度传感器数据读取过程如代码?所示，详见 temperature_hs300x.c 文件。代码行 2~3 声明两个变量。代码行 5~6 用于查找到温度传感器，然后将其打开。代码行 8~16 用于循环读取温度传感器的数据。最后，代码行 18 关闭传感器。

代码 6-1-14 HS300X 温度传感器数据读取

```

01 void TempHs300x(void){
02     int i = 0;
03     int32_t temperature;
04
05     struct SensorQuantity *temp =
SensorQuantityFind(SENSOR_QUANTITY_HS300X_TEMPERATURE,
SENSOR_QUANTITY_TEMP);
06     SensorQuantityOpen(temp);
07
08     for (i = 0; i < 10; i++) {
09         temperature = SensorQuantityReadValue(temp);
10         if (temperature > 0)
11             printf("Temperature : %d.%d °C\n", temperature/10, temperature%10);
12         else
13             printf("Temperature : %d.%d °C\n", temperature/10, -temperature%10);

```

```

14
15     PrivTaskDelay(500);
16 }
17
18     SensorQuantityClose(temp);
19 }

```

HS300X 温度传感器的数据读取如图 2.2 所示。一共读取了 10 次数据，显示当前室内温度在 24.1 度。

```

initialize sensor_framework success.
initialize hs300x_temperature success
SensorDeviceRegister: sensor with the same name already registered
initialize hs300x_humidity success
Device hs300x_1 open success.
Temperature : 24.1 °C
Temperature : 24.2 °C
Temperature : 24.1 °C
Temperature : 24.1 °C
Temperature : 24.1 °C
Temperature : 24.1 °C
Temperature : 24.1 °C
Temperature : 24.1 °C
Temperature : 24.1 °C
Temperature : 24.1 °C

```

图 6-1-24 温度传感器 HS300X 数据读取显示

1. 3. 2 湿度传感器

湿度传感器用于采集环境的湿度，根据实时数据对环境的湿度进行调节和控制。HS300X（设备型号）具有温度传感和湿度传感两种功能，因而仍采用 HS300X 作为湿度传感器进行实验。具体的湿度采集过程实现在 hs300x_humi.c 和 humidity_hs300x.c 文件中，其中 hs300x_humi.c 实现了 HS300X 湿度传感器设备初始化、注册及数据读取操作的方法；humidity_hs300x.c 文件用于查找到湿度传感器，并调用 hs300x_humi.c 文件中的接口进行数据读取。

hs300x_humi.c 中的湿度传感器初始化过程如代码 2.3 所示。代码行 3 是对设备硬件进行初始化；代码行 5 至代码行 14 对静态变量 hs300x_humidity 结构体成员进行初始化，即进行初值的赋值；代码行 16 将静态变量 hs300x_humidity 注册到系统中。

代码 6-1-15 HS300X 湿度传感器初始化

```

01 int Hs300xHumidityInit(void)
02 {
03     SensorDeviceHs300xInit();
04
05     hs300x_humidity.name = SENSOR_QUANTITY_HS300X_HUMIDITY;
06     hs300x_humidity.type = SENSOR_QUANTITY_HUMI;

```

```

07     hs300x_humidity.value.decimal_places = 1;
08     hs300x_humidity.value.max_std = 1000;
09     hs300x_humidity.value.min_std = 0;
10     hs300x_humidity.value.last_value = SENSOR_QUANTITY_VALUE_ERROR;
11     hs300x_humidity.value.max_value = SENSOR_QUANTITY_VALUE_ERROR;
12     hs300x_humidity.value.min_value = SENSOR_QUANTITY_VALUE_ERROR;
13     hs300x_humidity.sdev = &hs300x;
14     hs300x_humidity.ReadValue = ReadHumidity;
15
16     SensorQuantityRegister(&hs300x_humidity);
17
18     return 0;
19 }

```

湿度传感器数据读取过程如代码 2.4 所示，详见 humidity_hs300x.c 文件。代码行 3 至 4 进行了变量的初始化声明。代码行 5 根据配置好的传感器名称寻找到对应的传感器结构体指针。在代码行 6，即成功找到传感器结构体指针后，可调用结构体中的 open 方法打开传感器设备；代码行 7 到 11 根据传感器指针中的读取方法的函数指针读取传感器数值并打印至 shell 中。代码行 12 调用 Close 方法关闭设备。

代码 6-1-16 HS300X 湿度传感器数据读取

```

01 void HumiHs300x(void)
02 {
03     int i = 0;
04     int32_t humidity;
05     struct SensorQuantity *humi = SensorQuantityFind
                                (SENSOR_QUANTITY_HS300X_HUMIDITY,
                                SENSOR_QUANTITY_HUMI);
06     SensorQuantityOpen(humi);
07     for (i = 0; i < 10; i++) {
08         humidity = SensorQuantityReadValue(humi);
09         printf("Humidity : %d.%d %%RH\n", humidity/10, humidity%10);
10         PrivTaskDelay(500);
11     }
12     SensorQuantityClose(humi);
13 }

```

HS300X 湿度数据的读取如图 2.3 所示。一共读取并打印了 10 次读取数据的结果，显示当前室内相对湿度为 60.9% 左右。

```

hs300x_1 successfully closed.
Device hs300x_1 open success.
Humidity : 61.0 %RH
Humidity : 61.0 %RH
Humidity : 60.9 %RH
Humidity : 60.9 %RH
Humidity : 60.9 %RH
Humidity : 60.9 %RH
Humidity : 60.9 %RH
Humidity : 60.9 %RH
Humidity : 60.9 %RH
Humidity : 60.9 %RH
Humidity : 60.9 %RH
hs300x_1 successfully closed.

```

图 6-1-25 湿度传感器 HS300X 数据读取显示

1.3.3 风向传感器

风速传感器用于监测环境的风速。QS-FX（设备型号）风速传感器的端口采用串口，协议采用 SP3485。风速采集过程体现在 `qs_fx.c` 和 `winddirection_qs_fx.c` 文件中，其中 `qs_fx.c` 实现了 QS-FX 风速传感器设备初始化；`winddirection_qs_fx.c` 用于查找到风速传感器，并进行数据读取。其系统配置过程与 2.1.1 中的配置过程类似。

`qs_fx.c` 中的风向传感器初始化过程如代码 2.5 所示。代码行 2 对传感器设备进行初始化。代码行 4~13 对结构体类型为 `SensorQuantity` 的变量 `qs_fx_wind_direction` 参数进行赋值。代码行 15 将 `qs_fx_wind_direction` 注册到传感器列表中。

代码 6-1-17 QS-FX 风向传感器初始化

```

01 int QsFxWindDirectionInit(void){
02     QsFxInit();
03
04     qs_fx_wind_direction.name = SENSOR_QUANTITY_QS_FX_WINDDIRECTION;
05     qs_fx_wind_direction.type = SENSOR_QUANTITY_WINDDIRECTION;
06     qs_fx_wind_direction.value.decimal_places = 1;
07     qs_fx_wind_direction.value.max_std = 600;
08     qs_fx_wind_direction.value.min_std = 0;
09     qs_fx_wind_direction.value.last_value = SENSOR_QUANTITY_VALUE_ERROR;
10     qs_fx_wind_direction.value.max_value = SENSOR_QUANTITY_VALUE_ERROR;
11     qs_fx_wind_direction.value.min_value = SENSOR_QUANTITY_VALUE_ERROR;
12     qs_fx_wind_direction.sdev = &qs_fx;
13     qs_fx_wind_direction.ReadValue = ReadWindDirection;
14
15     SensorQuantityRegister(&qs_fx_wind_direction);
16
17     return 0;

```

18 }

QS-FS 风向传感器数据读取过程如代码 2.6 所示，详见 winddirection_qs_fx.c 文件。代码行 2~3 用于查找到风向传感器，然后将其打开。代码行 6~12 用于循环读取风向传感器的数据并打印显示。最后，代码行 14 关闭传感器。

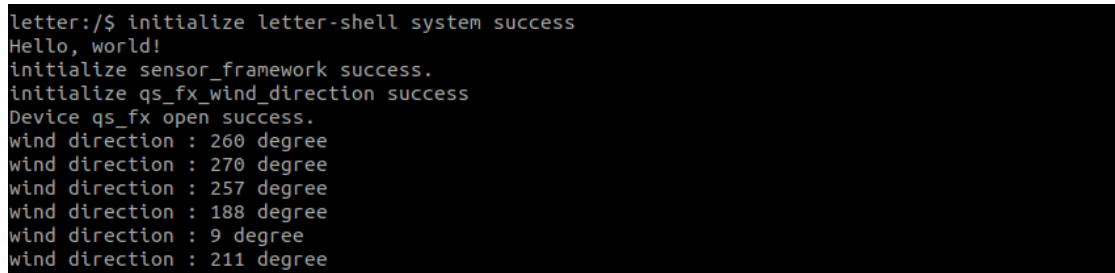
代码 6-249-18 QS-FS 风向传感器数据读取

```

01 void WindDirectionQsFx(void){
02     struct SensorQuantity *wind_direction =
SensorQuantityFind(SENSOR_QUANTITY_QS_FX_WINDDIRECTION,
SENSOR_QUANTITY_WINDDIRECTION);
03     SensorQuantityOpen(wind_direction);
04     PrivTaskDelay(2000);
05     int result = 0;
06     for(int i=0;i<2000;i++){
07         result = 0;
08         PrivTaskDelay(1000);
09         result = SensorQuantityReadValue(wind_direction);
10         if(result > 0)
11             printf("wind direction : %d degree\n", result);
12     }
13
14     SensorQuantityClose(wind_direction);
15 }

```

QS-FS 风向传感器的数据读取如图 2.4 所示，显示当前环境的风向一直在改变。



```

letter:/$ initialize letter-shell system success
Hello, world!
initialize sensor_framework success.
initialize qs_fx_wind_direction success
Device qs_fx open success.
wind direction : 260 degree
wind direction : 270 degree
wind direction : 257 degree
wind direction : 188 degree
wind direction : 9 degree
wind direction : 211 degree

```

图 6-1-26 QS-FS 风向传感器数据读取显示

1.3.4 风速传感器

风速传感器用于监测环境的风速。QS-FS（设备型号）风速传感器的端口采用串口，协议采用 SP3-485。风速采集过程体现在 qs_fs.c 和 windspeed_qs_fs.c 文件中，其中 qs_fs.c 实现了 QS-FS 风速传感器设备初始化；windspeed_qs_fs.c 用于查找到风速传感器，并进行数据读取。其系统配置过程与 2.1.1 中的配置过程类似。

qs_fs.c 中的风速传感器初始化过程如代码 2.7 所示。代码行 2 对传感器设备进行初始化。代码行 4~13 对结构体类型为 SensorQuantity 的变量 qs_fs_wind_speed 参数进行赋值。代码行 15 将 qs_fs_wind_speed 注册到传感器列表中。

代码 6-1-19250 QS-FS 风速传感器初始化

```

01 int QsFsWindSpeedInit(void){
02     QsFsInit();
03
04     qs_fs_wind_speed.name = SENSOR_QUANTITY_QS_FS_WINDSPEED;
05     qs_fs_wind_speed.type = SENSOR_QUANTITY_WINDSPEED;
06     qs_fs_wind_speed.value.decimal_places = 1;
07     qs_fs_wind_speed.value.max_std = 600;
08     qs_fs_wind_speed.value.min_std = 0;
09     qs_fs_wind_speed.value.last_value = SENSOR_QUANTITY_VALUE_ERROR;
10     qs_fs_wind_speed.value.max_value = SENSOR_QUANTITY_VALUE_ERROR;
11     qs_fs_wind_speed.value.min_value = SENSOR_QUANTITY_VALUE_ERROR;
12     qs_fs_wind_speed.sdev = &qs_fs;
13     qs_fs_wind_speed.ReadValue = ReadWindSpeed;
14
15     SensorQuantityRegister(&qs_fs_wind_speed);
16
17     return 0;
18 }

```

QS-FS 风速传感器数据读取过程如代码 2.8 所示，详见 windspeed_qs_fs.c 文件。代码行 2~3 用于查找到风速传感器，然后将其打开。代码行 6~12 用于循环读取风速传感器的数据并打印显示。最后，代码行 14 关闭传感器。

代码 6-251-20 QS-FS 风速传感器数据读取

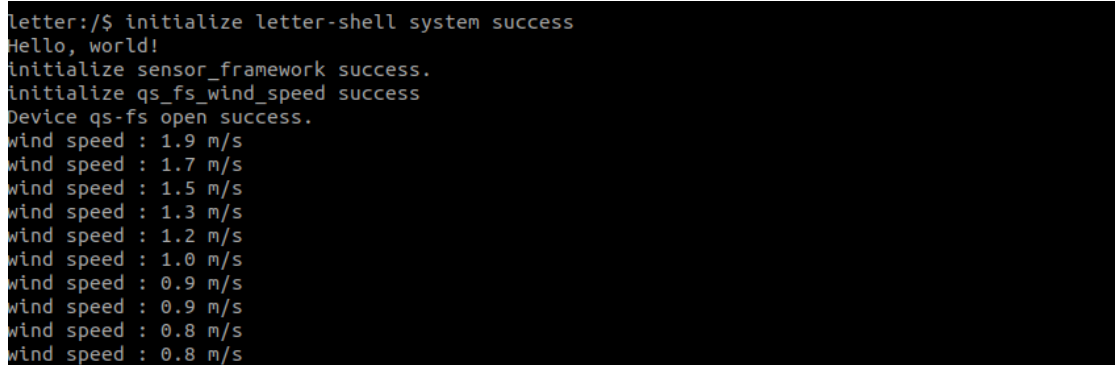
```

01 void WindSpeedQsFs(void){
02     struct SensorQuantity *wind_speed =
SensorQuantityFind(SENSOR_QUANTITY_QS_FS_WINDSPEED,
SENSOR_QUANTITY_WINDSPEED);
03     SensorQuantityOpen(wind_speed);
04     PrivTaskDelay(2000);
05     int result = 0;
06     for(int i=0;i<2000;i++){
07         result = 0;
08         PrivTaskDelay(1000);
09         result = SensorQuantityReadValue(wind_speed);

```

```
10         if(result > 0)
11             printf("wind speed : %d.%d m/s\n", result/10, result%10);
12     }
13
14     SensorQuantityClose(wind_speed);
15 }
```

QS-FS 风速传感器的数据读取如图 2.5 所示，显示当前环境的风速一直在变动。



```
letter:/$ initialize letter-shell system success
Hello, world!
initialize sensor_framework success.
initialize qs_fs_wind_speed success
Device qs-fs open success.
wind speed : 1.9 m/s
wind speed : 1.7 m/s
wind speed : 1.5 m/s
wind speed : 1.3 m/s
wind speed : 1.2 m/s
wind speed : 1.0 m/s
wind speed : 0.9 m/s
wind speed : 0.9 m/s
wind speed : 0.8 m/s
wind speed : 0.8 m/s
```

图 6-1-27 QS-FS 风速传感器数据读取显示

第2章 “泛在互联” 实践

2.1 “泛在互联” 框架介绍

多数嵌入式操作系统，应具备传输多种网络协议数据的能力，但是网络协议种类繁多，为每种协议都开放单独的接口，势必会增加操作系统使用者的开发负担，亦不利于嵌入式操作系统的普及与推广。

XiUOS 的“泛在互联”网络框架采用以网络适配器为中心的抽象方式，发送与接收数据时，只需要像 Linux socket 网络通信一样，仅关注发送与接收的数据，而不用把重点放在所使用的具体协议上。这种抽象方式有效屏蔽了种类繁多的网络协议细节，对外提供统一的网络接口，简化了基于嵌入式操作系统的网络开发。网络框架对各类协议的网络设备进行了抽象，即一个网络协议设备，被抽象为一个 Adapter。

Adapter 可以被设计为采用类似面向对象的方法，针对不同的网络协议，扩展其数据成员与接口，从而为种类繁多的网络协议实现统一的管理架构。在网络应用开发的过程中，只需要使用对应网络协议的 Adapter 实例，而无需关心网络协议的硬件细节，从而实现了网络数据收发功能与底层硬件的解耦。“泛在互联”网络框架图如下：

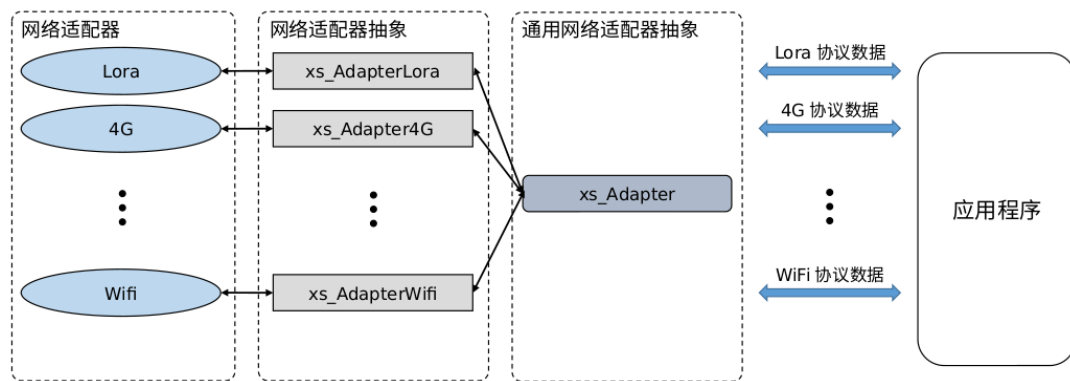


图 6-193-1 “泛在互联”框架结构示意图

下面介绍 XiUOS 的“泛在互联”网络框架关键数据结构定义和解析，具体详见 adapter.h 文件。主要介绍 Adapter 结构体的定义，以及一些接口函数。

2.1.1 Adapter 结构体

Adapter 结构体定义如下：

代码 6-2-1 Adapter 结构体定义

```
struct Adapter
{
    char name[NAME_NUM_MAX];
    int fd;
    int product_info_flag;
    struct AdapterProductInfo *info;
    ATAgentType agent;
    struct Socket socket;
    int net_role_id;
    enum NetProtocolType net_protocol;
    enum NetRoleType net_role;
    enum AdapterStatus adapter_status;
    char buffer[ADAPTER_BUFFSIZE];
    void *done;
    void *adapter_param;
    sem_t sem;
    pthread_mutex_t lock;
    struct DoublelistNode link;
};
```

其中，name 成员是一个可读的名字，用于唯一标识一个 Adapter 结构。fd 成员是一个标识符编号，用于唯一标识此网络设备所用串口。product_info_flag 用于根据型号给设备进

行分类。AdapterProductInfo 结构体包含该网络设备的一些信息，如厂家名 vendor 与型号 product_model，其定义如下：

代码 6-2-2 AdapterProductInfo 结构体定义

```
struct AdapterProductInfo
{
    uint32_t functions;
    char vendor_name[NAME_NUM_MAX];
    char model_name[NAME_NUM_MAX];
    uint32_t work_mode;
    void *model_done;
};
```

其中 model_done 成员包含统一的 API，用于对特定网络适配器进行实际的数据读写。model_done 成员有 IpProtocolDone/PrivProtocolDone 两套接口，根据网络适配器种类而定，两套接口都具有的基础 API 如下。在使用一个网络适配器前后需要打开（open）/关闭（close）该网络适配器，打开适配器后需要配置(ioctl)其属性，setup 和 setdown 用于使能/关闭网络传输功能，setaddr/setdns/setdhcp/ping/netstat 为一些基础的网络调试接口，send/recv 分别用于从网络适配器接收数据与向网络适配器发送数据，connect/disconnect 用于建立/断开一个网络连接。具体结构定义如下：

代码 6-2-3 IpProtocolDone/PrivProtocolDone 结构体定义

```
struct IpProtocolDone/PrivProtocolDone
{
    int (*open)(struct Adapter *adapter);
    int (*close)(struct Adapter *adapter);
    int (*ioctl)(struct Adapter *adapter, int cmd, void *args);
    int (*setup)(struct Adapter *adapter);
    int (*setdown)(struct Adapter *adapter);
    int (*setaddr)(struct Adapter *adapter, const char *ip,
const char *gateway, const char *netmask);
    int (*setdns)(struct Adapter *adapter, const char *dns_addr, uint8 dns_count);
    int (*setdhcp)(struct Adapter *adapter, int enable);
    int (*ping)(struct Adapter *adapter, const char *destination);
    int (*netstat)(struct Adapter *adapter);
    int (*connect)(struct Adapter *adapter, enum NetRoleType net_role, const char *ip, const
char *port, enum IpType ip_type);
    int (*send)(struct Adapter *adapter, const void *buf, size_t len);
```

```

int (*recv)(struct Adapter *adapter, void *buf, size_t len);
int (*disconnect)(struct Adapter *adapter);
};

```

agent 结构体是串口收发数据结构，用于辅助串口通信。网络设备通过 agent 通信,具有两个缓冲区分别用于向 CPU 接收和发送信息。涉及的 ATAgent 结构体如下：

代码 6-2-4 ATAgent 结构体定义

```

struct ATAgent
{
    char agent_name[64];
    int fd;
    char *maintain_buffer;
    uint32 maintain_len;
    uint32 maintain_max;
    int lock;
    ATReplyType reply;
    char reply_lr_end;
    char reply_end_last_char;
    char reply_end_char;
    uint32 reply_char_num;
    int rsp_sem;
    pthread_t at_handler;
    #define ENTM_RECV_MAX 256
    char entm_recv_buf[ENTM_RECV_MAX];
    uint32 entm_recv_len;
    enum ReceiveMode receive_mode;
    int entm_rx_notice;
};
typedef struct ATAgent *ATAgentType;

```

Adapter 结构体中的 socket 成员用于实现上层通信协议。建立多个安全稳定的 tcp/udp 套接字通信。具体结构体定义如下：

代码 6-2-5 Socket 结构体定义

```

struct Socket
{
    uint8_t type;           /* socket type:DGRAM->UDP,STREAM->TCP */
    uint8_t protocol;       /* udp or tcp */
    unsigned short listen_port; /* 0-65535 */
    uint8_t socket_id;
    uint8_t recv_control;

```

```
uint8_t af_type;          /* IPv4 or IPv6 */
char *src_ip_addr;        /* source ip address */
char *dst_ip_addr;        /* destination IP address */
};
```

Adapter 结构体中的 `net_protocol` 成员用于指示此网络设备支持的协议，`IP_PROTOCOL` 类型表示支持 IP 层相关协议，`PRIVATE_PROTOCOL` 表示此网络设备支持额外的协议。用一个枚举变量表示：

代码 6-2-6 NetProtocolType 枚举变量

```
enum NetProtocolType
{
    PRIVATE_PROTOCOL = 1,
    IP_PROTOCOL,
    PROTOCOL_NONE,
};
```

Adapter 结构体中的 `net_role_id` 成员用于辅助 `net_protocol`，用于标识区分该协议下支持的多个设备。`net_role` 成员用于设置此网络设备通信过程中的模式和角色，包括主、从、路由、终端等。用一个枚举变量表示：

代码 6-2-7 NetRoleType 枚举变量

```
enum NetRoleType
{
    CLIENT = 1,
    SERVER,
    MASTER,
    SLAVE,
    COORDINATOR,
    ROUTER,
    END_DEVICE,
    GATEWAY,
    ROLE_NONE,
};
```

Adapter 结构体中的 `adapter_status` 成员用来指示网络设备注册状态。用一个枚举变量表示：

代码 6-2-8 AdapterStatus 枚举变量

```
enum AdapterStatus
{
```

```

REGISTERED = 1,
UNREGISTERED,
INSTALL,
UNINSTALL,
};

```

Adapter 结构体中的 buffer/done/adapter_param 成员均为预留给 adapter 使用的数据缓冲区和用户数据接口。最后，在系统中每种网络协议的 Adapter 被分别组织成不同双链表，如 Lora 的 Adapter 链表、4G 的 Adapter 链表等，使用的链表节点即为 link 成员。

2.1.2 互联框架接口函数

另外，在 adapter.h 文件中还定义了一些函数接口：

- int AdapterDeviceRegister(struct Adapter *adapter)，用于注册适配器，将其加入到链表中，其中 adapter 表示需要操作的适配器。
- int AdapterDeviceUnregister(struct Adapter *adapter)，用于注销一个适配器，其中 adapter 表示需要操作的适配器。
- int AdapterDeviceOpen(struct Adapter *adapter)，用于打开一个适配器，其中 adapter 表示需要操作的适配器。
- int AdapterDeviceClose(struct Adapter *adapter)，用于关闭一个适配器，其中 adapter 表示需要操作的适配器。
- ssize_t AdapterDeviceRecv(struct Adapter *adapter, void *dst, size_t len)，用于从适配器接收数据，其中 adapter 表示需要操作的适配器，dst 表示接收缓冲区指针，len 表示接收缓冲区长度。
- ssize_t AdapterDeviceSend(struct Adapter *adapter, const void *src, size_t len)，用于发送数据到适配器，其中 adapter 表示需要操作的适配器，src 表示发送缓冲区指针，len 表示发送缓冲区长度。
- int AdapterDeviceControl(struct Adapter *adapter, int cmd, void *args)，用于配置适配器，其中 adapter 表示需要操作的适配器，cmd 表示配置命令，args 表示配置命令参数。
- int AdapterDeviceConnect(struct Adapter *adapter, enum NetRoleType net_role, const char *ip, const char *port, enum IpType ip_type)，用于适配器连接 ip 网络，只支持 IP_PROTOCOL，其中 adapter 表示需要操作的适配器，net_role 表示此网络设备通信过程

中的模式和角色，ip 表示需要连接的 ip 地址，port 表示需要连接的端口，ip_type 表示 ip 类型，IPV4 或 IPV6。

- int AdapterDeviceJoin(struct Adapter *adapter, unsigned char *priv_net_group), 用于适配器组入专用网络，只支持 PRIVATE_PROTOCOL，其中 adapter 表示需要操作的适配器，priv_net_group 表示专用网络组名。
- int AdapterDeviceDisconnect(struct Adapter *adapter, unsigned char *priv_net_group), 用于适配器从专用网络组断开连接，其中 adapter 表示需要操作的适配器，priv_net_group 表示专用网络组名。
- int AdapterDeviceSetUp(struct Adapter *adapter), 用于使能适配器网络传输功能，其中 adapter 表示需要操作的适配器。
- int AdapterDeviceSetDown(struct Adapter *adapter), 用于关闭适配器网络传输功能，其中 adapter 表示需要操作的适配器。
- int AdapterDeviceSetAddr(struct Adapter *adapter, const char *ip, const char *gateway, const char *netmask), 用于配置适配器 ip 地址、端口和网络掩码，其中 adapter 表示需要操作的适配器，ip 表示需要配置的 ip 地址，gateway 表示需要配置的端口，netmask 表示需要配置的网络掩码。
- int AdapterDeviceSetDns(struct Adapter *adapter, const char *dns_addr, uint8 dns_count), 用于配置适配器的域名服务器，其中 adapter 表示需要操作的适配器，dns_addr 表示需要配置的域名服务器地址，dns_count 表示需要配置的域名服务器数量。
- int AdapterDeviceSetDhcp(struct Adapter *adapter, int enable), 用于配置适配器 DHCP 功能，其中 adapter 表示需要操作的适配器，enable 表示使能 DHCP。
- int AdapterDevicePing(struct Adapter *adapter, const char *destination), 用于配置适配器 PING 功能，其中 adapter 表示需要操作的适配器，destination 表示目的 ip 地址。
- int AdapterDeviceNetstat(struct Adapter *adapter), 用于显示适配器网络状态，其中 adapter 表示需要操作的适配器。

2.2 ARM 架构的网络连接

2.2.1 LoRa

ARM 砒璜硬件终端的 LoRa 模组型号为 RA-01，采用 SPI 接口。该模组的射频芯片为 SX1278，其主要采用 LoRa™远程调制解调器，用于超长距离扩频通信，具有抗干扰性强、低功耗、高灵敏度等特点。

通过 lora 实现主从设备通信的过程体现在 connect_lora_spi.c 文件中。其中，lora 设备的打开、关闭、读、写功能被封装进 lora_done，具体如代码 6-2-9：

代码 6-2-9 Lora 设备对应操作封装

```

01 static const struct LoraDevDone lora_done =
02 {
03     .open = SpiLoraOpen,
04     .close = SpiLoraClose,
05     .write = SpiLoraWrite,
06     .read = SpiLoraRead,
07 };

```

在使用 lora 设备前需要先进行初始化，LoraSx12xxSpiDeviceInit 函数用于通用 sx12xx 系列芯片的初始化，具体如代码 6-2-10：

代码 6-2-10 Lora 设备初始化过程

```

01 int LoraSx12xxSpiDeviceInit(void)
02 {
03     #ifdef BSP_USING_SPI1
04         if(NONE == SpiLoraInit(SPI_BUS_NAME_1, SPI_1_DEVICE_NAME_0,
05                               SPI_1_DRV_NAME, SX12XX_DEVICE_NAME)) {
06             return ERROR;
07         }
08     #endif
09     return EOK;
10 }

```

其中，使用 BSP_USING_SPI1 宏命令将 spi1 总线与 lora 设备相关联，当 spi1 总线打开时，将自动进行 lora 设备的初始化操作。在此过程中，实现 lora 设备的具体初始化由 SpiLoraInit 函数实现，其具体如下：

代码 6-2-11 SPI 初始化过程

```

01 SpiLoraDeviceType SpiLoraInit(char *bus_name, char *dev_name, char *drv_name, char
02                               *lora_name)

```

```

02  {
03      NULL_PARAM_CHECK(dev_name);
04      NULL_PARAM_CHECK(drv_name);
05      NULL_PARAM_CHECK(lora_name);
06      NULL_PARAM_CHECK(bus_name);
07
08      x_err_t ret;
09      static HardwareDevType haldev;
10
11      haldev = SpiDeviceFind(dev_name, TYPE_SPI_DEV);
12      if (NONE == haldev) {
13          KPrintf("SpiLoraInit find spi haldev %s error! \n", dev_name);
14          return NONE;
15      }
16
17      SpiLoraDeviceType spi_lora_dev = (SpiLoraDeviceType)malloc(sizeof(struct
SpiLoraDevice));
18      if (NONE == spi_lora_dev) {
19          KPrintf("SpiLoraInit malloc spi_lora_dev failed\n");
20          free(spi_lora_dev);
21          return NONE;
22      }
23
24      memset(spi_lora_dev, 0, sizeof(struct SpiLoraDevice));
25
26      spi_lora_dev->spi_dev = CONTAINER_OF(haldev, struct SpiHardwareDevice,
haldev);
27
28      spi_lora_dev->lora_dev.spi_dev_flag = RET_TRUE;
29      spi_lora_dev->lora_dev.haldev.dev_done = (struct HalDevDone *)&lora_done;
30
31      struct Driver *spi_driver = SpiDriverFind(drv_name, TYPE_SPI_DRV);
32      if (NONE == spi_driver) {
33          KPrintf("SpiLoraInit find spi driver %s error! \n", drv_name);
34          free(spi_lora_dev);
35          return NONE;
36      }
37
38      //spi drv get spi dev param (SpiDeviceParam)
39      spi_driver->private_data = spi_lora_dev->spi_dev->haldev.private_data;
40      spi_lora_dev->spi_dev->haldev.owner_bus->owner_driver = spi_driver;

```

```

41
42     ret = SpiDeviceRegister(&spi_lora_dev->lora_dev,
spi_lora_dev->spi_dev->haldev.private_data, lora_name);
43     if (EOK != ret) {
44         KPrintf("SpiLoraInit SpiDeviceRegister device %s error %d\n", lora_name, ret);
45         free(spi_lora_dev);
46         return NONE;
47     }
48
49     ret = SpiDeviceAttachToBus(lora_name, bus_name);
50     if (EOK != ret) {
54         KPrintf("SpiLoraInit SpiDeviceAttachToBus device %s error %d\n", lora_name,
ret);
55         free(spi_lora_dev);
56         return NONE;
57     }
58
59     g_spi_lora_dev = &spi_lora_dev->spi_dev->haldev;
60     return spi_lora_dev;
61 }

```

在 connect_lora_spi.c 文件中, 可以通过 LoraOpen、LoraReceive、LoraSend 三个函数接口进行 lora 设备间的通信, 这三个函数分别定义如下:

代码 6-2-12 Lora 的打开过程

```

01 void LoraOpen(void)
02 {
03     x_err_t ret = EOK;
04
05     bus = BusFind(SPI_BUS_NAME_1);
06     dev = BusFindDevice(bus, SX12XX_DEVICE_NAME);
07     ret = SpiLoraOpen(dev);
08     if (EOK != ret) {
09         KPrintf("LoRa init failed\n");
10         return;
11     }
12     KPrintf("LoRa init succeed\n");
13     return;
14 }

```

代码 6-2-13 Lora 的接收过程

```

01 static void LoraReceive(void)

```

```

02 {
03     uint32 read_length = 0;
04     struct BusBlockReadParam read_param;
05     memset(&read_param, 0, sizeof(struct BusBlockReadParam));
06     read_param.buffer = malloc(SPI_LORA_BUFFER_SIZE);
07     read_length = SpiLoraRead(dev, &read_param);
08
09     KPrintf("LoraReceive length %d\n", read_length);
10     for (int i = 0; i < read_length; i++) {
11         KPrintf("i %d data 0x%x\n", i, ((uint8 *)read_param.buffer)[i]);
12     }
13
14     free(read_param.buffer);
15 }

```

代码 6-2-14 Lora 的关闭过程

```

01 static void LoraSend(int argc, char *argv[])
02 {
03     char Msg[SPI_LORA_BUFFER_SIZE] = {0};
04     struct BusBlockWriteParam write_param;
05     memset(&write_param, 0, sizeof(struct BusBlockWriteParam));
06
07     if (argc == 2) {
08         strncpy(Msg, argv[1], SPI_LORA_BUFFER_SIZE);
09         write_param.buffer = Msg;
10         write_param.size = strlen(Msg);
11         KPrintf("LoraSend data %s length %d\n", Msg, strlen(Msg));
12         SpiLoraWrite(dev, &write_param);
13     }
14 }

```

在测试 lora 设备间的通信前，还需要进行一些配置，在终端输入“make BOARD=hc32 f4a0 menuconfig”命令，进入编译配置菜单，如图 6-2-2 所示。

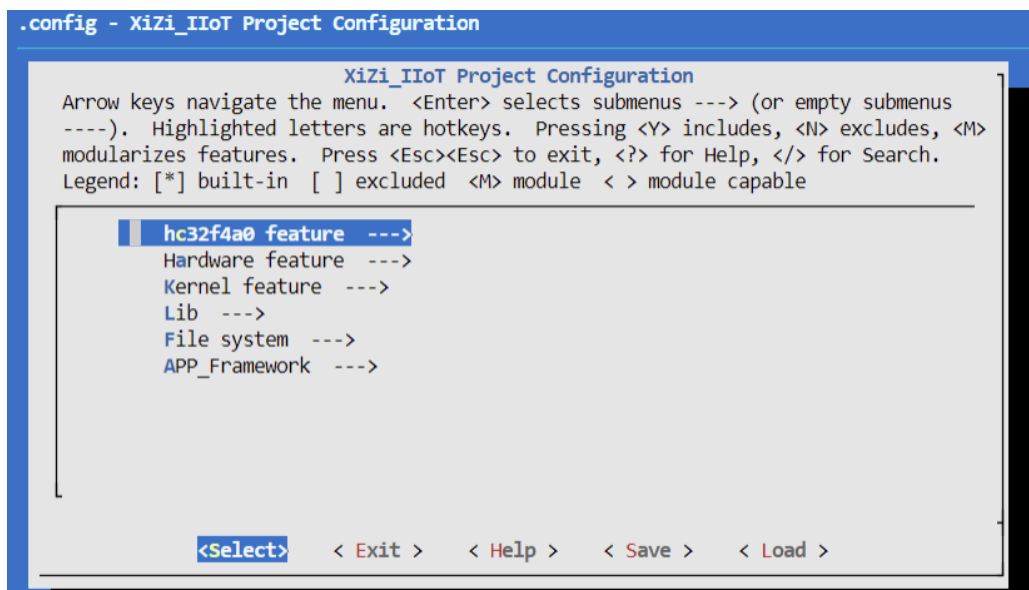


图 6-194 Lora 应用示例的配置过程

首先进入 hc32f4a0 feature 配置菜单，选中 Using SPI device；键入回车键，进入 Using SPI device 配置菜单，选中 Using spi1 和 Using spi lora function，配置完成后具体如图 6-2-3 所示。

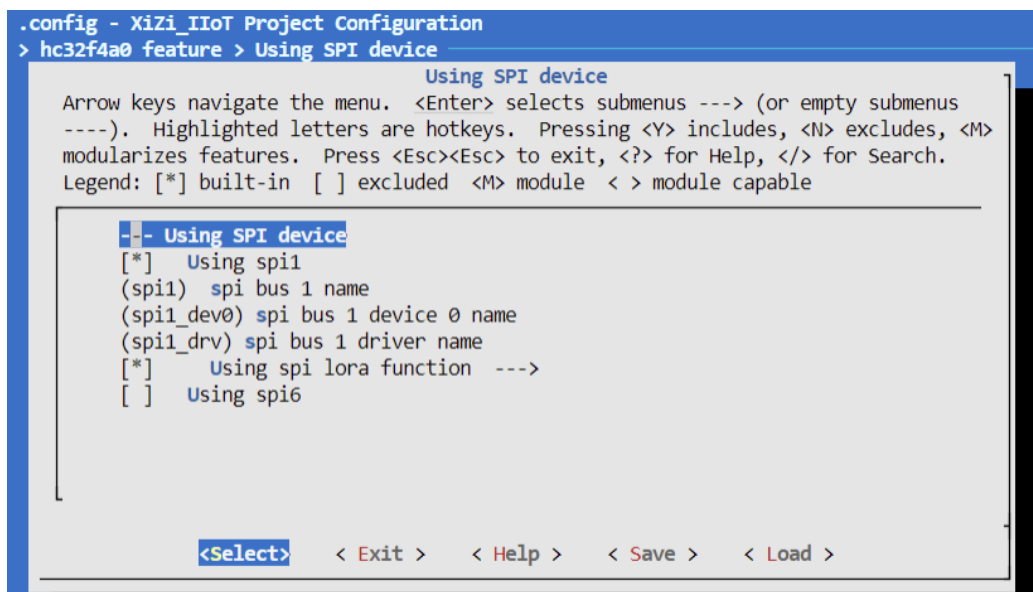


图 6-2-3195 Lora 应用示例的配置过程

最后启动终端进行测试，lora 设备间的通信如下图所示：


```

letter:/$ LoraOpen
SpiLoraOpen start
version code of the chip is 0x1:
SX1278 Lora parameters are :
RFFrequency is 433000000
RFPower is 20
SignalBw is 9
SpreadingFactor is 12
LoRa check ok!
Note: The length of the message
LoRa init succeed
Return: 18, 0x00000012

letter:/$ LoraReceive
SpiLoraRead Ready!
LoraReceive length 6
i 0 data 0x31
i 1 data 0x32
i 2 data 0x33
i 3 data 0x34
i 4 data 0x35
i 5 data 0x36
Return: 0, 0x00000000

Build: Feb 21 2023 20:17:37
Version: 3.0.5
Copyright: (c) 2020 Letter

letter:/$ initialize letter-shell system success
Hello, world!

letter:/$ LoraOpen
SpiLoraOpen start
version code of the chip is 0x12
SX1278 Lora parameters are :
RFFrequency is 433000000
RFPower is 20
SignalBw is 9
SpreadingFactor is 12
LoRa check ok!
Note: The length of the message that can be sent i
time is 256 characters
LoRa init succeed
Return: 18, 0x00000012

letter:/$ LoraSend 123456
LoraSend data 123456 length 6
SpiLoraWrite success!
Return: 0, 0x00000000

```

图 6-196-4 Lora 应用示例的配置过程

2.3 RISC-V 架构的网络连接

2.3.1 WIFI

WIFI 的实现主要在 `adapter_wifi.c` 和 `esp8285_wifi.c` 文件中。`adapter_wifi.c` 文件主要实现了 WIFI 的初始化；`esp8285_wifi.c` 文件中主要完成 WIFI 的打开、关闭、发送和接收功能。

代码 6-2-15 展示的是 `adapter_wifi.c` 文件中的 `AdapterWifiInit` 函数。代码行 4~10 声明一个结构体，是所有通信都用到的结构体，并对其进行内存分配和初始化。代码行 12~17 将 WIFI 驱动注册到适配器链表。代码行 19~27 将 `Esp8285Wifi` 挂接到适配器上。

代码 6-2-15 WIFI 初始化

```

01 int AdapterWifiInit(void){
02     int ret = 0;
03
04     struct Adapter *adapter = PrivMalloc(sizeof(struct Adapter));
05     if (!adapter) {
06         printf("AdapterWifiInit malloc error\n");
07         PrivFree(adapter);
08         return -1;
09     }
10     memset(adapter, 0, sizeof(struct Adapter));
11
12     ret = AdapterWifiRegister(adapter);
13     if (ret < 0) {
14         printf("AdapterWifiInit register wifi adapter error\n");
15         PrivFree(adapter);

```

```

16     return -1;
17 }
18
19 AdapterProductInfoType product_info = Esp8285WifiAttach(adapter);
20 if (!product_info) {
21     printf("AdapterWifiInit ESP8285 attach error\n");
22     PrivFree(adapter);
23     return -1;
24 }
25 adapter->product_info_flag = 1;
26 adapter->info = product_info;
27 adapter->done = product_info->model_done;
28
29 return ret;
30 }

```

代码 6-2-16 是 esp8285_wifi.c 文件中的 Esp8285WifiOpen 函数，用于打开一个 WIFI 设备。

代码 6-2-16252 WIFI 打开流程

```

01 static int Esp8285WifiOpen(struct Adapter *adapter){
02     Esp8285UartOpen(adapter);
03
04     if (!adapter->agent) {
05         char *agent_name = "wifi_uart_client";
06         if (EOK != InitATAgent(agent_name, adapter->fd, 512)) {
07             printf("at agent init failed !\n");
08             return -1;
09         }
10         ATAgentType at_agent = GetATAgent(agent_name);
11         adapter->agent = at_agent;
12     }
13
14     AtSetReplyEndChar(adapter->agent, 'O', 'K');
15     ADAPTER_DEBUG("Esp8285Wifi open done\n");
16
17     return 0;
18 }

```

代码 6-2-17 是 esp8285_wifi.c 文件中的 Esp8285WifiClose 函数，用于关闭一个 WIFI 设备。

代码 6-2-17 WIFI 关闭流程

```

01 static int Esp8285WifiClose(struct Adapter *adapter){
02     Esp8285WifiSetDown(adapter);
03     PrivClose(adapter->fd);
04     return 0;
05 }

```

代码 6-2-18 是 esp8285_wifi.c 文件中的 Esp8285WifiSend 函数，作用是通过 WIFI 发送数据。

代码 6-2-18 WIFI 发送流程

```

01 static int Esp8285WifiSend(struct Adapter *adapter, const void *data, size_t len){
02     x_err_t result = EOK;
03     if (adapter->agent) {
04         EntmSend(adapter->agent, (const char *)data, len);
05     } else {
06         printf("Esp8285WifiSend can not find agent!\n");
07     }
08     return result;
09 }

```

代码 6-2-19 是 esp8285_wifi.c 文件中的 Esp8285WifiReceive 函数，作用是通过 WIFI 接收数据。

代码 6-2-19 WIFI 接收流程

```

01 static int Esp8285WifiReceive(struct Adapter *adapter,
void *rev_buffer, size_t buffer_len){
02     x_err_t result = EOK;
03     printf("esp8285 receive waiting ... \n");
04
05     if (adapter->agent) {
06         return EntmRecv(adapter->agent, (char *)rev_buffer, buffer_len, 40);
07     } else {
08         printf("Esp8285WifiReceive can not find agent!\n");
09     }
10     return result;
11 }

```

使用 ESP8285 Wifi 模块时，需要对 UART 串口和“泛在互联”框架进行配置。如图 6-2-5 所示，需要使能 UART1，并将 UART1 的发送引脚 TXD 和接收引脚 RXD 分别设置为 7 和 6。

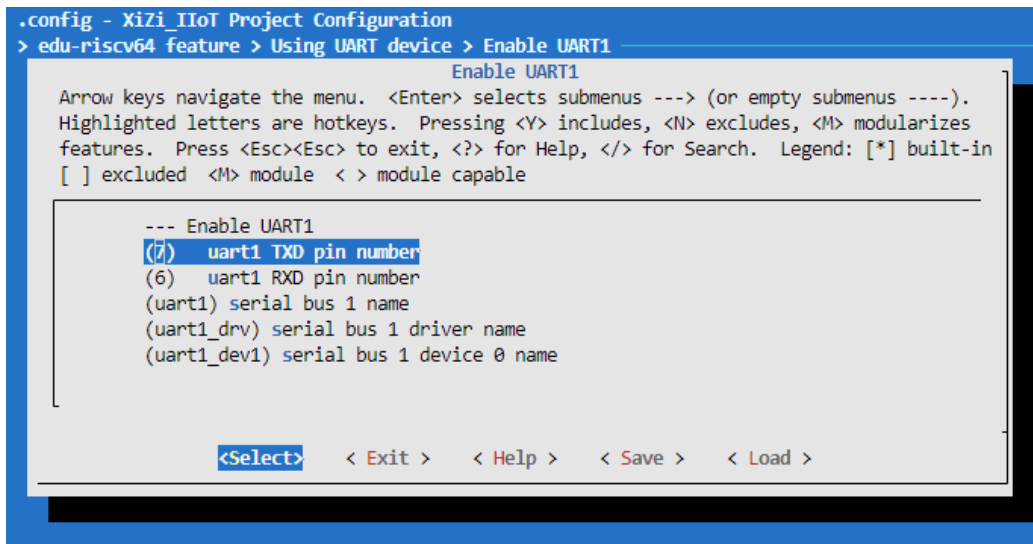


图 6-2-5 UART1 相关配置

如图 6-2-6 所示，在选中 support connection framework 后，进入子菜单，再选中 Using wifi adapter device 即可。

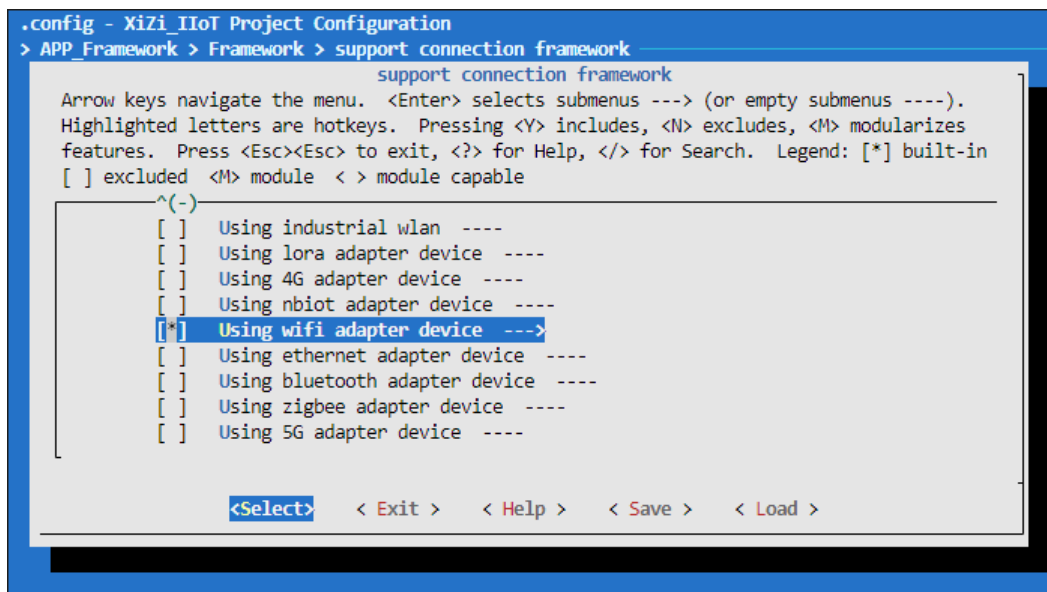


图 6-2-6 框架层配置

在完成程序的编译和烧录后，进入 letter-shell，即可通过命令行对 WiFi 功能进行控制。执行 WifiOpen 命令后，显示如图 6-2-7 所示的结果即意味着命令成功执行。

```
WifiOpen
esp8285 uart config ready
Device wifi param 0x0 open success.
Return: 0, 0x00000000
```

图 6-2-7 WifiOpen 命令执行结果

WifiSetup 命令用于连接 WiFi 热点，其参数依次为 WiFi SSID 和密码。如图 6-2-8 所示，输入 WifiSetup 命令后，终端将显示模块的相关回复，并返回本机和热点对应的 IP 和 MAC 地址。

```
letter:/$ WifiSetup test 12345678
AT send ATE0
len 6
GetCompleteATReply done
[reply result:
OK]
AT send AT
len 4
GetCompleteATReply done
[reply result:
OK]
AT send AT+CWMODE=3
len 13
GetCompleteATReply done
[reply result:
OK]
AT send AT+CWJAP="test","12345678"
len 28
GetCompleteATReply done
[reply result: WIFI CONNECTED
WIFI GOT IP

OK]
AT send AT+CIFSR
len 10
GetCompleteATReply done
[
+R:APIP,"192.168.4.1"
+CIFSR:APMAC,"3e:61:05:8a:fa:10"
+CIFSR:STAIP,"192.168.35.54"
+CIFSR:STAMAC,"3c:61:05:8a:fa:10"

OK]
Device wifi setup success.
Return: 0, 0x00000000
```

图 6-2-8 WifiSetup 命令执行结果

在连接 Wifi 热点后，可通过 WifiPing 命令对特定 IP 执行 Ping 操作。如图 6-2-9 所示，对 180.101.50.242 进行了 Ping 操作，并返回网络延时为 98ms。

```
letter:/$ WifiPing 180.101.50.242
ping 180.101.50.242
AT send AT+PING="180.101.50.242"
len 26
GetCompleteATReply done
[reply result: +98

OK]
ping [180.101.50.242] ok
Device wifi ping success.
Return: 0, 0x00000000
```

图 6-2-9 WifiPing 命令执行结果

第3章 “实时认知” 实践

3.1 “实时认知” 框架介绍

深度学习在近年来随着研究的深入逐渐实用化，并具备许多传统算法无可替代的优势，实现了传统算法难以实现的功能，目前较为成熟的应用包括分类、检测等。嵌入式系统的计算和内存资源通常是十分受限的，因而目前将深度学习部署在嵌入式系统尤其是 MCU 中还具备一定的挑战。另外，由于开发环境和资源的问题，MCU 的深度学习网络部署在软件适配方面也是一大挑战。

XiUOS 通过集成多种不同的主流开源深度学习框架，来尽可能解决各种资源受限、软件适配的问题，并提供一套完整的部署流程和方法，简化部署的复杂度。

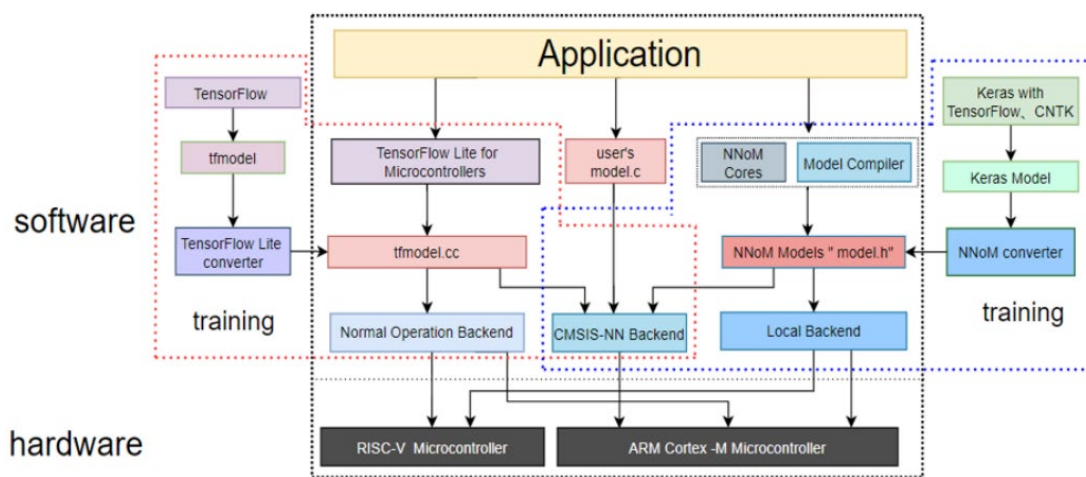


图 6-197-1 “实时认知”框架整体结构图

如图 6-3-1 所示，XiUOS 集成了 TensorFlow Lite for Microcontroller、CMSIS-NN、NN oM 三套深度学习框架，为不同的内核架构提供完善的软件支持和硬件加速功能。

MCU 和 NNOM 以及直接调用 CMSIS-NN 算子，其中针对 ARM 系列处理器，CMSIS-NN 算子也可以为 TensorFlow Lite for MCU 和 NNOM 在推理过程中加速使用。针对非 ARM 处理器的 MCU 可以直接使用二者自带的算子进行推理。另外，利用 TensorFlow Lite 和 NNOM 自带的相关工具可以将相应的训练好的模型转换成对应的格式。针对不同的硬件资源以及硬件特性可以选择不同的部署方式。

此外，对于具备特殊硬件加速功能的芯片如 K210，XiUOS 也具备有相应的软件支持，可以加速模型的部署进程。为了保证接口的主流性，XiUOS 并未对各深度学习框架进行过多的修改，因而读者可以根据自身需求根据各框架的官方文档进行相关程序的编写。

3.1.1 TensorFlow Lite for Microcontroller

TensorFlow Lite for Microcontroller 是由谷歌公司开发的一款基于 C++实现的神经网络框架。相比于 TensorFlow Lite，前者进一步裁剪了计算及内存资源的占用。TensorFlow Lite for Microcontroller 对 TensorFlow 框架有着极好的支持，也可以从 Pytorch、Keras、ONNX 等主流深度学习框架中进行模型转换。TensorFlow Lite for Microcontroller 的主要优点为模型转换简单、不依赖于内核架构且算子支持相对完善。

XiUOS 中 TensorFlow Lite for Microcontroller 框架在 Framework/knowning 目录下，开发者可以在 Application 下的自行创建头文件并编写相关程序，程序头文件的包含方法如代码 6-3-1 所示。读者应当在代码行 11 至 13、60 至 62、70 至 72 分别包含模型文件和编写模型的输入和输出的相关代码即可。

代码 6-3-1 TensorFlow Lite for Microcontroller 框架代码示例

```

01 #include <stdio>
02 #include <transform.h>
03 #include <stdio.h>
04
05 #include "tensorflow/lite/micro/all_ops_resolver.h"
06 #include "tensorflow/lite/micro/micro_error_reporter.h"
07 #include "tensorflow/lite/micro/micro_interpreter.h"
08 #include "tensorflow/lite/schema/schema_generated.h"
09 #include "tensorflow/lite/version.h"
10
11 /*
12 include your model file here
13 */
14
15 namespace {
16 tflite::ErrorReporter* error_reporter = nullptr;
17 const tflite::Model* model = nullptr;
18 tflite::MicroInterpreter* interpreter = nullptr;
19 TfLiteTensor* input = nullptr;
20 TfLiteTensor* output = nullptr;
21 constexpr int kTensorArenaSize = 110 * 1024;
22 uint8_t* tensor_arena = nullptr;
23 //uint8_t tensor_arena[kTensorArenaSize];
24 }
25

```

```
26 extern "C" void mnist_app() {
27     tflite::MicroErrorReporter micro_error_reporter;
28     error_reporter = &micro_error_reporter;
29
30     model = tflite::GetModel(my_model);
31     if (model->version() != TFLITE_SCHEMA_VERSION) {
32         TF_LITE_REPORT_ERROR(error_reporter,
33                               "Model provided is schema version %d not equal "
34                               "to supported version %d.",
35                               model->version(), TFLITE_SCHEMA_VERSION);
36     }
37     return;
38 }
39
40 tensor_arena = (uint8_t *)malloc(kTensorArenaSize);
41 if (tensor_arena == nullptr) {
42     TF_LITE_REPORT_ERROR(error_reporter,
43                           "malloc for tensor_arena failed");
44     return;
45 }
46
47 tflite::AllOpsResolver resolver;
48 tflite::MicroInterpreter static_interpreter(
49     model, resolver, tensor_arena, kTensorArenaSize, error_reporter);
50 interpreter = &static_interpreter;
51
52 // Allocate memory from the tensor_arena for the model's tensors.
53 TfLiteStatus allocate_status = interpreter->AllocateTensors();
54 if (allocate_status != kTfLiteOk) {
55     TF_LITE_REPORT_ERROR(error_reporter, "AllocateTensors() failed");
56     return;
57 }
58
59 input = interpreter->input(0);
60 output = interpreter->output(0);
61
62 /*
63  * transfer your input to your model here
64  */
65
66 TfLiteStatus invoke_status = interpreter->Invoke();
67 if (invoke_status != kTfLiteOk) {
```

```

66     TF_LITE_REPORT_ERROR(error_reporter, "Invoke failed on x_val\n");
67     return;
68 }
69
70 /*
71 Read the predicted y value from the model's output tensor
72 */
73 }

```

3.1.2 CMSIS-NN

CMSIS-NN 是由 ARM 公司设计的一款基于 ARM 架构的强大的嵌入式深度学习框架，其主要特点是针对 ARM 的相关指令集进行了专门的优化，使得 CMSIS-NN 在 ARM 架构上的运行速度十分可观。

CMSIS NN 架构设计简单，其核心可分为两个部分。如图 6-3-2 所示，一部分是基础神经网络算子的实现函数，另一部分是算子实现所对应的实用函数。

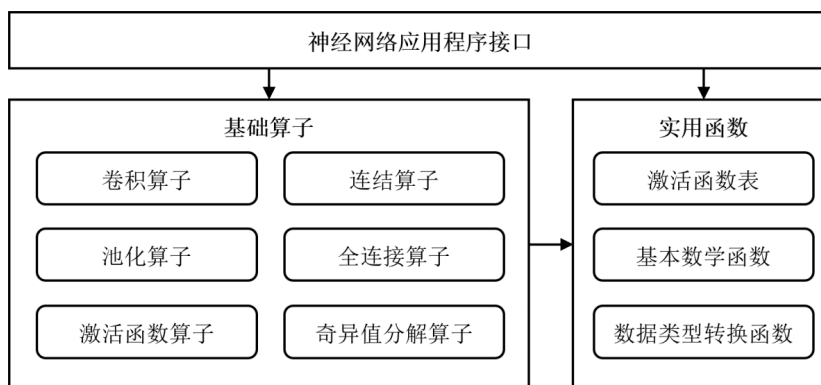


图 6-3-2 CMSIS-NN 框架结构图

CMSIS-NN 通过对算子计算原理和硬件结构的分析，实现了部分基础算子的优化以减少计算过程中的内存占用，并合理运用 SIMD 指令、数字信号处理单元（DSP）与浮点运算单元（FPU）等相关硬件单元，提高了计算效率，大幅缩短了模型推理的时间。基于 CMSIS-NN 编写的代码需要开发者根据深度学习的模型进行编写。

3.1.3 NNoM

NNoM 是一款专为 MCU 设计的高性能神经网络推断框架。其支持丰富的算子模型结构，良好的易用性和高性能的后端选项。NNoM 软件架构如图 6-3-3 所示。NNoM 支持本地后端或是 CMSIS-NN 后端，因而可以兼容 RISC-V 架构和 ARM 架构。同时 NNoM 还提供了大量相关的软件支持工具，包括代码构建工具、模型转换器等应用程序。

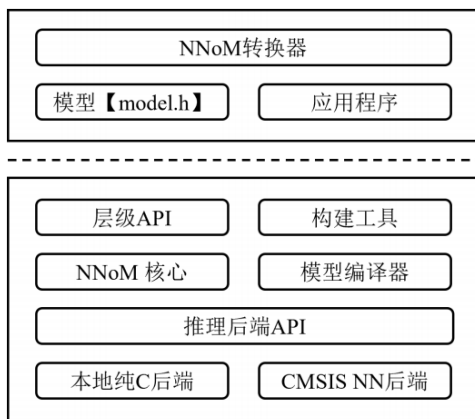


图 6-3-3 NNoM 框架结构图

NNoM 框架下的代码编写较为简单，相关模板可见代码 6-3-2。其中 `nnom_model_create`、`nnom_input_data` 均在 `.h` 的模型文件中进行了声明和定义。需要更改的部分为代码行 7 处的推断过程中完成每一层运算后的回调函数 `layer_callback` 和代码行 9 处的数据加载部分。

代码 6-3-2 NNoM 框架代码示例

```

01 int main(int argc, char* argv[])
02 {
03     nnom_model_t* model;
04     int8_t* input;
05
06     model = nnom_model_create();
07     model_set_callback(model,
                        layer_callback); // callback to save output of each layer.
08
09     input = load(data); // get your test data
10
11     memcpy(nnom_input_data, input, sizeof(nnom_input_data));
12     model_run(model);
13
14     model_delete(model);
15     free(input);
16
17     return 0;
18 }
  
```

3.2 ARM 架构下的“知”框架应用

ARM 架构矽璜硬件终端支持了 3 套主流的神经网络框架，包括 Tensorflow Lite for microcontrollers、NNoM 和 CMSIS-NN。

3.2.1 ARM 架构下的 Tensorflow Lite for microcontrollers 框架应用

Tensorflow Lite for microcontrollers 的模型文件为 tflite 或.h 头文件，二者的不同之处在于 tflite 文件需要文件系统的支持，即需要从 U 盘或 SD 卡中进行读取；而.h 头文件是在编程过程中直接进行包含并调用其中参数，最终直接编译进入可烧录的二进制文件中，也因此而不依赖于文件系统的支持。Tensorflow Lite for microcontrollers 的模型文件包含了模型结构和参数，因而对于框架支持的算子，可以通过直接调用框架内提供的 API 来直接实现程序的运行。

对于不支持的算子，则需要额外编写自定义代码来进行具体实现。

tflite 文件可通过 TensorFlow 模型转换而来。C 语言头文件形式的模型文件。其可通过 tflite 文件转换而来，其执行命令为：

```
$ xxd -i converted_model.tflite > model_data.h
```

在 XiZi 中的应用代码主体部分和包含文件与 Tensorflow Lite for microcontrollers 官方用例和使用方法一致，读者亦可以参考 Tensorflow Lite for microcontrollers 官方文档进行参考。

通常地，Tensorflow Lite for microcontrollers 是基于 C++实现框架的调用。代码 6-3-3 是基于生成的头文件模型编写的示例代码。从代码行 1 至 12 是对相关头文件进行了包含，代码行 14 至 23 创建了相关全局变量的指针和结构体变量。代码行 25 至 91 是代码的主体实现部分。其主要步骤为：获取模型（对应代码行 29 至 36）、创建解释器（对应代码行 44 至 47）、分配运算张量（对应代码行 50 至 54）、获取输入输出的相应地址并填充相应数据（对应代码行 56 至 72）、执行模型推断（对应代码行 74 至 78）、获取模型输出（对应代码行 80 至 90）。

代码 6-3-3 Tensorflow Lite for microcontrollers 应用示例

```
01 #include <stdio>
02 #include <transform.h>
03 #include <stdio.h>
04
05 #include "tensorflow/lite/micro/all_ops_resolver.h"
06 #include "tensorflow/lite/micro/micro_error_reporter.h"
07 #include "tensorflow/lite/micro/micro_interpreter.h"
08 #include "tensorflow/lite/schema/schema_generated.h"
```

```

09 #include "tensorflow/lite/version.h"
10
11 #include "digit.h"
12 #include "model.h"
13
14 namespace {
15 tflite::ErrorReporter* error_reporter = nullptr;
16 const tflite::Model* model = nullptr;
17 tflite::MicroInterpreter* interpreter = nullptr;
18 TfLiteTensor* input = nullptr;
19 TfLiteTensor* output = nullptr;
20 constexpr int kTensorArenaSize = 110 * 1024;
21 uint8_t *tensor_arena = nullptr;
22 //uint8_t tensor_arena[kTensorArenaSize];
23 }
24
25 extern "C" void mnist_app() {
26     tflite::MicroErrorReporter micro_error_reporter;
27     error_reporter = &micro_error_reporter;
28
29     model = tflite::GetModel(mnist_model);
30     if (model->version() != TFLITE_SCHEMA_VERSION) {
31         TF_LITE_REPORT_ERROR(error_reporter,
32                               "Model provided is schema version %d not equal "
33                               "to supported version %d.",
34                               model->version(), TFLITE_SCHEMA_VERSION);
35     }
36 }
37
38 tensor_arena = (uint8_t *)malloc(kTensorArenaSize);
39 if (tensor_arena == nullptr) {
40     TF_LITE_REPORT_ERROR(error_reporter, "malloc for tensor_arena failed");
41     return;
42 }
43
44 tflite::AllOpsResolver resolver;
45 tflite::MicroInterpreter static_interpreter(
46     model, resolver, tensor_arena, kTensorArenaSize, error_reporter);
47 interpreter = &static_interpreter;
48
49 // Allocate memory from the tensor_arena for the model's tensors.

```

```
50  TfLiteStatus allocate_status = interpreter->AllocateTensors();
51  if (allocate_status != kTfLiteOk) {
52      TF_LITE_REPORT_ERROR(error_reporter, "AllocateTensors() failed");
53      return;
54  }
55
56  input = interpreter->input(0);
57  output = interpreter->output(0);
58
59  printf("----- Input Digit -----\\n");
60  for (int i = 0; i < 28; i++) {
61      for (int j = 0; j < 28; j++) {
62          if (mnist_digit[i*28+j] > 0.3)
63              printf("#");
64          else
65              printf(".");
66      }
67      printf("\\n");
68  }
69
70  for (int i = 0; i < 28*28; i++) {
71      input->data.f[i] = mnist_digit[i];
72  }
73
74  TfLiteStatus invoke_status = interpreter->Invoke();
75  if (invoke_status != kTfLiteOk) {
76      TF_LITE_REPORT_ERROR(error_reporter, "Invoke failed on x_val\\n");
77      return;
78  }
79
80  // Read the predicted y value from the model's output tensor
81  float max = 0.0;
82  int index;
83  for (int i = 0; i < 10; i++) {
84      if (output->data.f[i] > max) {
85          max = output->data.f[i];
86          index = i;
87      }
88  }
89  printf("----- Output Result -----\\n");
90  printf("result is %d\\n", index);
```

[illegible]

图 6-198 Tensorflow Lite for microcontrollers 应用例程的运行结果

NNoM 框架的模型文件为 C 语言头文件形式，其可从 keras 框架模型文件转换而来，其具体转换方法为调用 NNoM 提供的 python 脚本中的 generate_model 函数来实现。

在 NNoM 生成的模型文件中会包含模型调用的主要函数的声明和框架内接口调用，如 `nnom_model_create` 等。代码 6-3-4 是 NNoM 框架下基于手写数字的一个示例代码，其中代码行 3 为调用模型文件中的创建模型接口来创建一个完整的模型并加载相关参数。代码行 5 至 14 进行相关变量的声明并获取命令参数。代码行 18 至 19 将数据输入进模型中，并执行模型的推断，其中 `nnom_input_data` 为在模型文件中的数组变量。代码行 22 至 26 执行了相关输入与输出数据的打印。

代码 6-3-4 NNoM 框架应用示例

```
01 void mnist_nnom(int argc, char **argv)
02 {
03     model = nnom_model_create();
04
05     uint32_t tick, time;
06     uint32_t predic_label;
```

```

07 float prob;
08 int32_t index = atoi(argv[1]);
09
10 if (index >= TOTAL_IMAGE || argc != 2)
11 {
12     printf("Please input image number within %d\n", TOTAL_IMAGE - 1);
13     return;
14 }
15
16 printf("\nprediction start.. \n");
17
18 memcpy(nnom_input_data, (int8_t *)&img[index][0], 784);
19 nnom_predict(model, &predic_label, &prob);
20
21 // print original image to console
22 print_img((int8_t *)&img[index][0]);
23
24 printf("Truth label: %d\n", label[index]);
25 printf("Predicted label: %d\n", predic_label);
26 printf("Probability: %d%%\n", (int)(prob * 100));
27 }

```

最终运行结果如图 6-3-5 所示，输入为图中所示的手写字体，识别结果为 6。

```

NNoM version 0.4.3
To disable logs, please void the marco 'NNOM_LOG(...)' in 'nnom_port.h'.
Data format: Channel Last (HWC)
Start compiling model...
Layer(#) Activation output shape ops(MAC) mem(in, out, buf) mem blk lifetime
-----
#1 Input - - ( 28, 28, 1,) ( 784, 784, 0) 1 - - - - -
#2 Conv2D - ReLU - ( 28, 28, 12,) 84k ( 784, 9408, 0) 1 1 - - - -
#3 MaxPool - - ( 14, 14, 12,) ( 9408, 2352, 0) 1 1 1 - - - -
#4 Conv2D - ReLU - ( 14, 14, 24,) 508k ( 2352, 4704, 0) 1 - 1 - - - -
#5 MaxPool - - ( 7, 7, 24,) ( 4704, 1176, 0) 1 1 1 - - - -
#6 Conv2D - ReLU - ( 7, 7, 48,) 508k ( 1176, 2352, 0) 1 - 1 - - - -
#7 MaxPool - - ( 4, 4, 48,) ( 2352, 768, 0) 1 1 1 - - - -
#8 Flatten - - ( 768, ) ( 768, 768, 0) - - 1 - - - -
#9 Dense - ReLU - ( 96, ) 73k ( 768, 96, 1536) 1 1 1 - - - -
#10 Dense - - ( 10, ) 960 ( 96, 10, 192) 1 1 1 - - - -
#11 Softmax - ( 10, ) ( 10, 10, 0) 1 - 1 - - - -
#12 Output - - ( 10, ) ( 10, 10, 0) 1 - - - - -
-----
Memory cost by each block:
blk_0:4704 blk_1:9408 blk_2:2352 blk_3:0 blk_4:0 blk_5:0 blk_6:0 blk_7:0
Memory cost by network buffers: 16464 bytes
Total memory occupied: 130424 bytes

prediction start..

      ]]tt0011
      ::qq88@bb88>>
      ~dd@pp<<,ppCC
      ~\@e88qg:: ::_
      \@e88qg:: ::_
      llzz@ee
      nne@88{
      j]s@ett
      ~\@e88qg::
      \\\@e88aacil
      qq@e88aacil
      >88@e88aacil
      L@e88-.. me@
      ~\@e88qg::
      j]s@ett LL@e))
      })@e88 LL@ecc
      {(e88eo LL@e[[
      dd@e88CC ~^pp@e
      {(e88eo~^pp@e
      ^^{**@e88qg::
      @e88o11

Truth label: 6
Predicted label: 6
Probability: 100%
Return: 18, 0x00000012

```

图 6-199 NNoM 应用例程的运行结果

3.2.3 ARM 开发板下的 CMSIS-NN 库应用

CMSIS-NN 可以作为 Tensorflow Lite for microcontrollers、NNoM 等框架的后端，也可独立编写应用代码。CMSIS-NN 库的模型文件包含多个 C 语言头文件和源文件，通常 CMSIS-NN 框架模型的生成以文件夹的形式。其中生成的 `nn.h` 中包含有相应的调用接口。

代码 6-3-5 是 CMSIS-NN 的应用代码示例，其功能为从文件系统中读取并识别 32×32 分辨率 RGB 图像，最终输出该图像的分类。代码中除了使用 CMSIS-NN 库外，还调用了 TJPGDEC 库文件。TJPGDEC 库的主要作用是解析 jpg 格式的图片文件来获取其 RGB 数值作为输入。代码行 3 至 6 中创建了包含相关变量；代码行 8 至 19 中对命令参数进行了判断，并打开指定的图像文件；代码行 21 至 27 对工作的缓存内存空间进行了分配；代码行 29 至 51 调用了 TJPGDEC 库相关接口对已读取到的图像文件进行了解析。代码行 53 将解析出的 RGB 数据输入模型进行推断。代码行 55 至 73 进行了了相关结果的打印并关闭文件。

代码 6-3-5 CMSIS-NN 应用示例

```

01 int cmsisnn_demo(int argc, char *argv[])
02 {
03     void *work;
04     JDEC jdec;
05     JRESULT res;
06     IODEV devid;
07
08     if (argc < 2)
09     {
10         printf("Jpeg_Dec illegal arguments ...\n");
11         return -1;
12     }
13
14     devid.fp = fopen(argv[1], "r+");
15     if (!devid.fp)
16     {
17         printf("Jpeg_Dec open the file failed...\n");
18         return -1;
19     }
20
21     work = malloc(WORK_POOL_SIZE);
22     if (work == NULL)
23     {

```

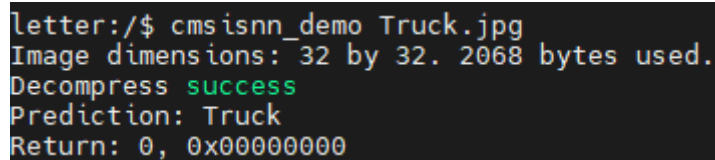
```
24     printf("Jpeg_Dec work malloc failed...\n");
25     res = -1;
26     goto __exit;
27 }
28
29     res = jd_prepare(&jdec, in_func_cmsisnn, work,
                    WORK_POOL_SIZE, (void*)&devid);
30     if (res == JDR_OK)
31     {
32         printf("Image dimensions: %u by %u. %u bytes used.\n", jdec.width,
                    jdec.height, 3100 - jdec.sz_pool);
33
34         devid.fbuf = malloc(3 * jdec.width * jdec.height);
35         if (devid.fbuf == NULL)
36         {
37             printf("Jpeg_Dec devid.fbuf malloc failed,
                    need to use %d Bytes ...\n", 3 * jdec.width * jdec.height);
38             res = -1;
39             goto __exit;
40         }
41         devid.wfbuf = jdec.width;
42
43         res = jd_decomp(&jdec, out_func_cmsisnn, 0);
44         if (res == JDR_OK)
45         {
46             printf("\rDecompress success \n");
47         }
48         else
49         {
50             printf("Failed to decompress: rc=%d\n", res);
51         }
52
53         cmsisnn_inference(devid.fbuf);
54
55         if (devid.fbuf != NULL)
56         {
57             free(devid.fbuf);
58         }
59     }
60     else
61     {
```

```

62     printf("Failed to prepare: rc=%d\n", res);
63 }
64
65 __exit:
66 if (work != NULL)
67 {
68     free(work);
69 }
70
71 fclose(devid.fp);
72
73 return res;
74 }

```

基于 CMSIS-NN 库编写的该应用程序的运行结果如下图 6-3-6 所示:



```

letter:/$ cmsisnn_demo Truck.jpg
Image dimensions: 32 by 32. 2068 bytes used.
Decompress success
Prediction: Truck
Return: 0, 0x00000000

```

图 6-200 CMSIS-NN 应用例程的运行结果

3.3 RISC-V 架构下 K210 的“知”框架

K210 是一款具有卷积网络加速单元(KPU)的嵌入式芯片,通过 KPU 进行卷积网络计算算力可达 1TFPOS(INT8),因而能够实现一些量化后的中小规模卷积网络的计算。K210 原生支持的模型存储格式为 Kmodel 类型,其可通过主流深度学习框架 Tensorflow、Kereas 等训练后模型文件转换得来。不同的应用场景加载训练好的不同的存储格式为 Kmodel 的模型,从而运行不同的算法任务。为了便于开发者专注模型算法,省去编写其他重复代码过程,为此设计了 KPU 加载器。该加载器是针对 K210 芯片在加载模型实现硬件加速而设计的。

3.3.1 RISC-V K210 的 KPU 模型加载器

图 6-3-7 为针对 K210 所设计的 KPU 模型加载器,利用 XiUOS 具备文件系统(elf, fat fs、romfs 等)的特性,可以将.kmodel 结尾的模型文件和.json 格式的描述模型的文件成对匹配存储到 tf 卡或者其他存储设备,然后 XiUOS 成功挂载,通过标准 POSIX 接口读入描述文件,获得与之相应的模型参数,然后调用 K210 的 KPU SDK 进行模型加载。通过该方式,可大大简化模型加载过程,开发者只需关注模型本身而不必关注模型部署过程。同时,可以通过线程的创建和删除,根据实际需求创建不同的应用。

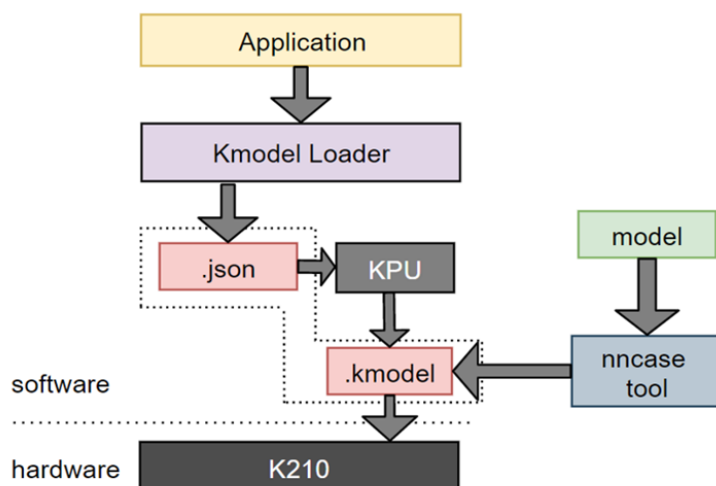


图 6-3-7 K210 KPU 加载器

3.3.2 RISC-V K210 利用 KPU 加载器的应用

在 XIZI 中存在一套加载 YOLOv2 模型的框架，通过读取 json 配置文件和 kmodel 模型文件，可以较容易地实现机器视觉检测功能。k210_yolov2_detect.c 文件实现了对 KPU、摄像头、LCD 显示屏硬件设备的调用和神经网络运行的主体逻辑；json_parser.c 和 region_layer.c 文件对子功能进行了实现。其中 json_parser.c 实现了 kmodel 对应的 json 文件的解析以获得配置参数，region_layer.c 实现了一些辅助操作，如 NMS 非极大值抑制算法、检测框的渲染等。调用接口为 void k210_detect(char *json_file_path)，其参数为 JSON 配置文件的路径，kmodel 模型文件需要与 json 配置文件同名。

“知”框架涉及到了摄像头、LCD 屏幕和 TF 卡的底层驱动的调用。如需要插入一张在根目录下存储有 *.kmodel、*.json 文件的存储卡或 U 盘。以插入 SD 卡的情况下为例，其配置过程如下：

首先，在终端输入“make BOARD=edu-riscv64 menuconfig”命令进入编译配置菜单，进入 edu-riscv64 feature 菜单栏中，将相关硬件设备配置如图 6-3-8，选中 Using SPI device、Using SOFT_SPI device、Using LCD device、Using DMA device、Using GPIO device、Using PLIC device、Using SYSCLOCK device、Using UART device、Using TIMER device、Using camera device、Using camera device、Using kpu device 选项。

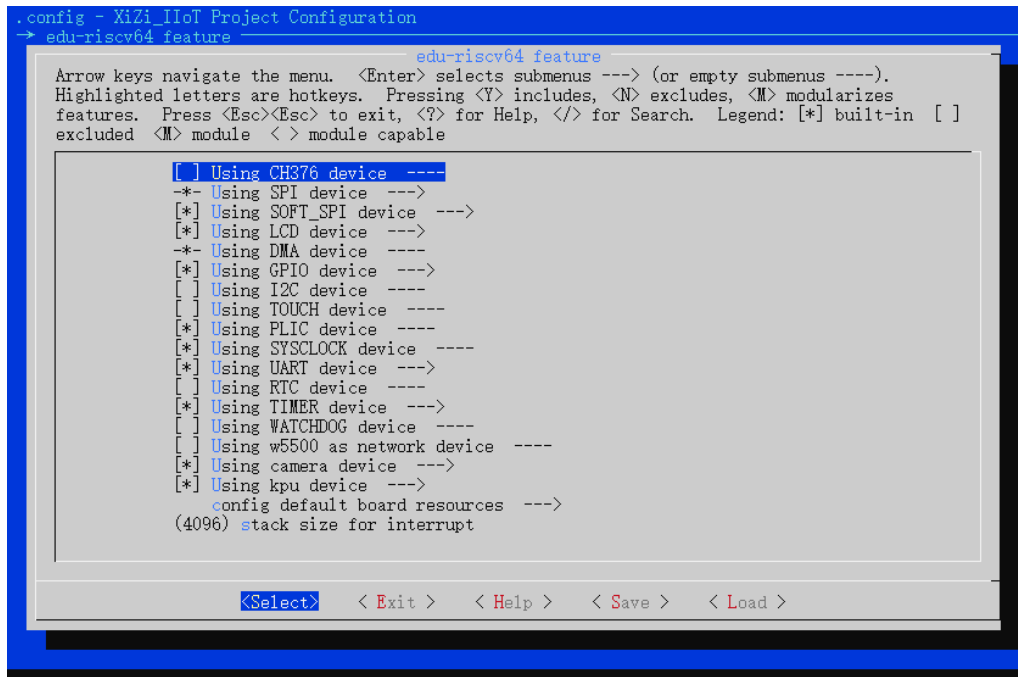


图 6-3-8 硬件设备配置过程

进入 Hardware feature/Using SPI bus drivers 菜单栏下，选中 Using SD/TF card driver with spi 选项。

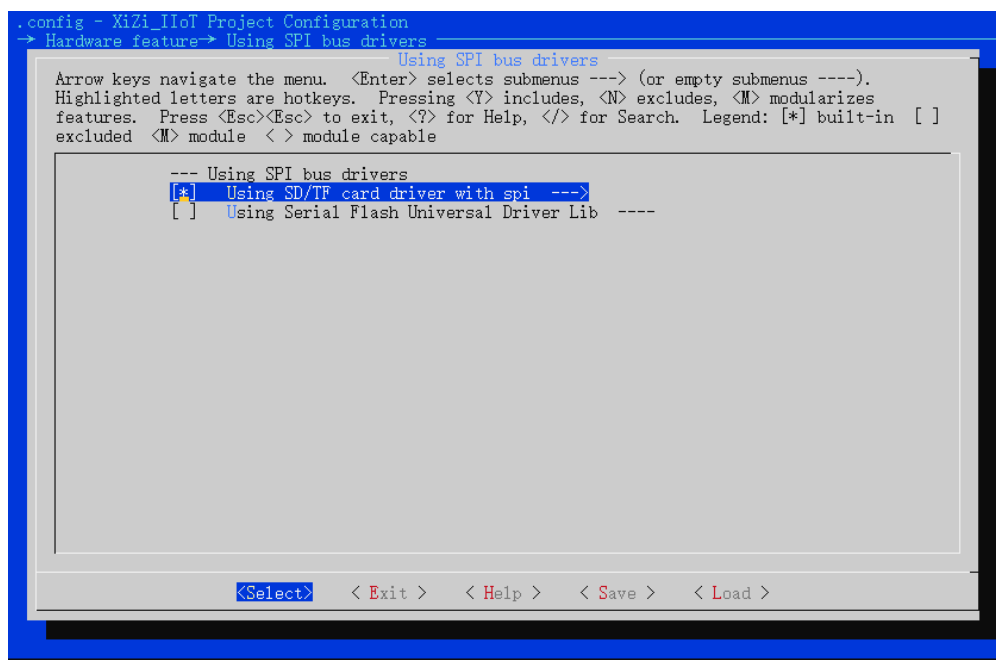


图 6-3-9 设备特性配置

进入 APP_Framework/Framework 选中 support knowing framework，并进入子菜单，选中 kpu model processing，选中 yolov2 region layer、yolov2 model json parser、k210 yolov2 detect procedure 选项。

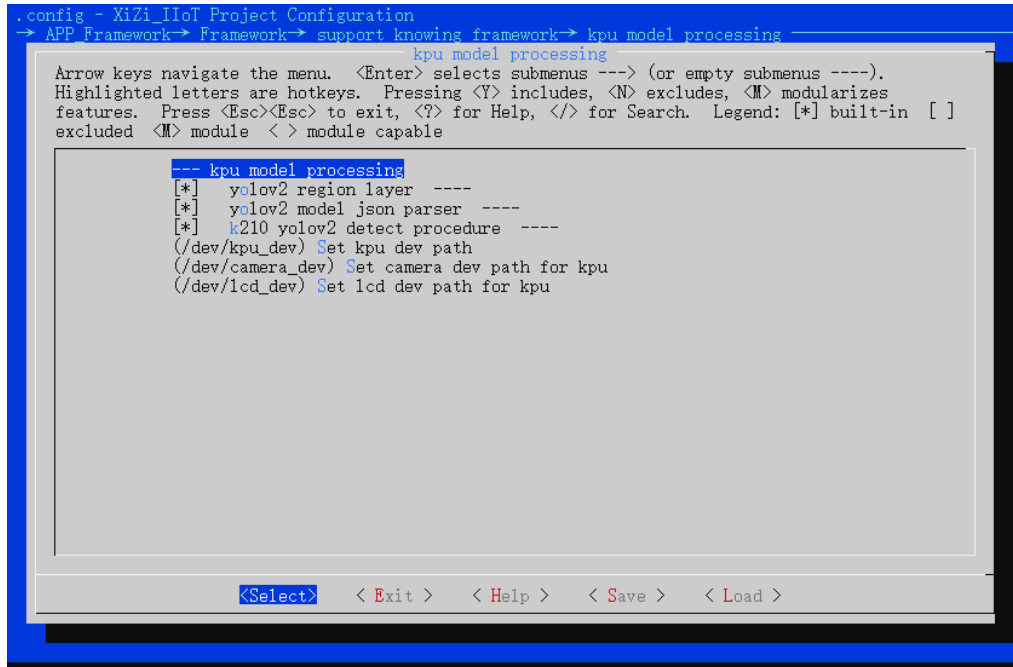


图 6-3-10201 知框架的配置过程

进入 App_Framework/Application/config stack size and priority of main task 子菜单中，将 set main task stack size 这一项设置为 409600。

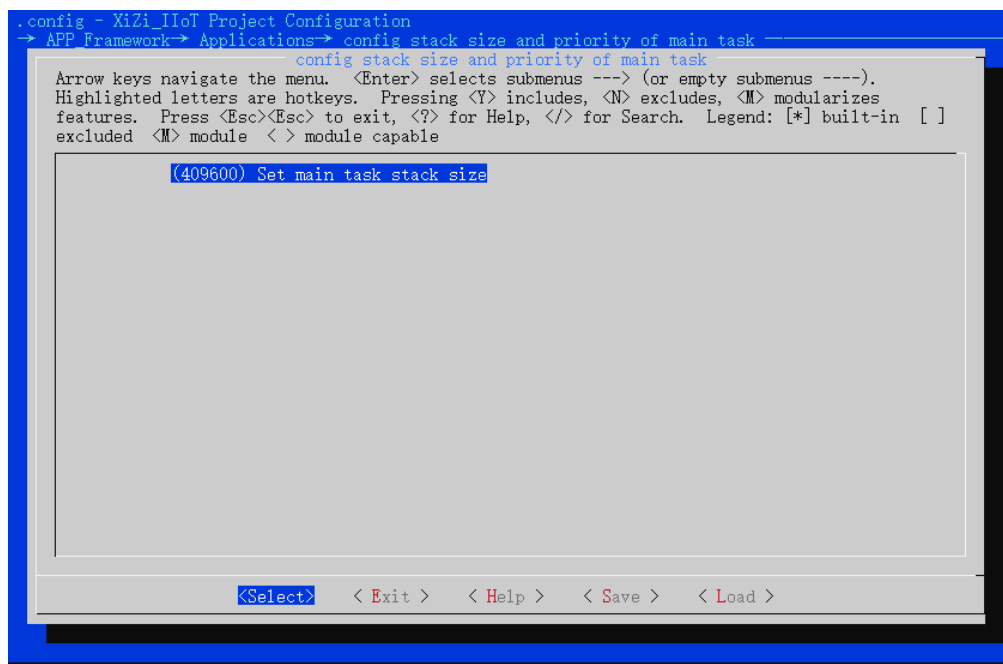


图 6-3-11 任务内存堆大小配置过程

最后，在 APP_Framework/lib 子菜单中勾选 Using cJSON 选项。

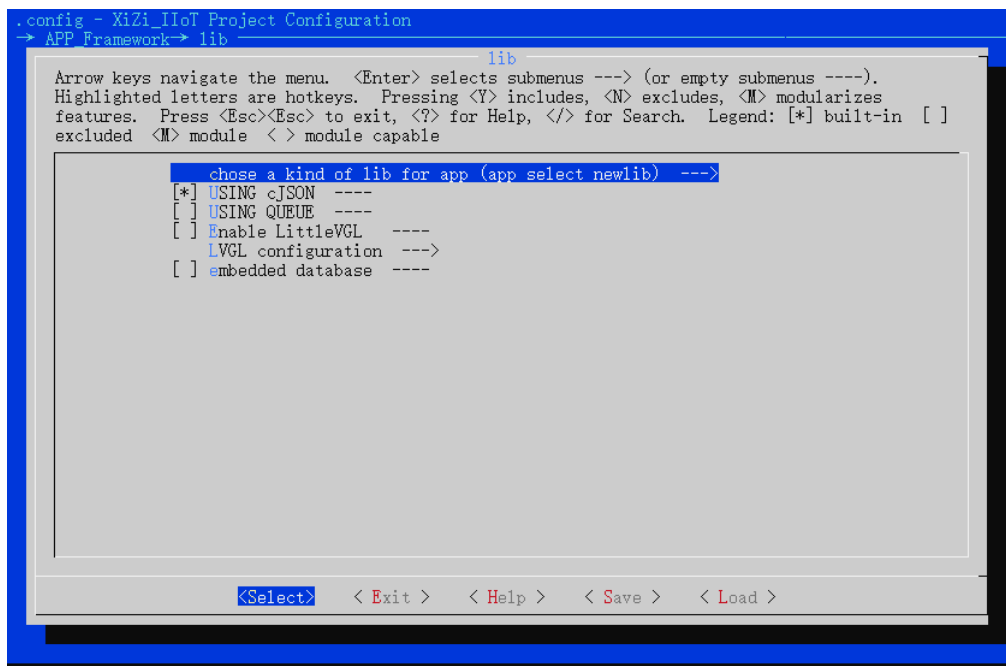


图 6-3-12 cJSON 库配置过程

接下来对 region_layer.c 和 k210_yolov2_detect.c 文件的接口函数的功能进行介绍。

(1) region_layer.c 文件

region_layer.c 文件里主要实现了一些接口函数，处理一些不利于使用 KPU 进行计算的操作。

- int region_layer_init(region_layer_t *rl, int width, int height, int channels, int origin_width, int origin_height), 用于实现对 region_layer_t *结构体指针的内存分配及数值初始化，其中 rl 为用于存储 region 层所需参数的结构体指针；width、height、channels 分别是模型输入的宽、高和通道数；origin_width、origin_height 分别是原图像的宽和高。
- static inline float sigmoid(float x), 通过内联函数的形式，用于计算单个浮点数 x 经 sigmoid 激活以后的数值输出。
- static void activate_array(region_layer_t *rl, int index, int n), 用于对一维数组计算经 sigmoid 激活后的输出。
- static void softmax(region_layer_t *rl, float *input, int n, int stride, float *output), 用于实现 softmax 函数激活。

- `static int entry_index(region_layer_t *rl, int location, int entry)`, 用于将 `entry` 入口值转换为输入数值的索引。
- `static void softmax_cpu(region_layer_t *rl, float *input, int n, int batch, int batch_offset, int groups, int stride, float *output)`, 用于对二维数组计算 sigmoid 激活值。
- `static void forward_region_layer(region_layer_t *rl)`, 根据 `anchor` 的数量计算若干次激活数组的值, 完成 YOLOv2 的最后一层的计算。
- `static void correct_region_boxes(region_layer_t *rl, box_t *boxes)`, 矫正模型输出的检测框的位置, 这是由于模型输入尺寸和图像尺寸不一致所导致需要的额外操作, 其中 `rl` 是存储 `region` 层相关参数的结构体指针, `boxes` 为原输出的检测框列表。
- `static box_t get_region_box(float *x, float *biases, int n, int index, int i, int j, int w, int h, int stride)`, 用于获得 YOLOv2 模型最后一层所输出的对应索引的检测框。
- `static void get_region_boxes(region_layer_t *rl, float *predictions, float **probs, box_t *boxes)`, 获得 YOLOv2 模型最后一层所输出的所有检测框, 用于通过调用 `box_t get_region_box` 函数实现, 其中 `rl` 是 `region_layer` 结构体指针, `predictions`、`probs`、`boxes` 分别为用于存储检测框对应分类可信度、检测框可信度和检测框的坐标。
- `static int nms_comparator(const void *pa, const void *pb)`, 比较两个不同检测框的可信度大小, 其中 `pa`、`pb` 分别是两个检测框的对应指针。如果 `pa` 的可信度小于 `pb` 的可信度, 则函数返回 1; 如果 `pa` 的可信度大于 `pb` 的可信度, 则函数返回 -1; 如果 `pa` 的可信度等于 `pb` 的可信度, 则函数返回 0。
- `static float overlap(float x1, float w1, float x2, float w2)`, 用于计算检测框在一个维度上的重叠长度。
- `static void do_nms_sort(region_layer_t *rl, box_t *boxes, float **probs)`, YOLOv2 的非极大值抑制(NMS)算法的具体实现, 其中 `rl` 是 `region_layer_t` 结构体指针, 包含 NMS 算法的计算参数; `boxes` 和 `probs` 分别是检测框的坐标值和可信度大小。NMS 的计算结果将直接保存在 `boxes` 和 `probs` 指针对应的内存上。
- `static void region_layer_output(region_layer_t *rl, obj_info_t *obj_info)`, 获取 YOLOv2 `region` 层的运算结果, 其中 `rl` 是 `region_layer_t` 结构体指针, 用于获取相关参数; `obj_info` 是 `obj_info_t` 结构体指针, 用于存储输出结果。

- void region_layer_run(region_layer_t *rl, obj_info_t *obj_info), 调用 forward_region_layer、correct_region_boxes、do_nms_sort、region_layer_output 函数，完成 region 层的运算，其中 rl 是 region_layer_t 结构体指针，用于获取相关参数；obj_info 是 obj_info_t 结构体指针，用于存储输出结果。
- void draw_edge(uint32_t *gram, obj_info_t *obj_info, uint32_t index, uint16_t color, uint16_t image_width, uint16_t image_height), 实现在 LCD 缓存绘制检测框，其中 gram 是 LCD 缓存对应的指针；obj_info 和 index 分别是检测框的结构体数组指针和索引；color 是检测框的颜色；image_width 和 image_height 分别是图像总共的宽和高。

(2) k210_yolov2_detect.c 文件

k210_yolov2_detect.c 文件实现了从 json 文件解析文件，调取摄像头获取图像，利用 KPU 进行 kmodel 模型文件的推断过程，并最终显示在 LCD 屏幕上。具体实现见代码 6-3-6。

代码 6-3-6 k210_detect 函数过程

```

01 void k210_detect(char *json_file_path){
02     int ret = 0;
03     int result = 0;
04     int size = 0;
05     char kmodel_path[127] = {};
06
07     yolov2_params_t detect_params = param_parse(json_file_path);
08     if (!detect_params.is_valid) return;
09
10     #ifdef ADD_XIZI_FEATURES
11         kpu_fd = PrivOpen(KPU_DEV_DRIVER, O_RDONLY);
12         if (camera_fd < 0) return;
13     #endif
14
15     camera_fd = PrivOpen(CAMERA_DEV_DRIVER, O_RDONLY);
16     if (camera_fd < 0) return;
17
18     _ioctl_set_reso set_dvp_reso = {detect_params.sensor_output_size[1],
                                     detect_params.sensor_output_size[0]};
19     struct PrivIoctlCfg camera_cfg;
20     camera_cfg.args = &set_dvp_reso;
21     camera_cfg.ioctl_driver_type = CAMERA_TYPE;
22     PrivIoctl(camera_fd, IOCTL_CAMERA_OUT_SIZE_RESO, &camera_cfg);

```

```

23  image_height = set_dvp_reso.height;
24  image_width = set_dvp_reso.width;
25
26  model_data = (unsigned char *)malloc(detect_params.kmodel_size + 255);
27  if (NULL == model_data){
28      free(showbuffer);
29      free(kpurgbbuffer);
30      PrivClose(camera_fd);
31      return;
32  }
33  showbuffer = (unsigned char *)malloc(detect_params.sensor_output_size[0]
                                     * detect_params.sensor_output_size[1] * 2);
34  if (NULL == showbuffer){
35      PrivClose(camera_fd);
36      return;
37  }
38  kpurgbbuffer = (unsigned char *)malloc(detect_params.net_input_size[0]
                                     * detect_params.net_input_size[1] * 3);
39  if (NULL == kpurgbbuffer){
40      PrivClose(camera_fd);
41      free(showbuffer);
42      return;
43  }
44  memset(model_data, 0, detect_params.kmodel_size + 255);
45  memset(showbuffer, 0, detect_params.sensor_output_size[0]
                                     * detect_params.sensor_output_size[1] * 2);
46  memset(kpurgbbuffer, 0, detect_params.net_input_size[0]
                                     * detect_params.net_input_size[1] * 3);
47  shoot_para_t.pdata = (uintptr_t)(showbuffer);
48  shoot_para_t.length = (size_t)(detect_params.sensor_output_size[0]
                                     * detect_params.sensor_output_size[1] * 2);
49
50  memcpy(kmodel_path, json_file_path, strlen(json_file_path));
51  int idx_suffix_start = strlen(json_file_path) - 4;
52  const char kmodel_suffix[7] = "kmodel";
53  int kmodel_suffix_len = 6;
54  while (kmodel_suffix_len--){
55      kmodel_path[idx_suffix_start + 5 - kmodel_suffix_len] =
                                     kmodel_suffix[5 - kmodel_suffix_len];
56  }
57  unsigned char *model_data_align =

```

```

                    (unsigned char *)(((uintptr_t)model_data + 255) & (~255));
58
59     kmodel_fd = PrivOpen(kmodel_path, O_RDONLY);
60     if (kmodel_fd < 0){
61         PrivClose(camera_fd);
62         free(showbuffer);
63         free(kpurgbbuffer);
64         free(model_data);
65         return;
66     }
67     else{
68         size = PrivRead(kmodel_fd, model_data_align, detect_params.kmodel_size);
69         if (size != detect_params.kmodel_size){
70             PrivClose(camera_fd);
71             PrivClose(kmodel_fd);
72             free(showbuffer);
73             free(kpurgbbuffer);
74             free(model_data);
75             return;
76         }
77         else{
78             PrivClose(kmodel_fd);
79         }
80     }
81
82 #ifdef ADD_RTTHREAD_FETURES
83     dvp_set_ai_addr((uintptr_t)(kpurgbbuffer + detect_params.net_input_size[1]
84                               * (detect_params.net_input_size[0] - detect_params.sensor_output_size[0])),
85                     (uintptr_t)(kpurgbbuffer + detect_params.net_input_size[1]
86                               * (detect_params.net_input_size[0] - detect_params.sensor_output_size[0])
87                               + detect_params.net_input_size[0] * detect_params.net_input_size[1]),
88                     (uintptr_t)(kpurgbbuffer + detect_params.net_input_size[1]
89                               * (detect_params.net_input_size[0] - detect_params.sensor_output_size[0])
90                               + detect_params.net_input_size[0] * detect_params.net_input_size[1] * 2));
91 #else
92     RgbAddress ai_address_preset;
93     ai_address_preset.r_addr = (uintptr_t)kpurgbbuffer
94                               + detect_params.net_input_size[1];
95     ai_address_preset.g_addr = ai_address_preset.r_addr
96                               + detect_params.net_input_size[0] * detect_params.net_input_size[1];
97     ai_address_preset.b_addr = ai_address_preset.g_addr

```

```

        + detect_params.net_input_size[0] * detect_params.net_input_size[1];
91     camera_cfg.args = &ai_address_preset;
92     PrivIoctl(camera_fd, SET_AI_ADDR, &camera_cfg);
93 #endif
94 #ifdef ADD_RTTHREAD_FETURES
95     if (kpu_load_kmodel(&detect_task, model_data_align) != 0){
96         PrivClose(camera_fd);
97         PrivClose(kmodel_fd);
98         free(showbuffer);
99         free(kpurgbbuffer);
100        free(model_data);
101        return;
102    }
103 #else
104     struct PrivIoctlCfg kpu_cfg;
105     kpu_cfg.args = model_data_align;
106     kpu_cfg.ioctl_driver_type = KPU_TYPE;
107     if (PrivIoctl(kpu_fd, LOAD_KMODEL, &kpu_cfg) != 0){
108         PrivClose(camera_fd);
109         PrivClose(kmodel_fd);
110         free(showbuffer);
111         free(kpurgbbuffer);
112         free(model_data);
113         return;
114     }
115 #endif
116
117     detect_rl.anchor_number = ANCHOR_NUM;
118     detect_rl.anchor = detect_params.anchor;
119     detect_rl.nms_value = detect_params.nms_thresh;
120     detect_rl.classes = detect_params.class_num;
121     result = region_layer_init(&detect_rl, detect_params.net_output_shape[0],
        detect_params.net_output_shape[1], detect_params.net_output_shape[2],
        detect_params.net_input_size[1], detect_params.net_input_size[0]);
122     for (int idx = 0; idx < detect_params.class_num; idx++){
123         detect_rl.threshold[idx] = detect_params.obj_thresh[idx];
124     }
125
126     size_t stack_size = STACK_SIZE;
127     pthread_attr_t attr;
128     struct sched_param prio;

```

```

129     prio.sched_priority = THREAD_PRIORITY_D;
130     pthread_attr_init(&attr);
131     pthread_attr_setschedparam(&attr, &prio);
132     pthread_attr_setstacksize(&attr, stack_size);
133
134     result = pthread_create(&tid, &attr, thread_detect_entry, &detect_params);
135     if (0 == result){
136         printf("thread_detect_entry successfully!\n");
137     }
138     else{
139         PrivClose(camera_fd);
140     }
141 }

```

调用方法在 Application 文件夹下的 main.c 里编写代码如下：

代码 6-3-7 main 函数调用过程

```

01 #include <stdio.h>
02 #include <string.h>
03 #include <transform.h>
04
05 extern int FrameworkInit();
06 extern void ApplicationOtaTaskInit(void);
07 extern void k210_detect(char *json_file_path);
08
09 int main(void){
10     printf("Hello, world! \n");
11     FrameworkInit();
12 #ifdef APPLICATION_OTA
13     ApplicationOtaTaskInit();
14 #endif
15     k210_detect("screw_v0.json");
16     return 0;
17 }

```

最终实现效果如图 6-3-13 所示。图片红框代表所识别到的物体位置。

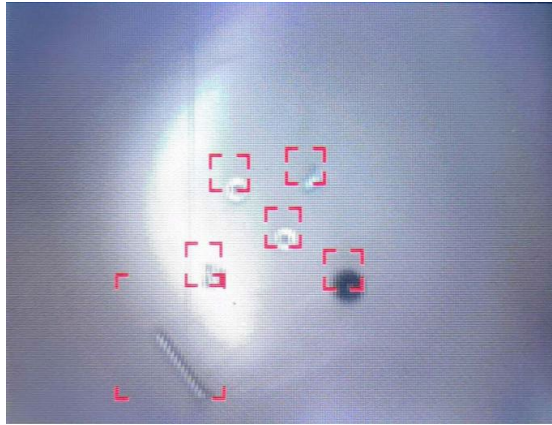


图 6-3-13 物体检测应用示例结果

第4章 “精准控制” 实践

4.1 “精准控制” 框架介绍

工业物联应用场景下“精准控制”应用框架拟解决的关键问题在于：全面互联互通难，尤其是各型生产设备、工业控制系统如 PLC 难以打通，难以从其获取生产数据，也难以自动对其进行调整控制从而优化生产。

“精准控制”应用框架遵循软件定义工业控制的设计思路，从工业控制的动作分解和合理抽象、工业控制软件模型的构建、工业控制功能块的划分等角度展开研究，支持和常见 PLC、工控机设备的互联互通，建立信息控制和交互的通道，实现自顶向下的执行流。设计中以 ControlProtocol 为基本单位，对上屏蔽 PLC 协议具体实现，提供“控制元”，即数据、指令交互 API 接口，对下对接操作系统通信接口，满足 PLC 设备通信需求。同时，为简化现场调试和开发流程，设计 PLC 配方 ControlRecipe，利用可插拔介质，如 SD 卡、USB 等，中的 json 文件，软件定义 PLC 协议参数、通信配置、交互变量信息等可能变化的重要信息，通过解析加载 json 配方文件，动态构建对应协议交互报文，配置依赖通信驱动，从而实现随时随地接入 PLC 设备、简化 PLC 复杂需求开发流程的目的。如图 6-4-1 所示。

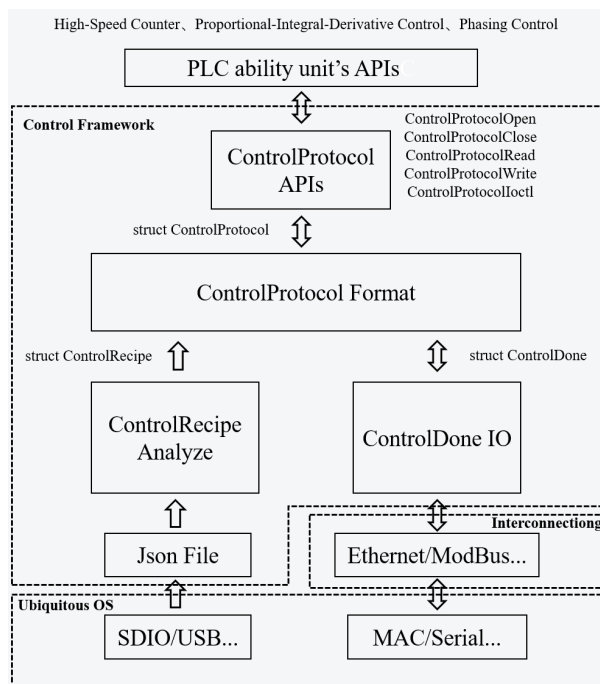


图 6-4-1 “精准控制”框架结构示意图

下面介绍 XiUOS 的“精准控制”框架关键数据结构定义和解析，具体详见 control.h、control_def.h、control_io.h 文件。

4.1.1 Control.h

Control.h 中声明了控制协议的结构体 ControlProtocol，协议（设备）所处阶段的结构体 ProtocolStatus，支持协议的类型的结构体 ProtocolType 和相关的一些函数。下面详见代码 6-4-1、代码 6-4-2、代码 6-4-3。

代码 6-4-1 ControlProtocol 结构体

```

struct ControlProtocol
{
    char *name;
    ProtocolType protocol_type;
    ProtocolStatus protocol_status;
    struct ControlRecipe *recipe;
    struct ControlDone *done;
    void *args;
    sem_t sem;
    pthread_mutex_t lock;
};

```

其中，name 是协议的名字，protocol_type 是协议的类型，所有的类型会在结构体 ProtocolType 中声明。

代码 6-4-2 ProtocolStatus 结构体

```
typedef enum
{
    CONTROL_INIT,
    CONTROL_REGISTERED,
    CONTROL_UNREGISTERED,
}ProtocolStatus;
```

其中，CONTROL_INIT 代表初始化，CONTROL_REGISTERED 代表协议已经注册，CONTROL_UNREGISTERED 代表协议尚未注册。

代码 6-4-3 ProtocolType 结构体

```
typedef enum
{
    PROTOCOL_S7 = 1,
    PROTOCOL_MODBUS_TCP,
    PROTOCOL_MODBUS_UART,
    PROTOCOL_OPC_UA,
    PROTOCOL_FINS,
    PROTOCOL_MELSEC_1E,
    PROTOCOL_MELSEC_3E_Q_L,
    PROTOCOL_MELSEC_3E_IQ_R,
    PROTOCOL_MELSEC_1C,
    PROTOCOL_MELSEC_3C,
    PROTOCOL_END
}ProtocolType;
```

另外，在 control.h 文件中还定义了一些外部调用的函数接口：

- ControlProtocolType ControlProtocolFind(void)，用于控制框架来寻找特定的协议。
- int ControlProtocolOpen(struct ControlProtocol *control_protocol)用于控制框架协议打开，传入的参数是控制协议机构体类型。打开成功返回 0，失败则返回-1。
- int ControlProtocolClose(struct ControlProtocol *control_protocol)为控制协议关闭。
- int ControlProtocolRead(struct ControlProtocol *control_protocol, void *buf, size_t len)为控制协议读操作。传入的参数中，void *buf 是数据缓冲区，size_t len 为读取的数据的长度。读取成功则返回读取数据的长度，失败则返回-1。

- `int ControlProtocolWrite(struct ControlProtocol *control_protocol, const void *buf, size_t len)` 为控制协议写操作。传入的参数中, `void *buf` 是数据缓冲区, `size_t len` 为写入的数据的长度。写入成功则返回写入数据的长度, 失败则返回-1。
- `int ControlProtocolIoctl(struct ControlProtocol *control_protocol, int cmd, void *args)` 为控制框架协议控制函数, 其中 `int cmd` 为传入的命令, `void *args` 为传入的参数。成功则返回 0, 失败返回-1。
- `int ControlFrameworkInit(void)` 为控制框架初始化。初始化成功返回 0, 失败则返回-1。

4.1.2 Control_def.h

`Control_def.h` 中定义了控制需要的基本信息, 下面详见代码 6-4-4、代码 6-4-5、代码 6-4-6、代码 6-4-7、代码 6-4-8、代码 6-4-9。

`BasicSocketPlc` 结构体中定义了 `ip` 地址, 端口号和套接字, 同时还定义了是否有第二个连接的标志。

代码 6-4-4 BasicSocketPlc 结构体

```
typedef struct
{
    uint8_t ip[4];
    uint16_t port;
    int32_t socket;
    int8_t secondary_connect_flag;//0: enable, no connected; 1: enable, connected; -1: disable
}BasicSocketPlc;
```

`BasicPlcDataInfo` 结构体中定义了命令的长度, 数据的大小, 和存放数据和命令的指针。

代码 6-4-5 BasicPlcDataInfo 结构体

```
typedef struct
{
    uint16_t command_length;
    uint16_t data_size;
    uint8_t command_ready;
    uint8_t *p_command;
    uint8_t *p_data;
}BasicPlcDataInfo;
```

`ProtocolData` 结构体中定义了数据的长度与存放数据的指针。

代码 6-4-6 ProtocolData 结构体

```

struct ProtocolData
{
    uint8_t *data;
    uint16_t data_length;
};

```

SerialConfig 结构体中定义了波特率，比特率，停止位和校验模式。

代码 6-4-7 SerialConfig 结构体

```

struct SerialConfig
{
    uint8_t station;
    uint32_t baud_rate;
    uint8_t data_bits;
    uint8_t stop_bits;
    uint8_t check_mode;
};

```

ProtocolFormatInfo 结构体中定义了协议数据格式相关的信息，这个结构体在协议初始化时需要将其初始化为对应协议的数据格式。

代码 6-4-8 ProtocolFormatInfo 结构体

```

typedef struct
{
    cJSON *read_single_item_json;
    uint8_t *p_read_item_data;
    uint16_t read_item_index;//Variable item index(1++)
    uint8_t last_item_size;
}ProtocolFormatInfo;

```

ControlRecipe 结构体是在初始化协议时需要初始化的一个很重要的结构体。

代码 6-4-9 ControlRecipe 结构体

```

struct ControlRecipe
{
    uint8_t device_name[20];
    uint16_t device_id;
    uint16_t read_period;
    uint16_t read_item_count;
    uint16_t total_data_length;
    uint8_t communication_type;
    ProtocolType protocol_type;
    void *read_item;
};

```

```

    struct ControlDone *done;
    struct SerialConfig serial_config;
    struct SocketConfig socket_config;
    struct ProtocolData protocol_data;
    int (*ControlProtocolFormatCmd)(struct ControlRecipe *p_recipe, ProtocolFormatInfo
    *protocol_format_info);
};

```

其中，device_name[20]定义了设备的名称，device_id 定义了设备的 id。ControlDone *done 对应了协议的打开，关闭，读，写，IO 控制的函数，对这些函数的调用以回调的形式进行。在协议初始化时，需要将对应的操作初始化为对应协议的操作，*read_item 定义了读的缓冲区，大小是 read_item_count 乘以对应协议的数据单元大小，以 fins 协议为例，需要将大小初始化为：sizeof(FinsReadItem*read_item_count)。

另外，在 control_def.h 文件中还定义了一些外部调用的函数接口：

- uint8_t GetValueTypeMemorySize(UniformValueType uniform_value_type)，获取对应的内存大小，传入的参数是数据类型。
- int RecipeBasicInformation(struct ControlRecipe *p_recipe, cJSON *p_recipe_file_json)，从接收到的文件中获取基本的信息，传入的参数位接收的 cJSON 的数据和协议接收数据结构体。
- int ControlPeripheralInit(struct ControlRecipe *p_recipe)，控制协议外设初始化，传入的参数位接收到的信息。
- int ControlConnectSocket(BasicSocketPlc *p_plc)，控制协议接入连接，传入的参数为 plc 的套接字结构体。
- int ControlDisconnectSocket(BasicSocketPlc *p_plc)，控制协议断开链接，传入的参数为 plc 的套接字结构体。
- int ControlProtocolOpenDef(struct ControlProtocol *control_protocol)，控制协议打开子协议，初始化接收任务和内存。传入的参数是控制协议结构体的指针。
- int ControlProtocolCloseDef(void)，控制协议关闭子协议，释放接收任务和内存。

4.1.3 Control_io.h

控制框架目前支持 socket 和 serial 两种通信方式，头文件 control_io.h 中定义了 socket 初始化和 serial 初始化、读写等接口，供框架内部使用。具体如下：

- void SocketInit(char *ip, char *mask, char *gw), 对网络端口进行初始化。其中 char *ip 为本地 ip, char *mask 为子网掩码, char *gw 为网关。
- void SerialInit(uint32_t baud_rate, uint8_t data_bits, uint8_t stop_bits, uint8_t check_mode), 对串口进行初始化。其中 uint32_t baud_rate 为波特率, uint8_t data_bits 为比特数, uint8_t stop_bits 为停止比特, uint8_t check_mode 为校验模式, 校验模式分为奇校验, 偶校验, 和不进行校验 (NONE)。
- void SerialWrite(uint8_t *write_data, int length), 串口写。uint8_t *write_data 为写入缓冲区, int length 为写入长度。
- int SerialRead(uint8_t *read_data, int length), 串口读。uint8_t *read_data 为读入缓冲区, int length 为读取数据的长度。若读取成功则返回读取长度, 失败则返回-1。

4.2 PLC 协议介绍

PLC 全称是: Programmable Logic Controller, 即可编程逻辑控制器, 是一种具有微处理器的用于自动化控制的数字运算控制器, 可以将控制指令随时载入内存进行储存与执行。下面对一些常见协议进行描述, 具体详见 fins.c、melsec.c、modbus_tcp.c、modbus_uart.c 文件。

4.2.1 Fins 协议

要使用 fins 协议, 首先要进行初始化, 主要的目的是将结构体 struct ControlRecipe *p_recipe 进行初始化。

代码行 3 给对应缓冲区分配内存。若分配内存失败则返回-1, 说明初始化失败。若分配内存成功则首先对分配的内存进行初始化。代码行 11 将命令格式设置为 fins 协议对应的命令格式, 代码行 13 将对应的操作设置为 fins 协议对应的操作。

代码 6-4-10 Fins 协议初始化函数

```

01 int FinsProtocolInit(struct ControlRecipe *p_recipe)
02 {
03     p_recipe->read_item = PrivMalloc(sizeof(FinsReadItem) *。
p_recipe->read_item_count);
04     if (NULL == p_recipe->read_item) {
05         PrivFree(p_recipe->read_item);
06         return -1;
07     }

```

```

08
09     memset(p_recipe->read_item, 0, sizeof(FinsReadItem));
10
11     p_recipe->ControlProtocolFormatCmd = FinsProtocolFormatCmd;
12
13     p_recipe->done = &fins_protocol_done;
14
15     return 0;
16 }

```

Fins 协议对应的操作有打开，关闭和读，具体定义在代码 6-4-11 中：

代码 6-4-11 fins 协议对应操作

```

01 static struct ControlDone fins_protocol_done =
02 {
03     ._open = FinsOpen,
04     ._close = FinsClose,
05     ._read = FinsRead,
06     ._write = NULL,
07     ._ioctl = NULL,
08 };

```

4.2.2 melsec 协议

和 fins 协议一样，首先要进行初始化，主要的目的是将结构体 struct ControlRecipe *p_recipe 进行初始化。

代码行 3 给对应缓冲区分配内存。若分配内存失败则返回-1，说明初始化失败。若分配内存成功则首先对分配的内存进行初始化。代码行 11 将命令格式设置为 melsec 协议对应的命令格式，代码行 13 将对应的操作设置为 melsec 协议对应的操作。

代码 6-4-12 melsec 协议初始化函数

```

01 int MelsecProtocolInit(struct ControlRecipe *p_recipe)
02 {
03     p_recipe->read_item = PrivMalloc(sizeof(MelsecReadItem) *
p_recipe->read_item_count);
04     if (NULL == p_recipe->read_item) {
05         PrivFree(p_recipe->read_item);
06         return -1;
07     }
08
09     memset(p_recipe->read_item, 0, sizeof(MelsecReadItem));

```

```

10
11     p_recipe->ControlProtocolFormatCmd = MelsecProtocolFormatCmd;
12
13     p_recipe->done = &melsec_protocol_done;
14
15     return 0;
16 }

```

melsec 协议对应的操作有打开，关闭和读，具体定义在代码 6-4-13 中：

代码 6-4-13 melsec 协议对应的操作有打开，关闭和读操作

```

01 static struct ControlDone melsec_protocol_done =
02 {
03     ._open = MelsecOpen,
04     ._close = MelsecClose,
05     ._read = MelsecRead,
06     ._write = NULL,
07     ._ioctl = NULL,
08 };

```

4.2.3 modbus_tcp 协议

想要使用 modbus_tcp 协议，首先要进行初始化，主要的目的是将结构体 struct Control Recipe *p_recipe 进行初始化。

代码行 3 给对应缓冲区分配内存。若分配内存失败则返回-1，说明初始化失败。若分配内存成功则首先对分配的内存进行初始化。代码行 11 将命令格式设置为 modbus_tcp 协议对应的命令格式，代码行 13 将对应的操作设置为 melsec 协议对应的操作。

代码 6-4-14 modbus_tcp 协议初始化函数

```

01 int ModbusTcpProtocolInit(struct ControlRecipe *p_recipe)
02 {
03     p_recipe->read_item = PrivMalloc(sizeof(ModbusTcpReadItem) *
p_recipe->read_item_count);
04     if (NULL == p_recipe->read_item) {
05         PrivFree(p_recipe->read_item);
06         return -1;
07     }
08
09     memset(p_recipe->read_item, 0, sizeof(ModbusTcpReadItem));
10
11     p_recipe->ControlProtocolFormatCmd = ModbusTcpProtocolFormatCmd;

```

```

12
13     p_recipe->done = &modbustcp_protocol_done;
14
15     return 0;
16 }

```

Modbus_tcp 协议对应的操作有打开、关闭、读、写、IO 控制。具体定义在代码 6-4-15 中：

代码 6-4-15 modbus_tcp 协议对应操作

```

01 static struct ControlDone modbustcp_protocol_done =
02 {
03     ._open = ModbusTcpOpen,
04     ._close = ModbusTcpClose,
05     ._read = ModbusTcpRead,
06     ._write = ModbusTcpWrite,
07     ._ioctl = ModbusTcpIoctl,
08 };

```

4.2.4 modbus_uart 协议

想要使用 modbus_uart 协议，首先要进行初始化，主要的目的是将结构体 struct ControlRecipe *p_recipe 进行初始化。

代码行 3 给对应缓冲区分配内存。若分配内存失败则返回-1，说明初始化失败。若分配内存成功则首先对分配的内存进行初始化。代码行 11 将命令格式设置为 modbus_uart 协议对应的命令格式，代码行 13 将对应的操作设置为 melsec 协议对应的操作。

代码 6-4-16 modbus_uart 协议初始化函数

```

01 int ModbusUartProtocolInit(struct ControlRecipe *p_recipe)
02 {
03     p_recipe->read_item = PrivMalloc(sizeof(ModbusUartReadItem) *
p_recipe->read_item_count);
04     if (NULL == p_recipe->read_item) {
05         PrivFree(p_recipe->read_item);
06         return -1;
07     }
08
09     memset(p_recipe->read_item, 0, sizeof(ModbusUartReadItem));
10
11     p_recipe->ControlProtocolFormatCmd = ModbusUartProtocolFormatCmd;
12

```

```

13     p_recipe->done = &modbusuart_protocol_done;
14
15     return 0;
16 }

```

Modbus_uart 协议对应的操作有打开、关闭、读、写、IO 控制。具体定义在代码 6-4-17 中：

代码 6-4-17 modbus_uart 协议对应操作

```

01 static struct ControlDone modbusuart_protocol_done =
02 {
03     ._open = ModbusUartOpen,
04     ._close = ModbusUartClose,
05     ._read = ModbusUartRead,
06     ._write = ModbusUartWrite,
07     ._ioctl = ModbusUartIoctl,
08 };

```

4.3 PLC 应用

4.3.1 ARM 架构下 PLC 应用

(1) 小型 PLC FX3U-32MT/ES-A 应用

在本书中选取的小型 PLC 使用样例为：CPU FX3U-32MT/ES-A+以太网模块 FX3U-EN ET-L。对应 test_recipe_mc_1e.json 配方文件，使用的协议为 melsec_1e 协议。其中配方文件字段说明如代码 6-4-18 所示：

代码 6-4-18 配方文件字段说明

```

01 {
02     "device_id": 771, //设备的 ID
03     "device_name": "S03", //设备的名称
04     "communication_type": 0, //0 表示以太网，1 表示串口
05     "socket_config": { //网络配置
06         "plc_ip": "192.168.250.20", //PLC 的 IP
07         "local_ip": "192.168.250.233", //终端的 IP
08         "gateway": "192.168.250.1", //网关
09         "netmask": "255.255.254.0", //子网掩码
10         "port": 2000 //端口号
11     },
12     "protocol_type": 6, //对应的协议，见程序定义
13     "read_period": 100, //读取周期，单位为 ms
14     "read_item_list": [ //读取列表

```

```

15      {
16          "value_name": "启动", //变量名称
17          "value_type": 1,      //变量类型，见程序定义
18          "device_code": "M",  //变量存储区
19          "head_device_number_string": "0", //变量存储区的起始地址
20          "device_points_count": 1, //读取的数据长度
21          "command_type": 0, //命令格式：变量的读取方式，0 表示按位来读，1 表
示按字来读
22          "monitoring_timer": 100 //监控时间
23      },
24      {
25          "value_name": "停止", //变量名称
26          "value_type": 1, //变量类型，见程序定义
27          "device_code": "M", //变量存储区
28          "head_device_number_string": "1", //变量存储区的起始地址

29          "device_points_count": 1, //读取的数据长度

30          "command_type": 0, //命令格式：变量的读取方式，0 表示按位来读，1 表
示按字来读
31          "monitoring_timer": 100 //监控时间

32      }
33  ]
34 }

```

使用配置流程可参考如下步骤：

Step1: 将上述 json 文件放进 sd 卡；

Step2: 裁剪配置，menuconfig 相关配置如下：

1) plc 协议选择，配置配方名字为 test_recipe.json，与 sd 卡中 json 文件名字相同：

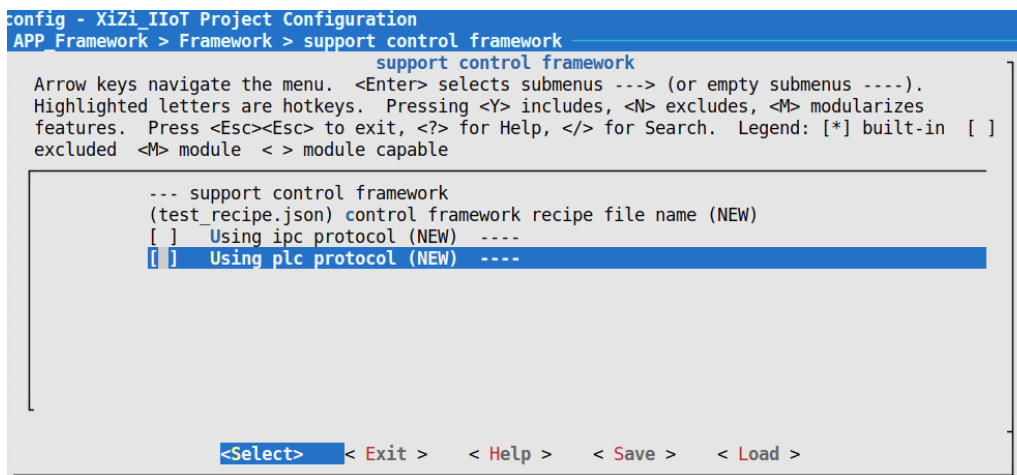


图 6-4-2 plc 协议选择

2) 使能 melsec_control_protocol, 选择基于网络 socket 通信的 melsec_le protocol:

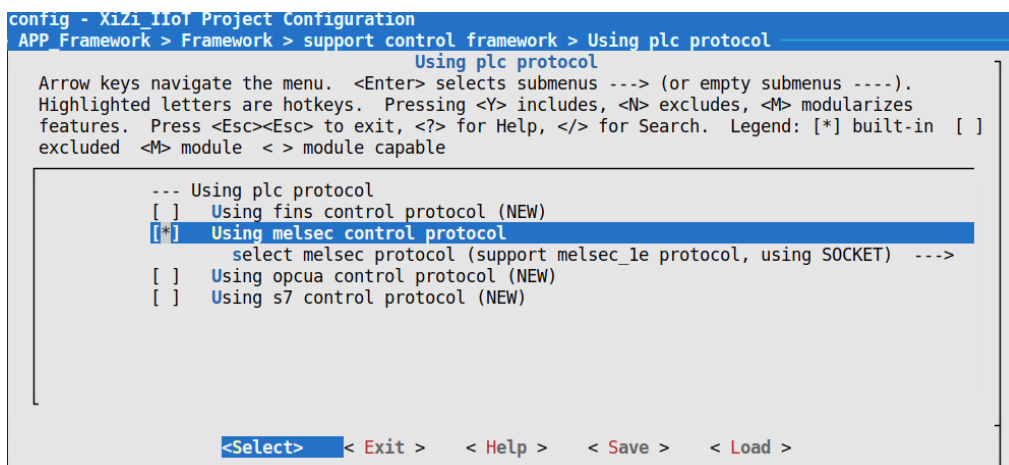


图 6-4-3 使能 melsec_control_protocol

3) 配置 main 线程栈大小:

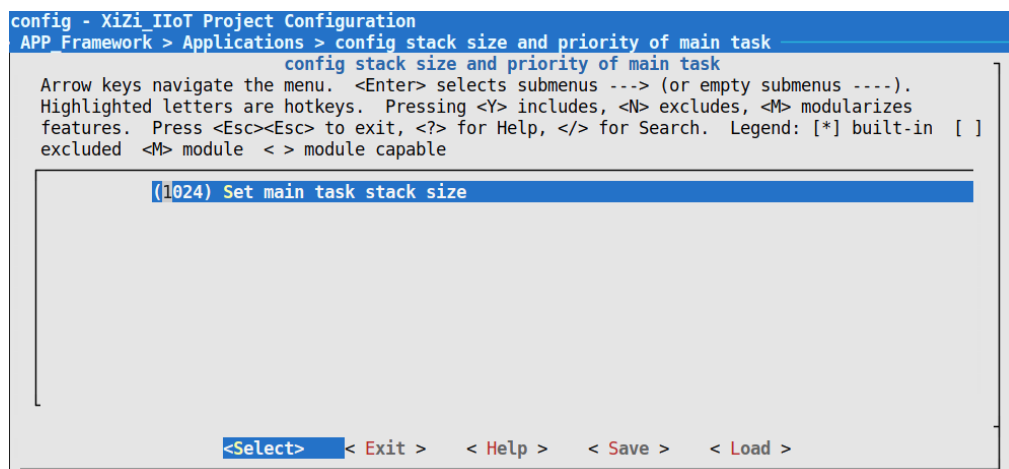


图 6-4-4 配置 main 线程栈

4) 使能 control app, 编译 plc 应用程序:

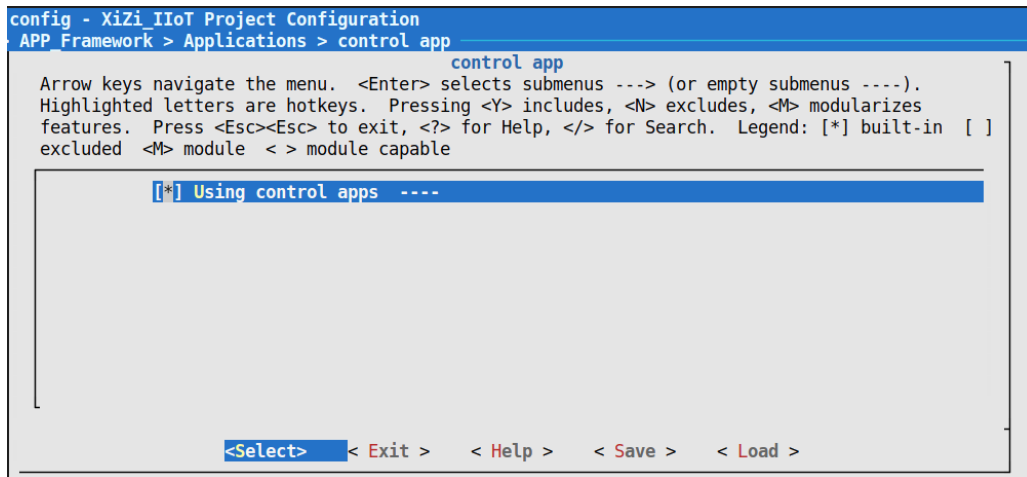


图 6-4-5202 使能 control app

5) 然后根据上述配置，编译成 bin 文件，烧录进砂达通进行配置即可。

Step3: 连接 PLC 进行测试。在 json 文件解析完成且 socket 初始化完成之后，输入 ControlFx3uTest 命令，即可进行数据读取。ControlFx3uTest 测试函数详见代码 6-4-19。

代码行 03-05 定义了存放数据的缓冲区和数据长度，代码行 11 将协议初始化为 melsec_1e_protocol，代码行 20 进行协议打开，代码行 22-33 进行数据的读取。

代码 6-4-19 ARM 架构下的 Melsec-1E PLC 应用测试

```

01 void ControlFx3uTest(void)
02 {
03     int i, j = 0;
04     int read_data_length = 0;
05     uint8_t read_data[128] = {0};
06
07 #ifdef CONNECTION_ADAPTER_4G
08     Adapter4GActive();
09 #endif
10
11     ControlProtocolType melsec_1e_protocol = ControlProtocolFind();
12     if(NULL == melsec_1e_protocol) {
13         printf("%s get melsec 1e protocol %p failed\n", __func__, melsec_1e_protocol);
14         return;
15     }
16
17     printf("%s get melsec 1e protocol %p successfull\n", __func__, melsec_1e_protocol);
18
19     if(CONTROL_REGISTERED == melsec_1e_protocol->protocol_status) {
20         ControlProtocolOpen(melsec_1e_protocol);

```

```

21
22     for (;;) {
23         read_data_length = ControlProtocolRead(melsec_1e_protocol, read_data,
sizeof(read_data));
24         printf("%s read [%d] melsec 1e data %d using receipe file\n", __func__, i,
read_data_length);
25         if (read_data_length) {
26             for (j = 0; j < read_data_length; j++) {
27                 printf("j %d data 0x%x\n", j, read_data[j]);
28             }
29         }
30         i++;
31         memset(read_data, 0, sizeof(read_data));
32         PrivTaskDelay(10000);
33     }
34
35     //ControlProtocolClose(modbus_tcp_protocol);
36 }
37 }

```

(2) 大型 PLC QO2UCPU 应用

在本书中选取的大型 PLC 使用样例为：CPU QO2UCPU+以太网模块 QJ71E71-100。对应 test_recipe_mc_3e.json 配方文件，使用的是 melsec_3e 协议。其中配方文件字段说明如代码 6-4-20 所示：

代码 6-4-20 配方文件字段说明

```

01 {
02     "device_id": 771, //设备的 ID
03     "device_name": "S03", //设备的名称
04     "communication_type": 0, //0 表示以太网，1 表示串口
05     "socket_config": { //网络配置
06         "plc_ip": "192.168.250.20", //PLC 的 IP
07         "local_ip": "192.168.250.233", //终端的 IP
08         "gateway": "192.168.250.1", //网关
09         "netmask": "255.255.254.0", //子网掩码
10         "port": 2000 //端口号
11     },
12     "protocol_type": 6, //对应的协议，见程序定义
13     "read_period": 100, //读取周期，单位为 ms
14     "read_item_list": [ //读取列表

```

```

15      {
16          "value_name": "启动", //变量名称
17          "value_type": 1,      //变量类型，见程序定义
18          "device_code": "M",  //变量存储区
19          "head_device_number_string": "0", //变量存储区的起始地址
20          "device_points_count": 1, //读取的数据长度
21          "command_type": 0, //命令格式：变量的读取方式，0 表示按位来读，1 表
示按字来读
22          "monitoring_timer": 100 //监控时间
23      },
24      {
25          "value_name": "停止", //变量名称
26          "value_type": 1, //变量类型，见程序定义
27          "device_code": "M", //变量存储区
28          "head_device_number_string": "1", //变量存储区的起始地址

29          "device_points_count": 1, //读取的数据长度

30          "command_type": 0, //命令格式：变量的读取方式，0 表示按位来读，1 表
示按字来读
31          "monitoring_timer": 100 //监控时间

32      }
33  ]
34  }

```

使用配置流程可参考如下步骤：

Step1: 将上述 json 文件放进 sd 卡；

Step2: 裁剪配置，menuconfig 相关配置如下：

1) plc 协议选择，配置配方名字为 test_recipe.json，与 sd 卡中 json 文件名字相同：

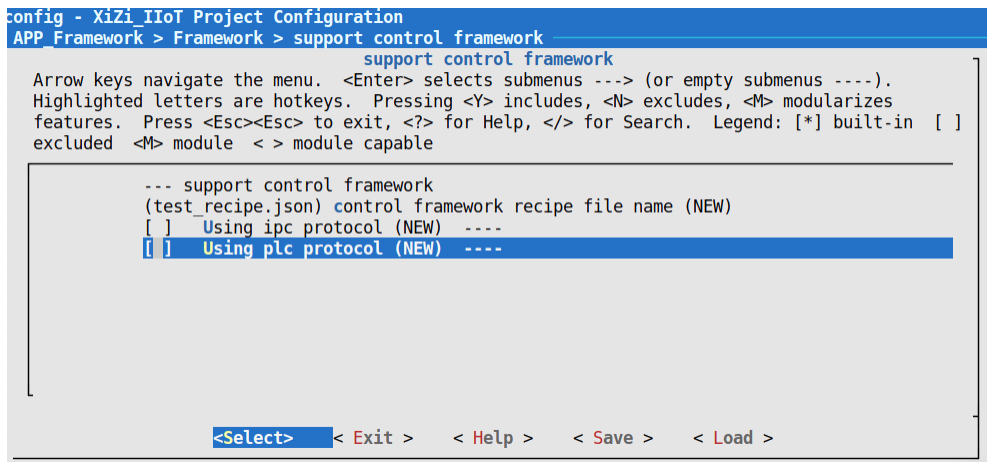


图 6-4-6 plc 协议选择

2) 使能 melsec_control_protocol, 选择基于网络 socket 通信的 melsec_3e_q_l protocol:

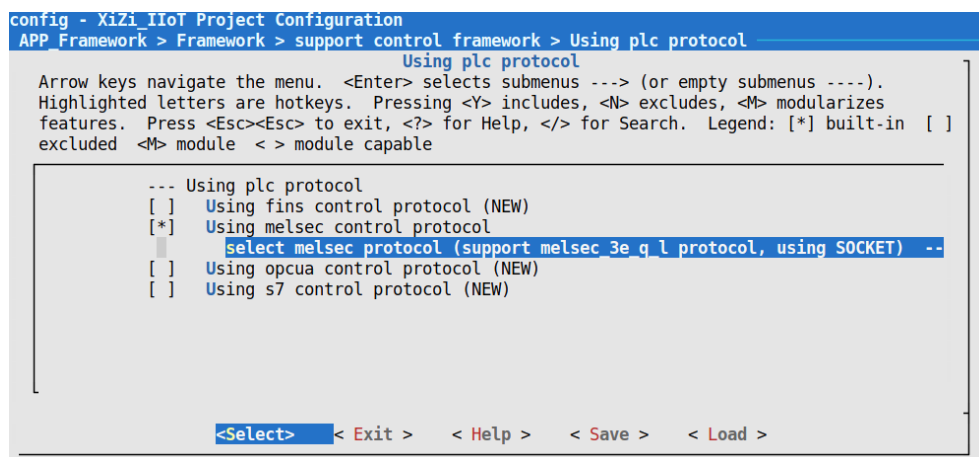


图 6-4-7 使能 melsec_control_protocol

3) 配置 main 线程栈大小:

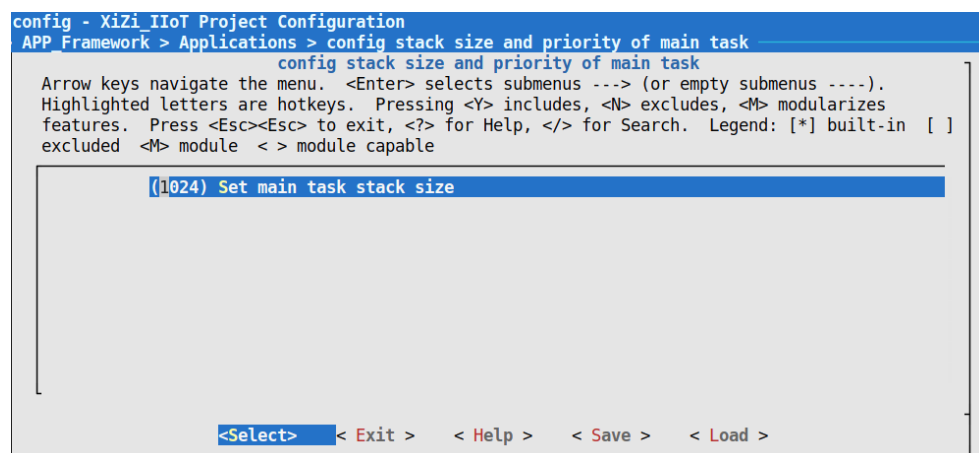


图 6-4-8 配置 main 线程栈大小

4) 使能 control app, 编译 plc 应用程序:

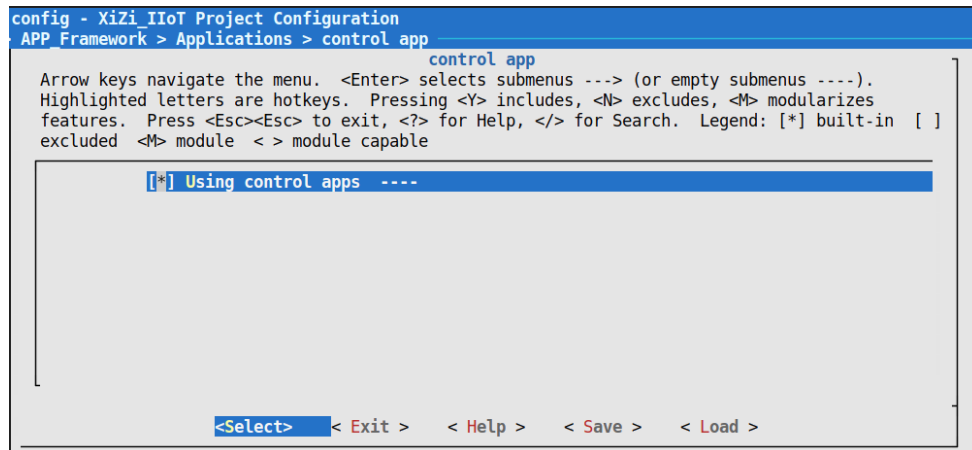


图 6-4-9 使能 control app

5) 然后根据上述配置，编译成 bin 文件，烧录进砂达通进行配置即可。

Step3: 连接 PLC 进行测试。在 json 文件解析完成且 socket 初始化完成之后，输入 ControlQ02ucpuTest 命令，即可进行数据读取。ControlQ02ucpuTest 测试函数详见代码 6-4-21。

代码行 03-05 定义了存放数据的缓冲区和数据长度，代码行 11 将协议初始化为 melsec_3e_protocol，代码行 20 进行协议打开，代码行 22-33 进行数据的读取。

代码 6-4-21 ARM 架构下的 Melsec-3E PLC 应用测试

```

01 void ControlQ02uTest(void)
02 {
03     int i, j = 0;
04     int read_data_length = 0;
05     uint8_t read_data[128] = {0};
06
07 #ifdef CONNECTION_ADAPTER_4G
08     Adapter4GActive();
09 #endif
10
11     ControlProtocolType melsec_3e_protocol = ControlProtocolFind();
12     if (NULL == melsec_3e_protocol) {
13         printf("%s get melsec 3e protocol %p failed\n", __func__, melsec_3e_protocol);
14         return;
15     }
16
17     printf("%s get melsec 3e protocol %p successfull\n", __func__, melsec_3e_protocol);
18
19     if (CONTROL_REGISTERED == melsec_3e_protocol->protocol_status) {
20         ControlProtocolOpen(melsec_3e_protocol);

```

```

21
22     for (;;) {
23         read_data_length = ControlProtocolRead(melsec_3c_protocol, read_data,
sizeof(read_data));
24         printf("%s read [%d] melsec 3c data %d using receipe file\n", __func__, i,
read_data_length);
25         if (read_data_length) {
26             for (j = 0; j < read_data_length; j++) {
27                 printf("j %d data 0x%x\n", j, read_data[j]);
28             }
29         }
30         i++;
31         memset(read_data, 0, sizeof(read_data));
32         PrivTaskDelay(10000);
33     }
34
35     //ControlProtocolClose(melsec_3c_protocol);
36 }
37 }

```

4.3.2 RISC-V 架构下 PLC 应用

针对 RISC-V 架构下的 PLC 应用，本书选用的 PLC 型号为 CPU 1215c（中型）和 CPU 1512SP-1 PN（大型）。这两款 PLC 使用的都是 S7 协议。其中 CPU 1215c 对应的配方文件为 test_recipe_s7_1215.json，CPU 1512SP-1 PN 对应的配方文件为 test_recipe_s7_1512.json。两种 PLC 的配置方式基本一致，在这里仅对 CPU 1512SP-1 PN 进行介绍。其中配方文件字段说明如代码 6-4-22 所示：

代码 6-4-22 配方文件字段说明

```

01 {
02     "device_id": 1500,           //设备 ID 号
03     "device_name": "demo",      //设备名称
04     "communication_type": 0,    //0 表示以太网，1 表示串口
05     "socket_config": {          //网口配置
06         "plc_ip": "192.168.250.2", //PLC 的 IP 地址
07         "local_ip": "192.168.250.233", //终端的 IP 地址
08         "gateway": "192.168.250.252", //终端的网关
09         "netmask": "255.255.255.0", //终端的子网掩码
10         "port": 102              //端口号
11     },                          //

```

```

12  "protocol_type": 1,           //选择协议，见程序定义
13  "read_period": 100,          //读取周期，单位为 ms
14  "read_item_list": [          //读取列表
15      {
16          "value_name": "扭矩", //变量名称
17          "value_type": 9,       //变量类型，见程序定义
18          "area": "DB",          //西门子寄存器区块
19          "wordlen": "Real",     //变量类型
20          "db_number": 18,       //区块号
21          "start": 18,           //地址起始位
22          "amount": 1            //读取寄存器数量，默认为 1
23      },
24      ...
25  ]
26  }

```

使用配置流程可参考如下步骤：

Step1: 将上述 json 文件放进 sd 卡：

Step2: 裁剪配置，menuconfig 相关配置如下：

1) plc 协议选择，配置配方名字为 test_recipe.json，与 sd 卡中 json 文件名字相同：

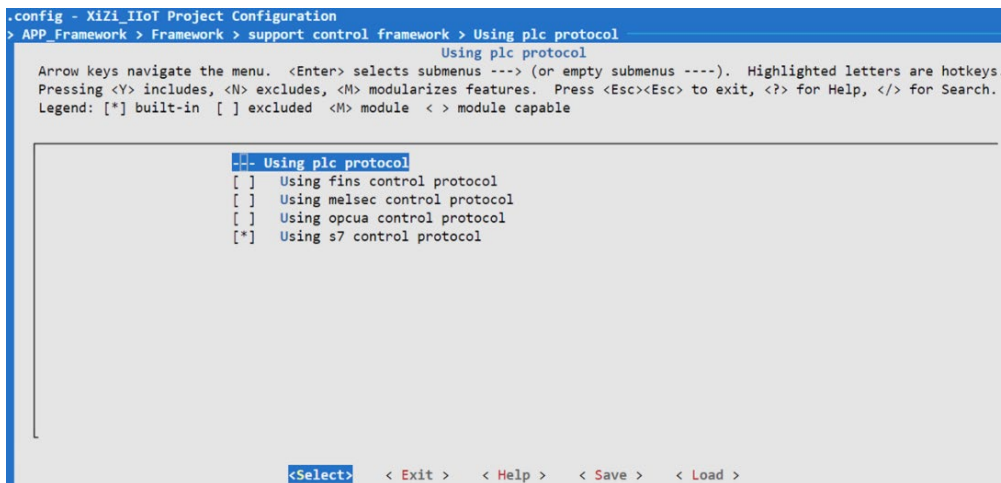


图 6-4-10 plc 协议选择

2) 使能 melsec_control_protocol，选择基于网络 socket 通信的 s7 protocol：

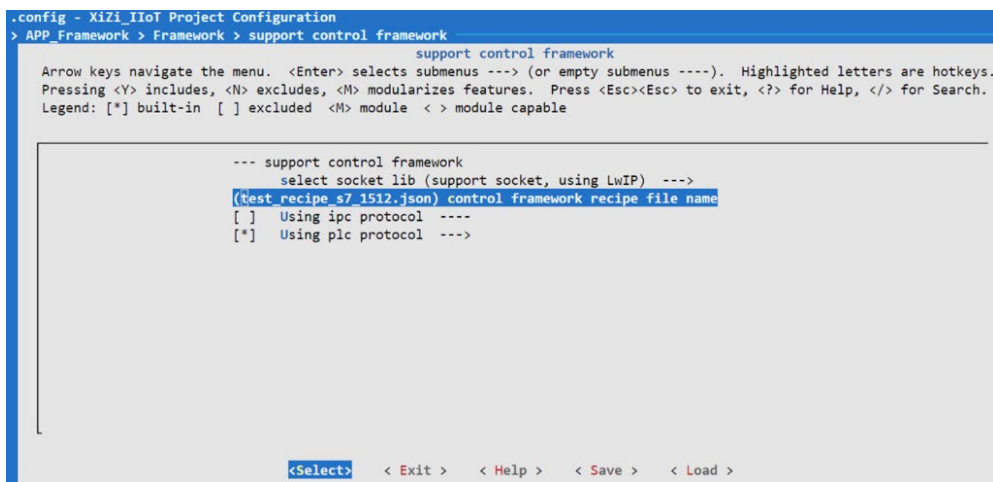


图 6-4-11 使能 melsec_control_protocol

3) 配置 main 线程栈大小:

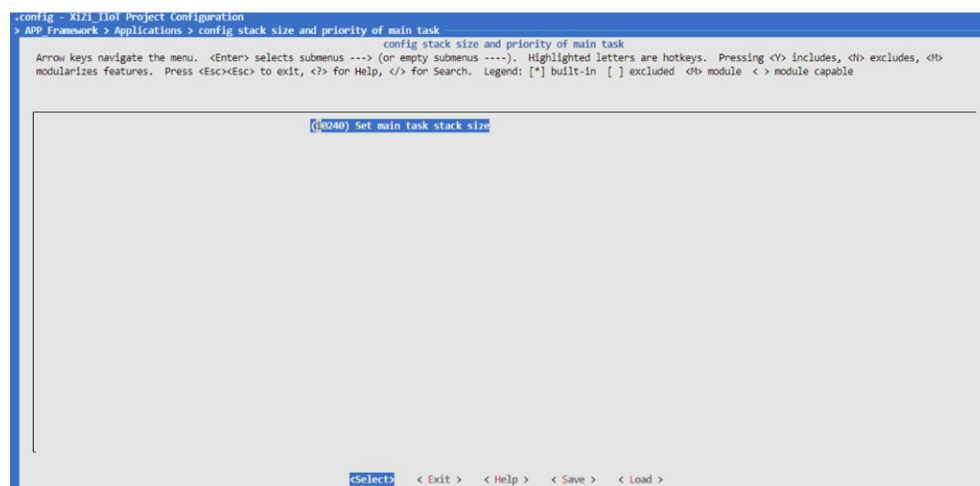


图 6-4-12 配置 main 线程栈

4) 使能 control app, 编译 plc 应用程序:

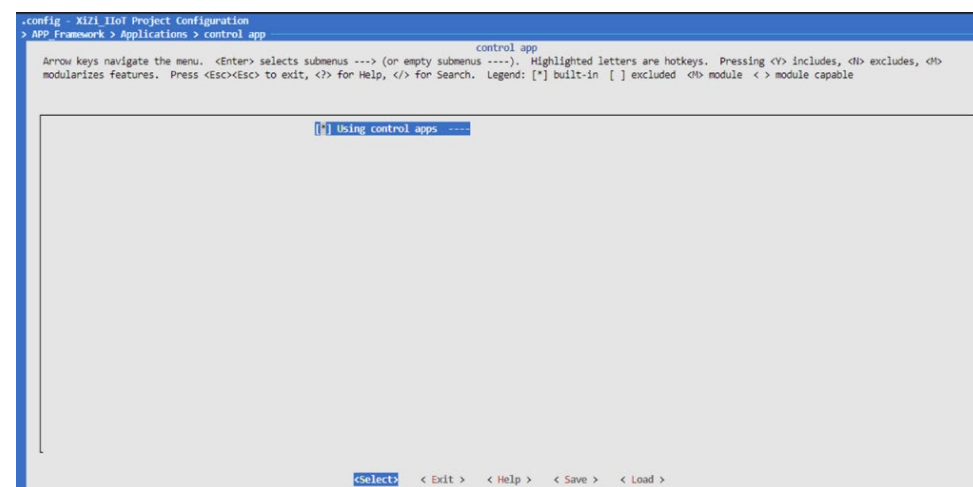


图 6-4-13 使能 control app

Step3: 连接 PLC 进行测试。在 json 文件解析完成且 socket 初始化完成之后, 输入 ControlS71500Test 命令, 即可进行数据读取。ControlS71500Test 测试函数详见代码 6-4-23。

代码行 03-05 定义了存放数据的缓冲区和数据长度, 代码行 06 将用到的协议设置为 S7 协议。代码行 13 进行协议打开, 代码行 15 进行数据的读取。

代码 6-4-23 RISC-V 架构下的西门子 CPU 1512SP-1 PN 应用测试

```

01 void ControlS71500Test(void)
02 {
03     int i = 0;
04     uint16_t read_data_length = 0;
05     uint8_t read_data[1024] = {0};
06     ControlProtocolType s7_protocol = ControlProtocolFind();
07     if (NULL == s7_protocol) {
08         printf("%s get s7 protocol %p failed\n", __func__, s7_protocol);
09         return;
10     }
11     printf("%s get s7 protocol %p successfull\n", __func__, s7_protocol);
12     if (CONTROL_REGISTERED == s7_protocol->protocol_status) {
13         ControlProtocolOpen(s7_protocol);
14         for (;;) {
15             read_data_length = ControlProtocolRead(s7_protocol, read_data,
sizeof(read_data));
16             printf("%s read [%d] s7 data %d using receipe file\n", __func__, i,
read_data_length);
17             i++;
18             memset(read_data, 0, sizeof(read_data));
19             PrivTaskDelay(10000);
20         }
21         //ControlProtocolClose(s7_protocol);
22     }
23 }

```

附录 A

表 A ARM 架构矽璞硬件终端引脚功能列表

GPIO 编号	功能（外设连接）	备注
PA0	WAKEUP	对应为 WAKRUP 按键
PA1	MII_RXCLK	
PA2	SMI_MDIO	
PA3	ADC123_IN3	
PA4	DAC1_OUT1	
PA5	Undefined	
PA6	Undefined	
PA7	MII_RXDV	
PA8	SDIO_D1	
PA9	USBFS_VBUSDET	
PA10	USBFS_ID	
PA11	USBFS_DM	
PA12	USBFS_DP	
PA13	SWDIO	SW 调试数据口
PA14	SWDCLK	SW 调试时钟口
PA15	Undefined	
PB0	MII_RXD2	
PB1	MII_RXD3	
PB2	Undefined	
PB3	Undefined	

PB4	Undefined	
PB5	SDIO_D3	
PB6	MII_TXCLK	
PB7	SDIO_DO	
PB8	MII_TXD3	
PB9	MII_TXD2	
PB10	USART3_TX	
PB11	USART3_RX	
PB12	Undefined	
PB13	SDIO_CD	
PB14	Undefined	
PB15	Undefined	
PC0	Undefined	
PC1	SMI_MDC	
PC2	Undefined	
PC3	Undefined	
PC4	MII_RMII_RXD0	
PC5	MII_RMII_RXD1	
PC6	QSPI_SCK	
PC7	QSPI_NSS	
PC8	Undefined	
PC9	USBFS_DRVVBUS	
PC10	SDIO_D2	

PC11	EXT_GPIO0	
PC12	SDIO_CK	
PC13	Undefined	
PC14	RTC_OUT	RTC 时钟引脚，接 32.768KHz 晶振，通常不额外配置
PC15	RTC_IN	RTC 时钟引脚，接 32.768KHz 晶振，通常不额外配置
PD0	LED0	
PD1	EXT_GPIO1	
PD2	SDIO_CMD	
PD3	I2C1_SCL	
PD4	CAN1_RX	
PD5	CAN1_TX	
PD6	CAN_RX	
PD7	CAN_TX	
PD8	QSPI_IO0	
PD9	QSPI_IO1	
PD10	QSPI_IO2	
PD11	QSPI_IO3	
PD12	Undefined	
PD13	Undefined	
PD14	Undefined	
PD15	USBFS_OC	
PE0	Undefined	

PE1	Undefined	
PE2	TRACECLK	
PE3	USART6_CK	
PE4	USART6_CTS	
PE5	USART6_RTS	
PE6	USART6_TX	
PE7	RS485_RXD	
PE8	Undefined	
PE9	Undefined	
PE10	Undefined	
PE11	Undefined	
PE12	Undefined	
PE13	Undefined	
PE14	Undefined	
PE15	Undefined	
PF0	Undefined	
PF1	Undefined	
PF2	Undefined	
PF3	Undefined	
PF4	Undefined	
PF5	Undefined	
PF6	Undefined	
PF7	Undefined	

PF8	Undefined	
PF9	Undefined	
PF10	I2C1_SDA	
PF11	Undefined	
PF12	Undefined	
PF13	Undefined	
PF14	Undefined	
PF15	Undefined	
PG0	RS485_TXD	
PG1	RS485_DIR	
PG2	Undefined	
PG3	Undefined	
PG4	Undefined	
PG5	Undefined	
PG6	Undefined	
PG7	Undefined	
PG8	Undefined	
PG9	Undefined	
PG10	Undefined	
PG11	MII_RMII_TXEN	
PG12	Undefined	
PG13	MII_RMII_TXD0	
PG14	MII_RMII_TXD1	

PG15	Undefined	
PH0	XTAL_IN	外部晶振输入引脚，接 8MHz 晶振，通常情况下不额外配置
PH1	XTAL_OUT	外部晶振输出引脚，接 8MHz 晶振，通常情况下不额外配置
PH2	MII_CRS	
PH3	MII_COL	
PH4	Undefined	
PH5	Undefined	
PH6	USART6_RX	
PH7	Undefined	
PH8	Undefined	
PH9	Undefined	
PH10	Undefined	
PH11	ETH_RST	
PH12	Undefined	
PH13	Undefined	
PH14	RW_SCK	
PH15	RW_MISO	
PI1	RW_MOSI	
PI2	RW_CS	
PI3	RW_RST	
PI4	Undefined	
PI5	Undefined	

PI6	Undefined	
PI7	USER_KEY	
PI8	Undefined	
PI9	USART10_TX	
PI10	MII_RMII_RXER	
PI11	USART10_RX	
PI12	Undefined	
PI13	MD	作 BOOT 引脚，通过引脚帽配置

附录 B

表 B RISC-V 架构矽璞硬件终端引脚功能列表

硬件 IO 编号	功能（外设连接）	备注
IO0	JTAG_TCK/WIFI_SPI_CS	用作 SPI 接口时无法调试
IO1	JTAG_TDI/WIFI_SPI_CLK	用作 SPI 接口时无法调试
IO2	JTAG_TMS/WIFI_SPI_MISO	用作 SPI 接口时无法调试
IO3	JTAG_TDO/WIFI_SPI_MOSI	用作 SPI 接口时无法调试
IO4	ISP_RX	ISP 程序烧录串口
IO5	ISP_TX	ISP 程序烧录串口
IO6	WIFI_TX	内置 WIFI 模块通信串口
IO7	WIFI_RX	内置 WIFI 模块通信串口
IO8	WIFI_EN	内置 WIFI 模块使能引脚
IO9	SPI1_ENET_SCLK	
IO10	SPI1_ENET_MISO	
IO11	SPI1_ENET_MOSI	
IO12	SPI1_ENET_nCS	
IO13	SPI1_ENET_nRST	
IO14	SPI1_ENET_nINT	
IO15	CTP_SDA	
IO16	BOOT	连接在 SW3 上
IO17	CTP_SCL	
IO18	RS485_RXD	SPS3485 的 RO 引脚
IO19	RS485_TXD	SPS3485 的 DI 引脚

IO20	E220_RXD	
IO21	E220_TXD	
IO22	CH376T_RXD	
IO23	CH376T_TXD	
IO24	RS485_DIR	连接 SPS3485 的 nRE 和 DE 引脚
IO25	SPI2_SD_MISO	
IO26	SPI2_SD_SCLK	
IO27	SPI2_SD_MOSI	
IO28	SPI2_SD_nCS	
IO29	LED0	
IO30	CTP_INT	
IO31	KEY	
IO32	E220_M0	
IO33	E220_M1	
IO34	Undefined	
IO35	Undefined	
IO36	LCD_RST	
IO37	LCD_CS	
IO38	LCD_WR	
IO39	LCD_DC	
IO40	DVP_RST	
IO41	DVP_PWDN	
IO42	DVP_XCLK	

IO43	DVP_PCLK	
IO44	DVP_HSYNC	
IO45	DVP_VSYNC	
IO46	DVP_SCL	
IO47	DVP_SDA	